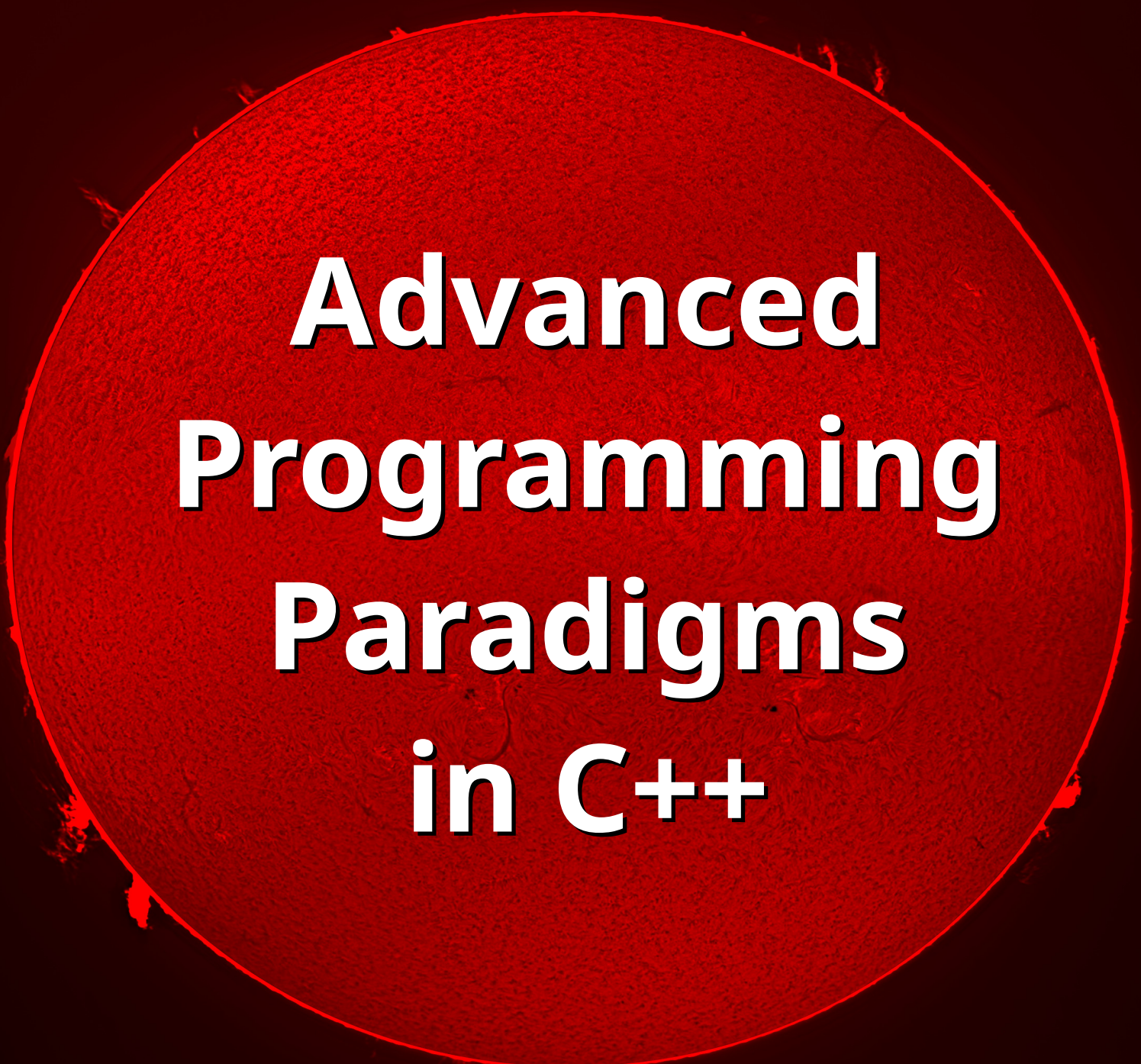


Ulrich W. Eisenecker



Advanced Programming Paradigms in C++

Version 0.93 - June 22, 2026

Ulrich W. Eisenecker

Advanced Programming Paradigms in C++

Version 0.93 - June 22, 2026

Dr. Ulrich W. Eisenecker is a full professor of software development at the Institute of Information Systems at the Leipzig University.

All information in this book has been written and compiled with care. Nevertheless, it may contain errors. Therefore, the author assumes no responsibility for the use of this book or the information contained therein.

The use of common names, trade names, product names, etc. in this book, even without special identification, does not justify the assumption that such names are to be considered free in the sense of the trademark and brand protection laws and may therefore be used by anyone.

The photo on the cover was taken by the author.

The book and all accompanying materials can be downloaded from <https://github.com/iwi-se/Advanced-Programming-Paradigms>. Updated versions will be made available from this repository as they are released. Suggestions for improvements or corrections can be submitted via issues directly on the GitHub page.

If you feel that this work causes a legal issue that concerns you, please submit a corresponding issue with detailed information via issues directly on the GitHub page.

Preface

The target audience for this book is students of computer science or information systems. It is the companion book to the Advanced Programming Paradigms lecture. A prerequisite for reading this book is to be familiar with Basic Programming Paradigms (U. W. Eisenecker 2022) and to have worked through the companion material of this book.

The LibreOffice suite was used for writing. Google Noto fonts were used for the typesetting. For programming, the tools mentioned in the book were used.

The book was originally written in English by a non-native speaker. To check the linguistic quality <https://www.deepl.com> was very helpful.

I would like to thank Nico Willert Stark, Janik Eriksson, Marie Ehnert, Eric Minor, Yannis Hübenthal, Enny May, Felix Obermann, Hannes Gött, Kolja Mailahn, Agathe Freywald and also students whose names have slipped my mind for pointing out mistakes and suggesting improvements.

I am particularly grateful to Hoang Nguyen, one of my students. He submitted a very long list of errors after reading an intermediate version of the book. He pointed out many linguistic errors, but also errors related to the C++ code.

In his bachelor's thesis, Leon Pfahl developed the structure of the repositories used to manage all artifacts for the book and its associated sources, and created a prototype implementation that included several basic workflows.

Enny May converted many manually created diagrams into textual Mermaid specifications, which can be more easily integrated and maintained as artifacts.

Special thanks go to Hannes Gött, who converted the book from the LibreOffice format to LaTeX. This involved not only clever conversion scripts but also numerous manual adjustments.

This book is intended for online reading, e.g. on a tablet computer. It does not have a book layout. If a printed version is desired, a color printer should be used. This is also one reason why there is no keyword index in the appendix. Instead, the electronic version is simply searched for the desired keyword or its stem.

If you would like to suggest changes or improvements, report bugs, or share other feedback related to this book, please use the relevant features on the GitHub project page.

This book is not yet complete. It also contains numerous errors of various kinds. Updated versions of the book will be made available as soon as possible. Chapters 4 and 5 have not yet been proofread, and a chapter on parallel programming, as well as the concluding notes, are still missing.

Contents

List of Figures	12
List of Tables	13
List of Listings	21
1 The Object-Oriented Programming Paradigm	23
1.1 From Procedures to Objects	23
1.1.1 Inheritance	27
1.1.2 Access Specifiers	30
1.1.3 Static Binding	32
1.2 Late Binding	37
1.3 Kinds of Polymorphism	40
1.4 Override	41
1.5 Pure Virtual Functions	44
1.6 Constructors and Destructor	55
1.6.1 Constructors and List Initialization	56
1.6.2 Classic Syntax for Calling Constructors	57
1.6.3 Fundamental Datatypes and Constructors	58
1.6.4 Global and Static Variables	59
1.6.5 Dynamic Objects	62
1.6.6 Copy Constructor	63
1.6.7 Explicit	64
1.6.8 Direct Construction vs. Copy Construction	65
1.6.9 Initializer List	66
1.6.10 Delegating Constructor	68
1.6.11 Virtual Destructor	69
1.6.12 Order of Calling Constructors and Destructors	73
1.7 The Virtual Function Table	75
1.8 final	80
1.9 Static Members	85
1.10 A Detour Into Fictitious Practice	95

1.10.1	A Nice and Inexpensive Feature	95
1.10.2	Options for Implementation	95
1.10.3	Use of the DOT Language	96
1.10.4	Implementation of the Feature	98
1.10.5	Evaluation of the Implementation	101
1.11	Using Declaration	102
1.12	Default and Delete	105
1.13	[[nodiscard]] for Classes	108
1.14	Object With Dynamic Resource	110
1.14.1	Prohibited Copying and Assigning	110
1.14.2	Dynamic Resource Objects Lab	113
1.14.3	Object Without Dynamic Resource	114
1.14.4	Object With Dynamic Resource—The Wrong Way	118
1.14.5	Big3—The Better Way	123
1.14.6	Big5—The Ambitious Way	126
1.14.7	Big5Deleted—A Convenient Alternative	130
1.15	Multiple Inheritance	131
1.15.1	Subobjects	131
1.15.2	Losing the Late Binding	133
1.15.3	Name Collisions	136
1.15.4	Virtual Base Classes	137
1.15.5	Draw Generators	140
1.15.5.1	Urn With Order And With Repetitions	141
1.15.5.2	Urn With Order And Without Repetitions	142
1.15.5.3	Urn Without Order And With Repetitions	144
1.15.5.4	Urn Without Order And Without Repetitions	146
1.15.5.5	Unit Testing of Urns	147
1.15.5.6	Implementation of the UrnOR Class	150
1.15.5.7	Implementation of the UrnO Class	153
1.15.5.8	Implementation of the UrnR Class	154
1.15.5.9	Implementation of the Urn Class	156
1.15.6	Criticism of Inheritance	157
1.16	Cast Operators	157
1.16.1	Traditional Cast and Functional Cast	158
1.16.2	static_cast<>()	162
1.16.3	const_cast<>()	163
1.16.4	dynamic_cast<>()	164
1.16.4.1	Syntactic Aspects of Using dynamic_cast<>()	164
1.16.4.2	Implementation of Correct Assignments for Inclusion Poly- morphism	166
1.16.5	reinterpret_cast<>()	175
1.16.5.1	Accessing Bits of a Float Variable With reinterpret_cast<>()	175

1.16.5.2	Implementation of Object Persistence With reinterpret_ cast<>()	177
1.17	Type Identification	190
1.17.1	Static Type	190
1.17.2	Dynamic Type	194
1.17.3	Multi-Methods	195
1.18	Exceptions	200
1.18.1	Bad Use of Exceptions	203
1.18.2	Stack-Unwinding	205
1.18.3	Incomplete Objects	209
1.18.4	More on Exception Handling	211
1.19	Smart Pointers	212
1.19.1	Exclusive Ownership	212
1.19.1.1	Object With Altering Behavior	213
1.19.1.2	The Pointer-to-Implementation Idiom	220
1.19.1.3	Explicit Transfer of Ownership	221
1.19.2	Shared Ownership	223
1.19.2.1	A Shared-Resource Example	223
1.19.2.2	Design for the Shared-Resource Example	224
1.19.2.3	Implementing the Shared-Resource Example	224
1.19.2.4	Running the Shared-Resource Example	227
1.19.2.5	Observations From the Shared-Resource Example	231
1.19.3	Interdependent Pointers	233
1.20	Criticism of OOP and Inclusion Polymorphism	237
2	The GUI-Programming Paradigm	241
2.1	Text-based vs. Graphical User Interfaces	242
2.1.1	An Example of a Text-based User Interface	242
2.1.2	The Properties of a Terminal Window	245
2.1.3	The Event Loop	246
2.1.4	The Procedural Nature of Ncurses	248
2.2	FLTK	248
2.2.1	Moving an Asterisk	249
2.2.2	Building an FLTK Application	250
2.2.3	GuiMove.cpp vs. Move.cpp	254
2.2.4	The main() Function of GuiMove.cpp	255
2.2.5	The MyWindow Class	257
2.2.5.1	Base Classes of MyWindow	257
2.2.5.2	Overriding the draw() Member Function	258
2.2.5.3	Overriding the handle() Member Function	258
2.2.5.4	Data Member and Constructor of MyWindow	259
2.2.5.5	About Fonts and Font-related Functions	259
2.2.5.6	Drawing Text	263

2.2.5.7	MyWindow Constructor Revisited	265
2.2.5.8	MyWindow::draw()	266
2.2.5.9	MyWindow::flash()	266
2.2.5.10	MyWindow::handle()	267
2.2.6	Portability of FLTK Programs	269
2.2.6.1	Different Look and Behavior	269
2.2.6.2	Information about Events	269
2.2.6.3	MS Windows Specific Behavior and Workaround	273
2.2.6.4	Further Specifics	274
2.2.6.5	More About Drawing Text	274
2.2.6.6	Coding Style	280
2.2.7	Child Widgets, Focus and Dialogs	280
2.2.7.1	Children	280
2.2.7.2	Focus and Dialogs	283
2.2.7.3	Nesting Widgets	292
2.2.7.4	Giving Focus With the Keyboard	300
2.2.7.5	Giving Focus With the Mouse	302
2.2.7.6	Giving Focus to Node Exemplars	304
2.2.8	Menus and Callbacks	311
2.2.8.1	Declarative Representation of Menus	311
2.2.8.2	Lambda Expressions as Callback Functions	320
2.2.8.3	Programmed Definition of Menus	322
2.2.9	Observer	324
2.2.9.1	An Implementation of Observer	325
2.2.9.2	Sub-classing of Observer and Subject	329
2.2.9.3	Using Observers and Subjects	339
2.2.9.4	Afterthoughts on Observer	341
2.2.10	More Topics of GUI Programming With FLTK	342
2.2.11	Human-Computer Interaction and User Experience	343
3	The Generic Programming Paradigm	345
3.1	Function Templates	345
3.1.1	Syntax of Function Templates	350
3.1.2	Template Argument Deduction	350
3.1.3	Implicit Requirements for a Template Parameter	351
3.1.4	Explicit Template Arguments	355
3.1.5	Full Specialization of Function Template	356
3.1.6	Constant Template Parameters	360
3.1.7	Overloading of Function Templates	361
3.2	Class Templates	363
3.2.1	More About Implicit Requirements for Template Parameters	368
3.2.2	Lazy Instantiation of Templates	370
3.2.3	Template Default Arguments	371

3.2.4	Specialization of Class Templates	373
3.2.5	Member Templates	374
3.2.6	Template Template Parameters	382
3.2.7	Typename and Template in Templates	384
3.2.8	Structuring Template Files	387
3.2.9	Name Look-up in Templates	394
3.2.10	Template Alias	397
3.3	Variadic Templates	397
3.4	Concepts	411
3.5	Final Advice for Implementing Templates	417
4	Containers, Iterators and Algorithms	419
4.1	The Classic C-Array	420
4.2	Migrating from Array to Vector<>	426
4.3	Use of Algorithms From the C++ Standard Library	428
4.4	Further Simplification by Using Ranges	430
4.5	Containers	432
4.5.1	Sequence Containers	432
4.5.2	Associative Containers	436
4.5.2.1	std::set<>	436
4.5.2.2	std::multiset<>	443
4.5.2.3	std::multimap<>	445
4.5.2.4	Another Look at std::multiset<>	450
4.5.3	Unordered Containers	455
4.5.4	More Container Types	456
4.5.4.1	std::stack<>	457
4.5.4.2	Views for Containers	467
4.5.5	Closing Remarks on Containers, Iterators and Algorithms	469
4.6	Descriptive Statistics—A Case Study	471
4.6.1	(Pseudo-)Random Numbers	471
4.6.2	Descriptive Statistics as an Application Example	476
4.6.3	Benchmarking	490
4.6.4	Simple Benchmarking with Catch2	490
4.6.5	Catch2 Generators	494
4.6.6	Benchmarking of Sequence Containers	496
4.6.7	Comprehensive Benchmarking	498
4.6.8	Unit Testing for Calculating Statistical Parameters	507
4.6.9	A Systematic Measurement Error and Maps	510
4.7	The Final Draw	513
5	The Functional Programming Paradigm	517
5.1	Template Metaprogramming	518
5.2	Basic Principles of Functional Programming	520
5.3	The Classic C Function and its Modern Form	523

5.3.1	New Syntax for Specifying the Return Type	523
5.3.2	Tail Recursion	524
5.3.3	Further Optimizations	529
5.3.4	Higher Order Functions	530
5.3.5	Functions With Memory	534
5.4	C++-specific Functions	537
5.4.1	Function Objects	537
5.4.1.1	Partial Function	539
5.4.1.2	std::optional<>	540
5.4.2	Lambda Expression	543
5.4.2.1	Lambda Syntax	543
5.4.2.2	Capture Clause	547
5.4.2.3	Capturing this and *this	550
5.4.3	Currying	557
5.4.4	Partial Function Application	559
5.4.5	Functor	564
5.4.6	Monad	572
5.4.6.1	The Optional Monad	574
5.4.6.2	The Expected Monad	578
5.4.6.3	The Vector Monad	580
5.4.7	Algebraic Data Types	582
5.4.7.1	Sum Type	583
5.4.7.2	Product Type	597
5.4.7.3	Recursive Type	601
5.4.8	Lazy Evaluation	602
5.4.8.1	From Containers to Ranges to Views	604
5.4.8.2	A View as Functor and Monad	607
5.4.8.3	Prime Factorization	610
5.4.9	Game of Life	614
5.4.10	Discussion	621
References		629

List of Figures

1	Parameter passing and inclusion polymorphism	38
2	OBCC—UML class diagram	45
3	OBCE—UML class diagram	47
4	OBC—UML class diagram	50
5	OBC—Inheritance Hierarchy	51
6	Compile, link, and run Global.cpp	61
7	Output of objects/Destructor.cpp	71
8	Output of objects/Order.cpp	74
9	Classes, virtual function tables, and objects	79
10	ObjectManager class—UML class diagram	86
11	Spreadsheet with persons.csv imported	91
12	Output when executing ./oidmOBJECT	94
13	Customer sketch of person graph	95
14	Graph for objects/manual.viz produced by dot	97
15	Person graph in landscape format	97
16	Creating a person graph with DOT	102
17	Output of Resources.cpp for using Class = NoDynamicResource;	117
18	Automatically generated copy constructor and dynamic resource	121
19	Automatically generated assignment operator and dynamic resource	122
20	Output of Resources.cpp for using Class = Big3;	124
21	Output of Resources.cpp for using Class = Big5;	129
22	Output of objects/LateBindingLost.cpp	135
23	Most-derived classes and virtual base class	139
24	Permutations for $n = 3$ and $k = 3$ (permutations with crossed-out repetitions)	143
25	Combinations for $n = 3$ and $k = 3$ (combinations affected by sorting are crossed out)	145
26	Output of objects/ProblematicAssignment.cpp	168
27	Output of program objects/StackUnwinding.cpp	208
28	Output of program objects/IncompleteObject.cpp (excerpt)	210
29	Class diagram for Butterfly and related classes	217
30	Butterfly stages (https://openclipart.org/detail/322882/ butterflys-life-cycle-english,remixed)	218

31	Screenshot of the execution of gui/MoveWithInfo.cpp	245
32	Screenshot of the execution of gui/GuiMoveWithInfo.cpp	249
33	Output of executing gui/Main.cpp	256
34	FLTK font faces (screenshot of gui/Fonts.cpp under MS Window)	260
35	FLTK font faces (screenshot of gui/Fonts.cpp under MS Window)	261
36	Further font features (screen shot of gui/FontFeatures.cpp under Ubuntu) .	262
37	Aligning text under macOS (left), Ubuntu (middle), and MS Windows (right)	263
38	Various ways of drawing text (gui/MoreAboutDrawing.cpp)	278
39	Screenshot of the execution of gui/Children.cpp	283
40	Screenshot of the execution of gui/NestedChildren.cpp	296
41	Screenshot of the execution of the modified gui/NestedChildren.cpp program	303
42	Processing compiler output	355
43	Adding the letters A to K to a ring buffer with the capacity 5	363
44	Template instantiations for quiet::sum1<3,5,1>()	400
45	Selected positions in the C-array values	424
46	Output of cia/HanoiVisualization.cpp for 3 disks	465
47	Spreadsheet replicating calculations of DescriptiveStatistics.cpp	489
48	Output of executing cia/BM_RandomData100.cpp	492
49	Mean values of benchmarks from BM_RandomData1_1000.cpp	496
50	Mean values of benchmarks from BM_Containers.cpp	497
51	Mean values of benchmarks from BM_Comprehensive.cpp	506
52	Benchmarking results for various optimization levels	528
53	Output of 2nd block in main() of functional/CaptureThis.cpp for g++ 15.2.0	555
54	Output of 2nd block in main() of functional/CaptureThis.cpp for clang++ 21.1.6	555
55	Cone	559
56	Conical segment	560
57	Category	564
58	Functor	566
59	Output of functional/Flatten.cpp	582
60	Numbers and prime factors	613
61	Only prime factors	613
62	Only products	613

List of Tables

1	Inheritance and accessibility	31
2	Variable categories, scope, lifetime, and initialization	63
3	Aggregated subobjects	132
4	Inherited subobjects	132
5	Inherited and aggregated subobjects	133
6	Multiple base objects	138
7	Urn models	141
8	Test code and textual specification	149
9	Two persons in a discussion	228
10	<i>Several</i> persons in several discussions	229
11	Discussion with a clone	230
12	Output of program objects/SharedPointer.cpp	232
13	Platform-specific text extents under macOS (left), Ubuntu (middle), and MS Windows (right)	265
14	Variadic function templates	398
15	Fold expressions	404
16	begin()/end() and start/stop iterators	430
17	Aligning substrings from ABACAB	448
18	Parameter names and formulas of frequency statistics	477
19	Output of different implementations of calculating statistical parameters	507
20	Mean values of data sets of different size	511

List of Listings

1	objects/OBC_TypeFunctions.cpp	24
2	objects/OBC_ADT.cpp	25
3	objects/OBC_TypeFunctionsAdvanced.cpp (excerpt)	26
4	objects/OBC_Inheritance.cpp	27
5	objects/OBC_IO.cpp (excerpt)	33
6	objects/OBC_LB.cpp (excerpt)	39
7	objects/OBC_Error.cpp (excerpt)	42
8	objects/OBC_ErrorOverride.cpp (excerpt)	43
9	objects/OBC_Override.cpp (excerpt)	43
10	objects/OBCC.cpp (excerpt)	46
11	objects/OBCE.cpp (excerpt)	48
12	objects/OBCH.hpp (excerpt)	52
13	Undefined behaviour after declaration of a variable of fundamental type .	59
14	objects/Global.cpp	59
15	objects/UseGlobal.cpp	60
16	objects/Static.cpp	62
17	objects/DoubleAsOBCE.cpp	64
18	objects/OBCHE.hpp (excerpt)	65
19	objects/CopyInitialization.cpp (excerpt)	65
20	objects/InitializerList.cpp	66
21	objects/RN.hpp (excerpt)	68
22	objects/RNI.hpp (excerpt)	69
23	objects/Destructor.cpp	70
24	objects/Order.cpp	73
25	objects/Sorting.cpp	75
26	objects/FinalClass.cpp	80
27	objects/FinalMethod.cpp	82
28	objects/oid_manager.hpp	86
29	objects/Acquaintances.cpp	88
30	objects/OIDManagerUnitTests.cpp	92
31	Manually created DOT file for person relations (objects/manual.viz)	96
32	Outline of graph specification in DOT	98
33	objects/RelationsGraph.cpp	99

34	objects/UsingConstructors.cpp	103
35	objects/UsingPrivateBaseClass.cpp	104
36	objects/Default.cpp	105
37	objects/Delete.cpp	106
38	objects/Nodiscard.cpp	109
39	objects/Diary.cpp	111
40	Class, test, and main() of objects/Resource.cpp (excerpt)	113
41	Resource and NoDynamicResource of objects/Resource.cpp (excerpt)	114
42	Wrong of objects/Resource.cpp (excerpt)	118
43	Big3 of objects/Resource.cpp (excerpt)	123
44	Splitting complex statements over multiple lines	126
45	Big5 of objects/Resource.cpp (excerpt)	127
46	Big5Deleted of objects/Resource.cpp (excerpt)	130
47	objects/LateBindingLost.cpp	134
48	objects/ResolvingNameCollision.cpp	136
49	objects/UrnUnitTests.cpp (excerpts)	147
50	UrnOR class (excerpt from objects/urn.hpp)	150
51	UrnO class (excerpt from objects/urn.hpp)	153
52	UrnR class (excerpt from objects/urn.hpp)	154
53	Urn class (excerpt from objects/urn.hpp)	156
54	objects/SimpleLegacyCasts.cpp	158
55	objects/ConstAndDownLegacyCasts.cpp	160
56	objects/StaticCast.cpp	163
57	objects/ConstCast.cpp	164
58	objects/DynamicCast.cpp	164
59	objects/ProblematicAssignment.cpp	167
60	objects/assignment.hpp	170
61	objects/AssignmentUnitTests.cpp (excerpt)	173
62	objects/ReinterpretCast.cpp	176
63	objects/persistence.hpp	178
64	Object class (excerpt from objects/PersistenceUnitTests.cpp)	184
65	Person class (excerpt from objects/PersistenceUnitTests.cpp)	185
66	TEST_CASE("PERSONS_DESERIALIZE") (excerpt from objects/Persistence- UnitTests.cpp)	188
67	objects/TypeTraits.cpp	190
68	objects/TypeID.cpp	191
69	objects/Specifics.cpp	192
70	objects/DynamicType.cpp	194
71	Account classes (excerpt from objects/MultiMethodUnitTests.cpp)	196
72	transfer() function (excerpt from objects/MultiMethodUnitTests.cpp)	198
73	objects/ExceptionBasics.cpp	200
74	objects/ExceptionBadUse.cpp	203
75	objects/ExceptionCorrectUse.cpp	204

76	objects/StackUnwinding.cpp	205
77	objects/IncompleteObject.cpp (excerpt)	209
78	objects/Butterfly.cpp	213
79	objects/TransferOwnership.cpp	221
80	objects/Discussion.cpp	224
81	objects/Circular.cpp	233
82	objects/BreakingTheCircle.cpp	236
83	gui/Move.cpp	243
84	gui/GuiMove.cpp	251
85	gui/Main.cpp	255
86	run.bat (MS Windows batch file) executing main.exe	256
87	gui/Events.cpp	270
88	gui/EventsSimplified.cpp	271
89	gui/TerminalOutput.cpp	273
90	gui/MoreAboutDrawing.cpp	275
91	gui/Children.cpp	281
92	gui/Focus.cpp	285
93	gui/NestedChildren.cpp	292
94	gui/NestedChildrenImproved.cpp	301
95	Selecting a child of window (excerpt from gui/NestedChildrenImproved.cpp)	302
96	Change in main() (excerpt from gui/NestedChildren.cpp)	302
97	gui/NestedChildrenImproved.cpp	304
98	gui/Menu.cpp	312
99	Fl_Menu_Item (taken from (Costantini, Gibson, et al. n.d.))	317
100	gui/MenuWithLambdas.cpp	320
101	gui/ProgrammedMenu.cpp	322
102	Observer (excerpt from gui/Observer.cpp)	325
103	Star (excerpt from gui/Observer.cpp)	330
104	StarWindow (excerpt from gui/Observer.cpp)	331
105	eventHandler() (excerpt from gui/Observer.cpp)	339
106	main() (excerpt from gui/Observer.cpp)	340
107	templates/SwapInts.cpp	346
108	templates/SwapOverloads.cpp	346
109	templates/SwapOverloads.cpp (excerpt, differences highlighted)	348
110	templates/SwapTemplate.cpp	349
111	templates/UnmetRequirements.cpp	352
112	templates/SwapChecked.cpp (excerpt)	353
113	templates/SwapMoreTemplates.cpp (excerpt)	356
114	templates/SwapSpecialized.cpp (excerpt)	357
115	templates/SwapMoreSpecialized.cpp (excerpt)	358
116	templates/SwapWithIntegralTemplateParameter.cpp (excerpt)	360
117	templates/SwapPointers.cpp	362
118	templates/SwapOverloaded.cpp (excerpt)	362

119	templates/BufferChar5.cpp (excerpt)	364
120	templates/BufferTemplateT.cpp (excerpt)	365
121	templates/ClassTemplateWithTAD.cpp (excerpt)	366
122	templates/BufferTemplateTN.cpp (excerpt)	367
123	templates/RingBufferSquaring.cpp (excerpt)	370
124	templates/BufferDefaultArguments.cpp (excerpt)	372
125	templates/BoolBuffer.cpp (excerpt)	373
126	templates/GenericSerialization.cpp	376
127	templates/StringSerialization.cpp (excerpt)	379
128	templates/TemplateTemplateParameter.cpp	382
129	templates/TypenameTemplate.cpp	385
130	templates/all.hpp	388
131	templates/separated.hpp	389
132	templates/definitions.t	391
133	templates/header.hpp	392
134	templates/instantiations.cpp	393
135	templates/UseInstantiations.cpp	393
136	templates/LookUp.cpp	395
137	templates/AliasTemplate.cpp (excerpt)	397
138	templates/quiet::sum1<>() (VariadicTemplates.cpp)	399
139	templates/InputCppInsights.cpp	401
140	templates/OutputCppInsights.cpp	401
141	templates/verbose::sum1<>() (VariadicTemplates.cpp)	403
142	templates/quiet::sum2<>() (VariadicTemplates.cpp)	404
143	templates/OutputFoldCppInsights.cpp (excerpt)	405
144	templates/verbose::sum2<>() (VariadicTemplates.cpp)	405
145	templates/quiet::sum3<>() (VariadicTemplates.cpp)	407
146	templates/verbose::sum3<>() (VariadicTemplates.cpp)	408
147	templates/quiet::sum4<>() (VariadicTemplates.cpp)	409
148	templates/verbose::sum4<>() (VariadicTemplates.cpp)	409
149	templates/Folds.cpp	410
150	templates/SwapWithRequires.cpp (excerpt)	411
151	templates/SwapWithConceptAndRequires.cpp (excerpt)	412
152	templates/SwapWithConcept.cpp (excerpt)	413
153	templates/SwapNewSyntax.cpp (excerpt)	413
154	templates/SwapImproved.cpp	414
155	templates/SwapIncompleteDiagnostics.cpp (excerpt)	415
156	templates/SwapCompleteDiagnostics.cpp (excerpt)	415
157	templates/SumWithConcept.cpp (excerpt)	416
158	SumWithConcept.cpp—modified (excerpt)	416
159	cia/ClassicArray.cpp	421
160	cia/VectorA.cpp	426
161	std::swap<>() and std::for_each<>() in cia/VectorB.cpp (excerpt)	428

162	<code>std::ranges::reverse<>()</code> and <code>std::ranges::for_each<>()</code> in <code>cia/VectorD.cpp</code> (excerpt)	431
163	<code>std::ranges::reverse_view<></code> and <code>std::ranges::reverse</code> in <code>cia/VectorE.cpp</code> (excerpt)	431
164	<code>cia/Containers.cpp</code> (excerpt)	433
165	<code>cia/Set.cpp</code>	437
166	<code>to_string<>()</code> with <code>std::ranges::fold_left</code> (excerpt from <code>cia/FoldLeft.cpp</code>) . . .	439
167	The insertion in the middle of a <code>std::list<></code>	442
168	<code>cia/InsertionSort.cpp</code>	444
169	<code>cia/MultiMap.cpp</code>	446
170	<code>cia/SetOfPairs.cpp</code>	451
171	<code>cia/SetOfTuples.cpp</code>	451
172	Specialization of <code>std::less<></code> for <code>std::pair<></code> (excerpt from <code>cia/SetOfPairsAdapted.cpp</code>)	453
173	Specialization of <code>std::less<></code> for a parameter pack (excerpt from <code>cia/SetOfTuplesAdapted.cpp</code>)	454
174	<code>cia/UnorderedSet.cpp</code>	455
175	<code>cia/HanoiSimple.cpp</code>	457
176	<code>cia/HanoiStack.cpp</code>	458
177	<code>cia/HanoiSmarter.cpp</code> (excerpt)	460
178	<code>cia/HanoiVisualization.cpp</code> (excerpt)	462
179	<code>cia/MultiDimensions.cpp</code> (excerpt)	467
180	<code>cia/PRN_Generator</code>	471
181	<code>cia/RandomDevice.cpp</code>	472
182	<code>cia/SeedValue.cpp</code>	473
183	<code>cia/Distribution.cpp</code>	474
184	<code>cia/GermanFemales.cpp</code>	475
185	<code>generateRandomData<>()</code> (excerpt from <code>cia/DescriptiveStatistics.cpp</code>) . . .	478
186	<code>modes<>()</code> (excerpt from <code>cia/DescriptiveStatistics.cpp</code>)	480
187	<code>calculateParameters<>()</code> (excerpt from <code>cia/DescriptiveStatistics.cpp</code>)	482
188	<code>outputData<>()</code> (excerpt from <code>cia/DescriptiveStatistics.cpp</code>)	485
189	<code>outputParameters<>()</code> (excerpt from <code>cia/DescriptiveStatistics.cpp</code>)	486
190	<code>main()</code> (excerpt from <code>cia/DescriptiveStatistics.cpp</code>)	488
191	<code>cia/BM_RandomData100.cpp</code>	491
192	<code>cia/BM_RandomData1_1000.cpp</code>	494
193	<code>generateRndDtaAsMap<>()</code> (excerpt from <code>cia/extended_statistics.hpp</code>) . . .	498
194	<code>modes<>()</code> (excerpt from <code>cia/extended_statistics.hpp</code>)	500
195	<code>median<>()</code> (excerpt from <code>cia/extended_statistics.hpp</code>)	500
196	<code>calculateParameters<>()</code> (excerpt from <code>cia/extended_statistics.hpp</code>)	502
197	<code>calcParamsFaster<>()</code> (excerpt from <code>cia/extended_statistics.hpp</code>)	503
198	<code>cia/BM_Comprehensive.cpp</code>	504
199	Parts of test case Calculations (excerpt from <code>cia/UnitTesting.cpp</code>)	508
200	Part of test case Calculations (excerpt from <code>cia/UnitTesting.cpp</code>)	509

201	cia/WrongMean.cpp	510
202	cia/MapWithDoubles.cpp	511
203	cia/FinalDraw.cpp	513
204	functional/FactorialTMP.cpp (excerpt)	518
205	Classic C function (excerpt from functional/Factorial.cpp)	523
206	Trailing return type specification (excerpt from functional/Factorial.cpp)	524
207	Factorial function for tail recursion (excerpt from functional/BM_TailRe- cursion.cpp)	525
208	functional/BM_TailRecursion.cpp (excerpt)	526
209	functional/HOFasParameter.cpp	530
210	functional/HOFasReturnValue.cpp	532
211	functional/FactorialMemo.cpp	534
212	functional/FunctionObject.cpp	537
213	functional/Composition.cpp (excerpt)	540
214	functional/BasicLambda.cpp	543
215	functional/ComplexLambda.cpp	545
216	functional/SimplifiedLambda.cpp	546
217	functional/CaptureClause.cpp	548
218	functional/CaptureThis.cpp	550
219	functional/Currying.cpp	557
220	coneVolume() (excerpt from functional/Cone.cpp)	560
221	conicalSegmentVolume() (excerpt from functional/Cone.cpp)	561
222	TEST_CASE("BIND1") (excerpt from functional/Cone.cpp)	562
223	TEST_CASE("BIND2") (excerpt from functional/Cone.cpp)	562
224	TEST_CASE("BIND3") (excerpt from functional/Cone.cpp)	563
225	TEST_CASE("BIND4") (excerpt from functional/Cone.cpp)	563
226	functional/Functor.cpp	567
227	functional/TypePlay.cpp	569
228	functional/NestedFunctor.cpp	572
229	functional/Optional.cpp	574
230	functional/OptionalOrElse.cpp	577
231	functional/Expected.cpp	579
232	functional/Flatten.cpp	580
233	functional/booleanenum.hpp	583
234	functional/booleanclass.hpp	585
235	functional/booleanvariant.hpp (excerpt)	591
236	functional/BooleanVariantUT.cpp (excerpt)	592
237	functional/Butterfly.cpp	594
238	functional/Tuple.cpp	598
239	functional/StrictOrder.cpp	602
240	functional/ReferentialTransparency.cpp	603
241	functional/CRV.cpp	605
242	functional/ViewMonad.cpp	607

243	functional/Join.cpp	608
244	functional/PrimeFactors.cpp	610
245	functional/LifeSimple.cpp	616
246	functional/LifeTUI.cpp (excerpt)	619

1. The Object-Oriented Programming Paradigm

Object-oriented programming is currently the dominant programming paradigm. Classes and objects not only promote the grouping of data and functions that are closely related to each other, they also enable the definition of new classes using existing classes and the writing of code that can also execute code that has not yet been written. The terminology of object-oriented programming is also used to create functional models in system analysis and technical models in system design.

An *object* is an *exemplar* of a *class*. It exists at runtime. Another synonym for object or exemplar is *instance*. A class is comparable to an *abstract data type* (ADT for short) as described in (U. W. Eisenecker 2022), Section 4.3. *Abstract Data Types*. The two fundamental extensions of object-orientation with respect to the ADT are inheritance between classes and *late binding* of methods, i.e. member functions. Late binding in combination with inheritance is also called *inclusion polymorphism*.

1.1. From Procedures to Objects

Programs with procedural and object-oriented design have similarities. A closer look at them can make it easier to understand how a procedural program can be converted into an object-oriented program and vice versa.

Therefore, the transition from procedural programming to programming with abstract data types is briefly explained. Listing 1 shows a program that implements a rudimentary *on-board computer* (OBC for short) for managing the fuel quantity of a vehicle with an internal combustion engine.

Listing 1: objects/OBC_TypeFunctions.cpp

```
1 // OBC_TypeFunctions.cpp by Ulrich Eisenecker, January 5, 2023
2
3 #include <iostream>
4 using namespace std;
5
6 struct OBC
7 {
8     double m_fuel;
9 };
10
11 OBC createOBC()
12 {
13     return OBC { 0.0 };
14 }
15
16 double fuel(const OBC& obc)
17 {
18     return obc.m_fuel;
19 }
20
21 void addFuel(OBC& obc, double fuel)
22 {
23     obc.m_fuel += fuel;
24 }
25
26 void consumeFuel(OBC& obc, double fuel)
27 {
28     obc.m_fuel -= fuel;
29 }
30
31 int main()
32 {
33     auto obc { createOBC() };
34     cout << "Fuel: " << fuel(obc) << endl;
35     addFuel(obc, 42.0);
36     cout << "Fuel: " << fuel(obc) << endl;
37     consumeFuel(obc, 12.0);
38     cout << "Fuel: " << fuel(obc) << endl;
39 }
```

The OBC struct has only one data member, namely `m_fuel` of type `double`. The `createOBC()` function returns a new variable of type OBC with `m_fuel` initialized to 0.0. The `fuel()` function returns the value of `m_fuel` of the OBC exemplar passed as a reference to `const`. The `addFuel()` and `consumeFuel()` functions increase and decrease, respectively, the value of `m_fuel` of the first parameter passed as OBC reference by the amount of the second parameter passed by value. The `main()` function demonstrates the use of the user-defined type OBC and its associated functions. With the exception of the `createOBC()` function, every other function requires at least one parameter of type OBC, which is always passed as the first parameter.

OBC and its associated functions are closely related. Therefore, it is only logical to define an abstract data type that encapsulates them. Listing 2 shows the result of this approach. It is not necessary to go into the technical details, as they are covered in (U. W. Eisenecker 2022), Section 4.3. *Abstract Data Types*.

Listing 2: objects/OBC_ADT.cpp

```
1 // OBC_ADT.cpp by Ulrich Eisenecker, January 5, 2023
2
3 #include <iostream>
4 using namespace std;
5
6 class OBC
7 {
8     public:
9         OBC(): m_fuel { 0.0 }
10        {}
11        double fuel() const
12        {
13            return m_fuel;
14        }
15        void addFuel(double fuel)
16        {
17            m_fuel += fuel;
18        }
19        void consumeFuel(double fuel)
20        {
21            m_fuel -= fuel;
22        }
23     private:
24         double m_fuel;
25 };
26
27 int main()
28 {
29     OBC obc;
30     cout << "Fuel: " << obc.fuel() << endl;
31     obc.addFuel(42.0);
32     cout << "Fuel: " << obc.fuel() << endl;
33     obc.consumeFuel(12.0);
34     cout << "Fuel: " << obc.fuel() << endl;
35 }
```

The important thing here is to realize how much these two programs correspond to each other. It is obvious that one program can easily be transformed into the other and vice versa. Of course, the first program is written specifically to emphasize this.

There is a decisive advantage of the ADT version: The member `OBC::m_fuel` cannot be accessed directly. In this way, the information hiding principle is realized. Whether a free function is called with one or two arguments—as in the first program—or whether a member function of a receiver object with no or only one parameter is called—as in the second program—, does not matter much.

This changes if a version of an on-board computer that also manages the vehicle's mileage is additionally required. In a purely procedural design, all the code written for OBC must be duplicated for the extended on-board computer OBCA, and new functions for the odometer reading must also be provided. Listing 3 shows part of the code specific to OBCA.

Listing 3: `objects/OBC_TypeFunctionsAdvanced.cpp` (excerpt)

```
1 // OBC_TypeFunctionsAdvanced.cpp by Ulrich Eisenecker, January 5, 2023
2
3 #include <iostream>
4 using namespace std;
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31 struct OBCA
32 {
33     double m_mileage,
34           m_fuel;
35 };
36
37 OBCA createOBCA()
38 {
39     return OBCA { 0.0, 0.0 };
40 }
41
42 double fuel(const OBCA& obc)
43 {
44     return obc.m_fuel;
45 }
46
47 void addFuel(OBCA& obc, double fuel)
48 {
49     obc.m_fuel += fuel;
50 }
51
52 void consumeFuel(OBCA& obc, double fuel)
53 {
54     obc.m_fuel -= fuel;
55 }
56
57 double mileage(const OBCA& obc)
58 {
59     return obc.m_mileage;
60 }
61
62 void addMileage(OBCA& obc, double distance)
63 {
```

```

64 |     obc.m_mileage += distance;
65 | }
66 |
67 | int main()
68 | {
69 |     auto obca { createOBCA() };
70 |     cout << "Fuel: " << fuel(obca) << endl;
71 |     cout << "Mileage: " << mileage(obca) << endl;
72 |     addFuel(obca, 42.0);
73 |     cout << "Fuel: " << fuel(obca) << endl;
74 |     addMileage(obca, 200.0);
75 |     consumeFuel(obca, 12.0);
76 |     cout << "Fuel: " << fuel(obca) << endl;
77 |     cout << "Mileage: " << mileage(obca) << endl;
78 | }

```

If there were no function overloading, i.e. overloading polymorphism, in C++, the fuel-related functions for OBCA would have to be named differently. Since OBC and OBCA are different types, it is not permissible to pass, for example, an exemplar of OBCA instead of an exemplar of OBC to the `mileage()` function. Therefore, it is not possible to avoid providing fuel-related functions for OBCA.

1.1.1. Inheritance

A new class can be defined using an existing class so that identical members that are already present in the existing class do not have to be defined again in the derived class. Inheritance therefore solves exactly the problem mentioned above.

Inheritance makes it possible to define a new type so that it is based on an existing type, optionally define new members, and—if necessary—provide new definitions for member functions of the base type. Listing 4 illustrates all of this.

Listing 4: objects/OBC_Inheritance.cpp

```

1 | // OBC_Inheritance.cpp by Ulrich Eisenecker, April 8, 2024
2 |
3 | #include <iostream>
4 | using namespace std;
5 |
6 | class OBC
7 | {
8 |     public:
9 |         OBC(): m_fuel { 0.0 }
10 |         {}
11 |         double fuel() const
12 |         {
13 |             return m_fuel;
14 |         }
15 |         void addFuel(double fuel)
16 |         {
17 |             m_fuel += fuel;

```

```
18     }
19     void consumeFuel(double fuel)
20     {
21         m_fuel -= fuel;
22     }
23 private:
24     double m_fuel;
25 };
26
27 class OBCA: public OBC
28 {
29 public:
30     OBCA(): OBC { }, m_mileage { 0.0 }
31     {}
32     double mileage() const
33     {
34         return m_mileage;
35     }
36     void addMileage(double distance)
37     {
38         m_mileage += distance;
39     }
40 private:
41     double m_mileage;
42 };
43
44 void testFuel(OBC& obc)
45 {
46     cout << "Fuel: " << obc.fuel() << endl;
47     obc.addFuel(42.0);
48     cout << "Fuel: " << obc.fuel() << endl;
49     obc.consumeFuel(12.0);
50     cout << "Fuel: " << obc.fuel() << endl;
51 }
52
53 int main()
54 {
55     OBC obc;
56     testFuel(obc);
57
58     OBCA obca;
59     testFuel(obca);
60     cout << "Mileage: " << obca.mileage() << endl;
61     obca.addMileage(200.0);
62     cout << "Mileage: " << obca.mileage() << endl;
63 }
```

The OBCA class specifies that it inherits from OBC. Syntactically, this is done by a colon after the name of the class to be defined, here OBCA, followed by the specifier `public` and the name of the base class, here OBC. The class OBCA is also called *derived class*, *child class* or *subclass*, while the base class OBC is also called *parent class* or *super class*. The inheritance relationship must be specified with the access-specifier `public`. Only then does OBCA become a *subtype* of OBC. ***Subtype means that wherever an exemplar of***

OBC is expected, an exemplar of OBCA or another direct or indirect public subclass of OBC can also be used. Accordingly, the base class in this case is called a *super type*; here OBC is a super type of OBCA. **It is also said that an OBCA is an OBC, i.e. it is an Is-a-inheritance.**

In the sense of a very precise use of terms, a subtype or super type relationship is also always a subclass or super class relationship. The reverse is not necessarily true: a subclass need not be a subtype.

The OBCA class inherits `m_fuel` from OBC. Since `m_fuel` is private in OBC, OBCA cannot access `m_fuel`. The `addFuel()` and `consumeFuel()` methods, as well as the constructor, are public in OBC and therefore can be called from OBCA. The relationship between access specifiers and inheritance will be discussed in more detail later. Since OBCA inherits publicly from OBC, all public methods inherited from OBC can be accessed from outside OBCA. It is as if all publicly inherited members were defined in the child class.

The `testFuel()` function demonstrates the consequences of *subtyping*. It specifies a reference of type OBC as a parameter. Therefore, any exemplar of OBC that is not `const` can be passed as the actual parameter when `testFuel()` is called. Because of sub-typing an exemplar of OBCA that is not `const` can be passed as a parameter to `testFuel()`, as shown in the `main()` function. Actually, an exemplar of all classes that are directly or indirectly publicly derived from OBC can be used whenever an exemplar of OBC passed by reference is expected. Also, any pointer to an exemplar of a class that publicly inherits from OBC can be used if a pointer to OBC is expected. Of course, care must be taken whether it is a reference to `const` or a pointer to `const`. Then only `const` members can be accessed in the corresponding function. **The fact that a subtype can be used instead of its super type is called type substitutability. Furthermore, if the subtype does not break any code written with respect to its super type, this corresponds to the Liskov (type) substitution principle** ((Liskov and Wing 1994), (Liskov Substitution Principle 2022)).

Inheritance has obvious advantages. It helps avoid code duplication, and in the form of type substitution, it also helps reduce the amount of client code. But it also has a disadvantage. When OBC and OBCA are considered as modules, the inheritance relationship increases the coupling between OBC and OBCA. Whenever OBC is changed, the possible consequences for its child classes must be considered. In addition, if the Liskov substitution principle is to be followed, the developer of a subtype must ensure that the subtype does not break the client code that uses the super type.

The coupling between base class and derived class can be minimized by

- defining only public and private members in the base class (and no protected ones, which will be explained next) and by
- reducing the number of calls to inherited methods in methods of the derived class.

1.1.2. Access Specifiers

In C++ there are three *access specifiers*, namely `public`, `protected` and `private`. They can be used to specify the type of inheritance and the accessibility of members of a struct or a class. The only syntactic difference between a struct and a class is the specific form of access.

When defining a struct, everything is public by default. By specifying `struct A { void f() {} };`, `A::f()` is public by default. It can be accessed from inside A, from outside A, and from any class derived from A. If this specification is followed by `struct B: A { };`, then struct B is publicly derived from A. This means that whenever an exemplar of A is expected, an exemplar of B can also be provided. Therefore, a B can be said to be an A.

When defining a class, everything is private by default. By specifying `class A { void f() {} };`, `A::f()` is private by default. It can only be accessed from within A. It is not accessible from outside or from a subclass. If the definition of A is followed by `class B: A { };`, then the class B is privately derived from A, with the consequence that A and B are not related types from the point of view of the type system, i.e. B is not a subtype of A. It is not possible to provide an exemplar of B if an exemplar of A is expected.

Any form of access that differs from the default must be explicitly specified.

When a struct or class A specifies a member as `protected`, that member can be accessed within A and in any descendant struct or class that inherits public from A. If A is specified as a `protected` or `private` base struct or class in B, B can still access public or `protected` members of A.

If A is a `protected` base class of B, an exemplar of B cannot be used when an exemplar of A is expected. However, all public and `protected` members of A can be accessed within B. Therefore, `protected` inheritance is similar to private inheritance.

Table 1 gives an overview of all combinations of inheritance and member specification, starting with grandparent classes and ending with grandchild classes.

The column labeled *Class name* contains the name of the corresponding class. The class named `Base` is the first base class for all other classes. Any other class name encodes how and in what role it is derived from its base class. For example, `PubChild` means that this class inherits publicly as a direct child of `Base`, and `PriGrandChild` means that this class inherits privately from `PriChild`, which in turn inherits privately from `Base`.

The following columns list the names of the methods that can be called from outside the defining class, from inside the defining class, from its direct descendant class, and from its grandchild class. The names indicate whether the methods are declared public (`pub()`), `protected` (`pro()`), or private (`pri()`). For example, if `Base` is the base class, all methods declared by `Base` are accessible within `Base`. If `PubChild` is the base class, `Base::pub()` and `Base::pro()` are accessible. The child class of `PubChild` is `PubGrandChild`. `PubGrandChild` can also access `Base::pub()` and `Base::pro()`. Since

Table 1: Inheritance and accessibility

Class name	Outside	Base class	Child class	Grandchild class
Base	pub()	pub() pro() pri()	pub() pro()	pub() pro()
PubChild	pub()	pub() pro()	pub() pro()	—
PubGrandChild	pub()	pub() pro()	—	—
ProChild		pub() pro()	pub() pro()	—
ProGrandChild		pub() pro()	—	—
PriChild		pub() pro()		—
PriGrandChild			—	—

there is no grandchild class for `PubChild`, the corresponding cell contains a minus. A minus always means that there is no corresponding class. An empty cell means that there is no method that can be called in this context.

Table 1 shows that there is no difference between protected and private inheritance in the immediate descendant class. Only in the case of indirect descendants does a difference become apparent.

The content of Table 1 can be checked with the program `Access.cpp`. In general, it is a good approach to create a program that uses the programming language itself to provide information about the desired properties. This is demonstrated by the program `Access.cpp`. Nevertheless, it is not reproduced here, but is included in the accompanying material in the `sources/objects` directory. `Base` is the base class for all other classes. It contains one public, one protected, and one private member function. There are three lines of descendant classes, one for public, protected, and private inheritance. Each class defines a `test()` member function. In this method each member function declared by `Base`, i.e. `Base::pub()`, `Base::pro()`, and `Base::pri()`, is called. The calls that are not legal are converted to comments.

The most important form is public inheritance. There are some rare cases where a descendant class may not be used as a subtype of its base class. Nevertheless, it can be implemented by using some or all of the methods of the base class candidate. This form of inheritance is called implementation inheritance. It is best realized in C++ by private inheritance.

1.1.3. Static Binding

All functions and all member functions implemented so far are statically bound. This means that the compiler determines which function should be called. This decision is thus made at compile time. The consequences of static binding are illustrated by object serialization and deserialization, which allows on-board computers to be persisted.

Next, member functions for serialization and deserialization are added to the `OBC` and `OBCA` classes. They are called `output()` and `input()`. Then the `>>` (stream extraction) and `<<` (stream insertion) operators are overloaded. Listing 5 shows only the code that is changed or new compared to Listing 4.

Listing 5: objects/OBC_IO.cpp (excerpt)

```

1 // OBC_IO.cpp by Ulrich Eisenecker, January 9, 2023
2
3 #include <fstream> // because of file streams
4 using namespace std;
5
6 class OBC
7 {
8     public:
9     void output(ostream& os) const
10     {
11         os << m_fuel << endl;
12     }
13     void input(istream& is)
14     {
15         is >> m_fuel;
16     }
17 };
18
19 ostream& operator<<(ostream& os,const OBC& o)
20 {
21     o.output(os);
22     return os;
23 }
24
25 istream& operator>>(istream& is,OBC& o)
26 {
27     o.input(is);
28     return is;
29 }
30
31 class OBCA: public OBC
32 {
33     public:
34     void output(ostream& os) const
35     {
36         os << m_mileage << '\t';
37         OBC::output(os);
38     }
39     void input(istream& is)
40     {
41         is >> m_mileage;
42         OBC::input(is);
43     }
44 };
45
46 /*
47 ostream& operator<<(ostream& os,const OBCA& o)
48 {
49     o.output(os);
50     return os;
51 }
52 */

```



```
82 | istream& operator>>(istream& is,OBCA& o)
83 | {
84 |     o.input(is);
85 |     return is;
86 | }
87 | */
88 | int main()
89 | {
90 |     const string filename1 { "obc1.csv" };
91 |     const string filename2 { "obc2.csv" };
92 |
93 |     {
94 |         ofstream out { filename1 };
95 |         OBC obc;
96 |         obc.addFuel(42.0);
97 |         obc.output(out);
98 |         OBCA obca;
99 |         obca.addFuel(30.0);
100 |         obca.addMileage(150.0);
101 |         obca.output(out);
102 |     }
103 |     {
104 |         ifstream in { filename1 };
105 |         OBC obc;
106 |         obc.input(in);
107 |         cout << obc.fuel() << endl;
108 |         OBCA obca;
109 |         obca.input(in);
110 |         cout << obca.fuel() << endl;
111 |         cout << obca.mileage() << endl;
112 |     }
113 |
114 |     {
115 |         ofstream out { filename2 };
116 |         OBC obc;
117 |         obc.addFuel(42.0);
118 |         out << obc;
119 |         OBCA obca;
120 |         obca.addFuel(30.0);
121 |         obca.addMileage(150.0);
122 |         out << obca;
123 |     }
124 |     {
125 |         ifstream in { filename2 };
126 |         OBC obc;
127 |         in >> obc;
128 |         cout << obc.fuel() << endl;
129 |         OBCA obca;
130 |         in >> obca;
131 |         cout << obca.fuel() << endl;
132 |         cout << obca.mileage() << endl;
133 |     }
134 | }
```

The `main()` function of this program uses external file streams to store and retrieve the state of on-board computers. For this reason, the `<fstream>` header file must be included. The knowledge needed to use file streams in this example is presented in (U. W. Eisenecker 2022), Section 5.5.1 *Include Files*. The input/output member functions and the stream insertion and extraction operators are implemented according to (U. W. Eisenecker 2022), Section 4.5.3 *Overloading Operators for RationalNumber*.

The member function `OBC::output()` takes a reference of type `ostream` named `os`. It sends the value of `OBC::m_fuel` to `os`. The purpose of this member function is to serialize the state of an `OBC` exemplar so that it can be restored later. The point is not to print interesting information about an `OBC` exemplar. Therefore, the member variable `OBC::m_fuel` is accessed directly instead of using the getter function `OBC::fuel()`. `OBC::output()` inserts a new line into the stream by sending `endl` to `os`. It must be possible for derived classes to serialize data members that are defined by themselves. Since it is not known in advance how many data members this may be, the data of an exemplar of `OBC` or a derived class is serialized in a single line, i.e. `output()` cannot serialize more than one object in a single line of text. The state of an exemplar of `OBC` or a derived class is serialized as plain text. Consequently, the content of the data stream is human readable. `OBC::output()` is declared as a `const` member function because it must not change the receiver object.

The method `OBC::input()` is implemented symmetrically. It takes a reference of type `istream` and extracts a value from `is` into the `OBC::m_fuel` data member. No special care is needed when extracting the terminating new line, since it is considered a white space and is silently consumed when the next `OBC` data line is read. Since `OBC::input()` modifies the receiver object, it is not declared as `const`.

Immediately after the definition of the `OBC` class, the `<<` and `>>` operators are overloaded as free functions. The `<<` operator takes a reference of type `ostream` named `os` as its first parameter and a reference to `const` of type `OBC` named `o` as its second parameter. In its implementation, it calls `output()` for `o` and passes `os` as an argument. Finally, `os` is returned, so it is possible to chain calls to the stream insertion operator.

The `>>` operator takes as its first parameter an `istream` reference named `is` and as its second parameter an `OBC` reference named `o`. It calls `input()` for `o`, passes `is` as an argument, and then returns `is`. The latter allows chaining of calls to the stream extraction operator.

The `OBCA::output()` method is a new definition of `OBC::output()`, i.e. it specifies the same interface but provides a new implementation. It is said that the definition of `OBCA::output()` *hides* the inherited method `OBC::output()`. The first statement of the `OBCA::output()` method sends the value of `OBCA::m_mileage` to the `os` output stream. It should be emphasized that immediately after the mileage, an additional tab is sent to `os`. The second statement calls `OBC::output()`. As mentioned before, `OBCA::output()` hides `OBC::output()`. To call the base class method, it must be preceded by the name of the base class, here `OBC`, followed by the scope operator, i.e. `::`. The call to `OBC::output()`

outputs the value of `OBC::m_fuel` immediately after the tab. Finally, a new line is inserted into the output stream. As a result, the corresponding line in the output stream contains two integers separated by a tab and terminated by a new line. The first value refers to the mileage, the second to the fuel. Files containing data separated by tabs or other standard delimiters are called *Comma-Separated-Value* (CSV for short) files. Originally, commas were used instead of tabs to separate values, which can be numbers (integers or floating point numbers) or strings (without spaces). In some languages, such as German, a comma is used instead of a decimal point for floating point numbers. Therefore, using a tab instead of a comma is a better choice. However, many programs that are able to import CSV files also allow the use of other symbols as separators, e.g. semicolons. Most popular spreadsheet programs can import the CSV file created by the program shown in Listing 5.

The two operator definitions after the `OBCA` class are ignored for now; they are converted to comments.

In `main()` two string constants are defined and initialized. They contain the names of the files that will be used later to store the values of the `OBC` and `OBCA` exemplars.

Now four blocks follow. In each block a file stream is declared and initialized. The associated file objects are opened immediately with the specified names. At the end of the declaration block, each file stream is automatically closed by its destructor, which is called when the file object's lifetime ends.

The first block serializes an `OBC` and an `OBCA` object by calling their `output()` methods directly. In the second block, exemplars of `OBC` and `OBCA` are created and their state is deserialized from the input stream `in`. For control, the corresponding values are output to `cout`.

The third block does the same as the first block. However, now the stream insertion operator is used to send the state of the `OBC` and `OBCA` objects to the output stream `out`. In the fourth block, new exemplars of `OBC` and `OBCA` are created and their states are deserialized from the input stream `in`, now using the stream extraction operator. Then their values are output by calling the appropriate methods on `cout`. Checking this output, it can be seen that an incorrect value is given for the mileage. It should be 150, but the reported value is 0.

A debugger can be used to find out the cause. When debugging the statement `out << obc;` in the third block, `OBC::output()` is called, which is correct. Debugging the statement `out << obca;` will again call `OBC::output()`, now for an `OBCA` object. Similarly, the input operator always calls `OBC::input()`, even if an `OBCA` exemplar is passed to it as the second parameter. This is probably not what one would naively expect, but it is correct.

When the compiler processes the statement `o.output(os);` in the `<<` operator, it knows from the declaration of the parameter that `o` is an exemplar of type `OBC`. This type is called *static type* because it is what the compiler sees in the declaration. The static type

may be different from the *true type*, which is also called the *dynamic type*. The true type corresponds to the actual type of the object, which may be the same as the static type or a subtype of it.

According to the static type, the compiler generates a call to `OBC::output()` regardless of the true type of the actual parameter. This is called *static binding*. ***Static binding is the de facto binding of functions and member functions.***

Static binding is not to be confused with static member functions, which were briefly introduced in (U. W. Eisenecker 2022), Section 4.5.3 *Overloading Operators for Rational-Number* and are discussed in detail in *Static Members*. A static member function is also statically bound.

When the two operator definitions contained in the comment following the `OBCA` class are turned from comments into active statements, the resulting program produces the expected results. The largely redundant operator definitions use overloading polymorphism to achieve the desired behavior. Overall, this solution is not scalable and is difficult to maintain. It can also be easily circumvented by using references to `OBC` for objects of type `OBCA`. For example: `OBCA obca; OBC& obc { obca }; cout << obc;` would result in a call to the `<<` operator, which is overloaded for `OBC`.

1.2. Late Binding

Late binding enables the execution of code that was written at a later point in time than the code from which it is called. Alongside inheritance, this is the second important feature that object-oriented programming adds to the concept of ADTs.

The above problem caused by static binding can be remedied by so-called *virtual methods*, also called *virtual (member) functions*. They are *dynamically bound* and are said to have *late binding*. Late binding allows a member function to be called that is specific for the true type of an object. It is decided at runtime which virtual member function will be called. The use of virtual functions is effective only in conjunction with inheritance and passing arguments as references or pointers. To illustrate this, the on-board computer and its specializations are used as an example.

It is easy to turn a member function into a virtual function. Syntactically, this is achieved by prefixing the declaration with the `virtual` keyword. For example, in the definition of the `OBC` class, the `output()` member function must be declared as `virtual void output(ostream& os) const`. Then this member function will be bound dynamically. Once a function is declared `virtual` in a base class, it is automatically `virtual` in all derived classes. Thus, if `OBC::output()` is `virtual`, the `OBCA::output()` method is also automatically `virtual`. Although it is possible to declare it as `virtual` again in a derived class, this has no effect. Some authors recommend providing redundant `virtual` declarations in derived classes. Basically, this is not really helpful, because one always has to check if there is a base class in which this method is already declared as `virtual`. So,

in order to fully understand a class, it is inevitable in object-oriented programming to go through the inheritance hierarchy. As mentioned earlier, this shows that inheritance reduces modularity by increasing coupling between related classes.

When a subclass provides a new definition for a virtual member function of its base class, this is called overriding. This is different from providing a new definition for a non-virtual member function of a base class, which hides the inherited method in the subclass.

Virtual functions are the first ingredient to the so-called *inclusion polymorphism*, which is a core concept of object-oriented programming. Another ingredient is public inheritance between classes and the passing parameters either as reference or as pointer. When a parameter is passed by reference, its name is just an alias for the object being passed. This preserves the true type of the object. If a pointer is passed as an argument, the object pointed to by the pointer is also not changed or replaced, so the true type of the object is preserved. Passing an argument by value always results in the creation of a new object of the specified type. If an exemplar of a subtype is passed instead of a super type, the compiler automatically generates and calls a corresponding type conversion constructor. Thus, the true type of the passed argument is always the same as the specified static type. However, the most important point is that an object passed by value is always a different object in the function than the object specified as an argument. Figure 1 shows the various options side by side.

Figure 1: Parameter passing and inclusion polymorphism

<pre> SubClass c; // call by reference void f(BaseClass& b); f(c); void f(BaseClass& b) { /* c is an alias of type BaseClass for c; */ &b == &c; } </pre>	<pre> // call by pointer void f(BaseClass* b); f(&c); void f(BaseClass* b) { /* b is a pointer of type BaseClass to c */ &(*b) == &c; } </pre>	<pre> // call by value void f(BaseClass b); f(c); void f(BaseClass b) { /* b is of type BaseClass and different from c */ &b != &c; } </pre>
--	--	--

Listing 6 shows only the changes in the new program compared to Listing 5 with late binding enabled for `OBC::output()` and `OBC::input()`. There are no modifications in `OBCE`, because the changes in `OBC` automatically affect `OBCE::output()` and `OBCE::input()`. The comment with the stream operators overloaded for `OBCE` has been removed as they are no longer needed.

Listing 6: objects/OBC_LB.cpp (excerpt)

```

1 // OBC_LB.cpp by Ulrich Eisenecker, January 10, 2023
2
7
8 class OBC
9 {
10     public:
25         virtual void output(ostream& os) const
26         {
27             os << m_fuel << endl;
28         }
29         virtual void input(istream& is)
30         {
31             is >> m_fuel;
32         }
35 };

```

If the declaration and the definition of a member function are separate, `virtual` may only be used in the declaration. It must not be repeated in the definition of the member function. As mentioned before, only member functions can be declared as `virtual`. Free functions or static member functions cannot be declared as `virtual`.

It is important that the `OBC` exemplar is passed by reference to the overloaded `<<` and `>>` operators so that its true type is preserved. When the `<<` operator sends an exemplar of `OBC` to `cout`, `OBC::output()` is called. When an exemplar of `OBCE` is sent to `cout`, `OBCE::output()` is called. It is very instructive to observe this behavior with a graphical debugger. The same is true for the call to the `>>` operator. When inputting an `OBC` exemplar, `OBC::input()` is called, when inputting an `OBCE` exemplar, `OBCE::input()` is called.

Interestingly, this also applies to classes that do not yet exist. If a new `OBCE` class (on-board computer electric) were derived from `OBCE`, providing a new definition for `output(ostream& os) const`, `OBCE::output()` would be called when an exemplar of `OBCE` is output. That is, both operators, `<<` and `>>`, work for any future class publicly derived from `OBC`.

Another interesting aspect is that it depends only on the true type of an object, which virtual member function is called. For the `OBCE` `obca; OBC& obc { obca }; cout << obc;` statements, sending `obc` to `cout` would result in a call to `OBCE::output()`, since the true type of `obc` is `OBCE`. Similarly, compiling and executing `OBCE obca; OBC* obc {`

`&obca }; cout << *obc;` will also result in a call to `OBca::output()`, since `obc` actually points to an `OBca` exemplar, i.e. the pointer `obc` points to an object whose true type is `OBca`.

This underlines the importance of late binding: Only when a call to a virtual member function is made at runtime is it decided which member function is actually called, depending on the true type of the object.

1.3. Kinds of Polymorphism

Polymorphism comes from the Greek and means that something can exist in several different forms. The different types of polymorphism are essential for understanding programming languages and can be regarded, at least in part, as programming paradigms in their own right.

According to (Cardelli and Wegner 1985), there are four kinds of polymorphism: *parametric polymorphism*, *inclusion polymorphism*, *overloading polymorphism*, and *coercion polymorphism*.

Parametric polymorphism, also known as *generic polymorphism*, is introduced in the Chapter *The Generic Programming Paradigm*.

Coercion polymorphism deals with type conversion, e.g. converting an `int` into a `double` when necessary. An example of coercion polymorphism is presented in (U. W. Eisenecker 2022), Section 4.3.3 *Class Design Based on Reference Semantics*. There, the constructor of the class `RationalNumber` acts as a type conversion constructor when called with an `int` as its only argument. This is a case of coercion polymorphism.

Function overloading is the manifestation of the overloading polymorphism introduced in (U. W. Eisenecker 2022), Section 3.3.5 *Function Overloading*, and used frequently thereafter.

Public inheritance in combination with late binding and passing arguments by reference or as pointer realizes inclusion polymorphism.

Care must be taken when these various polymorphisms interact. For example, when passing a parameter to a function, the coercion polymorphism and the overloading polymorphism may compete with each other. This conflict cannot be resolved by the compiler. Therefore, it is reported as an error.

A different situation arises when the stream insertion and extraction operators overloaded for `OBca` are reintroduced into the program with late binding (Listing 6). Then the compiler calls them when an `OBca` exemplar is sent to `cout` or read from `cin` because the overloaded functions are considered more specific than the virtual member functions. Here, the overloading polymorphism takes precedence over the inclusion polymorphism.

Coercion polymorphism also takes precedence over inclusion polymorphism. This is easily demonstrated by the overloaded operators in the program *OBC_LB.cpp* (Listing 6, the definitions of the input/output operators are included in Listing 5). Here, the parameter *o* is passed as a reference to `const` of type `OBC` to the `<<` operator and as a reference of type `OBC` to the `>>` operator. Passing *o* by reference is an essential requirement for inclusion polymorphism. If *o* were passed by value, the stream insertion operator would always call `OBC::output()` and the stream extraction operator would always call `OBC::input()`. The reason for this is that passing a parameter by value always creates a new exemplar of the specified type either by copying or by applying a type conversion if available. Consequently, there is always an exemplar of the specified type, here `OBC`, for which a particular method is called, here `OBC::input()` or `OBC::output()`. Even if these methods have a late binding, this does not matter, because they are called exactly for the specified class of the parameter.

Another way to enable inclusion polymorphism is to pass a pointer to an object. Although the pointer itself is copied, the object to which the pointer points is not copied. In this way, inclusion polymorphism is also preserved.

The interplay between the different varieties of polymorphism is sometimes difficult, sometimes confusing, and sometimes indispensable to finding a solid solution to difficult problems. The interplay of polymorphisms will be discussed again as needed.

1.4. Override

When a member function is overridden, the programmer can easily make a mistake, an example of which is given below. The override specification helps to avoid such errors. `OBC` and its subclasses are used for illustration.

It is assumed that—for whatever reason—an error is made when overriding `OBCA::output()`. Instead of passing *os* as a reference of type `ostream`, a reference of type `ofstream` is specified. `ofstream` is a subtype of `ostream` defined in `<fstream>`, which is included at the beginning of the program because `main()` uses file stream objects. Compared to *OBC_LB.cpp*, the only changes in *OBC_Error.cpp* are in line 64 (Listing 7) and the two comment lines inserted above it. The rest of the two programs is identical.

Listing 7: objects/OBC_Error.cpp (excerpt)

```
49 | class OBCA: public OBC
50 | {
51 |     public:
52 |         // Next, ostream& is replaced by its
53 |         // subtype ofstream.
54 |         void output(ofstream& os) const
55 |         {
56 |             os << m_mileage << '\t';
57 |             OBC::output(os);
58 |         }
59 | };
76 |
```

Interestingly, this program can be compiled and runs without errors. Only on closer inspection of the program output does it become noticeable that an OBCA exemplar with the values 30 (fuel) and 150 (mileage) has been serialized, but an OBCA exemplar with the values 0 (fuel) and 30 (mileage) is deserialized. This can be easily determined by checking the contents of the *obc1.csv* and *obc2.csv* files. *obc1.csv* contains the correct values, namely a first line for an OBC and a second line for an OBCA. But *obc2.csv* contains two OBC lines. The cause of this error can be found by thinking about it or, perhaps more easily, by using a debugger. The debugger shows that sending *obca* to *cout* calls `OBC::output()` and not `OBCA::output()`, as would be expected. **When a method is overridden, its signature must be exactly the same as in the base class** (without the return type). If this is not the case, the method in the derived class does not override the corresponding method in the base class. Instead, it *hides* the base class method.

The direct call to `OBCA::output()`, as made in the first nested block of `main()`, works fine because *cout* is an exemplar of type `ofstream`. But the indirect call in the third nested block by the statement `out << obc;` results in a call to `OBC::output()`. In fact, `OBCA::output()` does not override `OBC::output()` and therefore has only a static binding. Only if `OBCA::output()` correctly overrides `OBC::output()` does it benefit from late binding by specifying `OBC::output()` as `virtual`. Incidentally, specifying `OBCA::output()` as `virtual` is not a solution to this problem either. This would create a second `OBCA::output()` function, which is defined the first time in the OBCA class with late binding.

Thorough and comprehensive unit tests are required to detect this artificially introduced error. Fortunately, there is an excellent way to detect this error during compilation. Whenever a method should override a virtual method defined in one of its direct or indirect base classes, it should be specified as `override`. This is done by adding the `override` keyword to the end of the function declaration, as Listing 8 shows.

Listing 8: objects/OBC_ErrorOverride.cpp (excerpt)

```

50 class OBCA: public OBC
51 {
52     public:
53     void output(ofstream& os) const override
54     {
55         os << m_mileage << '\t';
56         OBC::output(os);
57     }
58 };

```

Now, when compiling the program, an error occurs. It says that `OBCA::output()` is marked as `override`, but actually does not override any member function. A warning can also be generated for `OBCA::input()`, since it is also not marked as `override`.

Only methods that are already declared as virtual in a base class can be specified with `override`.

It is strongly recommended that any method that overrides is marked as `override`. This helps to avoid errors that are otherwise difficult to detect. The use of `override` is much more useful than a redundant `virtual` specification.

Listing 9 shows the code snippet that specifies `OBCA::output()` and `OBCA::input()` with `override`. Also, the type of the `ofstream` parameter has been changed back to `ostream`. This program compiles without errors and executes correctly.

Listing 9: objects/OBC_Override.cpp (excerpt)

```

49 class OBCA: public OBC
50 {
51     public:
52     void output(ostream& os) const override
53     {
54         os << m_mileage << '\t';
55         OBC::output(os);
56     }
57     void input(istream& is) override
58     {
59         is >> m_mileage;
60         OBC::input(is);
61     }
62 };

```

It has already been mentioned that the return type of a member function is not considered in the same way as the number and types of the parameters when overriding. C++ allows the so-called *covariant specialization of the return type*, also known as *covariant return types*, which is not explained here.

1.5. Pure Virtual Functions

Sometimes it is desirable to declare a virtual function without providing a definition, i.e. an implementation, to force a subclass to define this member function. It may also be desirable to define classes that cannot be instantiated. Both can be achieved by using *pure virtual functions*.

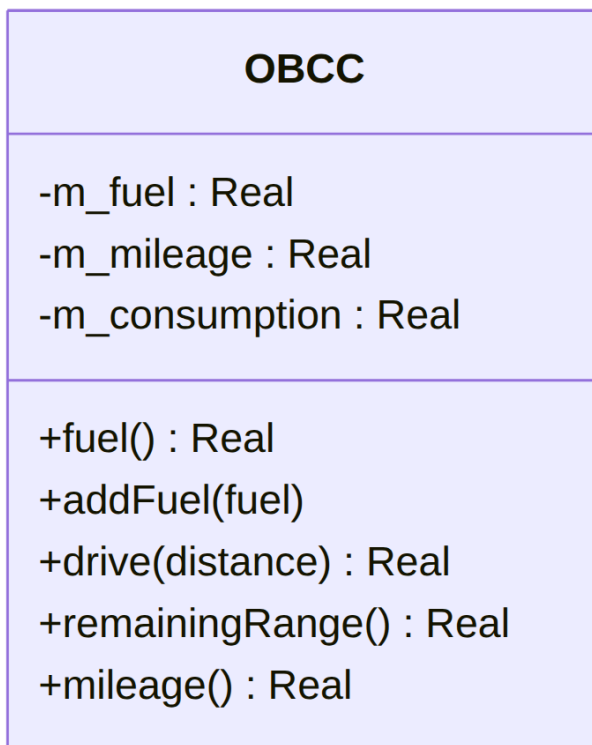
Sometimes, a base class needs to call a member function for which only a subclass can provide a valid implementation. An *abstract method* (object-oriented terminology), also called a *pure virtual function* (C++ terminology), serves exactly this purpose. For the introduction, on-board computers are used again. The corresponding example includes a major refactoring.

Suppose a car manufacturer decides to install an on-board computer. The manufacturer produces cars with conventional combustion engines and cars with electric motors. The basic function of the on-board computer is to display the mileage and the remaining range depending on the current fuel level or battery charge status.

The current design of the OBC class does not support all of these requirements. Therefore, a new class called OBCC is being developed. Its name is an acronym for *on-board computer combustion*. Figure 2 shows a *UML* class diagram for OBCC.

The upper compartment contains the name of the class. The middle compartment contains the attributes of the class. If an attribute is preceded by a *minus*, this means that it is private. The type of an attribute is specified after the colon. The lower compartment contains the methods. They are all public, which is indicated by the preceding *plus* sign. Since `addFuel()` does not return a value, the return type specification has been omitted. For all other methods, `Real` is specified as the return type after the colon. If a method has a parameter, it is specified with its name but without its type. Here it is assumed that each parameter is of type `Real`. The constructor of the class OBCC is not specified. In the implementation, the *UML* type `Real` is mapped to the C++ type `double`.

Listing 10 shows the implementation of the OBCC class. The attributes from Figure 2 are implemented as private data members of type `double`. `OBCC::m_consumption` is specified as `const`. This means that it must be initialized in the initializer list of the constructor and cannot change afterwards. It contains the average consumption of the car, which is passed as the only argument of the single constructor.

Figure 2: OBCC—UML class diagram

Listing 10: objects/OBCC.cpp (excerpt)

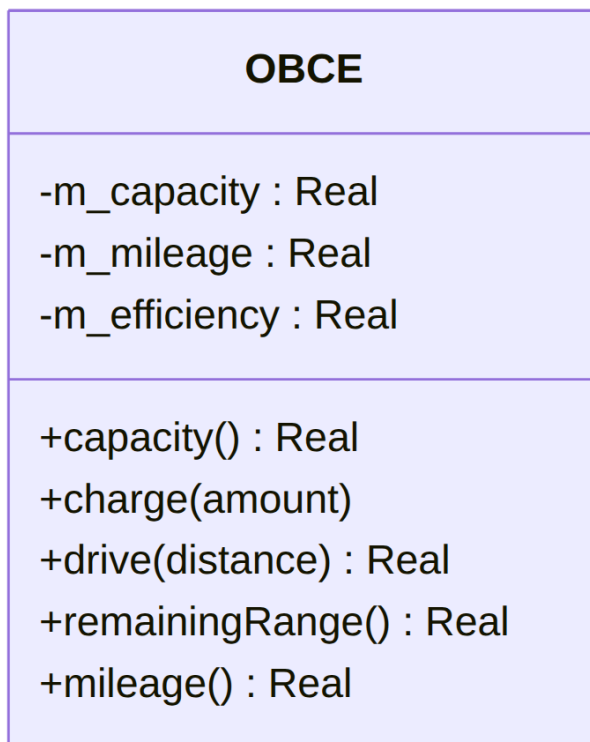
```
6 class OBCC
7 {
8     public:
9         OBCC(const double consumption): m_fuel { 0.0 }, m_mileage { 0.0 },
10                                         m_consumption { consumption }
11     {
12         [[nodiscard]] double fuel() const
13         {
14             return m_fuel;
15         }
16         void addFuel(double fuel)
17         {
18             m_fuel += fuel;
19         }
20         double drive(double distance)
21         {
22             auto requiredFuel { distance / 100.0 * m_consumption };
23             if (requiredFuel > m_fuel)
24             {
25                 distance = m_fuel * 100.0 / m_consumption;
26                 requiredFuel = m_fuel;
27             }
28             m_fuel -= requiredFuel;
29             m_mileage += distance;
30             return distance;
31         }
32         [[nodiscard]] double remainingRange() const
33         {
34             return m_fuel * 100.0 / m_consumption;
35         }
36         [[nodiscard]] double mileage() const
37         {
38             return m_mileage;
39         }
40     private:
41         double m_fuel, // liter or gallons
42               m_mileage; // km or miles
43         const double m_consumption; // liter or gallons per 100 km or miles
44 };
```

OBCC::fuel() is a getter method that returns the value of OBCC::m_fuel. OBCC::addFuel() is a kind of setter method, as it adds the specified amount of fuel to OBCC::m_fuel. OBCC::mileage() is also a getter method. It returns the value of OBCC::m_mileage. The other two methods perform calculations. OBCC::remainingRange() calculates the distance that can be traveled using the value of OBCC::m_fuel. OBCC::drive() takes a parameter for the distance to be covered. It is a bit more complex. First, it calculates the amount of fuel needed to travel the distance passed as a parameter. It stores the result in the requiredFuel local variable. Then it checks if the required fuel is greater than the available fuel. If this is the case,

it calculates how far the car can drive on the current amount of fuel. The result is assigned to `distance`. Therefore, this parameter is passed as a value to `OBCC::drive()`. Another consequence is that it assigns `requiredFuel` the value of `OBCC::m_fuel`. Then, it subtracts `requiredFuel` from `OBCC::m_fuel` and increments `OBCC::m_mileage` by `distance`. Finally, the method returns `distance`. This allows the caller to check whether the specified distance was covered completely or whether only part of the specified distance could be covered. The function `main()` tests the methods of `OBCC`, which is not shown here.

Next, the `OBCE` class is designed and implemented. `OBCE` is an acronym for *on-board computer electric*. Figure 3 shows its *UML* class diagram.

Figure 3: `OBCE`—*UML* class diagram



The private attribute `m_capacity` stands for the current capacity of the battery, the public method `capacity()` returns it. The private attribute `m_efficiency` indicates how much capacity is needed to drive 100 distance units. It is initialized in the constructor, which is not shown in the class diagram. The public method `charge()` increases the capacity of the battery by the specified value.

Listing 11 shows the implementation of the `OBCE` class. It is largely the same as the implementation of the `OBCC` class. Therefore, a detailed explanation of its members is omitted.

Listing 11: objects/OBCE.cpp (excerpt)

```
6 class OBCE
7 {
8     public:
9         OBCE(const double efficiency): m_capacity { 0.0 }, m_mileage { 0.0 },
10                                     m_efficiency { efficiency }
11     {
12         [[nodiscard]] double capacity() const
13         {
14             return m_capacity;
15         }
16         void charge(double amount)
17         {
18             m_capacity += amount;
19         }
20         double drive(double distance)
21         {
22             auto requiredCapacity { distance / 100.0 * m_efficiency };
23             if (requiredCapacity > m_capacity)
24             {
25                 distance = m_capacity * 100.0 / m_efficiency;
26                 requiredCapacity = m_capacity;
27             }
28             m_capacity -= requiredCapacity;
29             m_mileage += distance;
30             return distance;
31         }
32         [[nodiscard]] double remainingRange() const
33         {
34             return m_capacity * 100.0 / m_efficiency;
35         }
36         [[nodiscard]] double mileage() const
37         {
38             return m_mileage;
39         }
40     private:
41         double m_capacity, // kWh
42               m_mileage; // km or miles
43         const double m_efficiency; // kWh per 100 km or miles
44 };
```

OBCC and OBCE are classes that are completely separate from each other. The next step is to think about an abstraction of the two classes that captures their commonalities and can serve as a base class for deriving OBCC and OBCE. This common base class is called OBC.

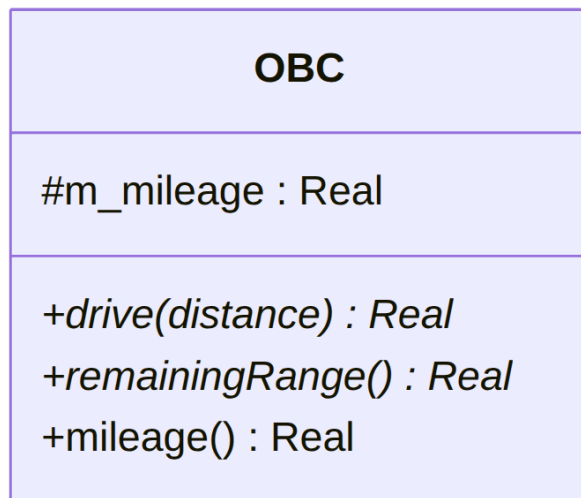
Obviously, `m_mileage` is a literal duplicate in both classes. So it can become an attribute of the OBC base class. The `mileage()` method simply returns the value of `m_mileage` and can therefore also be moved to the base class. `OBC::m_mileage` must be changed when a car is driving. As explained below, there will be no definition of the member func-

tion `drive()` in OBC. Therefore, `OBC::m_mileage` must be modified by a `drive()` member function defined in a subclass. If `OBC::m_mileage` is `private`, it is impossible to access it in a derived class. Introducing a public method for setting the mileage in OBC is not a viable option, as this would allow manipulation of the mileage by any external client. For this reason, `OBC::m_mileage` is declared as a protected member. This prevents modification from the outside, but preserves modifiability by derived classes. It should be mentioned that this action increases the coupling between all derived classes of OBC that access `OBC::m_mileage`. Removing `OBC::m_mileage` would affect all children of the base class.

Despite the structural similarity of `OBCC::drive()` and `OBCE::drive()`, these methods cannot be moved to the base class because they use differently named members for their computations. The question now is whether `OBCC::m_fuel` and `OBCE::m_capacity` can be renamed with an abstract and more general term. Consequently, `OBCC::addFuel()` and `OBCE::charge()` had to be renamed as well. This is not a good idea, because it would pretend that different things that do not in reality belong together are the same thing. A battery and a fuel tank are two different things that should not be confused with each other. Nonetheless, polling remaining range and driving are valid operations that apply to both classes. Fortunately, there is a solution to this problem. `drive()` and `remainingRange()` become so-called *abstract methods* in the OBC base class. If a class contains at least one abstract method, the entire class becomes *abstract*. **An abstract class cannot be instantiated, i.e. it is impossible to create an exemplar of it.** In the *UML* class diagram, the name of an abstract method and the name of an abstract class are in *italics*, as shown in Figure 4. Nevertheless, an abstract class is a valid base class for any class derived from it. The *UML* notation does not distinguish between early and late binding of methods. As a rule, *UML* assumes that methods have late binding by default. The hash in front of `m_mileage` means that it is a protected attribute.

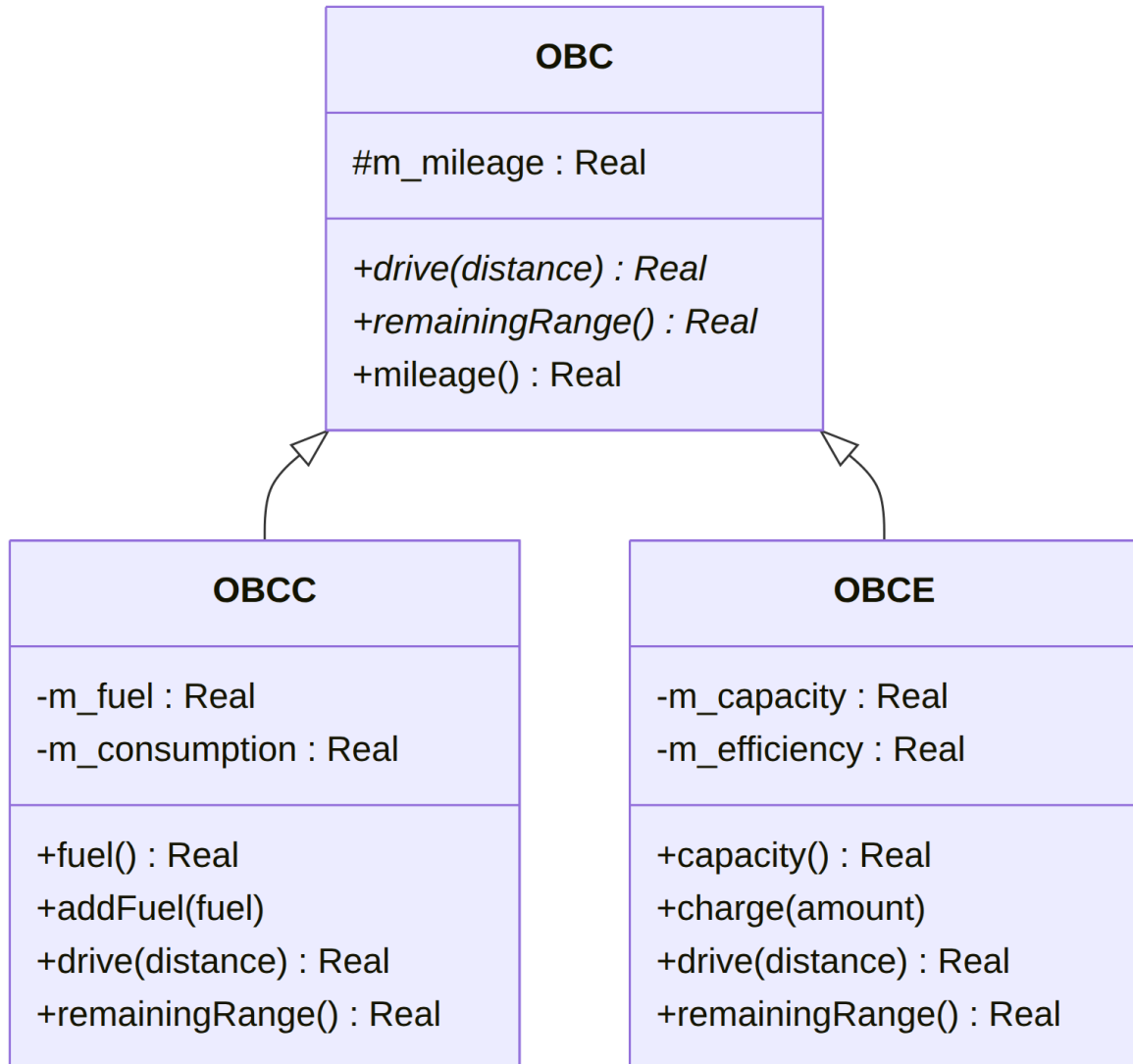
In C++, an abstract method is mapped to a *pure virtual member function*. A class that contains at least one pure virtual member function becomes an abstract class. It is also not allowed to instantiate an abstract class. Normally, a pure virtual member function is only declared, but not defined. Nevertheless, it may have a definition, but it must be outside the class. A pure virtual member function must be declared as `virtual` followed by `= 0` at the end of the function header, for example `virtual void drive(double distance) = 0;`. An abstract class is also a valid base class in C++. That is, it is possible to derive OBCC and OBCE as subclasses of OBC. If a pure virtual member function is not overridden in the derived class this child class is still abstract. To become a class that can be instantiated, a class must not contain a pure virtual member function, whether inherited or self-defined.

It is essential that a class that can later serve as a base class defines a virtual destructor. The reason for this is explained below in the Section *Virtual Destructor*. This also applies to the OBC class.

Figure 4: OBC—UML class diagram

The resulting *UML* class diagram is shown in Figure 5. The hash sign in front of `m_mileage` means that this attribute is protected. That is, the attribute can be accessed from the class that defines it and from its subclasses but not from outside. A line ending with an unfilled arrowhead represents an inheritance relationship. The line extends from the derived class to the base class, with the arrowhead attached to the base class.

Listing 12 shows the corresponding classes. This is an excerpt from the header file, which contains the definitions of the classes as well as all member functions in the form of inline definitions.

Figure 5: OBC—Inheritance Hierarchy

Listing 12: objects/OBCH.hpp (excerpt)

```
6 class OBC
7 {
8     public:
9         OBC(): m_mileage { 0.0 }
10        {}
11        [[nodiscard]] double mileage() const
12        {
13            return m_mileage;
14        }
15        virtual double drive(double distance) = 0;
16        [[nodiscard]] virtual double remainingRange() const = 0;
17        virtual ~OBC()
18        {}
19    protected:
20        double m_mileage; // km or miles
21};
22
23class OBCC: public OBC
24{
25    public:
26        OBCC(const double consumption): OBC { },
27                                         m_fuel { 0.0 },
28                                         m_consumption { consumption }
29        {}
30        [[nodiscard]] double fuel() const
31        {
32            return m_fuel;
33        }
34        void addFuel(double fuel)
35        {
36            m_fuel += fuel;
37        }
38        double drive(double distance) override
39        {
40            auto requiredFuel { distance / 100.0 * m_consumption };
41            if (requiredFuel > m_fuel)
42            {
43                distance = m_fuel * 100.0 / m_consumption;
44                requiredFuel = m_fuel;
45            }
46            m_fuel -= requiredFuel;
47            m_mileage += distance;
48            return distance;
49        }
50        [[nodiscard]] double remainingRange() const override
51        {
52            return m_fuel * 100.0 / m_consumption;
53        }
54    private:
55        double m_fuel; // km or miles
56        const double m_consumption; // liter or gallons per 100 km or miles
```

```

57 };
58
59 class OBCE: public OBC
60 {
61     public:
62         OBCE(const double efficiency): OBC { },
63                                         m_capacity { 0.0 },
64                                         m_efficiency { efficiency }
65     {}
66     [[nodiscard]] double capacity() const
67     {
68         return m_capacity;
69     }
70     void charge(double amount)
71     {
72         m_capacity += amount;
73     }
74     double drive(double distance) override
75     {
76         auto requiredCapacity { distance / 100.0 * m_efficiency };
77         if (requiredCapacity > m_capacity)
78         {
79             distance = m_capacity * 100.0 / m_efficiency;
80             requiredCapacity = m_capacity;
81         }
82         m_capacity -= requiredCapacity;
83         m_mileage += distance;
84         return distance;
85     }
86     [[nodiscard]] double remainingRange() const override
87     {
88         return m_capacity * 100.0 / m_efficiency;
89     }
90     private:
91         double m_capacity; // kWh
92         const double m_efficiency; // kWh per 100 km or miles
93 };

```

All methods of an abstract class that have a definition can be called. If an abstract method has its own definition outside the class, this also applies. For example, the `OBC` constructor initializes `OBC::m_mileage` and must be called by a constructor of a derived class. If a derived class does not specify a user defined constructor or a user-defined constructor does not explicitly call the base class constructor `OBC::OBC()`, it is called automatically. This is possible because `OBC::OBC()` is a standard constructor and as such it can be called automatically. All other constructors must be called explicitly from a subclass constructor (with a few exceptions).

`OBC::mileage()` returns the mileage. This method is statically bound, i.e. it is early bound. Therefore, it cannot be overridden. Next, the pure virtual member functions `OBC::remainingRange()` and `OBC::drive()` are declared. Since there is no code that can be executed in their defining class, no definitions are available for them outside the class.

Therefore, it is not possible to call them explicitly. In most cases, a pure virtual member function has no definition. If not, it cannot be defined inline. As mentioned before, it must be defined separately outside the class.

Then a virtual destructor is defined. Since it does not perform any specific action, it has an empty implementation. As stated before, any class that can be used as a base class must define a virtual destructor. Since an abstract class cannot be instantiated, its only purpose is to serve as a base class. **Therefore, all abstract classes must define a virtual destructor.** The reason for this is explained below in Section *Virtual Destructor*.

`OBC::m_mileage` is defined as a protected data member because it must be accessed in derived classes.

The `OBCC` class is publicly derived from `OBC`. Its constructor takes a parameter that specifies the fuel consumption. No units are specified. The only assumption is that consumption is the ratio of fuel units and 100 distance units. In its initialization list, the constructor first calls the base class constructor using `OBC { }`. After the comma, `OBCC::m_fuel` is initialized to 0.0, and after another comma, `OBCC::m_consumption` is initialized to consumption. No further initializations are required. Since there are no specific actions beyond initializing the members, the constructor body is empty.

The member functions `OBCC::fuel()` and `OBCC::addFuel()` are declared inline. The pure virtual member function `drive()` is overridden. The keyword `virtual` is not specified redundantly, but the member function is marked with `override`. The implementation is exactly the same as in the earlier `OBCC` class. Since `OBC::m_mileage` is protected, it can be used as if it were declared in the `OBCC` class. Everything applies accordingly to the member function `remainingRange()`. Now both inherited pure virtual member functions have definitions. So `OBCC` no longer contains a pure virtual member function and becomes a class that can be instantiated. Finally, the data members `OBCC::m_fuel` and `OBCC::m_consumption` are declared.

Everything that has been explained here also applies to `OBCE`. Therefore no further explanation is given.

It should be emphasized that the *UML* class diagram in Figure 5 and the class definitions in Listing 12 have much in common, even if they have minor differences in detail. This is one of the advantages of object-oriented programming: It provides the same concepts and their relationships for analysis, design, and implementation throughout.

The file `OBCH_UnitTests.cpp`, which is not reproduced here, includes `OBCH.hpp` and performs unit tests for all methods of all classes except their destructors with *Catch2* v3. It contains nothing new and is therefore not discussed here. It is contained in the `sources/objects` directory of the accompanying material. To compile the program, the environment variables `C_PLUS_INCLUDE` and `LIBRARY_PATH` should be set and contain the installation path of *Catch2* v3. Executing `g++ -std=c++20 OBCH_UnitTests.cpp`

`-lCatch2Main -lCatch2` in a terminal window then creates the executable file `a.out`. It is absolutely necessary to specify the `-lCatch2Main` option before `-lCatch2`. Otherwise, some compilers on some platforms may generate linker errors.

1.6. Constructors and Destructor

All exemplars of fundamental data types and user-defined types have a life cycle that begins with the creation of an exemplar and should end with its destruction. Therefore, special attention will be paid below to these two important points in the life cycle of an object.

The construction and destruction of objects is discussed for fundamental types and for user-defined types. The following kinds of objects are also highlighted in this context:

- They exist only once per program (extern),
- they exist outside of blocks, i.e. *global objects*,
- they are defined within a block but have a lifetime that does not depend on the defining block (static),
- they are created dynamically.

All of them have specific properties with respect to their lifetime and their accessibility. Artificial examples are used for illustration.

The life cycle of an object begins with its construction. After that, it is used by modifying or reading its state. Finally, an object is destroyed. The construction of an object is done by executing a *constructor*. **A class can have many constructors.** Some of them can be generated automatically by the compiler. **All constructors have in common that they do not specify a return type, not even `void`.** **An object is assumed to be fully initialized after the call to one of its constructors is completed.** **As a rule, an object should not require further initialization after creation.**

When an object terminates, its *destructor* is called. **A class has exactly one destructor**, which is either automatically generated or provided by the programmer. **A destructor has no arguments.** Normally, a destructor is called automatically when the object terminates. There are a few cases where a destructor is called explicitly. They are not discussed here.

In the following, various possibilities for calling constructors are presented. It is also outlined which types of constructors there are, which of them can be generated automatically and which can or must be implemented manually. In addition, some information is given about when objects are created and deleted.

1.6.1. Constructors and List Initialization

A class can have several types of constructors. Some of them can be generated automatically by the compiler if required, others must be programmed manually, and the existence of a manually defined constructor can prevent the automatic generation of another constructor.

A constructor that can be called without arguments is a so-called *standard constructor*. ***If no other constructor is manually defined, the compiler automatically generates a standard constructor.*** As soon as another constructor is defined manually, a standard constructor is no longer generated automatically by the compiler. Of course, a constructor that does not specify any parameters is in principle referred to as a standard constructor. Nevertheless, a constructor that specifies default values for each of its parameters can also be called as a standard constructor. The constructor of the `RationalNumber` class presented in (U. W. Eisenecker 2022), Section 4.3.3 *Class Design Based on Reference Semantics* is a perfect example. As its prototype shows, the constructor specifies two parameters, each of which has a default value:

```
RationalNumber::RationalNumber(const intmax_t& n = 0,  
                                const intmax_t& d = 1);
```

This allows an exemplar of `RationalNumber` to be created in three different ways:

1. `RationalNumber a { };`
2. `RationalNumber b { 2 };`
3. `RationalNumber c { 3,2 };`

After executing the previous statements, `a` is initialized to 0 for the numerator and 1 for the denominator, which corresponds to a so-called *zero-initialization*. The variable `b` is initialized to 2 for the numerator and 1 for the denominator, which corresponds to the value 2. The variable `c` is initialized to 3 for the numerator and 2 for the denominator. All of the above constructor calls use the so-called *list initialization* (***not to be confused with member initializer list***), which requires the use of curly braces. List initialization prevents so-called *narrowing*, i.e. implicit arithmetic conversions that can lead to a loss of precision. The effect can be easily demonstrated by changing the initialization of `b` to `RationalNumber b { 2.1 };`. Now the compiler should warn that the conversion from `double` to `intmax_t` truncates the decimal part 0.1.

The use of list initialization also supports automatic conversion of all parameters specified in curly braces into a temporary object of a particular type. Suppose there is a function `f(RationalNumber rn)` whose only parameter is `rn` of type `RationalNumber`, which is passed by value.

All of following calls to `f()` use list initialization to create a temporary object of type `RationalNumber`:

1. `f({ });`
2. `f({ 2 });`
3. `f({ 3,2 });`

List initialization is the preferred way of calling constructors in this book and in (U. W. Eisenecker 2022).

To clarify the terms: Object *initialization* means that each data member of an object is initialized so that the object can be used afterwards and does not need any further initialization. Object initialization should be performed by a constructor. A constructor should use a *member initializer list* to explicitly initialize each of its data members. Otherwise, for each data member not in the member initializer list, its default constructor is called. Assigning a value to a data member in the constructor body is not initialization, but assignment. *List initialization* is a special syntax of the constructor call, where the arguments are specified in an *initializer list* with curly braces. An initializer list is also a special generic data type that allows a different number of arguments to be passed to a constructor (see Section *Initializer List*).

1.6.2. Classic Syntax for Calling Constructors

In addition to list initialization, there is the classic syntax for calling constructors, which is demonstrated below:

1. `RationalNumber a;`
2. `RationalNumber b1(2);`
3. `RationalNumber b2 = 3;`
4. `RationalNumber c(3,2);`

The first example calls the constructor of `RationalNumber` as the standard constructor. Since no parameter values are specified, both standard values are used. This corresponds exactly to `RationalNumber a { };`. The second example uses parentheses to pass the first parameter for object initialization. Since the second parameter is omitted, its default value is used. This syntax is similar to a function call. The third example is a variation of the previous example. It is *not* an assignment, but a call to the constructor in combination with an initialization. This syntax does not support the specification of more than one argument. Nevertheless, the `=` character can be used in combination with list initialization to obtain the desired result, i.e. `RationalNumber b2 = { 3,1 };`. Although both statements have the same effect, there is an important semantic difference that will be discussed later.

The fourth example specifies values for both parameters of the constructor and therefore initializes the numerator to 3 and the denominator to 2.

The classic syntax supports only one specific form of calling the function `f()` that automatically creates a temporary object of type `RationalNumber`, namely `f(1)`. Omitting the parameter, i.e. `f()`, or specifying two values, i.e. `f(3, 2)`, is not legal. The call to `f(1)` results in the constructor of `RationalNumber` being called with one parameter value and the second to be substituted with the default value. The other two calls `f()` and `f(3, 3)` are reported as errors because no corresponding overloaded functions exist.

There is another interesting aspect of calling constructors with the classic syntax. As a variant of the first example, it is not allowed to call the constructor with parentheses: `RationalNumber a()`. The compiler recognizes this as a function declaration, which is not expected here. Normally the compiler issues a warning for this.

All code snippets presented before are contained in the program *objects/Constructors.cpp*. It is not reproduced here, but is included in the accompanying material.

1.6.3. Fundamental Datatypes and Constructors

As explained in (U. W. Eisenecker 2022), Section 2.5.2 *Initialization of Variables*, a list initialization with an empty pair of curly braces causes a zero initialization. This means that an exemplar should be initialized to a value that conceptually corresponds to the value zero, for example an empty `std::string` or a `RationalNumber` with 0 as numerator and 1 as denominator.

When declaring a variable of fundamental type, initialization with an empty list has exactly this effect. It initializes the variable with a value that is 0 or has the meaning of zero. For example, `int i { }`; defines `i` and initializes it to 0.

If the list specifies a single value, the variable is initialized with just that value. For example, `int j { 1 }`; defines `j` and initializes it to 1.

The question now is: What about using the classic constructor syntax in these cases?

`int i();` is not allowed. As explained earlier, the compiler considers this to be a function declaration. Since `RationalNumber a;` performs a zero initialization, one might conclude that `int i;` also performs a zero initialization. However, this is not the case. In fact, the value of `i` is undefined after `i` is defined. In practice, outputting the value of `i` after its definition mostly gives 0 as the result. But this can be easily changed. Compiling and executing the code fragment shown in Listing 13 usually outputs values other than 0. The reason for this is that `x`, which is created in the block, is initialized to a value other than 0 and is immediately destroyed when the end of the block is reached. This leaves the memory occupied by `x` in a state that contains some set bits. The local variable `i`, which is created immediately afterwards, uses memory previously occupied by `x`. Since `i` is not initialized, its value is undefined.

Listing 13: Undefined behaviour after declaration of a variable of fundamental type

```

1 {
2     int x { 99 };
3 }
4 int i;
5 cout << i << endl;

```

Fundamental types and structured types are handled differently during creation. While list initialization with an empty list performs a zero initialization, the mere declaration of a variable calls the standard constructor for structured types, while no initialization is performed for fundamental types.

This changes when a variable of fundamental type is declared as a global variable or as a static variable. Global and static variables are always zero initialized by calling their standard constructor, even for fundamental types.

1.6.4. Global and Static Variables

Global and static variables have a lifetime equal to that of the program. A global variable has no block scope, while a static variable usually is defined in a block.

A global variable is defined outside a block in a program, normally at the beginning. Defining it at the end of a program is pointless, since it is accessible only after its declaration. Nevertheless, it is defined and initialized at the beginning of the program, before any statement in `main()`. It is destroyed after the `main()` function returns. The standard input and output streams `std::cin` and `std::cout` are famous examples of global variables.

All of this is demonstrated by the program contained in Listing 14 and Listing 15.

Listing 14: objects/Global.cpp

```

1 // Global.cpp by Ulrich Eisenecker, February 9, 2023
2
11 #include <iostream>
12 using namespace std;
13
14 int global;
15
16 void useGlobal();
17
18 int main()
19 {
20     cout << "Global.cpp - global: " << global << endl;
21     ++global;
22     useGlobal();
23     cout << "Global.cpp - global: " << global << endl;
24 }

```

Listing 15: objects/UseGlobal.cpp

```
1 // UseGlobal.cpp by Ulrich Eisenecker, February 9, 2023
2
11 #include <iostream>
12 using namespace std;
13
14 extern int global;
15
16 class Global
17 {
18     public:
19         Global()
20         {
21             cout << "Global::Global()" << endl;
22         }
23         ~Global()
24         {
25             cout << "global: " << global << endl;
26             cout << "Global::~~Global()" << endl;
27         }
28 } globalObject;
29
30 void useGlobal()
31 {
32     cout << "UseGlobal.cpp - global: " << global << endl;
33     ++global;
34 }
```

Global.cpp defines a global variable of type `int` named `global`. Because it is a global variable, `global` is automatically initialized to zero when it is defined. The *Global.cpp* file contains the `main()` function. The output of `global` in `main()` prints 0 in the console window the first time. After that `global` is incremented. In the next line the function `useGlobal()` is called, which is not defined in *Global.cpp*. In the last line of `main()` the value of `global` is output again. Without knowing the implementation of `useGlobal()`, it is impossible to tell which value will be displayed in the console window. This changes if the call to `useGlobal()` is converted to a comment. Running the resulting program will now output 0 and 1.

After de-commenting `useGlobal()`;, *UseGlobal.cpp* is now examined. It should be emphasized that both source files are compiled separately. So there is no direct access to the global variable named `global`. If it were simply declared as `int global;`, an error would be reported when linking because there are two definitions of `global`. Therefore, `global` must be declared without defining it. This is accomplished with the `extern` declaration. Here, the keyword `extern` causes the following `int global;` to declare only the variable `global`. That is, the name `global` and its type are introduced, but no memory is allocated for `global` and `global` is not initialized. There are two more interesting parts in *UseGlobal.cpp*. The first is the definition of the `useGlobal()` function. This function

outputs the value of `global` and then increments its value. Thus `global` counts each call to `useGlobal()`. The second is the definition of the `Global` class and the declaration of its exemplar `globalObject`. Since `globalObject` is also a global variable, it is initialized before `main()` is executed and destroyed after `main()` is executed. To create a visible trail, the `Global` constructor emits an appropriate message and the destructor does the same. In addition, the destructor also outputs the value of the `global` variable.

Then both parts of the program are compiled and linked to an executable file. With `g++` this can be done as shown in Figure 6. This figure also shows the result of executing `a.out`.

Figure 6: Compile, link, and run `Global.cpp`

```
% g++ -std=c++20 -c UseGlobal.cpp -o glob.o
% g++ -std=c++20 Global.cpp glob.o
% ./a.out
Global::Global()
Global.cpp - global: 0
Global.cpp - global: 1
global: 1
Global::~~Global()
%
```

The creation and initialization of `global` has no visible effect, because it is a fundamental type. `GlobalObject` is created and initialized before `main()` is executed, as shown by the output of its constructor.

Then `main()` outputs the value of `global`, increments it and prints it again. This results in the values 0 and 1. After `main()` returns, the destructor for `globalObject` is called. The `global` variable still exists with the value that was also output with the last statement of `main()`, i.e. 1. Its destructor also outputs a hint that it has been executed.

Global variables of fundamental and user-defined type are created and initialized the same way. Within the same translation unit, this is done in the order of their declaration. Global objects are destroyed in the reverse order of their creation.

A *static variable* is declared with the prefixed keyword `static`. As a global variable it is initialized within its translation unit when the program starts, before a statement in `main()` is executed. Of course, a static variable can also be declared in the global scope, but usually it is defined in a block. The defining block limits its visibility but not its lifetime. Like a global variable, a static variable is destroyed after `main()` completes.

The program shown in Listing 16 demonstrates the use of a static variable named `c`. It is declared in the `count()` function. Unlike a normal variable of fundamental type, it is zero-initialized despite the lack of explicit initialization. Each time `count()` is called, `c` is incremented by one and then its value is returned. This is demonstrated in the `main()` function, which repeatedly calls `count()` and outputs its return value.

Listing 16: objects/Static.cpp

```
1 // Static.cpp by Ulrich Eisenecker, February 10, 2023
2
3 #include <iostream>
4 using namespace std;
5
6 unsigned int count()
7 {
8     static unsigned c;
9     return ++c;
10 }
11
12 int main()
13 {
14     for (auto i { 0u }; i < 10u; ++i)
15     {
16         cout << count() << endl;
17     }
18 }
```

It is refrained from creating a static variable of a user-defined type, especially a class, to demonstrate the timing of its creation and destruction.

As explained in (U. W. Eisenecker 2022), Section 6.5.2 *Logger Class*, a data member of a class (or a struct) can also be declared static. This has exactly the same effect as declaring the static variable `c` in the `count()` function. A static data member of a class is initialized when the program is started, and its lifetime extends throughout the lifetime of the program. It can be accessed from outside the class only if its name is fully qualified. This also requires that the static data member be declared as public. Otherwise, it is accessible only within the class (private and protected) or in derived classes (protected).

1.6.5. Dynamic Objects

Dynamic objects are introduced in (U. W. Eisenecker 2022), Section 3.3.2 *Dynamic Objects* with fundamental types. Everything explained there also applies to the creation and destruction of dynamic objects with user-defined composite types. A dynamic object is created with the `new` operator, which also calls one of its constructors to initialize the object. The pointer to the object returned by the `new` operator must be assigned to an appropriate pointer variable or used to initialize it. If this is not the case, the dynamically created object is lost because it can no longer be accessed. A dynamic object is never deleted automatically. It must always be deleted with the `delete` operator. One or more objects that were dynamically created with the `new[]` operator must be released with the `delete[]` operator. There is a difference to the classical syntax of calling constructors:

the `int()` in `auto ip = new int();` is legal and performs a zero initialization. Unlike `int i();`, it is not recognized by the compiler as an attempt to declare a function named `int()`.

It is strongly recommended not to use dynamically created objects.

Table 2 gives an overview of the variable categories, the scope, the lifetime and the possibilities of initialization. As the last line shows, list initialization applies uniformly to all the different variable categories. Therefore, list initialization is also referred to as *uniform initialization*. ***Whenever possible, it is advisable to use list initialization.***

Table 2: Variable categories, scope, lifetime, and initialization

	Tempo- rary		Local		Global	Static	Dynamic	
	Fundamen- tal Type	User- defined Type	Fundamen- tal Type	User- defined Type			Fundamen- tal Type	User- defined Type
Scope	not appli- cable	not appli- cable	block	block	global	block (nor- mally)	not appli- cable	not appli- cable
Lifetime	expression	expression	block	block	entire pro- gram	entire pro- gram	from cre- ation until deletion	from cre- ation until deletion
Default/Zero Initialization	not appli- cable	not appli- cable	no	yes	yes	yes	no	yes
Classic Constructor Call	yes	yes	yes, but not standard construc- tor	yes	yes	yes	yes	yes
List Initialization	yes	yes	yes	yes	yes	yes	yes	yes

1.6.6. Copy Constructor

When an exemplar is constructed as a copy of an existing exemplar of the same type, its so-called *copy constructor* is called, for example `RationalNumber a { 2,3 }; RationalNumber b { a };`. If there is no user-defined copy constructor, the compiler will automatically generate one. This compiler-generated copy constructor is usually sufficient. If an object has a dynamic resource, such as a file stream or a classical pointer, either a suitable copy constructor must be defined or the copy constructor must be deleted. A class that defines a dynamic resources for its exemplars is introduced in Section *Dynamic Objects*. Defaulting and deleted member functions and also constructors is described in Section *Default and Delete*.

1.6.7. Explicit

Some constructors that perform type conversions can be called automatically. This can lead to undesirable effects. It is therefore possible to suppress the automatic call of these constructors.

Listing 12 introduces the classes OBCC and OBCE. Each class defines a constructor that takes a double parameter as an argument. This has an interesting consequence. Any function that expects an exemplar of OBCC or OBCE passed by value or reference to const also takes a double value, as Listing 17 shows.

Listing 17: objects/DoubleAsOBCE.cpp

```
1 // DoubleAsOBCE.cpp by Ulrich Eisenecker, February 13, 2023
2
3 #include "OBCH.hpp"
4 #include <iostream>
5
6 using namespace std;
7
8 void printInfos(const OBCE& o)
9 {
10     cout << "mileage: " << o.mileage() << endl
11         << "remaining capacity: " << o.capacity() << endl
12         << "remaining range: " << o.remainingRange() << endl;
13 }
14
15 int main()
16 {
17     OBCE anOBCE { 15.0 };
18     anOBCE.charge(45.0);
19     anOBCE.drive(75.0);
20     printInfos(anOBCE); // That's fine!
21     printInfos(99.9); // Not necessarily intended.
22 }
```

The constructor of OBCE is automatically called to create a temporary exemplar when an exemplar of OBCE is expected and a double value is specified. Because of the automatic conversion between fundamental number types, it is also possible to call the printInfos() function with an arbitrary integer value, for example printInfos('x'); or printInfos(true);. The automatic call to the OBCE constructor can be disabled by declaring it with the explicit keyword. Listing 18 shows only the revised constructor definitions.

Listing 18: objects/OBCHE.hpp (excerpt)

```

22 |
23 | class OBCC: public OBC
24 | {
25 |     public:
26 |         explicit OBCC(const double consumption)
27 |             : OBC { },
28 |               m_fuel { 0.0 },
29 |               m_consumption { consumption }
30 |         {}
31 |
32 | class OBCE: public OBC
33 | {
34 |     public:
35 |         explicit OBCE(const double efficiency)
36 |             : OBC { },
37 |               m_capacity { 0.0 },
38 |               m_efficiency { efficiency }
39 |         {}
40 |         double capacity() const

```

Now it is not allowed to pass a number when calling the `printInfos()` function as demonstrated in program *objects/Explicit.cpp* (only available in the accompanying material).

1.6.8. Direct Construction vs. Copy Construction

What has been demonstrated so far is the so-called *direct initialization* when creating an exemplar. This form of initialization directly calls a corresponding constructor. Another possibility is the so-called *copy construction*. Here, a temporary exemplar of the corresponding type is first created first which is then used to call the copy constructor. Listing 19 demonstrates copy initialization for `RationalNumber` exemplars.

Listing 19: objects/CopyInitialization.cpp (excerpt)

```

33 |     RationalNumber a = { };
34 |     RationalNumber b = { 2 };
35 |     RationalNumber c = { 3, 2 };
36 |     cout << a << '\t' << b << '\t' << c << endl;

```

Since copy initialization first creates a temporary exemplar, the constructor called to create the temporary exemplar must not be declared with `explicit`. Only non-explicit user-defined constructors (or more generally: *conversion functions*) can be called implicitly to create the required temporary exemplar.

In contrast, direct initialization considers all constructors and user-defined conversion functions. Please note that user-defined conversion functions are not explained here.

For this reason, it is recommended to use direct initialization in general.

1.6.9. Initializer List

Sometimes it is desirable to initialize an exemplar with a varying number of arguments of the same type. It is assumed that a person has exactly one last name but any number of first names. When creating an exemplar, all of a person's names should be specified, the last name first followed by any number of first names. Listing 20 illustrates how to accomplish this.

Listing 20: objects/InitializerList.cpp

```
1 // InitializerList.cpp by Ulrich Eisenecker, April 22, 2026
2
3 #include <iostream>
4 #include <initializer_list>
5 #include <string>
6 #include <vector>
7
8 using namespace std;
9
10 class Person
11 {
12     public:
13         Person(string surname,initializer_list<string> firstnames = { })
14             : m_surname { surname },
15               m_firstnames { firstnames }
16         {}
17         string fullname() const
18         {
19             string result { };
20             for (const auto& name : m_firstnames)
21             {
22                 result += name + ' ';
23             }
24             result += m_surname;
25             return result;
26         }
27     private:
28         string m_surname;
29         vector<string> m_firstnames;
30 };
31
32 int main()
33 {
34     Person somePerson { "Johnson",{ "Jaden","Asa","Evan" } };
35     cout << somePerson.fullname() << endl;
36     Person anotherPerson { "Miller" };
37     cout << anotherPerson.fullname() << endl;
38 }
```

The `<initializer_list>` header file must be included because it contains the `initializer_list<>` class template, which allows passing a list of arguments of the same type. The `Person` class has two data members, namely `m_surname`, which is of type

`std::string`, and `m_firstnames`, which is of type `std::vector<std::string>`. The latter will hold an arbitrary number of first names in the exact order they were specified when an exemplar of `Person` is created. The `Person` constructor actually takes two arguments, namely a `std::string` exemplar named `surname` and a `std::initializer_list<std::string>` exemplar named `firstnames`, for which an empty list is specified as the default value. The latter allows this constructor to be called even with a last name as the only argument. In the member initializer list `m_surname` is initialized to `surname`, and `m_firstnames` is initialized to `firstnames`. The second initialization is possible because `vector<>` also defines a constructor that takes an `initializer_list<>` as an argument. No further actions of the `Person` constructor are necessary beyond initializing the members, so the constructor body is empty.

`Person::fullname()` returns a `std::string` containing all names of a person. First the first names appear, then comes the last name. Two consecutive names are always separated by a space.

After declaring a local string variable named `result`, a range-based for loop iterates through all the first names. If there is a first name, it is appended to `result` by applying the `+=` operator. It also checks if there are any first names at all. If so, a space is appended to `result`. After the loop ends, the last name is appended to `result` and `result` is returned.

The `main()` function illustrates the creation of `Person` exemplars. The object `somePerson` is created with one last name and three first names. The first names must be enclosed in curly braces to successfully create a temporary `std::initializerList<std::string>` exemplar using list initialization. The `anotherPerson` exemplar is created by specifying only one last name. In this case, the empty list is inserted as the default value for the second parameter. If the default value for the second parameter were not specified in the constructor declaration, an empty list would always have to be specified if no first names are present, e.g. `Person anotherPerson { "Miller", { } };`. Of course, it is also legal to call the actual constructor specifying the empty list as the default value, as here.

One important aspect must be emphasized. The second argument of the `Person` constructor could be specified as `vector<string> firstnames = { }`, with exactly the same effect and without further changes in the program. The advantage of using an `initializer_list<>` becomes clear if the type of the data element `Person::m_firstnames` is changed to a different container type. If `vector<string>` was specified as the type of the second constructor argument, it would have to be consistently changed to the other container type. Otherwise the code would become invalid. This consequence is avoided if `initializer_list<>` is specified as the type of the second constructor argument, as the container templates of the *STL* provide a constructor that allows initialization with an `initializer_list<>`.

1.6.10. Delegating Constructor

Sometimes a class defines multiple constructors to meet different initialization requirements. These constructors may have commonality and possibly a single point of modification. In this case, a class may define multiple constructors that contain unwanted redundancy, and perhaps another method that is called in the body of each constructor. An alternative to this code duplication in constructors is the use of a delegating constructor. Both cases are demonstrated below.

Listing 21 shows an excerpt from the *objects/RN.hpp* header file. It defines a rudimentary `RationalNumber` class with two constructors. The first one is known, but the second is new. It takes two string numbers, i.e. strings with digits, to create a `RationalNumber` exemplar. Since this constructor is intended to be used only on purpose, it is declared with `explicit` to prevent any implicit use. For example, one possible use of this constructor is when values have been entered as strings in a *graphical user interface* (*GUI* for short). This constructor would then allow these strings, which actually contain numbers, to be processed directly.

Listing 21: *objects/RN.hpp* (excerpt)

```
24 |  
25 | class RationalNumber  
26 | {  
27 |     public:  
28 |         RationalNumber(const std::intmax_t& n = 0, const std::intmax_t& d = 1)  
29 |             : m_numerator { n }, m_denominator { d }  
30 |         {  
31 |             normalize();  
32 |         }  
33 |     explicit RationalNumber(const std::string& n, const std::string& d)  
34 |         : m_numerator { std::atoll(n.c_str()) },  
35 |           m_denominator { std::atoll(d.c_str()) }  
36 |         {  
37 |             normalize();  
38 |         }  
58 | };
```

The constructor, taking two string exemplars as arguments, is very similar to the first one. The only difference is that it retrieves the *C-strings* from both `std::string` arguments and calls the *C function* `atoll()` to convert them to long long unsigned int values. To use the `atoll()` function, the `<cstdlib>` header must be included.

Assuming a check that ensures `m_denominator` is non-zero would be added later, this had to be done consistently in both constructors. Besides this maintenance problem, reading redundant code costs more time than necessary.

A *delegating constructor* can be used to overcome this deficiency. Listing 22 shows only an excerpt from *objects/RNI.hpp*, which contains the delegating constructor that takes two strings as parameters that calls the other constructor.

Listing 22: objects/RNI.hpp (excerpt)

```

32     explicit RationalNumber(const std::string& n,const std::string& d)
33         : RationalNumber { std::atoll(n.c_str()),std::atoll(d.c_str()) }
34     {
35     }

```

The first constructor is called in the initializer list and receives the two integers that result from the conversion of the string parameters. This also executes the body of the first constructor. Thus, no duplicate call to `normalize()` is required in the body of the second constructor. If a method to check for zero denominators were added later, it would only need to be called in the first constructor. The second constructor would not need to be touched again.

Of course, a delegating constructor can have the same arguments as another constructor plus additional arguments. In this case, it can call the other constructor with the same arguments and perform member initialization with the additional arguments. It can also execute additional code in its own constructor body.

Before implementing a class with delegating constructors, it should be considered that additional constructors cannot introduce new or different data members. That is, each constructor must initialize each data member. Thus, no constructor can do more than any other constructor. In addition, a solution with only one or a minimal number of constructors that use default values for arguments to achieve the same result is preferable over delegating constructors.

The *objects/RN_UnitTests.cpp* file, which is not reproduced here but is contained in the accompanying material, includes *RN.hpp* and performs a minimal set of unit tests using *Catch2* v3. For testing *RNI.hpp*, the simplest possibility is to include it in *RN_UnitTests.cpp* instead of *RN.hpp*, and compile and re-run the test.

1.6.11. Virtual Destructor

It has already been mentioned that a class that can be used as a base class must define a virtual destructor. The reasons for this are explained in detail below.

A destructor performs the necessary cleanup when an object is destroyed. Normally, nothing special has to be done in this case. Therefore, the automatically generated destructor is sufficient.

In the *Logger* class (see (U. W. Eisenecker 2022), Section 6.5.2 *Logger Class*) a destructor is defined which produces an output when the exemplar is destroyed. This is a common use case for the manual definition of a destructor. A related example might be the manual implementation of destructors to produce a side-effect that informs about their execution in an inheritance hierarchy.

Another use case that requires the explicit definition of a destructor is an object that acquires a dynamic resource such as heap memory, a file, or a thread. This dynamic resource must be released again, which is the responsibility of the destructor. Objects with dynamic resources are presented in Section *Dynamic Objects*.

As already mentioned, the destructor of a base class must be declared as `virtual`. The consequences of not doing so are illustrated by the *Destructor.cpp* program shown in Listing 23.

Listing 23: objects/Destructor.cpp

```
1 // Destructor.cpp by Ulrich Eisenecker, February 17, 2023
2
3 #include <iostream>
4 #include <array>
5 using namespace std;
6
7 class Base
8 {
9     public:
10         Base()
11         {
12             cout << "Base::Base()" << endl;
13         }
14         ~Base()
15         {
16             cout << "Base::~Base()" << endl;
17         }
18     private:
19         unsigned char m_base_data;
20 };
21
22 class Derived: public Base
23 {
24     public:
25         Derived()
26         {
27             cout << "Derived::Derived()" << endl;
28         }
29         ~Derived()
30         {
31             cout << "Derived::~Derived()" << endl;
32         }
33     private:
34         array<unsigned char,1024 * 1024> m_derived_data;
35 };
36
37 int main()
38 {
39     cout << "Assigning new Base to Base * and delete it afterwards."
40         << endl;
41     Base * basePointer { new Base };
42     delete basePointer;
```

```

43 |
44 |     cout << "\nAssigning new Derived to Derived * and delete it afterwards."
45 |         << endl;
46 |     Derived * derivedPointer { new Derived };
47 |     delete derivedPointer;
48 |
49 |     cout << "\nAssigning new Derived to Base * and delete it afterwards."
50 |         << endl;
51 |     Base * actuallyDerivedPointer { new Derived };
52 |     delete actuallyDerivedPointer; // Ouch!
53 | }

```

The `<array>` header file defines the `std::array<>` container, which is an adaptor for *C-arrays*. It is used here to define a data member of one mebibyte size.

The Base class defines a standard constructor and a destructor. Both of them simply output that they are executed. Base has a data member that occupies only a single byte of memory.

Derived is a public subclass of Base. It also defines a constructor and a destructor that output messages to inform about their execution. Derived defines the data member `m_derived_data`, which is of type `std::array<unsigned char, 1024 * 1024>`. This means, that `m_derived_data` stores 1,048,576 unsigned characters, each of one byte size.

The `main()` function first defines a pointer to Base called `basePointer` and initializes it with a pointer to a dynamically allocated Base exemplar. This pointer is deleted immediately afterwards. As the output of the corresponding statements shows (Figure 7), `new Base` causes the execution of the Base constructor, and `delete basePointer;` causes the execution of the Base destructor.

Figure 7: Output of `objects/Destructor.cpp`

```

Assigning new Base to Base * and delete it afterwards.
Base::Base()
Base::~Base()

Assigning new Derived to Derived * and delete it afterwards.
Base::Base()
Derived::Derived()
Derived::~Derived()
Base::~Base()

Assigning new Derived to Base * and delete it afterwards.
Base::Base()
Derived::Derived()
Base::~Base()

```

In the second group of statements, an exemplar of `Derived` is dynamically created and assigned to a pointer to `Derived` named `derivedPointer`. Afterwards, this exemplar is released by calling `delete derivedPointer;`. As the program output shows, when the object is created, the `Base` constructor is executed first, followed by the `Derived` constructor. **Constructors are always executed in strict order, from the top-level base class to the class of which an exemplar is actually created.** This makes sense because the constructor in a child class can always rely on the fact that the constructor of the immediate parent class has already done its job. When `derivedPointer` is deleted, the destructor of the `Derived` class is executed first, followed by the destructor of the `Base` class. **Destructors are always called in the reverse order of constructors.** As a result, the most specific cleanup is performed first by the destructor of the most derived class, which is then followed by the cleanup of the destructor of the immediate base class. Here, the execution of the `Derived` destructor is particularly important, since this returns one mebibyte of memory to the free store.

In the last group of statements, the address of a dynamically created `Derived` object is used to initialize a pointer to `Base`, called `actuallyDerivedPointer`. This is legal because `Derived` is a public subclass of `Base`, which means that whenever an exemplar of `Base` is expected, an exemplar of `Derived` can be used. Everything about object creation is fine. First the `Base` constructor is executed, then the `Derived` constructor. The mishap occurs when deleting `actuallyDerivedPointer`. Since the static type of this pointer is `Base`, the compiler generates code only for calling the `Base` destructor. The compiler does not know that `actuallyDerivedPointer` indeed points to a `Derived` exemplar. As a result, only the `Base` part of the object is destroyed and only one byte of memory, occupied by `m_base_data` is returned to the free store. The one mebibyte of `m_derived_data` is now orphaned and cannot be reclaimed for the free store. The resulting program has a fatal error, and worse, this error cannot be reliably detected.

Fortunately, the problem can be easily fixed. The solution is to declare the `Base` destructor as `virtual`, that is `virtual ~Base()`. As explained in Section *Late Binding*, this has the effect that all subclass destructors are also `virtual`, even if they are automatically generated by the compiler. The application of this correction and the recompilation and execution of the program is left to the reader as an exercise.

Whenever a class CAN serve as a base class, it SHOULD HAVE a virtual destructor. If the destructor does nothing special, it can be declared either with `virtual ~SomeClass() { }` or with `virtual ~SomeClass() = default;`. Defaulting a method will be explained in the Section *Default and Delete*.

If a class may not be used as a base class, it should be declared `final`, as explained in the Section *final*.

1.6.12. Order of Calling Constructors and Destructors

The order in which constructors and destructors are called for classes and their descendants as well as for data members of classes is precisely defined. This is explained in detail below. ***Attention should be paid not to write code that depends on the order in which constructors or destructors are called.***

The output of the *objects/Destructor.cpp* program (Listing 23) suggests that the constructor of a base class is executed before the constructor of its subclass, while the destructors are called in the reverse order. It is also interesting to note the order in which the data members are constructed and deleted. The program shown in Listing 24 explains this and again illustrates the order of calling constructors and destructors between related classes.

Listing 24: objects/Order.cpp

```

1 // Order.cpp by Ulrich Eisenecker, February 17, 2023
2
3 #include <iostream>
4 #include <string>
5 using namespace std;
6
7 class Class {
8
9     public:
10         Class(const string& description):m_description { description }
11         {
12             cout << "Class::Class(" << m_description << ')' << endl;
13         }
14         virtual ~Class()
15         {
16             cout << "Class::~~Class() " << m_description << endl;
17         }
18     private:
19         string m_description;
20 };
21
22 class Derived: public Class
23 {
24     public:
25         Derived(const string& description): Class { description }
26         {
27             cout << "Derived::Derived()" << endl;
28         }
29         ~Derived()
30         {
31             cout << "Derived::~~Derived()" << endl;
32         }
33     private:
34         Class m_1 { "This is Derived::m_1" },
35         m_2 { "This is Derived::m_2" };
36 };
37

```



```
38 | int main()  
39 | {  
40 |     Derived d { "This is d" };  
41 | }
```

The `Class` class defines a constructor that takes a reference to `const` of type `string`. This is used to initialize `Class::m_description`, which describes the object when its constructor or its destructors are executed. Their purpose is to print a message informing that they are called. Note that `Class::~Class()` is declared as `virtual`, because `Class` can (and will) serve as a base class. The `Derived` class is a public subclass of `Class`. It defines a constructor that also takes a `string` describing the object that calls the base class constructor, passing the description as an argument. The constructor and destructor of `Derived` are also defined in such a way that they generate a corresponding output when called. `Derived` defines two data members, namely `m_1` and `m_2`, both of type `Class`. They are initialized at the point of their declaration.

The only purpose of `main()` is to define a `Derived` exemplar named `d`. Figure 8 shows the output when the program is executed.

Figure 8: Output of `objects/Order.cpp`

```
Class:Class(This is d)  
Class:Class(This is Derived::m_1)  
Class:Class(This is Derived::m_2)  
Derived::Derived()  
Derived::~Derived()  
Class::~Class() This is Derived::m_2  
Class::~Class() This is Derived::m_1  
Class::~Class() This is d
```

The body of the base class constructor is executed first. This is followed by the constructors of the two `Derived` members `m_1` and `m_2`, in exactly that order. After that, the body of the `Derived` constructor is executed. This suggests that the constructors of the base class are executed first, then the constructors of the members, and finally the constructor body of the object to be constructed.

The order of calling the member constructors depends on the order of their declaration. If `m_2` is declared before `m_1` in `Derived`, the constructor of `m_2` is called before that of `m_1`. Another assumption is that the order depends on the order of initialization in the initializer list. However, changing the initializer list to `Derived(const string& description): Class { description }, m_1 { "This is Derived::m_1" }, m_2 { "This is Derived::m_2" } // ...` does not affect the execution order of the constructors.

As mentioned earlier, destructors are executed in exactly the reverse order of constructors.

Finally, the recommendation from the beginning of the section is repeated: ***Any dependencies between the construction and destruction of data members should be strictly avoided.***

1.7. The Virtual Function Table

The C++ standard does not specify how virtual function calls are to be implemented. The most common and most widely used mechanism for their implementation is explained below.

A *function pointer* allows *dynamic binding*, also known as *late binding*, of free functions. A *virtual function table* uses function pointers to implement late binding of member functions.

The C++ standard does not define how virtual function calls are implemented. Usually, they are implemented with the help of a so-called *virtual function table* (*vtable* or *vtbl* for short). Before virtual function calls and their possible implementation are discussed, function pointers are introduced. They are the basis of virtual function calls.

The `ascending()` function included in Listing 25 returns `true` if the first of its string arguments is greater than the second with respect to a particular sorting order. When executed as `ascending("B", "A")`, it returns `false` because "B" and "A" are not in ascending order.

Listing 25: objects/Sorting.cpp

```

1 // Sorting.cpp by Ulrich Eisenecker, April 27, 2026
2
3 #include <algorithm> // Because of sort<>()
4 #include <cstdlib> // Because of atoll()
5 #include <iostream>
6 #include <string>
7 #include <vector>
8
9 using namespace std;
10
11 bool ascending(const string& a, const string& b)
12 {
13     return (a < b);
14 }
15
16 bool descending(const string& a, const string& b)
17 {
18     return (a > b);
19 }
20
21 bool numericallyAscending(const string& a, const string& b)
22 {
23     return (atoll(a.c_str()) < atoll(b.c_str()));
24 }
```

```
25
26 void sortAndPrint(vector<string> v,bool(*fp)(const string&,const string&))
27 {
28     sort(v.begin(),v.end(),fp);
29     for (const auto& e : v)
30     {
31         cout << e << ' ';
32     }
33     cout << endl;
34 }
35
36 int main()
37 {
38     cout << boolalpha
39         << ascending("B","A") << endl;
40     vector someStrings { "Dog"s,"Cat"s,"Mouse"s,"Zebra"s, "Alligator"s };
41
42     sort(someStrings.begin(),someStrings.end(),ascending);
43     for (const auto& s : someStrings)
44     {
45         cout << s << ' ';
46     }
47     cout << endl;
48
49     sortAndPrint(someStrings,descending);
50     sortAndPrint( { "2"s,"-9"s,"1000"s,"-17"s },numericallyAscending);
51
52     using functionPointerType = bool (*)(const string&,const string&);
53     functionPointerType f { numericallyAscending };
54     sortAndPrint( { "3"s,"-2"s,"5"s },f);
55
56     f = ascending;
57     cout << f("a","b") << endl;
58 }
```

The `ascending()` function can also be passed to the `std::sort<>()` algorithm of the STL. This allows the elements of a `std::vector<>` to be sorted in alphabetically ascending order, as demonstrated in line 42. What is passed to the `sort<>()` algorithm is only the name of this function—without the following pair of parentheses. But what does `ascending` mean without parentheses? It is a pointer pointing to the function `ascending()`. Only when this pointer is followed by a pair of parentheses the function is actually called. This use of a pair of parentheses is called a *function call operator*.

One detail needs to be highlighted. In line 40, type deduction is combined with list initialization to declare the variable `someStrings`. Therefore, the elements in the initialization list are explicitly written as `std::string` literals. Otherwise, the result would be to declare `someStrings` as `vector<const char*>`, which is certainly not intended here. This would also lead to compilation errors, since there is no overloaded `ascending()` function for `const char*`. The string elements in lines 54 and 57 are also written as `std::string` literals.

The `std::sort<>()` function is a so-called *mutating algorithm*, i.e. it can change the order of the elements in the container to be sorted. Therefore, the container to be sorted cannot be specified as `const`, and it cannot be a temporary variable.

The `sortAndPrint()` function demonstrates how to specify a function pointer and its type. The second argument of this function is `bool(*fp)(const string&, const string&)`. `fp` is the name of the pointer variable. The asterisk in front of `fp` declares `fp` as a pointer. The asterisk and the pointer name must be enclosed in parentheses. Everything to the left and right of the parentheses specifies the type of the function pointer. The left part specifies the return type, here `bool`. The right part specifies the argument list of the function. It is also enclosed in parentheses and contains two parameters, which are references to `const` of type `string` here. The parameter names are not given because they are not required. If they are considered to be useful for understanding the declaration of the function pointer, they can be included. They are ignored by the compiler.

The first argument of the `sortAndPrint()` function is a `std::vector<std::string>` that is passed by value. This is indispensable for sorting it, since `std::sort<>()` is a mutating algorithm.

Lines 49 and 50 demonstrate how to call the `sortAndPrint()` function. Since the first parameter is passed by value, the function can also be called with a temporary object of type `std::vector<std::string>` by providing an initialization list with `std::string` literals.

The `descending()` and `numericallyAscending()` functions are additional examples of comparison functions. The `descending()` function simply causes an alphabetically descending order when passed to `sort<>()`. `numericallyAscending()` assumes that its arguments are strings of numbers and converts them to integer values before comparison. Since the arguments are of type `std::string`, the member function `string::c_str()` must be used to obtain a corresponding C-string, which is required by the `std::atoll()` function.

If a pointer to a comparison function had to be declared repeatedly, this would be cumbersome and error-prone. Writing declarations repeatedly, as in the `printAndSort()` function, lowers the code quality. Line 54 shows how to declare an alias for this function pointer type. This is done with the `using` declaration which specifies the alias before the equal sign. To the right, the function pointer type is specified. Since no function pointer variable is declared, the first pair of parentheses contains only the asterisk. To the left of this, the return value of this function pointer type is specified, which here is `bool`. The parameter list is shown on the right-hand side. Afterwards, the function pointer `f` is declared to be of type `functionPointerType` and initialized to `numericallyAscending`. In the next line, `f` is passed as pointer to the comparison function as the second parameter of `sortAndPrint()`.

Because `f` is a variable, it is perfectly legal to assign it another compatible function pointer, e.g. `f = ascending`; as done in line 56. Then `f` is called as a function with `"a"` and `"b"` as arguments.

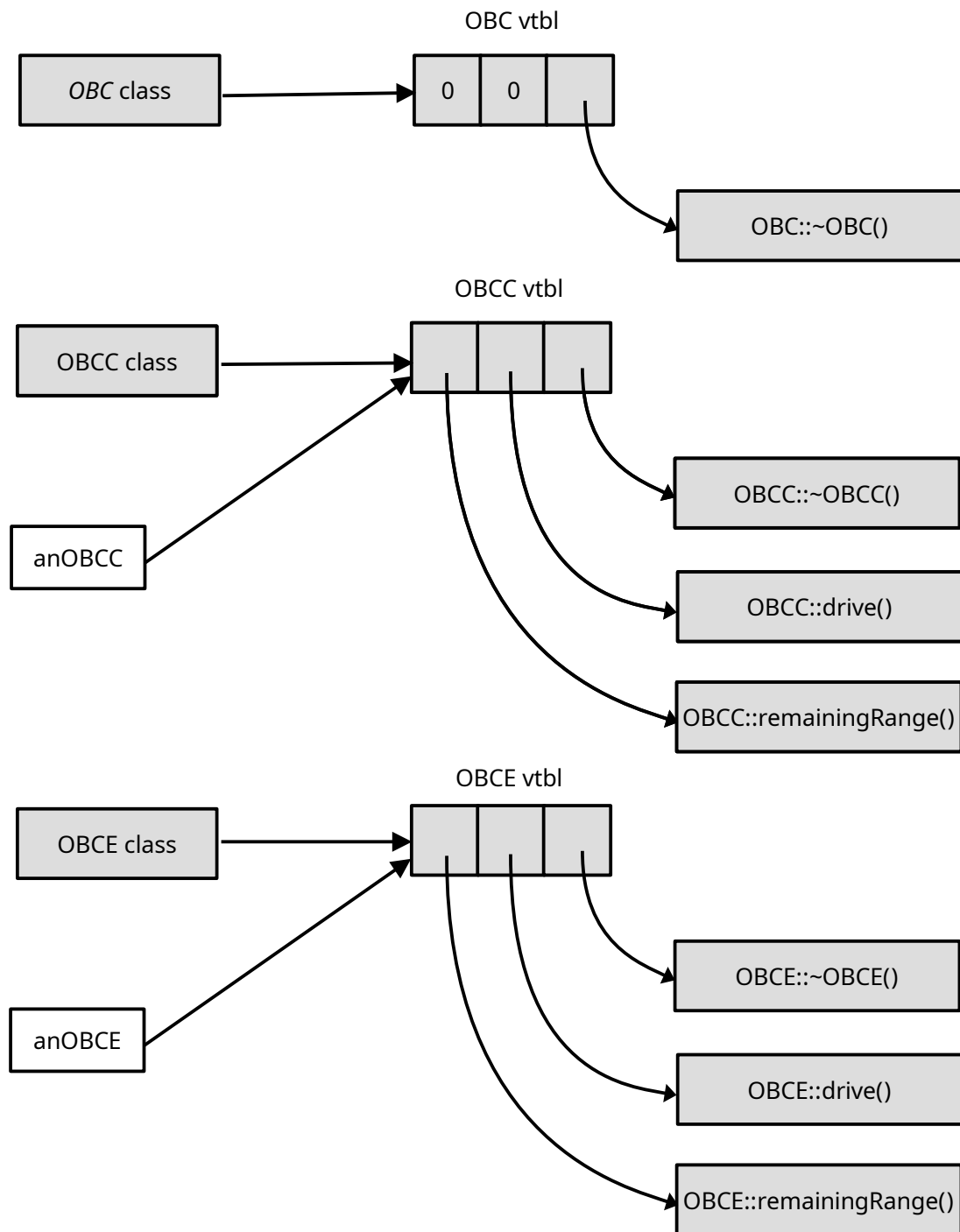
A function pointer type allows declaring function pointer variables that can point to any compatible function—at runtime. This corresponds to dynamic binding or late binding of virtual member functions—only without receiver object.

A virtual function table can be imagined as an array with function pointers. This array is associated with a class. Each exemplar of this class contains a pointer to the vtbl of its class. Figure 9 illustrates this using `objects/OBCHE.hpp` (Listing 18).

To call a virtual member function, the compiler generates code that uses an object's pointer to its vtbl to execute the function pointed to by the corresponding element of the vtbl. This is an indirection that incurs a small runtime loss. In many applications, especially interactive applications, this is of no significance. Also, each object must store a pointer to the vtbl, which increases the object's memory footprint. Considering the large amount of memory available in modern computers, this is also not significant. The situation is different on platforms with low CPU power and severe memory restrictions, such as embedded devices or IOT devices.

The vtbl pointer contained in an object is the basis for dynamic type casts and runtime type identification, which are presented below.

The structure of a vtbl and the location where the pointer to a vtbl is stored in an object is implementation-defined and not standardized. ***Never write code that directly accesses or manipulates a vtbl or the pointer to it. The resulting program would not be portable and would have potentially disastrous effects.***

Figure 9: Classes, virtual function tables, and objects

1.8. final

In C++, it is also possible to completely prohibit the use of a specific class as a base class. Additionally, it can be specified that a member function may not be overridden in a subclass.

If a class does not have a virtual destructor, it is not suitable as a base class. Static meta programming makes it possible to check for the presence of a virtual destructor. The alternative is to prohibit inheritance from a class altogether by declaring a class `final`. Another use of `final` is to forbid overriding of virtual functions in subclasses. For demonstration purposes, an abstract program and variations of the `RationalNumber` class are used. The latter example also introduces a class hierarchy design with classes representing specific values.

The program in Listing 26 first demonstrates the case where a class that is not declared as `final` is used as a base class. Instead, the presence of a virtual destructor is checked at compile time.

Listing 26: `objects/FinalClass.cpp`

```
1 // FinalClass.cpp by Ulrich Eisenecker, February 20, 2023
2 // Warning: This program will not compile on purpose!
3
4 #include <type_traits>
5
6 class DontUseAsBaseClass
7 {
17     // public: virtual ~DontUseAsBaseClass() {};
18 };
19
20 class ChildWithBadParent: public DontUseAsBaseClass
21 {
32     static_assert(std::has_virtual_destructor_v<DontUseAsBaseClass>,
33                 "Base class has no virtual destructor!");
34 };
35
36 class NotABaseClass final
37 {
38 };
39
40 class ImpossibleChild: public NotABaseClass
41 {
42 };
43
44 int main()
45 {
46 }
```

DontUseAsBaseClass is an empty class. Therefore, the compiler automatically generates a standard constructor and a default destructor, the latter being non-virtual. If line 17 is decommented, the class gets a virtual destructor and thus can be used as a valid base class.

ChildWithBadParent inherits from DontUseAsBaseClass as a public base class. If the last two lines of its definition are converted to a comment, the class definition is compiled without error. Normally, this error would not be detected. With static meta-programming it is possible to check for this error, as shown in lines 32f. The `static_assert` declaration defines an assertion that must be satisfied at compile time, i.e. it is checked by the compiler. The first argument of `static_assert` is an assertion that must evaluate to a `bool` value. If it evaluates to `true`, the `static_assert` statement has *no* effect. The second argument is optional. If present, it must be a string literal. If the assertion evaluates to `false`, the compiler reports an error and includes the string literal if present. In the condition, the type trait `has_virtual_destructor<>` from the meta-programming library is used to check for the presence of a virtual destructor in `DontUseAsBaseClass`. Here, instead of `std::has_virtual_destructor<DontUseAsBaseClass>::value`, the short form `std::has_virtual_destructor_v<DontUseAsBaseClass>` is used. For using type traits, the `<type_traits>` header must be included (see (*Containers Library - Cppreference.Com* n.d.); (U. W. Eisenecker 2022), Section *Information About Types*Information About Types).

Because of this `static_assert` statement the compiler issues a corresponding error message. If lines 32f. are converted to a comment, this error is suppressed.

Next, the alternative is examined. If it is determined that a class may not be used as a base class, it is advisable to declare it as a `final` class. This prevents the programmer from using that class as a base class.

Unlike `DontUseAsBaseClass`, the `NotABaseClass` class is declared `final` by placing the `final` specifier after the class name. Attempting to derive `ImpossibleChild` from `NotABaseClass` will consequently be reported as an error when the program is compiled.

One might ask, what is the advantage of the `final` specification over explicitly checking for the presence of a virtual destructor in the base class? The answer is both manifold and simple. The most important advantage is that the `final` specification is intentional. It clearly states what is and avoids any ambiguity. The second advantage is that is in the right place, namely in the base class. There it only has to be written once and is checked by any number of attempts to derive a subclass of the corresponding class. And last but not least, it is much shorter.

The second use of `final` is to declare a virtual member function as `final`. This means that a subclass may no longer override this member function. Listing 27 shows this use of `final`. The underlying idea is to design a hierarchy of classes that represent rational numbers with certain values, e.g. the values $0/1$, $1/1$, $n/1$ and n/d . In the context of rational numbers, this goal seems purely academic. However, the situation is different

when other data types are relevant, e.g. certain matrix types that enable optimized implementations of algorithms and at the same time require less memory for some matrix types.

Listing 27: objects/FinalMethod.cpp

```
1 // FinalMethod.cpp by Ulrich Eisenecker, February 20, 2023
2
3
4 #include <iostream>
5 #include <stdint> // Because of maxint_t
6
7
8 class AbstractRationalNumber
9 {
10 public:
11     virtual std::intmax_t numerator() const = 0;
12     virtual std::intmax_t denominator() const = 0;
13     virtual ~AbstractRationalNumber()
14     {};
15 };
16
17 class RationalNumber1: public AbstractRationalNumber
18 {
19 public:
20     std::intmax_t numerator() const override
21     {
22         return 111;
23     }
24     std::intmax_t denominator() const final
25     {
26         return 111;
27     }
28 };
29
30 class RationalNumber0: public RationalNumber1
31 {
32 public:
33     std::intmax_t numerator() const final
34     {
35         return 011;
36     }
37 };
38
39 class RationalNumberI: public RationalNumber0
40 {
41 public:
42     RationalNumberI(const std::intmax_t numerator):m_numerator { numerator }
43     {}
44     std::intmax_t numerator() const final
45     {
46         return m_numerator;
47     }
48 private:
49     std::intmax_t m_numerator;
50 };
51
52
53
```

```

54 |
55 | void test(const AbstractRationalNumber& n)
56 | {
57 |     std::cout << '(' << n.numerator()
58 |               << '/' << n.denominator()
59 |               << ')' << std::endl;
60 |
61 | }
62 |
63 | int main()
64 | {
65 |     RationalNumber1 one;
66 |     test(one);
67 |     RationalNumber0 zero;
68 |     test(zero);
69 |     RationalNumberI answer { 42 };
70 |     test(answer);
71 | }

```

This program again uses rational numbers as an example. Since there is no general import of all identifiers of the namespace `std` this time, `std::` is prefixed to all identifiers of the namespace `std`. If this were a header file, this would even be mandatory, so that programs that include the header file are not forced to import all identifiers of the namespace `std`.

The `AbstractRationalNumber` class has only pure virtual member functions. Therefore, it acts as an interface for all of its subclasses, which must provide implementations for the pure virtual member functions. All arithmetic member functions, such as addition or multiplication, have been omitted for sake of brevity. The `RationalNumber1` class is publicly derived from `AbstractRationalNumber`. Its exemplars exclusively represent the rational number with numerator 1 and denominator 1. Therefore, the overrides of the two corresponding member functions each return 111 as the result. `RationalNumber1::numerator()` is specified with `override`. Thus, the compiler checks whether the base class has a corresponding virtual function, which is the case. The member function `RationalNumber1::denominator()` is specified as `final`. This also requires that the base class has a corresponding virtual function, but prevents this virtual function from being overridden in any subclass of `RationalNumber1`. Next, there is the `RationalNumber0` class, which publicly inherits from `RationalNumber1`. This class represents the rational number with numerator 0 and denominator 1. Accordingly, it overrides the member function `numerator()`. Since it is not intended to return a non-zero numerator in a subclass this member function is specified as `final`.

Next, the `RationalNumberI` class is defined. It represents all rational numbers with denominator 1. That is, only the numerator must be stored in an exemplar. This is done with the private data member `m_numerator`. Since the member function `numerator()` must return the value of `m_numerator`, it must be overridden. This is exactly the point that is reported as an error by the compiler. The `numerator()` member function is already specified as `final` in `RationalNumber0`. Therefore, it can no longer be overridden.

Actually, the base class of `RationalNumberI` is incorrectly chosen. Obviously, the intention of this class hierarchy design is that each class only defines those data members whose values can vary. There are three cases that contain non-variable data members, namely $(1/1)$, $(0/1)$, and $(x/1)$, where x is any integer. The commonality of all three cases is a denominator of 1. For this reason, `RationalNumber1::denominator()` returns 1 and is specified as `final`, meaning that this member function cannot be overridden in any subclass. `RationalNumber0` always returns 0 as the numerator. Therefore, `RationalNumber0::numerator()` returns 0 and is specified as `final`. `RationalNumberI` allows the numerator to vary while the denominator is always 1. For this reason, a valid base class for deriving `RationalNumberI` is `RationalNumber1`, because `RationalNumber1` allows the `numerator()` member function to be overridden, which is not the case for `RationalNumber0`. Once `RationalNumber0` is changed to `RationalNumber1` as base class of `RationalNumberI`, the program will compile and execute correctly.

This class hierarchy is an example of designing classes to represent specific values. Apart from the memory required for the `vtbl`, `RationalNumber0` and `RationalNumber1` are classes whose exemplars require significantly less memory than other classes in this hierarchy, e.g. `RationalNumber`, which is not shown here. But there is not only an advantage in terms of memory. When adding an arbitrary rational number to a `RationalNumber0` exemplar, the corresponding `add()` method is very simple and extremely efficient. It simply returns the receiver object, since a `RationalNumber0` exemplar is the neutral element for addition. Similarly, when calculating the product of any rational number and an exemplar of `RationalNumber1`, the exemplar of `RationalNumber1` is returned as the result.

As mentioned above, the presence of classes representing specific values of rational numbers does not pay off in terms of memory consumption and execution speed. In high-performance computing, significant performance gains can be achieved, for example, if there are special types for unit matrices and sparse matrices.

The attentive reader may have noticed that the problem introduced earlier could have been at least partially avoided if `RationalNumber0` and `RationalNumberI` had been specified as `final` classes. This would even lead to a more specific error message when mistakenly trying to derive `RationalNumberI` as a subclass of `RationalNumber0`. This would also make it unnecessary to specify the methods of `RationalNumber0` and `RationalNumberI` as `final`.

The final specifier is not a keyword. Therefore, it is legal to define a class named final as final. With the goal in mind of writing programs that are easy to read and maintain, the use of final in any way other than as a final specifier should be strictly avoided.

1.9. Static Members

Unlike classic data members, which are bound to exemplars of a class, static members are bound to the class that defines them. Static members exist independently of exemplars of a class. They are introduced by the example of associating unique numbers, or *IDs* for short, to objects. These object IDs are the basis for serializing objects in streams.

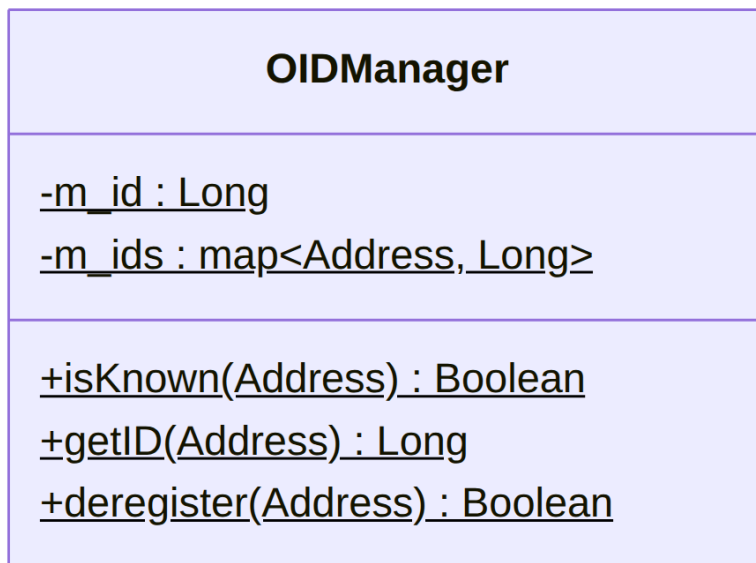
In addition to attributes and methods that are specific to exemplars, the object-oriented programming paradigm also provides attributes and methods that are specific to a class. They can also be understood as attributes and methods that are shared by all exemplars of a class without being bound to a specific object. So-called *class attributes* are mapped to *static data members* in C++, and so-called *class methods* are mapped to *static member functions* in C++. A static class member is declared with the keyword `static`.

There are many use cases for static members. The following example introduces a class that creates and manages identifiers for objects. This class is called `OIDManager` and has the following tasks:

1. It allows to query whether an object is already known to the `OIDManager`.
2. It allows to retrieve the identifier of an object; if the object is not known, a new identifier is created and returned.
3. An object can be deregistered, which means that the object is subsequently destroyed; consequently, the identifier for this object must be deleted.

Apparently, there must be only one exemplar of `OIDManager` per program to ensure consistent management of IDs. Interestingly, a design pattern called *Singleton* was described some decades ago (see (Gamma et al. 1995) for information) to enforce the creation of exactly one exemplar per class. Since then, the use of the *Singleton* design pattern has become obsolete for several reasons. An alternative solution with only static members is presented below. The `OIDManager` class is shown as a *UML* class diagram in Figure 10. According to the *UML* specification, static members are underlined.

The definition of this class, including its full implementation, is shown in Listing 28.

Figure 10: ObjectManager class—UML class diagram**Listing 28:** objects/oid_manager.hpp

```
1 // oid_manager.hpp by Ulrich Eisenecker, March 28, 2023
2
3 #ifndef OID_MANAGER_HPP
4 #define OID_MANAGER_HPP
5
6 #include <map>
7
8 namespace oid_manager
9 {
10
11     class Object
12     {
13     public:
14         virtual ~Object()
15         {}
16     };
17
18     class OIDManager
19     {
20     public:
21         [[nodiscard]] static bool isKnown(const Object& o)
22         {
23             return m_ids.find(&o) != m_ids.end();
24         }
25         [[nodiscard]] static std::size_t getId(const Object& o)
26         {
```

```

27         if (!isKnown(o))
28         {
29             m_ids[&o] = ++m_id;
30         }
31         return m_ids[&o];
32     }
33     static bool deregister(const Object& o)
34     {
35         if (isKnown(o))
36         {
37             m_ids.erase(m_ids.find(&o));
38             return true;
39         }
40         return false;
41     }
42 private:
43     inline static std::size_t m_id { 0 };
44     static inline std::map<const Object*,std::size_t> m_ids;
45 };
46 }
47 #endif // OID_MANAGER_HPP

```

It is designed as a header file and all its entities are defined in the `oid_manager` namespace. Since it uses only object-oriented programming means, the `Object` class serves as the base class for all classes whose exemplars can be managed by `OIDManager`. As a base class, it defines a virtual destructor as its only public member.

The `OIDManager` class has two static data members, namely `m_id` and `m_ids`. `m_id` is a counter of type `std::size_t` that is initialized to 0. To initialize a static data member in class, its declaration must contain `inline`. `inline` can either precede the static keyword (line 43 in Listing 28) or follow it (line 44 in Listing 28). `m_ids` is a `std::map<>` container template that is instantiated with a pointer to a `const Object` as its key and an integer of type `std::size_t` as value. A `map<>` contains unique keys plus associated values. Here, this feature is used to associate an object with its ID in the form of an integer.

Without the use of `inline`, the static data member must be defined and initialized outside the class in most cases, which is not shown here. There are many variations and exceptions to when the use of `inline` is mandatory for initializing static data members and when it is not. In addition, it is also possible to declare static data members that are `const`. In this case, it is usually best to declare them as `constexpr`. If `inline` was omitted even though it is required, the linker usually complains about a missing static data member.

A static member function is always bound to the defining class itself. Therefore, the pointer `this` is not defined in a static member function. Furthermore, a static member function is always bound at compile time. A static member function cannot be declared as `const`, because there is no exemplar that may not be modified.

The static member function `isKnown()` takes a reference to `const` of type `Object` named `o` as its only argument. It returns a `bool` value which must not be ignored. Therefore, the `[[nodiscard]]` attribute is used. The implementation simply returns the result of calling the `find()` member function of `map<>` with the address of `o`. `map<>::find()` returns a so-called *iterator* that points to the key-value pair if the key is found, or is equal to `map<>::end()`, which points exactly to the position immediately after the last element of `map<>`. Iterators are briefly presented in (U. W. Eisenecker 2022), Section 6.5.4 *LegalAnswers Class*. More knowledge is not required for using them here. Iterators are introduced in more detail in the Chapter *Containers, Iterators and Algorithms*. The result of checking whether the value returned by `m_ids.find()` differs from `m_ids.end()` is returned immediately.

The static member function `getID()` also takes a reference to `const` of type `Object` with the name `o`. It first checks, if `o` is already associated with an identifier. If this is not the case, the index operator `[]` takes the address of `o` as index and the value of `m_id` pre-incremented by one is assigned to the result of the index operator evaluation. If `&o` were already contained in `m_ids`, this would associate the key `&o` with a new value. If `&o` is not contained in `m_ids`, a new pair with `&o` and `m_id` is stored in `m_ids`. Then the identifier associated with `&o` is retrieved again using the index operator and returned as a function result that must not be ignored, as specified with the `[[nodiscard]]` attribute.

The static member function `deregister()` also takes a reference to `const` of type `Object` named `o` as its only parameter. If an entry with `&o` as its key exists, it removes the corresponding entry by first retrieving it with `map<>::find()` and then calling `map<>::erase()` with the iterator returned by `map<>::find()`. After that, it returns `true`. If the `&o` key is not known, the member function returns `false`. In some cases, it can be neglected whether an object was successfully deregistered or not. Therefore, the return value of `deregister()` can be ignored.

Next, a class is implemented whose exemplars can have relationships to other exemplars. Using the `OIDManager`, it is possible to serialize the exemplars of this class into a CSV file. Listing 29 shows the `Person` class and a `main()` function that illustrates how to serialize exemplars of `Person` to a CSV file.

Listing 29: objects/Acquaintances.cpp

```
1 // Acquaintances.cpp by Ulrich Eisenecker, April 28, 2026
2
3 #include "oid_manager.hpp"
4 #include <fstream>
5 #include <iostream>
6 #include <set>
7 #include <string>
8
9 using namespace std;
10 using namespace oid_manager;
11
12 class Person: public Object
```

```

13 {
14     public:
15         Person(const string& name): m_name { name }
16         {}
17         void introduce(const Person& p)
18         {
19             m_acquaintances.insert(&p);
20         }
21         void print() const
22         {
23             cout << "I\'m " << m_name;
24             if (m_acquaintances.size() > 0)
25             {
26                 cout << " and I know ";
27             }
28             for (size_t i { 1 }; const auto& a : m_acquaintances)
29             {
30                 cout << a->m_name;
31                 if (i < m_acquaintances.size() - 1)
32                 {
33                     cout << ", ";
34                 }
35                 if (i == m_acquaintances.size() - 1)
36                 {
37                     cout << " and ";
38                 }
39                 ++i;
40             }
41             cout << '.' << endl;
42         }
43         void serialize(ostream& os) const
44         {
45             auto id { OIDManager::getId(*this) };
46             os << id << '\t'
47                 << m_name << '\t'
48                 << m_acquaintances.size();
49             for (const auto& a : m_acquaintances)
50             {
51                 os << '\t' << OIDManager::getId(*a);
52             }
53             os << endl;
54         }
55     private:
56         string m_name;
57         set<const Person*> m_acquaintances;
58 };
59
60 ostream& operator<<(ostream& os, const Person& p)
61 {
62     p.serialize(os);
63     return os;
64 }
65
66 int main()
67 {

```



```
68 {
69     Person p1 { "Arnet" },
70           p2 { "Marn" },
71           p3 { "Fry" },
72           p4 { "Kat" },
73           p5 { "Tat" },
74           p6 { "Pash" },
75           p7 { "Pesh" };
76
77     p1.introduce(p2);
78     p1.introduce(p3);
79     p1.introduce(p4);
80     p2.introduce(p1);
81     p2.introduce(p5);
82     p3.introduce(p6);
83     p4.introduce(p2);
84     p4.introduce(p3);
85     p5.introduce(p6);
86     p7.introduce(p7);
87
88     p1.print();
89     p2.print();
90     p3.print();
91     p4.print();
92     p5.print();
93     p6.print();
94     p7.print();
95
96     ofstream persons { "persons.csv" };
97     persons << p1 << p2 << p3 << p4 << p5 << p6 << p7 << flush;
98 }
99 }
```

The data members of `Person` are `m_name` of type `std::string` and `m_acquaintances`, a `std::set<>` container template instantiated for a pointer to a `const Person`. The `Person` constructor takes a string as an argument that initializes `m_name`. The member function `introduce()` takes a reference to `Person` named `p` and inserts the address of `p` into `m_acquaintances`. The `print()` member function produces output for users, informing about the name of the `Person` exemplar and the names of its acquaintances. The range-based for loop defines an additional loop variable named `i`. This loop variable is used in two conditions to decide whether a comma should be printed as a separator or whether nothing should be printed if there are no other known persons. After evaluating the following two if statements, the loop variable must be incremented.

The member function `serialize()` takes a reference to `ostream` named `os`. It first requests an ID for itself from the `OIDManager`, which is used to initialize the local variable `id`. Then `id`, `m_name`, and the number of elements contained in `m_acquaintances` are sent to `os`. Now a range-based for loop iterates over `m_acquaintances` with `a` as loop variable. For each `a`, its `id` is queried and sent to `os`. Finally, a new line is inserted to `os`.

For convenience, the stream insertion operator for `Person` is overloaded. It takes a reference of type `ostream` as its first parameter and a reference to `const` of type `Person` named `p` as its second parameter. In its body, it calls the `serialize()` member function for `p` and then returns `os`.

In `main()`, seven exemplars of `Person` are created and initialized with different names. Then several persons are introduced to others. An introduction is not symmetrical, i.e. if `p2` is introduced to `p1`, this does not mean that `p1` is also introduced to `p2`. A person who is not introduced to any other person is also possible (`p6`), as is a person who only introduces themselves (`p7`). Then an output file stream named `persons` is declared. This is opened with the file name `persons.csv` in the working directory. The last statement sends all `Person` exemplars to the `persons` output file. After compiling and running the program, a file named `persons.csv` exists in the working directory. It can be imported using a spreadsheet program, as shown in Figure 11.

Figure 11: Spreadsheet with `persons.csv` imported

A	B	C	D	E	F
1	Arnet	3	2	3	4
2	Marn	2	1	5	
3	Fry	1	6		
4	Kat	2	2	3	
5	Tat	1	6		
6	Pash	0			
7	Pesh	1	7		

The first column contains the object identifiers assigned by `OIDManager`, the second column contains the name, the third column contains the number of known persons, and the following cells contain as many object identifiers as there are persons known to the person of that row. For example, 3 in *C1* means that *Arnet* knows three other persons, those with identifiers 2, 3, and 4, which corresponds to *Kat*, *Fry*, and *Marn*.

Interestingly, the names in column *B* appear exactly in the order of the declaration of the objects in `main()`. A `std::map<>` sorts its elements internally. Pointers to objects are sorted like integers. Here, the order reflects the ascending memory positions of the objects in `main()`.

Without a doubt, it is nice to have a representation of the state of `Person` exemplars as a CSV file. It would also be useful to deserialize the content of the CSV file back into objects again. This step is significantly more difficult and is performed in the *reinterpret_cast<>()* Section.

The `OIDClass` unit tests highlight some problems that give initial insights into what needs to be considered when deserializing a CSV file containing objects with relationships. Listing 30 shows two simple unit tests implemented with *Catch2* v3.

Listing 30: `objects/OIDManagerUnitTests.cpp`

```
1 // OIDManagerUnitTests.cpp by Ulrich Eisenecker, March 18, 2025
2
3 #define CATCH_CONFIG_MAIN
4 #include <catch2/catch_test_macros.hpp>
5 // This code is based on Catch2 v3,
6 // therefore only the minimal header
7 // file for test macros is included.
8 // The C++ library path must be set
9 // accordingly for compilation and
10 // linking, and the Catch2Main and
11 // explicitly in exactly this order!
12
13 #include "oid_manager.hpp"
14
15 TEST_CASE("CLASS")
16 {
17     using namespace oid_manager;
18     Object o1,o2;
19     REQUIRE(OIDManager::isKnown(o1) == false);
20     REQUIRE(OIDManager::getId(o1) == 11l);
21     REQUIRE(OIDManager::isKnown(o1) == true);
22     REQUIRE(OIDManager::isKnown(o2) == false);
23     REQUIRE(OIDManager::getId(o2) == 21l);
24     REQUIRE(OIDManager::isKnown(o2) == true);
25     REQUIRE(OIDManager::deregister(o1) == true);
26     REQUIRE(OIDManager::deregister(o2) == true);
27     REQUIRE(OIDManager::deregister(o1) == false);
28     REQUIRE(OIDManager::deregister(o2) == false);
29 }
30
31 // CAUTION: The following test case
32 // requires the prior execution of
33 // TEST_CASE("StaticCall").
34 // This violates inherent Catch2
35 // assumptions!
36 TEST_CASE("OBJECT")
37 {
38     using namespace oid_manager;
39     OIDManager om;
40     Object o1,o2;
41     REQUIRE(om.isKnown(o1) == false);
42     REQUIRE(om.getId(o1) == 31l);
43     REQUIRE(om.isKnown(o1) == true);
44     REQUIRE(om.isKnown(o2) == false);
45     REQUIRE(om.getId(o2) == 41l);
46     REQUIRE(om.isKnown(o2) == true);
47     REQUIRE(om.deregister(o1) == true);
48     REQUIRE(om.deregister(o2) == true);
49 }
```

```

50 | REQUIRE(om.deregister(o1) == false);
51 | REQUIRE(om.deregister(o2) == false);
52 | }

```

The test case named `CLASS` creates two exemplars of `Object` named `o1` and `o2`. The first assertion requires that `OIDManager::isKnown()` returns `false` for `o1`. The next assertion uses knowledge about the implementation of `OIDManager`. Therefore, it is not a black-box test, but a *white-box test* or *glass-box test*. Here, requesting an identifier for the first unknown object `o1` returns the result `111`. If the test were converted into a pure black-box test, this assertion had to be deleted. The third assertion makes no assumptions about the implementation of `OIDManager`. Since `o1` previously requested an ID, `o1` must now be known to `OIDManager`. The next assertions are repetitions of the previous ones, now for `o2`. The last four assertions of the `CLASS` test case are about deregistration. First, there are two assertions that deregister `o1` and `o2`. Since both objects are registered, `OIDManager::deregister()` must return `true` to indicate successful deregistration. The next two assertions repeat the deregistration of `o1` and `o2`. Now the deregistration must return `false` because both objects are not registered.

The test case named `OBJECT` demonstrates a different syntax for accessing a static class member. Here, an exemplar of `OIDManager` is first created with the name `om`. Then all static methods are called as if they were normal member functions of `om`. This is perfectly legal in C++. But reading the corresponding code without knowing that all these methods are actually static member functions, one has absolutely no clue what these methods really are. ***It is strongly discouraged to use the object syntax for calling static member functions. Instead, they should be called with the class name followed by the scope operator***.

This test case is problematic because it is not independent of the `CLASS` test case. Actually, the `CLASS` test case must be executed before the `OBJECT` test case. If not, the `OBJECT` test case fails. This can be easily verified. Assuming that `OBJECTManager.cpp` was compiled to the executable `oidm`, only the `OBJECT` test case can be run by typing `./oidm OBJECT` in the terminal window. Figure 12 shows the resulting output.

It should be noted that `Catch2 v3` does not guarantee that test cases will be executed exactly in the order in which they appear in the code, even if all test cases are executed. Therefore, running the binary file for a test may result in failed test cases on some platforms.

The reason for this is that there is indeed only the `OIDManager` class with its static data members. They remain the same for all test cases. If the `CLASS` test case modifies the state of the static data members of `OIDManager`, this state applies to all other test cases as well. Therefore, the test cases can no longer be executed independently or in a pseudo-random order. One could argue that the problem would be fixed by not including the glass-box assertion that checks for a specific ID. This is of course correct. In contrast, one could argue for being able to reset the state of the `OIDManager` class, or

Figure 12: Output when executing `./oidmOBJECT`

```
% ./oidm OBJECT
Filters: "OBJECT"
Randomness seeded to: 2126269195

~~~~~
oidm is a Catch2 v3.8.0 host application.
Run with -? for options

-----
OBJECT
-----
OIDManagerUnitTests.cpp:37
.....

OIDManagerUnitTests.cpp:43: FAILED:
  REQUIRE( om.getId(o1) == 311 )
with expansion:
  1 == 3

=====
test cases: 1 | 1 failed
assertions: 2 | 1 passed | 1 failed
```

to convert all static members to normal members and provide different exemplars of `OIDManager` in different contexts. The former would require the introduction of a corresponding static `reset()` method, the latter would mean abandoning static members in `OIDManager` altogether and relying on data member and member functions that are bound to an object rather than its class.

Overall, the `OIDManager::getId()` class method and the `Person::serialize()` method are sufficient for serializing `Person` objects. A `Person::deserialize()` method would face the problem of resolving IDs from the CSV file back to pointers to `Person`. Having accomplished this, one must deal with the problem that some objects have not yet been created when their IDs are extracted. All this is followed up in the *reinterpret_cast<>()* Section.

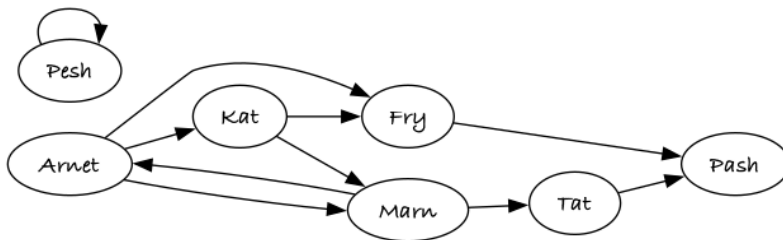
1.10. A Detour Into Fictitious Practice

This section describes the desire for an extension to an existing program and its implementation as it might actually occur in practice. It is about how such an extension can be implemented with minimal effort and without compromising comprehensibility and maintainability. In some cases, there are jumps between different aspects of requirements, design and implementation. This should give an impression of the fact that in development practice, the various activities are not executed in strict sequence. Rather, iterative development switches back and forth several times between various activities.

1.10.1. A Nice and Inexpensive Feature

Assuming the program *objects/Acquaintances.cpp* (Listing 29) has been developed for a customer. After the demonstration of the program, the customer is satisfied, but asks for an optional requirement: The provision of an additional graph for the relationships between persons would be nice. As an example, the customer sketches the relationships as shown in Figure 13.

Figure 13: Customer sketch of person graph



The customer is willing to pay only a small amount of money to implement this requirement, as it is not seen as crucial. The customer also states that a console-based application is completely sufficient.

1.10.2. Options for Implementation

Of course, the entire program plus extension could be re-implemented as an application with a graphical user interface (see Section *The GUI-Programming Paradigm*). To do this, the console-based application would have to be rewritten as a GUI application, which would be very expensive. Lightweight alternatives are therefore being sought.

For example, *Mermaid* (<https://mermaid.js.org>) is a tool that allows certain diagram types to be described textually. However, the diagram desired by the customer does not correspond to any of *Mermaid*'s diagram types and *Mermaid* primarily runs in a web browser, which makes the exchange of data somewhat more difficult. These two arguments speak against the use of *Mermaid*.

Another option is *Graphviz* (<https://graphviz.org>). It allows the textual specification of graphs in a flexible way, without prescribing certain diagram types. It supports a variety of output formats and several layout engines are available. Interestingly, *Doxygen* also uses *Graphviz*. Therefore, *Graphviz* will likely be available when *Doxygen* is fully installed. All this suggests the use of *Graphviz*.

1.10.3. Use of the DOT Language

Next, it is outlined how to implement the generation of a person graph specification for *Graphviz*.

First, it is advisable to read the online documentation of *Graphviz* in order to familiarize oneself with the grammar of the text language for the graph specification, namely *DOT*. The graph specification shown in Listing 31 is then created manually. It contains the persons and their relationships that are used in Listing 29.

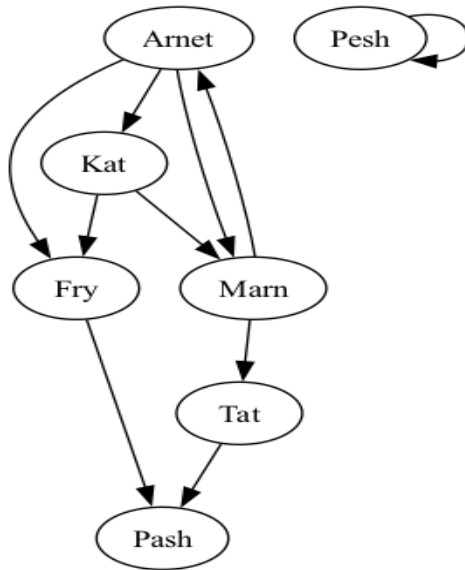
Listing 31: Manually created DOT file for person relations (objects/manual.viz)

```
1 digraph relations
2 {
3     1 [ label = "Arnet" ]
4     2 [ label = "Kat" ]
5     3 [ label = "Fry" ]
6     4 [ label = "Marn" ]
7     5 [ label = "Tat" ]
8     6 [ label = "Pash" ]
9     7 [ label = "Pesh" ]
10
11     1 -> { 2 3 4 }
12     4 -> { 5 1 }
13     3 -> { 6 }
14     2 -> { 3 4 }
15     5 -> { 6 }
16     7 -> { 7 }
17 }
```

The body `digraph relations { /* ... */ }` defines that a directed graph with the internal name `relations` is to be specified. It would be legal to omit the name of the graph, in which case an anonymous graph would be created. The specification of the nodes and the relations is enclosed by a pair of curly brackets. This is preceded by a multi-line comment as in C++ to indicate that some details have been omitted. The upper half of Listing 31 contains the specification of the nodes. Here they are simply numbered, starting with 1 as their IDs. For each node, the attribute name is given in square brackets. The second half contains the relations. For each relation, first a node ID and then the arrow is specified, followed by a list of node IDs to which relations exist, which are specified in curly brackets. This manual specification of a graph is available in the `objects` subdirectory under the name `relations.viz`. The ending was deliberately chosen as `.dot` would conflict with the extension for older *Microsoft Word*[®] style sheets, which are still widely used.

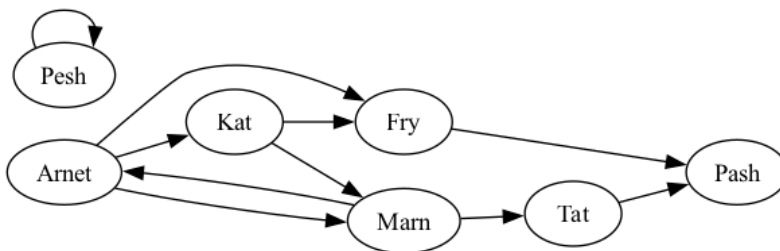
Executing `dot -Tpng < manual.viz > manual.png` in a console window creates the file `manual.png`, a graphics file in *portable network graphics* format (PNG for short). The `dot` command calls the processor for the *DOT* language. The `-T` option followed immediately by `png` specifies that a graphics file is to be created in PNG format. The `dot` command takes the argument `manual.viz` as input and creates `manual.png` by applying I/O redirection. Opening the `manual.png` file produces the result shown in Figure 14.

Figure 14: Graph for objects/manual.viz produced by dot



The customer is almost satisfied with the result. A final wish is that the diagram should be drawn from left to right and not from top to bottom. Therefore, the option `rankdir = LR` is inserted in `manual.viz` immediately after the opening curly bracket, which gives the result shown in Figure 15.

Figure 15: Person graph in landscape format



One further detail should be mentioned. The *DOT* syntax is very flexible. This also applies to the order in which nodes and relations can be specified. It is also possible to define the relations first and then the nodes. The definitions of nodes and relations can also be mixed. This will be important when implementing the creation of a person graph.

1.10.4. Implementation of the Feature

Knowing this, the question arises as to how the generation of a *DOT* file can be implemented in the program shown in Listing 29. The given restrictions are that the implementation should be inexpensive. For the programmer, this means that the implementation should cause as little effort as possible. Furthermore, it should not be too complex, it should be easy to use, and—additionally—one could argue that it should be easy to remove it with only minimal intervention, if it is no longer desired or is to be replaced by an improved implementation.

The ease of use should be discussed first. In the original program, information about person relations is output by calling the stream insertion operator for them, i.e. `persons << p1 << /* ... */ << p7 << flush;`. It would be fairly simple, if a *DOT* specification could be created in a similar way. How to do this is described below.

A *DOT* specification is plain text, i.e. it could be written to an output file stream, e.g. `ofstream graph { "persons.viz" };`. Listing 32 sketches its possible content.

Listing 32: Outline of graph specification in DOT

```
1 digraph relations
2 {
3     rankdir = LR
4
5     // Nodes and edges
6
7 }
```

The information about the graph and the beginning of the graph body are highlighted **green**. This part is identical for all person graphs. The following comment, which is highlighted in **blue**, contains information that is specific to each person graph. The end of the graph body is highlighted in **red**. This part is also identical for all graphs.

The invariant parts could be produced by a constructor (**green**) and the destructor (**red**) of a `RelationsGraph` class, which still needs to be implemented in detail.

Obviously, a `RelationsGraph` class does not know about persons and their relationships, i.e. the variable part of the graph (**blue**). Therefore, this part must become the responsibility of a new member function of the `Person` class. It will be implemented as an overload of the `Person::serialize()` member function, e.g. `void Person::serialize(RelationsGraph& graph) const.`

This member function is called by an overloaded stream insertion operator, i.e. `RelationsGraph& operator<<(RelationsGraph& os, const Person& p)`.

Listing 33 shows an excerpt from the resulting program, which focuses on the additions mentioned above.

Listing 33: `objects/RelationsGraph.cpp`

```

1 // RelationsGraph.cpp by Ulrich Eisenecker, April 28, 2026
2
12 {
13     public:
14         RelationsGraph(ostream& os):m_os { os }
15         {
16             m_os << "digraph relations\n"
17                 << "{\n"
18                 << "    rankdir = LR\n";
19         }
20         ~RelationsGraph()
21         {
22             m_os << "}" << flush;
23         }
24         operator ostream& ()
25         {
26             return m_os;
27         }
28     private:
29         ostream& m_os;
30 };
31
32 class Person: public Object
33 {
34     public:
35         Person(const string& name): m_name { name }
36         {
37             #ifndef __clang__
38                 ostream& os { graph }; // When not compiling with clang++.
39             #else
40                 ostream& os = graph; // When compiling with clang++.
41             #endif
42             auto id { OIDManager::getId(*this) };
43             os << "    " << id
44                 << " [ label = \"" << m_name << "\" ]\n    "
45                 << id << " -> {";
46             for (const auto& a : m_acquaintances)
47             {
48                 os << ' ' << OIDManager::getId(*a);
49             }
50             os << " }\n";
51         }
52     private:
53
54     ostream& operator<<(ostream& os,const Person& p)
55     {
56

```

```
105 |     p.serialize(os);
106 |     return os;
107 | }
108 |
109 | int main()
110 | {
111 |     {
139 |         {
140 |             ofstream persons { "persons.csv" };
141 |             persons << p1 << p2 << p3 << p4 << p5 << p6 << p7 << flush;
142 |         }
143 |         {
144 |             ofstream graph { "persons.viz" };
145 |             RelationsGraph persons { graph };
146 |             persons << p1 << p2 << p3 << p4 << p5 << p6 << p7;
147 |         }
148 |     }
149 | }
```

The most prominent addition is the `RelationsGraph` class. It is marked as `final` because it is not intended to serve as a base class. Considering a possible reuse for a nice addition is not appropriate. The class has only one data member, namely `m_os`, which is a reference of type `std::ostream`. This means that an exemplar of `RelationsGraph` does not own an exemplar of a subclass of `std::ostream`, it merely uses an alias for an output stream that exists somewhere outside.

Consequently, the `m_os` reference must be initialized in the constructor. A reference of type `ostream` called `os` is passed there and is used to initialize `m_os` in the initialization list. This has two major consequences. The first is that an exemplar of a subclass of `std::ostream` must be created in the calling context before a `RelationsGraph` exemplar is created, which makes the use of `RelationsGraph` somewhat more complicated. The second is that a `RelationsGraph` exemplar can be used with an exemplar of any class that has `ostream` as a direct or indirect base class, even with a class that does not yet exist. This is a powerful feature that outweighs the somewhat more complex creation of a `RelationsGraph` exemplar and is very inexpensive to implement. In the constructor body, the necessary header information for a DOT graph specification including the invariant body part of the specification is output to `m_os`.

The destructor ends the *DOT* graph specification by outputting the closing curly bracket to `m_os`. Please note that the parts of the code that generate the invariant parts of the specification are highlighted accordingly.

A special feature is the type conversion operator, which returns `m_os` as an `ostream` reference. It is called automatically if a non-constant `ostream` exemplar is expected, but a `RelationsGraph` exemplar is provided. A type conversion operator must be declared as a member function. It has a special syntax in that it specifies the return type after the operator keyword. As it is a unary operator, the return type is followed by an empty pair

of parentheses. In this way, the coercion polymorphism compensates for the inclusion polymorphism, which is not applied here to allow subtyping in relevant situations, as shown below.

Unfortunately, there is a compiler-specific issue that has to be resolved. *clang++* (version 22.1.3) does not allow to define the ostream reference with the initializer list syntax. The reported error is that the constructor of ostream is explicitly deleted, which is correct per se, but is not relevant here. Instead, the more classic syntax `ostream& os = graph;` must to be used. Therefore, the preprocessor directives `#ifndef` (which means *if not defined*), `#else` and `#endif` are used to provide the special syntax for declaring the reference for *clang++*. If *clang++* is used as compiler, the `__clang__` flag is defined. *g++* (version 14.2.0_1 or newer) compiles the code without problems.

The next important addition is an overloaded `serialize()` member function of `Person` that takes a reference to `RelationsGraph` as an argument. In its implementation, an ostream reference named `os` is initialized with `graph` passed by reference. This is possible because of the type conversion operator discussed immediately above. When called, the output stream is returned, which is managed as a reference by the graph exemplar. All subsequent outputs are sent to this output stream. The remaining implementation of the member function is structurally equivalent to `Person::serialize(ostream&)`. The only difference is that the output is prepared for a *DOT* specification. The implementation is therefore not discussed further here. The code that generates the variable output of a person graph is highlighted in blue.

The last addition is the overloaded stream insertion operator, which accepts a `RelationsGraph` reference and a reference to `const` of type `Person` as arguments. Its implementation is exactly the same as the stream insertion operator overloaded for an ostream reference and a `const Person` reference.

1.10.5. Evaluation of the Implementation

To summarize, in this implementation for creating a *DOT* specification of a person graph, coercion polymorphism and overloading polymorphism are combined in a clever way. The changes required are relatively minor and each is simple in its own right, although their combination is—admittedly—somewhat tricky. This is justified insofar as a—just—nice feature has been implemented very cheaply in this way. The entire implementation is not designed for reuse. However, it can be easily used and also easily removed, as it is implemented in a few, well-defined places and is not distributed throughout the entire code.

The excerpt from the `main()` function shows both the use of stream insertion for text output and stream insertion for the creation of a *DOT* specification. As mentioned earlier, the output file stream named `graph` must be created first, before the persons exemplar is defined for generating the *DOT* specification. But there is another difference. The manipulator `flush` is not inserted into persons when the *DOT* specification is created. This is because an overloaded stream insertion operator that accepts a manipu-

lator as a second argument is not (automatically) available for `RelationsGraph`, since `RelationsGraph` is not a subtype of `ostream` for which this operator is defined in the standard library. This could be fixed by manually defining the appropriate overloading of this stream insertion operator for `RelationsGraph` and a manipulator. However, this would require a deeper insight into manipulators, their definition and their interaction with streams, which would no longer support a cheap implementation of a nice feature.

Using the solution presented is somewhat clumsy. Figure 16 illustrates the procedure for compiling and executing *RelationsGraph.cpp* and calling *dot* with *persons.viz* as the specification and creating *graph.png* as the graph representation.

Figure 16: Creating a person graph with DOT

```
ulrich@Macmini adding_functionality % g++-14 -std=c++23 RelationsGraph.cpp -o rg
ulrich@Macmini adding_functionality % ./rg
I'm Arnet and I know Marn, Fry and Kat.
I'm Marn and I know Arnet and Tat.
I'm Fry and I know Pash.
I'm Kat and I know Marn and Fry.
I'm Tat and I know Pash.
I'm Pash.
I'm Pesh and I know Pesh.
ulrich@Macmini adding_functionality % dot -Tpng persons.viz > graph.png
ulrich@Macmini adding_functionality %
```

Since *graph.png* looks exactly the same as in Figure 15 it is not reproduced here.

1.11. Using Declaration

The using declaration has many facets. Besides importing identifiers from namespaces or declaring type aliases, it can make methods of a base class available in a derived class, especially those that would be otherwise not be accessible from outside due to private or protected inheritance.

The using keyword has different meanings depending on the context in which it is used. One of the most common uses is to import identifiers from namespaces either fully or selectively, such as with `using namespace std;`. It can also be used to declare aliases for types, as presented in the *The Virtual Function Table* Section, for example `using integer = signed long long;`

In the context of object-oriented programming, it can be used to make methods of the base class available that would otherwise have to be redefined in a subclass. A first example is the using declaration for constructors in a subclass (Listing 34).

Listing 34: objects/UsingConstructors.cpp

```

1 // UsingConstructors.cpp by Ulrich Eisenecker, February 23, 2023
2
3 #include "RN.hpp"
4
5 class DerivedRationalNumber: public RationalNumber
6 {
7     using RationalNumber::RationalNumber;
8 };
9
10 int main()
11 {
12     using namespace std;
13     DerivedRationalNumber a { 3,5 },
14                             b { "4", "-7" };
15     a.output(cout);
16     cout << endl;
17     b.output(cout);
18     cout << endl;
19 }

```

The `DerivedRationalNumber` class is a public subclass of the `RationalNumber` class, available through the inclusion of `"RN.hpp"`. Without further precautions, the compiler would automatically generate a standard constructor for `DerivedRationalNumber`. This standard constructor would in turn call the base class constructor as standard constructor using the specified default values. If a constructor for `DerivedRationalNumber` were defined manually, it would also have to call the base class constructor. And if the base class constructor would also suffice for the subclass, it would still have to be defined in the subclass and call the corresponding base class constructor.

To avoid this tedious and error-prone manual work, the `using` declaration (line 7 in Listing 34) allows to import all constructors of `RationalNumber` and make them publicly available. Note that the parentheses must not be used here, otherwise the compiler would consider this a function call. Two important details need to be emphasized. First, all constructors are imported into the subclass, it is not possible to selectively import constructors. Second, the `using` declaration implies a public import. Here it occurs in the implicitly private section of `DerivedRationalNumber`. Nevertheless, the constructor in `main()` is publicly available to create exemplars of the class. Placing `private:` immediately before the `using` declaration would not change this. This feature applies only to `using` declarations of constructors.

If a subclass does not add attributes, it is convenient to make the base class constructors available through a `using` declaration. If new attributes are appropriately initialized by their standard constructors or inline at the point of their declaration, a `using` declaration for the base class constructors can also be appropriate.

Listing 35 shows another form of the using declaration. Sometimes a class inherits from a base class because it reuses some relevant members without the subclass being a subtype of the base class. Not being a subtype means that an exemplar of the subclass may not be used if an exemplar of the base class is expected, i.e. the *Liskov type substitution principle* does not apply. In such a case it is appropriate to use private inheritance.

Listing 35: objects/UsingPrivateBaseClass.cpp

```
1 // UsingPrivateBaseClass.cpp by Ulrich Eisenecker, February 23, 2023
2
3 #include <iostream>
4 using namespace std;
5
6 class Fish
7 {
8     public:
9         void swim() const
10         {
11             cout << "I can swim" << endl;
12         }
13 };
14
15 class Whale: private Fish
16 {
17     public:
18         using Fish::swim;
19 };
20
21 int main()
22 {
23     Fish { }.swim();
24     Whale { }.swim();
25 }
```

Since a whale is a mammal, it is of course not a fish. Nevertheless, from a certain point of view, a whale shares some properties or behavior with a fish. Therefore, Fish is specified as the private base class of Whale. This has the effect that any public or protected method of Fish can be used inside Whale, but not outside Whale.

Here, Fish::swim() should be made publicly available for Whale. The using declaration does exactly that. The statement using Fish::swim;—again without parentheses—imports all overloaded member functions named swim() from Fish. Unlike the using declaration for constructors, the using declaration of other member functions must be placed in a public section if the corresponding methods are to be made publicly available. Again, it is not possible to selectively import a method with the using declaration. As demonstrated in main(), it is now possible to call the swim() method for a Whale as well. After the using declaration has been converted to a comment, this is no longer pos-

sible and the compiler will print an error message. For the sake of brevity, temporary objects are created with list initialization to show that the `swim()` method can be called for exemplars of both classes.

1.12. Default and Delete

For certain member functions, it is possible to force the creation of default implementations. On the other hand, it is possible to forbid member functions, i.e. to delete them. Then neither standard implementations are created nor manual definitions are allowed. Deleting operators for creating and deleting dynamically created objects is used as an example of this.

Once a user-defined constructor is available for a class, the compiler no longer automatically generates a standard constructor. This would mean that the standard constructor would have to be implemented manually. A simpler option is to enforce its presence by means of a default declaration. Listing 36 shows how to default the standard constructor.

Listing 36: objects/Default.cpp

```

1 // Default.cpp by Ulrich Eisenecker, February 24, 2023
2
3 #include <string>
4
5 using namespace std;
6
7 class Person
8 {
9     public:
10         Person(string name):m_name(name)
11         {}
12         Person() = default;
13     private:
14         string m_name;
15 };
16
17 int main()
18 {
19     Person p1 { "Jude" };
20     Person p2;
21 }
```

Defaulting the standard constructor requires more typing than explicitly providing an implementation for it. However, the use of `= default` is more intentional and has some subtle differences that will not be discussed here.

Besides the standard constructor, the compiler can automatically generate a copy constructor, a copy-assignment operator, a destructor and some more special member functions which are not covered in this book. They all can be defaulted as well. Another group of operators that can be defaulted under certain circumstances are comparison operators, which is not explained in this book.

The opposite of defaulting is deleting. All special member functions that can be defaulted can also be deleted. In addition, it is possible to delete any other member function. When a member function is deleted, no manual definition needs to be specified for it. Listing 37 demonstrates a number of deleted member functions and the consequences of deleting them.

Listing 37: objects/Delete.cpp

```
1 // Delete.cpp by Ulrich Eisenecker, January 17, 2024
2
3 #include <string>
4
5 using namespace std;
6
7 class Person
8 {
9     public:
10         Person(string name):m_name(name)
11         {}
12         Person() = default;
13         Person(const Person&) = delete;
14         Person& operator=(const Person&) = delete;
15         void* operator new(std::size_t) = delete;
16         void* operator new[](std::size_t) = delete;
17         void operator delete(void*,std::size_t) = delete;
18         void operator delete[](void*,std::size_t) = delete;
19         Person* operator&() const = delete;
20     private:
21         string m_name;
22 };
23
24 int main()
25 {
26     Person p1 { "Jude" };
27     Person persons[2]; // Possible
28     // Person p2 { p1 }; // Not possible
29     Person p3 { "Jaden" };
30     // p1 = p3; // Not possible
31     // Person* p4 = { new Person { "Jill" } }; // Not possible
32     // Person* p5 { new Person[2] }; // Not possible
33     // delete p4; // Not possible
34     // delete[] p5; // Not possible
35     Person* p6 { nullptr };
36     // delete p6;
37     // delete[] p6;
38     // Person* p7 { &p1 }; // Not possible
```

In the `Person` class, the standard constructor is defaulted so that it is possible to create an array of `Person` without the need to call another constructor.

Next, member functions are deleted for which the compiler would otherwise automatically generate default implementations. Most of them are operators specified exactly as they had to be overloaded as member functions. For a better understanding, it is recommended to try out each of the actions described below.

In line 13, the copy constructor is deleted. Normally, it is generated automatically when an object is created with another exemplar of the class. If line 13 is converted into a comment, line 28 can be uncommented and the program can be compiled.

In line 14, the copy assignment operator is deleted. It is automatically generated when an object is assigned to an object of the same class. If line 14 is converted to a comment, line 30 can be uncommented to compile the program successfully.

In line 15 the new operator for dynamically creating a single `Person` exemplar on the heap is deleted. If this line and line 17 are converted into a comment, line 31 can be uncommented and the program will compile successfully. Interestingly, not converting line 17 into a comment causes the compiler to report an error because the new operator is deleted and the delete operator is explicitly marked as deleted.

In line 16 the new operator for dynamically creating a variable number of `Person` exemplars on the heap is deleted. Like the new operator, this line and line 18 must be converted to comments so that line 32 can be uncommented and the program can be compiled.

In line 17 the delete operator to release exactly one dynamically created `Person` exemplar is deleted. If this line is converted into a comment, line 33 can be uncommented and the program compiled. Interestingly, in this case it is not necessary to convert the corresponding new operator (line 15) into a comment in order to successfully compile the program. This is not symmetrical to the new operator.

In line 18, the delete[] operator to release a variable number of dynamically created `Person` exemplars is deleted. After turning this line into a comment, line 34 can be uncommented and the program will compile successfully. Again, it is not necessary to turn the corresponding new[] (line 16) into a comment.

In line 19, the & operator that takes the address of a `Person` exemplar is deleted. If this line is converted to a comment, line 35 can be uncommented and the program will compile successfully.

One might assume that all deleted member functions leave only two constructors, namely a conversion constructor that takes a `std::string` argument and the standard constructor. However, this is not the case. A close look at lines 26 and 29 shows that

the arguments when calling the `Person` constructor are of type `const char *`, but not of `std::string`. This means that the constructor in line 10 also works for `const char *` arguments. To prevent this, another constructor had to be deleted, namely `Person(const char*) = delete;`. Now, lines 26 and 29 cannot be compiled. To fix this, `std::string` arguments had to be used.

Another thought could be that the `Person` class prohibits everything that has to do with dynamic objects. But that would not be entirely true. If `Person` had a public member, such as a member function `mf()`, the statement `p6->mf();` would compile successfully. However, the program would still generate a runtime error, because dereferencing a pointer with the value `nullptr` is not permissible. Although it is possible to overload the `->` operator for the `Person` class (with a `Person` exemplar as the implicit first and only argument), it is not possible to delete or overload the built-in `->` operator that takes a pointer to `Person` as its argument.

In the context of object-oriented programming and inheritance, measures such as those presented before should always be considered for possible subclasses as well. Despite the fact that `Person` has no virtual destructor and therefore cannot be a valid base class, it is legal to derive a subclass, for example `NewPerson`, that explicitly defines all the member functions that have been deleted for the base class. As long as these member functions do not call the deleted member functions of the base class, the `NewPerson` class will compile without errors. Given all this, it would make sense to mark `Person` as `final`. This would prevent publicly deriving a subclass of `Person` that can recover undesirable behavior. On the other hand, it becomes obvious that it will be impossible to completely restrict the behavior of subclasses by imposing restrictions on the base class.

1.13. `[[nodiscard]]` for Classes

The `[[nodiscard]]` attribute is introduced in (U. W. Eisenecker 2022), Section 2.3.3 *Ignoring Return Values* and has been widely used since then. The attribute can also be used in a class definition. Then it has the effect that the compiler warns if a temporary of this class is not used. Listing 38 contains a reduced version of the `RationalNumber` class defined with `[[nodiscard]]` after the `class` keyword. The member function `RationalNumber::copy()` demonstrates that the effect of `[[nodiscard]]` at class level also applies to member functions that return a temporary object of this class.

Listing 38: objects/Nodiscard.cpp

```

1 // Nodiscard.cpp by Ulrich Eisenecker, May 5, 2026
2
3 #include <cstdlib>
4 #include <cstdint> // Because of maxint_t
5
6 using namespace std;
7
8 class [[nodiscard]] RationalNumber
9 {
10     public:
11         RationalNumber(const intmax_t& n = 0, const intmax_t& d = 1):
12             m_numerator { n }, m_denominator { d }
13         {}
14         intmax_t numerator() const
15         {
16             return m_numerator;
17         }
18         intmax_t denominator() const
19         {
20             return m_denominator;
21         }
22         RationalNumber copy() const
23         {
24             return *this;
25         }
26     private:
27         intmax_t m_numerator,
28                 m_denominator;
29 };
30
31
32 int main()
33 {
34     RationalNumber rn { };
35     rn.numerator();
36     rn.denominator();
37     rn.copy(); // Warning: return value not used
38     RationalNumber { }; // Temporary not used
39     RationalNumber { 1 }; // Temporary not used
40     RationalNumber { 1, 3 }; // Temporary not used
41 }

```

Compiling this program generates four warnings. The first warning occurs at line 35, because the `RationalNumber` exemplar, which is returned as a value by calling `rn.copy()`, is not used. This warning can be silenced in two ways. First, a reference to `RationalNumber` can be returned in the declaration of `RationalNumber::copy()`. In this case, there is no longer a temporary object of type `RationalNumber` that is ignored. Second, line 35 is

changed to `auto rn2 { rn.copy() };`. Now the return value is used. The following three warnings create temporaries of type `RationalNumber`, but do not use them, e.g. to initialize a variable.

When compiling the program, differences can be detected with regard to the compiler used. Version 15.2.0 of `g++` only generates a warning for not using the result of the call to `rn.copy()`. The creation of the three temporary and unnamed copies of `RationalNumber` does not cause a warning. This is different when using `clang++` 22.1.4, which issues a warning in all four cases.

It has to be pointed out that `[[nodiscard]]` after the `class` keyword does not have the same effect as attributing a (member) function with `[[nodiscard]]`. In the `RationalNumber` class of Listing 38 the member functions `numerator()` and `denominator()` are not prefixed with `[[nodiscard]]`. Therefore, the result of calling `rn.numerator()` and `rn.denominator()` in `main()` can be ignored without warning.

If a public subclass of `RationalNumber` is not defined with `[[nodiscard]]`, temporary exemplars of it can be created without warnings. Calling the `RationalNumber::copy()` member function for an object of this subclass will nevertheless issue a warning. Therefore, the use of `[[nodiscard]]` in a class definition should also be considered with respect to inheritance.

1.14. Object With Dynamic Resource

A dynamic resource is acquired and released at runtime. Most modern operating systems automatically release a dynamic resource when the acquiring program exits. However, that cannot be taken for granted. Not releasing dynamic resources can result in faulty programs or even a crash of the operating system. The best-known example of a dynamic resource is heap memory; others are files and processes. The usage pattern for a dynamic resource is to acquire it only when it is needed and to release it immediately when it is no longer needed.

1.14.1. Prohibited Copying and Assigning

Objects that own a dynamic resource are not only responsible for acquiring and releasing the dynamic resource. They must also take special care with respect to copying and assigning. This is illustrated by a `Person` class that provides a diary as a dynamic resource for each of its exemplars. In this example the `mutable` specifier is introduced which allows to modify data members also for `const` objects.

First, it is shown how the `C++` standard library handles classes that use a dynamic resource (Listing 39).

Listing 39: objects/Diary.cpp

```

1 // Diary.cpp by Ulrich Eisenecker, March 1, 2023
2
3 // Caution: To successfully compile this program,
4 // the lines "Person kane { josh };" and
5 // "josh = josh;" must be turned into comments.
6
7 #include <iostream>
8 #include <fstream>
9 #include <string>
10
11 using namespace std;
12
13 class Person
14 {
15     public:
16         Person(const string& name)
17             : m_diary { name + "s_diary.txt" },
18               m_name { name }
19         {
20             m_diary << name
21                   << " was created."
22                   << endl;
23         }
24         const string& name() const
25         {
26             m_diary << m_name
27                   << " was asked for the name."
28                   << endl;
29             return m_name;
30         }
31         void name(const string& newName)
32         {
33             m_diary << m_name
34                   << " changed the name to "
35                   << newName
36                   << '.'
37                   << endl;
38             m_name = newName;
39         }
40         virtual ~Person()
41         {
42             m_diary << m_name
43                   << " passed away."
44                   << endl;
45         }
46     private:
47         mutable ofstream m_diary;
48         string m_name;
49 };
50
51 int main()

```

```
52 | {  
53 |     Person josh { "Josh" };  
54 |     cout << josh.name() << endl;  
55 |     josh.name("Jaden");  
56 |     cout << josh.name() << endl;  
57 |     Person kane { josh };  
58 |     josh = josh;  
59 | }
```

The `Person` class defines a constructor, a destructor, and a getter and a setter method for its attribute `Person::m_name`, which is of type `std::string`. The idea is that a `Person` exemplar records everything that happens to it in its life in a kind of electronic diary. The `Person::m_diary` member is an output file stream (therefore, `<fstream>` must be included) and as such is an example of a dynamic resource.

Each exemplar of `Person` has its own file stream. This file stream is declared with the mutable specifier. This allows this data member to be modified even for an object that is `const`. This is important, because the getter method `Person::name()` is declared as `const` because it is not allowed to modify the data member `Person::m_name`, but nevertheless the call of this method should be recorded in the diary. In addition, it is imaginable to create `Person` exemplars that are `const`. If `Person::m_diary` were not declared as mutable, it would be impossible to record anything in it. This can be seen as an interesting example of `const` objects that cannot be fully `const`.

It is obvious that two `Person` exemplars cannot share a diary, i.e. each `Person` exemplar must use its diary exclusively. It therefore makes no sense to copy the `Person::m_diary` attribute or assign it a different diary. In the `main()` function, the `Person josh` is created and then used. However, the compiler reports a number of errors when it compiles the last two lines of `main()`. The statement `Person kane { josh };` creates the `Person kane` by copy initialization, that is, it calls the copy constructor with `josh` as an argument.

The corresponding error message may vary depending on the compiler. Essentially, it should say that the copy constructor for the `Person` class is implicitly deleted because the copy constructor for the `m_diary` attribute is deleted. The statement `josh = josh;` performs an assignment. It is somewhat nonsensical because it assigns `josh` to itself, yet it is syntactically valid. Here the error message informs that the assignment operator for `Person` is implicitly deleted because it is deleted for `m_diary`.

The designers of the `C++` standard library anticipated this problem and as a precaution deleted member functions for output string streams that cannot be reasonably applied. Replacing the type of `Person::m_diary` with `std::ostringstream` (`<sstream>` must be included for this), the errors remain the same. An `ostringstream` data member is also a dynamic resource which should be owned exclusively by an object. Input file streams and input string streams are further examples of this category of dynamic resources.

In modern C++, it is no longer necessary to treat the heap memory directly as a dynamic resource. Instead, a so-called *smart pointer* can and should be used (see Section *Smart Pointers*).

1.14.2. Dynamic Resource Objects Lab

Sometimes a deeper understanding of a class that has a dynamic resource is necessary. In the following, the program *Resource.cpp* is presented, which is a laboratory for the study of objects, especially with regard to dynamic resources. It is not related to any specific application. It simply helps to identify the difficulties that may arise when using a class with a dynamic resource. The simplest action in this case is to delete potentially dangerous member functions. It also briefly touches on *move semantics*, which is a complex concept. For this reason, its various components will be presented piece by piece.

Resource.cpp is an experimental workspace that allows to detect a certain behavior using only the means of the C++ language. Listing 40 presents the beginning of the program and its two parts that perform the operations of interest.

Listing 40: Class, test, and main() of objects/Resource.cpp (excerpt)

```

1  // Resource.cpp by Ulrich Eisenecker, March 1, 2023
2
3  #include <iostream>
4  #include <string>
5
6  using namespace std;
7
155
156 using Class = NoDynamicResource;
157
158 Class test(Class o)
159 {
160     o = Class { };
161     return o;
162 }
163
164 int main()
165 {
166     Class a;
167     Class b { a };
168     a = b;
169     auto c { test(Class { }) };
170 }
```

In line 156, the `Class` identifier is declared as an alias for `NoDynamicResource`. The definition of the class `NoDynamicResource` is given later. This using declaration is the central point that allows to select one of several classes to examine its specific properties.

The `test()` function takes a parameter of type `Class` with name `o` passed by value. In its implementation it performs a copy assignment. A temporary `Class` object is assigned to `o`. Finally, `o` is returned as a value. All of these are important details. Here is what can be expected based on the knowledge presented before:

1. Passing `o` by value in line 158 means that `test()` uses a local `Class` exemplar. If necessary, the local exemplar must first be created by copy initialization. Since line 169 uses a temporary `Class` exemplar as a parameter for the call to `test()`, an additional copy is not required.
2. In line 160, the standard constructor of `Class` is called to create a temporary `Class` exemplar, which is then assigned to `o`.
3. The temporary `Class` exemplar, which is used as the right operand of the assignment, must be deleted which involves calling its destructor (line 160).
4. Since `o` is returned by value, the `Class` exemplar to be returned is copy initialized (line 161).
5. The local `Class` exemplar `o` is destroyed when the function terminates (line 162).

In `main()`, the standard constructor is called to create `a`. Then `b` is created using copy initialization. `b` is then assigned to `a`. Finally, `c` is created using copy initialization. The `test()` function itself is called with a temporary `Class` exemplar created using list initialization. Finally, the destructor for `c`, `b`, and `a` is called in exactly this order.

In summary, `main()` and `test()` call constructors, destructors, and assignment operators.

1.14.3. Object Without Dynamic Resource

In order to examine objects with different types of resources, a class that serves as a resource and a class that does not have a dynamic resource are presented first.

Listing 41 introduces the `Resource` and `NoDynamicResource` classes.

Listing 41: Resource and NoDynamicResource of objects/Resource.cpp (excerpt)

```
8 | class Resource
9 | {
10 |     public:
11 |         Resource():m_myId { ++m_nextId }
12 |         {
13 |             report("standard constructor");
14 |         }
15 |         Resource(const Resource&):m_myId { ++m_nextId }
16 |         {
17 |             report("copy constructor");
18 |         }
19 |         Resource(Resource&&):m_myId { ++m_nextId }
20 |         {
21 |             report("move constructor");
```

```

22     }
23     virtual ~Resource()
24     {
25         report("destructor");
26     }
27     Resource& operator=(const Resource&)
28     {
29         report("assignment");
30         return *this;
31     }
32     Resource& operator=(Resource&&)
33     {
34         report("move assignment");
35         return *this;
36     }
37 private:
38     void report(const string& mf) const
39     {
40         std::cout << m_myId
41                   << '\t' << mf
42                   << std::endl;
43     }
44     int m_myId;
45     inline static int m_nextId { };
46 };
47
48 class NoDynamicResource final
49 {
50 private:
51     Resource m_resource;
52 };

```

The Resource class defines three constructors, two assignment operators and one destructor in its public section. First, the private members of Resource are examined. The static class variable `m_nextId` is defined inline and thus initialized to 0 with an empty initialization list. `m_myId` is a data member of each Resource exemplar. Both data members are of type `int`. `Resource::report()` is a private `const` member function that outputs `m_myId` followed by a tab and a message passed as a reference to `const` of type `std::string` to `report()`. If desired, the program output can be redirected to a file to provide a simple CSV file for tracking.

Now to the constructors. The standard constructor initializes `m_myId` with the pre-incremented value of `m_nextId`. In this way, each exemplar of Resource is created with a unique value of `Resource::m_myId`. In its body, the standard constructor calls the `report()` member function with the message "standard constructor" to indicate that it is being called.

Next, the copy constructor is defined as shown several times before. Since the copy initialization also creates a new object, `m_my_Id` is initialized with the pre-incremented value of `m_nextId` and the `report()` method is called to inform about the execution of the copy constructor.

The next constructor is a very special one that has not been presented yet. It is a so-called *move constructor*, which is called for a so-called *rvalue* (synonym *R-value*) which means *right value*. An rvalue can only be used on the right side of an assignment, in contrast to an *lvalue* (synonym *L-value*), which can be used on both sides of an assignment, hence the name lvalue which means *left value* (including the possibility to be used as rvalue as well).

A temporary object is an rvalue. It is assumed to be deleted immediately after the current operation is complete. `Resource r { Resource { } };` is an example of calling the move constructor. The `Resource { }` exemplar inside the curly brackets is destroyed immediately after the creation of `r` is completed by calling the move constructor. Knowing this, the compiler simply replaces `r` with the temporary `Resource { }` object. This can be imagined as *moving* the temporary object to the destination object, hence the name *move constructor*. The argument object must be passed as a so-called *rvalue reference*, denoted by `&&`. An rvalue reference should not be specified as `const`, as explained below.

Then a virtual destructor is defined, which informs about its execution. Since the destructor is virtual, `Resource` is a valid base class.

The assignment operator is overloaded the first time. The correct specification of the assignment operator passes the object to be assigned as a reference to `const`. This prevents the creation of a local copy that must be subsequently destroyed. It returns the receiver object, i.e. the left operand of the assignment, as a reference. This is necessary to allow chained assignments, such as `a = b = c;`. The assignment operator is *right-associative*, i.e. it is evaluated from right to left.

If a variable is `const`, the attempt to assign another variable to it must fail. Therefore, assignment is valid only for variables that are not `const`. For this reason, the result of an assignment must be a reference. Returning a reference to `const` as the result of an assignment would also prevent chained assignments. The implementation of the assignment operator informs about its execution. It then returns `*this`, which is the object for which the assignment operator was called.

Finally, the operator for *move assignment* is specified. Instead of a reference to `const` of type `Resource` it takes an rvalue reference. In its implementation it informs that it is called. It also returns `*this` as result. Move assignment is applied when a temporary object is assigned, e.g. `o = Class { };` as in the `test()` function.

From the syntactic point of view, an rvalue reference can be specified as `const`. Since the rvalue reference is normally changed by moving one of its resources to another object, it is semantically nonsensical to specify an rvalue reference as `const`. This would prevent the `const` object from being changed.

Together with the *move assignment*, the move constructor belongs to the so-called *move semantics*, which deals with moving objects whose lifetime expires instead of copying them. This means that data is transferred from the expiring object to the other object.

For now, it must suffice that C++ has the concepts of rvalues, move constructors, and move assignments. It is sufficient to know when these special member functions are called. Later, it will be demonstrated how to correctly implement a move constructor and a move assignment.

The `NoDynamicResource` class is simple. It has only one private data member, namely `m_resource` of type `Resource`. It does not define a destructor and the automatically generated destructor has static binding. Therefore, `NoDynamicResource` is not a valid base class and is specified as `final`.

With the code in Listing 40 and the class definitions in Listing 41, the program *Resources.cpp* can be compiled and executed. Figure 17 displays the resulting output.

Figure 17: Output of *Resources.cpp* for using Class = `NoDynamicResource`;

```

1      standard constructor
2      copy constructor
1      assignment
3      standard constructor
4      standard constructor
3      move assignment
4      destructor
5      move constructor
3      destructor
5      destructor
2      destructor
1      destructor

```

Now the output and the executed statements in `main()` and `test()` are set in relation to each other.

1. `Class a;` (line 166 in Listing 40) indirectly creates the `Resource` object with ID 1 as a member of `a` by calling its standard constructor.
2. `Class b { a };` (line 167 in Listing 40) indirectly creates `Resource` object 2 by calling its copy constructor.
3. `a = b;` (line 168 in Listing 40) also assigns `b.m_resource` (ID 2) to `a.m_resource` (ID 1).
4. Line 169, `auto c { test(Class { }) };`, is more difficult to analyze. The innermost part `Class { }` indirectly creates the temporary `Resource` object with ID 3 by calling its standard constructor. The aggregating temporary object is not copied again, but used directly as a local variable `o` in `test()`.

5. Next, the indirect creation of the Resource object with ID 4 is reported by calling its standard constructor. This corresponds to `Class { }` in line 160 of Listing 40. Then, a move assignment is reported that indirectly assigns Resource object 4 (the ID is inferred) to the Resource object 3. Then the Resource object 4, i.e. the temporary object created indirectly in line 160, is then destroyed by calling its destructor.
6. For `return o;` (line 161 in Listing 40), indirect execution of the move constructor for Resource object 5 with Resource object 3 (inferred) is reported.
7. In line 162 of Listing 40, the `test()` function terminates and destroys the local `Class` exemplar `o` including its member Resource object 3, which is reported accordingly.
8. The `Class` object returned by `test()` is used directly as `c` in line 169 in Listing 40. Therefore, nothing is reported here.
9. In line 170 (Listing 40), the `main()` function is terminated. The `Class` object with the Resource member 5 was created last and is therefore deleted first. After that, objects `a` and `b` are deleted in the reverse order of their creation. Therefore, the destructor calls to the member objects with IDs 5, 2, and 1 are reported in this order.

There may be a legitimate complaint that the output of the *Resource.cpp* program could be improved. Indeed, it would be beneficial to additionally specify the ID of the second object involved in some operations, e.g. *2 copy constructor with 1*. Doing this is strongly recommended and left to the reader as an exercise. A lot of new and complex knowledge has been introduced in this section. In order to understand and learn, the reader should actively work through everything that has been presented in this section so far. Making the Resource class more conversational is an excellent way to do this.

1.14.4. Object With Dynamic Resource—The Wrong Way

Next, a class with a dynamic resource that is used in the wrong way is presented.

Listing 42 shows the `Wrong` class, which has a dynamic resource. It defines a pointer to Resource named `m_dynamic` as a private data member.

Listing 42: Wrong of objects/Resource.cpp (excerpt)

```
54 | class Wrong final
55 | {
56 |     public:
57 |         Wrong():m_dynamic { new Resource }
58 |         {}
59 |         ~Wrong()
60 |         {
61 |             delete m_dynamic;
62 |             m_dynamic = nullptr;
63 |         }
64 |     private:
65 |         Resource* m_dynamic;
66 | };
```

Wrong is specified as a final class, that is, it cannot be used as a base class. Consistently, the destructor is not declared as virtual. In its destructor, it releases the dynamic object by calling the delete operator for the `m_dynamic` pointer. Then `nullptr` is assigned to `m_dynamic` to set the pointer to a valid state and to indicate that it is no longer pointing to an object. This might be considered superfluous, but it is a good practice and should never be ignored.

Since the class Wrong is now introduced, the using declaration in the *Resource.cpp* program is changed to `using Class = Wrong;`. The resulting program compiles correctly, but should crash when executed. It is reported that Resource objects with IDs 1, 2, and 3 are created (indirectly) by the standard constructor, then Resource object 3 is destroyed (indirectly), and afterwards the program should terminate unexpectedly. Note, that the output is only due to Resource exemplars, not to Wrong exemplars. Therefore, the exact reason for the crash is not apparent.

To narrow down the cause of the crash, the last statement in `main()`, `auto c { test(Class { }) };`, is converted to a comment. Compiling and executing this version of the program still crashes. Interestingly, the call of its constructor and destructor is only reported for the Resource object with ID 1.

Next, `a = b;` is also converted to a comment. Still, the program compiles and crashes on execution, reporting that Resource object 1 was created and destroyed.

This changes when `Class b { a };` is also converted into a comment. Now the program is compiled and executed without errors. The Resource object 1 is (indirectly) created and destroyed.

How can all this be explained?

The Wrong class defines only a standard constructor and a destructor. If necessary, the compiler automatically generates a copy constructor and an assignment operator. The statement `Class b { a };` (Class is an alias for Wrong) forces the compiler to generate a copy constructor because a user-defined copy constructor is not available. Since the compiler is principally unaware of the dynamic resource, it treats `Wrong::m_dynamic` as an ordinary data member that is simply copied. Thereafter, `a.m_dynamic` and `b.m_dynamic` point to the same dynamically allocated Resource exemplar.

Figure 18a) shows the state of the program after the execution of `Class a;`. Figure 18b) shows the state of the program after the execution of `Class b { a };`. When `main()` terminates, `b` is destroyed and `Wrong::~Wrong()` is called. The destructor releases the Resource exemplar pointed to by `b.m_dynamic` and assigns `nullptr` to `b.m_dynamic` as depicted in Figure 18c). Eventually, `a` is also destroyed. But `a` is unaware that the Resource object pointed to by `a.m_dynamic` was previously freed. And `a` does not know that `b.m_dynamic` has been assigned `nullptr`. As a result, `a`'s destructor releases the Resource object that `a.m_dynamic` is still pointing to a second time. This causes the program to

crash. However, there is no guarantee that the program will crash. The program may not crash at all, or it may crash later in its execution. After the dynamic object is released again, the program is in an undefined state.

The reason for the malfunction of the program is that the compiler does not know about the dynamic resource of a `Wrong` object and therefore automatically generates a copy constructor which is not appropriate.

The same is true for the automatically generated assignment operator. Suppose there are two `Wrong` exemplars, both created by calling the standard constructor, that is, `Wrong w1, w2;`. This does not correspond to the situation in the *Resource.cpp* program, where the `m_dynamic` pointers of `a` and `b` are already messed up after copy creating `b`. Now, each `Wrong` object points to its own dynamic resource, as depicted in Figure 19a). When `w2` is assigned to `w1`, the assignment operator simply copies `w2.m_dynamic` to `w1.m_dynamic`. Now both `w1.m_dynamic` and `w2.m_dynamic` point to the `Resource` object originally created by the constructor of `w2`. The `Resource` object originally created by the constructor of `w1` is no longer accessible and thus cannot be released. In this way, it becomes an *orphaned object* (see (U. W. Eisenecker 2022), Section 3.2.4 *Dynamic Objects*). This situation is illustrated in Figure 19b). Next, `w2` is destroyed. As a result, the `Resource` exemplar pointed to by `w2.m_dynamic` is released, as shown in Figure 19c). Next, `w1` is destroyed, which in turn releases the object already released by `w2`. Ideally, this will crash the program.

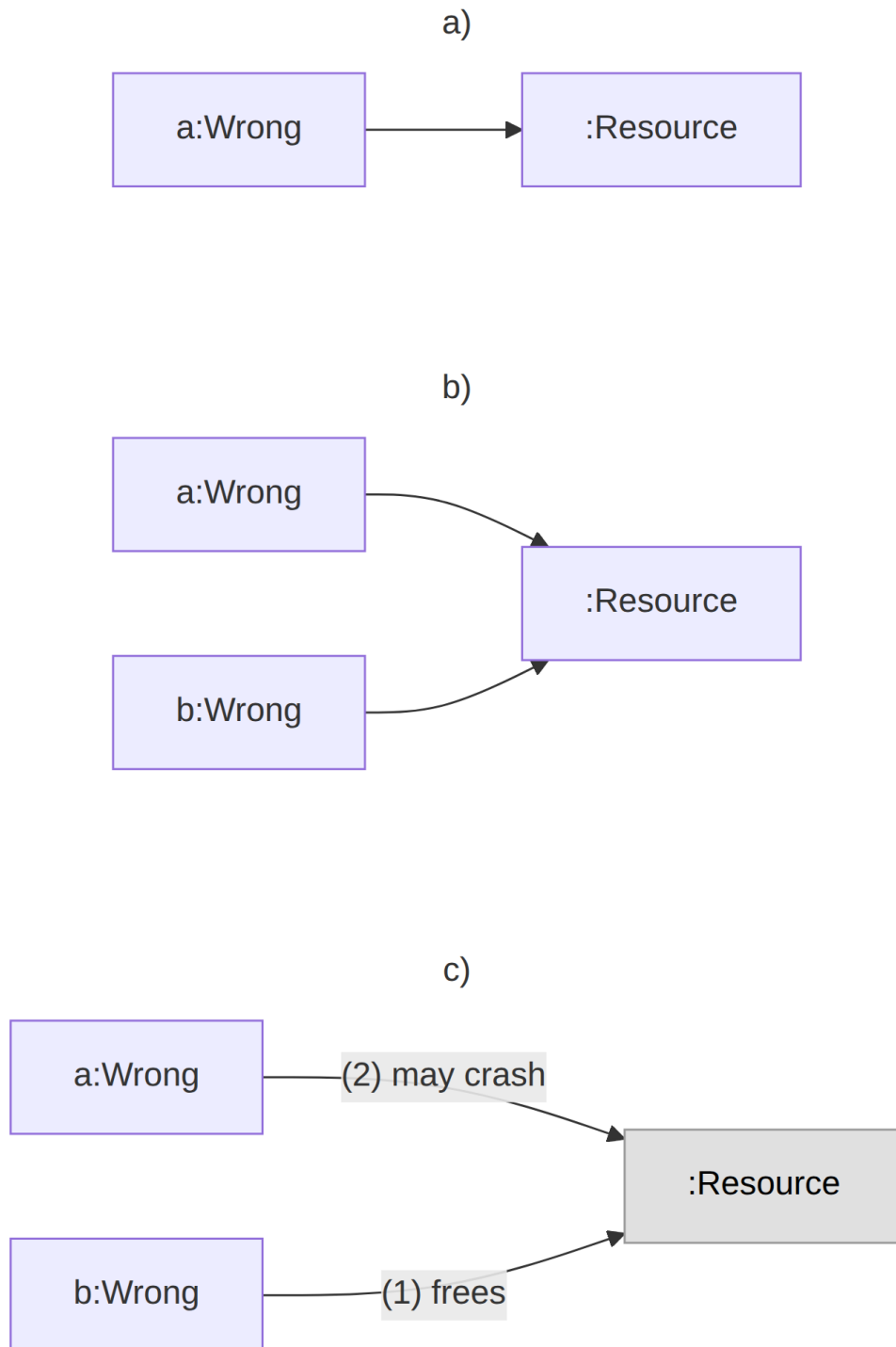
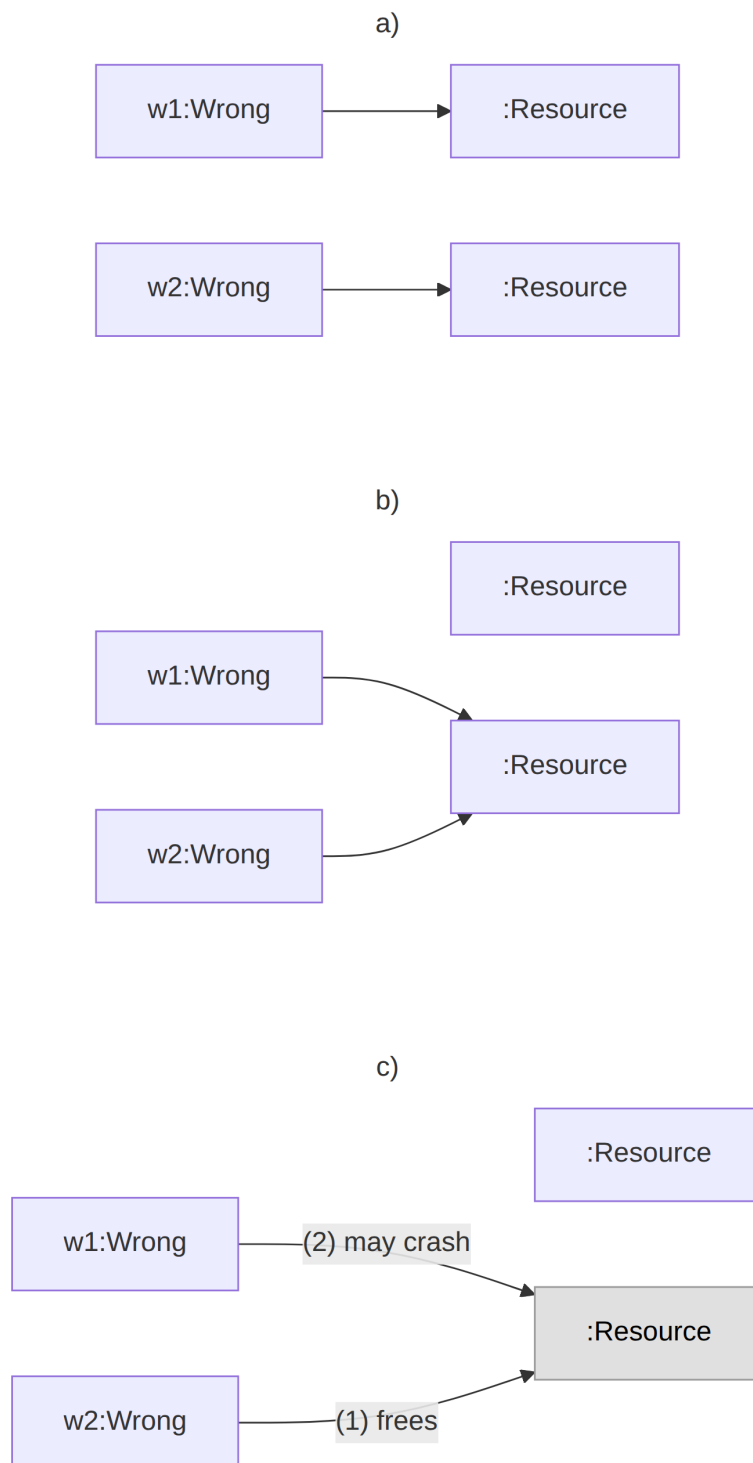
Figure 18: Automatically generated copy constructor and dynamic resource

Figure 19: Automatically generated assignment operator and dynamic resource

1.14.5. Big3—The Better Way

A class with a dynamic resource that is managed correctly is presented below.

The Wrong class shows that the automatically generated copy constructor and assignment operator are not sufficient for an object with a dynamic resource. In addition to the Wrong class, Big3 (Listing 43) provides a user-defined copy constructor and assignment operator. The name of the class is derived from the so-called *rule of three*. This rule states that if one of the special member functions *copy constructor*, *assignment operator*, or *destructor* has a user-defined implementation, it probably also requires a manual implementation of the remaining special member functions.

Listing 43: Big3 of objects/Resource.cpp (excerpt)

```

68 class Big3 final
69 {
70     public:
71         Big3():m_dynamic { new Resource }
72         {}
73         Big3(const Big3& big3)
74             :m_dynamic { new Resource { *big3.m_dynamic } }
75         {}
76         Big3& operator=(const Big3& big3)
77         {
78             if (this != &big3)
79             {
80                 Resource* newResource { new Resource { *big3.m_dynamic } };
81                 delete m_dynamic;
82                 m_dynamic = newResource;
83             }
84             return *this;
85         }
86         ~Big3()
87         {
88             delete m_dynamic;
89             m_dynamic = nullptr;
90         }
91     private:
92         Resource* m_dynamic;
93 };

```

In its initializer list, the Big3 copy constructor creates a new dynamic Resource object by using its copy constructor, that is, `new Resource { *big3.m_dynamic }`. The member access operator in `*big3.m_dynamic` has the higher priority. Therefore, evaluating `big3.m_dynamic` gives the pointer to the dynamic Resource object managed by `big3`. Then, this pointer is dereferenced by applying the dereference operator `*` to it. The newly created Resource object has the same state as the Resource object pointed to by `big3.m_dynamic`, but the copied Big3 object and the Big3 object under creation both manage their own dynamic Resource object.

The implementation of the `Big3` assignment operator is a bit more sophisticated. First, it checks whether the object is assigned to itself. The assignment of an object to itself seems to be nonsensical. However, when `a` is a reference for `b`, the statement `a = b;` is a self-assignment. Moreover, a self-assignment is a syntactically and semantically valid part of a program. For this reason, self-allocation must be taken into account. To avoid unnecessary operations in the case of a self-assignment, the if statement checks whether `this` and the address of `big3`, i.e. `&big3`, are different. If this is not the case, none of the code implementing the actual assignment is executed, and the assignment operator returns the receiver object with `return *this;`. Otherwise, the first action is to acquire a new dynamic resource. Here, this is done by calling the `new` operator for `Resource` and calling the `Resource` copy constructor, namely `new Resource { *big3.m_dynamic }`. This is exactly the same as creating a dynamic `Resource` object in the initializer list of the `Big3` copy constructor. The pointer to the new dynamic object is used to initialize the local `newResource` pointer. When this statement is successfully executed, the dynamic object currently pointed to by `m_dynamic` is released. Immediately afterwards, `newResource` is assigned to `m_dynamic`. Now, `this->m_dynamic` points to the correct dynamic `Resource` object.

`Big3` is also specified as `final` class. Accordingly, its destructor is not declared as `virtual`.

After changing the using declaration to `using Class = Big3;` the *Resource.cpp* program is compiled and executed without errors. Figure 20 shows the output of the execution in a terminal window.

Figure 20: Output of *Resources.cpp* for using `Class = Big3;`

```
1      standard constructor
2      copy constructor
3      copy constructor
1      destructor
4      standard constructor
5      standard constructor
6      copy constructor
4      destructor
5      destructor
7      copy constructor
6      destructor
7      destructor
2      destructor
3      destructor
```

Compared to Figure 17, two more objects are created and deleted, seven versus five. Again, it should be emphasized that the output is caused by calling special member functions of `Resource`, but not directly of `Big3`. Next, the code of `main()` and `test()` is compared to the output shown in Figure 20.

1. The statement `Class a;` (line 166 in Listing 40) indirectly calls the `Resource` standard constructor, which creates `Resource` object 1.
2. `Class b { a };` (line 167 in Listing 40) calls the copy constructor of `Big3`. The copy constructor of `Big3` calls the copy constructor of `Resource`, thus creating `Resource` object 2.
3. The statement `a = b;` (line 168 in Listing 40) calls the assignment operator of `Big3`. `Big3::operator=()` does not call `Resource`'s assignment operator. Instead, a new dynamic `Resource` object 3 is created by calling its copy constructor. Then the `Resource` object with ID 1 is released.
4. In line 169 in Listing 40, `auto c { test(Class { } ) };`, the first evaluated expression is `Class { }`. This indirectly creates a new `Resource` object with ID 4 by calling its standard constructor. The temporary object being created with `Class { }` is actually a `Big3` exemplar and is passed as a value to the `test()` function and used directly as a local variable in `test()`. Therefore, nothing is reported here.
5. The first statement of `test()`, `o = Class { };` (line 160 in Listing 40), first creates a temporary `Big3` exemplar by calling its standard constructor, which in turn calls the standard constructor of `Resource` exemplar 5. Then, the temporary `Big3` exemplar is assigned to `o`. The `Big3` assignment operator first creates a new dynamic `Resource` object with ID 6 by calling its copy constructor. Then the object with ID 4 (pointed to by `o.m_dynamic`) is destroyed. Next, the temporary object `Class { }` is destroyed, implicitly destroying the `Resource` object with ID 5.
6. The last statement of `test()` is `return o;` (line 161 in Listing 40). Since `test()` returns a new `Class` exemplar, the `Big3` copy constructor is called, which in turn copy initializes the new, dynamically created `Resource` object with ID 7. On return, the local `Big3` variable `o` is destroyed, which indirectly releases `Resource` object 6, as reported.
7. Again, the compiler uses the `Big3` object returned by `test()` as object `c`. Therefore, `c` is the owner of the `Resource` object with ID 7. Otherwise, an additional call to the copy constructor of a `Resource` object would have to be reported. In addition, the temporary `Class { }` object (actually of type `Big3`) that served as parameter for the call to `test()` is destroyed. This implicitly destroys `Resource` object 6 as well.
8. Now, `main()` has been terminated. The last object created is `c`. Therefore, it is destroyed first, as the destructor call for `Resource` object 7 shows. Then `b` and `a` are destroyed, as the destructor calls for `Resource` objects 2 and 3 suggest. While `b` still owns `Resource` object 2, `a` was previously assigned to `b`, resulting in the original `Resource` object 1 owned by `a` being destroyed and replaced by the newly created `Resource` object with ID 3.

Analyzing and understanding the program behavior and the interaction between objects and their dynamic resources is challenging. Therefore, the reader should take a closer look at the *Resources.cpp* program. One suggestion is to add additional output messages that clarify at which point of execution the program is currently performing certain operations. For this purpose, the `Logger` class introduced in (U. W. Eisenecker 2022)

could be useful (Section 6.5.2 *Logger Class*). Another possibility is to debug the program. In this case, lines such as `o = Class { };` and `auto c { test(Class { }) };` can be a hurdle. They combine many operations into a single line. While multiple outputs appear in the console window, the active debugging line remains the same. This can be changed by splitting such statements across multiple lines, as shown in Listing 44.

Listing 44: Splitting complex statements over multiple lines

```
1 | o =  
2 | Class { }  
3 | ;  
4 | // ...  
5 | auto c {  
6 | test(  
7 | Class { }  
8 | )  
9 | }  
10| ;
```

1.14.6. Big5—The Ambitious Way

For a class that manages a dynamic resource, performance optimizations can be implemented thanks to the *move semantics*. Move semantics and their application in the context of classes with dynamic resources are presented below.

The move constructor and the move assignment operator are an essential part of the so-called *move semantics* in C++. The move semantics supports a very efficient construction or assignment of objects in combination with rvalue objects.

Manually providing or defaulting a standard constructor, destructor, or assignment operator affects the automatic generation of member functions related to move semantics. If a standard constructor is manually implemented or defaulted, a move constructor is no longer generated. This applies similarly to the assignment operator, although the situation is somewhat more complicated due to copy assignment. Once a destructor is available—whether virtual or not, manually implemented or defaulted—none of the special member functions associated with the move semantics are automatically generated.

If a class has a dynamic resource, it must at least implement all elements of the *rule of three* (Big 3). However, this still does not recover any optimizations through the move semantics. To achieve this, a move constructor and a move assignment operator must be defined as additional elements, hence the name *rule of five* (Big 5).

The details of move semantics and their correct implementation are not the subject of this book. A comprehensive introduction to this topic is (N. Josuttis 2020).

The Big5 class (Listing 45) shows how to correctly implement a move constructor and a move assignment operator.

Listing 45: Big5 of objects/Resource.cpp (excerpt)

```

95 class Big5 final
96 {
97     public:
98         Big5():m_dynamic { new Resource }
99         {}
100        Big5(const Big5& big5)
101            :m_dynamic { new Resource { *big5.m_dynamic } }
102        {}
103        Big5(Big5&& big5)
104            :m_dynamic { big5.m_dynamic }
105        {
106            big5.m_dynamic = nullptr;
107            cout << "Big5 move constructor" << endl;
108        }
109        Big5& operator=(const Big5& big5)
110        {
111            if (this != &big5)
112            {
113                Resource* newResource { new Resource { *big5.m_dynamic } };
114                delete m_dynamic;
115                m_dynamic = newResource;
116            }
117            return *this;
118        }
119        Big5& operator=(Big5&& big5)
120        {
121            cout << "Big5 move assignment" << endl;
122            if (this != &big5)
123            {
124                delete m_dynamic;
125                m_dynamic = big5.m_dynamic;
126                big5.m_dynamic = nullptr;
127            }
128            return *this;
129        }
130        ~Big5()
131        {
132            delete m_dynamic;
133            m_dynamic = nullptr;
134        }
135        private:
136            Resource* m_dynamic;
137    };

```

The elements of the rule of three have already been sufficiently introduced in the Section *Big3—The Better Way*. Therefore, they will not be discussed for the Big5 class.

To understand what the move constructor and move assignment are good for, a situation must be constructed that forces the compiler to apply optimization based on the move semantics. Simply `Big5 o { Big5 { } };` is not enough. Obviously, copy initialization

is used to initialize `o` with a temporary `Big5` exemplar. However, the compiler recognizes this and inserts the temporary `Big5` exemplar directly in place of `o`, so it is no longer a temporary exemplar. Even writing funny statements like `Big5 o { Big5 { Big5 { } } }`; are perfectly recognized by the compiler and result in the creation of only one `Big5` exemplar that becomes `o`.

To achieve this, a function is needed that takes a `Big5` parameter passed as a value and is returned as a value, that is,

```
Big5 callByValue(Big5 b) {    return b; }.
```

Calling `callByValue()` and passing the `Big5 o` exemplar forces the use of the move constructor. First, a local `Big5` copy is created by copy initializing with `o` before calling the `callByValue()` function. When `b` is returned, the compiler knows that `b` will be destroyed immediately. This means that `b` no longer needs its dynamic resource `b.m_dynamic`. If no move constructor were available, `Big5`'s copy constructor would now be called. This would mean to copy initialize `m_dynamic` of the object to be returned and then destroying the dynamic Resource object pointed to by `b.m_dynamic`. Both are potentially expensive operations.

This is where the move constructor comes into play. Since `b` no longer needs its dynamic resource, it is taken away from `b` (usually called *stolen*) and given to the object to be created. Nevertheless, the non-dynamic members of `b` and `b` itself must be destroyed when the function terminates. Therefore, it is imperative to assign `nullptr` to `b.m_dynamic` after copying its value. This allows a safe delete in the `Big5` destructor, since a delete on `nullptr` is always harmless, as mentioned in (U. W. Eisenecker 2022), Section 3.2.4 *Dynamic Objects*. Taking the dynamic resource away from an object means *changing* that object. Therefore, this object must not be `const`.

It is simpler to force the call of the move assignment operator. If `o` is a `Big5` object, the statement `o = Big5 { };` calls the move assignment operator. Here, `Big5 { }` is a temporary object whose life will end soon, at the latest with the semicolon that terminates the statement. For this reason, it is safe to steal its dynamic resource. If the temporary object and the receiver object are not identical, the `m_dynamic` pointer of the right operand is copied to the `m_dynamic` pointer of the left operand of the assignment. Then, `nullptr` is assigned to `m_dynamic` of the right operand so that the right operand can be safely destroyed.

With this knowledge, the two special member functions of the move semantics can be explained.

In the move constructor the double ampersand `&&` specifies the parameter `big5` as rvalue reference. Since `big5` must be modified in the move constructor, it must not be passed as a `const` parameter. In the initializer list, `m_dynamic` is copy initialized with `big5.m_dynamic`, i.e. only the value of the pointer is copied, but not the object to which `big5.m_`

dynamic points. The constructor body assigns `nullptr` to `big5.m_dynamic` so that `delete` can be safely applied to `big5.m_dynamic` in its destructor. Also, a message is sent to `cout` informing about the execution of the move constructor.

The move assignment operator specifies the right operand of the assignment as an rvalue reference. Its implementation will modify `big5`, thus it must not be passed as `const`. First, a message is sent to `cout` informing that the move assignment operator is executed. Then it checks if `this` and `&big5` are different, which means, that they are both different objects. One could argue that this is redundant, because `big5` is a temporary object and will soon be destroyed, and the receiver object, i.e. the left operand of the assignment operator, must be a different object with a longer lifetime. However, it cannot be guaranteed that this is really true. Therefore, checking for different identities is appropriate. Then, the dynamic resource is taken away from `big5` with the statement `m_dynamic = big5.m_dynamic;`. After that, `big5.m_dynamic` is assigned `nullptr` to allow safe destruction of `big5`. Finally, `*this` returns the active object as a reference.

`Big5` is specified as a `final` class. Consequently, its destructor is not declared as `virtual`.

Changing the using declaration to `using Class = Big5;` and compiling and executing the program, it becomes apparent that the operations related to the move semantics are enabled again. A total of only 5 objects are now created, as the terminal window output shows (Figure 21).

Figure 21: Output of `Resources.cpp` for using `Class = Big5;`

```

1      standard constructor
2      copy constructor
3      copy constructor
1      destructor
4      standard constructor
5      standard constructor
Big5 move assignment
Big5 move constructor
5      destructor
2      destructor
3      destructor

```

A detailed comparison of program execution and output is not given here. The application of the move assignment and the move constructor is triggered exclusively by the function `test()`. The statement `o = Class { };` causes a move assignment to be generated, the statement `return o;` activates the move constructor.

1.14.7. Big5Deleted—A Convenient Alternative

Providing all standard member functions for a class with a dynamic resource sometimes causes either too much effort or is semantically difficult. In these cases, it is possible to simply delete some of these member functions and still get a useful class implementation.

The `Big5Deleted` class (Listing 46) shows a convenient alternative for dealing with an object that has a dynamic resource, namely deleting four of the five elements of the rule of five. The destructor cannot be deleted, otherwise an exemplar of `Big5Deleted` could not be destroyed. But the destructor is not `virtual` and `Big5Deleted` is specified as a `final` class. The consequences of deleting the copy constructor, the move constructor, the assignment operator, and the move assignment operator can be easily studied by changing the using declaration to `using Class = Big5Deleted;`. When compiling the program, a number of errors should be issued. They all state that the special member functions mentioned above are not available because they have been (explicitly) deleted.

Listing 46: `Big5Deleted` of `objects/Resource.cpp` (excerpt)

```
139 | class Big5Deleted
140 | {
141 |     public:
142 |         Big5Deleted():m_dynamic { new Resource }
143 |         {}
144 |         virtual ~Big5Deleted()
145 |         {
146 |             delete m_dynamic;
147 |         }
148 |         Big5Deleted(const Big5Deleted&) = delete;
149 |         Big5Deleted(Big5Deleted&&) = delete;
150 |         Big5Deleted& operator=(const Big5Deleted&) = delete;
151 |         Big5Deleted& operator=(Big5Deleted&&) = delete;
152 |     private:
153 |         Resource* m_dynamic;
154 | };
```

Actually, this is not the worst situation. Of course, the program won't compile, but that is because it uses potentially critical member functions of objects with dynamic resources. The alternative, enforced by `Big5Deleted`, is *to not use* these possibly dangerous operations. It is still possible to develop programs with classes that have these limitations. Moreover, even some classes of the C++ standard library are designed this way (Section *Prohibited Copying and Assigning*). Otherwise, all elements of the rule of three or the rule of five would have to be implemented correctly, and these implementations would also have to be thoroughly tested.

Interestingly, `Big5Deleted` can be used as a base class. It is not specified as `final` and it defines a virtual destructor. All other classes, including `Resource`, can cause problems if the static type and dynamic type of variables used in assignments are different. This problem and possible solutions are discussed in the *`dynamic_cast<>()`* Section.

Another issue not presented here is that an object should have either no dynamic resource or at most one. Objects with two or more dynamic resources should be strictly avoided. This problem is addressed in Section *Exceptions*.

1.15. Multiple Inheritance

Some object-oriented programming languages support only *single inheritance*, i.e. a class can have at most one base class. In C++, a class can have any number of base classes, i.e. none, one, or more than one. The latter is called *multiple inheritance*. The Section *Order of Calling Constructors and Destructors* gives some background about the construction and destruction of objects, which is also relevant for multiple inheritance. For a deeper understanding, the idea of a class with inherited and aggregated subobjects is presented below.

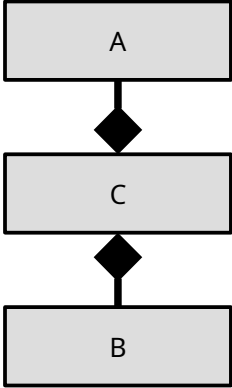
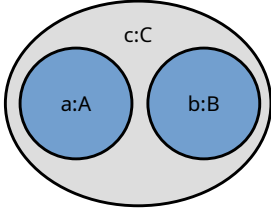
1.15.1. Subobjects

For a proper understanding of inheritance and aggregation, the concept of so-called *subobjects* is introduced. It provides insight into the construction, deletion and copying of objects.

Class C defines `a` as data member of class A and `b` as data member of class B. Then `a` and `b` can be considered as *aggregated subobjects* of a C object. Table 3 contains different representations for this situation. The first column shows a *UML* class diagram. Here C aggregates A and B. The second column shows rudimentary class definitions with classes A and B and class C containing data members `a` and `b` of types A and B, respectively. The third column shows a *Venn* diagram used in set theory to visualize relationships between sets and subsets. Here, the Venn diagram represents the containment relationships between objects and subobjects. `c:C` is the object named `c` of class C that contains `a:A` and `b:B` as subobjects, The textual notation corresponds to the *UML* object diagrams. Here, the name of an object is to the left of the colon, and its class is to the right of the colon. If there is no name to the left of the colon, it is an anonymous object. A blue area represents an aggregated subobject. The right column indicates the order of construction with respect to the class names. The tilde symbolizes a destructor call.

Table 4 shows four other classes. F specifies D and E as public base classes, and G specifies F as public base class. F has D and E as inherited subobjects, and G has F as an inherited subobject. The C++ code fragment shows rudimentary definitions for all classes involved. When `g` is created as an object of class G, the Venn diagram contains `g:G` as the outermost object that has `f:F` as inherited subobject. `f:F` in turn has `d:D` and `e:E` as inherited subobjects.

Table 3: Aggregated subobjects

UML class diagram	C++ code	Venn diagram	Construction/destruction
	<pre> class A {}; class B {}; class C { A a; B b; }; // C c; </pre>		<pre> ABC ~C~B~A </pre>

Red areas represent inherited subobjects. Objects are created from top to bottom with respect to the inheritance hierarchy. If a class specifies more than one base class, the inherited subobjects are created in the order in which they are specified as bases. A constructor call of a derived class is placed on a new line unless there are two or more base classes. Their constructor calls are placed side by side in one line. As usual, the destructors are called in the reverse order of the constructors.

Table 4: Inherited subobjects

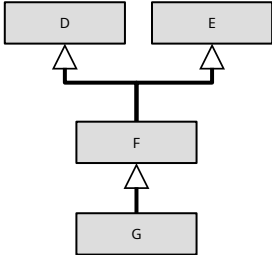
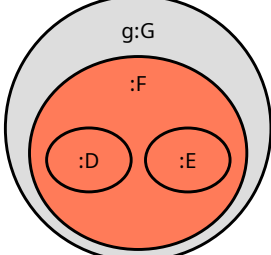
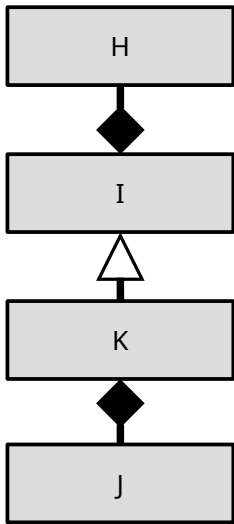
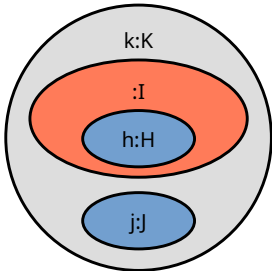
UML class diagram	C++ code	Venn diagram	Construction/destruction
	<pre> class D {}; class E {}; class F: public D, public E {}; class G: public F {}; // ... G g; </pre>		<pre> DE F G ~G ~F ~E~D </pre>

Table 5 shows a combination of aggregated and inherited subobjects. Class H has a data member named *i* of class I. K is a public subclass of H and has a data member *j* of class J.

Table 5: Inherited and aggregated subobjects

UML class diagram	C++ code	Venn diagram	Construction/ destruction
	<pre> class H { }; class I { H h; }; class J { }; class K: public I { J j; // ... K k; </pre>		HI JK $\sim K \sim J$ $\sim I \sim H$

When an object is initialized, its inherited subobjects are initialized first from top to bottom with respect to the inheritance hierarchy. After initializing all inherited subobjects of an object, its aggregated subobjects are initialized. Finally, the constructor body of the object to be constructed is executed. The program *objects/Subobjects.cpp* is included only in the accompanying material. It combines the code fragments from Tables 3, 4, and 5 in a somewhat elaborate form.

It is strongly recommended never to write code that depends on the order of object initialization or object destruction.

1.15.2. Losing the Late Binding

It has been explained that virtual functions are dynamically bound, i.e. it is decided at runtime depending on the true type (a.k.a. dynamic type) of an object which overriding function is actually called. However, there are special circumstances under which virtual functions lose their late binding.

When objects are constructed and destroyed, virtual member functions are bound early. The program in Listing 47 demonstrates this.

Listing 47: objects/LateBindingLost.cpp

```
1 // LateBindingLost.cpp by Ulrich Eisenecker, May 11, 2026
2
3 #include <iostream>
4 using namespace std;
5
6 class GrandParent
7 {
8     public:
9         GrandParent()
10         {
11             cout << "GrandParent::GrandParent(): ";
12             virtualFunction();
13         }
14         void nonVirtualFunction() const
15         {
16             cout << "nonVirtualFunction(): ";
17             virtualFunction();
18         }
19         virtual ~GrandParent()
20         {
21             cout << "GrandParent::~~GrandParent(): ";
22             virtualFunction();
23         };
24         virtual void virtualFunction() const
25         {
26             cout << "GrandParent::virtualFunction()" << endl;
27         }
28 };
29
30 class Parent: public GrandParent
31 {
32     public:
33         Parent()
34         {
35             cout << "Parent::Parent(): ";
36             virtualFunction();
37         }
38         void virtualFunction() const override
39         {
40             cout << "Parent::virtualFunction()" << endl;
41         }
42 };
43
44 class Child: public Parent
45 {
46     public:
47         Child()
48         {
49             cout << "Child::Child(): ";
50             virtualFunction();
51         }
```

```

52     void virtualFunction() const override
53     {
54         cout << "Child::virtualFunction()" << endl;
55     }
56 };
57
58
59 int main()
60 {
61     Child c;
62     c.nonVirtualFunction();
63 }

```

The `GrandParentClass` defines the virtual member function `virtualFunction()`. This member function is overridden in its descendant classes `Parent` and `Child`. In its constructor and destructor, `GrandParent` calls `virtualFunction()`. In addition, it defines the `nonVirtualFunction()` member function, which calls the `virtualFunction()` member function. As expected, calling `nonVirtualFunction()` for a `Child` object as in `main()` indirectly calls `Child::virtualFunction()` because of the late binding.

This is not the case when the `Child` object is created. Of course, the `GrandParent` subobject must be initialized first. But the `GrandParent` constructor executes `GrandParent::virtualFunction()` when calling `virtualFunction()` and not `Child::virtualFunction()`. The same is true when the `GrandParent` destructor calls `virtualFunction()`. This also executes `GrandParent::virtualFunction()`, but not `Child::virtualFunction()`. The constructors of `Parent` and `Child` also call `virtualFunction()` in their bodies. But even in this case, always the `virtualFunction()` is executed, which is overwritten in the actual class. That is, in constructors and destructors, virtual member functions lose late binding. Instead, the compiler generates static function calls for them, that is, they are bound early. Compiling and running the program demonstrates this, as the output in Figure 22 shows.

Figure 22: Output of `objects/LateBindingLost.cpp`

```

GrandParent::GrandParent(): GrandParent::virtualFunction()
Parent::Parent(): Parent::virtualFunction()
Child::Child(): Child::virtualFunction()
nonVirtualFunction(): Child::virtualFunction()
GrandParent~GrandParent(): GrandParent::virtualFunction()

```

The first three lines are caused by the creation of a `C` object, which in turn calls the constructors of its base classes. Each constructor calls exactly the `virtualFunction()` defined in its class. When the object is fully constructed, late binding is re-enabled, as demonstrated by the call to `nonVirtualFunction()`. The last line of output shows that late binding is disabled in the `GrandParent` destructor. Otherwise `Child::virtualFunction()` would be called, but this is not the case.

The implementation of constructors and destructors is not completely symmetrical in *LoseLateBinding.cpp*. While each constructor calls `virtualFunction()`, only the `GrandParent` destructor calls `virtualFunction()`. Since `GrandParent::~~GrandParent()` is declared as a virtual destructor, the automatically generated destructors of the descendent classes also have late binding. If desired, destructors calling `virtualFunction()` in the subclasses can be defined manually.

1.15.3. Name Collisions

Name collisions can occur when libraries or frameworks are developed by independent teams. They can be managed through the use of namespaces. Interestingly, the use of multiple inheritance can also lead to name collisions. This problem and its solution are presented below.

If a class has two or more base classes, it may happen that two or more of the bases define a member with exactly the same name. In this case, this member cannot be used in the subclass without further precaution. The compiler would then not know which definition to use in which class. This ambiguity must be resolved by the programmer. To achieve this, the class name followed by the scope operator must precede the name of the member, which is called *full qualification*. Listing 48 shows a corresponding example that uses member functions with the same signature. As demonstrated, the name collision can occur inside a class or from outside a class. In either case, it must be resolved by fully qualifying the otherwise ambiguous identifier.

Listing 48: objects/ResolvingNameCollision.cpp

```
1 // ResolvingNameCollision.cpp by Ulrich Eisenecker, March 7, 2023
2
3 #include <iostream>
4
5 using namespace std;
6
7 class FirstBase
8 {
9     public:
10         virtual ~FirstBase() = default;
11         virtual void ambiguous() const
12         {
13             cout << "FirstBase::ambiguous()" << endl;
14         }
15 };
16
17 class SecondBase
18 {
19     public:
20         virtual ~SecondBase() = default;
21         virtual void ambiguous() const
22         {
23             cout << "SecondBase::ambiguous()" << endl;
```

```

24     }
25 };
26
27 class Derived: public FirstBase, public SecondBase
28 {
29     public:
30         void resolved() const
31         {
32             // ambiguous(); // Ambiguous --> compilation error
33             FirstBase::ambiguous(); // Not ambiguous
34             SecondBase::ambiguous(); // Not ambiguous
35         }
36 };
37
38 int main()
39 {
40     Derived d;
41     d.resolved();
42     // d.ambiguous(); // Ambiguous --> compilation error
43     d.FirstBase::ambiguous(); // Not ambiguous
44     d.SecondBase::ambiguous(); // Not ambiguous
45 }

```

It is sometimes argued that name collisions are an inherent shortcoming of multiple inheritance. This is wrong. The problem of name collisions exists independently of multiple inheritance. It can also occur when two namespaces define the same identifier. The solution to the resulting name collision—when both namespaces are required—is to fully qualify the otherwise ambiguous identifier. For ambiguous identifiers in different namespaces, this is also done by prepending the namespace name followed by the scope operator to the identifier.

1.15.4. Virtual Base Classes

Inherited subobjects have already been introduced in Section *Subobjects*. With multiple inheritance, it can happen that an object has several subobjects of the same indirect base class. This can be a problem. The origin of this problem and the remedy to avoid it are described in detail below.

Table 4 shows a program fragment with the class *F* having *D* and *E* as public base classes. The code included in Table 6 builds on this fragment and introduces *Z* as common base class for classes *D* and *E*. This inheritance pattern is sometimes called a *diamond* or *rhombus*.

The program *objects/MultipleBaseObjects.cpp* is an elaborated version of the code snippet included in Table 6 that can compile to an executable program. It is only included in the accompanying material.

Table 6: Multiple base objects

UML class diagram	C++ code	Venn diagram	Construction/destruction
<pre> classDiagram Z < -- D Z < -- E D < -- F E < -- F F < -- G </pre>	<pre> class Z {}; class D: public Z {}; class E: public Z {}; class F: public D, public E {}; class G: public F {}; // ... G g; </pre>		ZD ZE F G ~G ~F ~E~Z ~D~Z

When executed, the resulting output is very interesting. Because D inherits from Z, an object or subobject of D must contain an inherited subobject of type Z. The same is true for the class E. This is exactly what is displayed in the terminal window. The construction of a D implies the preceding construction of a Z, and the construction of an E also implies the preceding construction of a Z. Looking at the end of the output, these subobjects are correctly destroyed in the reverse order of their construction. There are no changes in the construction and destruction of F and G with respect to Table 4.

This duplicated Z subobject can cause problems when it is not needed, which is the most common case. The duplicate can waste memory, and if the members of the duplicated subobjects are to be accessed from descendant classes F or G, or if an object of F or G is used from outside, their names must be fully qualified to distinguish between the G subobject of D and the Z subobject of E. This is annoying and usually not wanted.

Virtual base classes eliminate the occurrence of multiple subobjects of the same class. All classes derived from Z must specify Z as a virtual base class. This is done either with class D: public virtual Z or with class E: virtual public Z, that is, whether virtual is placed before or after the access specifier, here public, is of no significance. As a result, an object of any subclass of Z is guaranteed to have only one aggregate subobject of Z.

The question now is: Who is responsible for initializing the Z subobject? Without virtual base classes, the answer is obvious: It is always the constructor of the subclass. When creating an F object, there is no criterion to decide whether D or E is responsible for initializing the Z subobject. The solution is that the object of the *most derived class* is responsible for initializing the Z subobject. When creating an exemplar of F, F is the most derived class, and therefore the F constructors must call a Z constructor. When creating a G-exemplar, G is the most derived class and therefore must initialize Z.

That has a serious consequence: **Any class that can become the most derived class must call a constructor of the virtual base class.** If the most derived class does not explicitly specify the constructor for initializing the virtual base class, the compiler tries to call the standard constructor for the virtual base class. If the standard constructor is not available, the compiler issues an error. Accordingly, the most derived class is also responsible for destroying the inherited subobject of the virtual base class.

For the programmer, it is important to fully understand the consequence of a subobject in motion when a virtual base class is involved. First, there is a problem of pre-planning. How can one know if a particular class will be used as a virtual base at some later time? One extreme would be to specify every base class as a virtual base class. Another extreme would be to forbid multiple inheritance altogether in the coding guidelines, thereby also eliminating the problems of multiple subobjects or subobjects in motion. Somewhere in between is the conscious design of specific inheritance patterns with a beneficial use of multiple inheritance and virtual base classes. Second, when using multiple inheritance and virtual base classes, each descendant class must call an appropriate constructor of the virtual base class. Third, an earlier piece of advice (Section *Subobjects*) needs to be repeated: **Never write code that depends on the order of object initialization or object destruction.**

Figure 23 shows Venn diagrams for creating objects of classes Z, D, E, F, and G as most-derived classes. Special attention should be paid to which most-derived class the Z (sub) object moves to.

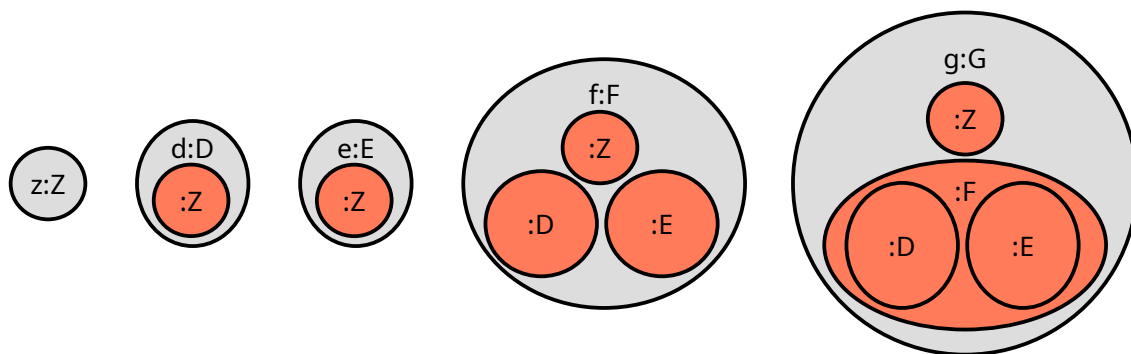


Figure 23: Most-derived classes and virtual base class

A class cannot be specified more than once as a direct base class. Multiple sub-objects can only occur because of their indirect specification as base classes, as presented in Table 6.

1.15.5. Draw Generators

Many sources describe multiple inheritance in C++ as superfluous, if not dangerous. The latter because of possible name collisions, which can be completely resolved as already explained, and because of its complexity. This section will demonstrate that this feared complexity is perfectly acceptable if multiple inheritance enables elegant solutions that directly reflect the structure of a problem.

After the purely abstract and syntactic introduction of multiple inheritance, an application of multiple inheritance in practice is therefore presented below.

Combinatorics is a branch of mathematics that deals with counting (*Combinatorics* 2023). *Permutations* (*Permutation* 2023) and *combinations* (*Combination* 2023) are problems of combinatorics that have many applications in practice. Since it is sometimes difficult to understand exactly what they are, a simplified model based on an urn is presented for them. The problem domain has three main concepts: *Urn*, *Balls*, and *Draw*.

An *urn* can be filled with n balls (n is a natural number). The balls are numbered strictly sequentially, starting with 0, i.e. there may be no duplicate number and no number may be omitted between the first and the last ball. For example, if there are 5 balls, they are numbered 0, 1, 2, 3 and 4.

A *draw*, used here as a general term for combination and permutation, has the size k (k is a natural number) of the balls drawn from the urn. There are two criteria that can vary for a draw. First, a ball may or may not be returned to the urn immediately after being drawn. This is called *drawing with repetition* and *drawing without repetition*. Second, the order in which the balls are drawn may be important or not, that is, the *order is important* or *order is not important*. Both criteria for drawing can be combined, resulting in four different drawing methods.

Sometimes these methods are also called *urn models*. This is interesting in that the urn and balls never change, but the method of drawing does. Table 7 shows the four urn models and the formulas for calculating the number of different possible draws plus the relevant restrictions.

The goal is to implement a class for each urn type that generates each draw for given n and k in *lexicographically ascending order*. *Lexicographical* means here that if all elements of a draw are interpreted as a number, the resulting numbers are strictly monotonically increasing. Functions to calculate the number of possible draws for given urn model with n and k are not to be included.

The urn types will be named as follows:

Table 7: Urn models

	With repetition	Without repetition
Order is important (Permutation)	$n^k, \neg(n = 0 \wedge k = 0) \vee (n > 0)$	$\frac{n!}{(n-k)!}, k \leq n$
Order is not important (Combination)	$\frac{(k+n-1)!}{k!(n-1)!}, n > 0$	$\frac{n!}{(n-k)!k!}, k \leq n$

- UrnOR—urn where the order is important and which contains repetitions.
- UrnO—urn where the order is important and does not include repetitions.
- UrnR—urn with unimportant order and with repetitions.
- Urn—urn with unimportant order and without repetitions.

The chosen names emphasize the properties that are present for a particular urn.

1.15.5.1. Urn With Order And With Repetitions

A complete and precise understanding of the problem is essential for the successful implementation of the draw generators, as this mathematical topic is not entirely easy. For this reason, and as a modification of the previously used methods, a *test-first approach* is followed in the development of the urn classes. First, special cases for UrnOR are considered.

1. $n = 0$ and $k = 0$ are valid values for creating an UrnOR exemplar. For this case there is exactly one permutation which is empty. Immediately after the creation of the urn, this single permutation is available, but it is not possible to access any of its elements since it does not contain any elements ($k = 0$). When the urn is asked for the next draw, it must return false to indicate that there is no next draw. Only valid uses are to be tested.
2. $n = 0$ and $k = 1$ are not valid values for creating an UrnOR exemplar. This also applies to $n = 0$ and values of $k > 1$. Since 0^n always gives 0, there can be no valid draws for such an urn. This case cannot yet be tested adequately.
3. $n = 1$ and $k = 0$ are valid values for creating an UrnOR exemplar. For this case exactly one permutation exists, which is empty. Immediately after creating the urn, this single permutation is available, but it is not possible to access any of its elements since it contains no elements ($k = 0$). When the urn is asked for the next draw, it must return false to indicate that there is no next draw. Only valid uses are to be tested.
4. $n = 1$ and $k = 1$ are valid values for creating an UrnOR exemplar. In this case exactly one permutation with one element ($k = 1$) exists. The value of this element at index position 0 is 0. When the urn is asked for the next draw, it returns false because there is no next draw. Only valid uses are to be tested.

Next, some representative cases for `UrnOR` are discussed.

5. $n = 2$ and $k = 1$ are valid values for creating an `UrnOR` exemplar. In this case, two permutations exist. The first draw contains 0 as the only element, the second draw contains 1.
6. $n = 2$ and $k = 2$ are valid values for creating an `UrnOR` exemplar. The urn results in four valid draws: 0 0, 0 1, 1 0, and 1 1. A further overload of the `std::to_string()` function can be implemented and used to create a valid string representation of the numbers in a draw.

All these cases are implemented in a test case named "`UrnOR`" using *Catch2* v3. The test case "`UrnOR`" includes the above cases in corresponding sections named "1" to "6". All test cases for urn models are contained in the *objects/UrnUnitTests.cpp* file, which is only included in the accompanying material.

Considering the draws of an `UrnOR` with $n = 2$ and $k = 2$, this corresponds to counting in a binary *positional numeral system* (two values) with two digits (two places) starting from 0. In fact, this can be generalized: The draws of any `UrnOR` correspond to counting from 0 in a numeral system with n values and k places.

1.15.5.2. Urn With Order And Without Repetitions

Next, a permutation without repetitions is examined. In preparation, all draws for a `UrnOR` with $n = 3$ and $k = 3$ are shown in Figure 24. Each line containing a draw is crossed out if it is not a valid draw for an `Urn0`, i.e. an urn without repetitions.

The series of draws corresponds to counting from 0 in a numeral system with three values and three places. In total, this results in $3^3 = 27$ permutations with repetitions. After crossing out each draw that contains a duplicated number, six draws remain. This corresponds to $\frac{3!}{0!} = 6$ permutations without repetitions. All valid permutations without repetitions are obtained by generating all permutations with repetitions and eliminating those that contain duplicated elements. Therefore, `Urn0` is based on `UrnOR` with invalid draws discarded internally. To detect an invalid draw, the current draw is checked for duplicates.

The following cases are specific to `Urn0`.

7. $n = 0$ and $k = 0$ are valid values for creating an `UrnOR` exemplar. For this case there is exactly one permutation which is empty. Immediately after the creation of the urn, this single permutation is available, but it is not possible to access any of its elements because it contains none ($k = 0$). When the urn is asked for the next draw, it returns `false` to indicate that there is no next draw. Only valid uses are to be tested.

Figure 24: Permutations for $n = 3$ and $k = 3$ (permutations with crossed-out repetitions)



~~0 0 0~~
~~0 0 1~~
~~0 0 2~~
~~0 1 0~~
~~0 1 1~~
0 1 2
~~0 2 0~~
0 2 1
~~0 2 2~~
1 ~~0 0~~
1 ~~0 1~~
1 ~~0 2~~
1 1 ~~0~~
1 1 1
1 1 ~~2~~
1 2 0
1 ~~2 1~~
1 2 2
2 ~~0 0~~
2 0 1
2 ~~0 2~~
2 1 0
2 1 1
2 1 2
2 2 ~~0~~
2 2 1
2 ~~2 2~~

8. $n = 0$ and $k = 1$ are invalid values for creating an `Urn0` exemplar. This is also true if $k > n$, as there can be no valid draws. Furthermore, for $n = 0$ and $k > 1$, the factorial of a negative number had to be computed in the denominator of the formula for calculating the number of different draws from an `Urn0`. The calculation of the factorial of a negative number is possible, but leads to a result that cannot be used here. Therefore, this case cannot be adequately tested.
9. $n = 1$ and $k = 0$ are valid values for creating an `Urn0` exemplar. In this case exactly one permutation exists, which is empty. Immediately after creating the urn, this single permutation is available, but it is not possible to access any of its elements since it contains none ($k = 0$). When the urn is asked for the next draw, it returns `false` to indicate that there is no next draw. Only valid uses are to be tested.
10. $n = 1$ and $k = 1$ are valid values for creating an `Urn0` exemplar. In this case there is exactly one permutation with one element ($k = 1$). The value of this element at index position 0 is 0. When the urn is asked for the next draw, it returns `false` because there is no next draw. Only valid uses are to be tested.
11. $n = 2$ and $k = 1$ are valid values for creating an `Urn0` exemplar. There are two permutations for this case. The first draw contains 0 as the only element, the second draw 1.
12. $n = 2$ and $k = 2$ are valid values for creating an `Urn0` exemplar. The urn results in two valid draws: 0 1 and 1 0. The `to_string()` function introduced above is used to create a valid string representation of the numbers of a draw.
13. $n = 3$ and $k = 3$ are valid values for creating an `Urn0` exemplar. The urn gives the valid draws shown in Figure 24.

1.15.5.3. Urn Without Order And With Repetitions

An exemplar of the `UrnR` class generates draws that can contain repetitions without the order being important. Figure 25 shows all possible draws for an `UrnOR` with $n = 3$ and $k = 3$. All draws that would change their order if sorted in ascending order are crossed out.

It contains 10 draws that are not crossed out. Evaluating $\frac{120!}{3! \cdot 2!}$ gives the same result. Actually, `UrnR` can be based on `UrnOR`, internally discarding invalid draws. Any draw that would be changed when sorting the draw is invalid. To detect an invalid draw, it is checked if a preceding element is larger than the following element.

The cases specific to `UrnR` are discussed below.

14. Looking at the formula for calculating the number of draws for an `UrnR`, it is obvious that $n = 0$ is problematic, as this would require calculating the factorial of -1, which is not desired here. Therefore, no `UrnR` should be initialized for $n = 0$, regardless of the k value. This case cannot be adequately tested yet.

Figure 25: Combinations for $n = 3$ and $k = 3$ (combinations affected by sorting are crossed out)



```
0 0 0
0 0 1
0 0 2
0 1 0
0 1 1
0 1 2
0 2 0
0 2 1
0 2 2
1 0 0
1 0 1
1 0 2
1 1 0
1 1 1
1 1 2
1 2 0
1 2 1
1 2 2
2 0 0
2 0 1
2 0 2
2 1 0
2 1 1
2 1 2
2 2 0
2 2 1
2 2 2
```


15. $n = 1$ and $k = 0$ are valid values for creating an `UrnR` exemplar. For this case there is exactly one permutation which is empty. Immediately after creating the urn, this single permutation is available, but it is not possible to access any of its elements since it contains none ($k = 0$). When the urn is asked for the next draw, it returns `false` to indicate that there is no next draw. Only valid uses are to be tested.
16. $n = 1$ and $k = 1$ are valid values for creating an `UrnR` exemplar. In this case exactly one permutation with one element ($k = 1$) exists. The value of this element at index position 0 is 0. When the urn is asked for the next draw it returns `false` because there is none. Only valid uses are to be tested.
17. $n = 2$ and $k = 1$ are valid values for creating an `UrnR` exemplar. For this case, two permutations exist. The first draw contains 0 as the only element, the second draw contains 1.
18. $n = 2$ and $k = 2$ are valid values for creating an `UrnR` exemplar. The urn results in three valid draws: 0 0, 0 1, and 1 1. The `to_string()` function introduced above is used to create a valid string representation of the numbers in a draw.
19. $n = 3$ and $k = 3$ are valid values for creating an `UrnR` exemplar. The urn produces the valid draws shown in Figure 25.

1.15.5.4. Urn Without Order And Without Repetitions

The `Urn` class generates draws without the order being important and without repetitions. This suggests combining `UrnR` and `UrnO` to define the `Urn` class. When generating the next draw, draws that are modified by sorting and draws that contain duplicates are discarded.

A representation as in Figure 24 or Figure 25 would be disappointing, because an `Urn` with $n = 3$ and $k = 3$ generates only one draw: 0 1 2. Each `Urn` with n equal to k can produce only one draw, because it is impossible to arrange the balls in any other order, and all balls are taken from the `Urn`. Therefore, only `Urns` with $k < n$ can generate more than one draw.

But first, the cases for `Urn` are presented.

20. $n = 0$ and $k = 0$ are valid values for creating an `Urn` exemplar. For this case, exactly one permutation exists, which is empty. Immediately after the urn is created, this single permutation is available, but it is not possible to access any of its elements since it contains no elements ($k = 0$). When the urn is asked for the next draw, it returns `false` as there is no next draw. Only valid uses are to be tested.
21. $n = 0$ and $k = 1$ are invalid values for creating an `Urn` exemplar. All value pairs of $n = 0$ and $k > n$ lead to the calculation of the factorial of a negative number, which is not desired here. This case cannot be adequately tested yet.
22. $n = 1$ and $k = 1$ are valid values for creating an `Urn` exemplar. For this case exactly one permutation with one element ($k = 1$) exists. The value of this element at index position 0 is 0. When the urn is asked for the next draw, it returns `false` because there is no next draw. Only valid uses are to be tested.

23. $n = 2$ and $k = 1$ are valid values for creating an Urn exemplar. There are two permutations for this case. The first draw contains 0 as the only element, the second draw contains 1.
24. $n = 2$ and $k = 2$ are valid values for creating an Urn exemplar. The only draw the urn generates is 0 1. The `to_string()` function introduced above is used to create a valid string representation of the numbers in a draw.
25. $n = 3$ and $k = 3$ are valid values for creating an Urn exemplar. As explained earlier, the only draw the urn generates is 0 1 2. The `to_string()` function introduced above is used to create a valid string representation of the numbers in a draw.
26. $n = 4$ and $k = 3$ are valid values for creating an Urn exemplar. This urn generates four different draws, namely 0 1 2, 0 1 3, 0 2 3, 1 2 3. The `to_string()` function is used to create a valid string representation of the numbers in a draw.

1.15.5.5. Unit Testing of Urns

As announced at the beginning, the urn classes are being developed with a test-first approach. The author assures that he did it exactly that way. The file with the unit tests written originally with *Catch2* v2 has about 300 lines and was later adapted to *Catch2* v3 with minimal changes. Therefore, it is not reproduced in its entirety. Listing 49 contains some excerpts to explain how it was created.

Listing 49: objects/UrnUnitTests.cpp (excerpts)

```

1 // UrnUnitTests.cpp by Ulrich Eisenecker, March 18, 2025
2
3 #define CATCH_CONFIG_MAIN
4 #include <catch2/catch_test_macros.hpp>
5 // This code is based on Catch2 v3,
6 // therefore only the minimal header
7 // file for test macros is included.
8 // The C++ library path must be set
9 // accordingly for compilation and
10 // linking, and the Catch2Main and
11 // explicitly in exactly this order!
12
13
14 #include "urn.hpp"
15 #include <iostream>
16 #include <string>
17
18 std::string to_string(const urn::UrnOR& urn)
19 {
20     std::string result;
21     for (urn::uint i { 0 }; i < urn.k(); ++i)
22     {
23         if (i > 0)
24         {
25             result += " ";
26         }
27         result += std::to_string(urn[i]);
28     }

```

```
29     return result;
30 }
31
32 TEST_CASE("UrnOR")
33 {
34     using namespace urn;
35     SECTION("1")
36     {
37         UrnOR u { 0,0 };
38         REQUIRE(u.n() == 0);
39         REQUIRE(u.k() == 0);
40         REQUIRE(u.next() == false);
41     }
42     SECTION("2")
43     {
44         //UrnOR u { 0,1 };
45         std::cout << "[UrnOR][2] is not implemented yet"
46                   << std::endl;
47     }
48     SECTION("26")
49     {
50         Urn u { 4,3 };
51         REQUIRE(u.n() == 4);
52         REQUIRE(u.k() == 3);
53         REQUIRE(to_string(u) == "0 1 2");
54         REQUIRE(u.next() == true);
55         REQUIRE(to_string(u) == "0 1 3");
56         REQUIRE(u.next() == true);
57         REQUIRE(to_string(u) == "0 2 3");
58         REQUIRE(u.next() == true);
59         REQUIRE(to_string(u) == "1 2 3");
60         REQUIRE(u.next() == false);
61     }
62 }
```

The beginning of the file contains the usual boilerplate code, here a comment with name, date, and author of the program as well as `#define`'s and `#include`'s for the use of *Catch2* v3, plus the remaining necessary `#include`'s.

Each test case is named with the identifier of the class it tests. There are four test cases in total. At the beginning of writing the test file it was assumed that sections could be helpful. Later it turned out that there was not much to reuse other than using namespace `std`; for each section, but it was convenient that within each section all urns could be named `u`.

Next, the helper function `to_string()` is defined, which takes an `UrnOR` as reference to `const` and returns a `std::string` representation of its elements.

Each section is named with the unique number of the respective urn-specific case. Table 8 relates the code of *Section 1* to the textual specification of *Case 1*, which is specific to the `UrnOR` class.

Table 8: Test code and textual specification

Code	Textual specification
SECTION("1")	1.
{	
UrnOR u{0,0};	$n = 0$ and $k = 0$ are valid values for creating an UrnOR exemplar.
REQUIRE(u.n() == 0);	
REQUIRE(u.k() == 0);	For this case there is exactly one permutation which is empty. Immediately after the creation of the urn, this single permutation is available, but it is not possible to access any of its elements since it does not contain any elements ($k = 0$).
REQUIRE(u.next() == false);	When the urn is asked for the next draw, it must return false to indicate that there is no next draw.
}	Only valid uses are to be tested.

For the test-first approach, a precise idea of names and interfaces is required. All unit tests contained in *UrnUnitTests.cpp* are black-box tests, since the implementation of the classes and their methods was not available when the tests were written. The development of each test case requires solid mathematical analysis and paper and pencil work.

Lines 44 – 46 show a placeholder for a section that cannot yet be tested, because the appropriate mechanism for handling errors has not been introduced.

The last section covers a representative use of an Urn object. It also demonstrates how the `to_string()` function facilitates writing some assertions.

1.15.5.6. Implementation of the UrnOR Class

Once a sufficient understanding of the problem has been gained and the test cases have been implemented, the draw generators are implemented.

All definitions for the four urn classes are included in the header file *urn.hpp*. There is no separate implementation file. Each identifier is defined in the namespace `urn`. There are two aliases for types, namely using `uint = unsigned int`; and using `Draw = std::vector<uint>;`.

Now for the class definition. Listing 50 shows the UrnOR class.

Listing 50: UrnOR class (excerpt from *objects/urn.hpp*)

```
15  class UrnOR
16  {
17      public:
18          explicit UrnOR(uint n,uint k,uint check = 1):m_n { n },
19                                                         m_k { k },
20                                                         m_balls(k,0)
21      {
22          if (check == 1 && m_n == 0 && m_k > 0)
23          {
24              std::cerr << "UrnOR with n = 0 and k > 0 is not valid, "
25                          "program aborted" << std::endl;
26              std::exit(check);
27          }
28      }
29      uint n() const
30      {
31          return m_n;
32      }
33      uint k() const
34      {
35          return m_k;
36      }
37      uint at(uint index) const
38      {
39          return m_balls.at(index);
40      }
```

```

41         uint operator[](uint index) const
42         {
43             return m_balls[index];
44         }
45         [[nodiscard]] virtual bool next()
46         // Member function improved by Agathe Freywald.
47         {
48             uint down { m_k };
49             while (down > 1 && m_balls[down - 1] == m_n - 1)
50             {
51                 m_balls[down - 1] = 0;
52                 --down;
53             }
54             if (down > 0 && m_balls[down - 1] < m_n - 1)
55             {
56                 m_balls[down - 1]++;
57                 return true;
58             }
59             return false;
60         }
61         virtual ~UrnOR() = default;
62     protected:
63         const uint m_n,
64                 m_k;
65         Draw m_balls;
66 };

```

UrnOR has only one constructor, which is declared as explicit, preventing its implicit invocation. It takes three parameters: *n*, *k*, and *check*, each of type `urn::uint`. The meaning of *n* and *k* is obvious. They are used to initialize then UrnOR data members *m_n* and *m_k*. The third parameter, *check*, has the default value 1 and was introduced when implementing the Urn class. Its purpose will be discussed in a moment. The last data member of UrnOR is *m_balls*. It is also initialized in the initializer list of the UrnOR constructor. For calling the *m_balls* constructor, ordinary parentheses are used. This ensures that a special `vector<>` constructor is called, which creates a `vector<>` with *k* elements, each with the value 0. If the *m_balls* constructor were called with an initialization list, a different constructor would be executed that would create a `vector<>` with two elements, each with the values *k* and 0.

The *check* parameter is not intended to be specified by the user. As explained above (Section *Losing the Late Binding*), all methods in constructors and destructors are bound early, even if they are declared as `virtual`. The constructor body performs a check that is specific for the UrnOR and the Urn class. The other classes, Urn0 and UrnR, must perform other checks. Thus, an indication of which check is to be performed is required. When an UrnOR exemplar is created, 1 is inserted as the value for the *check* parameter. Hence, the *if* statement first checks if *check* is equal to 1. If this is not true, the remaining checks are discarded because of boolean shortcut evaluation.

If the `UrnOR` exemplar to be created is invalid, the entire program is terminated by calling the `std::exit()` function. The call to `std::exit()` is a kind of emergency exit from a program where only minimal cleanup is performed. In addition, the value passed as a parameter to `std::exit()` is returned upon program completion—just as the return value of `main()`. The value of `check` is passed to `std::exit()`, so the return value of 1 means that an attempt was made to initialize an `UrnOR` with invalid arguments. This is not a precise information, since any other part of the program can also call `exit(1)`.

The getter methods `n()` and `k()` are defined next. They return the values of the data members `m_n` and `m_k`. It should be pointed out that `m_n` and `m_k` are declared as `const` data members. This is possible because they may not be changed after initialization of an `UrnOR`. Accordingly, `n()` and `k()` are specified as `const` member functions.

Then two getter functions are defined for accessing individual balls, i.e. elements, of a draw. The member function `at()` takes an index as argument and calls `m_balls.at()`. The `vector<>::at()` member function performs index checking. Thus, if an invalid index were passed to `UrnOR::at()`, this would be detected and reported by `vector<>::at()`. Index checking causes a minimal performance delay. The index operator is also overloaded for accessing individual balls. In its implementation it calls `vector<>::operator[]()`. The index operator for *STL* container templates does not perform any index checking. In the best case, passing an invalid index to `UrnOR::operator[]()` when calling `vector<>::operator[]()` causes the program to crash; in the worst case, no runtime error is reported. Omitting the index check gives the best performance. Both getter methods return the ball specified by index as value. Thus, both member functions are specified as `const`.

The member function `next()` is `virtual`. Its return value must not be ignored. The purpose of this function is to generate the next draw. If there is a valid next draw, the member function returns `true`, otherwise `false`. The revised implementation (contributed by *Agathe Freywald*) uses a single `while` loop that iterates downwards from the highest index down of a draw as long as `down` is greater than 1 and the ball at position `down - 1` corresponds to the highest digit. As long as the condition of the `while` loop is `true`, the current ball is assigned the value 0, which corresponds to the lowest digit. Then, `down` is decremented by 1. The following `if` statement checks whether `down` is greater than 1 and whether the ball at index `down - 1` is smaller than the highest digit. If this is the case, the digit of the current ball is incremented by 1 and the function returns `true`. If this is not the case, the function returns `false`.

To prepare `UrnOR` as a valid base class, its `virtual` destructor is defaulted. The data members are declared in a `protected` section. This enables direct access for child classes.

1.15.5.7. Implementation of the UrnO Class

Listing 51 shows the UrnO class. It specifies UrnOR as a virtual base class. Here, the keyword `virtual` follows `public`. The order `virtual public` would be equally valid.

Listing 51: UrnO class (excerpt from `objects/urn.hpp`)

```

68     class UrnO: public virtual UrnOR
69     {
70     public:
71         explicit UrnO(uint n,uint k,uint check = 2):UrnOR { n,k,check }
72         {
73             if (check == 2 && m_k > m_n)
74             {
75                 std::cerr << "UrnO with k > n is not valid, "
76                     "program aborted" << std::endl;
77                 std::exit(check);
78             }
79             for (uint i { 1 }; i < m_k; ++i)
80             {
81                 m_balls[i] = i;
82             }
83         }
84         [[nodiscard]] bool next() override
85         {
86             while (UrnOR::next())
87             {
88                 if (!repetitions())
89                 {
90                     return true;
91                 };
92             }
93             return false;
94         }
95     protected:
96         bool repetitions() const
97         {
98             for (uint i { 0 }; i < m_k - 1; ++i)
99             {
100                 for (uint j { i + 1 }; j < m_k; ++j)
101                 {
102                     if (m_balls[i] == m_balls[j])
103                     {
104                         return true;
105                     }
106                 }
107             }
108             return false;
109         }
110     };

```


Like `UrnOR`, the `UrnO` constructor is specified as `explicit` and takes three arguments. One difference is the third argument `check`, which specifies 2 as its default value. The initialization list of the constructor calls the `UrnOR` constructor by passing all three arguments `n`, `k`, and `check`. Since `check` is 2 (unless explicitly specified otherwise by the programmer), the `UrnOR` constructor does not perform any checking. The body of the `UrnO` constructor uses boolean shortcut evaluation to check if `m_k` is greater than `m_n` when `check` equals two. Then, each element of `m_balls` is initialized with values starting at 0 up to excluding `m_k`. Note that the `for` loop starts with 1 and thus the first element of `m_balls` at index position 0, which is already correctly initialized with 0, is omitted.

The member function `next()` overrides `UrnOR::next()`. Therefore, it is declared with `override`. Like `UrnOR::next()`, it is attributed with `[[nodiscard]]`. Its implementation consists of a `while` loop that calls `UrnOR::next()` in its condition. The statement of the `while` loop checks if the previously generated draw contains a duplicate element. If not, the member function immediately returns `true`. If the `while` loop ends without finding a next draw without duplicates, it returns `false`. Since `UrnO::next()` overrides and calls `UrnOR::next()`, this is an example of what is called *overriding and extending*. Overriding without extending means that the overriding method does not call the overridden method.

The `repetitions()` member function is declared in a protected section of `UrnO`. It uses two nested `for` loops to search for duplicate elements in `m_balls`. Conceptually, the two loops iterate over the right triangle of a square matrix. Therefore, the inner loop starts with the current value of the outer loop variable plus one. One possibility for an additional visualization—if desired—is to output each pair of index variables `i` and `j` in the inner loop in a terminal window.

Since no destructor is specified, the compiler automatically generates a default constructor that is `virtual` because the base class destructor is also `virtual`.

1.15.5.8. Implementation of the `UrnR` Class

Listing 52 shows the `UrnR` class. It is very similar to the `UrnO` class. It also specifies `UrnOR` as a virtual base class, now in the order `virtual public`.

Listing 52: `UrnR` class (excerpt from `objects/urn.hpp`)

```
112 | class UrnR: virtual public UrnOR
113 | {
114 |     public:
115 |         explicit UrnR(uint n,uint k,uint check = 3):UrnOR { n,k,check }
116 |         {
117 |             if (check == 3 && m_n == 0)
118 |             {
119 |                 std::cerr << "UrnR with n = 0 is not valid, "
120 |                             "program aborted" << std::endl;
121 |                 std::exit(check);
122 |             }
```

```

123     }
124     [[nodiscard]] bool next() override
125     {
126         while (UrnOR::next())
127         {
128             if (!unsorted())
129             {
130                 return true;
131             };
132         }
133         return false;
134     }
135     protected:
136     bool unsorted() const
137     {
138         for (uint i { 0 }; i < m_k - 1; ++i)
139         {
140             if (m_balls[i] > m_balls[i + 1])
141             {
142                 return true;
143             }
144         }
145         return false;
146     }
147     };

```

The next difference is relevant: The check parameter of the constructor specifies 3 as the default value. In its implementation, the constructor prevents an urn from being created with $n = 0$ if check equals three.

UrnR also overrides `UrnOR::next()` and extends it. In `UrnR::next()` the while loop runs as long as a next draw can be generated by calling `UrnOR::next()`. It returns true if the actual draw is not unsorted, that is, if it is sorted. The `unsorted()` member function is defined as a protected method. It uses a single for loop to check if an element with a lower index value is greater than its neighbor element with index value plus one. If this is the case, it returns true. Neither `UrnO::duplicates()` nor `UrnR::unsorted()` can be unit tested because they are protected and therefore cannot be called directly in a test case.

`UrnR::next()` and `UrnO::next()` are not prefixed with `[[nodiscard]]`. This can be considered sloppy style, as it allows calling these member functions and ignoring their return values, which is indeed nonsensical. A very thorough programmer would change this. An optimistic programmer would hope that the return value of `next()` would never be ignored by a trusted programmer in a derived class. Since `next()` is protected, it can only be called in derived classes, but never from outside.

1.15.5.9. Implementation of the Urn Class

The fourth and final class is `Urn`, which is reproduced in Listing 53.

Listing 53: Urn class (excerpt from `objects/urn.hpp`)

```
149 | class Urn: public Urn0, public UrnR
150 | {
151 |     public:
152 |         explicit Urn(uint n,uint k):UrnOR { n,k },
153 |                                     Urn0 { n,k,1 },
154 |                                     UrnR { n,k,1 }
155 |     {
156 |         for (uint i { 1 }; i < m_k; ++i)
157 |         {
158 |             m_balls[i] = i;
159 |         }
160 |     }
161 |     [[nodiscard]] bool next() override
162 |     {
163 |         while (UrnOR::next())
164 |         {
165 |             if (!unsorted() && !repetitions())
166 |             {
167 |                 return true;
168 |             };
169 |         }
170 |         return false;
171 |     }
172 | };
```

`Urn` specifies `Urn0` and `UrnR` as public base classes. Its constructor does not specify a third parameter. This is because the same restrictions apply to `Urn` as to `UrnOR`. The initialization list of the constructor must call all base class constructors, including that of `UrnOR`. When calling the `UrnOR` constructor, the third argument is not specified. Therefore, it is set to one by default. The `Urn0` and `UrnR` constructors are explicitly called with 1 as third argument. The body of the `Urn` constructor does not perform any check. The necessary checks are performed by the `UrnOR` constructor, since it is called with 1 as check parameter. The rest of its implementation duplicates the second part of the `Urn0` constructor. This is done to avoid any dependency related to the initialization order of the inherited subobjects.

`Urn` also overrides the `next()` member function of its base classes. This is also a case of overriding and extending, since `UrnOR::next()` is called in the condition of the `while` loop in the implementation of `Urn::next()`. The statement of the `while` loop checks that the current draw is not unsorted and that it does not contain repetitions. The order in which these two checks are performed has been carefully chosen. Checking whether a draw is unsorted is cheaper in terms of runtime than testing whether it contains repe-

titions. Therefore, the cheaper check is performed first. If a draw does not need to be sorted and does not contain repetitions, the member function returns `true`, otherwise `false`.

1.15.6. Criticism of Inheritance

Previously, several uses of inheritance and even multiple inheritance were presented with amazing examples. However, it is also necessary to briefly discuss the disadvantages and pitfalls of inheritance.

Each inheritance relationship between two classes compromises modularity by increasing the coupling in the design (see (Hunt 2003), for example). To fully understand a class, all of its base classes must be carefully examined. When a base class is changed, it affects all of its descendants. Multiple inheritance can introduce duplicate subobjects or subobjects of virtual base classes that move through the inheritance hierarchy. Therefore, even more caution is needed when using multiple inheritance. However, sometimes it is a good way to design a class hierarchy for certain problems, such as the urn classes presented earlier.

The unreflective use of multiple inheritance usually comes at the price of poor code quality and reduced intelligibility and maintainability. Consider a base class `Person` with a postal address. When this serves as the base class for `PersonWithEmail`, `PersonWithPhone`, and indirectly `PersonWithEmailAndPhone`, the resulting design has built-in problems. If it is necessary to pass objects to functions that accept only `Phone`, `Email`, `Person`, or `Address` as types for references or pointers, a so-called *mix-in design* can be considered. This would start with the basic property classes `Person`, `Address`, `Phone`, and `Email`. Inheritance and multiple inheritance could then be used as a kind of glue to combine the desired properties into new types. This imaginative example illustrates again that multiple inheritance should only be used in a well thought out design when no other options are available or practical.

1.16. Cast Operators

In the context of object-oriented programming, there are problems that cannot be solved satisfactorily with the available means of type conversion, i.e. coercion polymorphism. In these cases, it is often necessary to perform them explicitly.

Conversions between data types are so-called (*type*) *casts*. Some of them are performed automatically. There are various categories of type casts, for which different so-called *cast operators* are available to perform them manually.

C++ often automatically applies conversions and promotions between values of different types (see (U. W. Eisenecker 2022), Section 3.1.1.6 *Promotions and Conversions*). Sometimes it is desirable to explicitly code such a type cast, which would otherwise be per-

formed automatically. However, it is much more common that certain type conversions are not supported or are forbidden by the compiler's type system. These are the situations where (type) casts come into play.

After a short introduction to the *classic C casts*, which also exist in modern C++, the cast operators that replace them are introduced. The classic C casts are also called *legacy casts*. It is important to know them, as they are still used in some libraries and frameworks, e.g. *FLTK* (see Section *FLTK*).

Template meta-functions and template variables of the *type support library* are used in the programming examples. Also, the `decltype` specifier is introduced, which allows to determine the type of a variable or expression without evaluating it. The programming examples do not represent practical applications.

1.16.1. Traditional Cast and Functional Cast

The need for type casts was also present in C. Accordingly, appropriate means for performing type casts were introduced in C. They are briefly presented below, highlighting not only their use but also their specific weaknesses.

Listing 54 presents a simple case for applying a type cast. C++ has no suffix for declaring short int literals, neither signed nor unsigned. Therefore, `auto usi { 5 };` always results in the variable `usi` being defined of type signed int. Instead of writing `unsigned short int usi { 5 };`, which is a silent conversion of the signed int literal 5 to the unsigned short int variable `usi`, a so-called *traditional cast* can be used to cast the int literal 5 to a temporary value of type unsigned short int, as is done in line 11 of Listing 54.

Listing 54: objects/SimpleLegacyCasts.cpp

```
1 // SimpleLegacyCasts.cpp by Ulrich Eisenecker, March 21, 2023
2
3 #include <iostream>
4 #include <type_traits>
5
6 using namespace std;
7
8 int main()
9 {
10     // Traditional cast
11     auto usi { (unsigned short int) 5 };
12     cout << boolalpha
13           << is_same_v<decltype(usi),short unsigned int>
14           << endl;
15
16     // Functional cast
17     auto ssi { short(5) };
18     cout << boolalpha
19           << is_same_v<decltype(ssi),short signed int>
20           << endl;
```

The result type of a traditional cast is placed between parentheses. This also allows the specification of types with compound names as in the present example. It must be emphasized that the traditional cast creates a temporary object of type `unsigned short int`. It does not convert the `int` literal 5 to a `unsigned short int` literal 5, which is actually impossible.

The following lines check if the type of `usi` is short unsigned int. For this purpose, the template variable `std::is_same_v<>` of the *type support library* is used, which requires the inclusion of `<type_traits>`. The type support library is introduced in (U. W. Eisenecker 2022), Section 3.2.1 *Information About Types*. `std::is_same_v<>` expects two type arguments. The first argument is the type of `usi`. This (static) type is obtained by the `decltype` specifier. `decltype` **can be applied to a variable or expression and returns the type of the variable or value to which the expression would evaluate. `decltype` does this without evaluating the variable or the expression.** As long as the program is compiled and executed as is, it prints `true` in the terminal window. Deleting the traditional cast in line 11 or converting it into a comment will give `false` as output after recompiling and executing the program.

The next part of `main()` uses a similar setting. In line 17, a so-called *functional cast* is used to create a temporary object of type `short` from the `int` literal 5. As a result, `auto` defines a variable of type `short signed int`. With the functional cast, parentheses are placed around the argument of the cast. This is similar to the syntax of a function call, hence the name *functional cast*. The functional cast has the problem that it only works with elementary type names. For this reason, `short` is specified as target of the functional cast. Writing `signed short int(5)` would result in a compilation error, because this code fragment cannot be parsed as a functional cast. This is a disadvantage of the functional cast. How to overcome this drawback will be shown soon. Of course, a traditional cast could be used instead. It would also be possible to put both the result type and the argument in a separate pair of parentheses.

The program in Listing 55 is a bit more complex. It demonstrates the use of legacy casts to restore the dynamic type of an object and remove constness.

Listing 55: objects/ConstAndDownLegacyCasts.cpp

```
1 // ConstAndDownLegacyCasts.cpp by Ulrich Eisenecker, March 21, 2023
2
3 #include <iostream>
4 #include <string>
5
6 using namespace std;
7
8 class Base
9 {
10 public:
11     [[nodiscard]] string staticTypename()
12     {
13         return "Base";
14     }
15     [[nodiscard]] virtual string dynamicTypename()
16     {
17         return "Base";
18     }
19     virtual ~Base() = default;
20 };
21
22 class Derived: public Base
23 {
24 public:
25     [[nodiscard]] string staticTypename()
26     {
27         return "Derived";
28     }
29     [[nodiscard]] string dynamicTypename() override
30     {
31         return "Derived";
32     }
33 };
34
35 const Base& test(Base& b)
36 {
37     cout << "Static type: " << b.staticTypename()
38          << '\t'
39          << "dynamic type: " << b.dynamicTypename()
40          << endl;
41     return b;
42 }
43
44 int main()
45 {
46     Base b;
47     Derived d;
48
49     test(b);
50     test(d);
51 }
```

```

52     cout << endl;
53
54     Derived& d1 { (Derived&) test(d) };
55
56     cout << d1.staticTypename()
57           << endl;
58
59     using DerivedRef = Derived&;
60
61     Derived& d2 { DerivedRef(test(d)) };
62
63     cout << d2.staticTypename()
64           << endl;
65 }

```

The Base class defines two member functions: `staticTypename()` with early binding and `dynamicTypename()` with late binding. Both return a `std::string` with the content "Base". The definition of the two member functions is intentionally flawed. Although they can be assumed not to change the state of the receiver object, they are not declared as `const`. Base also specifies a virtual destructor.

The Derived class specifies Base as its base class. It then hides `staticTypename()`, which returns "Derived", and overrides `dynamicTypename()`, which also returns "Derived".

The `test()` function takes a reference of type Base as its only parameter `b`. In its implementation it calls `b.staticTypename()` and `b.dynamicTypename()` and outputs their return values. It returns `b` as a reference to `const` of type Base. This is possible because `b` is passed by reference. This prevents the use of a temporary or a `const` object as an argument. Therefore, the original (non-`const`) object referenced by `b` must exist in the scope of the caller of `test()`. Hence, it is safe to return `b` as a reference. The design of this function also contains a flaw: It returns `b` as a reference to `const`. This is odd because `b` is not passed as a `const` parameter. As a consequence, the value returned by `test()` is almost useless, because all of its member functions are not `const`.

In the `main()` function, a Base exemplar named `b` and a Derived exemplar named `d` are declared. The `test()` function is called for each of them. This produces the expected output, namely Base as both the static and dynamic type for Base, and Base as the static type for Derived and Derived as the dynamic type for Derived. This is consistent with the early binding of `Base::staticTypename()` and the dynamic binding of `Base::dynamicTypename()`.

In line 54, a reference to Derived named `d1` is defined for the result of calling `test()` with the `d` exemplar. Calling `test()` with a Derived object must return a Derived object, but with Base as the static type. Initializing a reference to a derived class with an object of a base class is not legal. Since it is certain that the result of the call to `test()` here is an object of dynamic type Derived, a traditional cast is applied to perform a so-called *downcast*. Such downcasts are common in object-oriented programming, because there are often functions or member functions similar to `test()` that return a reference or

pointer to a base class for an object whose dynamic type is a derived class. It is very important that the target of the traditional cast is `Derived&`. If the reference symbol were omitted, the cast would result in an attempt to create a temporary object of type `Derived` by providing an object of type `Base`. This would be a double error: First, there is no conversion operator for `Derived` that takes a `Base` object as a parameter, and second, if the previous error were fixed, it is not allowed to create a reference for a temporary object.

A second effect of this cast is that it cancels the constness of the result returned by `test()`. Thus, this cast mixes two purposes, namely a downcast to restore the static type of the object and the cancellation of its constness. As a consequence, the reference `d1` is not `const` and it is possible to call `staticTypeName()` for `d1`, which returns the expected result `"Derived"`.

To demonstrate the same with a functional cast, the `using` declaration defines `DerivedRef` as alias for `Derived&`. Since `DerivedRef` is not defined as an alias for `const Derived&`, when `DerivedRef` is used later in the functional cast, constness is also removed. `Derived&` is also a composite type specification that cannot be parsed by the compiler as the result of a functional cast. However, performing the functional cast with `DerivedRef` as the result type is straightforward, as shown in line 61. Like the traditional cast before, the functional cast here has a double effect: It removes the constness and performs a downcast.

This illustrates a weakness of these *C* legacy casts: They express no meaning for the purpose of their application. Another weakness is that the compiler performs minimal checks for them, since it also has no idea why these casts are applied. To overcome these deficits, more specific (type) casts were introduced in *C++*.

1.16.2. `static_cast<>()`

The *C++*-specific type casts replace the *C* type casts. It is preferable to use them because they make the purpose of their use explicit.

The new casts of *C++* have a common structure. Syntactically, they resemble function calls, more precisely explicit instantiations of function templates (see Section *Explicit Template Arguments*), which specify the result type in angle brackets, followed by the variable or expression to be cast passed as an argument in parentheses.

Listing 56 shows an excerpt from a program that rewrites the program from Listing 54, where `static_cast<>()` is used instead of the traditional cast and the functional cast.

Listing 56: objects/StaticCast.cpp

```

1 // StaticCast.cpp by Ulrich Eisenecker, March 21, 2023
2
10 // Formerly traditional cast, now static_cast
11 auto usi { static_cast<unsigned short int>(5) };
16 // Formerly functional cast, now static_cast
17 auto ssi { static_cast<short>(5) };
21 }

```

In the above example, the cast is required to fill the gap caused by the missing short suffix for integer literals. A redundant use of it is to highlight an automatic conversion or promotion to improve code readability. It also supports conversions between integral, enumeration, and floating point values. Care must be taken here, as the resulting values may be truncated or undefined if the result type cannot adequately represent the argument of the cast.

In addition, a `static_cast<>()` can be used for downcasts (as shown below) or for casts between related pointer types and any pointer type to a pointer to void or from a pointer to void to any pointer type. `static_cast<>()` is evaluated at compile time, there are no runtime checks.

Applying `static_cast<>()` is solely the responsibility of the programmer. A `static_cast<>()` must preserve constness. If the argument of `static_cast<>()` is `const`, the result type specification in the angle brackets must also contain `const`. The result type may be a temporary object, which is often the case, or it may be a reference, for example when performing a downcast that should not create a new exemplar of the desired type. For comprehensive information about the static cast operator, see (*Static_cast Conversion - Cppreference.Com* n.d.).

1.16.3. `const_cast<>()`

The only purpose of the `const` cast operator is to remove the constness of an object. It cannot change its type. The removal of constness only affects the `const` object. For this reason, the result type of `const_cast<>()` must always be a reference or a pointer to the argument type, which is not `const`. Listing 57 shows a snippet based on the program in Listing 54. The `test()` function returns the object passed to it by reference as a reference to `const`, so none of its non-`const` member functions can be called afterwards. That these member functions are not declared as `const` can be considered a design flaw of `Object` and `Derived`. But it is one of the rare occasions when the use of `const_cast<>()` is appropriate.

Listing 57: objects/ConstCast.cpp

```
1 // ConstCast.cpp by Ulrich Eisenecker, March 23, 2023
49 cout << const_cast<Base&>(test(b)).dynamicTypeName() << endl;
50 cout << const_cast<Base&>(test(d)).dynamicTypeName() << endl;
51 }
```

The const cast operator should not be applied to temporary objects or to objects that are intentionally declared as const. This can lead to unexpected consequences and programs with undefined behavior.

1.16.4. dynamic_cast<>()

The dynamic cast operator is bound to the inclusion polymorphism. After motivating its use and syntax in comparison to the static cast operator using artificial examples, its application is demonstrated using assignments in combination with inheritance. In addition, it is shown how conditional compilation can be used to also subject private or protected member functions to unit tests.

1.16.4.1. Syntactic Aspects of Using dynamic_cast<>()

With the dynamic cast, two usage scenarios must be considered, namely casting to a reference or casting to a pointer.

The dynamic cast operator is effective only for inclusion polymorphism. It allows to cast the type of any object to a super type or a subtype of it, but not to a subclass of the true type. For technical reasons, dynamic_cast<>() requires the vtbl pointer of an object. Therefore, both features of inclusion polymorphism, inheritance and late binding, must be present. Listing 58 shows the use of the dynamic cast operator, its limitations, and also a pitfall of static_cast<>() compared to dynamic_cast<>().

Listing 58: objects/DynamicCast.cpp

```
1 // DynamicCast.cpp by Ulrich Eisenecker, May 8, 2024
46 Base b;
47 Derived d;
48
49 test(b);
50 test(d);
51
52 cout << endl;
53
54 const Derived& d1 { dynamic_cast<const Derived&>(test(d)) };
55
56 cout << const_cast<Derived&>(d1).staticTypeName()
57     << endl;
58
59 const Derived* d2 { dynamic_cast<const Derived*>(&(test(d))) };
```

```

60
61     if (d2 != nullptr)
62     {
63         cout << const_cast<Derived*>(d2)->staticTypename()
64             << endl;
65     }
66
67     // The following static_cast<>() is invalid,
68     // but not reported as an error.
69     const Derived& d3 { static_cast<const Derived&>(test(b)) };
70     // The following line leads to nonsensical results.
71     cout << const_cast<Derived&>(d3).staticTypename() << endl;
72     // The following line produces a correct result.
73     cout << const_cast<Derived&>(d3).dynamicTypename() << endl;
74
75     // The following dynamic_cast is invalid
76     // and causes a runtime error.
77     const Derived& d4 { dynamic_cast<const Derived&>(test(b)) };

```

The missing code at the beginning is the same as in Listing 55. To refresh the memory: Base is a class with a vtbl, Derived inherits publicly from Base, and the test() function takes a reference of type Base or a subtype and returns it as a reference to const of type Base. In line 54, it is known that a Derived exemplar is passed to test(). Therefore, it is safe to downcast the reference of type Base returned by test() to Derived. The dynamic cast operator cannot remove constness. Hence, its result type is a reference to const of type Derived. The object returned by dynamic_cast<>() is used to initialize the reference to const of type Derived named d1. Now d1 is an alias for d, which has been upcast by test() and downcast by dynamic_cast<>() in the meantime.

The important property of the dynamic cast operator is that it checks at runtime whether the cast is valid. Attempting to cast an object into a reference of a class that is neither a super type of the object's class or a subtype beyond the object's true type will result in a bad cast exception being thrown. Exceptions and how to handle them is covered in the Section *Exceptions*.

Because d1 is const, it is not possible to call its member function staticTypename(). In line 54, the const cast operator removes the constness so that this member function can be called. As a result, the call to staticTypename() returns the correct value, namely "Derived". As illustrated, each cast has a specific function. Its intentionality improves the readability of the program. Due to the specific features of a modern cast, the compiler can also better verify whether it has been used appropriately.

Actually, it would be possible to apply const_cast<>() immediately to the result returned by dynamic_cast<>(). This is not done as the resulting statement would be difficult to read.

Line 59 demonstrates how dynamic_cast<>() is applied to a pointer. In its argument, the address of the result of the call to test() is taken by &(test(d)). Consequently, the result type is specified as const Derived*. The pointer must point to a const ob-

ject, because the dynamic cast operator cannot remove constness. The result of using `dynamic_cast<>()` is now a temporary pointer that is used to initialize the pointer variable `d2`. When casting to a pointer, `dynamic_cast<>()` checks its validity at runtime. If it is invalid, the value `nullptr` is returned. Therefore, line 61 tests whether `d2` is different from `nullptr`. If this is the case, the const cast operator is used to remove constness in line 63 and then call the `staticTypename()` member function.

The following lines clarify the difference between `static_cast<>()` and `dynamic_cast<>()`.

In line 69, the `Base` exemplar passed to `test()` and returned by `test()` is cast down to `Derived` with `static_cast<>()`. This cast is invalid, but it is not reported as an error by the compiler and does not cause a runtime error. The following two lines will probably be executable on most platforms. Since `Derived::staticTypename()` has an early binding, the result of calling this member function for `Derived` is consistent, but semantically incorrect. As this static cast does not change the vtbl pointer of an object, line 73 gives a semantically correct result. Nevertheless, lines 69 – 73 are wrong. The problem would be detected at runtime, if `dynamic_cast<>()` were used in line 69 instead of `static_cast<>()`.

Eventually, line 77 demonstrates what happens when an invalid dynamic cast is applied to a reference. During execution, a bad cast exception is reported here and the program is aborted.

1.16.4.2. Implementation of Correct Assignments for Inclusion Polymorphism

In *Object With Dynamic Resource—The Wrong Way* and following sections it was mentioned that assignment in combination with inclusion polymorphism can cause problems. These problems are illustrated below before presenting a solution using `dynamic_cast<>()`.

The program in Listing 59 defines two classes `A` and `B`. Each class defines a constructor that takes one or two strings as arguments that should identify the class and the object to be created. Both constructors initialize the corresponding data member, and `B::B()` also calls `A`'s constructor. The member function `A::print()` is virtual and overridden and extended in `B`. As required for a base class, a virtual destructor is defaulted for `A`.

Listing 59: objects/ProblematicAssignment.cpp

```

1 // ProblematicAssignment.cpp by Ulrich Eisenecker, March 24, 2023
2
3 #include <iostream>
4 #include <string>
5 using namespace std;
6
7 class A
8 {
9     public:
10         A(string a = "A::a"): m_a { a }
11         {}
12         virtual void print() const
13         {
14             cout << m_a << endl;
15         }
16         virtual ~A() = default;
17     private:
18         string m_a;
19 };
20
21 class B: public A
22 {
23     public:
24         B(string a = "B::a", string b = "B::b"): A { a }, m_b { b }
25         {}
26         void print() const override
27         {
28             A::print();
29             cout << m_b << endl;
30         }
31     private:
32         string m_b;
33 };
34
35 int main()
36 {
37     cout << "Assignment with static == dynamic type\n"
38          << "a" << endl;
39     A a;
40     a.print();
41     cout << "b" << endl;
42     B b;
43     b.print();
44     cout << "a = b" << endl;
45     a = b;
46     a.print();
47
48     cout << "\nAssignment with static != dynamic type"
49          << endl;
50     B b1 { "b1::a", "b1::b" };
51     B b2 { "b2::a", "b2::b" };

```

```
52 |     A& b1asa { b1 };  
53 |     A& b2asa { b2 };  
54 |     b1asa.print();  
55 |     b2asa.print();  
56 |     cout << "b1asa = b2asa" << endl;  
57 |     cout << "b1asa" << endl;  
58 |     b1asa = b2asa;  
59 |     b1asa.print();  
60 | }
```

The `main()` function has two parts. In the first part, only valid assignments are performed. For each object involved, its static type in the assignment is equal to its dynamic type. There are four possible combinations for assignments:

- Of course, it is possible to assign an A exemplar to another A exemplar, which is not shown here.
- It is permissible to assign a B exemplar to an A exemplar, as shown in line 45. The assignment `a = b;` assigns only the A subobject of B to a, as shown in Figure 26. The B specific part in this assignment is discarded.
- It is also legal to assign a B exemplar to another B exemplar.
- It is not possible to assign an A exemplar to a B exemplar because the A exemplar has no B specific part. The exemplar resulting from such an invalid assignment would be in a mixed state, i.e. assigned and unassigned.

Figure 26: Output of `objects/ProblematicAssignment.cpp`

```
Assignment with static == dynamic type  
a  
A::a  
b  
B::a  
B::b  
a = b  
B::a  
  
Assignment with static != dynamic type  
b1::a  
b1::b  
b2::a  
b2::b  
b1asa = b2asa  
b1asa  
b2::a  
b1::b
```

In the second part of `main()`, two B exemplars, `b1` and `b2`, are defined. Then, two A references named `b1asa` and `b2asa` are initialized with `b1` and `b2`, respectively. This is legal because a subclass exemplar may be used if a base class exemplar is expected (assuming public inheritance between the classes involved). Then the assignment `b1asa = b2asa` is executed. The result is probably not expected from a naive point of view. The output shows that only the A-specific part of both referenced B exemplars is assigned. The assignment ignores the B-specific part.

The reason for this is that the compiler generates an overloaded assignment operator when no user-defined assignment operator is available. Providing a virtual assignment operator in both A and B is not a solution because overriding requires that the specification of a member function remains the same. However, overriding an assignment operator in B that takes an A as an argument would convert illegal assignments into valid assignments. Also, the compiler would still automatically generate an overloaded version of the assignment operator for B that takes a B as an argument.

This strange situation can be at least partially solved by providing a virtual member function that performs the assignment, and an overloaded assignment operator that in turn calls the virtual member function that performs the assignment. All of this must be done for each class in the inheritance hierarchy from the base class down.

The member function that performs the assignment is usually called `assign()` and is overridden and extended in each derived class.

Listing 60 shows the corresponding solution for classes A and B. The code is implemented as a header file with multiple inclusion protection. All methods are implicitly defined inline, i.e. they are a direct part of the header file. Thus, no separate implementation file is required. All declarations are included in the namespace `assignment`.

To enable unit tests, conditional compilation is used. This allows unit tests to access private class members for testing purposes. If the `COMPILE_FOR_TESTING` macro is not defined, the `#ifndef` preprocessor directive in lines 33 and 63 includes the source code up to the following `#endif` directive. In this way, the program is compiled so that `A::m_a` and `B::m_b` are private data members. If the macro `COMPILE_FOR_TESTING` is defined, the corresponding code is not included. Thus, it can be accessed, ideally from within the unit tests. To achieve this, the macro `COMPILE_FOR_TESTING` must be defined before *assignment.hpp* is included the first time.

Listing 60: objects/assignment.hpp

```
1 // assignment.hpp by Ulrich Eisenecker, May 17, 2023
2
3 #ifndef ASSIGNMENT_HPP
4 #define ASSIGNMENT_HPP
5
6 #include <iostream>
7 #include <string>
8
9 namespace assignment
10 {
11     class A
12     {
13     public:
14         A(std::string a = "A::a"): m_a { a }
15         {}
16         virtual void print() const
17         {
18             std::cout << m_a << std::endl;
19         }
20         virtual void assign(const A& a)
21         {
22             if (this != &a)
23             {
24                 m_a = a.m_a;
25             }
26         }
27         A& operator=(const A& a)
28         {
29             assign(a);
30             return *this;
31         }
32         virtual ~A() = default;
33     #ifndef COMPILE_FOR_TESTING
34     private:
35     #endif // COMPILE_FOR_TESTING
36         std::string m_a;
37     };
38
39     class B: public A
40     {
41     public:
42         B(std::string a = "B::a", std::string b = "B::b"): A { a }, m_b { b }
43         {}
44         void print() const override
45         {
46             A::print();
47             std::cout << m_b << std::endl;
48         }
49         void assign(const A& a) override
50         {
51             if (this != &a)
```

```

52         {
53             const B& b { dynamic_cast<const B&>(a) };
54             A::assign(b);
55             m_b = b.m_b;
56         }
57     }
58     B& operator=(const B& b)
59     {
60         assign(b);
61         return *this;
62     }
63     #ifndef COMPILE_FOR_TESTING
64     private:
65     #endif // COMPILE_FOR_TESTING
66         std::string m_b;
67     };
68 }
69
70 #endif // ASSIGNMENT_HPP

```

The `assign()` member function is the heart of the solution. Its implementation in the base class `A` first checks whether there is a self-assignment. Despite the fact that a self-assignment is pointless, it is technically possible and cannot be prevented. If the `this` pointer and the address of the object to be assigned to the receiver object are different, there is no self-assignment and each `A`-specific member is assigned. In the case of a self-assignment, the function is exited without executing another instruction. The only argument to `assign()` is a reference to `const` of type `A`. This allows a temporary object to be used for the assignment. Otherwise, the object is used as it is without copying it. Of course, instead of an `A` exemplar, any exemplar of a direct or indirect public subclass of `A` can be used. It is important that `assign()` is declared as `virtual`, because it must be overridden in any descendant class.

To make overriding effective, `B::assign()` must specify exactly the same interface as `A::assign()`. Therefore, the argument of `assign()` is also a reference to `const` of type `A`. As a consequence, an object of the base class can be assigned to an object of the subclass, which breaks the type system. Although this assignment is invalid, it can no longer be recognized by the compiler. How to solve this problem will be explained soon.

The implementation of `B::assign()` first checks whether it is not a self-assignment. If this check evaluates to true, a `dynamic_cast<>()` is applied to `a`. It performs a downcast to a reference to `const` of type `B`. If this cast fails, an exception is thrown and the member function exits prematurely. If the cast succeeds, `A::assign()` is called first. Since `b` is a reference to `const` of type `B`, it can be passed instead of a reference to `const` of type `A` as specified in `A::assign()`. Then all `B`-specific parts are assigned. This implementation of the `assign()` member function in any derived class ensures that the receiver object remains in an unchanged state, even in the case that the dynamic cast fails.

Two aspects deserve to be highlighted:

- An object of the same class can always access private members of another exemplar of the same class. This possibility is exploited in `A::assign()` and in `B::assign()`.
- An object that is a subclass has more than one type: Namely its true type, also known as dynamic type, and all its direct and indirect base class types. That is, the object passed to `B::assign()` as a reference to `const` of type `A` can be of type `A`, of type `B`, or any type of a derived class. All of this is syntactically correct. The dynamic cast allows casting the true type `B` into the static type `B`, as well as casting any true subtype of `B` into the static type `B`.

Each subclass of `B` must override `B::assign()` in exactly the same way as it is done in `B::assign()`. In this way, an assignment is performed from the top-level base class subobject down to the true type of the receiver object.

The specification of `assign()` makes invalid assignments syntactically possible, but its implementation detects them with `dynamic_cast<>()`. As a consequence, the invalid assignments are not detected at compile time, but cause a runtime error. This is an undesirable but unavoidable trade-off to make assignment work in combination with inclusion polymorphism.

The `assign()` member function provides everything required to perform assignments in the context of inclusion polymorphism. Overloading the assignment operator provides only so-called *syntactic sugar*. It must be overloaded for each subclass of the base class—including the base class—that defines the member function `assign()` for the first time. It takes a reference to `const` of the same type as an argument and calls `assign()`. The `assign()` member function does all the rest. The assignment operator only has to return the assigned object as a reference.

The program *AssignmentUnitTests.cpp*, part of which is reproduced in Listing 61, includes two comprehensive test cases, one for the `assign()` member function named "ASSIGN" and one for the assignment operator named "OPERATOR=". Each test case uses *Catch2* sections to automate the creation of the test environment. The structure of the "OPERATOR=" test case is exactly the same as the "ASSIGN" test case. This is another indicator that the assignment operator adds nothing to the functionality realized by the `assign()` member function.

Listing 61: objects/AssignmentUnitTests.cpp (excerpt)

```

1 // AssignmentUnitTests.cpp by Ulrich Eisenecker, May 20, 2026
2
3 #define CATCH_CONFIG_MAIN
4 #include <catch2/catch_test_macros.hpp>
5 // This code is based on Catch2 v3,
6 // therefore only the minimal header
7 // file for test macros is included.
8 // The C++ library path must be set
9 // accordingly for compilation and
10 // linking, and the Catch2Main and
11 // explicitly in exactly this order!
12
13
14 #define COMPILE_FOR_TESTING
15
16 #include "assignment.hpp"
17 #include <exception>
18 #include <string>
19
20 using namespace std;
21 using namespace assignment;
22
23 TEST_CASE("ASSIGN")
24 {
25     A a1 { "a1::a" };
26     A a2 { "a2::a" };
27     B b1 { "b1::a", "b1::b" };
28     B b2 { "b2::a", "b2::b" };
29     A& b1asa { b1 };
30     A& b2asa { b2 };
31
32     SECTION("a1_assign_a2")
33     {
34         REQUIRE(a1.m_a == "a1::a");
35         REQUIRE(a2.m_a == "a2::a");
36         a1.assign(a2);
37         REQUIRE(a1.m_a == a2.m_a);
38     }
39     SECTION("b1_assign_b2")
40     {
41         REQUIRE(b1.m_a == "b1::a");
42     }
43     SECTION("b1asa_assign_a1")
44     {
45         REQUIRE(b1.m_a == "b1::a");
46         REQUIRE(b1.m_b == "b1::b");
47         REQUIRE(a1.m_a == "a1::a");
48         REQUIRE_THROWS_AS(b1asa.assign(a1), bad_cast);
49     }
50 }
51
52 TEST_CASE("OPERATOR=")
53 {

```

```
118 | SECTION("b1asa=b2asa")
119 | {
120 |     REQUIRE(b1.m_a == "b1::a");
121 |     REQUIRE(b1.m_b == "b1::b");
122 |     REQUIRE(b2.m_a == "b2::a");
123 |     REQUIRE(b2.m_b == "b2::b");
124 |     b1asa = b2asa;
125 |     REQUIRE(b1.m_a == b2.m_a);
126 |     REQUIRE(b1.m_b == b2.m_b);
127 | }
143 | }
```

For successfully compiling the test program, `COMPILE_FOR_TESTING` must be defined before "assignment.hpp" (line 16) is included. When compiling, it is also important to specify the `-lCatch2Main` option before `-lCatch`. In the "ASSIGN" test case two A and two B exemplars are defined. In addition, two references of type A are initialized with the B exemplars. Altogether, this allows testing all base class and subclass assignments where the static type is the same as the dynamic type, and all possible combinations of assignments where the receiver object has the same static type and dynamic type, as well as all combinations of two objects where the static type and dynamic type are different, and semantically incorrect assignments that cause runtime errors.

The "b1asa_assign_b2asa" section shows the interesting case where both variables involved in the assignment have a different static and dynamic type. Both `b1asa` and `b2asa` are of static type A, but their true type is B. When `b1asa.assign(b2asa);` is executed, the `B::assign()` member function for `b1asa` is called because of the late binding. This in turn calls `A::assign()` before executing the B-specific assignments. It must be emphasized that all `REQUIRES` assertions are formulated with the original variables and not the aliases for simplicity. Otherwise, additional type casts would be necessary to cast the static type of e.g. `b1asa` into the required true type.

The section "b1asa_assign_a1" assigns `a1`, which is of true type A, to `b1`, which is of true type B, which is semantically invalid. To check if the expected exception is thrown due to the invalid `dynamic_cast<>()`, the `REQUIRES_THROWS_AS` assertion is used. Its first argument is the expression to evaluate, here `b1asa.assign(a1)`, and the second argument the exception type to be thrown, here `bad_cast`. To use `bad_cast`, the `<exception>` header must be included (line 17).

The "b1asa=b2asa" section of the `TEST_CASE("OPERATOR=")` corresponds to the "b1asa_assign_b2asa" section of the `TEST_CASE("ASSIGN")`. It simply replaces the `b1asa.assign(b2asa);` statement with `b1asa = b2asa;`. When the assignment `b1asa = b2asa;` is executed, the assignment operator defined in A is called. This one in turn calls the `assign()` member function, and the following execution is the same as for `b1asa.assign(b2asa);`.

The general assignment pattern, the `assign()` member function and its implementation in each subclass, and the overloading of the assignment operator for each class in combination with inclusion polymorphism must be followed very carefully. Otherwise, subtle errors may occur that are hard to spot. The possible errors are not examined here.

Another issue is the implementation of the `assign()` member function for a class that owns a dynamic resource. In some cases, it is possible and makes sense to also perform the assignment for the dynamic resource. However, sometimes this is not useful. In these cases, the access handle to the old dynamic resource must be saved. Then a new exemplar of the dynamic resource should be copy-initialized, then it should be assigned to the access handle (if necessary), and eventually the old dynamic resource should be returned or deallocated. This is not explained in detail here, because it is recommended to either do it without a dynamic resource or to delete member functions that can cause problems in combination with dynamic resources, such as copy constructor or assignment.

Special attention must be paid if multiple inheritance also needs to be considered in the assignment.

Other solutions to the assignment problem in the context of inclusion polymorphism are based on the so-called *copy-and-swap idiom*, see for example (Gregoire 2020) and (Breymann 2020). These solutions are not presented here. All of these solutions have their advantages and disadvantages.

1.16.5. reinterpret_cast<>()

The *reinterpret cast operator* supports any conversion between types, but it cannot remove constness. This cast must be used with the greatest care. After showing its use in a simple example displaying the bits of a `float` variable, it is applied in a more realistic scenario to complete the serialization of objects (introduced in Section *Static Members*) with deserialization. This also requires the introduction of the `swap()` member function. The result is a minimal basis for object persistence.

1.16.5.1. Accessing Bits of a Float Variable With reinterpret_cast<>()

The basic representation of any value in the computer is a *bit sequence* or *bit string*. Depending on the type associated with this bit string, it is interpreted either as a floating point value, integer, Boolean value, and so on. With the *reinterpret cast*, a bit string can be interpreted as any type. Therefore, its destination type is usually specified as a reference, because the *reinterpret cast* does not create a new value. The only limitation of the *reinterpret cast* is that it cannot remove the constness of an object.

The compiler only checks for `const` correctness, but nothing else. Therefore, `reinterpret_cast<>()` should be used with extreme caution.

In Listing 62, `reinterpret_cast<>()` is used to interpret a float variable as an unsigned integer of the same size. This allows to inspect which bits are set for which values of the float variable.

Listing 62: objects/ReinterpretCast.cpp

```
1 // ReinterpretCast.cpp by Ulrich Eisenecker, March 28, 2023
2
3 #include <iostream>
4 #include <climits>
5
6 using namespace std;
7
8 using uint_t = unsigned int;
9
10 static_assert(sizeof(float) == sizeof(uint_t));
11
12 constexpr uint_t one { 1 };
13
14 int main()
15 {
16     float f { };
17     cout << "Float: ";
18     cin >> f;
19     uint_t& i { reinterpret_cast<uint_t&>(f) };
20     uint_t loop { sizeof(uint_t) * CHAR_BIT };
21     while (loop > 0)
22     {
23         cout << (i & one << (loop - 1) ? '1' : '0' );
24         --loop;
25     };
26     cout << endl;
27 }
```

To reliably retrieve the number of bits a byte has, `<climits>` is included. It provides the macro constant `CHAR_BIT`, which gives exactly this information. In line 8, the type alias `uint_t` is declared. It allows to easily change the unsigned integer type to be used, if necessary. In line 10, `static_assert()` checks if `float` and `uint_t` have the same size. If not, the compiler reports an error and aborts compilation. This is a necessary measure because further down in the program a `float` variable is interpreted as an unsigned integer variable. If the integer type would be represented with less bytes than the `float` variable, the program output would be wrong. If the `float` variable had more bytes than the unsigned integer variable, the program would access bits that were not previously allocated properly, resulting in a program with undefined behavior. This check is essential as a safeguard for the `reinterpret cast` that will be applied later. In line 12, the constant `one` is defined with `constexpr`. It is of type `uint_t` and initialized to one. This is important because this value will be shifted to the left in the further course of the program. Otherwise this can lead to erroneous results, especially if the value shifted to the left has fewer bytes than the value specifying the shift count.

The `main()` function defines the `float` variable `f` and inputs a value from the console which is stored in `f`. In line 19, the reinterpret cast is applied. It specifies a reference of type `uint_t` as the result type and `f` as the argument. The result is used to initialize the reference `i` of type `uint_t`. Consequently, `f` and `i` refer to the same bit string, but with different types.

Next, the variable `loop` with type `uint_t` is initialized with the number of bits that `uint_t` occupies multiplied with `CHAR_BIT`. This guarantees that `loop` is initialized to the actual number of bits that a `uint_t` value or variable occupies. The following `while` loop executes as long as `loop` is larger than 0. In the `while` loop the ternary operator `? :` is applied to determine whether to output 1 for a set bit or 0 for an unset bit. The condition of the ternary operator is to the left of the question mark. Here `i` is combined with bitwise AND with the value resulting from shifting one `(loop - 1)` bits to the left. It is important to subtract 1 from the `loop` value to get the correct shift count, because 1 is already the least significant bit (LSB). If it would be shifted to the left by `loop` positions, the most significant bit (MSB) would be shifted one position too far to the left and thus fall out. Therefore it must be shifted 0 positions to the left. If the bit resulting from bitwise OR of `i` and `(loop - 1)` is set, the ternary operator returns the value to the left of the colon; if not, the value to the right of the colon is returned. Then `loop` is decremented by one. It is important that the `while` loop runs from the MSB to the LSB, because humans are used to seeing the MSB at the leftmost position of the output. After the `while` loop terminates, a newline is sent to the standard output stream.

Depending on the platform, the compilation of the program may fail due to the `static_assert()` statement. In this case, `uint_t` should be adapted to an unsigned integer type which has the same number of bytes as the `float` type.

This small program not only demonstrates the syntax of `reinterpret_cast<>()`, but also shows the necessary measures for its safe use.

1.16.5.2. Implementation of Object Persistence With `reinterpret_cast<>()`

How to serialize objects into a file stream is shown in Listings 29 and 30. Now the serialization is to be complemented by a deserialization. This requires modifications to the `OIDManager` class and the introduction of the `Persistable` class, which serves as the base class for all classes with persistable objects. The program *PersistenceUnitTests.cpp* (available in the accompanying material in the *sources/objects* directory, but not reproduced here) defines new persistable classes and performs extensive unit tests for `OIDManager` and persistable classes. Deserializing associations between objects and updating them requires the use of a `reinterpret_cast<>()`. Updating associations between objects is simplified by the `std::set<>::swap()` member function.

Overall, this provides a minimal basis for object persistence. The approach used here is called *persistence by type* because it assumes that all classes with persistable objects are subtypes of a persistable base class. There are other approaches to realizing object persistence that are not presented here.

Listing 63 introduces the `Persistable` base class and the revised `OIDManager` class.

Listing 63: `objects/persistence.hpp`

```
1 // persistence.hpp by Ulrich Eisenecker, May 19, 2025
2
3 #ifndef PERSISTENCE_HPP
4 #define PERSISTENCE_HPP
5
6 #include <map>
7 #include <iostream>
8 #include <limits> // Because of numeric_limits<std::streamsize>::max()
9
10 namespace persistence
11 {
12     using IdType = std::size_t; // Provide a single point for changing the type
13                                // of object IDs.
14
15     class Persistable
16     {
17     public:
18         virtual void serialize(std::ostream& os,
19                               const bool newline = true) const = 0;
20         virtual void deserialize(std::istream& is,
21                                 const bool newline = true) = 0;
22         virtual void updateAssociations()
23         {
24             // Nothing to update for Persistable.
25         }
26         virtual ~Persistable() = default;
27         inline static constexpr char separator { '\t' };
28         static void nextLine(std::istream& is)
29         {
30             is.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
31         }
32         static void nextToken(std::istream& is)
33         {
34             is.ignore(1, separator);
35         }
36     };
37
38     static_assert(sizeof(IdType) == sizeof(Persistable*),
39                   "Incompatible sizes");
40
41     class OIDManager
42     {
43     public:
44         [[nodiscard]] static bool isKnown(const Persistable& o)
45         {
46             return m_ids.find(&o) != m_ids.end();
47         }
48         [[nodiscard]] static bool isKnown(IdType id)
49         {
50             return m_addresses.find(id) != m_addresses.end();
51         }
52     };
53 }
```

```

51     }
52     [[nodiscard]] static IdType getId(const Persistable& o)
53     {
54         if (!isKnown(o))
55         {
56             m_ids[&o] = ++m_id;
57         }
58         return m_ids[&o];
59     }
60     [[nodiscard]] static
61     persistence::Persistable* getAddress(IdType id)
62     {
63         if (!isKnown(id))
64         {
65             return nullptr;
66         }
67         return m_addresses[id];
68     }
69     static bool deregister(const Persistable& o)
70     {
71         if (isKnown(o))
72         {
73             auto id { getId(o) }; // Find id and remember it.
74             m_ids.erase(m_ids.find(&o));
75             if (isKnown(id)) // If id is still known, the corresponding
76                 // entry in m_addresses is deleted.
77                 m_addresses.erase(m_addresses.find(id));
78         }
79         return true;
80     }
81     return false;
82 }
83 static bool doregister(IdType id, Persistable* p)
84 {
85     if (isKnown(id))
86     {
87         return false;
88     }
89     m_addresses[id] = p;
90     m_ids[p] = id;
91     if (id > m_id) // Just continuously updating m_id!
92     {
93         m_id = id;
94     }
95     return true;
96 }
97 static void reset()
98 {
99     m_id = 0;
100     m_ids.clear();
101     m_addresses.clear();
102 }
103 static void updateAssociations()
104 {
105     for (auto& idAddress : m_addresses)

```

```
106         {
107             idAddress.second->updateAssociations();
108         }
109     }
110 private:
111     inline static IdType m_id { 0 };
112     static inline std::map<const Persistable*, IdType> m_ids;
113     static inline std::map<IdType, Persistable*> m_addresses;
114 };
115
116 void Persistable::serialize(std::ostream& os, const bool endl) const
117 {
118     // Since Persistable is an abstract class, it must not
119     // be called with endl = true.
120     auto id { OIDManager::getId(*this) };
121     os << id;
122 }
123
124 std::ostream& operator<<(std::ostream& os, const Persistable& p)
125 {
126     p.serialize(os);
127     return os;
128 }
129
130 void Persistable::deserialize(std::istream& is, const bool endl)
131 {
132     // Since Persistable is an abstract class, it must not
133     // be called with endl = true.
134     IdType id;
135     is >> id;
136     OIDManager::doregister(id, this);
137 }
138
139 std::istream& operator>>(std::istream& is, Persistable& p)
140 {
141     p.deserialize(is);
142     return is;
143 }
144 }
145
146 #endif // PERSISTENCE_HPP
```

The `oid_manager::Object` class in Listing 28 is a polymorphic type that is used as base class for objects processed by the `oid_manager::OIDManager` class. Apart from a virtual destructor, it provides nothing else. This class is replaced by `persistence::Persistable`. This abstract class defines the pure virtual member functions `serialize()` and `deserialize()`, which are the core of persistence. These two member functions must be overridden in a subclass. Since both pure virtual member functions also have implementations, it is essential that any immediate descendant of `Persistable` also calls these implementations.

`Persistable::serialize()` takes a reference of type `ostream` and a second parameter of type `bool`, for which the default value is `true`. This second parameter should never be specified by a client programmer. Since this member function must not modify the receiver object, it is specified as `const`. The implementation of the `Persistable::serialize()` member function cannot be specified inline. Therefore, it is included at the end of the *persistence.hpp* file. This is also necessary because it uses the `persistence::OIDManager` class. In its implementation, it requests its id by calling `OIDManager::getId(*this)`. Then the value of id is sent to the stream named `os`.

`Persistable::deserialize()` takes a reference of type `istream` as the first parameter. The second parameter is of type `bool` with the default value `true`. This second parameter should also never be specified directly by a client programmer. Its implementation simply reads the value for the local variable `id` from the input stream and calls `OID::doregister()` passing `id` and `this`. This static member function of `OIDManager` establishes a link between the ID and the actual address of the object. Interestingly, `Persistable::deserialize()` must not be called with `true` as value for `endline`. Since `persistence::Persistable` is an abstract class, it is impossible to deserialize an exemplar of this class. The newline character must exclusively be extracted from the exemplar of the most derived class, i.e. the `true` type. This is not done in the implementation of `Persistable::deserialize()`. It is therefore not harmful if this method is called with `endline` equal to `true`.

As a small detour, the stream insertion and extraction operators for `Persistable` are considered. The stream insertion operator calls the `serialize()` method for its second argument. This member function has late binding. Therefore, instead of a reference to `Persistable`, the stream operator can be passed an object of any subclass as an argument. This makes it sufficient to overload the output operator for `Persistable`, since it also works for exemplars of all subclasses of `persistence::Persistable`. Finally, it returns the output stream by reference. The stream extraction operator is built in parallel with the stream insertion operator. It calls the `deserialize()` member function for the object passed as a `Persistable` reference before returning the input stream by reference.

Another virtual member function is `updateAssociations()`. It has an empty implementation, which means that it can be overridden if necessary, but it is not mandatory to override it. A subclass that manages associations to other objects must override `updateAssociations()`. The responsibility of this method is to convert the deserialized object IDs into the actual memory addresses of the corresponding objects.

Next, a virtual destructor is defaulted so that `Persistable` becomes a valid base class. Eventually, several static members are defined. `Persistable::separator` specifies the tab character as the separator for the CSV files. Originally, CSV files used the comma to separate data files, as the name *comma separated value* file suggests. In fact, any character can be used to separate data fields. If another character is desired, this is the place to customize it. `Persistable::nextLine()` extracts the remaining content of an

input line, including the newline character. Its implementation uses the `ignore()` member function of an input stream. This member function extracts and discards at most the characters specified as first parameter, including the termination character being specified as second parameter. Since it is not known how many characters actually precede the termination character, here the newline character, a template meta-function defined in `<limits>` is used to determine the maximum number of characters an input stream can theoretically contain. If the termination character occurs as the next character in the input stream, only the termination character is extracted and discarded. `Persistable::nextToken()` consumes separator after having read a data field. Normally, when extracting data with the stream extraction operator `>>`, the character that terminates the data to be extracted is not extracted. Again, the `ignore()` member function of an input stream is used for this task. However, a maximum of 1 character plus the termination character, in this case separator, is extracted and discarded.

It is nonsensical to output memory addresses of objects. They can be completely different when a program or even a single block is re-executed. Therefore integer values of type `IdType`, an alias for `std::size_t`, are used as surrogates for the memory addresses. Of course, they are also deserialized later as after the extraction of all objects is complete, the IDs used by them to link to the associated objects must be replaced with the actual memory addresses of those objects. To allow for a slightly shorter implementation, the same `container<>` variable is used for the IDs as is used to store the memory addresses. As a result, it must be ensured that memory addresses and `IdType` occupy the same number of bytes. This is exactly what is checked by the `static_assert()` statement in lines 38f at compile time.

Now the `persistence::OIDManager` class will be discussed in detail, focusing on the differences compared to the `oid_manager::OIDManager` class. Therefore, it is best to open both files in different editor windows and match their contents.

The static member function `persistence::OIDManager::isKnown()` simply replaces the reference to `const` of type `Object` with a reference to `const` of type `Persistable`. The following static member function `isKnown()` is new. It takes an ID and returns `true`, if the ID is known, and `false` otherwise. In its implementation it looks for the ID in the container `m_addresses`. `m_addresses` is specified as a static data member of the container template `map<>`. It maps an ID as a key to a pointer to `Persistable` as a value. The member `m_addresses` is also new with respect to `oid_manager::OIDManager`.

The static member function `persistence::OIDManager::getId()` is fully equivalent to the static member function `oid_manager::OIDManager::getId()`. Therefore, it is not explained redundantly.

The static member function `persistence::OIDManager::getAddress()` is new. It takes an ID as a parameter and returns the associated memory address of an object. If the ID is unknown, the value `nullptr` is returned. The return value is attributed with `[[nodiscard]]`. It must therefore not be ignored and should always be checked before use to see whether it differs from `nullptr`.

There is also a change in the `deregister()` method. When adapting the `OIDManager` class from Listing 28, the static data member `m_addresses` was introduced. After deleting the entry with the address of the object `o` from `m_ids`, the corresponding entry in `m_addresses` with its ID must also be deleted so that both maps remain consistent. The ID for `o` is therefore determined first and stored in `id`. If `id` is known after deleting the entry from `m_ids`, the entry with it is deleted from `m_addresses`.

The static member function `persistence::OIDManager::doregister()` is new. It takes an ID and a pointer to `Persistable`. Normally, an ID should not be registered more than once, perhaps with different pointer values. Therefore, the implementation first checks, if the ID is already known. If so, it immediately returns `false`. Please note that the return value can be ignored by the caller of the function. Otherwise, the key/value pair is stored in `m_addresses`. In addition, `m_ids` will also be updated. Ideally, this is a superfluous action. However, if a serialization is immediately followed by a deserialization without `persistence::OIDManager::reset()` having been called before, `id` is assigned to `m_id` if its value is greater than that of `m_id`. This is useful because it cannot be ruled out that a new `Persistable` object is created before or after calling `doregister()`. Eventually, the method returns `true`. Preferably, this method should have been called `register()`, since it is the complement of `deregister()`. But `register` is a C/C++ keyword and thus not available as an identifier. It is somewhat unfortunate to have the names `deregister()` and `doregister()`, because they differ by only one character. Therefore, they can be easily confused. If readers want to revise the program and to use it for their own purposes, they could choose more appropriate names.

The `persistence::OIDManager::reset()` method is new. It sets `m_id` to 0 and clears the `m_ids` and `m_addresses` maps. This is necessary to allow more than one serialization and deserialization in a program. For this reason, `reset()` should always be called as first method in a context that serializes or deserializes objects. Otherwise, object addresses and IDs may be confused. An alternative to the `reset()` member function would have been to design and implement `OIDManager` as a class for stateful objects without static members. Then a new exemplar of `OIDManager` would have to be created for each serialization or deserialization context. In addition, this exemplar of `OIDManager` had to be explicitly passed as a parameter to each `Persistable` object. This was not considered to be very maintainable. Therefore, the design of `OIDManager` as an abstract data object was preferred and the `reset()` method was added.

The last method is `updateAssociations()`. It is also a static member function and returns nothing. It uses a range-based for loop to iterate through all `Persistable` pointers stored in `m_addresses`. For each pointer, it calls `updateAssociations()`. Note that this member function is implemented as an empty virtual method in `persistence::Persistable` and must be overridden in any class whose objects may have associations with other objects.

The static data member `m_addresses` is also a `map<>`. It allows the reverse lookup of object addresses by IDs.

PersistenceUnitTests.cpp performs unit tests for *persistence.hpp*. In addition, it contains classes that demonstrate the use of the *OIDManager* class and the implementation of persistence. Since it has more than 250 lines, it is presented only selectively.

The purpose of the *Object* class (Listing 64) is testing.

Listing 64: *Object* class (excerpt from *objects/PersistenceUnitTests.cpp*)

```
21 class Object: public persistence::Persistable
22 {
23     public:
24         void serialize(std::ostream& os,
25                       const bool endl = true) const override
26         {
27             persistence::Persistable::serialize(os, false);
28             if (endl == true)
29             {
30                 os << std::endl;
31             }
32         }
33         void deserialize(std::istream& is, const bool endl = true) override
34         {
35             persistence::Persistable::deserialize(is, false);
36             if (endl == true)
37             {
38                 nextLine(is);
39             }
40         }
41 };
```

It is a public subclass of *Persistable* and overrides the member functions *serialize()* and *deserialize()*. Whether serialization and deserialization work cannot be tested directly for *Persistable* because *Persistable* is an abstract class. Therefore, *Object* must override these two abstract methods in order to create exemplars of *Object*. The implementation of *serialize()* first calls the *persistence::Persistable::serialize()* member function. When calling the *serialize()* member function of a base class, the second argument must always be specified as *false*. Failing to do so may result in a misplaced newline character being inserted into the output stream. Since *Object* has no data members of its own whose values must be sent to the output stream, the final check is whether *endl* equals *true*. If this is the case, this method was supposedly called by the client programmer, and therefore a newline character is inserted into the output stream by calling the static member function *nextLine()* defined in *persistence::Persistable*.

Its *deserialize()* member function first calls *persistence::Persistable::deserialize()* with *false* as the second parameter. Since there are no data members whose values are to be extracted, it then checks whether it was called with

endlineset to true. If so, the newline character is extracted from the input stream by calling `nextLine()`, which is defined as a static member function in `persistence::Persistable`.

Object exemplars are used in the test case labeled "OBJECT", which is not reproduced here.

The `Person` class in Listing 65 corresponds to the `Person` class in Listing 29. Identical parts have been omitted.

Listing 65: `Person` class (excerpt from `objects/PersistenceUnitTests.cpp`)

```

43 class Person: public persistence::Persistable
44 {
45     public:
74     void serialize(std::ostream& os,
75                  const bool endlineset = true) const override
76     {
77         Persistable::serialize(os,false);
78         os << separator
79             << m_name << separator
80             << m_acquaintances.size();
81         for (auto& a : m_acquaintances)
82         {
83             os << separator << persistence::OIDManager::getId(*a);
84         }
85         if (endlineset == true)
86         {
87             os << std::endl;
88         }
89     }
90     void deserialize(std::istream& is,const bool endlineset = true) override
91     {
92         persistence::Persistable::deserialize(is,false);
93         nextToken(is);
94         std::getline(is,m_name,separator);
95         std::size_t numberOfPeople,
96                 id;
97         is >> numberOfPeople;
98         while (numberOfPeople > 0)
99         {
100             nextToken(is);
101             is >> id;
102             m_acquaintances.insert(reinterpret_cast<Person*>(id));
103             --numberOfPeople;
104         }
105         if (endlineset == true)
106         {
107             nextLine(is);
108         }
109     }
110     void updateAssociations() override
111     {

```



```
112         std::set<const Person*> acquaintances;
113         for (auto& id : m_acquaintances)
114         {
115             auto address { persistence::OIDManager::
116                             getAddress(reinterpret_cast<const std::size_t&>
117                                     (id))
118             };
119             if (address == nullptr)
120             {
121                 std::cerr << "Cannot convert id to address." << std::endl;
122                 exit(1);
123             }
124             else
125             {
126                 auto person { static_cast<Person*>(address) };
127                 acquaintances.insert(person);
128             }
129         }
130         m_acquaintances.swap(acquaintances);
131     }
132 private:
41 };
```

As for the `Person` class above, the `serialization()` member function is overridden accordingly. It first calls `Persistable::serialize()` with `false` as the second parameter. This inserts the object ID into the output stream. Then a separator is inserted, followed by `m_name` and another separator. Next the number of acquaintances is inserted by calling `m_acquaintances.size()`. Then a range-based for loop is used to output a separator, followed by the ID of the acquaintance by calling `persistence::OIDManager::getId(*a)`. If `serialize()` was called by the client programmer (by checking that `endline` equals `true`), the receiver object is assumed to be of the most derived type, i.e. the true type `Person`, and a newline character is inserted in the output stream.

The `deserialize()` member function is the inverse of `serialize()`. In its implementation, `persistence::Persistable::deserialize()` is called first, with the second parameter being `false`. Since this method does not extract the separator from the input stream, this must be done next by calling `nextToken(is)`. Now `getline()` is used to read the name from the input stream `is`. The result is placed in the data member `m_name`, and the separator is specified as termination character. This extracts all characters including the separator from the input stream. This way it is also possible to deserialize persons whose names consist of more than one part. Because `getline()` extracts and consumes the separator, the next input operation is not preceded by a call to `nextToken()`. The next value extracted from the input stream is the number of people. Now a while loop is started that runs as long as `numberOfPeople` is greater than 0. If `numberOfPeople` is initially 0, the while loop does nothing. Otherwise, the separator must be extracted by calling `nextToken()`. Then the ID is extracted. This ID is cast to a pointer to `Person`. This is done by a reinterpret cast that interprets the `std::size_t` variable `id` as a reference to

a pointer to `Person`. The result of this cast is immediately passed as an argument to `m_acquaintances.insert()`. In this way, the deserialized IDs of acquaintances are stored as pointers.

Of course, it is illegal to dereference these pointer values because the IDs are not valid memory addresses. Therefore, the IDs must first be converted to valid memory addresses, which is done by `updateAssociations()`, which must be called separately from the client program. A final action in the while loop is to decrement `numberOfPeople` by one so that the while loop eventually terminates. Upon completion of the while loop, it is checked to see if `endline` equals `true`. If so, the receiver object is assumed to have the most derived class as its true type, and the newline character is extracted from the input stream by calling `nextLine(is)`.

Since neither a static cast nor a dynamic cast can be applied to the conversion between `std::size_t` and `persistence::Persistable*`, resorting to `dynamic_cast<>()` is inevitable. The only viable alternative would be to manage a separate container for IDs in each `Persistable` subclass that maintains associations with other `Persistable` objects. This would increase the memory footprint of the corresponding class and also add to its complexity.

Next, the `updateAssociations()` member function must be overridden. Its responsibility is to replace the IDs stored as pointers to `Person` with valid `Person` pointers as soon as the deserialization process is complete. Omitting this will result in an undefined program that is likely to crash on execution.

This member function first defines the local variable `acquaintances`, which is of type `set<const Person*>`, i.e. a `set<>` that manages pointers to `const Person`'s. Next, a range-based for loop iterates over `m_acquaintances`. The loop variable is specified as an auto reference named `id`. Actually, it is a reference to a pointer to a `const Person`. In the range-based for loop, first the pointer to `const` is cast to a reference to `const` of type `std::size_t` by a reinterpret cast. The result of this cast is used to initialize the variable `address`, whose type is specified by using `auto`. It must be emphasized that `persistence::OIDManager::getAddress()` returns a pointer to `Persistable`, not `Person`. This becomes important later. In addition, `address` is no longer `const` because it is created as a new value. Using `auto` to declare a variable removes `const` and reference. Thus, this is a rare case where a new value is created when a reinterpret cast is applied. Next, it is checked whether `address` equals `nullptr`. If it is, an error has occurred and the function sends an error message to `std::cerr` and then terminates the program by calling `exit(1)`. The `std::exit()` function is defined in `<cstdlib>`, which is included at the beginning of the *objects/PersistenceUnitTests.cpp* program. If `address` is different from `nullptr`, a static cast is executed that specifies a pointer to `Person` as the result type. This is necessary because `address` is a pointer to `Persistable`, but not to `Person`. The result is used to initialize the auto variable `person`, which is now a valid pointer to `Person`. Afterwards, `person` is inserted to `acquaintances`.

One might ask why an ID value stored in `m_acquaintances` is not directly replaced by the corresponding pointer to `Person`? The reason is that `m_acquaintances` is specified as a `set<>` and a `std::set<>` is internally ordered. Therefore, the IDs and the memory addresses cannot be stored in the same place in a set without destroying the internal ordering of the set.

After the completion of the range-based for loop, the `swap()` member function is used to swap the contents of `m_acquaintances` and `acquaintances`. The `swap()` function is available as a free function, `std::swap()`, and for all *STL* container templates as a member function. It is a very efficient method to swap the contents of two containers of the same class. Here it is used to quickly replace all invalid values in `m_acquaintances` with the valid memory addresses in `acquaintances` that were just computed. Since the local variable `acquaintances` is destroyed immediately afterwards, this does no harm. The copy-and-swap idiom, briefly mentioned at the end of the Section *Implementation of Correct Assignments for Inclusion Polymorphism*, also builds on the use of `swap()`, more precisely, here `set<>::swap()`, to implement the `assign()` member function in an assignment.

Listing 66 shows the test case named "PERSONS_DESERIALIZE". It illustrates how to test the deserialization of `Person` exemplars.

Listing 66: TEST_CASE("PERSONS_DESERIALIZE") (excerpt from `objects/PersistenceUnitTests.cpp`)

```
246 TEST_CASE("PERSONS_DESERIALIZE")
247 {
248     using namespace persistence;
249     using namespace std;
250
251     Person p1 { "" },
252            p2 { "" },
253            p3 { "" },
254            p4 { "" },
255            p5 { "" },
256            p6 { "" },
257            p7 { "" };
258
259     constexpr auto& s { Persistable::separator };
260     string content { "1"s + s + "Arnet"s + s
261                    + "3"s + s + "2"s + s + "3"s + s + "4\n"
262                    + "4"s + s + "Marn"s + s + "2"s + s + "5"s + s + "1\n"
263                    + "3"s + s + "Fry"s + s + "1"s + s + "6\n"
264                    + "2"s + s + "Kat"s + s + "2"s + s + "3"s + s + "4\n"
265                    + "5"s + s + "Tat"s + s + "1"s + s + "6\n"
266                    + "6"s + s + "Pash"s + s + "0\n"
267                    + "7"s + s + "Pesh"s + s + "1"s + s + "7\n"
268                    };
269
270     OIDManager::reset();
271     istream input { content };
```

```

272 |     input >> p1 >> p2 >> p3 >> p4 >> p5 >> p6 >> p7;
273 |     OIDManager::updateAssociations();
283 |     ostream output;
284 |     OIDManager::reset();
285 |     output << p1 << p2 << p3 << p4 << p5 << p6 << p7 << std::flush;
286 |
287 |     REQUIRE(output.str() == content);
288 | }

```

After creating seven exemplars of `Person` with empty strings as names, a variable of type `std::string` named `content` is defined and initialized with the assumed output of a serialization of the seven `Person` variables. Each person appears on a new line enclosed in double quotes. This is a common way to represent a string that spans multiple lines. All these lines are automatically concatenated into a single C-string.

As the first step of the deserialization process `OIDManager::reset()` is called. This brings the `OIDManager` class into a clean state. After that, an `istringstream` named `input` is created and initialized with `content`. Then, all `Person` exemplars `p1` to `p7` are deserialized from `input` in a single statement. The last mandatory step of deserialization is the call to `OIDManager::updateAssociations()`, which in turn calls the method of the same name for each deserialized object. It should be mentioned that `OIDManager::updateAssociations()` and `Persistable::updateAssociations()` are unrelated methods, because they are defined in separate classes that are not connected in a class hierarchy.

How can it be checked whether the deserialization was correct? In the omitted part after line 273, the original program contains several lines that output the `Person` variables. But this is only for the human eye. As an automatic test, after calling `OIDManager::reset()`, all `Person` variables are serialized to the `ostream` variable `output`. Finally, the `REQUIRE` assertion checks whether `output.str()` is equal to `content`. If the string resulting from the serialization matches the string for deserialization, the test passed successfully.

Many authors advise against `reinterpret_cast<>()`. Indeed, using `reinterpret_cast<>()` should be done with precautions, as demonstrated in Listing 62. When developing libraries or frameworks such as the object persistence example, it can be an indispensable tool in a few places.

The example of persistence again emphasizes the importance of testing. Of course, the debugger can be used to understand how all these classes and their methods work together to achieve the desired result. But even debugging the serialization or deserialization of only seven persistent objects is a time-consuming challenge. In fact, every piece of *persistence.hpp* was developed using accompanying unit tests with *Persistence-UnitTests.cpp* as the result.

1.17. Type Identification

Some programming languages allow to retrieve information about the type of an object at runtime, for example about the members of a class. Sometimes it is even possible to define a new class at runtime and to create exemplars for it. This is referred to as *reflection*—a term with a somewhat narrower meaning—or as *metaprogramming*—a broader term that also encompasses reflection. C++ does not support runtime reflection or metaprogramming, but it does have the typeid operator, which provides information about the static or dynamic type. The identification of the dynamic type is called *Run-time Type Identification* (RTTI for short). C++ also offers a rich toolbox for so-called *static meta-programming*, which allows to obtain a lot of information about types at compile time, so-called *static meta-information*. Similarities and differences between static meta-information and RTTI are explained. Eventually, a code example for the implementation of a so-called *multi-method* using RTTI is presented. Starting with C++26, the standard defines a comprehensive library for compile-time reflection. However, this will not be discussed here, as it is not yet widely available.

1.17.1. Static Type

In the following, meta information about the static type of an object or properties that refer to the static type are discussed.

Some possibilities to obtain information about types are introduced in (U. W. Eisenecker 2022), Section 3.2.1 *Information About Types*. Suppose somewhere in a program it needs to be checked whether the variable `i` is of type `int`, and whether variable `l` is also of type `int`. Of course, these variables must be declared with their type somewhere in the program beforehand. But maybe a type alias is used that is declared somewhere else in the header file of a library, but it is not exactly known which one and where.

In this scenario, a combination of the `decltype` specifier and the `std::is_same_v<>` template variable (contained in the `<type_traits>` header file) can be used to accomplish this task, as Listing 67 illustrates.

Listing 67: objects/TypeTraits.cpp

```
1 // TypeTraits.cpp by Ulrich Eisenecker, March 15, 2024
2
3 #include <iostream>
4 #include <type_traits> // Because of is_same_v<>.
5
6 using namespace std;
7
8 using int_t = int;
9 int_t i { };
10 unsigned long int l { };
11
12 int main()
13 {
```

```

14     cout << "is type of i int? "
15         << boolalpha
16         << is_same_v<decltype(i),int>
17         << endl
18         << "is type of l int? "
19         << is_same_v<decltype(l),int>
20         << endl;
21 }

```

Lines 8 – 10 correspond to the type names and variables to be defined elsewhere. In `main()` it is important to find out if `i` is of type `int` and `l` is of type `int`. The `decltype` specifier inspects the *declared type* of anything that has a type, such as a literal, variable, or expression, without evaluating it. But beware, `decltype` does not remove `const` or `volatile` (the `volatile` keyword and its meaning are not introduced in this book) or a reference in the type declaration of the argument. Therefore `int` and `const int` or `char&` and `char` are not regarded as identical types. The term *declared type* is very important because it refers to the *static type*. As shown before, the *true* or *dynamic type* of an entity may be different from its static type. The result of the evaluation of `decltype`, which is done at compile time, is passed along with `int` as the second parameter to the `is_same_v<>` template variable, which is also evaluated at compile time. The result is `true` if both types are the same, and `false` otherwise. The header files `<type_traits>` and `<limits>` provide a wealth of more information about types. For example, it is possible to check whether the type of `i` is integral, and to retrieve the maximum and minimum values that can be represented by the type of `i`.

Listing 68 presents a program equivalent to the preceding one, but using the `typeid` operator for the same purpose.

Listing 68: objects/TypeID.cpp

```

1 // TypeID.cpp by Ulrich Eisenecker, March 30, 2023
2
3 #include <iostream>
4 #include <typeinfo> // Because of typeid.
5
6 using namespace std;
7
8 using int_t = int;
9 int_t i { };
10 unsigned long int l { };
11
12 int main()
13 {
14     cout << "is type of i int? "
15         << boolalpha
16         << (typeid(i) == typeid(int))
17         << endl
18         << "is type of l int? "
19         << (typeid(l) == typeid(int))

```

```
20 |         << endl;  
21 |     }
```

To use the `typeid` operator, the `<typeinfo>` header must be included. `typeid` can be applied to a type name, a variable, or an expression. The result of applying `typeid` is a reference to `const::std::type_info`. Objects of class `std::type_info` cannot be created, copied, or assigned. Most important for using `std::type_info` objects is the `==` operator. It checks if two `std::type_info` objects are the same. In addition, there is `type_info::name()`, which returns an implementation-specific name, and `type_info::hash_code()`, which returns a hash value specific for each `std::type_info` object. Both methods are almost superfluous, because their return values may be different for different compilers. There is also `type_info::before()`. It returns internal implementation-specific information and is also almost useless in practice.

The program in Listing 69 demonstrates the above mentioned difference between `is_same_v<>` and `typeid` and the specifics of the `std::type_info` member functions.

Listing 69: objects/Specifics.cpp

```
1 | // Specifics.cpp by Ulrich Eisenecker, May 20, 2025  
2 |  
3 | #include <iostream>  
4 | #include <type_traits> // Because of is_same_v<>,  
5 | #include <typeinfo> // Because of typeid.  
6 |  
7 | using namespace std;  
8 |  
9 | class A  
10 | {  
11 |     public:  
12 |         virtual ~A() = default;  
13 | };  
14 |  
15 | class B: public A  
16 | { };  
17 |  
18 | B notEvaluated()  
19 | {  
20 |     cout << "Not evaluated ";  
21 |     return B { };  
22 | }  
23 |  
24 | A& evaluated(A& a)  
25 | {  
26 |     cout << "Evaluated ";  
27 |     return a;  
28 | }  
29 |  
30 | A* alsoEvaluated(A& a)  
31 | {  
32 |     cout << "alsoEvaluated ";
```

```

33     return &a;
34 }
35
36 int main()
37 {
38     B aB;
39     int a { 42 };
40     const int& b { a };
41
42     cout << boolalpha
43           << "const is relevant for is_same_v<>: "
44           << is_same_v<int,const int>
45           << endl
46           << "reference is relevant for is_same_v<>: "
47           << is_same_v<int,int&>
48           << endl
49           << "Use remove_cvref_t<> to remove const, volatile or reference: "
50           << is_same_v<int,remove_cvref_t<decltype(b)>>
51           << endl
52           << "const is NOT relevant for typeid: "
53           << (typeid(int) == typeid(const int))
54           << endl
55           << "type_info.name() returns implementation specific value: "
56           << typeid(std::type_info).name()
57           << endl
58           << "type_info.hash_code() returns implementation specific value: "
59           << typeid(std::type_info).hash_code()
60           << endl
61           << "Operand of typeid is NOT evaluated "
62           << "if the type can be determined statically: "
63           << typeid(notEvaluated()).name()
64           << endl
65           << "Operand of typeid IS evaluated "
66           << "if the type cannot be determined statically: "
67           << typeid(evaluated(aB)).name()
68           << endl
69           << "Operand of typeid IS evaluated "
70           << "if the type cannot be determined statically: "
71           << typeid(*alsoEvaluated(aB)).name()
72           << endl;
73 }

```

To illustrate the conditions under which the typeid operator evaluates its arguments, two classes and three functions are defined. A is a valid base class because it defines a virtual destructor by defaulting it. B is a public subclass of A. Therefore, an exemplar of B can be used if an exemplar of A is expected. The notEvaluated() function returns an exemplar of B as a new value. Therefore, this function is not evaluated by typeid, as the compiler can justifiably assume that the result of the call to notEvaluated() must be of type B. The evaluated() function returns a reference of type A. Consequently, the compiler must evaluate this function and check the result at runtime. This also applies to the alsoEvaluated() function, which returns a pointer to A. When applying typeid

to the dereferenced result of the function call, the entire expression must also be evaluated. All functions send a corresponding message to `cout` as a side effect. In this way, the program is informed about the evaluation of the corresponding function calls (lines 58ff).

At the beginning of `main()`, it is shown that `is_same_v<>` actually takes a reference and `const` into account (volatile is omitted here). To remove any combination of `const`, `volatile` or reference, `remove_cvref_t<>` can be applied—here to `decltype(b)` (line 50).

1.17.2. Dynamic Type

Next, the means of obtaining meta-information about the dynamic type, also known as the true type of an object, are presented.

An object that has a pointer to a `vtbl` is a so-called *polymorphic object*. When `typeid` is applied to a polymorphic object, it returns a reference to a `std::type_info` object that is specific for the dynamic type of its argument. This is called *Runtime Type Identification* (RTTI for short). This cannot be achieved by static meta-programming in C++. Listing 70 demonstrates the application of `typeid` to polymorphic objects.

Listing 70: objects/DynamicType.cpp

```
1 // DynamicType.cpp by Ulrich Eisenecker, March 30, 2023
2
3 #include <iostream>
4 #include <typeinfo> // Because of typeid.
5
6 using namespace std;
7
8 class A
9 {
10     public:
11         virtual ~A() = default;
12 };
13
14 class B: public A
15 { };
16
17 class C: public B
18 { };
19
20 int main()
21 {
22     C c;
23     A& casa { c };
24     cout << boolalpha
25           << "Is casa an A? "
26           << (typeid(casa) == typeid(A))
27           << endl
28           << "Is casa a B? "
29           << (typeid(casa) == typeid(B))
```

```

30     << endl
31     << "Is casa a C? "
32     << (typeid(casa) == typeid(C))
33     << endl;
34 }

```

When this program is compiled and executed, it prints true if it checks for equality of the `std::type_info` for `casa` and `c`. All other comparisons evaluate to false. This is the possibility to accurately determine the dynamic type of an object. Technically, this requires the object to have a `vtbl` pointer, that is, it must be an exemplar of a polymorphic type. It does not work for objects of classes that do not have a `vtbl`.

In Listing 70, if line 11 is converted to a comment, compiling and executing the program produces a completely different result. Now `typeid(casa) == typeid(A)` evaluates to true and the two other checks evaluate to false. The reason for this is that `A` is no longer a polymorphic type because it lacks a `vtbl`. Now `typeid` returns a `type_info` reference for the static type of its operand, which is `A` here. Because of this, `typeid` can produce unexpected results when applied to a non-polymorphic type. Fortunately, the template variable `is_polymorphic_v<>` defined in the `<type_traits>` header can be used to check if a type is polymorphic. If the statement `static_assert(is_polymorphic_v<A>);` were inserted anywhere after line 12, the compiler would issue an error if `A` was not a polymorphic type, i.e. did not have a `vtbl`.

Of course, the dynamic type definition must be available in principle if it is necessary to know whether the dynamic type of an object corresponds to a certain type.

1.17.3. Multi-Methods

So far, member functions whose runtime behavior, or more precisely, whose implementations executed at runtime depend on the true type of the receiver object, have been discussed in detail. In addition, functions whose runtime behavior depends on the true types of two or more objects used as parameters are also conceivable. They are referred to as *multi-methods* and are not directly available as a language feature in C++. The following section explains how they can be implemented equivalently in C++ and provides an example of their use.

A virtual function depends only on the true type of its receiver object. A *multi-method* depends on the true types of more than one of its arguments, passed as references or pointers to objects.

When a virtual member function is called for an object, it actually calls the member function defined for the true type of that object. Therefore, the virtual member function could also be implemented as a free function that takes a reference or a pointer to a base type object that performs a specific action depending on the `std::type_info` reference

for the dynamic type of the object. Virtual member functions are thus just a convenient means, so-called *syntactic sugar*, to facilitate the definition and invocation of functions specific to the dynamic type of an object.

The selection of the function to be executed becomes more difficult when the code to be executed depends on the dynamic type of two or more objects, e.g. for money transfers between accounts in different currencies. In case of money transfer from one EUR account to another EUR account, no actions are required other than debiting the source account with the amount of money and crediting the destination account with the amount of money. However, when transferring money from a EUR account to a USD account, the amount must be debited from the source account and the destination account must be credited with an amount converted according to the current exchange rate. This also applies when money is transferred from a USD account to a EUR account, but now the USD to EUR exchange rate must be applied. Transfers between USD accounts are always done 1:1.

In fact, this problem can be solved by object-oriented means by applying a technique called *double dispatch*. Although this technique is superficially elegant, it is a maintenance nightmare and does not scale. Therefore, it is not explained here.

The alternative is a so-called *multi-method*. This is basically a free function that checks the dynamic type of two or more polymorphic arguments and performs a specific action for each relevant combination of dynamic types.

Listing 71 shows an excerpt of the *MultiMethodUnitTests.cpp* program, which contains an alias for the type to be used for money amounts, as well as the classes *Account*, *AccountEUR*, and *AccountUSD*. This program is unconventional because it contains the code of particular interest here, namely the multi-method plus the classes involved, and the corresponding unit tests. This is only to keep the overall program small and in a single file.

Listing 71: Account classes (excerpt from objects/MultiMethodUnitTests.cpp)

```
22 using money_t = float;
23
24 class Account
25 {
26     public:
27         explicit Account(money_t amount):m_amount { amount }
28         {}
29         void credit(money_t amount)
30         {
31             m_amount += amount;
32         }
33         void debit(money_t amount)
34         {
35             m_amount -= amount;
36         }
37         [[nodiscard]] string balance() const
```

```

38     {
39         ostreamstream os;
40         os << setprecision(2) << fixed
41             << m_amount << ' ' << currencyAbbreviation();
42         return os.str();
43     }
44     virtual ~Account() = default;
45 protected:
46     [[nodiscard]] virtual string currencyAbbreviation() const = 0;
47 private:
48     money_t m_amount;
49 };
50
51 class AccountEUR: public Account
52 {
53 public:
54     using Account::Account;
55 protected:
56     [[nodiscard]] string currencyAbbreviation() const override
57     {
58         return "EUR";
59     }
60 };
61
62 class AccountUSD: public Account
63 {
64 public:
65     using Account::Account;
66 protected:
67     [[nodiscard]] string currencyAbbreviation() const override
68     {
69         return "USD";
70     }
71 };

```

The declaration of the `money_t` alias for the `float` type is a special feature. Normally, the money amount would not be represented as a `float` type due to precision and representation problems. Instead, an integer type or a type for so-called *Binary Coded Decimals* (*BCD* for short) would be used. The `float` type is chosen here for simplicity, and `money_t` provides a single point for choosing a different type for representing money amounts.

`Account` is the abstract base class for all currency-specific accounts. To enable RTTI, `Account` must have a `vtbl`, which is achieved by defaulting a virtual destructor. This is also necessary to make `Account` a valid base class.

The `Account` constructor takes an argument of type `money_t`. In practice, it might be sufficient to specify 0 as the default value for `amount`, which is not the case here. The constructor is marked as `explicit` so that it cannot be called implicitly by accident. The `credit()` and `debit()` methods add or remove the specified amount of money from `Account::m_amount`. The `balance()` method is special in that it returns a `std::string` as the result. For didactic reasons and simplification, the returned string is composed of the

money amount and the currency abbreviation. To avoid errors due to inaccurate representation of the money amount and rounding, an `ostringstream` is used to format the result of the method. The implementation of `balance()` calls the virtual member function `currencyAbbreviation()`. Since this is a pure virtual member function, it must be overridden in any base class. The responsibility of `currencyAbbreviation()` is to return the correct currency abbreviation as a `std::string`. It must be emphasized that `currencyAbbreviation()` is declared as a protected member function. This means that it cannot be called from outside `Account`. One could debate whether this is really appropriate here. But it shows that virtual functions do not necessarily need to be declared as public. `Account::m_amount` represents the balance of the account.

`AccountEUR` and `AccountUSD` have very simple implementations. For convenience, the base class constructor is imported using `Account::Account;`. Only the `currencyAbbreviation()` method is overridden in the protected section so that it returns the correct currency abbreviation.

The Listing 72 contains the definition of the `transfer()` function.

Listing 72: `transfer()` function (excerpt from `objects/MultiMethodUnitTests.cpp`)

```
127 void transfer(Account& source, Account& destination, money_t amount)
128 {
129     money_t convertedAmount { amount };
130     if (typeid(source) == typeid(AccountEUR))
131     {
132         if (typeid(destination) == typeid(AccountUSD))
133         {
134             convertedAmount *= 1.09;
135         }
136     }
137     if (typeid(source) == typeid(AccountUSD))
138     {
139         if (typeid(destination) == typeid(AccountEUR))
140         {
141             convertedAmount *= 0.92;
142         }
143     }
144     source.debit(amount);
145     destination.credit(convertedAmount);
146 }
```

It accepts two parameters of type `Account`, which are passed as references. The third parameter of type `money_t` is passed by value. Its implementation first initializes `convertedAmount` with the value of `amount`. If necessary, `convertedAmount` is multiplied by the respective exchange rate. For each account type an `if` statement follows. The first `if` statement refers to `AccountEUR`. In its condition, the operator `typeid` is applied to both the source account and the type `AccountEUR`. The resulting references to `const type_info` objects are then checked for equality. If they are equal, the nested `if` statement is executed. This checks whether the target is of type `AccountUSD`. If it is, `convertedAmount`

is multiplied by the corresponding exchange ratio. It is not necessary to check if the target is of type `AccountEUR`, because in this case the exchange ratio is 1:1 and nothing needs to be done.

The second `if` statement has exactly the same structure. It checks if the type of the source account is `AccountUSD`. Then the nested `if` statement checks, if destination is of type `AccountEUR`. If it is, `convertedAmount` is adjusted according the exchange ratio. If both source and destination are of type `AccountUSD`, nothing needs to be done. Finally, the source account is debited with the amount, and the `convertedAmount` is credited to the destination account.

In practice, the implementation of multi-methods would not hard-code the exchange ratios. Rather, it would retrieve the desired exchange ratio from a cache, which in turn would be regularly updated by an appropriate web service.

Looking closely at the code, one notices that the `typeid` operator is used redundantly. When adding more account classes, it would be better to apply the `typeid` operator twice to source and destination at the beginning and initialize the references to `const` of type `type_info` to preserve the references to the corresponding `type_info` references for later use.

Adding a new account class does not affect the other account classes already defined. The interface of the `transfer()` function is also not changed by this. However, the implementation of the `transfer()` function must be adjusted. First, a new nested `if` statement must be provided in each of the existing top-level `if` statements to provide for the case where the destination account belongs to the newly introduced account class. Second, another top-level `if` statement must be added for the source account of the new class type, including nested `if` statements for all cases where the source account type is different from the destination account type. This is a schematic adaptation. The big advantage is that there is exactly one point in the entire program that needs to be adapted to the new account class. Of course, the file containing the `transfer()` implementation must be recompiled, and the entire application must be relinked. However, this is a minimal effort that does not affect the runtime efficiency and the locality of the multi-method definition.

In practice, there would not be a single monolithic program containing all the account classes, the `transfer()` function, and the test cases or a `main()` function. There would be a header file for the `Account` class and the prototype of the `transfer()` function. There would be at least one implementation file for the `Account` class and the `transfer()` function. If the implementations of `Account` and `transfer()` were too large, it would be advisable to split them into two separate implementation files. Each account subclass would be kept separate. If they were simple, as here, only one header file per class would suffice; otherwise, one header file and one implementation file per class would be appropriate. The test cases or the `main()` function would be in a separate file.

The test cases in *MultiMethodUnitTests.cpp* are not reproduced here, but they are included in the accompanying material in the *sources/objects* directory. Two of them test the AccountEUR and AccountUSD classes in isolation, and the third tests the transfer() function. This test case uses SECTIONS to simplify the implementation of the test case.

It should be noted that the Account::balance() function was developed for didactic reasons. In practice, it would either return only a numeric value, and there would be a separate function for the currency abbreviation, or it would return a special object containing a numeric value plus the currency abbreviation.

1.18. Exceptions

In the preceding code examples, there are many places where errors can occur. For example, in Listing 50 the UrnOR constructor checks for invalid values for the constructor arguments and exits the program if they are invalid. In many cases, terminating the program or, even worse, doing nothing at all is not the best solution. It is more desirable to inform the program about the specific error situation and give it a chance to fix the error and try again or to perform another appropriate action—if possible.

Exceptions are the de facto standard for error handling. In C++ they are used frequently. For example, if a dynamic_cast<> to a reference fails, an exception of the type bad_cast is thrown (Section *Syntactic Aspects of Using dynamic_cast<>()*). But there are also error situations which do not raise an exception, e.g. dereferencing the value nullptr results in undefined behavior, and dividing a floating point variable by 0.0 sets some error state in the runtime library and yields a special value representing infinity.

When an *error is detected*, an *exception is thrown*. This exception must be caught by a suitable *exception handler* in a superordinate caller of the corresponding (member) function. If there is no function that catches the specific exception, it is caught by a general exception handler that is part of the runtime library and catches all exceptions. The program shown in Listing 73 illustrates this.

Listing 73: objects/ExceptionBasics.cpp

```
1 // ExceptionBasics.cpp by Ulrich Eisenecker, May 21, 2024
2
3 #include <iostream>
4 #include <stdexcept>
5
6 using namespace std;
7
8 double division(int n,int d)
9 {
10     if (d == 0)
11     {
12         throw overflow_error { "Division by zero" };
13     }
14     return static_cast<double>(n) / d;
```

```

15 }
16
17 int main()
18 {
19     int numerator { }, denominator { };
20     cout << "Numerator: ";
21     cin >> numerator;
22     cout << "Denominator: ";
23     cin >> denominator;
24     double value;
25     bool exceptionCaught { false };
26     try
27     {
28         value = division(numerator, denominator);
29     }
30     catch (const out_of_range& e)
31     {
32         cerr << "Caught out_of_range exception: " << e.what() << endl;
33         exceptionCaught = true;
34     }
35     catch (const overflow_error& e)
36     {
37         cout << "Caught overflow_error exception: " << e.what() << endl;
38         exceptionCaught = true;
39     }
40     if (!exceptionCaught)
41     {
42         cout << "Value = " << value << endl;
43     }
44 }

```

At the beginning the header file `<stdexcept>` is included. It contains all exception classes defined in the standard library. These exception classes are arranged in a class hierarchy with `std::exception` as the base class (*std::exception* - *Cppreference.Com* n.d.). In addition to a standard constructor and a copy constructor, `exception` defines a virtual destructor, the assignment operator, and a `what()` member function. The latter returns an explanatory string of type `const char *`, i.e. not a `std::string`. The `exception` class is the parent class of `std::runtime_error`, which adds two constructors that take either a reference to `const std::string` or a pointer to `const char` as argument. Each argument should provide an explanation for the exception thrown. The `runtime_error` class represents an error that is not caused by the program itself and cannot be easily predicted. In the above example, `std::overflow_error` is chosen as the exception class. It is a direct subclass of `std::runtime_error` and represents an arithmetic overflow error. Dividing a number by zero gives an infinite result. Therefore, it is appropriate to throw an `overflow_error` when the denominator is 0.

In fact, it is possible to throw exceptions of fundamental types and define specific exception classes or derive them from the existing ones. **However, it is recommended to use one of the exception classes included in the C++ standard if possible.**

The `division()` function checks whether `d` is 0. If so, an `overflow_error` is thrown. The statement `throw overflow_error { "Division by zero" };` creates an anonymous `overflow_error` exemplar with "Division by zero" as argument and throws it immediately, as specified by the keyword `throw`. It is not necessary to first create a variable of type `overflow_error` and then throw it, since this variable cannot be accessed after throwing.

Immediately after throwing an exception, the normal control flow is interrupted. Therefore `division()` does not perform the division in this case and returns no result.

The `main()` function inputs the variables `numerator` and `denominator` and defines the variable `value` without initializing it. The `bool` variable `exceptionCaught` is initialized to `false`. In a catch handler, it is assigned the value `true` to indicate to the program part following the catch handlers that an exception was caught.

Then a so-called *try block* is defined with the keyword `try`. Please note the mandatory curly brackets. **A try block must always specify a compound statement.** It contains code whose execution can cause an exception to be thrown. Once an exception is thrown, the control flow is passed to an *exception handler* that can catch an exception of the specified type. It is possible to specify multiple catch handlers after a try block. Care must be taken to arrange the catch handlers in a strict order from more specific to more general exception types. The `std::out_of_range` exception inherits from `std::logic_error`, which is itself a child of `std::exception`. Therefore, `std::out_of_range` and `std::overflow_error` are not in a subtype relationship and can appear in any order in catch handlers. Since the exception thrown by `divide()` is neither a `std::out_of_range` exception nor an exemplar of one of its base classes, this catch handler cannot catch it. The following catch handler specifies an `overflow_error` exception. Therefore, it will catch the exception thrown by `divide()`.

An important detail is that the catch handler catches an exception as a reference to `const`. This is for two reasons. The first is that the catch handler should not modify the caught exception object. The second reason is that the true type, also known as the dynamic type, of the caught exception exemplar should be preserved. If the catch handler were to catch the exception by value, a new exemplar would be created with the specified type. After that, it would be impossible to recover the true type of the original exception exemplar. **An exception should generally be caught as a reference to `const`.**

When the second catch handler has caught the exception thrown by `divide()`, It outputs a corresponding message plus the result of the call to `e.what()`. It then assigns the value `true` to `exceptionCaught`. Please note that the output is on `cerr` and not on `cout`. This emphasizes that a situation caused by an error is handled.

When an exception handler completes, the program continues with the next statement following the last catch handler belonging to the respective try block.

Here the `if` statement checks whether no exception has occurred. If so, it outputs the result of the division, which now has a defined value. Otherwise, no output is produced and `main()` terminates.

Now the program can be varied a little. Instead of throwing an exception of type `overflow_error` in `divide()`, an exception of type `bool` is thrown, namely `throw bool { false };`. After compiling, executing and entering 0 as the denominator value, the program issues a message that the program has been terminated because an exception of type `bool` has occurred. The catch handlers provided in `main()` are not capable of catching a `bool` exception. Therefore, it is caught by the runtime library's general catch handler, which catches an exception of any type.

The preceding program demonstrates the basic use of exceptions and their handling. Before going into further details, it is first explained for what purpose exceptions should *not* be used.

1.18.1. Bad Use of Exceptions

Many modern programming languages provide an exception mechanism to deal with errors. But it is easy to use exceptions in situations where they are not appropriate. This is discussed below.

Listing 74 shows a variant of the program in Listing 73.

Listing 74: `objects/ExceptionBadUse.cpp`

```

17 int main()
18 {
19     int numerator { }, denominator { };
20     cout << "Numerator: ";
21     cin >> numerator;
22     double value;
23     bool exceptionCaught { false };
24     do
25     {
26         cout << "Denominator: ";
27         cin >> denominator;
28         exceptionCaught = false;
29         try
30         {
31             value = division(numerator, denominator);
32         }
33         catch (const overflow_error& e)
34         {
35             cout << "Caught overflow_error exception: " << e.what()
36                 << "\nPlease retry." << endl;
37             exceptionCaught = true;
38         }
39     } while (exceptionCaught);
40     cout << "Value = " << value << endl;
41 }
```

All differences occur in `main()`. First, the catch handler for the `std::out_of_range` exception has been removed, since it contains code that can never be reached in this program. Second, a do-while loop has been introduced. It is executed as long as `exceptionCaught` is true. The statements for inputting the value for denominator have been moved to the beginning of the loop body. Then `exceptionCaught` is assigned the value false. When the catch handler becomes active, it reports the error, informs the user to try again, and assigns true to `exceptionCaught`. In this case, the condition of the do-while loop is true and the loop body repeats. Please note that the output related to the caught exception is now on `cout`, as it is considered a normal interaction with the user.

In this way, the triad of try, throw and catch implements code that interacts with the user to make them enter valid values for denominator.

However, this is an abuse of the exception mechanism!

From the point of view of the `divide()` function it is indeed an unexpected situation if it is called with 0 for the parameter `d`, and it cannot take appropriate action in this situation. Hence, it is adequate for `divide()` to throw an exception when `d` is 0. But this is different for the code that calls `divide()`. Here, it is obvious that `divide()` should not be called with a value of 0 for `d`. In addition, the client code can take appropriate measures to prevent this situation. The program in Listing 75 shows a rewritten `main()` function without a try block.

Listing 75: `objects/ExceptionCorrectUse.cpp`

```
17 int main()
18 {
19     int numerator { }, denominator { };
20     cout << "Numerator: ";
21     cin >> numerator;
22     double value;
23     do
24     {
25         cout << "Denominator: ";
26         cin >> denominator;
27         if (denominator == 0)
28         {
29             cout << "Denominator must not be equal to 0."
30                  << "\nPlease retry." << endl;
31         }
32     } while (denominator == 0);
33     value = division(numerator, denominator);
34     cout << "Value = " << value << endl;
35 }
```

Compared to Listing 75 `main()` without a try block is obviously shorter and easier to understand. It also has a small performance advantage, since exception handling causes minimal execution time loss when an exception is thrown.

Thorough consideration should always be given to whether using a `try` block and catching exceptions is an appropriate option.

1.18.2. Stack-Unwinding

As mentioned earlier, throwing an exception interrupts the normal flow of control. This has other consequences. Any object that was *fully constructed* before the exception was thrown is automatically destroyed, and the function call stack is reset to the point where a catch handler intercepts the thrown exception. Together, these actions are referred to as *stack unwinding*. The attentive reader may have noticed the term *fully constructed object*. It will be explained later. Next, stack unwinding is demonstrated using the program shown in Listing 76. It also introduces some new features in the context of logging, which are also explained.

Listing 76: objects/StackUnwinding.cpp

```

1 // StackUnwinding.cpp by Ulrich Eisenecker, August 22, 2023
2
3 #include <iostream>
4 #include <stdexcept>
5 #include <string>
6 #include <source_location>
7 using namespace std;
8
9 class Logger final
10 {
11     public:
12         Logger(string fu = source_location::current().function_name(),
13              string fi = source_location::current().file_name()):
14             m_functionname { fu },
15             m_filename { fi }
16         {
17             cout << "Entering " << m_functionname
18                  << " (" << m_filename << ")."
19                  << endl;
20         }
21         ~Logger()
22         {
23             cout << "Leaving " << m_functionname
24                  << " (" << m_filename << ")."
25                  << endl;
26         }
27     private:
28         string m_functionname,
29             m_filename;
30 };
31
32 class Object
33 {
34     public:
35         Object(string name = "anonymous"s,
```

```
36         unsigned long line = source_location::current().line(),
37         unsigned long column = source_location::current().column():
38             m_name { name }
39     {
40         cout << m_name << " created in line " << line
41             << " column " << column << '.' << endl;
42     }
43     virtual ~Object()
44     {
45         cout << m_name << " deleted." << endl;
46     }
47     private:
48         string m_name;
49 };
50
51 void top()
52 {
53     Logger l;
54     Object A { "A"s };
55     throw exception { };
56     Object B { "B"s };
57 }
58
59 void middle()
60 {
61     Logger l;
62     Object C { "C"s };
63     top();
64     Object D { "D"s };
65 }
66
67 int main()
68 {
69     Logger l;
70     try
71     {
72         Object E { "E"s };
73         middle();
74         Object F { "F"s };
75     }
76     catch (...)
77     {
78         cout << "Unspecified exception caught" << endl;
79     }
80 }
```

At the beginning the header file `<source_location>` is included. It provides the `std::source_location` class. Its static member function `current()` creates a new `source_location` exemplar corresponding to the call site. Note, that `source_location` has no public constructor. New exemplars can only be created through a call to

`current()`. `source_location` has four (non-static) member functions: `file_name()` returns the file name, `line()` returns the line number, `column()` returns the column number, and `function_name()` returns the name of the function.

Then `Logger` is defined as `final` class (see Section *final*). The `Logger` class was introduced in (U. W. Eisenecker 2022), Section 6.5.2 *Logger Class*. Its constructor has two arguments, each with a default value. The default value for the parameter `fu` (short for *function* name) is `source_location::current().function_name()`. This evaluates to the function name of the caller site. In this case, the caller site is the function that defines a `Logger` exemplar. For the `fi` parameter (short for *file* name) the default `source_location::current().file_name()` evaluates to the file name containing the function definition. The values of the arguments are used to initialize the corresponding member variables. They are used to report when a function is entered and when it is exited.

After that, the `Object` class is defined. Its purpose is to report when an object is created and when it is deleted. Its constructor has three arguments, for each of which a default value is specified. The first argument contains a name for the object being created. If no name is specified, "anonymous" is taken as the default value. The second argument, `line`, is initialized with `source_location::current().line()` as the default value, which evaluates to the line number of the file in which the object is created. For the third argument, `column`, the result of calling `source_location::current().column()` is used as default value, namely the column in the file where the declaration of the object ends. Only the value of the parameter `name` is preserved in the member variable `m_name`. The values of `line` and `column` are output in the constructor body. Reporting them again in the destructor is pointless because the line and column numbers at which the destructor is called are not accessible. The constructor and destructor report the creation and the deletion of a particular object.

The `top()` function is called last in this program, hence its name: it becomes the *top entry* of the function call stack. First, the `Logger` exemplar `l` is declared. Then, the exemplar `A` of the class `Object` is created. In its constructor, the `std::string` literal "A"s is passed as its name. The third statement unconditionally throws an exception of type `exception`. The constructor of the exception class does not take an explanatory string, so the exception is created without it. The last statement of the `top()` function is the declaration of an `Object` exemplar named `B`. According to the runtime output of the program, this exemplar is never created because of the exception thrown earlier.

The `middle()` function also declares a `Logger` exemplar to keep track of when the function is entered and when it is exited. Next, the `Object` exemplar `C` is declared. Then `top()` is called. Because an exception is thrown in `top()`, the `Object` exemplar `D` is never created in `middle()`.

A `Logger` exemplar is also created in `main()`. The `try` block creates an `Object` exemplar named `E` and then calls `middle()`. The following statement `Object F { "F"s };` is not executed because an exception was previously thrown.

The catch handler introduces a new feature: it catches the ellipsis `...`. This catches an exception of arbitrary type. Therefore, the type of the exception is not known and no reference to `const` for an exception type is specified. Actually, an exception of type `std::exception` was thrown, but since the exception is caught with the ellipsis, this knowledge is not available at this point. Since the ellipsis catches all exceptions, it should be specified as the last catch handler in a series of catch handlers.

Figure 27 shows the output when running the compiled program.

Figure 27: Output of program `objects/StackUnwinding.cpp`

```
Entering int main() (StackUnwinding.cpp).
E created in line 72 column 23.
Entering void middle() (StackUnwinding.cpp).
C created in line 62 column 20.
Entering void top() (StackUnwinding.cpp).
A created in line 54 column 20.
A deleted.
Leaving void top() (StackUnwinding.cpp).
C deleted.
Leaving void middle() (StackUnwinding.cpp).
E deleted.
Unspecified exception caught
Leaving int main() (StackUnwinding.cpp)
```

As the output demonstrates, `main()` is entered, `E` is created, `middle()` is entered, `C` is created, `top()` is entered and `A` is created. Then the exception is thrown. `A` and `l` in `top()` are a fully constructed objects. Therefore, their destructors are called in the reverse order of the declaration of the objects during stack unwinding. Since the creation of `B` is not reported, it can be reasonably assumed that no subsequent part of `top()` was executed after the exception was thrown. The same is true for `l` and `C`, which were created in `middle()`. Both are fully constructed objects. Therefore, they are deleted upon stack unwinding. The creation of `D` is not reported, which means that no further statement is executed in this function after the exception is thrown.

Stack unwinding aims at leaving the program in a well-defined state as far as possible in an automated way. In particular this means that the function call stack is reset to the function that catches the exception and that all objects are destroyed that were previously fully constructed.

Apart from this, it is the programmer's responsibility to ensure that any state change that cannot be undone by stack unwinding is reversed, usually in the appropriate catch handler. Ideally, a `try` block behaves like a database transaction: either it succeeds completely, and the state changes it makes are preserved, or, if only a portion fails, the entire transaction fails and is rolled back. This cannot always be achieved. Then the catch handler should at least finish the program in a defined way, i.e. return dynamic resources (e.g. flushing and closing files) and provide a comprehensive error report.

Stack unwinding and the programmer cannot adequately deal with objects that are not fully constructed. Such incompletely constructed objects are dealt with below.

1.18.3. Incomplete Objects

So far, only objects that are completely constructed have been presented. In the following, an example of an *incomplete object* is presented and the resulting consequences are discussed.

The program in Listing 77 again uses the `Object` class from Listing 76.

Listing 77: objects/IncompleteObject.cpp (excerpt)

```

1 // IncompleteObject.cpp by Ulrich Eisenecker, August 23, 2023
2
3 #include <iostream>
4 #include <stdexcept>
5 #include <string>
6 #include <source_location>
7 using namespace std;
8
28 class IncompleteObject: public Object
29 {
30     public:
31         IncompleteObject(): m_pointer { nullptr }
32         {
33             m_pointer = new Object { "DynamicObject"s };
34             throw exception { };
35             cout << "We will not see this." << endl;
36         }
37         IncompleteObject(const IncompleteObject& ) = delete;
38         IncompleteObject& operator=(const IncompleteObject& ) = delete;
39         ~IncompleteObject() override
40         {
41             delete m_pointer;
42         }
43     private:
44         Object *m_pointer;
45 };
46
47 int main()
48 {
49     try
50     {
51         IncompleteObject io;
52     }
53     catch (...)
54     {
55         cout << "Unspecified exception caught" << endl;
56     }
57 }

```


The `IncompleteObject` class is a subclass of `Object`. It defines a pointer to `Object` as a data member, which is initialized to `nullptr` in the initializer list of the constructor. Because the initializer list does not explicitly call the base class constructor, namely `Object::Object()`, it is called implicitly by defaulting the name argument to "anonymous"s. This can be easily changed by inserting `Object { "anonymous"s }` as the first element after the colon followed by a comma in the initializer list.

The constructor of `IncompleteObject` creates a new dynamic `Object` exemplar and assigns the resulting pointer to the data member `m_pointer`. Then an exception is thrown unconditionally. The following statement, which produces output, is never executed because of the previously thrown exception.

Since `IncompleteObject` owns a dynamic resource, its copy constructor and assignment operator are deleted. Thus, no extra action needs to be taken for these two cases.

Next, the destructor is defined with the `override` keyword. The destructor applies the `delete` operator to `m_pointer` to release the dynamically created object.

The `main()` function has a `try` block with only one statement that declares the variable `io`. The `catch` block catches exceptions of all types by using the ellipsis.

After compiling and running the program it produces the output shown in Figure 28.

Figure 28: Output of program `objects/IncompleteObject.cpp` (excerpt)

```
anonymous created in line 31 column 47.  
DynamicObject created in line 33 column 52.  
anonymous deleted.  
Unspecified exception caught
```

As can be seen, the inherited subobject of type `Object` is created with the name "anonymous". Then the first statement of the constructor of `IncompleteObject` is executed, which is obvious because it is reported that an `Object` with the name "DynamicObject" was created. The next statement throws an exception. And now comes the amazing point: the destructor of the base class `Object` is called, but nothing else. Especially the destructor of `io` is *not* called—because it is an incomplete object! Otherwise the deletion of the `Object` called "DynamicObject" would be reported. The only fully constructed object is the inherited subobject of `IncompleteObject`. Consequently, its destructor is called, but not the destructor of the derived class `IncompleteObject`.

To avoid an incomplete object, it should never directly own a dynamic resource if a constructor of it can throw an exception after acquiring the dynamic resource. Therefore, a dynamic resource should always be acquired by a so-called *wrapper class*, which throws an exception only if the resource cannot be successfully acquired in a constructor. This way, no exception is thrown if the resource is successfully acquired, and the destructor of the wrapper class reliably releases the dynamic resource. This principle is called *Resource Acquisition Is Initialization* (RAII for short), see (RAII - Cplusplus.com n.d.)

for more information. This is also a reason why the C++ standard includes special class templates for wrapping dynamic resources, especially memory pointers. They are called *smart pointers* and are discussed in Section *Smart Pointers*.

1.18.4. More on Exception Handling

Much more could be written about the handling of exceptions. Some selected features will be treated only superficially in the following, namely the specification of exceptions for (member) functions, special functions related to exception handling, nested catch handlers, exception safety, and some recommendations for the use of exception handling.

Functions and member functions can have an exception specification. If no exception specification is given, this means that a (member) function can throw any exception. This can also be specified as `noexcept(false)`. Thus, the function declarations `void f();` and `void f() noexcept(false);` are equivalent. Conversely, it can be specified that a function does *not* throw any exception. This can be done with `noexcept` or `noexcept(true)`. Therefore, the function declarations `void f() noexcept;` and `void f() noexcept(true);` are identical. Both functions promise *not* to throw an exception. This is different for destructors. Since **a destructor should never throw an exception** it is implicitly specified as `noexcept(true)`. The exception specification is part of the function type, but not part of the function signature. It is not possible to overload two functions that differ only by their exception specification. Earlier versions of the C++ standard allowed more detailed exception specifications. They are now all deprecated or have been removed. ***It is recommended not to use any explicit exception specification.***

When exceptions are thrown in situations where they should not occur, for example in a function with `noexcept`, the special error handling function `std::terminate()` (defined in the `<exception>` header) is called. It is also possible to replace this special error handling function by calling `std::set_terminate()` and passing a pointer to a self-defined error handling function. This is not shown here.

Nested catch handlers are one option for fine-grained handling of an exception. When an exception is caught, it is handled as best as reasonably possible by the most specific catch handler. Then the exception is thrown again with the `throw;` statement. This gives a surrounding, less specific catch handler the opportunity to also catch that exception and handle it again in a specific way, and so on. In this design, an exception is handled *piecemeal*. Although this design is well thought out, a corresponding program can be difficult to understand and maintain. Therefore, careful consideration should be given to the extent to which exceptions should be handled by nested catch handlers.

A further topic is it to write functions, classes and programs *exception-safe*, whereby several levels of *exception safety* can be distinguished. The most ambitious level of exception safety corresponds to the *transaction* concept in the data base design, which was already briefly mentioned. More to this topic will not be described here.

For those who are interested, both, (Breymann 2020) and (Gregoire 2020) devote a chapter to exception handling, but without presenting all the details.

As a general recommendation, exceptions should be used consistently but with restraint. The handling of exceptions should not implement normal program logic. In addition, too fine-grained try blocks should be avoided. Wrapping each statement in a separate try block results in a program that is difficult to read and hard to maintain. When exceptions are caught, the main effort should be to terminate the program in a defined way, i.e. to free dynamic resources and write meaningful information to an error log file. Attempting to write programs at the highest level of exception safety is very ambitious and potentially time consuming and expensive.

A completely different approach to error handling in C++ is inspired by functional programming, namely the use of `optional<>` and `expected<>`. They are introduced in Section *The Functional Programming Paradigm*.

1.19. Smart Pointers

In (U. W. Eisenecker 2022), sections *Pointers* and following, pointers, or more precisely, *raw pointers*, were introduced and some risks of their use were pointed out. In the Section *Incomplete Objects* it was demonstrated that the use of dynamic resources such as pointers can lead to an incomplete object and thus to code that is not exception-safe.

Smart pointers help to mitigate these risks. They are so-called *wrapper classes* that manage dynamic resources—particularly raw memory pointers—and eliminate or minimize the dangers associated with the use of raw pointers. The C++ standard includes the class templates `std::unique_ptr<>`, `std::shared_ptr<>`, and `std::weak_ptr<>`, which offer the functionality of various smart pointers, each of which serves a specific purpose. The smart pointer classes overload the operators `*` and `->` as member functions. This allows them to be used like raw pointers, i.e. dereferencing or accessing members using a raw pointer. The other member functions of smart pointers are mostly specific to each smart pointer class template. How and for what purpose smart pointers can be used is explained below.

1.19.1. Exclusive Ownership

In most cases, an object is the exclusive owner of a dynamic resource. Of course, the dynamic resource can be acquired in a constructor and released in the destructor. But much care must be taken when implementing objects with dynamic resources as explained in the Section *Object With Dynamic Resource* and its sub sections. However, all the measures described there cannot guarantee exception safety. When it comes to dynamic memory, the `std::unique_ptr<>` class template defined in the `<memory>` header helps to avoid incomplete objects. The dynamic resource managed by a `unique_ptr<>` is deleted when the `unique_ptr<>` to which it belongs goes out of scope or is assigned. A `unique_ptr<>` supports move semantics for construction and assignment, but copy

construction and copy assignment are disabled. To create a unique pointer, the use of the free function template `std::make_unique<>()` is recommended. Detailed information about `unique_ptr<>` can be found in (*std::unique_ptr* - *Cppreference.Com* n.d.). In addition, some information is given about objects that can change their behavior, about effectively decoupling a class from its implementation, and on explicitly transferring the ownership of a dynamic resource.

1.19.1.1. Object With Altering Behavior

Since the C++ standard library provides a variety of container templates, it is rarely necessary to allocate memory via raw pointers to deal with large objects or a number of objects not known before runtime. Therefore, a different application scenario is chosen to motivate the use of `std::unique_ptr<>`. The program shown in Listing 78 implements a *Butterfly* class that covers the entire life cycle of a *butterfly* from *egg* to *caterpillar* to *chrysalis* and finally to *imago*. Here, a `unique_ptr<>` is used to safely enable inclusion polymorphism.

Listing 78: objects/Butterfly.cpp

```

1 // Butterfly.cpp by Ulrich Eisenecker, May 27, 2025
2
3 #include <iostream>
4 #include <memory>
5 #include <string>
6 #include <typeinfo>
7 using namespace std;
8
9 // If verbose is initialized to true, the program will report
10 // information about which member function has been called,
11 // as well as additional bookkeeping information in the terminal
12 // window.
13 constexpr bool verbose { false };
14
15 // Abstract base class for Egg, Caterpillar, Chrysalis, and Imago.
16 // It serves as an interface that abstracts from the
17 // specific development stage of a Butterfly exemplar.
18 // It must be defined before Butterfly, as Butterfly::whatAmI()
19 // calls the member function with the same name for the Stage
20 // pointer m_stage.
21 class Stage
22 {
23 public:
24     Stage()
25     { if constexpr (verbose)
26       {
27         cout << "After construction = " << ++m_n << endl;
28       }
29     }
30     virtual ~Stage()
31     { if constexpr(verbose)
32       {

```

```
33         cout << "After destruction = " << --m_n << endl;
34     }
35 }
36 virtual unique_ptr<Stage> next() const = 0;
37 virtual string whatAmI() const = 0;
38 private:
39     static inline size_t m_n { 0 };
40 };
41
42 // This is the class to be instantiated by the client programmer.
43 // It internally manages a stage pointer that refers to the specific
44 // development stage of a butterfly and thus enables stage-specific behavior.
45 class Butterfly
46 {
47 public:
48     Butterfly();
49     bool next();
50     string whatAmI() const
51     {
52         return m_stage->whatAmI();
53     }
54 private:
55     unique_ptr<Stage> m_stage;
56 };
57
58 class Egg: public Stage
59 {
60 public:
61     unique_ptr<Stage> next() const override;
62     string whatAmI() const override
63     {
64         return "Egg"s;
65     }
66 };
67
68 class Caterpillar: public Stage
69 {
70 public:
71     unique_ptr<Stage> next() const override;
72     string whatAmI() const override
73     {
74         return "Caterpillar"s;
75     }
76 };
77
78 class Chrysalis: public Stage
79 {
80 public:
81     unique_ptr<Stage> next() const override;
82     string whatAmI() const override
83     {
84         return "Chrysalis"s;
85     }
86 };
87
```

```
88 class Imago: public Stage
89 {
90     public:
91         unique_ptr<Stage> next() const override;
92         string whatAmI() const override
93         {
94             return "Imago"s;
95         }
96 };
97
98 // The Butterfly constructor must be defined after the
99 // definition of Egg, as it creates a dynamic Egg exemplar.
100 Butterfly::Butterfly(): m_stage { make_unique<Egg>() }
101 {
102 }
103
104 // Butterfly::next() must be defined after the
105 // definition of Imago, as it uses the complete
106 // type of Imago.
107 bool Butterfly::next()
108 {
109     auto& stage { *m_stage }; // To avoid a warning in the next line.
110     if (typeid(stage) != typeid(Imago))
111     {
112         m_stage = m_stage->next();
113         return true;
114     }
115     return false;
116 }
117
118 // The following member functions must be defined after the
119 // definition of the classes representing the different stages,
120 // as each of these member functions uses a particular subclass
121 // of Stage.
122
123 unique_ptr<Stage> Egg::next() const
124 {
125     return make_unique<Caterpillar>();
126 }
127
128 unique_ptr<Stage> Caterpillar::next() const
129 {
130     return make_unique<Chrysalis>();
131 }
132
133 unique_ptr<Stage> Chrysalis::next() const
134 {
135     return make_unique<Imago>();
136 }
137
138 unique_ptr<Stage> Imago::next() const
139 {
140     return nullptr;
141 }
142
```

```
143 int main()
144 {
145     Butterfly bf;
146     cout << bf.whatAmI() << endl;
147     cout << bf.next() << endl;
148     cout << bf.whatAmI() << endl;
149     cout << bf.next() << endl;
150     cout << bf.whatAmI() << endl;
151     cout << bf.next() << endl;
152     cout << bf.whatAmI() << endl;
153     cout << bf.next() << endl;
154     cout << bf.whatAmI() << endl;
155 }
```

It is instructive to compile and run the program with `verbose` initialized to `false` and `verbose` initialized to `true`. When `verbose` is `false`, the program output shows that the `Butterfly` class behaves as expected. The `Butterfly` exemplar `bf` starts out as an egg, calling the member function `next()` yields `true` and moves `bf` to the caterpillar stage, the following call to `next()` also yields `true` and `bf` becomes a chrysalis. Another call to `next()` again returns `true` and puts `bf` into the imago stage. Since this is the adult stage, another call to `next()` returns `false` and does not change `bf`'s stage.

If `verbose` is `true`, the program additionally informs about the number of active inherited `Stage` subobjects. Whenever the `next()` member function of `Egg`, `Caterpillar` or `Chrysalis` is called, a second `Stage` subobject exists. This is automatically deleted when the assignment has completed the stage change.

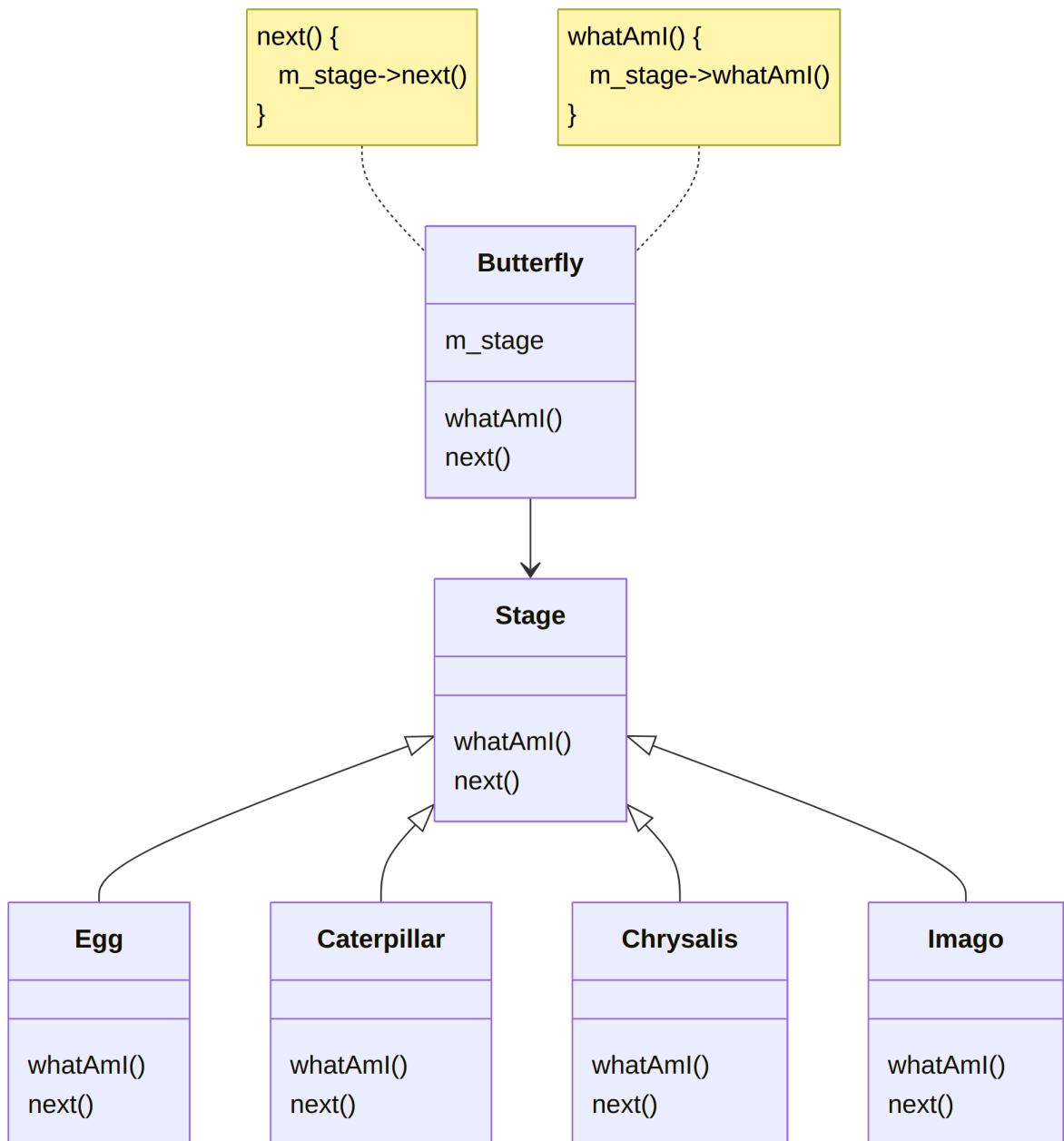
The program includes `<memory>` because of `std::unique_ptr<>` and `std::make_unique<>()` and `<typeinfo>` because the `typeid` operator is also used.

The `Butterfly` class has a data member called `m_stage`, which is of type `unique_ptr<Stage>`. `Stage` is the base class from which the other stage classes are derived, namely `Egg`, `Caterpillar`, `Chrysalis`, and `Imago`. `Butterfly` owns the `Stage` exemplar to which `m_stage` refers.

The *UML* class diagram in Figure 29 shows all classes of the program and their relationships.

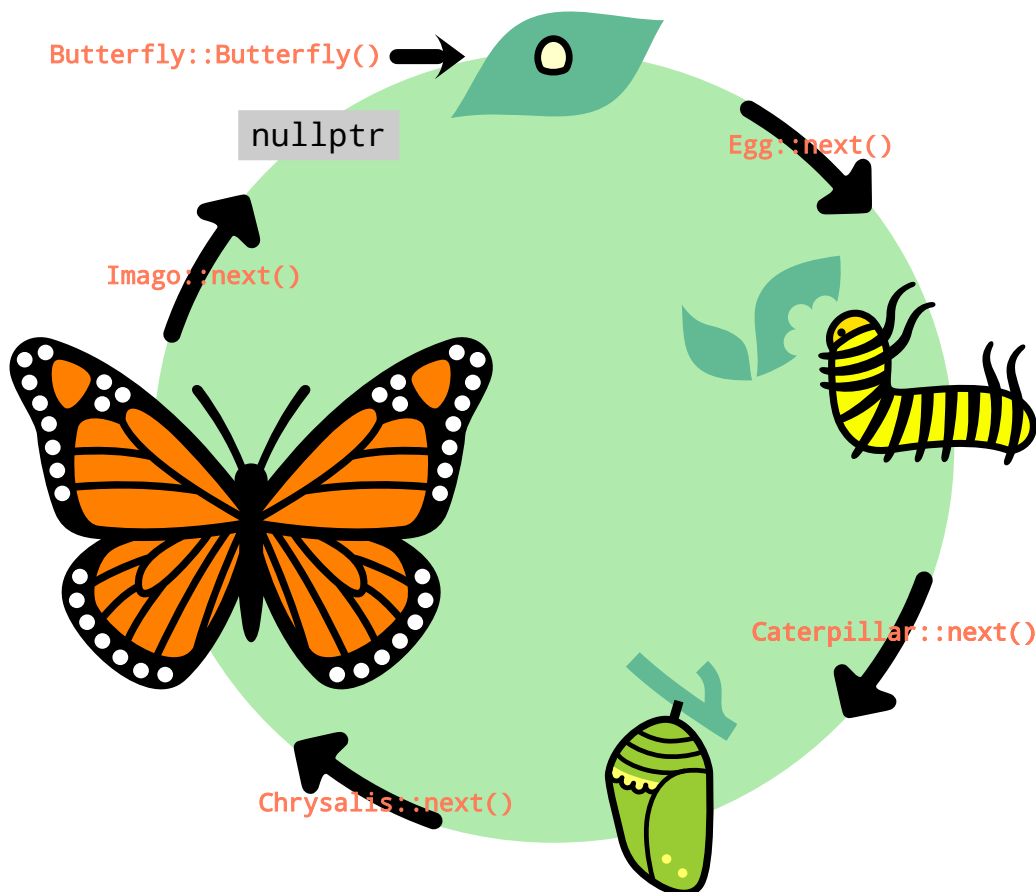
The member functions `Butterfly::next()` and `Butterfly::whatAmI()` are intended to behave differently depending on the current stage of a `Butterfly` exemplar. Both methods have a static binding. How their stage-dependent behavior is implemented will be explained in a moment.

The `whatAmI()` member function is implemented inline. `Butterfly::whatAmI()` calls the `whatAmI()` member function of the exemplar pointed to by `m_stage`. This technique is called *forwarding*. Since a `Butterfly` exemplar conceptually owns a `Stage` exemplar, it is—even more precisely—*aggregation-based forwarding*. The `Butterfly::next()` member function must be defined outside the class after the definition of the `Stage` and the `Imago` class have been defined. Another example of aggregation-based forwarding is

Figure 29: Class diagram for Butterfly and related classes

`Butterfly::next()` which forwards to the `next()` member function of the exemplar pointed to by `m_stage`. Figure 30 illustrates the stages that result from repeated calls to `Stage::next()`.

Figure 30: Butterfly stages
(<https://openclipart.org/detail/322882/butterflies-life-cycle-english>, remixed)



Stage is an abstract class. To become a valid base class, its destructor is declared as virtual. In this way, Stage also becomes a polymorphic class. It would be sufficient to define the destructor as virtual, i.e. `virtual ~Stage() = default;`. Here, the destructor and the constructor should keep track of the creation and destruction of exemplars, if desired. Therefore, depending on the global bool constant `verbose`, it is checked at compile time whether accounting code should be compiled or not. The static member variable `Stage::m_n`, which is defined inline, is used to count the number of Stage exemplars present. The member function `next()` is a pure virtual function without a definition. It specifies a `unique_ptr<>` instantiated for Stage as return value. `whatAmI()` is also a pure virtual member function which returns a string describing the stage of the current exemplar.

Since `Butterfly` declares `m_stage` as a data member of type `unique_ptr<Stage>`, it must be defined after the `Stage` class.

The following classes are all publicly derived from `Stage`. `Egg` overrides the `next()` member function and the `whatAmI()` member function. Since the implementation of `Egg::next()` requires `Caterpillar` as a complete type, it is defined outside the class. It calls `make_unique<Caterpillar>()` to create a unique pointer for a dynamic `Caterpillar` exemplar and returns it as a value of type `unique_ptr<Stage>`. As a result, ownership of the dynamically created exemplar is transferred to the caller of `Egg::next()`. The `Egg::whatAmI()` member function is defined inline and returns the string "Egg"s.

The classes `Caterpillar` and `Chrysalis` are very similar to the class `Egg`. Therefore, no redundant explanation is given for them.

The `Imago` class is also similar to the `Egg` class, but `Imago::next()` is implemented slightly differently. The imago is the adult stage of a butterfly. It can lay eggs, but *it is not* an egg. As an imago, the life cycle of a butterfly ends. Therefore, `Imago::next()` returns a `unique_ptr<Stage>` initialized to `nullptr`. One might object that this is nonsensical. But it is not possible to delete this member function from the class or to throw an exception when it is called. Both actions would violate *Liskov's substitution principle* (see Section *Inheritance*). Therefore, the special return value of this member function must be handled appropriately on the caller's side. Instead of calling `return unique_ptr<Stage>{ nullptr };`, the statement `return unique_ptr<Stage>;` would be equally valid, since a `unique_ptr<>` automatically performs a zero initialization with `nullptr` as its value.

The `Butterfly` constructor and `Butterfly::next()` still need to be explained. A butterfly always starts as an egg. Consequently, `Butterfly::Butterfly()` initializes `m_stage` with a dynamically created `Egg`. Furthermore, reporting code is compiled if `verbose` is `true`.

`Butterfly::next()` first defines `stage` as a reference to `*m_stage`. Like a raw pointer, a `unique_ptr<>` is dereferenced with the `*` operator to access the object it points to. In the next line it is checked if `stage` and `Imago` are of different types. Since `Stage` is a polymorphic base class, `typeid` can be applied to obtain a `type_info` object that represents the dynamic type of an object. This `if` statement prevents the call to `Imago::next()`. Only if the exemplar pointed to by `m_stage` is not an `Imago` will `m_stage` be assigned the result of the call to `m_stage->next()`. This assignment has two relevant effects. First, if `m_stage` is currently pointing to a valid exemplar, that exemplar is released. Second, the result of the call to `m_stage->next()` is assigned to `m_stage`. This transfers ownership of the `unique_ptr<Stage>` exemplar returned from `m_stage->next()` to `m_stage`. After the assignment, `m_stage` points to another dynamically created exemplar of a `Stage` subclass. If a new `Stage` exemplar has been dynamically created, `Butterfly::next()` returns `true`, otherwise `false`. This member function is not attributed with `[[nodiscard]]`. So the caller can check or ignore the function result. The design of this member function, as well as the overall design of the `Butterfly` class and the classes associated with it, is not

entirely consistent with regard to the allocation of responsibilities. The two extreme poles would be to make only `Butterfly::next()` responsible for the transition to the next stage or to distribute this responsibility exclusively to the Stage subclasses. In the current design, the responsibility for moving to the next stage is assigned to the `next()` member functions of the Stage subclasses, with an exception for `Butterfly::next()`, which prevents further transformation when the Imago stage is reached.

The `main()` function is simple. First, a `Butterfly` exemplar named `bf` is created. Then the two statements that output the result of `bf.next()` and `bf.whatAmI()` are repeated four times. The last call to `bf.next()` returns `false`. Therefore, the final call to `bf.whatAmI()` yields "Imago" as the result.

The structure and collaboration of the `Butterfly` class and its companion classes are an instance of the application of the design pattern *State*. This will not be pursued further here, but the interested reader is referred to (*State Pattern* 2023).

1.19.1.2. The Pointer-to-Implementation Idiom

An *idiom* is a general problem-solving pair that is specific to a particular programming language. In C++, for example, a class must be fully defined in a header file in order to use it. However, this requirement does not imply that the definitions of the member functions of this class must also be contained in the header file. But sometimes the presence of the class definition reveals too much information for the user of this class, even without the definitions of the member functions. This is therefore a general problem that is more or less specific to C++. In fact, there is a general solution to largely hide the actual definition of the class, which relies entirely on the language means of C++, but does not exist as a built-in feature of the C++ language. Both criteria, being specific to a particular programming language and being a general problem-solution pair, are the constitutive features of the term *idiom*.

The separation of `Butterfly` and `Stage` is also the basis for the application of the *Pointer to Implementation C++* idiom (*pImpl* for short) (*PImpl - Cppreference.Com* n.d.). This idiom helps to hide the actual implementation of a class by forwarding calls to member functions to a pointer to an object that implements the actual functionality. This also improves the stability of the interface.

To fully implement *pImpl* in this case, some additional measures would be required. `Butterfly` would have to be defined in a separate header file preceded by a pending declaration of `Stage`. In the header file, `Butterfly` can only define a `unique_ptr<Stage>` as a data member. This is possible even if `Stage` is an incomplete type. No methods for `Stage` may be called here. All definitions of `Butterfly` member functions must be done in an implementation file. Now `Stage` and its subclasses are defined in another separate header file. If desired, the member functions of `Stage` and its subclasses can be defined in a separate implementation file. After that, an object file for `Butterfly` can

be created, which can be passed on to other programmers together with the Butterfly header files. All implementation details of the Butterfly class and files related to Stage and its subclasses do not have to be passed on to other programmers.

It is important to note that familiarity with the *pImpl* idiom is relevant only for understanding legacy code. When writing new code, one should use *modules* instead. Modules were introduced with the C++20 standard, and allow for a perfect implementation of the information hiding principle.

Using `unique_ptr<>` ensures exclusive ownership. Therefore its copy constructor and default assignment operator are deleted, see `(std::unique_ptr<T,Deleter>::unique_ptr - Cppreference.Com n.d.)` and `(std::unique_ptr<T,Deleter>::operator= - Cppreference.Com n.d.)`. Their absence can be easily tested by adding the statements `Butterfly bfCopy { bf };` and `Butterfly bf2; bf2 = bf;` at the end of `main()`. The first and the third line will not compile because they call operations that were deleted for `unique_ptr<>`.

But what about the statement `m_stage = m_stage->next();` in `Butterfly::next()`? Obviously this is an assignment! Yes, but it is a move assignment that takes an rvalue reference to `unique_ptr<>` as its right operand. This means that ownership of the underlying raw pointer is transferred from the right operand of an assignment to its left operand, i.e. this resource is moved from the source object to the destination object (see Section *Big5—The Ambitious Way*). After the assignment, the right operand no longer owns the pointer and the underlying raw pointer has the value `nullptr`.

1.19.1.3. Explicit Transfer of Ownership

Transfer of ownership means that a dynamic resource is taken from its current owner and transferred to a new owner. The previous owner must be changed so that they will never attempt to release the dynamic resource.

The program in Listing 79 shows how to make a transfer of ownership explicit.

Listing 79: objects/TransferOwnership.cpp

```

1 // TransferOwnership.cpp by Ulrich Eisenecker, June 3, 2026
2
3 #include <iostream>
4 #include <memory>
5 #include <utility>
6 using namespace std;
7
8 class Object
9 {
10 public:
11     Object()
12     {
13         cout << "Object " << ++m_id << " created" << endl;
14     }
15     ~Object()

```

```
16     {
17         cout << "Object " << m_id << " deleted" << endl;
18     }
19 private:
20     inline static size_t m_id { 0 };
21 };
22
23 void transferOwnership(unique_ptr<Object>&& o)
24 {
25     unique_ptr<Object> local { std::move(o) }; // <-- Some compilers
26     // issue a warning if a call to move() is not fully qualified.
27 }
28
29 int main()
30 {
31     transferOwnership(unique_ptr<Object> { new Object });
32     unique_ptr<Object> o { new Object };
33     cout << boolalpha << (o.get() == nullptr) << endl;
34     transferOwnership(std::move(o)); // <--Some compilers
35     // issue a warning if a call to move() is not fully qualified.
36     cout << boolalpha << (o.get() == nullptr) << endl;
37 }
```

The `Object` class visualizes the construction and destruction of an object by sending a message to `cout` that contains an object identifier. This object identifier is managed by the static member variable `Object::m_id`. It is pre-incremented in the constructor when the output is created.

The `transferOwnership()` function takes an rvalue reference to `unique_ptr<Object>`. Thus, it can be called automatically on temporary objects that are neither `const` nor literals. Without further provision, it cannot be called with an lvalue, such as a local variable. In `transferOwnership()`, the variable `local` of type `unique_ptr<Object>` is defined and initialized by calling its move constructor. For this purpose, the `std::move()` function is called (*std::move* - *Cppreference.Com* n.d.). It converts its argument object into a reference to an object that can expire. Therefore, it may be deprived from its resources. Here, this means that the raw pointer contained in the `o` parameter can be moved to `local`, which is actually done by calling the move constructor of `unique_ptr<>`. After that, `o` will be in an unspecified state and should no longer be used. At the end of `transferOwnership()`, `local` is destroyed. This will also release the raw pointer managed by `local`.

In `main()`, `transferOwnership()` is called with an anonymous temporary object. Since it is an rvalue, `transferOwnership()` can be called directly for it.

The next statement in `main()` defines the `unique_ptr<Object> o` and initializes it with a dynamically created `Object` exemplar. This shows that it is also possible to call the `new` operator to create the dynamic object directly instead of calling `make_unique<>()`, which does not mean that the recommendation to use `make_unique<>()` for exactly this pur-

pose is revoked. The member function `get()` returns the raw pointer from a `unique_ptr<>`, as it does in the next line. This checks if the raw pointer is equal to `nullptr`, which evaluates to `false` here.

Then `transferOwnership()` is called with `o` as an argument. Since `o` is not an rvalue, `std::move()` must be applied to it first. After moving the raw pointer from `o` to `local`, the raw pointer is released when `transferOwnership()` finishes.

The last statement in `main()` checks again if `o.get()` equals to `nullptr`, which is now `true`.

In the rare case that ownership of a dynamic resource is to be transferred, it is recommended that it be done this way, since the use of rvalue references and `std::move()` at the receiving side and the use of `std::move()` for an lvalue at the calling side makes the ownership transfer explicit and prevents unintended use.

Much more could be written about `std::unique_ptr<>` and related topics. For more information, the references above can serve as a starting point.

1.19.2. Shared Ownership

In some situations, it is necessary to *share ownership* of a dynamic resource, i.e. a dynamic resource may belong to multiple owners. This does not exclude the possibility that ownership can be transferred from one owner to another. The class template `std::shared_ptr<>` is a smart pointer designed specifically for this purpose. As will be shown, shared ownership can have many pitfalls to be aware of.

1.19.2.1. A Shared-Resource Example

The following outlines the requirements for a discussion and the persons participating in a discussion. The only activities of interest are starting, joining, and leaving a discussion. Other activities, such as talking to each other and taking minutes, are neglected. This simplifies the scenario to demonstrate the use of `shared_ptr<>`. The following requirements should be met:

1. A person exists independently of a discussion.
2. A person can participate in no or exactly one discussion.
3. A person can start a new discussion at any time .
4. When a person opens a discussion, they automatically join the discussion.
5. A person can join a discussion at any time. If the persons is already participating in another discussion, they will leave that discussion immediately.
6. A person can leave a discussion at any time.
7. When the last person leaves a discussion, the discussion will automatically end.
8. A discussion cannot exist independently of a person.
9. The program should inform when a new discussion is started, when a person joins a discussion, when a persons leaves a discussion, and when a discussion is ended.

10. It should also be indicated how many people are participating in the discussion.

1.19.2.2. Design for the Shared-Resource Example

Two classes will implement these requirements, `Discussion` and `Person`. Since `Discussion` is the simpler class, its responsibilities are presented first.

Because a `Discussion` has a topic, it must manage its name in a corresponding data member. To create a `Discussion` exemplar, the topic and the initiating `Person` are needed. `Discussion` needs a member function `join()` so that a `Person` can join it. Another member function, `leave()`, allows a `Person` to leave the `Discussion`.

A `Person` manages their name and a pointer to a `Discussion`. To create a `Person` their name must be specified. The name of a `Person` can be retrieved with the member function `name()`. `Person` provides the member function `startDiscussion()` to start a new discussion on a given topic. If a `Person` is already participating in a `Discussion`, they first has to leave this `Discussion`. Then, a new `Discussion` is created and a shared pointer to it is returned as the result. The member function `join()` allows a `Person` to join a `Discussion` that is passed as a shared pointer. If the `Person` is already participating in that exact `Discussion`, no action needs to be taken. Otherwise, if the `Person` is participating in another `Discussion`, the `Person` leaves the current `Discussion` and joins the new one. The member function `leave()` removes a `Person` from a `Discussion`. First, it checks whether the `Person` is participating in a `Discussion` at all. If so, it leaves the discussion and assigns `nullptr` to the data member pointing to the `Discussion`. The member function `report()` outputs the use count of the shared pointer.

1.19.2.3. Implementing the Shared-Resource Example

The program in Listing 80 shows the implementation of the classes `Discussion` and `Person` as well as a few lines from the beginning of the `main()` function. In the following, the different parts of the `main()` function are matched with their output.

Listing 80: `objects/Discussion.cpp`

```
1 // Discussion.cpp by Ulrich Eisenecker, October 18, 2023
2
3 #include <string>
4 #include <iostream>
5 #include <memory>
6 using namespace std;
7
8 class Person;
9
10 class Discussion final
11 {
12     public:
13         explicit Discussion(const string& topic, const Person& p);
14         ~Discussion()
```

```

15     {
16         cout << "Discussion about " << m_topic << " ends." << endl;
17     }
18     void leave(const Person& p);
19     void join(const Person& p);
20 private:
21     string m_topic;
22 };
23
24 class Person final
25 {
26 public:
27     explicit Person(const string& name):m_name { name }
28     {
29     }
30     auto& startDiscussion(const string& topic)
31     {
32         if (m_discussion)
33         {
34             m_discussion->leave(*this);
35         }
36         m_discussion = make_shared<Discussion>(topic,*this);
37         return m_discussion;
38     }
39     void join(shared_ptr<Discussion>& discussion)
40     {
41         if (m_discussion != discussion)
42         {
43             if (m_discussion)
44             {
45                 m_discussion->leave(*this);
46             }
47             m_discussion = discussion;
48             m_discussion->join(*this);
49         }
50     }
51     void leave()
52     {
53         if (m_discussion)
54         {
55             m_discussion->leave(*this);
56             m_discussion = nullptr;
57         }
58     }
59     const string& name() const
60     {
61         return m_name;
62     }
63     void report() const
64     {
65         cout << ", use count = " << m_discussion.use_count() << '.' << endl;
66     }
67 private:
68     shared_ptr<Discussion> m_discussion;
69     string m_name;

```



```
70 };
71
72 Discussion::Discussion(const string& topic,const Person& p):m_topic { topic }
73 {
74     cout << p.name() << " starts a discussion about " << m_topic << '.' << endl;
75 }
76
77 void Discussion::join(const Person& p)
78 {
79     cout << p.name() << " joins the discussion about " << m_topic;
80     p.report();
81 }
82
83 void Discussion::leave(const Person& p)
84 {
85     cout << p.name() << " leaves the discussion about " << m_topic;
86     p.report();
87 }
88
89 int main()
90 {
91     Person josh { "Josh"s },
92           jude { "Jude"s },
93           kyle { "Kyle"s };
```

The `shared_ptr<>` class template—like `unique_ptr<>`—is also defined in the `<memory>` header file. Therefore, this header file is included. Since the classes use `std::string` and produce output, the header files `<string>` and `<iostream>` are also included.

Since references to `Person` will be passed as parameters to member functions of the `Discussion` class, a pending declaration is required for `Person`.

`Discussion` is declared as a `final` class, thus it cannot be used as a base class. Its constructor is declared as `explicit` and takes a reference to `const` of type `std::string`, which contains the topic of a discussion, and a reference to `const` of type `Person`, which is the initiator of the `Discussion` to be created. The constructor is defined after the `Person` class because it must call the `Person` member functions. It initializes the data member `m_topic` with the argument `topic`, and outputs a message containing the name of the initiator of the discussion and its topic. The destructor outputs a message indicating the end of life of the `Discussion` exemplar. The member function `join()` takes a reference to `const` of type `Person`. It is defined after the `Person` class. It prints a message that the `Person` passed as an argument has joined the discussion and calls `p.report()`, which is a member function of `Person` and prints the use count for the shared pointer. The member function `leave()` is analogous and is called when a `Person` leaves a `Discussion`.

`Person` is also declared as a `final` class. Its constructor, which is also declared as `explicit`, takes the name of the `Person` as an argument and uses it to initialize the corresponding data member. The `startDiscussion()` member function takes a reference to `const` of type `std::string` named `topic` as a parameter. First, it checks whether `m_`-

discussion has the value true, which is the case if the raw pointer differs from the value nullptr. This is achieved by the type conversion operator defined for `shared_ptr<>`, which evaluates to a bool value. If the active Person is already participating in a Discussion, they first leaves this discussion by calling `m_discussion->leave(*this)`. Then a new Discussion is created by calling the `make_shared<>()` function template. It is explicitly instantiated with Discussion and the constructor parameters topic and *this are passed as function arguments. The result of this function call is assigned to `m_discussion`. If `m_discussion` already contains a raw pointer pointing to a dynamically created Discussion exemplar, its use counter is first decremented by 1 before the actual assignment is performed. The last statement returns `m_discussion` by reference. This is achieved by specifying `auto&` as the return type. The use of `auto` causes the compiler to deduce the return type from the value returned by the return statement. The reference symbol indicates that the value should be returned as a reference. This is an important detail because it prevents another shared pointer exemplar from being created—as would be the case if a value were returned.

The `join()` member function takes a reference to `shared_ptr<Discussion>` named `discussion`. Then it checks if its data member `m_discussion` and `discussion` are different. The `!=` operator defined for `shared_ptr<>` checks if the raw pointers managed by the two shared pointer exemplars are different. The nested check tests if there is an active discussion. If so, the Person exemplar does exit it first. Then `discussion` is assigned to `m_discussion`. The effects of this assignment are the same as described for the `startDiscussion()` member function. Then, the active Person exemplar joins the discussion with `m_discussion->join(this)`.

The `leave()` member function first checks if the raw pointer managed by `m_discussion` is different from the nullptr value. If it is, the active Person exemplar leaves the discussion and assigns nullptr to `m_discussion`. This safeguard is especially important for the call to `leave()`. Calling `leave()` for a raw pointer equal to nullptr would result in an undefined state.

The `name()` member function simply returns the data member `m_name` as a reference to const of type `std::string`. The `report()` member function prints how many owners currently share the shared pointer `m_discussion`. This is done by calling `m_discussion.use_count()`.

1.19.2.4. Running the Shared-Resource Example

Listing 80 shows only the beginning of the `main()` function. Here, three persons with different names are defined and initialized. Table 9 shows in its first column each of the following three statements of `main()` in a separate row. The second column shows the output produced by the statement on the left. Only two persons are involved in the discussion here.

Table 9: Two persons in a discussion

Statement	Output
<code>jude.join(josh.startDiscussion("Fishing"s));</code>	Josh starts a discussion about Fishing. Jude joins the discussion about Fishing, use count = 2.
<code>josh.leave();</code>	Josh leaves the discussion about Fishing, use count = 2.
<code>jude.leave();</code>	Jude leaves the discussion about Fishing, use count = 1. Discussion about Fishing ends.

The call to `josh.startDiscussion("Fishing"s)` creates a new `Discussion`. The shared pointer `m_discussion` returned by the call to this member function as a reference is passed as an argument to `jude.join()`. Correspondingly, the `report()` member function outputs 2 as the value for use count, which is correct since Jude and Josh are sharing this discussion.

Next, `josh.leave()` is executed. This first outputs the current use count, which is still 2 due to the call to `m_discussion->leave(*this)` in `Person::leave()`. Then Josh executes `m_discussion = nullptr;`, which decreases the use count of the shared pointer by 1 (no output).

Then `jude.leave()` is executed. Correspondingly, the use count of the shared pointer is output before Jude left, which is 1. Then, Jude also assigns `nullptr` to `m_discussion`. This decreases the use count by 1 again, resulting in zero objects sharing the `Discussion` exemplar. For this reason, the `Discussion` exemplar is deleted, which is adequately reported as the end of the discussion.

The member functions defined for `Person` do not allow to query the `Discussion` exemplar shared by a `Person`. Therefore, it is not possible for more than two `Person` exemplars to participate in a `Discussion` created in this way. How more than two `Persons` can share a `Discussion` is illustrated below.

Table 10 shows the statements and their output of the following nested block.

First, `josh` starts a new discussion about "Farming"s. This does not mean that the discussion that ended immediately before is resumed, but it is a new, dynamically created exemplar. The reference returned by the function call is used to copy initialize a `Discussion` exemplar named `farming`. The `auto` keyword is used to allow the compiler to automatically deduce the type of `farming` from the result of the `Person::startDiscussion()` call.

Table 10: *Several persons in several discussions*

Statement	Output
<code>auto farming { josh.startDiscussion("Farming"s) };</code>	Josh starts a discussion about Farming.
<code>josh.leave();</code>	Josh leaves the discussion about Farming, use count = 2.
<code>cout << "*Use count of farming: " << farming.use_count() << endl;</code>	*Use count of farming: 1
<code>jude.join(farming);</code>	Jude joins the discussion about Farming, use count = 2.
<code>kyle.join(farming);</code>	Kyle joins the discussion about Farming, use count = 3.
<code>cout << "*Use count of farming: " << farming.use_count() << endl;</code>	*Use count of farming: 3
<code>jude.startDiscussion("Gaming"s);</code>	Jude leaves the discussion about Farming, use count = 3. Jude starts a discussion about Gaming.
<code>jude.leave();</code>	Jude leaves the discussion about Gaming, use count = 1. Discussion about Gaming ended.
<code>kyle.leave();</code>	Kyle leaves the discussion about Farming, use count = 2.
<code>cout << "*Use count of farming: " << farming.use_count() << endl;</code>	*Use count of farming: 1
End of block	Discussion about Farming ends.

Next, josh immediately leaves the discussion. Interestingly, the use count before josh leaves is reported as 2. In fact, josh is not the only owner of the shared pointer. It is also used by the farming variable, which is not a Person but a `shared_ptr<Discussion>` exemplar. Next, jude and kyle join the farming discussion which is reported accordingly. The use count is three, since the farming variable is also an owner. Now jude leaves the farming discussion and starts a new discussion about "Gaming"s. This time, the returned reference from `Person::startDiscussion()` is discarded. Therefore, no other Person can join the discussion. Both steps are reported accordingly as can be seen in the cells to the right of the statements. After that, jude leaves the gaming discussion. Before he leaves, the use count of the gaming discussion is 1. Once he leaves, the gaming discussion ends. Next, kyle leaves the farming discussion. Before he leaves, the use count of farming is 2, after that it is 1. The reason for this is that the farming variable still points to the farming discussion. At the end of the block, the lifetime of farming ends, which also releases the dynamic resource for the "Farming"s topic.

Table 11: Discussion with a clone

Statement	Output
<code>auto programming { kyle.startDiscussion("Programming"s) };</code>	Kyle starts a discussion about Programming.
<code>auto kylesClone { kyle };</code>	
<code>kyle.leave();</code>	Kyle leaves the discussion about Programming, use count = 3.
<code>cout << "*Use count of programming: " << programming.use_count() << endl;</code>	*Use count of programming: 2
<code>kylesClone = jude;</code>	
<code>cout << "*Use count of programming: " << programming.use_count() << endl;</code>	*Use count of programming: 1
<code>cout << "**End of main()" << endl;</code>	*End of main()
End of block	Discussion about Programming ends.

Table 11 shows the remaining statements of `main()` along with the output that their execution produces. The result of starting a discussion by kyle is used to initialize the programming variable. Then a clone of kyle named `kylesClone` is created by calling the copy constructor of `Person`. Then kyle leaves the discussion, but immediately before the use count of programming is 3. This is due to the existence of the variable `programming`, which still counts kyle as a sharing owner, and because of `kylesClone`. After kyle leaves the discussion, the use counter of programming is reported as 2. Assigning jude to `kylesClone` decreases the use count of programming to 1. This is correct

because the raw pointer of programming is still pointing to a dynamic Discussion exemplar. jude is not currently participating in a discussion; thus, kylesClone is also not participating in a discussion after jude is assigned. The last statement of `main()` outputs that the end of `main()` is reached. However, the programming discussion is still active because the variable programming still exists. When programming is destroyed at the end of the block of `main()`, its end is reported.

1.19.2.5. Observations From the Shared-Resource Example

Unlike `std::unique_ptr<>`, it is possible to copy and assign a `std::shared_ptr<>` and objects that own it. The formulated requirements are met by the program, but it is obvious that the implementation allows uses that were not previously thought of. First, one may question whether returning a reference to a `shared_ptr<Discussion>` from `Person::startDiscussion()` is really useful. It allows the creation of `shared_ptr<Discussion>` variables outside of `Person`, which was not anticipated. These variables are also owners that share the dynamic resource and therefore affect its use count. In addition, such a variable may extend the lifetime of a `Discussion` exemplar beyond the participation or even existence of `Person` exemplars. An alternative could be that a `Person` can only join a discussion by asking another `Person` who is participating in that discussion.

The member function `Person::report()` outputs the use count of a shared pointer. This responsibility seems to be assigned incorrectly. For the `Discussion` class, one might actually expect this. But a `Discussion` exemplar does not know how many `Person`'s are participating. As discussed earlier, the use count of a `shared_ptr<Discussion>` does not necessarily correspond to the number of participating `Person` exemplars. Therefore, an additional requirement should be introduced and implemented to capture the actual number of `Person` exemplars participating in a discussion. The `report()` member function, which no longer depends on the shared pointer, could become a member function of `Discussion`.

The possibility to create a clone of a `Person` exemplar is a direct consequence of the automatic generation of a copy constructor for `Person`. The same is true for the assignment of `Person`'s. Although it is nice to have these operations available, one can debate whether they are really desirable in terms of the requirements, or more precisely, what is *not* included in the requirements.

To understand the special situation of circular dependencies between pointers, a closer look at copy construction, assignments and `shared_ptr<>::reset()` can be helpful.

The program *objects/SharedPointer.cpp*, which is only included in the accompanying material, generates the output shown in Table 12. The left-hand column contains the relevant statements and specifies the shared pointer for which the information is given in the right-hand column.

Table 12: Output of program objects/SharedPointer.cpp

Two shared pointers that share the same object.	
<code>sp1 = make_shared<string>("ok?"s);</code>	use count: 1, "ok?"
<code>sp2 { sp1 };</code>	use count: 2, "ok?"
<code>*sp1 = "OK!";</code>	use count: 2, "OK!"
<code>sp2</code>	use count: 2, "OK!"
Stop sharing now.	
<code>sp1.reset();</code>	use count: 0, nullptr
<code>sp2</code>	use count: 1, "OK!"
~	
Two shared pointers that do not share the same object.	
<code>shared_ptr<string> sp3 {};</code>	use count: 0, nullptr
<code>auto sp4 { sp3 };</code>	use count: 0, nullptr
<code>sp3 = make_shared<string>("?!"s);</code>	use count: 1, "?!"
<code>sp4</code>	use count: 0, nullptr
Start sharing now.	
<code>sp4 = sp3; sp4</code>	use count: 2, "?!"

As can be seen, `sp1` and `sp2` share the same string object. `sp1` is first created by a call to `make_shared<>()`, and `sp2` is created by copy initializing with `sp1`. Sharing is terminated after calling the `reset()` member function for `sp1`. This results in the use count reported for `sp2` being reduced to 1. Resetting a shared pointer only deletes a shared object if the use count before the reset is already 1. There is no way to delete an object referenced by a shared pointer so that it is removed from other pointers that share it.

`sp3` is created differently to `sp1`. The `shared_ptr<>` template is explicitly instantiated for `string` and `sp3` is initialized with an empty list. This leads to an initialization that is conceptually null, i.e. it is initialized here with `nullptr`. Despite the fact that `sp4` is also initialized with `sp3`, the resulting shared pointers actually have nothing in common! `nullptr` is a valid pointer, but it does not point to any object. Therefore, there is also no object that can be reference-counted. This changes when assigning a shared pointer to a string to `sp3`. Now `sp3` points to a valid object. By assigning `sp3` to `sp4`, the number of uses of the shared pointer is increased to 2.

1.19.3. Interdependent Pointers

If each of two objects has a smart pointer pointing to the other object, they are mutually dependent and it is impossible to delete them. In the following, an example of this situation is given and a new category of smart pointers is introduced to help solve this problem.

A special situation arises when two objects refer to each other by means of a shared pointer or when objects refer to other objects in a circular form by means of a shared pointer. The program in Listing 81 demonstrates this.

Listing 81: `objects/Circular.cpp`

```

1 // Circular.cpp by Ulrich Eisenecker, September 5, 2023
2
3 #include <string>
4 #include <iostream>
5 #include <memory>
6 using namespace std;
7
8 class Object final
9 {
10 public:
11     explicit Object(const string& name): m_name { name }
12     {
13         cout << m_name << " created." << endl;
14     }
15     Object(const Object&) = delete;
16     Object& operator=(const Object&) = delete;
17     ~Object()
18     {
19         cout << m_name << " deleted." << endl;
20     }

```



```
21     void connect(shared_ptr<Object>& p)
22     {
23         m_object = p;
24     }
25     void disconnect()
26     {
27         m_object.reset();
28     }
29 private:
30     string m_name;
31     shared_ptr<Object> m_object;
32 };
33
34 int main()
35 {
36     {
37         cout << "Shared pointers are released." << endl;
38         auto o1 { make_shared<Object>("Jaden"s) };
39         auto o2 { make_shared<Object>("Josh"s) };
40         o1->connect(o2);
41         o1->disconnect();
42         cout << "o1uc " << o1.use_count()
43              << " o2uc " << o2.use_count() << endl;
44         o1.reset();
45         o2.reset();
46     }
47
48     {
49         cout << "\nShared pointers are released." << endl;
50         auto o1 { make_shared<Object>("Kyle"s) };
51         auto o2 { make_shared<Object>("Rowan"s) };
52         auto o3 { make_shared<Object>("Jordan"s) };
53         o1->connect(o2);
54         o2->connect(o3);
55         o1.reset();
56         o2.reset();
57         o3.reset();
58     }
59
60     {
61         cout << "\nShared pointers are NOT released." << endl;
62         auto o1 { make_shared<Object>("Quinn"s) };
63         auto o2 { make_shared<Object>("Logan"s) };
64         o1->connect(o2);
65         o2->connect(o1);
66         o1.reset();
67         o2.reset();
68     }
69
70     {
71         cout << "\nShared pointers are NOT released." << endl;
72         auto o1 { make_shared<Object>("River"s) };
73         auto o2 { make_shared<Object>("Riley"s) };
74         auto o3 { make_shared<Object>("Avery"s) };
75         o1->connect(o2);
```

```
76 |         o2->connect(o3);  
77 |         o3->connect(o1);  
78 |         o1.reset();  
79 |         o2.reset();
```

The final class `Object` defines an explicit constructor that takes the argument name of type `std::string`, which is used to initialize the data member `m_name`. The constructor and destructor inform about the creation and deletion of an `Object`. The copy constructor and the assignment operator are deleted.

The data member `m_object` is of type `shared_ptr<Object>` and contains a pointer to another `Object` exemplar.

The `connect()` member function establishes a connection to another `Object` exemplar passed by reference. The `disconnect()` member function disconnects the connection by calling the overloaded member function `reset()` for `m_object`. It is not called at all in the `main()` function shown. Instead, the member function `shared_ptr<>::reset()` is used.

In the first block of the `main()` function the shared pointers `o1` and `o2` are created. Then the `o1` shared pointer is linked to `o2`. Next, the `reset()` member function is called for `o1` and `o2` with the result that both objects no longer point to another `Object`. At the end of the block, both `Object`'s referred by the shared pointers `o1` and `o2` are deleted as the output shows.

In the second block, the shared pointers `o1`, `o2` and `o3` are created and linked from `o1` to `o2` and `o2` to `o3`. All shared pointers are reset. At the end of the block all dynamically created `Object` exemplars are deleted.

After compiling and running the program, its output contains interesting details with respect to the first and second block. All objects referenced by shared pointers are deleted in the exact order in which they were created. However, this is not what one would usually expect. Normally the last created variable is deleted first! How can the opposite happen here?

After creation and initialization, the use counter of `o1` and `o2` is 1. After connecting `o2` to `o1`, the use count of `o2` is 2, but the use counter of `o1` is still 1. After resetting `o1`, its use count becomes 0 and the common object "Jaden" is deleted. As a result of the deletion, the data element `m_object` of the object "Jaden" is also deleted, which in turn reduces the use count of `o2` to 1. After resetting `o2`, its use count therefore becomes 0 and "Josh" is deleted. Of course, `o1` and `o2` are deleted in the correct order, namely `o2` before `o1`, but these deletions do not result in any output. Only the deletion of the shared objects produces an output. This is the reason for this seemingly puzzling observation.

The expected order of object destruction can be restored by executing the `disconnect()` member function of each referenced `Object` instead of calling `reset()` for the shared pointer everywhere in each block. For example, `o1.reset()` becomes `o1-`

>disconnect(), and so on. After this is done consistently throughout the program, all Object's are destroyed in the reverse order of their creation. If o1.disconnect() is called, o1.m_object is reset. This decrements the use count of o2 to 1, but the use count of o1 is still 1. Calling o2.disconnect() has no effect because o2.m_object has nullptr as the value for its raw pointer. Since reset() is not called for either o1 or o2, both shared pointers are deleted in the reverse order of their creation.

The same explanation applies to block 2.

Blocks 3 and 4 also exhibit interesting runtime behavior. In the third block of main(), a mutual link between o1 and o2 is established, i.e. o1 is linked to o2, and o2 is linked to o1. Despite resetting o1 and o2, the shared pointers at the end of the block cannot release the underlying objects because of the mutual reference. Thus, no information is given about the deletion of o1 and o2.

The fourth block creates a circle by linking o1 with o2, o2 with o3 and o3 with o1. As can be seen, the raw pointers cannot be released, although the smart pointers o1, o2 and o3 are reset.

The use of std::weak_ptr<> allows to solve this problem. A weak pointer holds a non-owning reference to an object that is managed by a shared pointer. A weak pointer is also said to have *temporary ownership*. To solve the problem of mutual or circular references, only two changes to the objects/Circular.cpp program are necessary. They are shown in Listing 82.

Listing 82: objects/BreakingTheCircle.cpp

```
8 | class Object final
9 | {
10 |     public:
21 |         void connect(shared_ptr<Object>& p)
22 |         {
23 |             m_object = weak_ptr<Object> { p };
24 |         }
29 |     private:
30 |         string m_name;
31 |         weak_ptr<Object> m_object;
32 | };
```

The first change is that Object::m_object is now of type weak_ptr<Object>. The next change takes place within the Object::connect() member function. Here, a converting constructor of weak_ptr<> is used to get a weak pointer from a shared pointer, which is then assigned to m_object.

These two changes are enough for the program to release all dynamically created objects when all smart pointers that own these objects are deleted.

In fact, solving the problem by using a `weak_ptr<>` is somehow artificial. If in blocks 3 and 4 of the original program `objects/Circular.cpp` each call to `.reset()` is replaced by a call to `->disconnect()`, the problem also no longer occurs. Another side effect of this measure is that all objects are destroyed in the expected order, namely from the last object created to the first object created. In addition, the `o3.reset()` or `o3->disconnect()` statement can be deleted in the second block, as it has no effect on the program run.

There are other scenarios for using `weak_ptr<>` that involve temporarily converting a weak pointer to a shared pointer. Furthermore, a weak pointer has other member functions that have not been described here. For a brief overview see for example (*std::weak_ptr* - *Cppreference.Com* n.d.).

Overall, this section is only a superficial introduction to smart pointers. Anyone with a deeper interest in understanding smart pointers or using them in practice should definitively seek further information, e.g. by studying web resources and other texts that deal specifically with smart pointers.

1.20. Criticism of OOP and Inclusion Polymorphism

The benefits and advantages of object-oriented programming are now summarized. In addition, some deficits and risks of this currently dominant programming paradigm are also mentioned.

In the early days of object-oriented programming, programmers and software engineers realized that classes sometimes allowed a more direct mapping of the problem space to the solution space, that inheritance was an effective means of structuring code by capturing commonalities in base classes (sometimes in accordance with given structures in the problem space), and that inclusion polymorphism made it possible to write code that allowed the execution of code that had not yet been written.

Object-oriented methods for designing software and for analyzing and modeling problems followed soon after object-oriented programming languages, and for the first time in software engineering the same concepts were used in programming, design and analysis, namely classes, methods, attributes, inheritance and late binding, i.e. inclusion polymorphism. Late binding was not particularly emphasized, it was simply adopted as the standard binding for methods. In addition, the *Object Management Group* (OMG for short) (<http://www.omg.org/>) published a number of industry standards focused on object orientation, most notably the *Unified Modeling Language* (UML for short), a notation for all aspects of object-oriented analysis and design. All of this contributed to the overwhelming success of object orientation and made it the dominant paradigm for programming today.

Nevertheless, there are also serious problems. Some of these are mentioned below.

Without taking generic programming into account (see Section *The Generic Programming Paradigm*), container types had to be implemented exclusively in an object-oriented manner, for example the *Booch components* (Booch and Vilot 1990). In order to store objects in such containers, the classes of all these objects must be (directly or indirectly) derived from a common base class. When a stored object is retrieved from a container, it is always returned as a type of that common base class. If the dynamic type of this object is required, it must be restored by applying an appropriate type cast. However, this cast can fail at runtime, which has a negative impact on the usability and security of the corresponding software system. Furthermore, it is not always obvious and easy to decide which container type should serve as the base class for another container type, as described in (Booch and Vilot 1990), for example.

The importance of a base class should not be underestimated. A single change to a base class can have serious consequences for its successor classes. If the change only affects the interface of the base class, this can lead to errors when compiling the successor classes. Fortunately, however, these errors are reported. Much worse is a change to the implementation of a base class method, perhaps with a malicious intention. Now, while no compilation errors will occur, any code that calls that particular base class method, perhaps by overriding and extending it (see Section *Implementation of the UrnO Class*), will execute malicious code. This problem is known as the *Fragile Base Class Problem*, and is described in detail in (Mikhajlov and Sekerinski 1998).

In general, inheritance has a negative effect on the modularization principle, as it introduces a dependency of the derived class on its base class. If a derived class also accesses non-public members of its base class, this also violates the principle of information hiding. This makes it more difficult to understand and change the code, as it always requires access to all its base classes. This problem leads to the conclusion that inheritance can (at least sometimes) be harmful (Hunt 2003).

Another popular promise of object-oriented programming was and is reuse. In short, reuse means that a new class is not created from scratch, but is derived from a base class and only the differences to the base class need to be defined and implemented in a derived class. In terms of the conventions and rules that have evolved over time, these differences are usually limited to adding new methods or attributes in the derived class or replacing the implementation of a base class method in the derived class. The consideration of relevant differences in individual classes leads to fine-grained classes that have complex relationships either through multiple inheritance or through the implementation of multiple interfaces. A library that tries to capture all relevant variations as a hierarchy of related classes (possibly including collaborating classes) grows enormously in size and complexity, which has a strong negative impact on comprehension and maintainability. This problem is described under the name *Library Scaling Problem* in (Biggerstaff 1994).

There are many other problems that arise from object-oriented programming, e.g. the problem of random vs. planned reuse, the combination of object-oriented frameworks and much more. They cannot be mentioned in detail here. To summarize, object-oriented programming is the dominant programming paradigm today, but is increasingly used in combination with other programming paradigms, especially generic and functional programming. Furthermore, the use of inheritance and polymorphism should be very well considered and only chosen if no other option is available that is less harmful in terms of comprehensibility and maintainability.

2. The GUI-Programming Paradigm

Programs with interactive graphical user interfaces are the de facto standard for desktop applications. In the following, the bridge is built from console programs that force their execution on the user, via event-processing programs that also run in a console window, to programs that graphically display their status and possible interactions and react to user input.

So far, only programs that are executed in a terminal window have been presented. A so-called *console application* runs either in *batch mode*, i.e. it does not interact at all with a human user in terms of input and output, or in *interactive mode*. Interactive mode means that the program generates output in the terminal window and requests input from the user. In this case, the console application has control over when and in what order which outputs are generated and which inputs are requested.

The user also needs to know which commands can be entered in a terminal window and the effect of entering a particular command. There are a few commands to get help on commands, e.g. *man* or *apropos*. But even with these, one must at least *actively memorize* the names of the possible commands. Basically, using a terminal window requires intensive training. This also applies to console applications if they are executed interactively.

The situation is different for programs with a *graphical user interface* (GUI for short). Here, the user is responsible for controlling the program. The program displays the interaction options graphically and the user selects the operation to be executed, decides when to make which inputs and requests certain outputs. The most noticeable difference to the terminal window is that the user can see which options exist and which of them are currently available, as they are not grayed out. This considerably reduces the need for active remembering. Sometimes it is enough for the user to look at the available options to know which is the right option for them. This is *passive remembering*. And even if there is no passive remembering, the user can explore the available options by hovering over them to see if help information is available.

Another possibility is to simply select an option and undo the consequences if they are not desired. This points to another important difference between console applications and GUI applications, namely the shift of control from the program to the user.

The ability to address individual screen pixels, the optional use of a pointing device and the provision of graphical elements are further features that distinguish GUI applications from console applications.

2.1. Text-based vs. Graphical User Interfaces

Shifting control of the interaction from the program to the user is not specific to GUI applications. The following shows how control can also be transferred to the user for applications that run in a terminal window.

There is no technical reason for the above-mentioned differences between GUI and console applications. The use of several windows, the addressing of individual character positions, the optional use of a pointing device and the control of the control flow by the user can also be implemented with purely *text-based user interfaces*, which can be realized with corresponding frameworks or libraries, e.g. *Ncurses* ((*NCURSES - New Curses* n.d.) and (*Ncurses* 2023)).

2.1.1. An Example of a Text-based User Interface

Ncurses is a *C* library that is not part of the *C* or *C++* standard. It can be easily installed on *Linux*, e.g. `sudo apt install ncurses`, or with `brew install ncurses` on *macOS*. Alternatively, *PDCurses* can be installed on *Windows*-based systems. This may require downloading it from its project homepage, executing the corresponding *Makefile* with `mingw32-make`, creating a library file with the `ar` command and hard links for `curses.h` to `ncurses.h` and possibly for the library file with the `mklink` command in a *CMD* console window or integrating the necessary headers and the library into the corresponding folders of the installed compiler. Explaining all this in detail is beyond the scope of this book.

Listing 83 shows a program implemented with *Ncurses*. When compiling, the *Ncurses* library must be specified explicitly for linking, e.g. `g++ -std=c++20 Move.cpp -lncurses`. When executed, it displays an asterisk in the center of the console window, which can be moved with the *h* (move left), *j* (move up), *k* (move down) and *l* (move right) keys. This assignment of keys to cursor movements is the same as in the *Vim* editor. Pressing the *Esc* key closes the program. A sound is played when another button is pressed if the console window supports this. Attempting to move the asterisk beyond the edge of the terminal window will cause the window to flash. Accidentally pressing a cursor key on the keyboard can also end the program. The reason for this is that the cursor keys are sometimes configured in such a way that pressing a cursor key generates several character codes, one of which is the *Escape* character.

Listing 83: gui/Move.cpp

```

1 // Move.cpp by Ulrich Eisenecker, May 28, 2024
2
3 #include <ncurses.h>
4
5 const char quit { 27 };
6 const char symbol { '*' };
7 const char space { ' ' };
8 const char left { 'h' };
9 const char right { 'l' };
10 const char up { 'j' };
11 const char down { 'k' };
12
13 // Get a (keyboard) event.
14 char getEvent()
15 {
16     char e;
17     bool invalid { true };
18     do
19     {
20         e = getch();
21         if ((e == quit) || (e == left) || (e == right) || (e == up) || (e == down))
22         {
23             invalid = false;
24         }
25         else
26         {
27             beep();
28         }
29     } while (invalid);
30     return e;
31 }
32
33 // Execution of code that is specific to an event.
34 void processEvent(char e)
35 {
36     static int x { getmaxx(stdscr) / 2 - 1 };
37     static int y { getmaxy(stdscr) / 2 - 1 };
38
39     mvaddch(y,x,space);
40     if (e == left)
41     {
42         if (x > 0)
43         {
44             --x;
45         }
46         else
47         {
48             flash();
49         };
50     }
51     if (e == right)

```

```
52 {
53     if (x < getmaxx(stdscr) - 1)
54     {
55         ++x;
56     }
57     else
58     {
59         flash();
60     }
61 }
62 if (e == up)
63 {
64     if (y > 0)
65     {
66         --y;
67     }
68     else
69     {
70         flash();
71     }
72 }
73 if (e == down)
74 {
75     if (y < getmaxy(stdscr) - 1)
76     {
77         ++y;
78     }
79     else
80     {
81         flash();
82     }
83 }
84 mvaddch(y,x,symbol);
85 }
86
87 int main()
88 {
89     initscr(); // Initialize Ncurses.
90     noecho(); // Disable echoing of an entered character.
91     curs_set(0); // Make the cursor transparent.
92     bool execute { true };
93     processEvent(0); // Calling processEvent() to initialize static variables.
94     do // Start the event loop.
95     {
96         auto event { getEvent() };
97         if (event == quit)
98         {
99             execute = false;
100         }
101         else
102         {
103             processEvent(event);
104         }
105     } while (execute);
106     curs_set(1); // Restore the normal appearance of the cursor.
```

```

107 |     endwin(); // Stop using Ncurses; restore the normal
108 | }           // behavior of the terminal window.

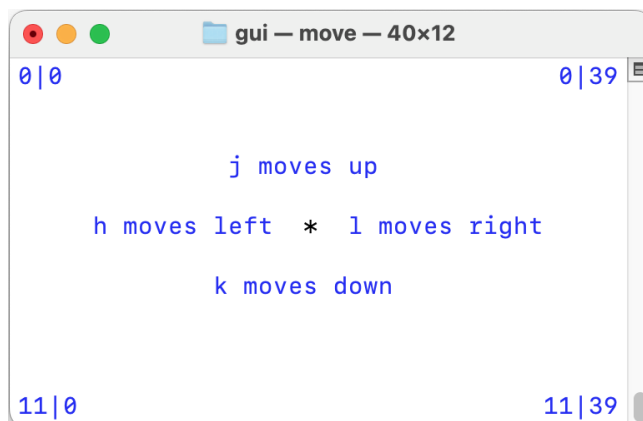
```

2.1.2. The Properties of a Terminal Window

Although a terminal window looks very simple, it has some interesting features that are important for creating and running interactive applications with a text-based user interface. These relevant properties are presented below.

Figure 31 shows a screenshot of a modified version of the program `gui/Move.cpp`. It is called *MoveWithInfo.cpp* and is not presented here, but is included in the accompanying material in the *gui* folder. It is executed in a reduced terminal window that is only 12 rows and 40 columns in size (normally set as properties of the terminal window). This modified version displays additional information about some terminal properties and the available buttons for movement control at the beginning. It helps to understand what the program `gui/Move.cpp` does and how *Ncurses* works. Executing this program in a terminal window whose properties are set differently (background color, font attributes, and screen size) results in a different appearance.

Figure 31: Screenshot of the execution of `gui/MoveWithInfo.cpp`



The file `<ncurses.h>` is included at the beginning of the program. The extension `.h` is mandatory as it is a *C* header file and not a *C++* header file. This is also the reason why no identifiers from a specific namespace are imported. All identifiers that originate from `<ncurses.h>` are in an anonymous namespace and therefore cannot be imported explicitly. Apart from `<ncurses.h>`, no other header files are required by the program.

Several character constants are defined at the beginning of the program. They make it easier to understand and adapt the program, e.g. selecting the *q* key instead of *esc* to exit the program. The two functions `getEvent()` and `processEvent()` are discussed after the `main()` function.

`main()` first calls `initscr()`. This retrieves the properties of the console window and initializes the internal data structures of *Ncurses* as well as the standard window. Calling `noecho()` suppresses the display of a pressed key in the terminal window. Calling `curs_ set(0)` makes the cursor transparent. Its normal appearance can be restored by calling `curs_ set(1)`. Calling this function with 2 as an argument, the cursor is highlighted in a terminal-specific way. Next, the `bool` variable `execute` is declared and initialized to `true`. Then the function `processEvent()` is called with 0, which draws an asterisk in the center of the terminal window. Next, a do-while loop is started, which runs as long as `execute` is `true`. In the loop, the function `getEvent()` is called to initialize the event variable. If `event` equals `esc`, `execute` is set to `false`. Otherwise, `event` is passed as an argument in the call to `processEvent()`. At the end of the loop, `curs_ set(1)` is called to reset the cursor to its default display, and `endwin()` is called to clean up any changes made by *Ncurses* and return the terminal window to its normal state.

The `getEvent()` function reads one of the character values `quit`, `left`, `right`, `up` or `down` and returns it. It first declares the variable `e` of type `char`. Then the `bool` variable `invalid` is declared and initialized with `true`. A do loop is started, which runs as long as `invalid` is `true`. In the do loop, the function `getch()` is called, the result of which is assigned to `e`. `getch()` waits until a key is pressed and then returns its value. The if statement has a complex condition whose elementary conditions are combined by a logical OR. Each elementary condition checks whether `e` is equal to one of the characters `quit`, `left`, `right`, `up` or `down`. If this is the case, `invalid` is assigned the value `false`. Otherwise, `beep()` is called, which generates a sound on terminals that support this function. As soon as `invalid` is `false`, the function returns `e`. This ensures that only valid key values are returned by `getEvent()`.

2.1.3. The Event Loop

Every interactive program, whether with a text-based user interface or a graphical user interface, has a loop as a central element that receives and processes events or forwards them to the intended recipient for processing.

The `processEvent()` function takes a `char` named `e` as an argument and has no return value. It initializes two static variables `x` and `y` to the approximate center of the terminal window. Static variables are introduced in the Section *Global and Static Variables*. They are initialized when the program is started before `main()` is called. The `getmaxx()` function determines the maximum number of columns of the window that is passed as an argument. Here, `stdscr` is passed as an argument, which refers to the *Ncurses* standard window. As `x` is to be initialized to the approximate middle of the screen width, the result of the call to `getmaxx()` is divided integrally by 2. The coordinate system starts at the top left-hand corner of the terminal window with 0|0. Therefore, 1 must be subtracted from the result. The static variable `y` is initialized with `getmaxy(stdscr) / 2 - 1`. The `getmaxy()` function returns the line number of the window passed as an ar-

gument, which is also `stdscr` in this case. Here too, it is divided integrally by 2 and decreased by 1. A blank character is then output at the `y` and `x` position in the standard window. This is done by calling `mvaddch()`.

The function `mvaddch()` inserts a single character at position `y` and `x` on the screen, in this case the space character. It is important to specify the row first and then the column, i.e. `y` comes first, followed by `x`. All functions whose names begin with *mv* normally move the cursor to the positions `y` and `x` before generating an output. The *add* in the middle of the function name indicates that the corresponding output is permanently inserted at the relevant screen position. It remains there until it is explicitly replaced by another output. Each of the following if statements checks whether it should be executed according to its specific condition. For example, the first if statement checks whether `e` equals `left`. If this is the case, the nested if statement checks whether `x` is greater than 0. If this is the case, `x` is decremented by one, otherwise the else branch causes the screen to flash. Regardless of whether one of the if statements has been executed, `mvaddch()` is called again at the end of the function. It now inserts `symbol` at the screen position specified by `y` and `x`. If one of the coordinates was previously changed, `symbol` now appears at a different screen position. If `processEvent()` is called with a value that is not processed by one of the if statements, only the statements that add a space and `symbol` at position `y` and `x` on the screen are executed. Exactly this behavior is exploited when `processEvent()` is called with 0 as argument in `main()`. In this way, the asterisk is initially placed in the center of the screen.

The control flow of the program is such that it waits for keystrokes until the quit key has been pressed. All keystrokes returned by `getEvent()` are processed by calling `processEvent()`. It should be noted that `getEvent()` only returns valid keystrokes. All other keystrokes are ignored. The core element of this program is the loop that waits for events and processes them. This is exactly the same process as in a program with a graphical user interface. Similar to `gui/Move.cpp`, a GUI program sets up everything that can be drawn and process events, and then starts the loop that receives events and forwards them to the corresponding recipients. All this is realized here bundled in the `processEvent()` function. GUI programs use a more sophisticated mapping of events to code to be executed. In addition, there is not just a single window in GUI programs, but a large number of windows. Therefore, an event must be forwarded to the corresponding recipient window. In a GUI program, everything that is used to output information, input information or interact with the user is usually a window. Thus, the single terminal window is replaced by a multitude of orchestrated windows. In addition, a GUI program can also use the console window.

For the sake of completeness, it should be mentioned that *Ncurses*, like other libraries for text-based user interfaces, can manage several windows.

2.1.4. The Procedural Nature of *Ncurses*

Ncurses is based entirely on the paradigm of procedural programming. For this reason, there is a mixture of function calls, some of which act on the currently active screen, but some of which also affect a screen that must be explicitly identified by an additional parameter. An example of the first category is the `mvaddch()` function. It does not take a specific screen as a parameter, but adds a character to the currently active screen. An example of the second category is the `getmaxx()` function, where the screen must be explicitly specified as a parameter. Here, `stdscr` is used as a parameter, which represents the currently active screen. The purely procedural programming paradigm makes the use of *Ncurses* somewhat cumbersome and confusing. Most GUI libraries and frameworks are based on the object-oriented paradigm and thus offer more stringent options for handling different windows. This also applies to the GUI library *FLTK*, which is presented below.

2.2. FLTK

In the following, the GUI library *FLTK* is introduced. The first GUI program is the port of the text-based user interface program that moves an asterisk.

The acronym *FLTK* stands for *Fast Light Toolkit*. It refers to a GUI library developed in C++ and an interactive GUI designer program called *FLUID*, which means *Fast Light User Interface Designer*. The official homepage for *FLTK* is <https://www.fltk.org> (last accessed on February 13, 2025). *FLTK* is maintained on *GitHub* (<https://github.com/fltk/fltk>, last accessed on February 13, 2025). A valuable source of information about *FLTK* is <https://www.seriss.com/people/erco/fltk/> (last accessed on February 13, 2025). The *FLTK* software and manual are released as open source under the terms of the *GNU Library General Public License*. The stable version of *FLTK* at the time of writing is 1.4.1. It can be installed on *MS Windows*, *Linux* and *macOS* and enables the development of portable GUI applications as well as GUI applications with a platform-specific appearance.

A comprehensive guide for installing *FLTK* on *MS Windows* is (Fleischer 2021), which is somewhat outdated, but can be adapted to the current status with a few considerations. On *macOS*, *FLTK* can be installed by entering `brew install fltk` in a terminal window, provided the *Homebrew* package manager (*Homebrew The Missing Package Manager for macOS (or Linux)* n.d.) has been installed beforehand. On *Linux*, *FLTK* can be downloaded from (Fltk/Fltk 2024) and built from scratch using *CMake* (possibly repeatedly to install any missing libraries) and finally running `make` and `make install`. In any case, the installation instructions should be read carefully and followed with caution. Further information on installing *FLTK* on different platforms cannot be given in this book. *FLTK* can also be used to create applications with a platform-specific look. However, these are no longer portable. In the following, the focus is therefore on portable GUI applications.

FLTK is programmed purely object-oriented. It makes no use of advanced features such as namespaces, exceptions, intelligent pointers or container templates. It also does not know the type `std::string` and many other helpful types and functions of the *C++* standard library. The interested reader is referred to (*Configuration Management Plan - Fast Light Toolkit (FLTK)* n.d.), Section *C++ Portability*, which discusses the decision to prohibit the use of these features in the *FLTK* code itself. This also applies to versions 1.4.x. This restriction does not extend to applications created using *FLTK*.

The functionality of *FLTK* is distributed across many fine-grained header files. Using *FLTK* therefore requires detailed knowledge of what is in which header file. One advantage of this philosophy is that statically linked *FLTK* applications are relatively small.

Now that all this has been explained, it is time for the first *FLTK* program.

2.2.1. Moving an Asterisk

The first *FLTK* program mimics the `gui/Move.cpp` program. It creates a window with a drawing area of 320×240 pixels and displays an asterisk that can be moved horizontally and vertically using the *h*, *j*, *k*, and *l* keys. Pressing the *Escape* key ends the program. Pressing any other key makes a sound. The asterisk cannot be moved beyond a window border. When a border is reached, the screen flashes. Figure 32 shows a screenshot of an extended version of this program. It is not reproduced here, but is included as `gui/GuiMoveWithInfo.cpp` in the accompanying material.

Figure 32: Screenshot of the execution of `gui/GuiMoveWithInfo.cpp`



Figures 31 and 32 are very similar. The differences include the fonts, the font color, the background color of the screen, and the exact alignment of the text in the corners. There is one major difference, namely the coordinates of the corners except for the origin. For example, the top right corner is `0|39` in the console version and `319|0` in the GUI version.

The console version has 12 rows (height) and 40 columns (width), whereby the rows are specified first and then the columns. The available drawing area of the GUI version is 320 pixels wide and 240 pixels high, where a pixel is indicated by its x-coordinate in the first position and its y-coordinate in the second position. The screen starts in the top left-hand corner at 0|0 and extends to $width - 1 | height - 1$, where *width* and *height* refer to the width and height of a window in pixels.

2.2.2. Building an FLTK Application

There are various ways to create and build an *FLTK* application. The simplest and most user-friendly way is presented below. It is also used in the rest of this book.

The *FLTK* installation contains a script called *fltk-config*, which is located in the *FLTK* installation directory. If a terminal window refuses to run this script, it should be checked whether the path variable includes the *FLTK* installation directory. One of the options of this script is to compile and create a platform specific executable *FLTK* program, for example by entering *fltk-config --compile GuiMove.cpp*. In the following, the single source programs are compiled in this way.

If *FLTK* has been installed on *MS Windows* according to (Fleischer 2021), an *MSYS* shell must be started to run *fltk-config* successfully. *MSYS* is the acronym of *Minimal SYStem*, which provides several *Unix* utilities for *MS Windows* based on a compatibility layer. It is an add-on to *MinGW* and should also be installed if the instructions from (Fleischer 2021) have been followed.

When using *fltk-config* errors may be reported because current C++ features cannot be compiled. In this case, it is advisable to check the *fltk-script* and—if necessary—change it. Depending on the given platform and compiler installed, it may use unsuitable or outdated options for calling the compiler. For example, it could contain the line `CXX="clang++ -std=gnu++11"`. This assigns a value to the shell script variable `CXX`, which causes *clang++* to be used as the compiler and *gnu++11* as the C++ standard. However, *gnu++11* is an old version of the C++ standard. As the following programs occasionally use features that require at least C++ 20, *gnu++11* should be replaced by *c++20*. The correspondingly modified script line then looks as follows:

```
CXX="clang++ -std=c++20"
```

As the *fltk-config* script is normally write-protected, overwriting the write-protected file must be forced. If this is not possible, the script must be set to read/write before editing. The read-only status must be restored after the changes have been written.

It must be emphasized that the *fltk-config* script is generated during the creation of *FLTK* on a specific platform and for a specific compiler. Therefore, its content may vary considerably.

If modifying *fltk-config* is not an option, most code examples will still compile successfully. In case of failure, the relevant part (calling the compiler plus all command line options and parameters) can be copied from the command line window, pasted and then changed before pressing the *Enter*-key. In this case, *gnu++11* has to be manually replaced by *c++20*, which is tedious and error-prone.

Listing 84 shows the GUI counterpart of the program from Listing 83.

Listing 84: gui/GuiMove.cpp

```

1 // GuiMove.cpp by Ulrich Eisenecker, February 13, 2025
2
3 #include <FL/Fl.H> // Must be included by each FLTK-program
4 #include <FL/Fl_Window.H> // Because of Fl_Window
5 #include <FL/Enumerations.H> // Because of FL_BLACK, FL_Escape, etc.
6 #include <FL/fl_draw.H> // Because of fl_... functions
7 #include <FL/fl_ask.H> // Because of FL_BEEP_DEFAULT
8
9 const char symbol[] { "*" };
10 const char left { 'h' };
11 const char right { 'l' };
12 const char up { 'j' };
13 const char down { 'k' };
14
15 // MyWindow is a subclass that offers special drawing and
16 // event handling as well as additional functionality.
17 class MyWindow: public Fl_Window
18 {
19     public:
20         // The member variables that manage the cursor position
21         // of the symbol to be drawn are initialized in the constructor.
22         MyWindow(int w,int h): Fl_Window { w,h }
23         {
24             fl_font(FL_HELVETICA,18);
25             fl_text_extents(symbol,m_dx,m_dy,m_width,m_height);
26             m_x = (w - m_width) / 2 - m_dx;
27             m_y = (h - m_height) / 2 - m_dy;
28         }
29         // Overriding and extending the member function draw().
30         void draw() override
31         {
32             Fl_Window::draw();
33             fl_color(FL_BLACK);
34             fl_draw(symbol,m_x,m_y);
35         }
36         // Provision of the flashing of a window as a member function.
37         void flash()
38         {
39             auto oldColor { color() };
40             color(FL_RED);
41             redraw();
42             Fl::wait(0.25);
43             color(oldColor);

```

```
44     redraw();
45 }
46 // Handling of the key events for the movement of the symbol.
47 int handle(int event) override
48 {
49     switch (event)
50     {
51     case FL_KEYBOARD:
52     {
53         auto key { Fl::event_key() };
54         // Unknown keys should be
55         // handled with Fl_Window::handle().
56         if ((key != left) && (key != right) &&
57             (key != up) && (key != down))
58         {
59             fl_beep(FL_BEEP_DEFAULT);
60             return Fl_Window::handle(event);
61         }
62         // Handle known keys now.
63         if (key == left)
64         {
65             m_x -= m_width;
66             if (m_x < m_dx)
67             {
68                 m_x = -m_dx;
69                 flash();
70             }
71         }
72         if (key == right)
73         {
74             m_x += m_width;
75             if (m_x > w() - m_width - m_dx - 1)
76             {
77                 m_x = w() - m_width - m_dx;
78                 flash();
79             }
80         }
81         if (key == up)
82         {
83             m_y -= m_height;
84             if (m_y < -m_dy)
85             {
86                 m_y = -m_dy;
87                 flash();
88             }
89         }
90         if (key == down)
91         {
92             m_y += m_height;
93             if (m_y > h() - 2 * m_height - m_dy)
94             {
95                 m_y = h() - m_height - m_dy - 1;
96                 flash();
97             }
98         }
99     }
100 }
```

```

99         // A known key has been handled.
100        // Therefore, redraw() is called
101        // and 1 is returned 1 to indicate
102        // that the event has been consumed.
103        redraw();
104        return 1;
105    }
106    }
107    // All other events should be handled by Fl_Window::handle().
108    return Fl_Window::handle(event);
109    }
110    private:
111        int m_dx,
112            m_dy,
113            m_width,
114            m_height,
115            m_x,
116            m_y;
117    };
118
119    int main(int argc, char* argv[])
120    {
121        MyWindow window { 320, 240 };
122        window.end();
123        window.show(argc, argv);
124        return Fl::run(); // Start the FLTK event loop.
125    }

```

The procedure for compiling the program is the same on every platform. A console window (an *MSYS* shell under *MS Windows*) is opened for the *gui* directory and the command *fltk-config --compile GuiMove.cpp* is entered. Under *MS Windows*, the result is an executable file called *GuiMove.exe*. In a *MSYS* shell, the command *./GuiMove.exe* must be entered to start the program. In the standard *MS Windows* terminal, it is started by entering *GuiMove*. Under *Linux*, the *GuiMove* file is then available, which can be started by entering *./GuiMove*. Under *macOS*, the files *GuiMove.app* and *GuiMove* are created. *GuiMove.app* can be started by double-clicking on its name in the *Finder*. Alternatively, it is possible to execute *GuiMove* in the console window by entering *./GuiMove*. In fact, *GuiMove.app* is a special type of directory, a so-called *application bundle*, which contains the executable file *GuiMove*, among other things.

First, a rough comparison of *gui/GuiMove.cpp* and *gui/Move.cpp* is made. Details of the various header files and the reasons for their inclusion are then discussed.

2.2.3. GuiMove.cpp vs. Move.cpp

The two previously presented programs with a text-based user interface and GUI are now compared in detail. Both the similarities and the differences between them will be highlighted.

Unlike `gui/Move.cpp`, `gui/GuiMove.cpp` includes six header files. *FLTK* requires that all its header files are present in the *FL* sub directory in the include directory, where the C++ compiler searches for header files by default. The names of most *FLTK* header files start with the prefix *FL_* or *fl_*. This seems to be redundant, as all the header files are located in the *FL* sub directory, but should not be changed, as otherwise the demo and test programs supplied with *FLTK* can no longer be compiled. The two exceptions without a prefix are `<FL/Enumerations.H>` and `<FL/Fl.H>`.

Another special feature is that every *FLTK* header file ends with *.H*. This differs from the standard header files, which have no extension, and from most *C* header files, which end with *.h*. The `<FL/Fl.H>` header must be included by every *FLTK* program, as it contains the *FLTK* event loop. The header file `<FL/Enumerations.H>` defines many constants for different purposes as enumeration constants or as macros. It is therefore also included by most *FLTK* programs. The header file `<FL/fl_draw.H>` provides drawing functions and `<FL/fl_ask.H>` defines the enumeration constant `FL_BEEP_DEFAULT` for the acoustic standard signal. The header file `<cstdlib>` contains the `exit()` function, with which the program is terminated normally and all necessary clean-up work is carried out.

The sections with definitions of constants are almost identical, with two exceptions. Firstly, the *Escape* key is not defined in `gui/GuiMove.cpp`, as `<FL/Enumerations.H>` defines the macro `FL_Escape` for precisely this purpose. Furthermore, it is not necessary to react to `FL_Escape`, as this is already handled accordingly by the `Fl_Window` base class. Secondly, `symbol` is not defined as a character constant, but as a constant pointer to constant characters, i.e. a classic *C*-string, since *C*-strings are the only type of strings *FLTK* recognizes. *FLTK* is not able to draw a value of type `char`. Therefore, the `symbol` to be drawn must be the only character in a *C*-string.

The free function `getEvent()` of `gui/Move.cpp` has no counterpart in `gui/GuiMove.cpp`, as the event loop of *FLTK* automatically receives and forwards events from the keyboard or mouse. The `processEvent()` function of `gui/Move.cpp` corresponds directly to the `MyWindow::handle()` member function. It checks for keyboard events, i.e. key presses, and executes key-specific code. The `MyWindow` class also defines the member functions that draw the window content and make a window flash. The `MyWindow::draw()` member function could also have been integrated directly into `MyWindow::handle()`, as it is relatively small and is only called once. This is different with `MyWindow::flash()`. It is needed because *FLTK* does not provide a function that corresponds to `flash()` in *Ncurses*, and it is called several times, which justifies its definition with a descriptive name. The `MyWindow` class must be defined because there is no global screen or window as in *Ncurses*. Therefore, either an exemplar of an *FLTK* window or a user-defined window class must be instantiated.

The two `main()` functions are partially similar. Despite some preliminary work, the `main()` function of `gui/Move.cpp` also contains the event loop, which is implemented here as a do-while loop. This is not necessary in `gui/GuiMove.cpp`. Here, `main()` calls `Fl::run()`, which implements the event loop of *FLTK*. The `main()` function of `gui/GuiMove.cpp` also contains less pre- and post-configuration code, as this is executed automatically by *FLTK*.

2.2.4. The `main()` Function of `GuiMove.cpp`

The `main()` function of an *FLTK* program should normally be defined with two parameters, namely the number of command line arguments and a C-string array containing the command line parameters. In addition to `main()` without parameters, this is the second overloading of the `main()` function as defined in the C++ standard. Both overloads return an `int` value that provides information on how the program was terminated and which can be checked by the calling process, e.g. a console window.

The name of the program to be executed can be accessed as the first entry in the array that is passed as the second parameter, here `argv[]`, i.e. `argv[0]`. Instead of `char* argv[]`, `argv` could also be declared as `char ** argv`, which is a pointer to any number of pointers to `char`. A pointer to `char`, i.e. `char*`, is the standard C-string terminated by a ASCII null byte. The `argv` identifier is followed by an empty pair of square brackets to emphasize that it can be treated as an array. Since a program must always be named for its execution (there are no unnamed programs), the value of the first parameter, here `argc`, must be at least 1. The following arguments correspond to the entries in the `argv[]` array from 1 to `argc - 1`.

The program in Listing 85 illustrates this. When the elements of `argv[]` are output in the for loop, no range check is performed. At the end, `main()` returns `argc`. After compilation with `g++ -std=c++20 Main.cpp -o main.exe`, execution in a *MS Windows PowerShell* with `./main one 2 t-h-r-e-e F_O_U_R -5.still,5` prints the result shown in Figure 33.

Listing 85: `gui/Main.cpp`

```

1 // Main.cpp by Ulrich Eisenecker, February 5, 2024
2
3 #include <iostream>
4 using namespace std;
5
6 int main(int argc, char* argv[])
7 {
8     for (int index { 0 }; index < argc; ++index)
9     {
10         cout << index << '\t' << argv[index] << endl;
11     }
12     return argc;
13 }
```

Figure 33: Output of executing gui/Main.cpp

```

0      C:\Users\ueise\Nextcloud\app\sources\gui\main.exe
1      one
2      2
3      t-h-r-e-e
4      F_O_U_R
5      -5.still,5

```

The command line interpreter first tokenizes the input in the terminal window. Normally, the first token is the name of the program to be executed. Each subsequent token that is not syntactically relevant for the command line interpreter is passed to the called program as a *command line argument* (also known as an *option*). As soon as the first token valid for the command line interpreter is found, the rest of the command line, including this token, is not passed to the called program. Tokens that are relevant for the command line interpreter are e.g. `|` (pipe symbol), `>` (output redirection) and `<` (input redirection).

Since `main()` returns `argc`, entering `echo $?` in the console window of a *Linux* or *macOS* terminal immediately after calling `main.exe` will output the number 6, i.e. the number of tokens passed to the called program, including the name of the program itself. `$?` is a so-called *environment variable*. The name of this variable is simply `?`, and the prefix `$` has the effect, that `?` is recognized as a variable. This cannot be reproduced directly under *MS Windows*. Here, a so-called *batch file* (called *shell script* under *Linux* or *macOS*) must be written to output the return value of `main()`, as shown in Listing 86.

Listing 86: `run.bat` (MS Windows batch file) executing `main.exe`

```

1 | echo off
2 | main one 2 t-h-r-e-e F_O_U_R -5.still,5
3 | echo %errorlevel%

```

The execution with `./run` in a *PowerShell* results in the output shown in Figure 33 plus an additional line with 6.

Now back to the program `gui/GuiMove.cpp`. The first statement in `main()` declares window as an exemplar of the class `MyWindow`. `MyWindow` is defined at the beginning of the program and explained below. The values in the initialization list of window represent the number of usable pixels in the width and height of the window. After the constructor has been executed, child windows, so-called *widgets* (a portmanteau of *window* and *gadget*, i.e. any window-like thing that can be displayed) can be added. This is not done here. Instead, `window.end()` is called in the next statement, which ends the configuration of the window exemplar. Next, the window is displayed using

`window.show(argc,argv)`. The member function `F1_Window::show()` takes `argc` and `argv[]` as arguments. This is relevant as GUI applications can also be called directly from a console window with additional command line arguments.

The last statement in `main()` returns the result of the call to `F1::run()`. `F1::run()` starts the event loop. It is a static member function of the class `F1`, which is defined in `<FL/F1.H>`. Each *FLTK* program must start the event loop and must therefore include the header file `<FL/F1.H>`. Since *FLTK* does not use namespaces, the `F1` class can be seen as a substitute for a namespace to avoid global variables and global functions (among others) by defining them as static class members. The fact that almost all *FLTK* identifiers are prefixed with *FL_*, *Fl_*, *fl_* or similar corresponds to the classic *C* convention for avoiding name collisions, because *C* does not recognize namespaces. In addition to the `F1` class, `<FL/F1.H>` contains macros, (pending) type declarations, function declarations and some inline definitions, many of which neutralize platform-specific features, and documentation comments.

2.2.5. The MyWindow Class

The `MyWindow` class demonstrates how to draw in a window and how to handle events. The former is achieved by overriding the `draw()` member function, the latter by overriding the `handle()` member function. Additional information is provided on fonts and drawing text.

When reading the following sections, it is advisable to have the *FLTK* documentation to hand, either offline (PDF) or online (Costantini, Melcher, et al. 2025).

2.2.5.1. Base Classes of MyWindow

`MyWindow` is defined as a public subclass of `F1_Window`. Therefore, the program includes `<FL/F1_Window.H>`. An exemplar of `F1_Window` can act either as a top-level window or as a child window. `F1_Window` offers many member functions for customizing its behavior and appearance. For drawing the asterisk and handling the key commands, the member functions `F1_Window::draw()` and `F1_Window::handle()` are overridden in the `MyWindow` class. `F1_Window` is a subclass of `F1_Group`. `F1_Group` is the *FLTK* container widget because it manages any number of widgets, including exemplars of `F1_Group` itself. It is also a base class for many other specialized widgets that coordinate other widgets. `F1_Group` is a direct subclass of the abstract class `F1_Widget`, which is the base class of all widgets.

2.2.5.2. Overriding the draw() Member Function

`Fl_Widget::draw()` is a pure virtual function. It is overridden by `Fl_Group::draw()`, which calls the `draw()` member function for all widgets it manages. `Fl_Window::draw()` also overrides `Fl_Group::draw()`, but cannot be made to display the asterisk. To do this, `Fl_Window::draw()` must be overridden and extended in the `MyWindow` class (see Section *Implementation of the UrnO Class* for overriding and extending). Please note the use of the override specifier in the declaration of the member function. *FLTK* does not use this feature, but it can be safely applied when declaring new subclasses of *FLTK*.

2.2.5.3. Overriding the handle() Member Function

`Fl_Widget` is the base class that defines the virtual member function `handle()` the first time. The only parameter of this member function is an integer value that identifies the passed event. This member function is overridden and extended in `Fl_Group`. It is not overridden in `Fl_Window`. In order to process the keystrokes for moving the asterisk, `handle()` must also be overridden and extended in `MyWindow`. The member function `handle()` must return 1 to indicate that the event has been *consumed*, i.e. *used*, and that it can be deleted. A return value of 0 means that the event has not been processed. If an event has not been consumed by the receiver object, the `handle()` member function defined in a base class could consume it. In this case, the return value of the `handle()` member function of the base class should be returned, e.g. `return Fl_Window::handle(event)` (as in `MyWindow::handle()` in Listing 84). This gives the base class object the chance to process the event. If 0 is returned, *FLTK* attempts to send the event to another suitable widget, which may prevent processing by the `handle()` member function of the base class. `MyWindow::handle()` is also specified with `override`.

The use of a switch statement for event is a little too complex here. It could certainly have been replaced by an if statement that checks whether event equals `FL_KEYBOARD`. Nevertheless, a switch statement is used to indicate that switch statements are the usual means of processing events in event handlers. In the only case branch, the key that triggered the `FL_KEYBOARD` event is retrieved with `Fl::event_key()` and used to initialize the auto variable `key`. It is then first checked whether it is a known key. If not, a beep is emitted on platforms that support this by calling `fl_beep(FL_BEEP_DEFAULT)`. Then the `handle()` member function is exited by returning the result of calling the `handle()` member function of `Fl_Window` and passing event as an argument.

The following if statements check for known keys and execute the corresponding action. After a known key has been handled, the window is marked for drawing by calling `redraw()`. The `draw()` member function must not be called directly, as only *FLTK* decides when it needs to be called and does so if necessary. The `handle()` member function is then exited with `return 1;`. As already mentioned, the return value 1 means that the event has been consumed.

The switch statement is followed by another statement that returns the value of the call to `Fl_Window::handle()` with `event` as an argument. This is necessary because the base class could process all other events, such as mouse events, which are not processed by `MyWindow::handle()`.

2.2.5.4. Data Member and Constructor of MyWindow

Next, the `MyWindow` class is analyzed in detail. It defines a series of private data members, namely `m_dx`, `m_dy`, `m_width`, `m_height`, `m_x` and `m_y`. They are all initialized in the `MyWindow` constructor. The `MyWindow` constructor takes the arguments `w` and `h`, which are both of type `int` and passed by value. They represent the width and the height of the window to be created. They are therefore passed as arguments for the constructor call of `Fl_Window` in the initializer list.

The `Fl_Window` class defines two constructors. The first, which is called here, accepts three arguments. The third parameter is a pointer to a `const char` with 0 as default value. In modern C++, a pointer should be initialized to `nullptr` and not 0 to indicate that it points to nothing and therefore should not be dereferenced. Since the third parameter is missing in the initializer list, the title of the `MyWindow` exemplar is *FLTK*. This constructor always places the window in the top left-hand corner of the screen.

The second `Fl_Window` constructor takes 5 arguments, the four integer values `x`, `y`, `w` and `h` as well as `title`, a pointer to `const char` with 0 as default value. When this constructor is called, the top left corner of the window appears at the position specified by `x` and `y` on the screen, and is `w` pixels wide and `h` pixels high.

2.2.5.5. About Fonts and Font-related Functions

The first statement in the `MyWindow` constructor, `fl_font(FL_HELVETICA,18);`, sets the font type, called *font face* or simply *font* in *FLTK*, and the font size. The `fl_font(Fl_Font,Fl_Fontsize)` function is declared in `<FL/fl_draw.H>`. Its first parameter specifies the font, the second parameter the font size. The first parameter used here is `FL_HELVETICA` of type `Fl_Font` (an alias for `int`), which identifies an *FLTK* standard font. The second parameter is 18 as font size. It is of type `Fl_Fontsize`, also an alias for `int`. This indicates that `Fl_Font` and `Fl_Fontsize` may be defined differently in future versions of *FLTK*. Hence, variables representing a font or a font size should be declared with their corresponding *FLTK* type names. `Fl_Font` and `Fl_Fontsize` are defined in `<FL/Enumerations.H>`, which is one of the reasons why `gui/GuiMove.cpp` includes this header file.

Each operating system comes with its own fonts, which are sometimes subject to proprietary licenses that restrict their use. In addition, all these fonts have individual characteristics that differ from platform to platform. This is the reason why *FLTK* logically defines its own palette of fonts. It includes a mono-spaced font (`FL_COURIER`), a grotesque font, i.e. a font without serifs (`FL_HELVETICA`), a serif font (`FL_TIMES`), fonts with symbols

and icons (`FL_SYMBOL`, `FL_ZAPF_DINGBATS`) and a mono-spaced screen font (`FL_SCREEN`). In addition, some of fonts can be modified so that they appear in bold or italics. All font numbers are defined in `<FL/Enumerations.H>`. It still needs to be explained what “logically defined” means. In fact, *FLTK* does not contain any fonts by itself. Rather, it assumes that some fonts are available on most operating systems and—if not—can be mapped to more or less suitable substitute fonts, which is impressively illustrated in figures 34 and 42.

In order to write programs that are portable and look more or less the same on all platforms, the *FLTK* fonts should be used.

Figures 34 and 35 show the *FLTK* fonts without their variations in bold or italics. They are screenshot under *macOS* and *MS Windows* of the *gui/Fonts.cpp* program, which is included in the accompanying material but is not presented here. The grid is 25 pixels wide and the font size has been set to 18 for all fonts. At the beginning are characters that are particularly small, wide, extend far down far up or both. This is followed by the name of the font as retrieved from the array `fl_fontnames`, which is defined inline in `<FL/names.h>`. As can be seen, the text examples for the individual fonts look very different. All texts are drawn with the value 4 for the x-position, but they all have a different width. The same applies to their height: `FL_SYMBOL` appears much smaller than `FL_SCREEN`.

Figure 34: FLTK font faces (screenshot of *gui/Fonts.cpp* under MS Window)

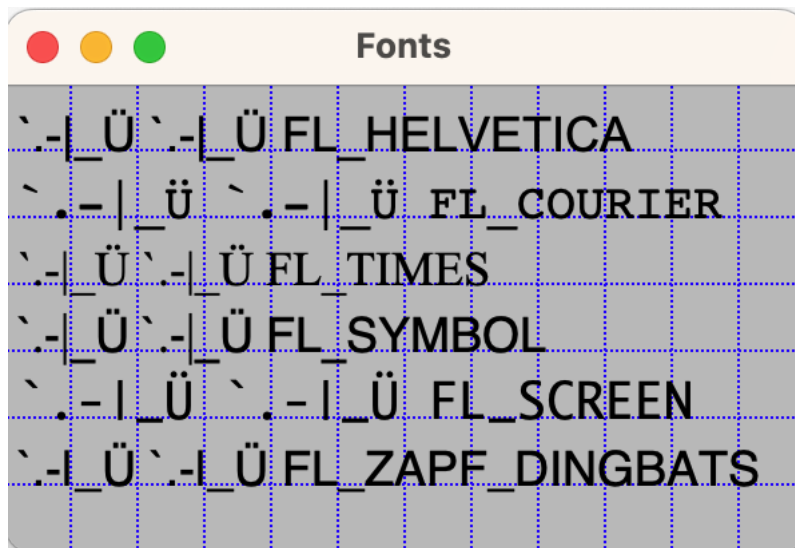
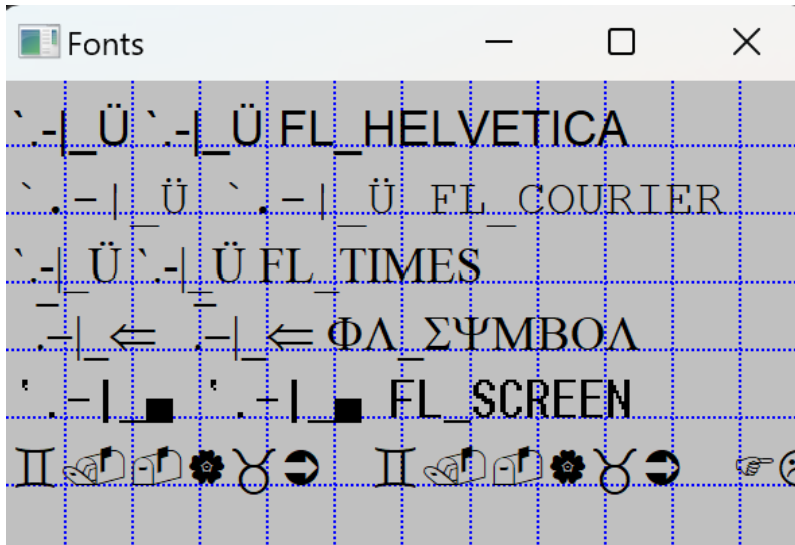


Figure 36 illustrates further characteristics of fonts. The characters of a font are aligned along a *baseline* on which most of the characters rest. In the first line of text, the dashed red line indicates the baseline of the font. The text in the second line has exactly one

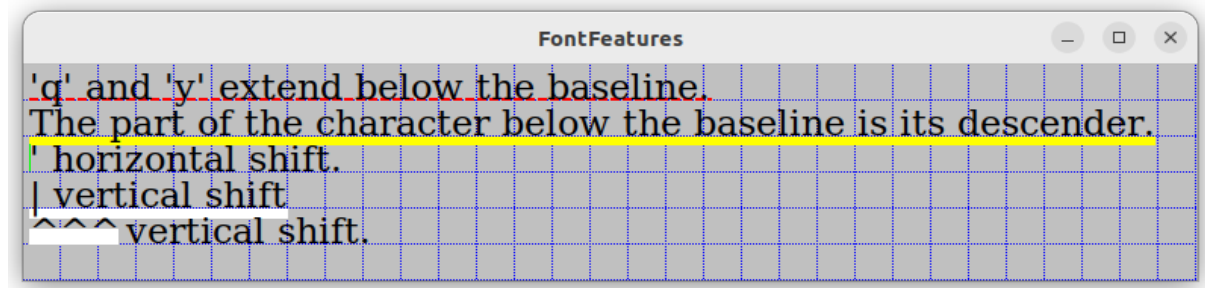
Figure 35: FLTK font faces (screenshot of gui/Fonts.cpp under MS Window)

character, the p , which lies below the baseline. The part of a character that extends below the baseline of a font is called *descender*. The character with the longest descender forms the descender of a text.

In a *proportional* font, each character has its own width, while in a *mono-spaced* font, all characters have the same width. In both cases, a character has a certain distance to its left hand. The third line illustrates this horizontal shift to the right. If the first pixel of the character is to be drawn exactly at the specified x-position, this right shift must be subtracted beforehand, as the vertical green bar in text line three shows. The subtraction of this right shift is particularly important if text in different fonts is to be drawn left-aligned.

Text line four illustrates the vertical shift caused by characters with descenders. If all characters are not to extend below the line of the grid, the height of the white box must be taken into account. By the way: Looking closely at the beginning of the fourth line, it can be seen that the horizontal shift has been retained. The last line shows a variant resulting from the use of characters that do not extend to the baseline. If this text was to be aligned to the bottom grid line, the height of the white box had to be added to the y-position for drawing the text. Compiling and running the program *gui/Font-Features.cpp*, which is only included in the accompanying material, under *MS Windows* probably shows a different last line. At least on the test computer under *MS Windows*, the height of a text with characters that do not extend to the baseline was always calculated incorrectly.

The font `FL_ROMAN` font with a size of 24 was selected for drawing the text in Figure 36. When comparing Figure 36 with Figure 34, it is easy to see that the same font looks different on different platforms.

Figure 36: Further font features (screen shot of gui/FontFeatures.cpp under Ubuntu)

One could argue that fonts that look different on different platforms are not really helpful. But the advantage of the standard *FLTK* fonts is that they look similar in some cases and a program can rely on their presence.

2.2.5.6. Drawing Text

FLTK offers several overloads of the `fl_draw()` function for drawing text. In the following, only the first overload will be considered. It takes three arguments, a character string that is passed as `const char *` and the `x` and `y` position (both of type `int`) of the lower left corner for drawing the character string. The null-terminated string contains characters in *UTF-8 encoding*. UTF-8 retains the ASCII character encoding for values between and including 0 to 127 and uses a variable number of bytes for encoding characters beyond the ASCII (see (UTF-8 2024) for more information). The `fl_draw()` function calls the underlying function of the operating system directly. It draws the character string in the currently set font and font size without taking into account the features of the font. This is not sufficient, as it is not possible to draw text that is adjusted to a specific vertical or horizontal line.

This is where the `fl_text_extents()` function comes into play, which is declared in the `<FL/fl_draw.H>` header file. There are two overloads of `fl_text_extents()`, only one of which is considered here. Once the font and font size have been set, the statement `fl_text_extents(symbol,dx,dy,width,height);` calculates the width and height in pixels of the C-string passed as the first argument, in this case `symbol`, as well as the necessary shift in the `x` and `y` directions in order to align the text horizontally and vertically in a fictitious box. The origin of this fictitious box is the top left corner, and the calculated height respects the descenders of the characters in the string. The calculated values are stored in the variables `dx`, `dy`, `width` and `height`, which are passed by reference. The height and width of this fictitious box naturally depend on the font and the font size, i.e. its height, and the number of characters in the C-string. On different platforms, the values of `width`, `height`, `dx` and `dy` can be different despite the same font and font size.

Figure 37: Aligning text under macOS (left), Ubuntu (middle), and MS Windows (right)

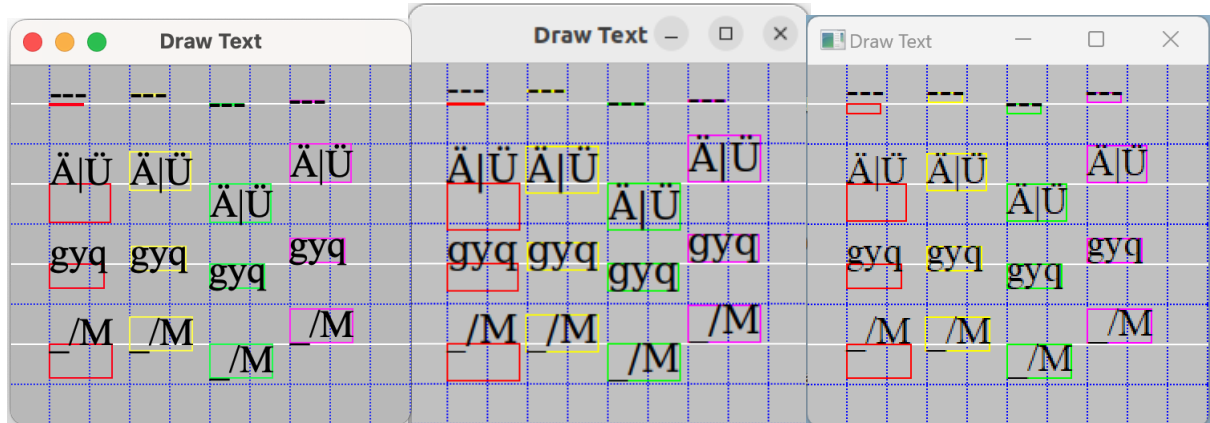


Figure 42 illustrates the meaning of the values `dx`, `dy`, `width` and `height`. It shows screenshots of the `gui/DrawText.cpp`, which was compiled and executed on *macOS*, *Ubuntu*, and *MS Windows*. This program is not shown here, but is included in the accompanying ma-

terial in the *gui* folder. Each of the three windows contains a grid with a width and height of 25 pixel. In the following, the coordinate pair $x|y$ denotes the screen position of the upper left corner for drawing a rectangle. For example, the top left corner of the first red rectangle is drawn at $x = 24$ and $y = 24$, the top left corner of the lowest magenta rectangle on the right is drawn at $x = 174$ and $y = 174$. The white horizontal lines serve as a reference for vertical alignment. The dashed vertical lines are references for horizontal alignment.

The first column with the red box shows the result of drawing text exactly at the coordinates $x|y$ without taking dx , dy , width and height into account. As already mentioned, the starting point for drawing text is the bottom left corner. The red rectangles are drawn at the same $x|y$ coordinates, ignoring dx , dy , width and height. Therefore, the lower left corners of the rectangles start exactly at the specified $x|y$ coordinates. As can be seen, the texts and rectangles are offset from each other. This means that despite having the same $x|y$ coordinates for drawing text and rectangles, the text and rectangles are not aligned because the drawing of text and rectangles has different starting points.

The second column with the yellow boxes shows the result of drawing text at the $x|y$ coordinates and rectangles at the $x + dx | y + dy$ coordinates. Now text and rectangles are almost well aligned, but overall they are not aligned to the specified $x|y$ coordinates, as shown by their different placement compared to the red rectangles on the left.

The third column with green rectangles draws the text at the coordinates $x - dx | y - dy$ and the rectangles at the coordinates $x|y$ with width and height. This is the desired and intended result. The top of the rectangles and the text are aligned with the white line that goes horizontally through y , and their left side is aligned with the dashed line of the grid that goes vertically through x .

The fourth column with magenta rectangles shows a variant of the third column with green rectangles. Here, the text is drawn at the coordinates $x|y - \text{height} - dx$ and the rectangles at the coordinates $x | y - \text{height}$.

Here too, the texts can vary greatly in height and width, e.g. the rectangle for `---` with small and thin letters is significantly smaller than the rectangle for `_ / M` with tall and wide letters.

A closer look at the results shows that the rectangles do not always fit perfectly around the text. In particular, the first line of the *MS Windows* version shows a rectangle that is too large for the text. As already mentioned, the height of a text with characters that do not reach the baseline is calculated incorrectly.

gui/DrawText.cpp also outputs to the standard output stream, which can be redirected to a text file. Table 13 contains the summarized output of the program when running on all platforms in the order *macOS*, *Ubuntu* and *MS Windows*. Interestingly, the numbers for *Font* are always the same, namely 8, but the *g* in the center of Figure 37 (*Ubuntu*) differs significantly from the *g* on the left (*macOS*) and right (*MS Windows*). Different values can be reported for one and the same operating system due to different versions.

Table 13: Platform-specific text extents under macOS (left), Ubuntu (middle), and MS Windows (right)

	ma- cOS						Ubuntu						MS Win- dows					
Text	Font	Size	Dx	Dy	Width	Height	Font	Size	Dx	Dy	Width	Height	Font	Size	Dx	Dy	Width	Height
—	8	26	1	-6	22	2	8	29	0	-8	24	2	8	27	0	-6	22	6
Ä Û	8	26	0	-20	39	25	8	29	-1	-23	46	30	8	27	0	-19	39	24
gyq	8	26	1	-11	35	16	8	29	0	-13	45	18	8	27	0	-11	34	16
_M	8	26	0	-17	40	22	8	29	-1	-18	46	24	8	27	0	-17	40	22

Although the font size in the program is set to 24, each platform reports a different value for size. Interestingly, the *Height* of --- on *MS Windows* is 6 pixels, which is not appropriate and could explain the displacement and distortion of the rectangle enclosing --- in Figure 37. Most of the values of the columns labeled *Dx*, *Dy*, *Width* and *Height* are different, which also indicates that the fonts are different for each platform, although they are all identified as *Font* 8.

2.2.5.7. MyWindow Constructor Revisited

Since setting the values of the `MyWindow` data members `m_dx`, `m_dy`, `m_width` and `m_height` requires the call of `fl_text_extents()`, which in turn requires the preceding call of `fl_font()`, these data members cannot be initialized in the initialization list of the constructor. Instead, they must be assigned the corresponding values in the constructor body. The data elements `m_x` and `m_y` represent the screen coordinate at which the upper left corner of the symbol text is to be drawn. In the statement `m_x = (w - m_width) / 2 - m_dx`; the calculated `m_width` of the symbol is first subtracted from `w`, the width of the screen. The result is divided by 2. This represents the left corner for drawing the symbol. If the difference is even, the symbol cannot be placed exactly in the middle of the screen width and is moved half a pixel to the right. Subtracting 1 to match the screen origin `0|0` is also not a valid option as the text would be moved half a pixel to the left. If the difference is odd, the result of the division by 2 is always rounded to the nearest smallest natural number. In this case, the text is centered exactly horizontally. Finally, `m_dx` must be subtracted to compensate for text with wide or thin characters. In principle, the same procedure is used to calculate the value of `m_y`. The `MyWindow` exemplar is now fully initialized for further use by its constructor.

2.2.5.8. `MyWindow::draw()`

One of the reasons for defining a new window class is to give it a specific appearance. The member function `MyWindow::draw()` overrides and extends the member function `Fl_Window::draw()`. Therefore, the first statement of this member function calls the method of the base class. Next, the color for drawing is set to `FL_BLACK`. Otherwise, drawing would be done in the currently active color, which is not necessarily black.

This would also be a place to set the font and font size. It is known here that both values are only set in the constructor and are not changed afterwards. In practice, other drawing operations could have changed both values. Therefore, it would make more sense to define the font and font size in one place in the program and set font and font size to these values before drawing text. A precautionary measure would have been to retrieve and save the font and font size before changing them and restore these values after the respective drawing operations have been completed. However, this would be superfluous if we assume that all other functions that draw something set their specific values immediately before drawing. The strategy of always setting everything needed to draw and not undoing these changes results in less code, but can be unfriendly with respect to other code. The policy of saving, setting and restoring values leads to more code, but can prevent unwanted effects in other code. However, the actual code follows neither the first nor the second strategy. According to (*Configuration Management Plan - Fast Light Toolkit (FLTK)* n.d.), *FLTK* itself should follow the first strategy. After upgrading from a previous version to *FLTK* 1.4.1, it was discovered that initially retrieving the font currently in use by calling `fl_font()` and then setting a different font size for the previously retrieved font causes a segmentation fault. This change is also a strong support for the first strategy.

The last statement draws the C-string symbol at position `m_x` and `m_y`.

2.2.5.9. `MyWindow::flash()`

FLTK does not provide a method for flashing a window. Therefore, the functionality corresponding to `flash()` in *Ncurses* is implemented in the member function `MyWindow::flash()`. This is done according to the save, set and restore policy mentioned in the section above. It calls the `Fl_Widget::color()` member function to initialize the variable `oldColor`, whose type is derived from the initialization value before the auto declaration. The overloaded member function `Fl_Widget::color()` is then called to set the color to `FL_RED`, which is defined in `<FL/Enumerations.H>`.

`Fl_Widget::redraw()` is called thereafter. This indicates that the `draw()` member function of the receiver object should be called by the *FLTK* runtime. As pointed out before, `Fl_Widget::draw()` or an override of this function is never called directly, as *FLTK* decides when to call a `draw()` method. Why is that? Imagine that the widget to be drawn is currently not visible, for whatever reason. In this case, drawing can either be delayed or completely ignored. In the meantime, the corresponding widget can even be marked

as to be drawn several times. However, `draw()` is not called until the widget becomes visible again. It is also possible for only the last call of `draw()` to be executed. In this way, *FLTK* avoids drawing the widget unnecessarily.

Next, `F1::wait()` is called. There are two overloads of this static member function. The first version takes no arguments and temporarily passes control to the *FLTK* runtime. It waits until an event occurs and then returns. The second overload takes a double value that specifies a maximum wait time in seconds. If an event occurs, it can return earlier, but it will return after the specified time in seconds at the latest. The return values of the two overloaded member functions have a platform-specific meaning. They are therefore not dealt with here.

After `F1::wait()` has returned, the color is reset to the saved value `oldColor` and the window is marked for drawing again by calling `redraw()`.

2.2.5.10. `MyWindow::handle()`

This section resumes the discussion of `MyWindow::handle()`. Some aspects are described repeatedly, but with more details.

Another reason to define a new widget or window class is to implement a specific behavior that often occurs in combination with a different appearance. For this purpose, `MyWindow::handle()` overrides and extends the member function `FL_Window::handle()`. Its only argument `event` is of type `int` and is passed by value. Normally its value is compared with constants that provide information about the type of the event (not the type of the event variable, which is `int`), i.e. whether it comes from the keyboard or the mouse. Normally this is done in a large switch statement. For this reason, the `gui/GuiMove.cpp` program also uses a switch statement, but only with a single case. If `event` equals `FL_KEYBOARD`, which is a synonym for `FL_KEYDOWN`, it tells that a key has been pressed. In this case, the static member function `F1::event_key()` is called to initialize the variable `key`, which is declared as `auto`.

There are two overloads of `F1::event_key()`. The first takes no argument, the second takes an `int` value as an argument. The first overload returns the key code that was last pressed. The second overload returns `true` if the key with the key code passed as argument was pressed during the last event, `false` otherwise.

The first overload of `F1::event_key()` is used here. It should be noted that calling this function returns a key code that does not always match the character or text returned by pressing the specific key, e.g. hitting the key labeled 5 on the alphanumeric part of the keyboard returns a key code that matches the character 5, while hitting 5 on the numeric keypad with *Numlock* enabled returns a different key code. In order to always receive the character 5 in this case, the function `F1::event_text()` should be called. It returns a null-terminated, i.e. classic *C-string*. However, this *C-string* can also contain additional ASCII zero values in between. Therefore, the function `F1::event_length()`

must be called as well, which returns the number of characters contained in the *C-string* returned by `F1::event_text()`. As the program only checks whether the key pressed contains lowercase letters, calling `F1::event_key()` is sufficient.

The first if statement checks whether key does not correspond to left, right, up or down. In this case, `f1_beep()` is called. To use `f1_beep()`, the header file `<FL/fl_ask.H>` must be included. The function has a single argument of type `int`, for which `FL_BEEP_DEFAULT` is specified as the default value. However, this default value is explicitly passed as a parameter here to indicate that other values can also be used. The possible values for this are defined as enumeration constants of enum `F1_Beep` in `<FL/fl_ask.H>`. `F1_Beep` is an enum in the old *C* style whose enumeration values are defined in the enclosing namespace. This differs from enum as presented in (U. W. Eisenecker 2022), Section 4.1 *Enumeration Types*. Finally, the result of the call to `F1_Window::handle()` is returned to allow the event handler of the base class the opportunity to process this key event, as explained above. For example, pressing the *Escape* key is handled by `F1_Window::handle()` and the program is terminated, which is the standard behavior of an *FLTK* window. If `F1_Window::handle()` were not called here, this would no longer be possible.

Next comes a series of if statements that check the equality of key and certain characters. This series of if statements could also have been written as a switch statement, which would have improved readability and made the code slightly smaller. This was not done here in order to distinguish it more clearly from the outer switch statement, which checks for certain event types.

The four following if statements have the same structure. First, they execute the specific change of `m_x` by subtracting or adding `m_width` or of `m_y` by subtracting or adding `m_height`. Secondly, they check whether the result is still within the limits of the window. If this is not the case, the value is set so that the content of `symbol` is exactly aligned with the corresponding border. In addition, `flash()` is executed to give a visual signal that the movement has pushed the symbol to an edge. All these if statements have only one then branch.

After the last if statement, a known key value has been processed. This requires the window to be updated. Therefore, `redraw()` is called.

After calling `redraw()`, the `handle()` function is exited with `return 1;`. The return of 1 means that the event has been *processed* by the `handle()` function. Synonyms for processed in this context are *used* or *consumed*. The return of 0 means that the event is not known in this override of the `handle()` member function and should therefore be discarded. However, this prevents other `handle()` functions from receiving and possibly processing this event. Therefore, a third option is to call the `handle()` function of the base class, here `F1_Window`, and return its result, as is done at the end of the implementation of `MyWindow::handle()`.

2.2.6. Portability of FLTK Programs

Portability has many facets, among others:

- The same binary code can be executed on different platforms, e.g. operating systems or web browsers (*binary code portability*).
- The same source code can be compiled on different platforms (*source code portability*).
- The program compiled on different platforms looks the same on each platform.
- The program compiled on different platforms behaves the same on all platforms.

Like compiled C++ programs, executable *FLTK* programs are also platform-specific. But in most cases, the same *FLTK* source program can be successfully compiled into an executable program on different platforms. However, the platforms differ widely in terms of the look of their applications and their behavior. Thus, *FLTK* can only compensate for these differences to a certain extent, but not completely.

2.2.6.1. Different Look and Behavior

The same *FLTK* program can look different when compiled and executed on different platforms, as Figure 37 demonstrates. One reason for this is that this first overload of `fl_draw()` directly calls the native drawing functions of the underlying operating system.

Not only can the appearance of the widgets vary, but the events can also be generated differently. On *macOS*, drawing keys must be pressed repeatedly, i.e. pressed and released, to move the asterisk in the `gui/GuiMove.cpp` program. On *MS Windows* and *Ubuntu*, holding down a key will still generate `FL_KEYBOARD` events. This makes a big difference when using the program.

2.2.6.2. Information about Events

Information about events and differences in the generation of events can be further investigated using the means available in *FLTK*. The program in Listing 87 defines the `EventsWindow` class. It first imports all constructors from its base class so that they do not have to be defined redundantly. Then it overrides and extends the member function `Fl_Window::handle()`.

Here, the static variable `c` is declared and initialized with 0 (see Section *Global and Static Variables* for static variables). The next statement outputs the value of `c`, incrementing it, and also outputs `fl_eventnames[event]`. `fl_eventnames` is a C-array of C-strings defined in `<FL/names.h>`. The event variable contains a value that represents the category of an event. By indexing the `fl_eventnames` array with `event`, the name of the event category can be determined as a C-string. This name is sent to `cout`.

Then it is checked whether event is equal to FL_KEYBOARD. Instead of FL_KEYBOARD, the enumeration constant FL_KEYDOWN could be used as well, as it is defined with the same value. If this is the case, additional information is sent to cout, namely the *event key* (retrieved by Fl::event_key()), which is an integer, and the *event text* (retrieved by Fl::event_text()), which is the result of a key combination, e.g. *Shift* plus key labeled *o* results in *O*. In this example, pressing the *Shift* key generates an FL_KEYDOWN event, pressing the *o* key generates an FL_KEYDOWN event, releasing these keys generates FL_KEYUP events, and the result is *O* as a C-string, i.e. *O* is followed by an ASCII zero to terminate the C-string. The function returns Fl_Window::handle(event) as the result.

Listing 87: gui/Events.cpp

```
1 // Events.cpp by Ulrich Eisenecker, April 12, 2024
2
3 #include <FL/Fl.H> // Must be included by each FLTK-program
4 #include <FL/Fl_Window.H> // Because of Fl_Window
5 #include <FL/names.h> // Because of fl_eventnames
6 #include <iostream>
7
8 class EventsWindow: public Fl_Window
9 {
10 public:
11     // Make all constructors of Fl_Window available for EventsWindow.
12     using Fl_Window::Fl_Window;
13     int handle(int event) override
14     {
15         static unsigned long long c { 0 };
16         std::cout << c++ << '\t' << fl_eventnames[event];
17         //if (event == FL_KEYBOARD)
18         if (event == FL_KEYDOWN)
19         {
20             std::cout << '\t' << Fl::event_key() << '\t' << Fl::event_text();
21         }
22         std::cout << std::endl;
23         return Fl_Window::handle(event);
24     }
25 };
26
27 int main()
28 {
29     const auto width { 320 };
30     const auto height { 240 };
31     EventsWindow window { (Fl::w() - width) / 2,
32                           (Fl::h() - height) / 2,
33                           width,height };
34     window.end();
35     window.resizable(window);
36     window.show();
37     return Fl::run();
```

In the `main()` function, the `int` constants `width` and `height` are declared and initialized with 320 and 240. Then an exemplar of `EventsWindow` is created and initialized by calling the constructor with four arguments. The first two arguments contain the `x|y` coordinate on the active screen at which the top left corner of the new window is drawn. The `x|y` coordinate is calculated using the same principle as for centering `symbol` in the `gui/GuiMove.cpp` program. The static member function `Fl::w()` returns the width of the screen in pixels, `Fl::h()` returns the height of the screen in pixels. The width and height are then passed as the next two arguments. Calling this constructor causes the window to appear almost centered on the screen after calling `window.show()`.

As no further child widgets are inserted in `window`, `window.end()` is called immediately after the window exemplar has been created. The statement `window.resizable(window);` is new. The recipient object is `window`, which is derived from `Fl_Group`. Calling this member function sets the resizable area to the (resizable) widget that is passed as an argument. Here, `window` itself is passed as an argument. This means that `window` can and will change its size depending on its own size. Now the size of the window can be changed by dragging the edges or corners with the mouse. The member function `resizable()` must be called before the member function `show()` is called. As can be seen here, it is also possible to call `show()` without passing an argument.

The last statement calls the *FLTK* event loop and returns its result.

It is not necessary to define a new subclass of `Fl_Window` to access the events received by this window. The program in Listing 88 shows how to implement and install a so-called *event handler* to intercept the events that are sent to an `Fl_Window` exemplar.

Listing 88: `gui/EventsSimplified.cpp`

```

1 // EventsSimplified.cpp by Ulrich Eisenecker, April 12, 2024
2
3 #include <FL/Fl.H> // Must be included by each FLTK-program
4 #include <FL/Fl_Window.H> // Because of Fl_Window
5 #include <FL/names.h> // Because of fl_eventnames
6 #include <iostream>
7
8 // Events can also be handled in a free event handler function.
9 int printAndForwardEvents(int event, Fl_Window* window)
10 {
11     static unsigned long long c { 0 };
12     std::cout << c++ << '\t' << fl_eventnames[event];
13     if (event == FL_KEYBOARD)
14     {
15         std::cout << '\t' << Fl::event_key() << '\t' << Fl::event_text();
16     }
17     std::cout << std::endl;
18     return Fl::handle_(event, window);
19 }
20
```

```
21 int main()
22 {
23     // Setting the free event handler function.
24     Fl::event_dispatch(printAndForwardEvents);
25     const auto width { 320 };
26     const auto height { 240 };
27     Fl_Window window { (Fl::w() - width) / 2,
28                        (Fl::h() - height) / 2,
29                        width,height };
30     window.end();
31     window.resizable(window);
32     window.show();
33     return Fl::run();
34 }
```

The function `printAndForwardEvents()` receives a parameter of type `int` as the first argument and a pointer to `Fl_Window` as the second argument. It returns an `int` that represents an event that has not yet been processed by the event handling function. The implementation is largely identical to that of `EventsWindow::handle()` in Listing 87. The difference lies in the return statement. Here, the static member function `Fl::handle_()` is called with two parameters, namely `event` and `window`. Obviously, `Fl::handle_()` then ensures that `window->handle(event)` is called.

The first statement in the `main()` function sets `printAndForwardEvents()` as event handler by calling `Fl::event_dispatch()`. Here, `printAndForwardEvents` is passed as a parameter. Since the function call operator is not used here, `printAndForwardEvents` is a function pointer that points to the `printAndForwardEvents()` function (see Section *The Virtual Function Table*).

Now it is sufficient to create `window` as an exemplar of `Fl_Window`. The previously installed event handler will receive all events before `Fl_Window::handle()` and output the desired information. It then passes each event to `Fl_Window::handle()` for processing.

After creating the executable program by calling `fltk-config --compile Events.cpp`, the program can be executed in a terminal window. Under *macOS* and *Linux*, a centered window is opened. Since it is focused after opening, it receives every event from the keyboard or mouse. The information about each event is displayed in the terminal window. A long mouse movement generates a series of `FL_MOVE` events. Now the advantage of printing an ascending number for each event becomes clear: This helps to distinguish events of the same category.

Pressing and releasing a character key results in a `FL_KEYDOWN` event, possibly followed by further events and finally a `FL_KEYUP` event. Pressing a modifier key, for example the *Shift* key, also generates an `FL_KEYDOWN` event. Holding down the *Shift* key and pressing a character key again generates a `FL_KEYDOWN` event. Now the corresponding capital letter is output by calling `Fl::event_text()`.

Of course, one can also press so-called *function keys*. They can initially be intercepted by the operating system and possibly forwarded to the currently running *FLTK* program. In some cases, this can disrupt the active session. Therefore, everything should be saved beforehand. However, the `gui/Events.cpp` program can be used to interactively explore the events that a *FLTK* window or—more generally—a widget can receive.

2.2.6.3. MS Windows Specific Behavior and Workaround

If the program `gui/Events.cpp` is executed in a terminal window under *MS Windows*, no output is displayed, which is specific to *MS Windows*. However, the output is available and can be redirected to a text file, e.g. by entering `Events > events.csv` in the command line. In this way, the generated events can at least be viewed after the program has ended. However, it would be more interactive to observe the events live. The program in Listing 89 makes this possible. It uses the function `std::getline()` to read an entire input line from `cin` into the `std::string` variable `s` and outputs `s` to `cout`, provided that `cin` is still in a good state, i.e. it has not been closed.

Listing 89: `gui/TerminalOutput.cpp`

```

1 // TerminalOutput.cpp by Ulrich Eisenecker, February 16, 2024
2
3 #include <iostream>
4 #include <string>
5
6 using namespace std;
7
8 int main()
9 {
10     string s;
11     while (cin)
12     {
13         getline(cin,s);
14         if (cin)
15         {
16             cout << s << endl;
17         }
18     }
19 }
```

After compiling this program with `g++ TerminalOutput.cpp -o to.exe`, the program that generates events can be restarted in an *MS Windows* terminal by entering `events | to`. The events should now be reported in the terminal window. The `|` character is the pipe symbol. It sends the output of the command to the left hand to the input of the command to the right hand. The difference to `Events > events.txt` is that the right operand of `|` is a program, in contrast to the right operand of `>`, which is a file.

2.2.6.4. Further Specifics

On *Ubuntu* or *macOS*, the `gui/GuiMove.cpp` program flashes reliably when the asterisk touches the edge of a window. Under *MS Windows*, the author could only rarely observe the flashing under the same circumstances. The reason for this could not be determined exactly. It may be computer-specific and not primarily related to the operating system.

Another peculiarity when executing `gui/GuiMove.cpp` under *MS Windows* is that it is not possible to move the asterisk so close to the bottom edge of the window that it touches the edge. It stops before reaching the bottom edge of the window and cannot be moved beyond the clearly visible gap. The reason for this has already been mentioned: the height of the asterisk as a character that does not reach the baseline of the font is not reported correctly under *MS Windows*.

In an earlier version of `gui/GuiMove.cpp`, the derived window class was called `MoveWindow`. Everything was fine until an attempt was made to compile it under *MS Windows*. There was a conflict with a function identifier in the underlying *application programming interface (API)* of *MS Windows*, which prevented a successful compilation.

Writing portable programs cannot be done exclusively on a single platform. Instead, compilation and execution should be tested on all target platforms from the outset.

2.2.6.5. More About Drawing Text

It has already been mentioned that there are several overloads of `fl_draw()`. Most of them are discussed using the program shown in Listing 90. Before going into the details of the program, some prerequisites and measures must be explained.

Firstly, a file with the name *psychedelic.png* must also be present in the directory containing the executable program. This must be a valid graphic file in *Portable Networks Graphic (PNG)* format. Such a file can also be found in the *gui* folder of the accompanying material. It is possible to replace it with another valid PNG file of the same name if desired.

Secondly, attempting to compile the program with `fltk-config --compile MoreAboutDrawing.cpp` will fail on all platforms. If a program uses functions related to images, the additional option `--use-images` should be specified before the `--compile` option, i.e. `fltk-config --use-images --compile MoreAboutDrawing.cpp`.

For the third, on *macOS*, if most development tools have been installed with *brew*, it is possible that `fltk-config --use-images --compile MoreAboutDrawing.cpp` will still fail with a linker error stating that a static library is missing. The name of the missing library is given as *png*, but the actual name is *libpng.a*, which is a naming standard on Unixoid operating systems. Here, the name of a library always begins with *lib* and the file name has the suffix *.a*. For whatever reason, it can happen that the paths of the libraries on which *FLTK* depends, i.e. third-party libraries, not those of *FLTK* itself, are not entered in

the library search path of the linker. If this error occurs, it must first be checked whether the requested library is installed, e.g. in `/usr/local/lib/`. To determine its location, the `locate` command can be used, which may need to be installed separately.

Next, `fltk-config` must be found, e.g. by entering `locate fltk-config` in a terminal window. More than one file may be reported. In this case, all files that have extensions are excluded. Then all paths that contain a version number are excluded. After this step, only one path with one file should remain, e.g. `/usr/local/bin/fltk-config`. This file should be loaded into an editor, e.g. `vim`. Now the text should be searched for `LDFLAGS=""`. Between the two quotation marks at the end of the line, `-L` is inserted directly followed by the path containing the png library. The result could look like this: `LDFLAGS="-L/usr/local/lib"`. The name of the library must not be included. Finally, `fltk-config` is saved. If necessary, overwriting of the read-only file must be forced. Editing the `fltk-config` script is also described above at the beginning of Section *Building an FLTK Application*.

Listing 90: gui/MoreAboutDrawing.cpp

```

1 // MoreAboutDrawing.cpp by Ulrich Eisenecker, March 6, 2024
2
3 #include <FL/Fl.H> // Must be included by each FLTK-program
4 #include <FL/Fl_Window.H> // Because of Fl_Window
5 #include <FL/Enumerations.H> // Because of FL_BLACK, FL_Escape, etc.
6 #include <FL/fl_draw.H> // Because of fl_... functions
7 #include <FL/fl_ask.H> // Because of FL_BEEP_DEFAULT
8 #include <FL/Fl_Image.H> // Because of ERR_FILE_ACCESS, etc.
9 #include <FL/Fl_PNG_Image.H> // Because of Fl_PNG_Image
10 #include <iostream>
11
12 class MyWindow: public Fl_Window
13 {
14     public:
15     MyWindow(int w,int h): Fl_Window { w,h },
16                           m_image { "../psychedelic.png" }
17     {
18         m_image_error = m_image.fail();
19         switch (m_image_error)
20         {
21             case Fl_Image::ERR_FILE_ACCESS:
22                 std::cerr << "ERR_FILE_ACCESS" << std::endl;
23                 break;
24             case Fl_Image::ERR_FORMAT:
25                 std::cerr << "ERR_FORMAT" << std::endl;
26                 break;
27             case Fl_Image::ERR_NO_IMAGE:
28                 std::cerr << "ERR_NO_IMAGE" << std::endl;
29                 break;
30         }
31     }
32     void draw() override
33     {
34         Fl_Window::draw();

```

```
35     fl_color(FL_BLACK);
36
37     // FL_ALIGN_BOTTOM_RIGHT
38     fl_font(FL_COURIER,12);
39     fl_color(FL_BLACK);
40     fl_draw("FL_ALIGN_BOTTOM_RIGHT",
41             0,0, w(), h(),
42             FL_ALIGN_BOTTOM_RIGHT,nullptr,false);
43
44     // FL_ALIGN_TOP_RIGHT with image
45     if (!m_image_error)
46     {
47         fl_font(FL_HELVETICA_BOLD,12);
48
49         fl_draw("FL_ALIGN_TOP_RIGHT",0,0,w(),h(),
50                 FL_ALIGN_TOP_RIGHT,&m_image,false);
51     }
52
53     // FL_ALIGN_BOTTOM_LEFT with symbol
54     fl_font(FL_SCREEN,12);
55     fl_draw("With symbol @refresh",0,0,w(),h(),
56             FL_ALIGN_BOTTOM_LEFT,nullptr,true);
57
58     // Variations of FL_ALIGN ...
59     fl_font(FL_TIMES,40);
60     const char* text { "Text" };
61     int dx,dy,width,height;
62     fl_text_extents(text,dx,dy,width,height);
63     auto x { (w() - width) / 2 };
64     auto y { (h() - height) / 2 };
65     fl_rect(x + dx,y + dy + height,width,height,FL_RED);
66     fl_color(FL_BLACK);
67     fl_draw(text,x,y,width,height,
68             FL_ALIGN_CENTER,nullptr,false);
69     fl_color(FL_MAGENTA);
70     fl_font(FL_TIMES,76);
71     fl_draw(text,x - (width * 1/3),y - (height * 1/3),
72             width + (width * 2/3),height + (height * 2/3),
73             FL_ALIGN_CENTER | FL_ALIGN_CLIP,nullptr,false);
74
75     // Rotated text
76     // Selecting FL_SCREEN in the next line,
77     // disables rotation under MS Windows.
78     fl_font(FL_HELVETICA,18);
79     fl_color(FL_BLACK);
80     fl_draw(315,"Rotated",5,10);
81 }
82 private:
83     Fl_PNG_Image m_image;
84     int m_image_error;
85 };
86
87 int main(int argc,char* argv[])
88 {
89     MyWindow window { 320,240 };
```

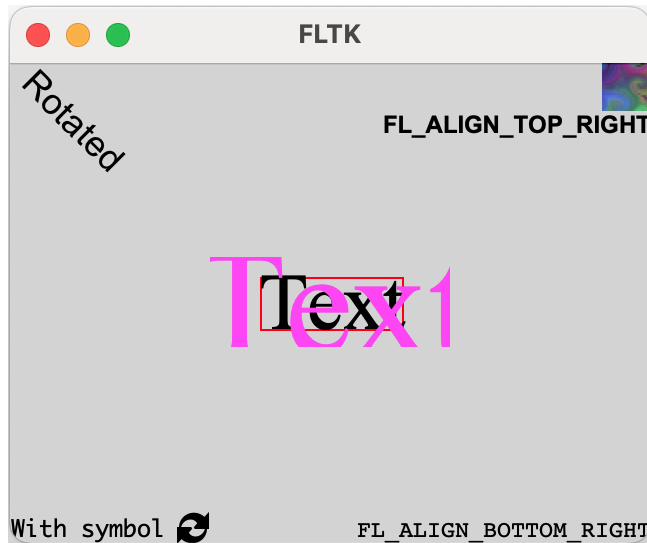
```

90 |     window.end();
91 |     window.resizable(window);
92 |     window.show(argc,argv);
93 |     return Fl::run();
94 | }

```

The `MyWindow` class inherits from `Fl_Window`. It defines two data members, `m_image` of type `Fl_PNG_Image`, which requires to include `<FL/Fl_PNG_Image.H>`, and `m_image_error` of type `int`. `Fl_PNG_Image` defines a constructor that accepts a C-string representing the name of a file containing an image in PNG format. The name for this file is specified in the initialization list of `MyWindow` as `"/psychedelic.png"`. If *FLTK* cannot load this file, no exception is thrown as *FLTK* does not use exceptions. It must therefore be checked immediately whether loading was successful. The result of the call to `m_image.fail()` returns 0 if the loading of the PNG file was successful, and another value otherwise. The value returned by `m_image.fail()` is assigned to `m_image_error`. A switch statement is then used to check whether this value corresponds to one of the static integer constants defined in `Fl_Image`, which represent certain errors. If an error has occurred, the name of the error is sent to `cerr` as text. Therefore `<iostream>` is included at the beginning of the program. As this error does not significantly affect the executability of the program, it is sufficient to output the error message to `cerr`. It is not necessary to throw an exception. As the error status is assigned to `m_image_error`, it can be checked later in the `draw()` member function whether the image was loaded successfully.

Now to `MyWindow::draw()`. Only the things that have not yet been explained are discussed below. The code section under the comment `// FL_ALIGN_BOTTOM_RIGHT` illustrates a `fl_draw()` function that requires eight parameters. The first parameter is a C-string with the text to be drawn. Parameters 2 and 3 specify the pair of coordinates of the fictitious box in which the text is to be drawn. The next two parameters specify the width and height of this fictitious box. In the code, the window origin `0|0` is specified as the origin of the box, and the width and height of the box are specified as the width and height of the window. The sixth parameter is specified as `FL_ALIGN_BOTTOM_RIGHT`, an enumeration constant defined by the `Fl_Align` enumeration type. This means that the text is drawn so that it is aligned with the bottom right-hand corner of the window. Parameter number seven is a pointer to an image exemplar. In *FLTK*, 0 is used to indicate a null pointer, which means that there is no image that could be drawn. However, it is also possible to use the `nullptr` keyword instead to emphasize that it is a pointer with the value zero. The last parameter is of type `int`. If it is specified as 0, the C-string containing the text is not processed during drawing. A value other than 0 means that *FLTK* processes the text before it is drawn. Instead of 0 and a value other than 0, `true` and `false` can also be used, which is perhaps a little more intentional. Here, `false` is passed as a parameter, which means that the text is to be drawn without processing. Figure 38 shows the result, where the text in the bottom right-hand corner is drawn in the `FL_COURIER` font.

Figure 38: Various ways of drawing text (gui/MoreAboutDrawing.cpp)

The next code section illustrates how an image can be drawn to which a pointer passed as the seventh argument points. The corresponding code is only executed if `m_image` has been loaded without errors, i.e. if the if condition `!m_image_error` evaluates to true. The sixth argument is `FL_ALIGN_TOP_RIGHT`, which aligns the image to the top right-hand corner of the window. The seventh parameter is the address of `m_image`, i.e. `&m_image`. The eighth parameter is again false, as the text passed as the first parameter should not be processed. The result is that the PNG image is placed exactly in the top right-hand corner, while the text is placed directly below the image. The result can be seen in the top right-hand corner of Figure 38.

The code under the comment `// FL_ALIGN_BOTTOM_LEFT with symbol` illustrates what it means to process a character string when it is drawn. A special feature of *FLTK* are so-called *label symbols*. A label symbol begins with `@`, followed by formatting characters. For example, `@arrow` draws an arrow pointing to the right and `@refresh` draws two intertwined arrows. To output the *at*-sign, it is doubled, i.e. `@@` draws a single *at* character. Detailed information is provided in the *FLTK* documentation (Costantini, Melcher, et al. 2025) in Section *Common Widgets and Attributes* under the headline *Labels and Label Types*. The string to be drawn "With symbol `@refresh`" contains such a symbol and the eighth parameter of `fl_draw()` is specified as true, which means that the string is processed when drawing. The result can be seen in the bottom left corner of Figure 38.

The next section of code is a little more complicated. After setting the font face and size, the dimensions of a fictitious box that encloses the C-string are calculated by calling `fl_text_extents()`. Then the variables `x` and `y` are declared and initialized to the relative window center. Now the enclosing rectangle is printed in red using an overload of the

`fl_rect()` function with a fifth parameter that specifies the character color. The next instruction draws the text in exactly the enclosing rectangle with `FL_ALIGN_CENTER` as the sixth parameter.

The question arises as to whether calling `fl_draw(text, x, y)` would have been a simpler but completely equivalent alternative? This is not the case, because `fl_draw()` with 8 parameters draws the text into the specified box, i.e. `x` and `y` specify the top left corner of this rectangle, while `fl_draw()` with 3 parameters draws the text above the rectangle, since `x` and `y` specify the bottom left corner for drawing the text. The next step is to significantly increase the font size and draw the text in a fictitious box that has been enlarged by $2/3$. As the frame should still be centered, $1/3$ of width and height is subtracted from `x` and `y`, and $2/3$ of width and height is added to width and height.

This time, `FL_ALIGN_CENTER | FL_ALIGN_CLIP` is specified as a parameter for the alignment. The symbol `|` is the bitwise OR operator. As `FL_ALIGN_CENTER` and `FL_ALIGN_CLIP` are integer constants, they can be combined with the bitwise OR operator. Their values are selected in such a way that it is possible to use them as so-called bit masks (*Mask (Computing)* 2024). The result of combining both integer constants with the bitwise OR therefore produces a value that represents both the centered alignment and the clipping. That is, `FL_ALIGN_CLIP` means that only the text inside the fictitious box will be drawn, while any text that would be drawn outside the box is clipped. Clipping is an important function in GUI programming, which will be discussed again later. The result can be seen in the middle of Figure 38.

The last code section shows how text can be drawn rotated. This is done by calling an overload of `fl_draw()` with four arguments. The first argument specifies the angle in radians counterclockwise. The center of the rotation is passed as parameters three and four and also specifies the lower left corner for drawing the text. Here 315 is specified as the value for the angle. As the first parameter is of type `int`, specifying -45 would give the same result.

This section of code reveals another difference between the platforms. In an earlier version, `FL_SCREEN` was selected as the font for drawing the rotated text. This worked without problems on *macOS* and *Ubuntu*. Under *MS Windows*, the text was not rotated. Instead, it was drawn as it would be expected if `draw` were called with only three parameters. Since the *FLTK* documentation points out possible problems with platforms using the *X Window System*, the standard error stream was also checked for possible problems—but there were none. After careful analysis, the problem could be traced back to the font being set to `FL_SCREEN`. If a different font is selected, the character string is drawn rotated under *MS Windows* without any problems.

2.2.6.6. Coding Style

The code examples presented in this book are not in line with (*Configuration Management Plan - Fast Light Toolkit (FLTK)* n.d.). Instead, they make extensive use of modern C++ concepts and features and show how easily the somewhat special *FLTK* style can be combined with this newer style.

When studying other code that uses *FLTK*, e.g. from internet sources, another difference becomes apparent: the other code examples mostly use dynamically generated widgets, whereas in this book widgets are generated statically, i.e. on the function call stack, whenever possible. This is for didactic reasons, as the resulting code can be perceived as less confusing by the reader. It also avoids potential confusion caused by uninitialized pointers or orphaned objects. However, *FLTK* provides excellent management of dynamic child widgets, which are automatically destroyed when their parent widget is destroyed. Therefore, the programmer has to worry less about releasing dynamic objects, at least when it comes to dynamic *FLTK* child widgets.

2.2.7. Child Widgets, Focus and Dialogs

An application window, i.e. an exemplar of `F1_Window` or a subclass of it, can manage several widgets, so-called *children*. To allow these children to receive and process events, they must be given the so-called *focus*.

There are also predefined widgets for interaction with the user, the so-called *dialogs*. *FLTK* offers several dialog classes and free functions to execute them and get the desired results. In the following program examples, some of these classes and functions are discussed and their use is explained.

2.2.7.1. Children

In the previous examples, everything was drawn directly in the drawing area of the main window. Here, the main window is the only object that can receive and process events. In addition to the main window, which is of type `F1_Window` or a subclass of it, there are other classes of widgets that can be drawn and can receive and process events. The `F1_Widget` class is the base class for all these classes. It is an abstract class because its `draw()` method is a pure virtual function. A special subclass of it is `F1_Group`. An exemplar of `F1_Group` manages any number of exemplars of `F1_Widget` or its descendants, which includes instances of `F1_Group` itself or derived classes. The class `F1_Window` is a direct subclass of `F1_Group`. It can therefore also manage any number of exemplars of `F1_Widget` or derived classes.

The program in Listing 91 shows how to define a subclass of `F1_Widget` and insert several exemplars of it into an `F1_Window` object.

Listing 91: gui/Children.cpp

```

1 // Children.cpp by Ulrich Eisenecker, March 11, 2024
2
3 #include <FL/Fl.H>
4 #include <FL/Fl_Window.H>
5 #include <FL/Fl_Widget.H>
6 #include <FL/fl_draw.H>
7 #include <FL/Enumerations.H>
8
9 class Leaf: public Fl_Widget
10 {
11     public:
12         // the constructor of the base class, as the constructor
13         // of the base class is declared as protected.
14         Leaf(int x, int y, int w, int h, const char* label = nullptr,
15             Fl_Color c = FL_BLACK):Fl_Widget { x,y,w,h,label },m_color { c }
16         {
17         }
18         void draw() override
19         {
20             draw_box(FL_FLAT_BOX,m_color);
21             draw_label();
22         }
23     private:
24         Fl_Color m_color;
25 };
26
27 int main()
28 {
29     // window //////////////////////////////////////
30     Fl_Window window { 320,240,"Parent" };
31
32     // Some computations //////////////////////////////////
33     auto wHalf { window.w() / 2 };
34     auto hHalf { window.h() / 2 };
35
36     Leaf green { 0,0,wHalf,hHalf,"Green",FL_GREEN };
37     Leaf magenta { wHalf - 1,0,wHalf,hHalf,"Magenta",FL_MAGENTA };
38     Leaf cyan { 0,hHalf - 1,wHalf,hHalf,"Cyan",FL_CYAN };
39     Leaf yellow { wHalf - 1,hHalf - 1,wHalf,hHalf,"Yellow",FL_YELLOW };
40     // window end //////////////////////////////////
41     window.end();
42     window.resizable(green);
43     window.show();
44     return Fl::run();
45 }
46

```

The header file `<FL/Fl_Widget.H>` is included because `Fl_Widget` is the base class for `Leaf`. The name of this class was chosen to make it clear that its exemplars can only act as leaves in the tree that structures the widgets according to their parent-child relationship.

Although it is an abstract class, the constructor of `Fl_Widget` is also declared as protected. Therefore, it cannot be made accessible via a using declaration, i.e. using `Fl_Widget::Fl_Widget`. Instead, a corresponding public constructor is defined for `Leaf`, which calls the protected constructor of `Fl_Widget` and passes all specified arguments for position, size and label. An additional data member defined in `Leaf`, `m_color` of type `Fl_Color`, is initialized with `c`, where this last parameter of the `Leaf` constructor is of type `Fl_Color` and has `FL_BLACK` as its default value.

Next, the `draw()` member function is declared. It draws a box in the style defined by `FL_FLAT_BOX` with `m_color` as the color. The label is drawn horizontally and vertically centered, which is the standard for `Fl_Widget`.

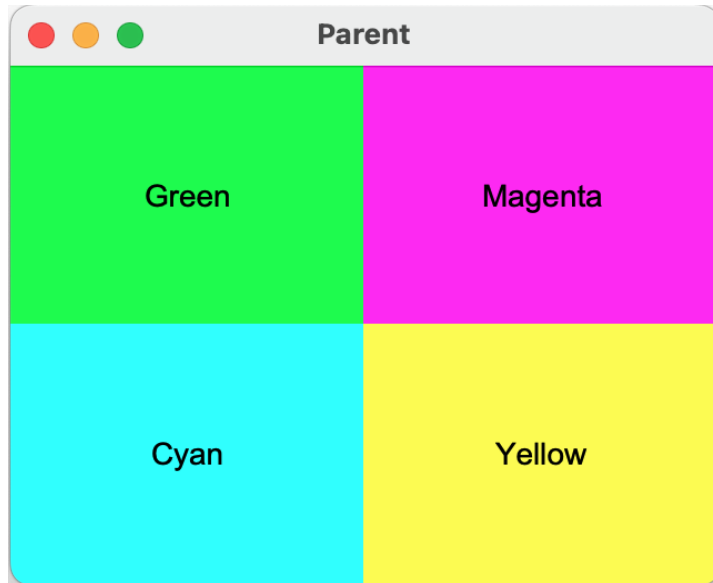
In the private section, `m_color` is declared as a data member of type `Fl_Color`.

The `main()` function creates an exemplar of `Fl_Window` that is 320 pixels wide and 240 pixels high. It then initializes the variables `wHalf` to half the width of the window and `hHalf` to half the height of the window. The values of `wHalf` and `hHalf` are then used to initialize four exemplars of `Leaf` named `green`, `magenta`, `cyan` and `yellow`, so that each `Leaf` exemplar covers exactly one quarter of the parent window and has the color specified by its variable name.

All these exemplars automatically become children of `window`, as it is in insert mode after being created as an `Fl_Window` exemplar. Each widget that is created thereafter is automatically added as a member of the group represented by the last `Fl_Group` exemplar created. The statement `window.end();` ends this special state of `window` so that no further children are automatically added.

Then `window` is made resizable with `green` as the primary object for resizing. This means that the application window can be resized, but its resizing depends on the resizing of `green`. All other widgets, `magenta`, `cyan` and `yellow`, as well as `window` itself, depend on `green` for their resizing. Consequently, `green` retains its proportions during resizing, but the other widgets may not retain their original ratio of width to height. The last statements display `window` and start the event loop.

Figure 39 shows the application window directly after the compiled program has been executed. Apart from moving, resizing and closing, nothing else can be done with this window. Some behavior will be added in the next example.

Figure 39: Screenshot of the execution of `gui/Children.cpp`

2.2.7.2. Focus and Dialogs

The program shown in Listing 91 extends the `gui/Children.cpp` program by enabling the children of the main window to receive focus and process some events. Before going into the details of the source program, the possible interactions during the execution of the compiled program should be discussed.

Immediately after starting the program, the main window itself, whose label has the value *Parent*, is the resizable window. So changing the size of the application window changes all subordinate windows proportionally to the main window.

Although the window looks largely the same as in Figure 39, there is a slight difference in its appearance. The green rectangle is slightly highlighted as it is drawn with the `FL_DOWN_BOX` box type. Once the application with the title *Parent* has been selected, it is possible to switch between the colored rectangles in a clockwise direction using the *Tab* key. Pressing the *Shift* key and the *Tab* key at the same time scrolls through the rectangles in an anti-clockwise direction. It is also possible to click on a rectangle with the left mouse button to select it. As soon as a new rectangle is selected, the previously selected rectangle changes its appearance as it is now drawn with the box type set to `FL_UP_BOX`.

First, the main window itself is selected as the resizable widget. After selecting a rectangle with a left mouse click, the *r* key can be pressed. A dialog then appears asking whether the corresponding widget should become a resizable widget, e.g. *Make cyan the resizable widget?* This dialog has two buttons for answering. The left button is labeled *No* and shows an additional icon indicating that it is the default button. Consequently,

this button is selected by pressing the *Enter* key. The right button is labeled *Yes*. When clicking on the *No* button, nothing changes, when clicking on the *Yes* button, the corresponding widget becomes the resizable widget. If a rectangle is already selected as the resizable widget, pressing the *r* button when it is selected will have no effect.

In most cases, the button that triggers the most harmless action should be the default button. In the example above, *No* only closes the dialog and does not change the state of the program, which is therefore an action without consequences. In *FLTK* dialogs, the first button is normally the default button. In detail, these design aspects are regulated by the *GUI design guidelines* for the relevant GUI framework or the specific platform.

It should be emphasized that it is possible to use the *Tab* key to switch between the two buttons of the dialog. When a button is selected, it is activated by pressing the *Enter* key. This is a design principle that makes it possible to operate a GUI application using only the keyboard. When implementing own widgets, care should be taken to ensure that this convention is adhered to. This enables the user to work with the program even if they cannot use a mouse or if a mouse is not available for whatever reason.

Another option is to press the *l* key. This opens a dialog that asks the user for a new name for the label, e.g. *New label:*. The input line contains the current label of the rectangle, which can be edited. There are also two buttons here, which represent the selection options *OK* and *Cancel*. Now *OK* is marked with a symbol to indicate that this button is activated when the *Enter* key is pressed. Of course, this default button does not correspond to the principle that pressing it should result in the most inconsequential action.

The terms *left-click* and *right-click* for mouse actions are actually to be understood symbolically. It is possible to swap the mouse buttons, e.g. to adapt to chirality. There may also be individual mouse buttons for which special actions are required to generate a right-click, e.g. holding down the control button and performing a left-click.

The last option is to press the *c* key. This opens a dialog for selecting a new color for the rectangle. The currently selected color is highlighted in the color field and a new color can be selected in various ways, which are not described in detail here as this is a standard *FLTK* dialog. Here too, the dialog can be closed by clicking *OK* after a new color has been selected, or by clicking *Cancel*, which closes the dialog without any further consequences.

A change to the label or the color of a rectangle is displayed as soon as it is confirmed.

Listing 92: gui/Focus.cpp

```

1 // Focus.cpp by Ulrich Eisenecker, July 9, 2025
2
3 #include <FL/Fl.H>
4 #include <FL/Fl_Window.H>
5 #include <FL/fl_ask.H>
6 #include <FL/Fl_Widget.H>
7 #include <FL/fl_draw.H>
8 #include <FL/Enumerations.H>
9 #include <FL/Fl_Color_Chooser.H> // Because of fl_color_chooser()
10 #include <string>
11
12 using namespace std;
13
14 class Leaf: public Fl_Widget
15 {
16     public:
17     Leaf(int x, int y, int w, int h, const char* label = nullptr,
18         Fl_Color c = FL_BLACK):Fl_Widget { x,y,w,h,label },
19         m_color { c }, m_boxtype { FL_UP_BOX }
20     {}
21     void changeColor()
22     {
23         uchar r,g,b;
24         Fl::get_color(m_color,r,g,b);
25         auto result { fl_color_chooser("Select color",r,g,b) };
26         if (result == 1)
27         {
28             m_color = fl_rgb_color(r,g,b);
29             redraw();
30         }
31     }
32     void changeLabel()
33     {
34         auto newLabel { fl_input("New label: ",label()) };
35         if (newLabel)
36         {
37             copy_label(newLabel);
38             redraw();
39         }
40     }
41     void changeResizable()
42     {
43         if (parent()->resizable() == this)
44         {
45             return;
46         }
47         using namespace std;
48         string question { "Make "s + label() + " the resizable widget?" };
49         // The following switch statement could easily be
50         // converted into an if statement.
51         switch (fl_choice("%s", "Yes", "No", 0, question.c_str()))

```

```
52     {
53         case 0: parent()->resizable(this);
54                 parent()->init_sizes();
55                 break;
56         case 1: break; // could be deleted
57     }
58 }
59 void draw() override
60 {
61     color(m_color);
62     box(m_boxtype);
63     draw_box();
64     draw_label();
65 }
66 int handle(int event) override
67 {
68     switch (event)
69     {
70         case FL_PUSH:
71             take_focus();
72             return 1;
73         case FL_FOCUS:
74             m_boxtype = FL_DOWN_BOX;
75             redraw();
76             return 1;
77         case FL_KEYBOARD:
78             {
79                 auto key { Fl::event_key() };
80                 switch (key)
81                 {
82                     case 'c':
83                         changeColor();
84                         return 1;
85                     case 'l':
86                         changeLabel();
87                         return 1;
88                     case 'r':
89                         changeResizable();
90                         return 1;
91                 };
92             }
93             break;
94         case FL_UNFOCUS:
95             m_boxtype = FL_UP_BOX;
96             redraw();
97             return 1;
98     }
99     return Fl_Widget::handle(event);
100 }
101 private:
102     Fl_Color m_color;
103     Fl_Boxtype m_boxtype;
104 };
105
106 int main()
```

```

107 {
108     // window ///////////////////////////////////
109     Fl_Window window { 320,240,"Parent" };
110
111     // Some computations ///////////////////////////////////
112     auto wHalf { window.w() / 2 };
113     auto hHalf { window.h() / 2 };
114
115     Leaf green { 0,0,wHalf,hHalf,"Green",FL_GREEN };
116     Leaf magenta { wHalf - 1,0,wHalf,hHalf,"Magenta",FL_MAGENTA };
117     Leaf cyan { 0,hHalf - 1,wHalf,hHalf,"Cyan",FL_CYAN };
118     Leaf yellow { wHalf - 1,hHalf - 1,wHalf,hHalf,"Yellow",FL_YELLOW };
119     // window end ///////////////////////////////////
120     window.end();
121     window.resizable(green);
122     window.show();
123     return Fl::run();
124 }

```

In Listing 92 includes the header file `<FL/Fl_Color_Chooser.H>` for the first time. In addition to the `Fl_Color_Chooser` class, it also contains the `fl_color_chooser()` function, which prompts the user to select a color.

When using the script command `fltk-config -compile Focus.cpp` to create the program, the compiler may complain about the use of the suffix `s` to identify a C++-string ("Make "s). That is caused by the script passing the option `-std=gnu++11` to the compiler. For correct compilation the script statement calling the compiler can be copied, pasted and edited in the command line. After replacing `-std=gnu++11` with `-std=c++20` the program should be compiled successfully. A detailed explanation of how to modify the `fltk-config` script can be found above, at the beginning of Section *Building an FLTK Application*.

The only change to the constructor of the `Leaf` class is the initialization of the newly introduced data member `m_boxtype`, which is of type `Fl_Boxtype`. It manages the way in which a `Leaf` exemplar is drawn that currently has the focus and how it is drawn if this is not the case. As all `Leaf` exemplars will be drawn in unpressed state, the data member `m_boxtype` is initialized with `FL_UP_BOX`.

The member functions `changeColor()`, `changeLabel()` and `changeResizable()` are new. They are discussed below.

Compared to `gui/Children.cpp`, the `draw()` member function shows an alternative method for drawing a box. Here, the single statement `draw_box(FL_FLAT_BOX,m_color);` is replaced by three statements that firstly set the color, secondly set the box type and thirdly draw the box. This is only done here to show the possibility of this variant. In terms of code parsimony, the variant in `gui/Children.cpp` is the preferable one.

The member function `Leaf::changeColor()` first defines three unsigned character variables called `r`, `g` and `b`. The static member function `Fl::get_color(m_color, r, g, b)` is then called to set the variables `r`, `g` and `b` to the red, green and blue values of the color specified by `m_color`. This corresponds to the *Red-Green-Blue color model* (*RGB* for short), which is one way of defining colors. Next, the `fl_color_chooser()` function is called. Its first argument is a C-string, which serves as a prompt for the user. The next three arguments are the variables `r`, `g` and `b`. Since they are set to the RGB values of the currently active color, the corresponding color is highlighted in the color field displayed by the dialog that opens when this function is called.

The dialog allows to select a new color from the color field. It is also possible to enter new values manually as byte or hex values or to switch to a different color model, namely the *Hue-Saturation-Brightness color model* (*HSB* for short). If the user confirms a color selection by pressing the *OK* button, the function sets the values of `r`, `g` and `b`, which are passed as a reference to the selected color, and returns 1. If the *Cancel* button is pressed, the function returns 0.

As long as the color selection dialog is open, no other dialog can be opened. It is also not possible to interact with the application window, except to select, move, maximize or iconize it. It cannot be resized or closed either. This special behavior is called *application modal*. **A modal application dialog must be closed before the application window can resume its normal behavior.**

The following if statement checks whether the result returned by the function is 1. If so, the `fl_rgb_color()` function is called. It returns a 24-bit color value that comes closest to the red, green and blue components specified by `r`, `g` and `b`.

Finally, `redraw()` is called to indicate that *FLTK* should redraw the window if necessary. It should be emphasized that this function could be further simplified. The result returned by `fl_color_chooser()` could be used directly in the condition of the if statement, eliminating the need to use the result variable. The version chosen is perhaps easier to understand.

The member function `Leaf::changeLabel()` demonstrates the use of another function that opens a dialog, namely `fl_input()`. To use it, `<FL/fl_ask.H>` must be included. `fl_input()` is one of a series of overloaded functions. It outputs the C-string that is passed as the first argument, displays the second argument, also a C-string, as the default value and prompts the user to enter a string that is returned as a C-string. Here, the result of the call to the member function `label()` is passed as the second parameter. This member function returns the currently used label as a C-string. The user can change the displayed default string or enter a completely new string. After pressing the *OK* button, the entered value is returned as a C-string. If the *Cancel* button is pressed, a pointer with the value 0 is returned. Here, the result of the call to `fl_input()` is used to initialize the variable `newLabel`, which is specified as `auto`.

The following if statement checks whether `newLabel` is different from 0. If so, it calls the member function `copy_label()` with `newLabel` as the value for the new label. The member function ensures that the currently active label is properly deleted and the passed C-string is copied as the new label value. Like `fl_color_chooser()`, this dialog is also application-modal. One more detail needs to be added. The first argument is a so-called *format string*. Format strings are quite common in C for the output of strings with the C functions `printf()` and `sprintf()`, which support a variable number of arguments. The following shows how these format strings can be used and what effects their use has. After copying the new label, the `redraw()` function is called to redraw the window if necessary. Unlike `changeColor()`, this member function cannot be simplified because the return value of `fl_input()` must be stored in a local variable in order to use it as the value for setting a new label.

Both member functions, `changeColor()` and `changeLabel()`, call `redraw()` when a change has been made. One could argue that this combines two purposes in one function. However, `redraw()` is only called when a change has been made. If the call to `redraw()` were omitted in this case, both functions would have to return a `bool` value, for example, to indicate the need to call `redraw()` on the caller's side. Furthermore, this would make the use of these member functions on the caller's side more complex. Therefore, the decision was made to call `redraw()` in both functions.

The member function `Leaf::changeResizable()` makes it possible to select the currently selected `Leaf` exemplar as the resizable widget if this is not already the case. Strictly speaking, this is the check made in the first statement. The parent element of the `Leaf` exemplar is the application window. The pointer to it is returned by the member function `parent()`, which is defined in `Fl_Group`. If the returned pointer is equal to `this`, the box is already the resizable widget and the function returns immediately. Next, all identifiers are imported from the namespace `std`. This enables the use of the type `std::string` and in particular string literals without namespace prefix and other restrictions. Next, the string variable `question` is initialized with a string that is composed of a `std::string` literal, a C-string that is returned by calling `label()` and another C-string literal by repeatedly applying the `std::string` concatenation operator `+`. This relatively complex procedure for creating a single character string as an output message can be considerably simplified by using other features of format strings, as will be shown later. The string variable `question` is then passed as the last argument in the call to `fl_choice()`.

Now more about the `fl_choice()` function, which can be called with a different number of arguments. As already mentioned, the first argument is a format string. In a format string, the `%` character introduces a so-called *conversion specification*. If an `s` follows, the occurrence of `%s` is replaced by a C-string, `%d` is replaced by a signed integer and `%f` by a floating point number. Further formatting specifications are available. An additional parameter must be passed to the function call for each conversion specification. To output a single `%` character, `%%` must be used. A detailed description of the formatting of char-

acter strings in C is (*std::printf*, *std::fprintf*, *std::sprintf*, *std::snprintf* - *Cppreference.Com* n.d.). All this indicates that a format string with additional conversion specifications can be used to replace the complex creation of the C++-string question.

If a formatting string does not contain a conversion specification, such as in `Leaf::changeLabel()`, no additional parameter needs to be specified. In `changeResizable()`, the format string contains exactly one conversion specification, namely `%s`. Therefore, `question.c_str()` must be passed as the last parameter. If "Make this to the resizable widget?" were used as the format string, no additional parameter would have to be specified. The `fl_choice()` function offers a maximum of three choices. This means that after the first parameter, which is the format string, three further parameters must be specified as C-strings. Here, "Yes" and "No" are passed as parameters two and three. There is no third option. Therefore, 0 is passed as a parameter. Sensible options for parameter selection would be one C-string and two null pointers, two C-strings and one null pointer or three C-strings and no null pointer. *FLTK* uses 0 as the value for specifying the null pointer. Instead, the keyword `nullptr` can also be used to make it clear that the null pointer and not an integer is passed here. It should be noted that after these first four parameters, there must be as many additional parameters as there are conversion specifiers in the format string.

The `fl_choice()` function returns an integer value that indicates which button was pressed, i.e. which C-string was selected. It is 0 for the first, 1 for the second and 2 for the third button. As there is no option to cancel the dialog, this option does not need to be displayed. The *FLTK* documentation (Costantini, Gibson, et al. n.d.) uses a switch statement to distinguish between the different return values. To indicate this possibility, the code shown here also uses a switch statement. In practice, it would be more efficient to replace it with an if statement. In addition, the case branch for 1 is superfluous, as nothing needs to be done in this case.

The use of the `fl_choice()` function here is completely equivalent to the `fl_ask()` function. However, according to (*Human Interface Guidelines - Fast Light Toolkit (FLTK)* n.d.), the `fl_ask()` is obsolete and should no longer be used.

The statements contained in the case branch for 0 are now explained. The first statement is obvious. The member function `F1_Widget::parent()` asks for the parent of the window, which is returned as a pointer of type `F1_Group`. The access operator of the pointer, i.e. `->`, is used to tell the parent with the member function `resizable()` that the `Leaf` exemplar pointed to is now the resizable object. Exemplars of the `F1_Group` class and its descendant classes manage an internal array of widget sizes and positions. After the children of a group have been rearranged, `F1_Group::init_sizes()` must be called to update this internal array. Although it is not mentioned in the documentation, calling `F1_Group::init_sizes()` is obviously also necessary when a new resizable child is set in the group.

The fact that it is the resizable widget is not reflected in the appearance of a Leaf exemplar. Therefore, `redraw()` does not need to be called when a Leaf exemplar becomes the resizable widget, as nothing needs to be drawn. This could be considered as a flawed design from the perspective of human-computer interaction, as it is the responsibility of the software to assist the user in perceiving its state.

Compared to `gui/Children.cpp`, the member function `Leaf::handle()` is also new. The switch statement processes those values of the event variable that are of interest for the Leaf widgets. The first case branch checks whether event equals `FL_PUSH`. `FL_PUSH` is a mouse event that is generated when a mouse click is made on the visible area of a widget. If this is the case, the widget must signal that it wants to have the focus. This is done by calling the inherited member function `take_focus()`. As the `FL_PUSH` event has been consumed, the `handle()` member function is exited with `return 1;`. Here and in the following sections, superfluous break statements have been removed to improve the readability of the code. Nevertheless, it should be remembered that case branches should normally be terminated by `break`, unless they are intended to fall through. The next branch is executed if event equals `FL_FOCUS`. In this case, the constant `FL_DOWN_BOX` is assigned to `m_boxtype` and the widget is marked for redrawing. The member function `handle()` then returns the value 1 to indicate that the event has been consumed.

The next case branch is executed if event equals `FL_KEYBOARD`. In this case, the auto variable `key` is initialized with the result of the call to `Fl::event_key()` (see Section *My-Window::handle()*). Then a nested switch statement is used to distinguish between the keys `c`, `l` and `r`. This can be seen as a more deliberate alternative to using chained if statements as in Listing `gui/GuiMove.cpp`. Each case of the nested switch statement ends with `return 1;` to indicate that the `FL_KEYBOARD` event for the specific key has been consumed.

The last case branch is executed when event equals `FL_UNFOCUS`. This event signals that the widget has lost focus. In this case, `FL_UP_BOX` is assigned to `m_boxtype`. Next, `redraw()` is called and the `handle()` function is exited with the return value 1. The last statement of `Leaf::handle()` returns the result of passing the event to the member function `Fl_Widget::handle()`.

The `main()` function has not changed compared to `gui/Children.cpp`. It is therefore not explained further.

2.2.7.3. Nesting Widgets

The widgets in the `gui/Focus.cpp` program are all direct children of the application window, which is the standard situation in *FLTK*. In addition, the child windows do not overlap.

However, one can also imagine widgets that have children, which in turn are parents. These widgets must be drawn in the order in which they are nested, i.e. from the root widget to the leaf widgets. Otherwise, subordinate widgets could be entirely covered by their parents. In addition, a parent must ensure that a nested widget can receive the focus in order to receive events.

Selecting a child widget, especially a grandchild widget, as the widget on which the resizing depends is a challenge, as this is not foreseen or intended in *FLTK*. Dealing with this problem promotes further understanding of events and their processing in *FLTK*.

The program in Listing 93 offers a first, but incomplete approach to implement nested widgets in this way. As it uses newer features of C++, the *fltk-config* script should be modified as described at the beginning of Section *Building an FLTK Application*. Otherwise errors will be reported. If this is not possible, line 134 can be changed to `static Fl_Widget* m_focused;` and the following line has to be inserted after line 136 `Fl_Widget* Node::m_focused = nullptr;`

Listing 93: `gui/NestedChildren.cpp`

```
1 // NestedChildren.cpp by Ulrich Eisenecker, March 14, 2024
2
3 #include <FL/Fl.H>
4 #include <FL/Fl_Window.H>
5 #include <FL/fl_ask.H>
6 #include <FL/Fl_Widget.H>
7 #include <FL/Fl_Group.H>
8 #include <FL/fl_draw.H>
9 #include <FL/Enumerations.H>
10 #include <FL/Fl_Color_Chooser.H>
11
12 class Node: public Fl_Group
13 {
14     public:
15         Node(int x, int y, int w, int h, const char* label = nullptr,
16             Fl_Color c = FL_BLACK):Fl_Group { x,y,w,h,label },
17             m_color { c }
18         {}
19         void changeColor()
20         {
21             uchar r,g,b;
22             Fl::get_color(m_color,r,g,b);
23             if (fl_color_chooser("Select color",r,g,b) == 1)
24             {
25                 m_color = fl_rgb_color(r,g,b);
26                 redraw();
27             }
28         }
29     private:
30         Fl_Color m_color;
```

```

27     }
28 }
29 void changeLabel()
30 {
31     auto newLabel { fl_input("New label: ",label()) };
32     if (newLabel)
33     {
34         copy_label(newLabel);
35         redraw();
36     }
37 }
38 void changeResizableUpwards(Fl_Widget* node)
39 {
40     parent()->resizable(node);
41     parent()->init_sizes();
42     Node* parentNode { dynamic_cast<Node*>(parent()) };
43     if (parentNode)
44     {
45         parentNode->changeResizableUpwards(this);
46     }
47 }
48 void resetResizableDownwards()
49 {
50     for (int i { 0 };i < children(); ++i)
51     {
52         Node* node { dynamic_cast<Node*>(child(i)) };
53         if (node)
54         {
55             node->resizable(nullptr);
56             node->resetResizableDownwards();
57         }
58     }
59     resizable(this);
60     init_sizes();
61 }
62 void changeResizable()
63 {
64     if (fl_choice("Make %s the resizable widget?",
65                  "Yes","No",0,label()) == 0)
66     {
67         resetResizableDownwards();
68         changeResizableUpwards(this);
69     }
70 }
71 void draw() override
72 {
73     // The following statement uses the ternary operator.
74     Fl_Boxtype boxtype { m_focused == this ? FL_DOWN_BOX : FL_UP_BOX };
75     draw_box(boxtype,m_color);
76     draw_label();
77     draw_children();
78 }
79 int handle(int event) override
80 {
81     switch (event)

```

```
82     {
83     case FL_PUSH:
84         // The statements of this case branch contain serious errors.
85         if (children())
86         {
87             for (int i { 0 }; i < children(); ++i)
88             {
89                 if (Fl::event_inside(child(i)))
90                 {
91                     if (child(i)->take_focus())
92                     {
93                         m_focused = child(i);
94                         return 1;
95                     }
96                 }
97             }
98         }
99         if (take_focus())
100         {
101             m_focused = this;
102             return 1;
103         }
104         break;
105     case FL_FOCUS:
106         redraw();
107         return 1;
108     case FL_KEYBOARD:
109         {
110             auto key { Fl::event_key() };
111             switch (key)
112             {
113                 case 'c':
114                     changeColor();
115                     return 1;
116                 case 'l':
117                     changeLabel();
118                     return 1;
119                 case 'r':
120                     changeResizable();
121                     return 1;
122             };
123         };
124         break;
125     case FL_UNFOCUS:
126         redraw();
127         return 1;
128     }
129     Fl::focus(m_focused);
130     return Fl_Widget::handle(event);
131 }
132 private:
133     Fl_Color m_color;
134     inline static Fl_Widget* m_focused { nullptr };
135 };
136
```

```

137 int main()
138 {
139     // window ///////////////////////////////////
140     Fl_Window window { 320,240,"Parent" };
141
142     // Some computations ///////////////////////////////////
143     auto wHalf { window.w() / 2 };
144     auto hHalf { window.h() / 2 };
145
146     // green in window ///////////////////////////////////
147     Node green { 0,0,wHalf,hHalf,"Green",FL_GREEN };
148     // red in green ///////////////////////////////////
149     Node red { green.w() / 6,green.h() / 6,
150              green.w() / 3,green.h() / 3,"Red",FL_RED };
151     red.align(FL_ALIGN_CENTER | FL_ALIGN_INSIDE);
152     // red end ///////////////////////////////////
153     red.end();
154     // white in green ///////////////////////////////////
155     Node white { green.w() * 2 / 3,green.h() * 2 / 3,
156               green.w() / 4,green.h() / 4,"White",FL_WHITE };
157     white.align(FL_ALIGN_CENTER | FL_ALIGN_INSIDE);
158     // white end ///////////////////////////////////
159     white.end();
160     // green end ///////////////////////////////////
161     green.end();
162     // magenta in window ///////////////////////////////////
163     Node magenta { wHalf,0,wHalf,hHalf,"Magenta",FL_MAGENTA };
164     // magenta end ///////////////////////////////////
165     magenta.end();
166     // cyan in window ///////////////////////////////////
167     Node cyan { 0,hHalf,wHalf,hHalf,"Cyan",FL_CYAN };
168     // cyan end ///////////////////////////////////
169     cyan.end();
170     // yellow in window ///////////////////////////////////
171     Node yellow { wHalf,hHalf,wHalf,hHalf,"Yellow",FL_YELLOW };
172     // yellow end ///////////////////////////////////
173     yellow.end();
174     for (int n { 0 }; n < window.children(); ++n)
175     {
176         window.child(n)->align(FL_ALIGN_TOP | FL_ALIGN_INSIDE);
177     }
178     // window end ///////////////////////////////////
179     window.end();
180     window.resizable(window);
181     window.show();
182     return Fl::run();
183 }

```

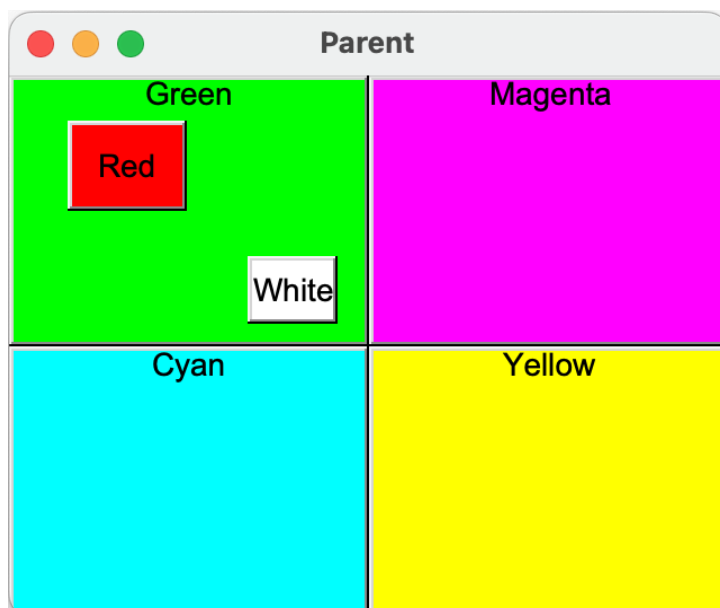
The `gui/Focus.cpp` and `gui/NestedChildren.cpp` programs have a similar structure. Instead of the `Leaf` class in `gui/Focus.cpp`, the `gui/NestedChildren.cpp` program defines the `Node` class. In contrast to `Leaf`, it is derived from `Fl_Group`, which allows an exemplar of `Node` to manage child widgets. In this way, the nesting of widgets of type `Node` or derived classes is supported.

The following discussion does not follow the different parts of the program, i.e. classes and (member) functions. Instead, it focuses on problems that arise and presents measures to solve them.

The nesting of Node exemplars is used in the `main()` function of `gui/NestedChildren.cpp`. Immediately after the green exemplar of type Node is created, the red exemplar, which is also of type Node, is created. All widgets that are created after a widget of type `F1_Group` or a subclass thereof are automatically inserted into the widget created immediately before. This automatic insertion as a child stops as soon as the `end()` member function of the parent widget is called. After inserting the node exemplar `red`, `red.end()` is executed to prevent the following widgets from being inserted as children in `red`. After `red.end()` is called, green becomes active again as the target for inserting newly created widgets. The underlying data structure that enables this behavior can be imagined as a stack. Thus, the next Node exemplar that is created, `white`, is also inserted as a widget in green as its parent. Since no children are to be inserted in `white`, `white.end()` is called. Then `green.end()` is executed to end the insertion of further children.

The magenta, cyan and yellow Node exemplars are then created and automatically inserted into window as children, as described in the Section *Children*. Figure 40 shows the resulting application window immediately after starting the program.

Figure 40: Screenshot of the execution of `gui/NestedChildren.cpp`



Since the exemplars with color names are of type Node, which is a subclass of `F1_Group`, their labels would be aligned differently when drawing. More precisely, they would be aligned outside their widget, but not inside. Therefore, a for loop runs through all children and sets the desired alignment for each child.

`Fl_Group` is not a container for which `begin()` and `end()` functions are defined. Therefore, a classic for loop must be used. It runs from 0 to the number of children minus one. The member function `Fl_Group::children()` returns the number of children managed by a parent. In the loop body, `Fl_Group::child()` is used to access each child widget. This member function returns a pointer to the child referenced by the index passed as an argument. Therefore, the class member access operator `->` is used to call the member function `align()` for this child to set the desired alignment as `FL_ALIGN_TOP` | `FL_ALIGN_INSIDE`. The consequences of not adjusting the alignment can be easily checked by converting the statement in the loop body into a comment.

When compiling and running the program, it is noticeable that the window size has been quadrupled. This was done to improve the appearance of the window and its children. Due to the relative calculation of positions and sizes, this only required a small change when creating the window exemplar.

During the development of `gui/NestedChildren.cpp` it turned out that it was impossible to select the red or white widget because green always received the `FL_FOCUS` event beforehand. In fact, this problem is the default behavior of *FLTK*. To work around it, the `Node::handle()` member function in the case branch for `FL_PUSH` first checks if the recipient widget has children. If so, a loop begins that checks whether the mouse position of the click falls within the range of a child. If this is the case, this child is asked to take over the focus. However, this solution suffers from two serious shortcomings, which will be explained later.

As a nested widget loses focus when the application window is redrawn, the focused widget must be managed separately. This is done by the static member variable `m_focused`, which is a pointer of type `Fl_Widget`. If a child element is ready to take focus, i.e. returns a non-zero value when its member function `take_focus()` is called, it is assigned to `Node::m_focused`. This static variable is initialized inline with `nullptr`. As a result, no child has the focus after the first drawing of the application window. This is different from the `gui/Focus.cpp` program. Here, green is initially focused and is therefore drawn with `FL_DOWN_BOX` as the box type. By default, *FLTK* automatically gives focus to the widget that is inserted first in a group. If no subordinate widget takes the focus, the parent widget asks itself whether it wants to take the focus. If `take_focus()` returns a non-zero value, this is assigned to `m_focused`.

Actually, this behavior is a deficiency of `gui/NestedChildren.cpp`, because it prevents to navigate through the child widgets only with the keyboard, i.e. with the *Tab* key and the *Shift + Tab* key. ***It is an important principle of GUI applications that full interaction with them must only be possible with keyboard. Interactions that depend exclusively on a pointing device such as a mouse are discouraged.***

Another issue that occurred was that losing focus did not always generate a `FL_UNFOCUS` event for a previously focused nested widget. One consequence of this was that the box type was no longer set correctly, resulting in the nested widget that had lost focus being drawn inadequately. Therefore, the management of the box type in the member `m_`

boxtype was abandoned. Instead, the static member `m_focused` is used for this purpose. In `Node::draw()`, the ternary operator `? :` is used to initialize the local variable `boxtype` accordingly. The first operand of the ternary operator is a condition. If it evaluates to true, the second operand (between the question mark and the colon) is returned as the result of the evaluation of the ternary operator; otherwise the third operand (after the colon) is returned as the result. Here the condition checks whether `m_focused` is equal to `this`. If true, `FL_DOWN_BOX` is returned for the initialization of `boxtype`, otherwise `FL_UP_BOX`.

Since `Node::draw()` overrides `Fl_Group::draw()` to implement the desired drawing behavior, it must call the member function `draw_children()`, which is a protected member function of `Fl_Group`. Drawing the children at the end of the member function ensures that they are drawn after the parent widget and are therefore visible on top of their parent.

This ordering of the widgets is referred to as Z-order. It is a fundamental principle in GUI and graphics programming. The term refers to the arrangement along the Z-axis beyond the X- and Y-axis. A specific value of the Z-order is referred to as the Z-index. A simple representation of the Z-order is a stack. The widget with the lowest Z-index is the first in the stack of widgets, while the widget with the highest Z-index is at the top of the stack. This idea does not take sibling widgets into account. For this reason, a tree is a more appropriate conceptualization. The window in Listing 93 has the lowest Z-index, i.e. 0, of all widgets in the application. This means that it is in the background and must be drawn first. Its children green, magenta, cyan and yellow are siblings that do not overlap. Therefore, they all have the same Z-index of 1. The green widget has two children that do not overlap. Therefore, both have a Z-index of 3. In `gui/NestedChildren.cpp`, the Z-order is managed by the nesting of the `Node` exemplars.

In a sense, the children of a `Fl_Group` exemplar can also be seen as a representation of a Z-order. Since they are drawn in the order in which they are added to their parent, a Z-order is realized in this way when drawing. The order in which children receive events is not explicitly stated in the documentation. However, from various results of a web search, it can be concluded that children receive events in the reverse order of being added to the group, which also sounds plausible. Once a child handles the event and returns 1, no other child of the same group receives this event. `Fl_Group` also provides member functions to manipulate its children and their order.

In the `gui/NestedChildren.cpp` program, the Z-order is crucial for resizing, which is performed in three member functions, namely `changeResizable()`, `resetResizableDownwards()` and `changeResizableUpwards()`. At first glance, the names chosen for these functions may seem counterintuitive, as `resetResizableDownwards()` means going from a given Z-index to higher Z-indexes, i.e. traversing to the leaves of the tree, and that `changeResizableUpwards()` refers to going from a given Z-index to 0, i.e. traversing the tree to the root.

In the freshly started program, no widget has the focus—which does not comply with the GUI design rules mentioned above, as this would prevent operation using only the keyboard. In addition, the window exemplar is resized in relation to itself. Therefore, when resizing, all widgets in the affected dimensions are changed. A mouse click on a widget selects it visually, it looks depressed and receives the focus. Pressing the *r* key will bring up a dialog asking if this widget should be made the resizable widget. For example, if it is the white widget, it is not appropriate to make it directly the widget whose size affects that of the window exemplar. This is because white is not a child of window. Consequently, white must become the resizable widget of green, and green must become the widget on which the resizing of the window exemplar depends.

The member function `changeResizableUpwards()`, which receives a pointer to `Fl_Widget` with the name `node` as a parameter, is responsible for this. In its implementation, it first calls `parent()->resizable(node);`. This retrieves the pointer to the parent—there must be a parent, it cannot be null, which is why there is no additional check—and calls `resizable()` for it, passing `node` as a parameter. Then `parent()->init_sizes()` must be called to update the internal array of widgets. Next, a `dynamic_cast<>()` is applied to `parent()`. If this cast to a pointer to `Node` is successful, i.e. results in a pointer that is not null, `changeResizableUpwards(this)` is called for this pointer. If `parent()` does not return a pointer to an object of type `Node` or a subclass thereof, `dynamic_cast<>()` fails and returns null. As a result, the upward traversal stops as soon as `parent()` points to a sibling class of `Node`, e.g. `Fl_Window`. But thanks to the first statement in `changeResizableUpwards()`, `node` becomes the object that determines the resizing of this instance, e.g. in the case of `gui/NestedChildren.cpp` the window exemplar. Since `node` must always be a child or grandchild of window, window can never be made dependent on itself in terms of resizing. It is impossible to return to the initial state of the program. Overall, this member function establishes a transitive resizing relationship from each node upwards to the root of a widget tree.

In another scenario, a leaf is the resizable widget, and now a parent or grandparent of that leaf becomes the resizable widget. Without terminating the relationship between this leaf and its parent, a conflict would arise. Therefore, all resizable relationships from the given node exemplar to its children must be reset. The member function `resetResizableDownwards()` is responsible for this. It runs through all children of a node and applies a `dynamic_cast<>()` to a pointer to `Node` to each pointer to a child. If `dynamic_cast<>()` is successful, i.e. `node` is a pointer other than null, `resizable()` is called for `node` with `nullptr` as argument. Then `resetResizableDownwards()` is called for `node`.

In the `gui/Focus.cpp` program, pressing *r* is ignored if the recipient object is already the widget on which the resizing depends. The corresponding check is no longer possible, as it only checks whether the resizable widget of the parent is the same. This test only works for one group, but not for hierarchically nested groups. For this reason, this check is removed completely. When the user clicks the *Yes* button, the selected widget becomes the resizable widget by first removing all resizable relationships of its children and then

creating resizable relationships from itself across all Z-levels up to the root widget. Finally, the widget sets itself as the resizable widget and updates its internal array of child widgets.

The member functions `changeColor()` and `changeLabel()` have only been slightly revised. Compared to their counterparts in `gui/Focus.cpp`, they have been simplified while retaining their original functionality.

The reader is strongly advised to run the compiled program and examine it closely. The next step is to make some changes to the source program and then recompile and run the program to explore the consequences. Before reading the next section, the program it is recommended to thoroughly analyze `gui/NestedChildren.cpp` to detect as much unexpected and unwanted behavior as possible.

2.2.7.4. Giving Focus With the Keyboard

After starting the compiled `gui/NestedChildren.cpp` program, it is not possible to operate it using only the keyboard. Initially, none of the widgets is focused. After focusing a widget by a mouse click, it reacts to the *c*, *l* and *r* keys, but not to *Tab*, *Shift + Tab* or a cursor key.

Two changes are necessary as a remedy. The first is that `Node::handle()` must also process the keys *Tab*, *Shift + Tab*, and *Cursor*. This alone would not be sufficient, as no widget is focused immediately after the program is started. Therefore, before starting the event loop, a child of window must be focused manually, i.e. by calling its `take_focus()` member function.

The extension of the member function `Node::handle()` is relatively simple. In the switch statement for key, additional case branches for `FL_Tab` and the *Cursor* keys `FL_Right`, `FL_Left` and possibly `FL_Up` and `FL_Down` need to be included. `FL_Up` and `FL_Down` could be used to navigate from a child to its parent and from a parent to its first child. Interestingly, the case branch for `FL_Tab` also covers *Tab + Shift*. For this purpose, an if statement is used to check whether `F1::event_state()` returns `FL_SHIFT`.

Listing 94 shows a modified excerpt from `Node::handle()` of the `gui/NestedChildrenImproved.cpp` program with the changes mentioned above. Pressing the *Tab* key alone or in combination with the *Shift* key results in `FL_Tab` in both cases. When the *Shift* key is pressed at the same time, `F1::event_state()` returns `FL_SHIFT`. The case branch for `FL_Tab` therefore differs from the case branches for the other keys.

Listing 94: gui/NestedChildrenImproved.cpp

```

209         case FL_KEYBOARD:
210             {
211                 auto key { Fl::event_key() };
212                 switch (key)
213                 {
214                     case 'c':
215                         changeColor();
216                         return 1;
217                     case 'l':
218                         changeLabel();
219                         return 1;
220                     case 'r':
221                         changeResizable();
222                         return 1;
223                     case FL_Tab:
224                         if (Fl::event_state() == FL_SHIFT)
225                         {
226                             focusPreviousSibling();
227                         }
228                         else
229                         {
230                             focusNextSibling();
231                         }
232                         return 1;
233                     case FL_Right:
234                         focusNextSibling();
235                         return 1;
236                     case FL_Left:
237                         focusPreviousSibling();
238                         return 1;
239                     case FL_Up:
240                         focusParent();
241                         return 1;
242                     case FL_Down:
243                         focusChild();
244                         return 1;
245                 };
246             };
247         break;
248         case FL_UNFOCUS:

```

This program also uses new C++ features. Therefore, it must be compiled with a modified *fltk-config* script (see the beginning of Section *Building an FLTK Application* for more details).

Listing 95 shows the code fragment that selects a child of the `Fl_Window` exemplar called `window`. It must be inserted into the `main()` function before the event loop is started. First, it is checked whether `window` has any children. Then a for loop iterates through all children. As soon as the first child accepts the focus, the loop is terminated with `break`.

Listing 95: Selecting a child of window (excerpt from gui/NestedChildrenImproved.cpp)

```
265 | int main()
266 | {
310 |     if (window.children())
311 |     {
312 |         for (int i { 0 }; i < window.children(); ++i)
313 |         {
314 |             auto node { asNode(window.child(i)) };
315 |             if (node && node->focus())
316 |             {
317 |                 break;
318 |             }
319 |         }
320 |     }
321 |     return Fl::run();
322 | }
```

2.2.7.5. Giving Focus With the Mouse

When running the gui/NestedChildren.cpp program, each widget can be clicked with the mouse to give it the focus. The corresponding code is completely in the case branch for `FL_PUSH` in the `Node::handle()` member function (Listing 93). Unfortunately, this code no longer works as expected after a small modification in the `main()` function, which is shown in Listing 96. The Node exemplar `white` is now inserted as a child of `red`.

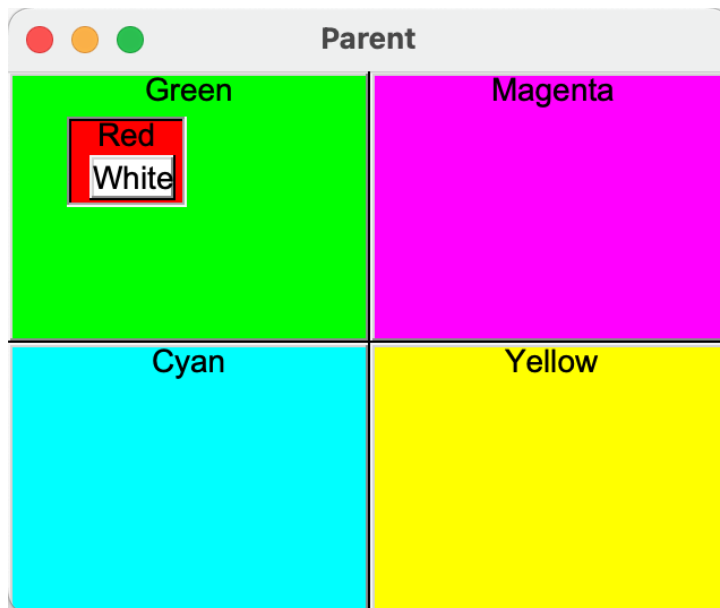
Listing 96: Change in `main()` (excerpt from gui/NestedChildren.cpp)

```
137 | int main()
138 | {
146 |     // green in window //////////////////////////////////
147 |     Node green { 0,0,wHalf,hHalf,"Green",FL_GREEN };
148 |     // red in green //////////////////////////////////
149 |     Node red { green.w() / 6,green.h() / 6,
150 |               green.w() / 3,green.h() / 3,"Red",FL_RED };
157 |     white.align(FL_ALIGN_CENTER | FL_ALIGN_INSIDE);
158 |     // white end //////////////////////////////////
159 |     white.end();
160 |     // green end //////////////////////////////////
161 |     green.end();
183 | }
```

It is then no longer possible to select the `white` widget with the mouse (Figure 41). Any attempt to select the `white` widget will result in the `red` widget being selected.

The reason for this is the hierarchical nesting of Node exemplars beyond the parent-child level. The `white` widget is not a child of `green`. Hence, it is never evaluated whether `Fl::event_inside(&white)` is true, because `child(i)` never points to `white`. To write a

Figure 41: Screenshot of the execution of the modified `gui/NestedChildren.cpp` program



function that recursively descends from a node to its leaves to check with `Fl::event_inside()` whether it should take the focus and then terminate the recursion seems obvious. Unfortunately, this does not work. The documentation for the first overload of `Fl::event_inside()` explicitly states that “*there must not be an intermediate subwindow ... because this method does not transform coordinates of child widgets, and thus the given widget *o* must be within the same window that is handling the current event*” (Costantini, Gibson, et al. n.d.).

Therefore, a special member function is required that descends to the leaves of a `Node` exemplar and asks each leaf if the event coordinates are in its range, updating the event coordinates with respect to its parent when descending. This member function is called `Node::focusWidget()`. In addition, another function is required that returns `true` if the event coordinates are within the range of the widget, and `false` otherwise. Since `inside()` is already the name of a member function of `Fl_Widget`, this free function is named `within()` to avoid the prefix `::` when it is called from another member function of `Node`.

The correspondingly revised program is presented and discussed in detail below.

2.2.7.6. Giving Focus to Node Exemplars

Listing 97 shows the program parts that have been changed or added compared to Listing 93. The changes and additions are discussed in the order in which they appear in the program.

If *fltk-config* reports errors during compilation, the same measures should be taken as for program `gui/NestedChildren.cpp`.

Listing 97: `gui/NestedChildrenImproved.cpp`

```
1 // NestedChildrenImproved.cpp by Ulrich Eisenecker, July 09, 2025
11 #include <stdexcept>
12
13 bool within(int x,int y,int w,int h)
14 {
15     if (x >= 0 && x < w &&
16         y >= 0 && y < h)
17     {
18         return true;
19     };
20     return false;
21 }
22
23 class Node; // Pending declaration
24
25 // Declaration of function casting to Node*
26 Node* asNode(Fl_Widget* widget);
27
28 class Node: public Fl_Group
29 {
30     public:
31     void changeResizableUpwards(Fl_Widget* node)
32     {
33         parent()->resizable(node);
34         parent()->init_sizes();
35         auto parentNode { asNode(parent()) };
36         if (parentNode)
37         {
38             parentNode->changeResizableUpwards(this);
39         }
40     }
41     void resetResizableDownwards()
42     {
43         for (int i { 0 };i < children(); ++i )
44         {
45             auto node { asNode(child(i)) };
46             if (node)
47             {
48                 node->resizable(nullptr);
49                 node->resetResizableDownwards();
50             }
51         }
52         resizable(this);
53     }
54 }
```

```

76     init_sizes();
77 }
94 void focusNextSibling()
95 {
96     if (parent())
97     {
98         if (parent()->children() == 0)
99         {
100             throw std::out_of_range
101                 { "focusNextSibling() - no. of children is zero" };
102         }
103         auto index { parent()->find(this) };
104         if (index == parent()->children())
105         {
106             throw std::out_of_range
107                 { "focusNextSibling() - child not in group" };
108         }
109         if (index == parent()->children() - 1)
110         {
111             index = 0;
112         }
113         else
114         {
115             ++index;
116         }
117         if (parent()->child(index)->take_focus())
118         {
119             m_focused = parent()->child(index);
120             redraw();
121         }
122     }
123 }
124 void focusPreviousSibling()
125 {
126     if (parent())
127     {
128         if (parent()->children() == 0)
129         {
130             throw std::out_of_range
131                 { "focusPreviousSibling() - no. of children is zero" };
132         }
133         auto index { parent()->find(this) };
134         if (index == parent()->children())
135         {
136             throw std::out_of_range
137                 { "focusPreviousSibling() - child not in group" };
138         }
139         if (index == 0)
140         {
141             index = parent()->children() - 1;
142         }
143         else
144         {
145             --index;
146         }

```



```
147         if (parent()->child(index)->take_focus())
148         {
149             m_focused = parent()->child(index);
150             redraw();
151         }
152     }
153 }
154 void focusParent()
155 {
156     // Only Node's should take focus!
157     auto parentNode { asNode(parent()) };
158     if (parentNode && parentNode->take_focus())
159     {
160         m_focused = parentNode;
161         redraw();
162     }
163 }
164 void focusChild()
165 {
166     if (children() > 0 && child(0)->take_focus())
167     {
168         m_focused = child(0);
169         redraw();
170     }
171 }
172 bool focusWidget(int ev_x = Fl::event_x(),int ev_y = Fl::event_y())
173 {
174     for (int i { 0 };i < children();++i)
175     {
176         auto childNode { asNode(child(i)) };
177         if (childNode && childNode->focusWidget(ev_x,ev_y))
178         {
179             return true;
180         }
181     }
182     if (within(ev_x - x(),ev_y - y(),w(),h()))
183     {
184         m_focused = this;
185         redraw();
186         return true;
187     }
188     return false;
189 }
190 Fl_Widget* focus()
191 {
192     if (take_focus())
193     {
194         m_focused = this;
195         return m_focused;
196     }
197     return nullptr;
198 }
199 int handle(int event) override
200 {
201     switch (event)
```

```

202     {
203         case FL_PUSH:
204             focusWidget();
205             return 1;
206         case FL_FOCUS:
207             redraw();
208             return 1;
209         case FL_KEYBOARD:
210             {
211                 auto key { Fl::event_key() };
212                 switch (key)
213                 {
214                     case 'c':
215                         changeColor();
216                         return 1;
217                     case 'l':
218                         changeLabel();
219                         return 1;
220                     case 'r':
221                         changeResizable();
222                         return 1;
223                     case FL_Tab:
224                         if (Fl::event_state() == FL_SHIFT)
225                         {
226                             focusPreviousSibling();
227                         }
228                         else
229                         {
230                             focusNextSibling();
231                         }
232                         return 1;
233                     case FL_Right:
234                         focusNextSibling();
235                         return 1;
236                     case FL_Left:
237                         focusPreviousSibling();
238                         return 1;
239                     case FL_Up:
240                         focusParent();
241                         return 1;
242                     case FL_Down:
243                         focusChild();
244                         return 1;
245                 };
246             };
247             break;
248         case FL_UNFOCUS:
249             redraw();
250             return 1;
251     }
252     Fl::focus(m_focused);
253     return Fl_Widget::handle(event);
254 }
255 };

```

```
260 Node* asNode(Fl_Widget* widget)
261 {
262     return dynamic_cast<Node*>(widget);
263 }
264
265 int main()
266 {
310     if (window.children())
311     {
312         for (int i { 0 }; i < window.children(); ++i)
313         {
314             auto node { asNode(window.child(i)) };
315             if (node && node->focus())
316             {
317                 break;
318             }
319         }
320     }
321     return Fl::run();
322 }
```

The header file `<stdexcept>` is included because the newly introduced methods can throw `std::out_of_range` exceptions in some cases. These cases should not occur in practice. However, it is more prudent to check for these errors and throw an exception if such an error is detected.

The free function `within()` requires four arguments. The first two arguments specify the *x* and *y* coordinates relative to a window origin of 0|0. Therefore, it is sufficient to pass the width and height of a given window and simply check that none of the coordinates are outside the limits of their ranges, i.e. negative or equal to the width or height in pixels. It returns `true` if both coordinates are within their ranges and `false` if not.

Next, a pending declaration of `Node` is required. In `gui/NestedChildren.cpp`, `dynamic_cast<>()` with `Node*` as the result type is performed twice. Exactly this cast occurs even more often in Listing 97. It therefore seems reasonable to provide a function that performs exactly this cast. For its declaration, `Node` must be introduced as the name of a type.

The free function `asNode()` takes a pointer to `Fl_Widget` and returns a pointer to `Node`. The pointer is null if the passed pointer cannot be successfully cast to a `Node` pointer. The function `asNode()` is inspired by the member functions `as_group()` and `as_window()`. They are virtual member functions of `Fl_Widget` that are overridden in `Fl_Group` and `Fl_Window` to return the desired result, namely a null pointer if the corresponding pointer does not point to an `Fl_Group` object or an `Fl_Window` object. This is a more complicated way to implement specific type casts using inclusion polymorphism, which is necessary because *FLTK* does not use the modern casts.

The definition, i.e. its implementation, is located after the definition of `Node`, as a `dynamic_cast<>()` cannot be applied to an incomplete result type.

The only changes in `changeResizableUpwards()` and `resetResizableDownwards()` are the replacements of `dynamic_cast<>()` by a call to the `asNode()` function.

The member function `Node::focusNextSibling()` is new. It is called to give focus to the nearest sibling object of the receiver object. First, it checks whether the receiver object has a parent at all. If not, the member function does nothing and returns implicitly at the end. If there is a valid parent, the function checks whether the number of children is 0. If this is the case, an exception is thrown. Under normal circumstances, this error cannot occur. However, if it does occur, it is detected and reported. The next step is to locate the receiver object in the parent's group. This is done by calling `parent()->find(this)`. The result is stored in the auto variable `index`. This member function returns the number of children in the group if the child being searched is not contained in this group. This is very similar to the return of the `end()` iterator if an element cannot be found in an *STL* container. Therefore, the checked condition is `index == parent()->children()` and not `index >= parent()->children()`. An exception is also thrown in this case. However, this case should not occur in practice. The next step is a simple check. If the receiver object is the last child in the group, `index` is set to 0, otherwise `index` is incremented by 1. Finally, if the child that is accessed with `parent()->child(index)` takes the focus, the corresponding child widget is assigned to the static variable `Node::m_focused` and the current window is marked to be redrawn if necessary.

The member function `focusPreviousSibling()` is very similar to the function `focusNextSibling()`. The only difference is that it checks whether `index` is 0. If this is the case, `index` is assigned the index of the last child of the group, otherwise `index` is decremented by one. Since this is the only difference between these two member functions, one could consider to combine the two functions into one with an additional parameter that controls whether the next sibling or the previous one should be focused. This should only be done if the resulting code is still easy to read and understand.

Both member functions are called in `Node::handle()` in response to the key events `FL_Tab`, `FL_Tab` plus `FL_SHIFT`, `FL_Right` or `FL_Left`.

The member function `focusParent()` is called in response to the `FL_Up` key event. Only if the parent of the widget is of type `Node` or a subclass of it, it is asked whether it takes the focus and `m_focused` is assigned accordingly. At a first glance, one could argue that a `FL_Window` exemplar would also be a valid parent. However, the problem is that once focus has been given to the `FL_Window` parent, it would be impossible to give it to one of its children, as this requires calling member functions that are only defined in `Node`.

The member function `focusChild()` is called in response to the `FL_Down` key event. Its purpose is to select the first child if the receiver object has any children at all. The condition of the if statement uses short-circuit evaluation. It first checks whether the number of children is greater than 0. This could have been written less redundantly simply as `children()`, as 0 would automatically be converted to `false`. The elementary condition

to the right hand side of the `&&` operator is evaluated as `true` if the first child takes the focus. Then `child(0)` is assigned to `m_focused` and `redraw()` is called to mark the widget to be redrawn at the next opportunity.

The member function `focusWidget()` is complicated. It takes two parameters of type `int`, which represent the coordinates of a mouse click. If the caller of the function does not specify these parameters, they are initialized with `Fl::event_x()` and `Fl::event_y()` as default values. These functions return the `x` and `y` coordinates of the mouse click. The method first iterates over all its children. A child pointer is first converted into a `Node` pointer, as otherwise the `focusWidget()` method cannot be called for it. If `childNodes` is not null, the `focusWidget()` member function is called for `childNodes`, whereby `ev_x` and `ev_y` are now explicitly passed as parameters. If the call to `childNodes->focusWidget(ev_x, ev_y)` returns `true`, the for loop is exited with `return true;`. This means that a descendant child took the focus. In this case, nothing further needs to be done. If the for loop is terminated, no child took the focus. In this case, an if statement checks whether the adjusted coordinates of the mouse click are within its range by calling the `within()` function. The adjustment is made by subtracting `x()` from `ev_x` and `y()` from `ev_y`. This makes sense, as a child must lie within or be equal to the dimensions of its parent. As parameters 3 and 4, `w()` and `h()` are passed to `within()`. If `within()` returns `true`, this is assigned to `m_focused`, `redraw()` is called and the function exits with `return true;`. By default, the function returns `false`.

The member function `Node::focus()` checks whether the receiver object can or wants to take the focus. If so, `Node::m_focused` is assigned this and `m_focused` is returned. Otherwise, the member function returns `nullptr`. This member function is called later in `main()` to give focus to the first child widget. A simple call to `take_focus()` would not be sufficient, as the static member variable `Node::m_focused` must be updated. As this variable is private and not visible to the outside world, the member function `Node::focus()` had to be introduced.

Several changes have been made to the member function `Node::handle()`. The case branch for `FL_PUSH` has been simplified and calls `focusWidget()` without specifying any arguments. Therefore, the current mouse coordinates are used as default values. Since the `FL_PUSH` event has been consumed, the case branch actually ends with `return 1;`. This part of `Node::handle()` is not included in Listing 97.

The next changes are made in the nested switch statement for key (Listing 97). In its case branches, the events generated by pressing the *Tab* key or one of the *Cursor* keys are processed. For each event, the corresponding member function is called, which navigates to the desired element—if it exists—and gives it the focus.

The methods for navigating to a `Node` exemplar and for giving focus to it serve two purposes, namely navigating to the widget—if it exists—and then asking whether it wants to take the focus, and if so, updating `m_focused` and calling `redraw()`. The code parts for the latter are duplicated in each corresponding member function. Therefore, one could consider refactoring by moving the corresponding code to a specialized member

function and calling it when needed. As a consequence, one might also consider not including this call in the member function that navigates to a widget. Instead, it might be necessary to call it separately if the navigation was successful and giving focus is accepted. In the current case, this could be so-called *over engineering*, as in this program, navigating to a widget and asking whether it wants to take the focus are always required in combination. Splitting both purposes into different member functions would therefore make them more difficult to use at the calling sites.

2.2.8. Menus and Callbacks

Most GUI applications offer menus for interaction with the user. The most common type of menu is the so-called *menu bar* with *drop-down menus*. Other types of menus are *pop-up menus* and *toolbars*. Only the classic menu bar is discussed below.

A menu bar consists of *menu items* that are linked in a list. However, they are displayed in a hierarchy with menu items at the top level which, when selected, display menu items at a lower level. The end of a menu level and the start of a subordinate menu level are determined by special values.

When a menu item is selected, this triggers an event that is sent to a widget. This is done via so-called *callback functions*. *FLTK* only allows the use of free functions or static member functions as callback functions, normal member functions are not supported.

Regardless of this limitation and the impression of a narrow focus, it will be shown that there are many technical possibilities for creating classic menus and combining them with callback functions.

2.2.8.1. Declarative Representation of Menus

Menus can be created in *FLTK* in various ways. One of these is a declarative representation of a menu in the form of an array with menu items as elements. In this way, the entire menu is implemented in the form of *C-strings* at a single point in the code.

The program in Listing 98 is based on the *gui/NestedChildrenImproved.cpp* program. It only shows the parts that are new or have been changed compared to Listings 93 and 97.

If *fltk-config* reports errors when compiling this program, the same measures should be taken as for program *gui/NestedChildren.cpp*.

Listing 98: gui/Menu.cpp

```
1 // Menu.cpp by Ulrich Eisenecker, July 9, 2025
2
11 #include <FL/Fl_Menu_Item.H> // Because of Fl_Menu_Item
12 #include <FL/Fl_Menu_Bar.H> // Because of Fl_Menu_Bar
13 #include <stdexcept>
14
15 class InvisibleNode: public Fl_Group
16 {
17     public:
18         InvisibleNode(int x, int y, int w, int h,
19                     const char* label = nullptr
20                     ):Fl_Group { x,y,w,h,label }
21     {}
22     virtual void draw() override
23     {
24         draw_children();
25     }
26     void end();
27 };
28
43 class Node: public InvisibleNode
44 {
45     public:
213     int handle(int event) override
214     {
215         switch (event)
216         {
217             case FL_PUSH:
218                 focusWidget();
219                 return 1;
220             case FL_FOCUS:
221                 redraw();
222                 return 1;
223             case FL_KEYBOARD:
224                 {
225                     auto key { Fl::event_key() };
226                     switch (key)
227                     {
228                         case FL_Tab:
229                             if (Fl::event_state() == FL_SHIFT)
230                             {
231                                 focusPreviousSibling();
232                             }
233                             else
234                             {
235                                 focusNextSibling();
236                             }
237                             return 1;
238                         case FL_Right:
239                             focusNextSibling();
240                             return 1;
```

```

241         case FL_Left:
242             focusPreviousSibling();
243             return 1;
244         case FL_Up:
245             focusParent();
246             return 1;
247         case FL_Down:
248             focusChild();
249             return 1;
250     };
251 };
252 break;
253 case FL_UNFOCUS:
254     redraw();
255     return 1;
256 }
257 Fl::focus(m_focused);
258 return Fl_Widget::handle(event);
259 }
260 };
261
262 void InvisibleNode::end()
263 {
264     Fl_Group::end();
265     if (children())
266     {
267         auto node { asNode(child(0)) };
268         if (node)
269         {
270             node->focus();
271         }
272     }
273 }
274
275 void cbAbout(Fl_Widget*,void*)
276 {
277     fl_message("Menu application, July 9, 2025\n"
278               "by Ulrich Eisenecker"
279               );
280 }
281
282 void cbQuit(Fl_Widget*,void*)
283 {
284     if (fl_choice("Do you really want to quit the program?",
285                  "Ok","Cancel",0) == 0)
286     {
287         exit(0);
288     }
289 }
290
291 void cbColor(Fl_Widget*,void*)
292 {
293     asNode(Node::focused())->changeColor();
294 }
295
296 void cbLabel(Fl_Widget*,void*)

```



```
309 {
310     asNode(Node::focused()->changeLabel());
311 }
312
313 void cbResizable(Fl_Widget*,void*)
314 {
315     asNode(Node::focused()->changeResizable());
316 }
317
318 int main()
319 {
320     // window ///////////////////////////////////
321     Fl_Window window { 640,480,"Parent" };
322     // menuItems ///////////////////////////////////
323     Fl_Menu_Item menuItems[]
324     {
325         { "Demo",          0,0,0,FL_SUBMENU },
326         { "&About",         FL_CTRL+'a',cbAbout,0,FL_MENU_DIVIDER },
327         { "&Quit",          FL_CTRL+'q',cbQuit,0 },
328         { 0 },
329         { "Edit",          0,nullptr,nullptr,FL_SUBMENU },
330         { "C&olor",        FL_CTRL+'o',cbColor,nullptr },
331         { "&Label",         FL_CTRL+'l',cbLabel,nullptr },
332         { "&Resizable",    FL_CTRL+'r',cbResizable,nullptr },
333         { nullptr },
334         { nullptr }
335     };
336     // menuBar in window ///////////////////////////////////
337     Fl_Menu_Bar menuBar { 0,0,window.w(),30,"Menu" };
338     menuBar.copy(menuItems);
339
340     // Some computations ///////////////////////////////////
341     auto wHalf { window.w() / 2 };
342     auto hHalf { (window.h() - menuBar.h()) / 2 };
343
344     // invisible in window ///////////////////////////////////
345     InvisibleNode invisible { 0,menuBar.h(),window.w(),window.h() - menuBar.h() };
346     // green in invisible ///////////////////////////////////
347     Node green { 0,menuBar.h(),wHalf,hHalf,"Green",FL_GREEN };
348     // red in green ///////////////////////////////////
349     Node red { green.w() / 6,green.h() / 6 + menuBar.h(),
350               green.w() / 3,green.h() / 2,"Red",FL_RED };
351     red.align(FL_ALIGN_CENTER | FL_ALIGN_INSIDE);
352     // blue in red ///////////////////////////////////
353     Node blue { red.x() + 5,red.y() + 5,35,20,"Blue",FL_BLUE };
354     blue.align(FL_ALIGN_CENTER | FL_ALIGN_INSIDE);
355     blue.end();
356     // red end ///////////////////////////////////
357     red.end();
358     // white in green ///////////////////////////////////
359     Node white { green.w() * 2 / 3,green.h() * 2 / 3,
360                green.w() / 4,green.h() / 4,"White",FL_WHITE };
361     white.align(FL_ALIGN_CENTER | FL_ALIGN_INSIDE);
362     // white in green ///////////////////////////////////
363     white.end();
```

```

364 // green end //////////////////////////////////
365 green.end();
366 Node magenta { wHalf,menuBar.h(),wHalf,hHalf,"Magenta",FL_MAGENTA };
367 // magenta end //////////////////////////////////
368 magenta.end();
369 // cyan in invisible //////////////////////////////////
370 Node cyan { 0,hHalf + menuBar.h(),wHalf,hHalf,"Cyan",FL_CYAN };
371 // cyan end //////////////////////////////////
372 cyan.end();
373 // yellow in invisible //////////////////////////////////
374 Node yellow { wHalf,hHalf + menuBar.h(),wHalf,hHalf,"Yellow",FL_YELLOW };
375 // yellow end //////////////////////////////////
376 yellow.end();
377 invisible.end();
378 for (int n { 0 }; n < invisible.children(); ++n)
379 {
380     invisible.child(n)->align(FL_ALIGN_TOP | FL_ALIGN_INSIDE);
381 }
382 // window end //////////////////////////////////
383 window.end();
384 window.resizable(invisible);
385 window.show();
386 return Fl::run();
387 }

```

The most noticeable change at the beginning is the introduction of a new class, `InvisibleNode`. It inherits from `Fl_Group`, declares a public constructor, overrides the `draw()` member function and provides a new implementation of `end()`. In addition to an exemplar of `Fl_Menu_Bar`, an exemplar of this class is inserted as a child in the application window, which is of type `Fl_Window` in `main()` and is named `window`. Although an `Fl_Menu_Bar` normally has a fixed position in the application window, its height would change if the application window changes its height. Therefore, another child must be selected as the resizable widget which prevents the menu bar from resizing. This is the first reason for the introduction of `InvisibleNode` and therefore it calls `draw_children()`, a protected member function declared in its parent class `Fl_Widget`.

It is also desirable to make the code that is responsible for selecting a child widget at the end of the `main()` function in Listing 97 superfluous in order to ensure operation with a keyboard. The member function `InvisibleNode::end()` does exactly this. Its definition appears after the definition of the `Node` class. The reason for this is that it performs a type cast from `Fl_Widget*` to `Node*`, which requires the `Node` class to be fully defined. The `end()` member function first calls `Fl_Group::end()`. Please note that this function does not override the member function of the base class, as it is not declared as `virtual` in the base class. Therefore, its declaration hides `Fl_Group::end()`, but nevertheless extends it.

This has another important consequence. The top-level widget must be declared with `InvisibleNode` as its static type. Otherwise, `InvisibleNode::end()` is not called. `InvisibleNode::end()` then checks whether children are present. If this is the case, it

calls the free function `asNode()`, which performs a dynamic cast with a pointer to `Node` as the result type. If `asNode()` returns a pointer that is not null, `node->focus()`; is executed to give this child the focus. The member function `focus()` cannot be called for a widget that is not of dynamic type `Node`, because `focus()` is defined the first time in `Node`. In fact, `InvisibleNode` could also have children that are not of type `Node`. Therefore, iterating over all children and returning once the first `Node` child has been found and focused would be a more precise solution.

Only minor changes are made to the `Node` class. Firstly, it inherits from `InvisibleNode` and must therefore also call the `InvisibleNode` constructor in its initializer list. Secondly, there is a small change in `focusChild()`. Here, the first condition in the if statement, `children() > 0`, which evaluates to a bool value, has been replaced with `children()`, which is automatically converted to a bool value. This was done to illustrate this possibility. If this is not covered by a programming guideline, it is up to personal taste which variant to use. However, the decision should be applied consistently. Thirdly and finally, the static member function `focused()` is declared. It returns the static data member `m_focused`. This member function is required by some of the callback functions presented below.

Next, five free functions are declared that serve as callback functions. When a function is used as callback function, a pointer to it is saved and called when a certain event occurs. Therefore, a callback function must have a type that is specifically intended for callback functions. One of these types is `F1_Callback`, which is defined in `<FL/FL_Widget.H>` as `typedef void() F1_Callback(FL_Widget *, void *)`. `F1_Callback` is a function pointer type as introduced in Section *The Virtual Function Table*. The only difference is that *FLTK* uses `typedef` for declaration, which is more traditional, while in the Section *The Virtual Function Table* using `using` is used, which is a more modern and general form of declaring type aliases. A function of type `F1_Callback` does not return a value, so its return value is specified as `void`. The first argument it takes is a pointer to `FL_Widget`, which identifies the sender object, not the intended receiver. The second argument is a pointer to `void`, which can point to any object in memory. Here it is used firstly to make data available to the callback function, which secondly can be passed on to the intended receiver object if it needs it. As a pointer to `void` cannot be dereferenced, it should first be checked to ensure that it is not null. Only in this case can a type cast be applied to it to restore a pointer to the assumed type. A pointer to `void` is always dangerous. As *FLTK* does not use any advanced C++ features, the use of pointers to `void` cannot be avoided in this case.

The first callback function is `cbAbout()`. Since it does not use the parameters passed to it, they are declared without names. The `cbAbout()` callback calls the `f1_message()` function, which starts a modal dialog that provides information about the program.

The second callback function is `cbQuit()`. It also does not use its parameters, which is why they are declared without names. It calls `f1_choice()` to ask the user if they really want to exit the program. If yes, the program is terminated by calling `exit(0)`.

The third callback function is `cbColor()`. It is called to execute the color choosing dialog for the currently focused `Node` exemplar. This is retrieved by calling `Node::focused()`. Since this pointer is of type `Fl_Widget`, it must be cast to `Node` before use. This is done by passing it as an argument to the `asNode()` function. The resulting pointer is dereferenced and `Node::changeColor()` is called for it. It should be noted that the present implementation of `cbColor()` is potentially dangerous as it does not check that the result of the call to `Node::focused()` or `asNode()` is not null before dereferencing the pointer.

The callback functions `cbLabel()` and `cbResizable()` do exactly what their names suggest. In addition, they have exactly the same structure as `cbColor()`. They are therefore not discussed further here.

The declarative description of the menu is the most eye-catching part at the beginning of `main()`. The statement `Fl_Menu_Item menuItems[];` declares a classic C-array called `menuItems`. The part `[]` means that `menuItems` is an array. Since the brackets do not contain any size information, the elements of the array must follow immediately so that compiler can count them to determine the size of the array. Its elements are of type `Fl_Menu_Item`. `Fl_Menu_Item` is defined as a struct in `<FL/Fl_Menu_Item.H>`. It neither inherits from a base class nor does it define any constructor or destructor.

According to (Costantini, Gibson, et al. n.d.), `Fl_Menu_Item` is defined as shown in Listing 99. The struct also defines member functions that are only present in the header file `<FL/Fl_Menu_Item.H>`. They are not of particular interest here.

Listing 99: `Fl_Menu_Item` (taken from (Costantini, Gibson, et al. n.d.))

```

1 struct Fl_Menu_Item {
2   const char*   text;           // label()
3   ulong        shortcut_;
4   Fl_Callback*  callback_;
5   void*        user_data_;
6   int          flags;
7   uchar        labeltype_;
8   uchar        labelfont_;
9   uchar        labelsize_;
10  uchar        labelcolor_;
11 };

```

When a struct is initialized, its members are initialized from left to right with the corresponding values from the initializer list, which is enclosed in curly brackets. Members of the struct for which no data is available are automatically zero-initialized. Data for initializing members can only be omitted from right to left. Consequently, the statement `Fl_Menu_Item item { };` defines the variable `item` of type `Fl_Menu_Item` and conceptually initializes all of its members to zero. This means that `item.text` is equal to `nullptr`.

The statement `Fl_Menu_Item item { "&About", FL_CTRL+'a', cbAbout, 0, FL_MENU_DIVIDER };` defines and initializes

- `item.text` with the *C-string* "&About",
- `item.shortcut_` with `FL_CTRL+'a'`,
- `item.callback_` with `cbAbout`,
- `item.user_data` with 0 (i.e. `nullptr`) and
- `item.flags` with `FL_MENU_DIVIDER`.

The other data members of `item` are initialized with 0. It is not allowed to omit initialization values for individual members.

If an array of `Fl_Menu_Item` is declared, the array itself must be enclosed in curly brackets. This pair of curly brackets contains all menu items. Each individual menu item must also be enclosed in curly brackets. The overall structure is therefore `Fl_Menu_Item item[] { { /* item1 */ }, { * item 2 * }, { /* ... */ }, { * item n * } };`. This is exactly the structure on which the definition and initialization of `menuItems` in `main()` is based.

Another detail needs to be explained. The definition and initialization of `item` from above is strictly linear, i.e. it is a one-dimensional array with all menu items in a linear order. There is no hierarchical nesting. But the initialization expression for `menuItems` in `main()` uses indentation of individual initialization expressions. This indentation has no meaning for the compiler or the structuring of the menu items. It reflects the visual appearance of the menu in the GUI of the program. In this way, it is much easier for the human programmer to understand what is meant by the code.

The label of a menu item can also contain special characters. For example, the `&` character defines the following character as a so-called *accelerator key*, the `_` character causes a divider to be inserted after the menu item, and the `/` character creates a submenu item for the following text. If `/` appears as the first character of a label, it is ignored. If the parent widget has the focus, any accelerator key available for the visible menu items can be pressed to activate the corresponding menu item.

The second parameter defines a *shortcut*, i.e. a key or key combination that activates the menu item in every state of the program—with the exception of modal dialogs. In `main()`, the `&About` menu item is initialized with `FL_CTRL + 'a'`. Therefore, pressing the key combination `Ctrl + a` will call the *About*-dialog from anywhere in the program. A value of 0 means that the menu item does not define a shortcut. It is not possible to use `nullptr` here, as a pointer to void cannot be automatically converted to `0u1`, but 0 can be automatically converted to `0u1`.

Setting the pointer for the callback function and the pointer for the user data is obvious. A little more can be said about the next data member, `flags`. The header file `<FL/Fl_Menu_Item.H>` defines a series of enumeration constants that can be used as flags. For example, `FL_MENU_INACTIVE` deactivates the menu item and displays it grayed out, `FL_MENU_INVISIBLE` hides the menu item completely, but its shortcut is still active, `FL_SUBMENU` starts a submenu that begins with the next menu item, and `FL_MENU_DIVIDER` inserts a divider after the menu item.

Menu items can be nested as deeply as required. To end a group of menu items in a submenu, the following menu item must be initialized as an empty entry, either by `{ }`, or by `{ 0 }` or by `{ nullptr }`. In addition, the menu items at the top level must also be terminated with an empty entry. The use of `nullptr` instead of `0` is possible here, as `0` is a permissible value for any pointer—just like `nullptr` in modern C++.

Further information on the meaning of the other data members can be found in (Costantini, Gibson, et al. n.d.).

The statement `Fl_Menu_Bar menuBar { 0,0,window.w(),30,"Menu" };` creates a menu bar. Normally, it starts at `0|0` and has the same width as its parent window, which is given here by `window.w()`. The *FLTK* documentation states that a menu bar should have a height of at least 30 pixels, otherwise the visibility of the label text may be impaired if it is drawn with the default font. Selecting a different font or font size for the menu items may make it necessary to adjust the height of the menu bar. The last parameter specifies the label of the menu bar, which is not visible. Since all subclasses of `Fl_Widget` have a label that can be queried, it can be useful to set it to a unique value, which can be used for debugging. Otherwise it can be specified with `nullptr` or `0`.

Next, the menu items must be inserted into the menu bar. This is done by calling the `copy()` member function of the menu bar and passing `menuItems` as a parameter.

Since the menu bar is created after `window` is created and before `window.end()` is called, it is automatically inserted as a child of `window`. As `Fl_Menu_Bar` inherits from `Fl_Widget`, it does not have an `end()` member function that needs to be called.

Please note that `Ctrl + o` was chosen as the shortcut for calling the `cbColor()` function. This makes sense because `Ctrl + c` as a system key can have a different meaning, e.g. to interrupt a console program, and as such is processed as a system event and not sent to the application window.

Then `invisible` is inserted into `window` as a child. It acts as a container for all the `Node` exemplars that are inserted in the following. This means that `window` only has two direct children, `menuBar` and `invisible`.

In the following, various `Node` exemplars are inserted as children or grandchildren of `invisible`. Finally, `window.end()` is called to end the insertion of children and `window` is made resizable depending on itself.

Another difference is that the member function `Node::handle()` no longer processes the keys `c`, `l` and `r`, i.e. the corresponding case branches in the nested switch statement for key have been deleted. They are superfluous, as the keys `Ctrl + o`, `Ctrl + l` and `Ctrl + r` have been defined as shortcuts to call the corresponding shortcut functions, which in turn call the corresponding member functions of the `Node` class.

When running the program, it turns out that it cannot be operated exclusively via the keyboard. The reason for this is that the focus is initially set to `invisible` and `window` as the application window never has the chance to receive the focus. Therefore, `window`

cannot give focus to the menu bar, which is why one has to click with the mouse on a top-level menu item to activate it. At the end of Section *Lambda Expressions as Callback Functions* a possibility is shown how this problem can be solved.

A good starting point to learn more about menus and menu items is to modify the program in Listing 98, recompile it and study the consequences when it is executed. It is possible to introduce more menu items and to define callback functions that simply call the `fl_message()` function to inform the user that the appropriate menu item has been selected.

2.2.8.2. Lambda Expressions as Callback Functions

C++ offers more function types and syntactic possibilities for declaring functions than have been presented so far. A so-called *lambda expression*, or *lambda* in common parlance, can be roughly considered an anonymous function that can be defined exactly where it is needed.

In this way, the use of a lambda expression can make the definition of a callback function elsewhere in the program obsolete. As long as a lambda is only needed once, its use reduces the code size and improves the locality of the program, as it is no longer necessary to go from the place that uses a callback function to the definition of the callback function to see exactly what it does.

Listing 100 shows an excerpt from a program in which the callback functions `cbColor()`, `cbLabel()` and `cbResizable()` have been replaced by corresponding lambda expressions. The callback functions mentioned above have therefore been deleted from the program. To demonstrate the use of classic callback functions as opposed to lambdas, `cbAbout()` and `cbQuit()` have been retained. Converting these functions into lambdas would also lead to a considerable increase in the initialization expression for the menu. Only the initialization expression is reproduced here, as it contains the relevant changes. The complete program is contained in the *sources/gui* folder in the accompanying material. If *fltk-config* initially refuses to compile the program, the same measures should be taken as for program `gui/NestedChildren.cpp`.

Listing 100: `gui/MenuWithLambdas.cpp`

```

327 |   Fl_Menu_Item menuItems[]
328 |   {
329 |       { "Demo",          FL_CTRL+'m',0,0,FL_SUBMENU },
330 |         { "&About",       FL_CTRL+'a',cbAbout,0,FL_MENU_DIVIDER },
331 |         { "&Quit",        FL_CTRL+'q',cbQuit,0 },
332 |         { 0 },
333 |       { "Edit",          0,          nullptr,nullptr,FL_SUBMENU },
334 |         { "C&olor",      FL_CTRL+'o',[](Fl_Widget*,void*)
335 |                                           { asNode(Node::focused())->changeColor(); }
336 |                                           },
337 |         { "&Label",       FL_CTRL+'l',[](Fl_Widget*,void*)
338 |                                           { asNode(Node::focused())->changeLabel(); }

```

```

339         },
340         { "&Resizable", FL_CTRL+'r', [] (Fl_Widget*, void*)
341             { asNode(Node::focused())->
342               changeResizable();
343             }
344         },
345     { },
346     { }
347 };

```

The definition of a lambda begins with the so-called *capture list* `[]`. Between the square brackets, it specifies which variables of the calling context are made available within the lambda and how this is done. Here, this capture list is empty, which means that none of the variables of the calling context are available within the lambda.

The parameters that are passed as function parameters are specified in the following pair of parentheses, as is the case with normal functions. The following pair of curly brackets contains the implementation of the lambda. No special restrictions apply to the implementation.

As might be noticed, the lambda does not specify a return type. If a lambda does not return a value, its return type is automatically assumed to be `void`. If it returns a value, the return type is automatically deduced from the return type. If the lambda has more than one return statement, the types of its values must not differ.

The above explanations are sufficient to understand the code shown in Listing 100. More information about lambdas can be found in the Section *Lambda Expression*.

The lambda expression of the callback function for the menu item "&Color" is `[] (Fl_Widget*, void*) { asNode(Node::focused())->changeColor(); }`.

As can be seen, the capture list is empty and the lambda has two parameters, namely a pointer to `Fl_Widget` and a pointer to `void`. Both parameters are not named as they are not used in the implementation of the lambda. Their implementation is taken directly from the callback function `cbColor()` in Listing 98. The definition of the lambda corresponds to a pointer of type `Fl_Callback`, which is used to initialize the data element `Fl_Menu_Item::callback_`.

The remaining two lambdas have exactly the same structure. Therefore, no further explanation is given for them.

There is a further change in the details of the menu items. *Ctrl + m* is defined as a shortcut for the top menu item *Demo*. This allows the user to open the first pull-down menu from anywhere in the program by pressing *Ctrl + m* and to navigate in the menu from there using the *Cursor* keys or the *Tab* key. The user must be informed about this in the documentation or in a help system, if one is available for the program.

2.2.8.3. Programmed Definition of Menus

Menus do not have to be defined statically, as shown in Sections *Declarative Representation of Menus* and *Lambda Expressions as Callback Functions*. Another possibility is to create and manage them with member functions of `Fl_Menu_Bar`, as shown in Listing 101. In addition, this program fragment demonstrates some of variants for defining menu items. Furthermore, another variant of the design of a callback function is presented. If *fltk-config* cannot compile the program, the same measures should be taken as for program `gui/NestedChildren.cpp`.

Listing 101: `gui/ProgrammedMenu.cpp`

```

304 void cb(Fl_Widget*,void* data)
305 {
306     static_assert(sizeof(unsigned long) == sizeof(void*));
307     auto value { reinterpret_cast<unsigned long*>(data) };
308     switch (value)
309     {
310         case 1: fl_message("Menu application, July 9, 2025\n"
311                             "by Ulrich Eisenecker");
312                 break;
313         case 2: if (fl_choice("Do you really want to quit the program?",
314                             "Ok","Cancel",0) == 0)
315                 {
316                     exit(0);
317                 }
318                 break;
319         case 3: asNode(Node::focused()->changeColor();
320                 break;
321         case 4: asNode(Node::focused()->changeLabel();
322                 break;
323         case 5: asNode(Node::focused()->changeResizable();
324                 break;
325         default: fl_alert("Unknown menu item %ld",value);
326                  break;
327     }
328 }
329
330
331 int main()
332 {
333     // window ///////////////////////////////////
334     Fl_Window window { 640,480,"Parent" };
335     // menuBar in window ///////////////////////////////////
336     Fl_Menu_Bar menuBar { 0,0,window.w(),30,"Menu" };
337     menuBar.add("Demo/_&About",FL_CTRL+'a',cb,(void*)1ul);
338     menuBar.add("Demo/_Quit",    FL_ALT+'q', cb,(void*)2ul);
339
340     menuBar.add("Edit/_&Color",    "#c", cb,reinterpret_cast<void*>(3ul));
341     menuBar.add("Edit/_&Label",    "+l", cb,reinterpret_cast<void*>(4ul));
342     menuBar.add("Edit/_&Resizable", "^c",cb,reinterpret_cast<void*>(5ul));
343     menuBar.add("Edit/_&Unknown",  0,    cb,reinterpret_cast<void*>(6ul));
344

```

First, the `main()` function is discussed.

After the menu bar has been created, new menu items are inserted with an overload of the `add()` member function. The overloads of the `add()` member functions are defined in `Fl_Menu_`, which is the base class of `Fl_Menu_Bar`. The second overload of `add()` is used first. It accepts the *C-string* label, the `int` shortcut, the `Fl_Callback*` callback, the `void*` userdata and the `int` flags as parameters. The default value for `userdata` and `flags` is 0, which means that this member function can be called in three different variants. Overall, these parameters correspond exactly to the members of the `Fl_Menu_Item` struct.

The statement `menuBar.add("&Demo/_&About", FL_CTRL+'a', cb, (void*)1ul);` inserts the menu item "Demo". The `/` character serves as a separator for the following submenu item `"_&About"`, which first inserts a separator because of `_` and then specifies `A` as the shortcut key. The following parameters all relate to the `"_&About"` submenu item. This means that the key combination *Ctrl + a* is specified as accelerator key for `"_&About"`, and `cb` is passed as a pointer to the callback function. The last parameter is special. When the `"_&About"` menu item is selected, `1ul` should be passed as a value to the callback function `cb()` to identify the `"_&About"` menu item. But `cb()` expects a pointer to `void`, while `1ul` is an unsigned long integer. A traditional cast is used here to obtain a void pointer value from `1ul` (see Section *Traditional Cast and Functional Cast*).

This is only shown here because *FLTK* itself does not make use of the C++ cast operators. As an alternative, a `reinterpret_cast<>()` is used to convert the `3ul` literal into a pointer to `void`. It is not allowed to convert a `void*` to an unsigned long int, as the compiler treats them as completely different and unrelated types. Therefore, a `reinterpret_cast<>()` must be used. Since `3ul` is a literal, it is not possible to take its address. This is the reason for specifying a new value of type `void*` as the result of `reinterpret_cast<>()`, whereas normally a reference is specified (see Section *Accessing Bits of a Float Variable With reinterpret_cast<>()*). The C++ standard does not stipulate that unsigned long int and `void*` must have the same size. This is why the `static_assert()` mentioned above was included in the program. It checks whether both types have the same size.

Next, more menu items are added to the menu bar using the third overload of the member function `Fl_Menu_::add()`. Instead of `int`, it uses `const char *` as the type of the second parameter. This enables further variants for specifying shortcuts:

- A hash sign in front of a key in the shortcut specification is the same as adding `FL_ALT` to the key, e.g. `"#c"` means that *Alt + c* must be pressed.
- A plus sign in front of a key in the shortcut specification is the same as adding `FL_SHIFT` to the key, e.g. `"+l"` means that *Shift + l* must be pressed.
- A caret sign in front of a key in the shortcut specification is the same as adding `FL_CTRL` to the key, e.g. `"^c "` means the same as *Ctrl + c*.

The last menu item specifies `reinterpret_cast<void*>(6ul)` as the value for the parameter `userdata`. As will be seen below, this value is *not known* to the only callback function that is specified in all menu items.

Please note that this menu again raises the problem that the application cannot be operated exclusively with the keyboard, as it is not possible to give the focus to the menu bar without using the mouse.

There is only one function defined that serves as a callback function, namely `cb()`. As required, it specifies two parameters. The first is a pointer to `Fl_Widget` and is not used in the function. It is therefore not named. The second parameter is a pointer to `void`. It has the name `data` and is used to tell the function which menu item was pressed to execute the corresponding code.

First, a `static_assert()` statement is included in the function definition. It checks whether a pointer to `void` and unsigned long int have the same size in bytes. This is because unsigned long int values are used below to identify the menu items. This check is performed at compile time and is normally successful.

Then a reinterpret cast is executed to initialize the auto variable `value` with `data` as a reference to unsigned long int. It should be noted that `value` is still a new variable and not a reference, as `auto` removes a reference. In order for `value` to become a reference for the result of the type conversion, the reference specifier `&` would have to be placed after `auto`.

The following switch statement for `value` executes the corresponding code for each known value of a menu item. The default branch calls the `fl_alert()` function to display a warning dialog informing about an unknown value for a menu item.

There are other ways to create menus that are not covered here. The information presented so far should be sufficient for the reader to find out about these other options through self-study.

2.2.9. Observer

In software engineering, ***the term design pattern refers to a canonical description of a generic problem-solution pair that is largely independent of a programming language.*** The best known book on design patterns is (Gamma et al. 1995). *Canonical* means that the description follows an agreed structure. *Generic* means that it is not specific, but refers to a group or a family. In this respect, generic also means that it abstracts from specific details. *Problem-solution pair* means that there is a recurring problem for which a proven solution exists. The solution can be associated with advantages and disadvantages. When applying a design pattern other design patterns may come into play. Because a design pattern is more or less abstract, a specific application of it must be implemented. For some design patterns, implementations can be made available to facilitate their application.

The *Observer* design pattern, also known as *Publisher-Subscriber*, is a pattern for which an implementation can be provided. In GUI applications, the problem often arises that data or an object is displayed in different windows or by different widgets. It is also possible to change these data or object, and these changes must be reflected in each visible representation. It is useful to think about this problem as an interaction between a so-called *Subject* and possible *Observers* of it.

2.2.9.1. An Implementation of Observer

There are many comprehensive descriptions of the Observer pattern, for example (*Observer Pattern* 2024) which also contain *UML* class diagrams and a sequence diagram for it. In the following, the focus is therefore on the C++ implementation shown in Listing Observer (excerpt from `gui/Observer.cpp`).

Listing 102: Observer (excerpt from `gui/Observer.cpp`)

```

14 class Observer; // Pending declaration
15
16 class Subject
17 {
18     public:
19         Subject()
20         {}
21         bool attach(Observer* observer);
22         bool detach(Observer* observer);
23         void notify() const;
24         virtual ~Subject();
25         Subject& operator=(const Subject&) = delete;
26         Subject(const Subject&) = delete;
27     private:
28         std::vector<Observer*> m_observers { };
29 };
30
31 class Observer
32 {
33     public:
34         Observer(Subject* subject = nullptr): m_subject { subject }
35         {
36             if (m_subject)
37             {
38                 m_subject->attach(this);
39             }
40         }
41         void subject(Subject* s = nullptr)
42         {
43             if (m_subject)
44             {
45                 m_subject->detach(this);
46             }
47             m_subject = s;
48             if (m_subject)

```

```
49         {
50             m_subject->attach(this);
51             update();
52         }
53     }
54     virtual void update() = 0;
55     virtual ~Observer()
56     {
57         if (m_subject)
58         {
59             m_subject->detach(this);
60         }
61     }
62     Observer(const Observer&) = delete;
63     Observer& operator=(const Observer&) = delete;
64 protected:
65     Subject* m_subject;
66 };
67
68 bool Subject::attach(Observer* observer)
69 {
70     if (observer != nullptr)
71     {
72         auto result { std::find(m_observers.begin(),m_observers.end(),observer) };
73         if (result == m_observers.end())
74         {
75             m_observers.push_back(observer);
76             observer->subject(this);
77             return true;
78         }
79     }
80     return false;
81 }
82
83 bool Subject::detach(Observer* observer)
84 {
85     auto result { std::find(m_observers.begin(),m_observers.end(),observer) };
86     if (result == m_observers.end())
87     {
88         return false;
89     }
90     m_observers.erase(result);
91     return true;
92 }
93
94 void Subject::notify() const
95 {
96     for (const auto& observer : m_observers)
97     {
98         observer->update();
99     }
100 }
101 }
102
103 Subject::~Subject()
```

```

104 | {
105 |     for (auto& observer : m_observers)
106 |     {
107 |         observer->subject(); // Remove subject from observer.
108 |     }
109 | }

```

Unlike in many articles and books, Subject is not an abstract class here. For its implementation, it uses the container template `std::vector<>` and the algorithm `std::find<>()`. Therefore, the header files `<vector>` and `<algorithm>` must be included, which is not shown here.

The pending declaration of Observer at the beginning is necessary because Subject uses pointers to Observer in its definition. As it would be necessary to consider in detail what copying or assigning a Subject exemplar means, the corresponding member functions have been deleted. Due to the deletion of the copy constructor, the standard constructor is no longer generated automatically. It is therefore defined manually with an empty implementation. The member function `attach()` registers a pointer to an Observer exemplar. It returns `true` if this has been done successfully and `false` otherwise.

At this point, Observer is an incomplete type. Therefore, the member function must be defined separately after the complete definition of the Observer class. The member function `detach()` removes a pointer to an Observer exemplar from the Subject exemplar. If this was done successfully, `detach()` returns `true`, otherwise `false`. Like `attach()`, `detach()` must be defined after the observer class for the same reason. As none of these member functions are prefixed with the `[[nodiscard]]` attribute, their return values can be ignored without warning. This also applies to the member function `notify()`. It is specified as `const` because it does not change the state of the receiver object, and it does not return anything, i.e. its return type is `void`. The purpose of `notify()` is to inform all observers of the subject of a change to the subject.

The destructor is `virtual`, which is necessary for Subject to be a valid base class. It must be emphasized that none of the other member functions are declared as `virtual`, which means that none of them can be overridden in a subclass.

The only data member of Subject, namely `m_observers`, is the container template `std::vector<>`, which is instantiated for `Observer*` and declared as `private`, i.e. it cannot be accessed from outside or by a child class.

Now to the implementation details of the Subject member functions.

The purpose of the member function `Subject::attach()` is to register a new Observer exemplar for a Subject exemplar. It first checks whether `observer` is not equal to `nullptr`. If this is not the case, no further code is executed and the member function returns `false` to indicate to the caller that this pointer to Observer has not been inserted as a valid observer. Otherwise, the auto variable `result` is initialized with the result of the call to `std::find<>()` to search for `observer` in all observers. If `observer` is not con-

tained in `m_observers`, i.e. `result` is equal to `m_observers.end()`, `observer` is appended to `m_observers` as the last element. Now the subject of the newly inserted observer must be set to the active object, i.e. `this`. Otherwise `observer` would not know which subject to observe. The member function then returns `true` to indicate that the observer has been successfully registered.

The member function `Subject::detach()` is called to deregister an `Observer` exemplar from the `Subject` exemplar. It takes a pointer to an `Observer` exemplar and searches `Subject::m_observers` for it. The auto variable `result` is initialized with the result of this search. If `result` is equal to `m_observers.end()`, the `Observer` exemplar is not registered as an observer of the `Subject` exemplar. Therefore, the `Observer` exemplar cannot be removed and the member function returns `false`. Otherwise, `result` is passed as an argument to `m_observers.erase()`, which removes `result` from the container. Then `true` is returned as the result.

The `Subject::notify()` member function is declared as `const`, since it does not modify the state of the receiver object. In its range-based for loop, it calls the `update()` member function for each pointer to a registered `Observer` exemplar. In this way, the corresponding `Observer` exemplar knows that its observed `Subject` exemplar has been changed.

If a `Subject` exemplar is destroyed, its destructor must inform all registered `Observer` exemplars and cancel the bidirectional relationship between `Observer` and `Subject`. For this purpose, the destructor executes the `subject()` member function without an argument for each pointer to `Observer` contained in `Subject::m_observers` in a range-based for loop.

The copy constructor and the assignment operator of the `Observer` class are also deleted. This is done for the same reason as for the `Subject` class. A constructor must therefore be defined manually for `Observer`. It takes a pointer to `Subject` with the name `subject` as its only argument. In addition, `nullptr` is specified as the default value for `subject`. An `Observer` exemplar has only one data member, namely `m_subject`, which is a pointer to `Subject`. In the initializer list, `m_subject` is initialized with `subject`. In the implementation, the constructor checks whether `m_subject` is different from `nullptr`. For this purpose, the automatic conversion from the integer value 0 to `false` and from any non-zero to `true` is used. If `m_subject` is not null, the `Observer` exemplar calls `attach()` for `m_subject` and passes this as an argument to register itself as an `Observer` exemplar of the subject.

The member function `Observer::subject()` establishes or removes a connection to a `Subject` exemplar. It takes a pointer to `Subject` with the name `s` as an argument, for which `nullptr` is specified as the default value. If `m_subject` is not null, the `Observer` exemplar first calls `detach()` for the `m_subject` pointer and passes itself with this as an argument. This first breaks the existing connection between the `Observer` exemplar and the current `Subject` exemplar. Next, `s` is assigned to `m_subject`. If `s` is `nullptr`, the intended effect, namely the removal of the connection between `Observer` and `Subject`, has already been achieved and nothing further needs to be done. Otherwise, the new

connection between the Observer exemplar and the Subject exemplar must be established by calling `attach()` for `m_subject` passing this as an argument. The following call to `update()` is important as it triggers the fetching of the data from the Subject exemplar. Omitting this call would result in an Observer observing a new Subject, but not immediately displaying the state of the Subject.

`Observer::update()` is a pure virtual member function, which makes Observer an abstract class. Therefore, a subclass must override `Observer::update()` and provide an implementation. Normally, this implementation will cast the `Observer::m_subject` pointer into a pointer of a known subclass of Subject to call all getter methods in order to update itself.

The destructor is also declared as `virtual`, which is necessary for Observer to become a valid base class. The destructor checks whether `m_subject` is not null. If this is the case, it calls `detach()` for the pointer `m_subject` passing this as an argument in order to remove the relationship between the Subject exemplar and itself.

The presented implementation of the Observer design pattern is sufficient and lightweight. It is imaginable to provide implementations for the deleted copy constructors and assignment operators. One could also think about extending the Observer class so that it can observe any number of Subject exemplars. To make it generally usable, it would have to be rewritten as a valid header file and its classes placed in a namespace. All member function shown here can easily be implemented inline, as they are lightweight and cause only a small amount of code bloat.

2.2.9.2. Sub-classing of Observer and Subject

This section shows how to correctly create subclasses of Observer and Subject.

The *gui/Observer.cpp* program is based on *gui/GuiMove.cpp*. It introduces a window and a symbol that can be moved in the window. The window offers menus for changing the symbol, the font, the size and the color.

The *Star* class is a subclass of *Subject* (Listing 103). An exemplar of *Star* manages a symbol, a font, a font size and a color for drawing. The *StarWindow* class (Listing 104) is a subclass of both *Observer* and *Fl_Window*. The use of multiple inheritance allows the functionality of *Observer* to be elegantly combined with the *FLTK* class *Fl_Widget*.

Listing 103: Star (excerpt from gui/Observer.cpp)

```
111 class Star: public Subject
112 {
113     public:
114         Star(char symbol = '*', Fl_Font font = FL_SCREEN,
115             Fl_Fontsize size = FL_NORMAL_SIZE,
116             Fl_Color color = FL_BLACK): m_symbol { symbol },
117                                     m_font { font },
118                                     m_size { size },
119                                     m_color { color }
120     {}
121     void symbol(char symbol)
122     {
123         m_symbol = symbol;
124         notify();
125     }
126     void font(Fl_Font f)
127     {
128         m_font = f;
129         notify();
130     }
131     void size(Fl_Fontsize s)
132     {
133         m_size = s;
134         notify();
135     }
136     void color(Fl_Color c)
137     {
138         m_color = c;
139         notify();
140     }
141     char symbol() const
142     {
143         return m_symbol;
144     }
145     Fl_Font font() const
146     {
147         return m_font;
148     }
149     Fl_Fontsize size() const
150     {
151         return m_size;
152     }
153     Fl_Color color() const
154     {
155         return m_color;
156     }
157     private:
158         char m_symbol;
159         Fl_Font m_font;
160         Fl_Fontsize m_size;
161         Fl_Color m_color;
```

162 | };

The `Star` class defines four data members, namely `m_symbol` of type `char`, `m_font` of type `Fl_Font`, `m_size` of type `Fl_Fonsize` and `m_color` of type `Fl_Color`. The use of the latter three types makes `Star` depend on `FLTK`. This dependency is tolerable as `Star` is a very light-weight class that offers no significant functionality apart from its three data members, which are types from *FLTK*.

The constructor of `Star` specifies arguments to initialize all data members as well as default values for each of its arguments. Therefore, it can also be called as a standard constructor without arguments. In its initializer list, it does not explicitly call a constructor of its base class `Subject`. For this reason, the standard constructor of `Subject` is automatically called here.

All other member functions of `Star` are either getter or setter methods. Each setter method calls `notify()` after it has assigned a new value to a data member. This triggers the call of the `update()` method for each observer. These member functions are very simple and are thus not explained further.

Apart from the fact that `Star` publicly inherits from `Subject`, there is no further dependency on `Subject` which improves its modularity.

Listing 104: `StarWindow` (excerpt from `gui/Observer.cpp`)

```

164 class StarWindow: public Fl_Window,public Observer
165 {
166     public:
167         StarWindow(int width,int height,const char* title,Star* star = nullptr)
168             : Fl_Window { width,height,title },Observer { star },
169             m_menu { 0,0,w(),30,"MenuBar" }
170     {
171         m_x = w() / 2;
172         m_y = (h() - m_menu.h()) / 2 + m_menu.h();
173         m_menu.add("&Star/_&About",FL_CTRL + 'a',
174                 [](Fl_Widget*,void*)
175                 { fl_message("Star Application"); });
176         m_menu.add("&Star/&Quit",FL_CTRL + 'q',
177                 [](Fl_Widget*,void*)
178                 { if (fl_choice("Really quit?","Ok","Cancel",0) == 0)
179                     exit(0); });
180         m_menu.add("&Edit/&Symbol",FL_CTRL + 's',
181                 cbMenu,reinterpret_cast<void*>(1ul));
182         m_menu.add("&Edit/_&Color",FL_CTRL + 's',
183                 cbMenu,reinterpret_cast<void*>(2ul));
184         m_menu.add("&Edit/&Font/&Helvetica",0,
185                 cbMenu,reinterpret_cast<void*>(3ul));
186         m_menu.add("&Edit/&Font/&Times",0,
187                 cbMenu,reinterpret_cast<void*>(4ul));
188         m_menu.add("&Edit/&Font/&Courier",0,
189                 cbMenu,reinterpret_cast<void*>(5ul));

```

```

190     m_menu.add("&Edit/&Font/&Default",0,
191               cbMenu,reinterpret_cast<void*>(6ul));
192     m_menu.add("&Edit/&Size/&8",0,
193               cbMenu,reinterpret_cast<void*>(7ul));
194     m_menu.add("&Edit/&Size/1&2",0,
195               cbMenu,reinterpret_cast<void*>(8ul));
196     m_menu.add("&Edit/&Size/2&4",0,
197               cbMenu,reinterpret_cast<void*>(9ul));
198     m_menu.add("&Edit/&Size/&Default",0,
199               cbMenu,reinterpret_cast<void*>(10ul));
200 }
201 void update() override
202 {
203     if (m_subject)
204     {
205         auto star { dynamic_cast<Star*>(m_subject) };
206         if (star)
207         {
208             m_symbol[0] = star->symbol();
209             m_font = star->font();
210             m_size = star->size();
211             m_color = star->color();
212             redraw();
213         }
214     }
215 }
216 void draw() override
217 {
218     if (m_subject)
219     {
220         Fl_Window::draw(); // Must call Fl_Window::draw() beforehand for the
221         fl_font(m_font,m_size); // font and font size setting to take effect
222         fl_text_extents(m_symbol,m_dx,m_dy,m_width,m_height);
223         fl_color(m_color);
224         fl_draw(m_symbol,m_x + m_dx - m_width / 2 + 1,
225               m_y - m_dy / 2 - m_height / 2);
226     }
227 }
228 void flash()
229 {
230     auto oldColor { color() };
231     color(FL_RED);
232     redraw();
233     Fl::wait(0.25);
234     color(oldColor);
235     redraw();
236 }
237 static void cbMenu(Fl_Widget* w,void* data)
238 {
239     auto window { dynamic_cast<StarWindow*>(w->parent()) };
240     Star* star { nullptr };
241     if (window)
242     {
243         star = dynamic_cast<Star*>(window->m_subject);
244     }

```

```

245     if (window && star)
246     {
247         auto menuId { reinterpret_cast<unsigned long>(data) };
248         switch (menuId)
249         {
250             case 1:
251             {
252                 bool again { false };
253                 do
254                 {
255                     auto newLabel { fl_input("New symbol: ",
256                                             window->m_symbol) };
257                     if (newLabel == nullptr)
258                     {
259                         break;
260                     }
261                     again = (strlen(newLabel) != 1);
262                     if (again)
263                     {
264                         fl_alert("Symbol must have only 1 char, "
265                                 "please retry.");
266                     }
267                     else
268                     {
269                         window->m_symbol[0] = newLabel[0];
270                     }
271                     } while (again);
272                     star->symbol(window->m_symbol[0]);
273                 }
274                 break;
275             case 2:
276                 uchar r,g,b;
277                 Fl::get_color(window->m_color,r,g,b);
278                 if (fl_color_chooser("Select color",r,g,b) == 1)
279                 {
280                     window->m_color = fl_rgb_color(r,g,b);
281                     star->color(window->m_color);
282                 }
283                 break;
284             case 3:
285                 window->m_font = FL_HELVETICA;
286                 star->font(window->m_font);
287                 break;
288             case 4:
289                 window->m_font = FL_TIMES;
290                 star->font(window->m_font);
291                 break;
292             case 5:
293                 window->m_font = FL_COURIER;
294                 star->font(window->m_font);
295                 break;
296             case 6:
297                 window->m_font = FL_SCREEN;
298                 star->font(window->m_font);
299                 break;

```

```
300         case 7:
301             window->m_size = 8;
302             star->size(window->m_size);
303             break;
304         case 8:
305             window->m_size = 12;
306             star->size(window->m_size);
307             break;
308         case 9:
309             window->m_size = 24;
310             star->size(window->m_size);
311             break;
312         case 10:
313             window->m_size = FL_NORMAL_SIZE;
314             star->size(window->m_size);
315             break;
316         default: fl_alert("Unknown menu id: %lu",menuId);break;
317     }
318 }
319 }
320 static void cbLeft(Fl_Widget* w,void*)
321 {
322     auto window { dynamic_cast<StarWindow*>(w) };
323     if (window)
324     {
325         window->m_x -= window->m_width;
326         if (window->m_x < window->m_dx + 1)
327         {
328             window->m_x = -window->m_dx + 1;
329             window->flash();
330         }
331     }
332     window->redraw();
333 }
334 static void cbRight(Fl_Widget* w,void*)
335 {
336     auto window { dynamic_cast<StarWindow*>(w) };
337     if (window)
338     {
339         window->m_x += window->m_width;
340         if (window->m_x > window->w() - window->m_width - window->m_dx + 1)
341         {
342             window->m_x = window->w() - window->m_width - window->m_dx + 1;
343             window->flash();
344         }
345     }
346     window->redraw();
347 }
348 static void cbUp(Fl_Widget* w,void *)
349 {
350     auto window { dynamic_cast<StarWindow*>(w) };
351     if (window)
352     {
353         window->m_y -= window->m_height;
354         if (window->m_y < -window->m_dy + window->m_menu.h() - 1)
```

```

355         {
356             window->m_y = -window->m_dy + window->m_menu.h() - 1;
357             window->flash();
358         }
359     }
360     window->redraw();
361 }
362 static void cbDown(Fl_Widget* w,void*)
363 {
364     auto window { dynamic_cast<StarWindow*>(w) };
365     if (window)
366     {
367         window->m_y += window->m_height;
368         if (window->m_y > window->h() - 2 *
369             window->m_height - window->m_dy - 1)
370         {
371             window->m_y = window->h() - window->m_height - window->m_dy - 1;
372             window->flash();
373         }
374     }
375     window->redraw();
376 }
377 private:
378     char m_symbol[2] { " " };
379     Fl_Font m_font { FL_SCREEN };
380     Fl_Fonsize m_size { FL_NORMAL_SIZE };
381     Fl_Color m_color { FL_BLACK };
382     int m_x { },
383         m_y { },
384         m_width { },
385         m_height { },
386         m_dx { },
387         m_dy { };
388     Fl_Menu_Bar m_menu;
389 };

```

The `StarWindow` class is somewhat more complicated. However, it is similar to the `MyWindow` class from Listing 84. It defines the members `m_x` and `m_y`, both of type `int`, which manage the position at which the symbol is to be drawn. The data members `m_width`, `m_height`, `m_dx` and `m_dy`, all of type `int`, manage the information on how the symbol can be drawn correctly aligned in a fictitious box. The data members `m_symbol`, a character array with two elements, `m_font` of type `Fl_Font`, `m_size` of type `Fl_Fonsize` and `m_color` of type `Fl_Color` cache the corresponding values of an observed `Star` exemplar. This makes it unnecessary to retrieve the corresponding values from the observed `Star` exemplar each time the window is drawn. All the data members mentioned above are initialized when they are declared. Please note the initialization of `m_symbol` with the C-String `" "`. As a result, the first element, which is the symbol, is actually initialized with the *Space* character, which has no visible effect when drawing. The second element is the value `0` which correctly terminates the C-String and must not be replaced by another value.

The last data member defined in `StarWindow` is `m_menu` of type `Fl_Menu_Bar`. It contains the menus of a `StarWindow` exemplar. No data member is defined that points to a `Star` exemplar, as this pointer is managed by the `Observer` base class as a pointer to `Subject`.

The `StarWindow` constructor takes four arguments. The first three are width, height and title. They correspond to the arguments of the `Fl_Window` constructor, which also requires three arguments. Therefore, they are simply passed as arguments to the call of the `Fl_Window` constructor in the initializer list. The fourth argument is a pointer to `Star` named `star`, for which `nullptr` is specified as the default value. It is passed as an argument to the call of the `Observer` constructor in the initializer list. The only data member that is initialized in the initializer list is `m_menu`. Its constructor is called with `0|0` as the origin of the menu bar in the main window, `w()` as the width of the parent window that owns the menu bar, `30` as the recommended height of the menu bar and the C-String `"MenuBar"` as the same name for the menu bars of all `StarWindow` exemplars.

In the implementation of the `StarWindow` constructor, `m_x` and `m_y` are assigned the values that correspond to the center of the application window. Please note that the height of the menu bar is taken into account when calculating `m_y`. Next, the menu is programmed. The callback functions for the menu items `"About"` and `"Quit"` are implemented as lambda expressions. The other menu items all use the pointer to the same static member function `cbMenu()`. In addition, each menu item specifies a unique unsigned long literal. It is cast into a pointer to `void` and identifies the corresponding menu item.

Next, the `Observer::update()` member function is overridden to provide an implementation for it. This is the only customization beyond specifying `Observer` as the base class to become a valid `Observer` subclass. If `m_subject` is not null, the auto variable `star` is initialized with the result of a `dynamic_cast<>()` with `Star*` as the result type. If the result is not null, the corresponding getter functions of `Star` are called and their return values are assigned to the corresponding data members of `StarWindow`. Since `Star` only manages a single character, this character is assigned to the first element of the `m_symbol` array, i.e. `m_symbol[0]`. By initializing `m_symbol` with `" "`, the first element of this character array is replaced with the corresponding symbol managed by the `Star` exemplar. After assigning the member variables that cache the `Star` values, `redraw()` is called to redraw the window as soon as necessary.

The `draw()` member function overrides the corresponding `Fl_Window` member function. If `m_subject` equals `nullptr`, no code is executed. Please note that `Observer::m_subject` is defined as a protected data member. This allows it to be used directly in a subclass, as is done here. Otherwise, the code for drawing the window is executed. It should be mentioned that `Fl_Window::draw()` is called first. This is mandatory because this call can and does change the settings for drawing, such as font, size and color. Next, the font and size are set by calling `fl_font()`. The fictitious box for drawing the symbol is then calculated by calling `fl_text_extents()`. After that, the color is set and the symbol is drawn.

The `flash()` member function was taken unchanged from `gui/GuiMove.cpp`.

The static `StarWindow::cbMenu()` member function is too large from a software quality point of view. Fortunately, it has a simple structure and offers a single point of customization, which has a positive effect on software quality. When `cbMenu()` is called, two arguments are passed by the caller. The first is a pointer of type `F1_Widget` with the name `w`, which identifies the caller of `cbMenu()`. As the menu item is only available via the menu bar, `w` points to the menu bar that belongs to the respective `StarWindow` exemplar. In principle, `cbMenu()` could also be registered as a callback function by other widgets, but this is not the case here. The second parameter `data`, which is passed as a pointer to void, contains the identifier that uniquely identifies the corresponding menu item.

Since the caller of `cbMenu()` is the menu bar, its parent is a `StarWindow` exemplar. A pointer to this window is obtained by calling `w->parent()`. A dynamic cast is performed with `StarWindow*` as the result type, and the result is used to initialize the auto variable `window`. Next, the `Star` pointer `star` is declared and initialized with `nullptr`. If `window` is not null, another dynamic cast with `Star*` as the result type is applied to `window->m_subject`, the result of which is assigned to `star`. The following if statement makes use of the Boolean short-circuit evaluation. The if branch is only executed if `window` is not null and `star` is not null. Otherwise, no further statements are executed and the member function is terminated.

In the if branch, a reinterpret cast with unsigned long as the result type is applied to `data`. This is necessary because a pointer to void and unsigned long are not related types. The result is used to initialize the auto variable `menuId`.

This is followed by a large switch statement for `menuId`. The case branch for 1 starts a new block in which the bool variable `again` is declared and initialized with false. Now a do-while loop is executed, which calls `f1_input()` to get a new label. The result of the call to `f1_input()` is used to initialize the auto variable `newLabel`. Since `f1_input()` returns a pointer to const char, `newLabel` is of type const char. If `newLabel` is equal to `nullptr`, the user has pressed the *Cancel* button and the case branch is exited immediately with `break`. Otherwise, the `strlen()` function is used to calculate the length of `newLabel`. Because `strlen()` is used, `<cstring>` must be included as a header file. If the size of `newLabel` is not exactly 1, an error message is displayed by calling `f1_alert()` and the user is prompted to repeat the input. As soon as exactly one character has been entered, `newLabel[0]` is assigned to `window->m_symbol[0]`. When the do-while loop has ended, `star->symbol(window->m_symbol[0])` is called to change the symbol of the observed subject.

There are a few points to consider here. First, it is not user-friendly to allow the input of an invalid result, as is the case with the call to `f1_ask()`. According to GUI design guidelines, a dialog should be used that allows the user to select only valid results, in this case only a single character that can be used as symbol (perhaps with the exception of the *Space* character). Secondly, `f1_ask()` returns a C-string and `f1_draw()` requires

a C-string as its first argument. For this reason, `m_symbol` is a C-string and not a single character. This is an almost unavoidable mismatch with respect to `Star::m_symbol`, being of type `char`, which is almost impossible to fix. Thirdly, in this and other case branches, new values are set for `Star` attributes, which goes beyond the boundaries of the *Observer* design pattern. By changing values of the observed subject, a `StarWindow` exemplar also acts in the role of a *controller*. A modification, or more precisely an extension of the *Observer* pattern is the *Model-View-Controller* design pattern, which explicitly adds the possibility of also modifying a Subject exemplar, which is called *Model* here. For more information, see (*Model-View-Controller* 2024).

The case branch for 2 asks for a new color and sets it for `star` if a new color has been chosen by the user. This corresponds to the member function `Node::changeColor()` in Listing 93, so no further explanation is given here.

The case branches for 3 to 10 have the same simple structure. In each case branch, the value of the cached data member is set first and then the corresponding setter method for the `Star` pointer is called.

The default case branch should never be executed. When using a new menu ID and forgetting to implement the corresponding case branch, a dialog is displayed that informs about the unknown menu ID.

The following four callback functions, `cbLeft()`, `cbRight()`, `cbUp()` and `cbDown()` are not called by any other member function of `StarWindow`. They provide the functionality to move the symbol in the drawing area of the window. To achieve this, a dynamic cast with `StarWindow*` as the result type is first applied to the argument `w`, which is of type `Fl_Window*`. The result is used to initialize the auto variable `window`. If `window` is not equal to `nullptr`, `window->m_x` and `window->m_y` are updated accordingly. A check is made to see whether the fictitious box in which the symbol is drawn is still within the window boundaries minus the menu bar. If this is not the case, the coordinate is adjusted accordingly and `window->flash()` is called.

As might be noticed, `StarWindow` does not override the member function `handle()`. The handling of *Cursor* keys is implemented in the free function `eventHandler()`, which is briefly explained in the following section.

Changing the font of the symbol and its size in the way shown above is not very effective and intuitive. In terms of user-friendliness, it would make more sense to use a font selection dialog for this purpose. Unfortunately, *FLTK* does not offer such a dialog and implementing one would go beyond the purpose of this book. Therefore, this overly complex way was chosen for a somewhat limited possibility to change these two font attributes.

2.2.9.3. Using Observers and Subjects

The focus of this section is on the use of concrete observers and subjects. In addition, some special features of the resulting behavior and the possibilities of the *Observer* design pattern are shown.

Two more functions are presented below, namely `eventHandler()` and `main()`.

The free function `eventHandler()` (Listing 105) has two arguments. The first, `event` of type `int`, identifies the event, and the second, `w`, a pointer to `Fl_Window`, identifies the window that received the event. First, a dynamic cast with `StarWindow*` as the result type is applied to `w`. The result is used to initialize the auto variable `window`. If `window` is different from `nullptr`, it is checked, whether `event` is equal to `FL_KEYBOARD`. If so, the corresponding key is retrieved by calling `Fl_event_key()` and used to initialize the auto variable `key`. A switch statement for `key` checks whether `key` is equal to a *Cursor* key. If yes, the corresponding callback function of `StarWindow` is called and 1 is returned as the result. If the event is not known, the function returns the result of the call to `Fl::handle_(event,w)`, so that the event handlers of other widgets can handle the event.

Listing 105: `eventHandler()` (excerpt from `gui/Observer.cpp`)

```

391 int eventHandler(int event, Fl_Window* w)
392 {
393     auto window { dynamic_cast<StarWindow*>(w) };
394     if (window)
395     {
396         if (event == FL_KEYBOARD)
397         {
398             auto key { Fl::event_key() };
399             switch (key)
400             {
401                 case FL_Right: StarWindow::cbRight(window, nullptr); return 1;
402                 case FL_Left:  StarWindow::cbLeft(window, nullptr);  return 1;
403                 case FL_Up:    StarWindow::cbUp(window, nullptr);    return 1;
404                 case FL_Down:  StarWindow::cbDown(window, nullptr);  return 1;
405             }
406         }
407     }
408     return Fl::handle_(event,w);
409 }
```

Two things should be mentioned here. Firstly, using the *Cursor* keys to move the symbol generates new events on all tested platforms (*macOS*, *MS Windows*, *Ubuntu*) when the *Cursor* key is held down. This makes it easier to move the symbol on *macOS*. Secondly, the use of static callback member functions of `StarWindow` seems to be too complex. This was done here to allow for later extensions of the program, such as a toolbar with widgets that can be used instead of the *Cursor* keys to move the symbol. In addition, an accelerator value could be passed as a second argument which controls the distance of a single movement of the symbol. In practice, this would be not appropriate if there were

not an optional requirement specifying exactly this possible extension of the program. Incidentally, this also applies to the use of static member functions as callbacks due to a speculative later extension. In practice, it is disadvantageous to provide such extension points if they are not desired.

The `main()` function (Listing 106) first sets `eventHandler()` as the event handling function by calling `F1::event_dispatch()`. Next, a first `Star` exemplar named `star1` is created. Then a first `StarWindow` exemplar called `window1` is created. Next, `star1.attach(&window1)` is called to register `window1` as an observer of `star1`. Finally, `window1` is shown. Now something happens that has not yet been demonstrated: an application can comprise more than one window. A second `StarWindow` exemplar called `star2` is created and also registered as an observer of `star1`. In addition, `window3` and `star2` are created and `window3` is registered as an observer of `star2`. Of course, `show()` must also be called for `window2` and `window3`. Finally, the event loop is started by calling `F1::run()`.

Listing 106: `main()` (excerpt from `gui/Observer.cpp`)

```
411 | int main()
412 | {
413 |     F1::event_dispatch(eventHandler);
414 |     Star star1 { '*' };
415 |     StarWindow window1 { 320,240,"Window 1" };
416 |     window1.end();
417 |     star1.attach(&window1);
418 |     window1.show();
419 |     StarWindow window2 { 160,120," Window 2" };
420 |     window2.end();
421 |     star1.attach(&window2);
422 |     window2.show();
423 |     StarWindow window3 { 640,480," Window 3" };
424 |     window3.end();
425 |     Star star2 { '+' };
426 |     star2.attach(&window3);
427 |     window3.show();
428 |     return F1::run();
429 | }
```

After compiling and running the program, three overlapping windows appear on the screen. They are labeled *Window 1*, *Window 2* and *Window 3*. Each of these windows can be moved to a new position so that they no longer overlap.

Changing the symbol in *Window 1* immediately affects *Window 2*. The same applies to changing the font, size and color of the symbol. It is important to note that moving the symbol in *Window 1* or *Window 2* has no effect on its position in the other window. This shows that it is a design decision which properties belong to a Subject and which are managed by an Observer. Here, the decision was made that the position of a symbol is specific to an Observer exemplar. The rationale for this decision could be that

StarWindow exemplars can be of different size and that moving a Star exemplar in a small window could cause that particular window to flash, but not the other observer window. It could also be decided that, for example, the color or size of a symbol should be specific to the Observer exemplar. This would be easy to implement. It would not be necessary to remove the corresponding setter and getter methods from the Star class. It would be completely sufficient not to call them in the StarWindow class.

The situation is different for *Window 3*, which observes *star2*. Therefore, any changes made to *star2* via the menus of *Window 3* only affect *Window 3* and have no influence on other windows.

The three windows can be closed independently of each other. As soon as the last window is closed, the application is terminated.

2.2.9.4. Afterthoughts on Observer

In principle, there is no limit for Observer exemplars and observed Subject exemplars. Multiple StarWindow exemplars can be created and attached to the same or different Star exemplars. In addition, Observer exemplars can also have different types. For example, it is conceivable to define a new subclass of Observer as a non-modal dialog class that only displays the symbol, its font and its size. In this way, it would be possible to permanently display some of the attributes of a Star exemplar without being able to change them. Of course, this possibility could be added in another dialog class.

It is also conceivable to create special dialog classes that display the existing Observer windows and the observed Subject exemplars. They could also make it possible to create new Observer and Subject exemplars, remove Subjects from Observers and add other Subjects to an Observer.

It is interesting to think about and implement all these possibilities in order to gain a deeper understanding of the *Observer* design pattern and *FLTK*. In practice, one has to limit oneself to the captured requirements, as the sheer number of possibilities is simply too high.

It is strongly recommended to run the program and to study it thoroughly. For example, what happens when a small symbol hits the edge of a window and makes it flash and a maximum size is selected for the symbol in another window that observes the same Star exemplar.

Investigating the consequences of code changes also provides valuable insights, e.g. if the windows are made resizable depending on themselves.

This section concludes the introduction to the GUI programming paradigm, which is actually more of an event-based programming paradigm. Everything described so far can also be implemented with a purely text-based user interface, with the exception of drawing lines or more complex shapes, of course.

The following sections outline topics that are not covered in this book and provide some guidance for those readers who wish to delve deeper into GUI design, particularly how to follow ergonomic design rules and create applications that provide a good user experience.

2.2.10. More Topics of GUI Programming With FLTK

There are many other topics that are relevant for programming with *FLTK* and for creating GUI applications. The topics that are not covered in this book are listed below. This list does not claim to be exhaustive.

- Every *FLTK* application has a built-in processor for the command line, which processes a range of options and can also be extended.
- In addition to the menu bar, there are other types of menus, e.g. toolbars, pop-up menus, drop-down menus and floating menus.
- Drag and drop is an essential feature of modern GUI applications that facilitates the use of a single application and possibly the interaction of different applications.
- The use of the clipboard is indispensable in standard GUI applications. It allows selected information to be copied to the clipboard or cut and pasted elsewhere.
- There are many more editing functions, e.g. for text, but also for graphic editors such as undo, redo, group or ungroup.
- Drawings can also be stored on internal buffers to be displayed at a later time. This is particularly important for applications that use animations.
- Drawing in 2D and 3D, including animated drawings, belongs to the field of graphic data processing.
- When a modal dialog is waiting to be completed, the waiting time can be used to perform other, especially time-consuming calculations. This is just one reason for using the parallel programming options that are also included in *FLTK*.
- The user can and should be supported by tool tips, bubble help and a comprehensive help system. The use of these options considerably improves the learning of an application and its occasional use.
- The *fltk-config* script offers further options and possibilities. However, its use severely limits the size, structure and number of modules an application can consist of, namely to one. Therefore, make files are required with which such applications can be built. Here, *fltk-config* can be used to retrieve platform-specific options and settings that are needed when writing these make files.
- Sometimes it may be desirable to debug a GUI application, which requires some specific tactics.
- Testing GUI applications is a separate domain. Some tests can be carried out in the form of unit tests. Testing user interaction with widgets requires special tools and considerable effort.
- The development and implementation of GUI applications can be very expensive. To cover these costs, it is often necessary to adapt an application to other languages or national regulations. Special tools and techniques can facilitate this endeavor.

- There are two opposing approaches to the development of GUI applications. One is the platform-specific design and implementation of an application. Normally, the user expects an application to look and behave exactly like other applications on a particular platform. *FLTK* also supports the development of platform-specific applications. Another approach is to design and develop applications that look and behave the same regardless of the platform they are running on. This is also supported by *FLTK*. Another approach, which combines some aspects of the opposing approaches, is to implement applications that can be built for any platform and that largely have a platform-specific look and behavior.
- Direct programming of graphical user interfaces can be very difficult as it requires an excellent understanding of the visual and behavioral implications of the code. It is also possible to largely decouple the design and creation of the GUI from the programming of a program's functionality. This is supported by so-called *GUI editors* or *GUI designers*. *FLTK* provides the *FLUID* tool for this purpose, which is an acronym for *FLtk User Interface Designer*.

2.2.11. Human-Computer Interaction and User Experience

The Section *FLTK* gives a rudimentary introduction to *FLTK*, a GUI toolkit for developing classic *desktop applications*. Desktop applications differ significantly from applications for touchscreens, smartphones and virtual or augmented reality glasses. *FLTK* is not intended for the development of non-desktop applications. In addition, non-desktop applications use different technologies, such as special sensors. They also require special design principles. It should therefore be noted that the following reading recommendations have a strong focus on desktop applications.

The use of *FLTK* for GUI programming focuses on the paradigm of GUI programming or event-based programming. It is not about programming appealing GUIs or GUIs that conform to GUI design guidelines. For example, the menus in the programming examples in this book in no way correspond to a GUI guideline. Ergonomic deficits and shortcomings have been pointed out several times. To design acceptable GUIs, the relevant GUI guidelines should be read first.

Normally, GUI toolkits provide their own user interface guidelines. For *FLTK* these are (*Human Interface Guidelines - Fast Light Toolkit (FLTK)* n.d.), which are rudimentary, incomplete, and partly outdated. There are also platform-specific guidelines, for example (*Human Interface Guidelines* n.d.) for the various *Apple* platforms, (jwmsft 2021a) (comprehensive but very outdated) and (jwmsft 2021b) for *MS Windows*, and (*GNOME Human Interface Guidelines* n.d.) for the *GNOME* desktop environment for some *Unixes* and *Linux* distributions.

A few more general resources on *human-computer interaction* (*HCI* for short) are the classic books (Norman 2013), (Norman 2013), (Shneiderman et al. 2017) and (Diaper and Stanton 2019). They are less technically oriented and also contain some relevant information on cognitive psychology.

A small selection of recent books on *interaction design* and *user experience* (UX for short) based on customer reviews are (Benyon 2017), (Rogers, Sharp, and Preece 2023) and (Pereyra 2023). UX goes beyond ergonomics, usability and human computer interaction, as it also encompasses a user's experiences, feelings and attitudes with a product, including the perception of a particular brand.

An introduction to research methods in the field of HCI is (Lazar 2017).

3. The Generic Programming Paradigm

C++ templates are a manifestation of *parametric polymorphism* (Cardelli and Wegner 1985) which is at the core of the *generic programming paradigm*. Templates, also known as *generics*, are of great importance in modern programming languages, as they make it possible to increase type safety in programs. This is also true for C++ and its standard library, which defines and uses many templates. Templates have been used several times in this book and its predecessor (U. W. Eisenecker 2022), e.g. as *template meta-functions* and *template variables* to get information about (numeric) types, as *class templates* for containers and as *function templates* for algorithms.

The following presentation focuses on the basics of templates, their definition, their usage, and some selected advanced features of templates. This book cannot provide a comprehensive introduction to C++ templates. Readers interested in more detail are referred to (Vandevoorde, N. M. Josuttis, and Gregor 2018), for example.

3.1. Function Templates

A *Function template* is a form for a function that can be instantiated with different types and even literals. It allows to convert a family of overloaded functions into a function template by converting the differences of the functions in terms of types and literals into corresponding parameters of a function template. To introduce function templates, the swapping of two variables passed as reference parameters is used.

The `swapVariables()` function in Listing 107 swaps the content of two `int` variables passed as reference parameters.

Listing 107: templates/SwapInts.cpp

```
1 // SwapInts.cpp by Ulrich Eisenecker, April 29, 2022
2
3 #include <iostream>
4 using namespace std;
5
6 void swapVariables(int& a,int& b)
7 {
8     int temp { a };
9     a = b;
10    b = temp;
11 }
12
13 int main()
14 {
15     int i1,i2;
16     cout << "i1 = ";
17     cin >> i1;
18     cout << "i2 = ";
19     cin >> i2;
20     swapVariables(i1,i2);
21     cout << "i1 = " << i1
22          << "\ni2 = " << i2
23          << endl;
24 }
```

To swap variables of a different type, appropriate functions can be provided by function overloading (Listing 108).

Listing 108: templates/SwapOverloads.cpp

```
1 // SwapOverloads.cpp by Ulrich Eisenecker, April 29, 2022
2
3 #include <iostream>
4 #include <string>
5 using namespace std;
6
7 void swapVariables(int& a,int& b)
8 {
9     int temp { a };
10    a = b;
11    b = temp;
12 }
13
14 void swapVariables(double& a,double& b)
15 {
16     double temp { a };
17     a = b;
18     b = temp;
19 }
20
```

```

21 void swapVariables(string& a,string& b)
22 {
23     string temp { a };
24     a = b;
25     b = temp;
26 }
27
28 int main()
29 {
30     int i1,i2;
31     cout << "i1 = ";
32     cin >> i1;
33     cout << "i2 = ";
34     cin >> i2;
35     swapVariables(i1,i2);
36     cout << "i1 = " << i1
37         << "\ni2 = " << i2
38         << endl;
39
40     double d1,d2;
41     cout << "d1 = ";
42     cin >> d1;
43     cout << "d2 = ";
44     cin >> d2;
45     swapVariables(d1,d2);
46     cout << "d1 = " << d1
47         << "\nd2 = " << d2
48         << endl;
49
50     if(cin.peek() == '\n')
51     {
52         cin.get();
53     }
54     string s1,s2;
55     cout << "s1 = ";
56     getline(cin,s1);
57     cout << "s2 = ";
58     getline(cin,s2);
59     swapVariables(s1,s2);
60     cout << "s1 = " << s1
61         << "\ns2 = " << s2
62         << endl;
63 }

```

This is nice, because it is easy to see that all these functions have the same purpose, namely to swap variables of a certain type. Of course, it would have been possible to encode the type of the variables to be swapped in the function names, e.g. `swapIntVariables()`, `swapDoubleVariables()`, and so on. However, the longer names may be less readable and the close relationship between these functions and their purpose less clear. Therefore, overloading is preferred here. An interesting detail in lines 50–53 deserves an explanation. When reading input with the `>>` operator, all characters that cannot be extracted remain in the input stream. This is why the newline character,

which terminates the previous input of a double value, is still in the input stream, here `cin`. In line 56, a so-called *line input* is made with `std::getline()`. This immediately returns an empty string, since `getline()` consumes the remaining newline character. To ensure that the code works as intended, `cin.peek()` is used to check whether a newline character is present. If so, it is consumed by `cin.get()` and discarded.

This implementation of entering a `std::string` is not foolproof. If more characters are entered after the preceding value before the *Enter* key is pressed, the input is garbled. Alternatively, the `istream::ignore()` member function could be called with `'\n'` as the terminating character to resolve this issue. However, this would always discard any additional input after the preceding value, which could also be undesirable behavior.

Obviously, the overloaded functions in Listing 108 have a lot in common and exhibit only minor differences, i.e. variability. But every time variables of a new type need to be swapped, a new overloaded function must be implemented. This is tedious and potentially error-prone. In computer science, and especially in software development, there is always an effort to get rid of repetitive tasks.

Function templates provide a solution to this problem by making the type of the reference parameters and the auxiliary variable a *template parameter* of a function template. In Listing 109, the function definitions from Listing 108 are repeated, with the differences now highlighted.

Listing 109: templates/SwapOverloads.cpp (excerpt, differences highlighted)

```
1 void swapVariables(int& a, int& b)
2 {
3     int temp { a };
4     a = b;
5     b = temp;
6 }
7
8 void swapVariables(double& a, double& b)
9 {
10    double temp { a };
11    a = b;
12    b = temp;
13 }
14
15 void swapVariables(string& a, string& b)
16 {
17    string temp { a };
18    a = b;
19    b = temp;
20 }
```

Listing 110 shows the `swapVariables<>()` function template, which eliminates all these differences by providing a single template parameter. To indicate that an identifier refers to a template, a pair of angle brackets is appended. The rest of the program re-

mains unchanged compared to Listing 108. The `swapVariables()` functions are called like ordinary functions. This is due to the *template argument deduction* (TAD for short), which is explained in Section *Template Argument Deduction*.

Listing 110: templates/SwapTemplate.cpp

```

1 // SwapTemplate.cpp by Ulrich Eisenecker, April 29, 2022
2
3 #include <iostream>
4 #include <string>
5 using namespace std;
6
7 template <typename T>
8 void swapVariables(T& a,T& b)
9 {
10     T temp { a };
11     a = b;
12     b = temp;
13 }
14
15 int main()
16 {
17     int i1,i2;
18     cout << "i1 = ";
19     cin >> i1;
20     cout << "i2 = ";
21     cin >> i2;
22     swapVariables(i1,i2);
23     cout << "i1 = " << i1
24         << "\ni2 = " << i2
25         << endl;
26
27     double d1,d2;
28     cout << "d1 = ";
29     cin >> d1;
30     cout << "d2 = ";
31     cin >> d2;
32     swapVariables(d1,d2);
33     cout << "d1 = " << d1
34         << "\nd2 = " << d2
35         << endl;
36
37     if(cin.peek() == '\n')
38     {
39         cin.get();
40     }
41     string s1,s2;
42     cout << "s1 = ";
43     getline(cin,s1);
44     cout << "s2 = ";
45     getline(cin,s2);
46     swapVariables(s1,s2);
47     cout << "s1 = " << s1
48         << "\ns2 = " << s2

```

```
49 |         << endl;  
50 |     }
```

It should be mentioned that the C++ standard library contains the function template `std::swap<>()`, which is defined in the headers `<algorithm>`, `<utility>` and `<string_view>`. It is therefore not necessary to define a corresponding function template for swapping variables oneself. `swapVariables<>()` is only introduced for didactic reasons.

3.1.1. Syntax of Function Templates

First, the syntax of a template definition is explained. A template definition starts with the keyword `template`, followed by a pair of angle brackets. One or more template parameters are specified within these brackets. Type template parameters are specified with the keyword `class` or `typename`. There is no difference between using `class` or `typename`. It is a convention to use `class` to indicate that a structured type, i.e. a class, should be used for instantiation. The use of `typename` expresses that any type can be used as a parameter, either a class or a simple type, i.e. an elementary type. `class` or `typename` is followed by the name of the template parameter, which is used to refer to the type parameter when needed.

Very often a single type parameter is simply referred to as `T`. In fact, any legal identifier can be used that is not a keyword or defined elsewhere in the same scope. If there are multiple type template parameters, they must be separated by commas. Each type template parameter must be preceded by `class` or `typename`. There are also *constant template parameters* (also known as *non-type template parameters*) and *template template parameters*. Constant template parameters will be introduced in the Section *Constant Template Parameters*, template template parameters in Section *Template Template Parameters*. The normal function definition follows the declaration of the template parameters.

Wherever appropriate, a type template parameter is used as a placeholder for a concrete type in the function definition. In `swapVariables<>()`, the type of the two reference parameters is denoted by `T`, as is the type of the auxiliary variable `temp`.

3.1.2. Template Argument Deduction

The compiler automatically instantiates the function template `swapVariables<>()`, taking into account the function parameters passed. The underlying mechanism for this is the *template argument deduction* (abbr. *TAD*). To understand this properly, the use of a function template must also be seen in the context of normal (overloaded) functions.

When the compiler sees a use of `swapVariables()`, it checks whether such a function with the given types, e.g. `swapVariables(int& a, int& b)`, exists. If so, the compiler generates a call to that function, as a matching function always takes precedence over the instantiation of a function template, unless special measures are taken. If there is

no suitable function, the compiler checks whether an already instantiated function template can be used. If this is not the case, the compiler tries to instantiate a suitable function template.

In the case of `swapVariables<>()`, this is possible because this function template declares exactly one type parameter, while the expression under consideration, `swapVariables(int&,int&)`, uses exactly one type, namely `int`. The decisive factor here is not the number of function parameters, but only the number of different types that are used. This allows the use of `int` for the type template parameter `T` and automatically creates a valid instantiation of the function template `swapVariables<>(T&,T&)` for `int`.

Now a case is considered where the template argument deduction fails. If there is an expression that contains the function call `swapVariables(int&,double&)`, the compiler first looks for a definition of a function with exactly this signature. If it does not find one, it checks whether the function template `swapVariables<>()` can be instantiated. In fact, `swapVariables(int&,double&)` contains parameters with two different types, `int` and `double`. If there is no corresponding function template that specifies two parameters of different type, the compiler checks whether `swapVariables<>(T&,T&)` can be instantiated. However, this is not possible as the function template only specifies one type parameter, `T`. For the automatic template parameter deduction to work, a function template would be required that specifies two type template parameters, namely template `<class T1,class T2> void swapVariables(T1&,T2&)`. The compiler would then deduce `int` for `T1` and `double` for `T2`.

3.1.3. Implicit Requirements for a Template Parameter

A requirement for a type defines an operation that must be available for that type. For example, a class must have a public destructor, otherwise an exemplar of that class cannot be destroyed. Another example of requirements are arithmetic operators that must be defined for an arithmetic type, such as the `+` operator or the `--` operator. Since these requirements arise from the use of the template argument, they are referred to as *implicit requirements for a template parameter*.

The `swapVariables<>()` function template in Listing 110 implicitly defines requirements for the type that is used to instantiate the template. These are now explained in detail. In line 10, the copy constructor is called to initialize `temp` with `a`. Therefore, `T` must have a public copy constructor. In lines 11 and 12, the assignment operator is called, so it must also be public for `T`. Not explicitly visible is the call of the destructor of `T` when exiting the function in line 13.

If a type lacks one of these public member functions, the compiler reports an error when it attempts to instantiate the template. Listing 111 illustrates this by defining three different classes, each missing exactly one of these member functions. Then `swapVariables<>()` is called with exemplars of these types to cause the compiler to report the appropriate errors.

Listing 111: templates/UnmetRequirements.cpp

```
1 // UnmetRequirements.cpp by Ulrich Eisenecker, April 29, 2022
2 // Warning: Program does not compile!
3
4 #include <iostream>
5 using namespace std;
6
7 template <typename T>
8 void swapVariables(T& a,T& b)
9 {
10     T temp { a };
11     a = b;
12     b = temp;
13 }
14
15 class NoCopyConstructor
16 {
17     public:
18         NoCopyConstructor()
19         {}
20         NoCopyConstructor(const NoCopyConstructor&) = delete;
21 };
22
23 class NoAssignmentOperator
24 {
25     public:
26         NoAssignmentOperator& operator=(const NoAssignmentOperator&) = delete;
27 };
28
29 class NoDestructor
30 {
31     public:
32         ~NoDestructor() = delete;
33 };
34
35 int main()
36 {
37     {
38         NoCopyConstructor a,b;
39         swapVariables(a,b);
40     }
41
42     {
43         NoAssignmentOperator a,b;
44         swapVariables(a,b);
45     }
46
47     {
48         NoDestructor* a = new NoDestructor();
49         NoDestructor* b = new NoDestructor();
50         swapVariables(*a,*b);
51     }
```

52 | }

Since C++ 11, a member function is made inaccessible by the special syntax `= delete;` (see Section *Default and Delete*). By appending `= delete;` to the declaration of a member function, this member function is made non-existent. It may neither be defined nor used in the following. Prior to C++ 11, the usual idiom to make a member function inaccessible was to declare it as `private`, which is not shown here.

When the program is compiled, the compiler reports at least three errors, usually with a variety of context information. Since these error messages can vary from compiler to compiler, no specific example is given here. Sometimes it is difficult for a programmer to understand these error messages. It is therefore possible to make the compiler output more meaningful error messages. Listing 112 shows how this can be achieved using template meta-programming functions. To use them, the header `<type_traits>` must be included. It should be noted that the implementation of the checks is more verbose than necessary, as template meta-functions are called. Equally valid, but shorter, would be the use of template variables that are automatically instantiated by calling the corresponding template meta functions, e.g. `is_destructible_v<T>` instead of `is_destructible<T>::value`. In addition, the explicit comparison with `true` is superfluous, because if, for example, `is_destructible_v<T>` is evaluated to `true`, this value is already `true`.

Listing 112: templates/SwapChecked.cpp (excerpt)

```

5  #include <type_traits>
6  using namespace std;
7
8  template <typename T>
9  void swapVariables(T& a, T& b)
10 {
11     static_assert(is_destructible<T>::value == true,
12                  "No destructor available for T");
13     static_assert(is_copy_constructible<T>::value == true,
14                  "No copy constructor available for T");
15     static_assert(is_assignable<T&, T>::value == true,
16                  "No assignment operator available for T");
17     T temp { a };
18     a = b;
19     b = temp;
20 }
```

The `static_assert` declaration was introduced in Section *final*. The three `static_assert` statements in Listing 112 check whether the type `T` meets the requirements of the function template and, if not, issue corresponding error messages. The great advantage of using the error message string as the second parameter is that after compilation, the output generated by the compiler can be searched for the occurrence of this string.

Line 12 contains a very important detail. Here it is essential to designate the first template parameter as `T&`. Using only `T` would mean that an attempt is made to assign to a newly created object, i.e. a temporary object. This is not allowed. Therefore, the assignment operator must principally take a reference as the first operand. In this way, the previously implicit requirements for type template parameters are converted into explicit requirements by checking them with `static_assert`.

The consequences of using `static_assert` can be easily demonstrated in a terminal window. On a *Unix*-based operating system, a terminal window is opened at the directory containing the file *SwapChecked.cpp*, here *templates*. The compiler is then invoked and its output is redirected to the file *compiler_output.txt*.

The attempt to redirect the output of the compiler, as was previously done with the output of normal programs, fails. This means that the command `g++ -std=c++20 SwapChecked.cpp > compiler_output.txt` creates an empty file named *compiler_output.txt*. But all messages of the compiler continue to appear in the terminal window. The reason for this is that the compiler sends its output to the *standard error stream* called *stderr*. However, with `>` as the redirection operator, the output is always redirected to the standard output stream *stdout*.

Internally, the operating system identifies these streams by numbers: *1* refers to *stdout* and *2* refers to *stderr*. If the redirection operator is preceded by a number, this number identifies the stream to be redirected. Thus, *program 1 > file* redirects the output of the program to *stdout* to *file*, and *program 2 > file* redirects the output of the program to *stderr* to *file*. Consequently, *program 1 > file 2 > file* would redirect both program outputs, *stdout* and *stderr*, to *file*. Fortunately, this can be abbreviated by `&>`, which combines both streams to be redirected. Thus, *program &> file* sends the program output to *stdout* and *stderr* to *file*.

Therefore, the complete command line that must be executed in the terminal is `g++ -std=c++20 SwapChecked.cpp &> compiler_output.txt`. The *compiler_output.txt* file then exists and it is *not* empty. Now it is possible to search this file for the occurrence of a specific string, for example with *grep* (acronym for *get regular expression*), which also searches for a simple string. Entering the command `grep "No destructor available for T" compiler_output.txt` results in the output `error: static_assert failed due to requirement 'is_destructible<NoDestructor>::value == true' "No destructor available for T"`. This is the complete line containing the searched string. If *grep* cannot find the string it is looking for, it outputs an empty line. Comprehensive information on redirecting output to *stdout* and *stderr* or on using *grep* can be found, for example, in (Shotts 2012).

Figure 42 demonstrates the abbreviated dialog for the above commands in a terminal window.

Figure 42: Processing compiler output

```

user@computer templates % g++ -std=c++20 SwapChecked.cpp > compiler_output.txt
SwapChecked.cpp:12:4: error: static_assert failed due to requirement
'is_copy_constructible<NoCopyConstructor>::value == true' "No copy constructor
available for T"
    static_assert(is_copy_constructible<T>::value == true,
    ^
    ~~~~~
...
8 errors generated.
user@computer templates % ls -l compiler_output.txt
-rw-r--r--  1 ulrich  staff   0  2 Mai 19:03 compiler_output.txt
user@computer templates % g++ -std=c++20 SwapChecked.cpp &> compiler_output.txt
user@computer templates % ls -l compiler_output.txt
-rw-r--r--  1 ulrich  staff 2657  2 Mai 19:04 compiler_output.txt
user@computer templates % grep "No destructor available for T" compiler_output.txt
SwapChecked.cpp:10:4: error: static_assert failed due to requirement
'is_destructible<NoDestructor>::value == true' "No destructor available for T"
user@computer templates %

```

The use of `static_assert` increases the number of error messages issued by the compiler. Nevertheless, capturing the compiler messages and their post processing has several uses, for example, in tuning of build systems and build pipelines, and in the automatic correction of programming tasks.

Alternatively, the type template parameter requirements can be checked using so-called *concepts*, which is explained in Section *Concepts*.

3.1.4. Explicit Template Arguments

There are cases in which the deduction of template arguments is not possible. Then template arguments that cannot be deduced automatically must be specified explicitly.

A closer look at Listing 110 shows that it contains more code that could be turned into a function template. Lines 17 – 25 and 27 – 35 are almost identical, except for the types, variable names and output strings involved. Lines 37 – 49 are different, they contain statements that are not present in the previous lines, and they also contain other statements.

First, a function template is created to replace lines 17 – 35. It is called `demonstrateSwap<>()` and specifies a type template parameter called `T`. It is noteworthy that it has no function parameter of type `T`. Its only function parameter is a character passed by value, which is used to create the prompt. When calling this function template, template argument deduction does not work because the function template does not specify a corresponding argument! Therefore, the template argument must be specified explicitly. This is done by enclosing the type parameter in angle brackets immediately after the function name, for example, `demonstrateSwap<int>('i');`.

The code in `main()` that inputs two strings, swaps them and outputs the result, remains unchanged in the program. Listing 113 only shows a snippet with the function template `demonstrateSwap<>()` and the explicit specification of its template argument when called for `int` and `double`.

If the signature of a function template does not use a template parameter, or if a template parameter is only used to specify the return type, the template argument for this template parameter must be specified explicitly.

Listing 113: `templates/SwapMoreTemplates.cpp` (excerpt)

```
1 // SwapMoreTemplates.cpp by Ulrich Eisenecker, May 3, 2022
15 template <typename T>
16 void demonstrateSwap(char t)
17 {
18     T v1,v2;
19     cout << t << "1 = ";
20     cin >> v1;
21     cout << t << "2 = ";
22     cin >> v2;
23     swapVariables(v1,v2);
24     cout << t << "1 = " << v1
25         << "\n" << t << "2 = " << v2
26         << endl;
27 }
31 demonstrateSwap<int>('i');
32 demonstrateSwap<double>('d');
33
```

3.1.5. Full Specialization of Function Template

Sometimes a function template needs to be instantiated differently for certain types or literals. The full *specialization of function templates* fulfills this need. Swapping variables is still used as an example. Furthermore, the implementation of a point for future changes, a so-called *hook*, is presented and discussed.

Listing 113 still contains the code from Listing 110 for swapping two `std::string` variables. The question is: can this code also be converted into a function template? Yes, it can, using the full specialization of the function template `demonstrateSwap<>()`, as shown in Listing 114. The syntax for full specialization of a template is to use an empty list for specifying the template parameters (line 15) and to insert an additional list with concrete types for the template parameters after the function template name (line 16). The number of template parameters must be exactly the same as that of the non-specialized function template, which must also come before the specialized template.

Listing 114: templates/SwapSpecialized.cpp (excerpt)

```

1 // SwapSpecialized.cpp by Ulrich Eisenecker, May 3, 2022
15 template <typename T>
16 void demonstrateSwap(char t)
17 {
27 }
28
29 template <>
30 void demonstrateSwap<string>(char t)
31 {
32     using T = string;
33     T v1,v2;
34     if(cin.peek() == '\n')
35     {
36         cin.get();
37     }
38     cout << t << "1 = ";
39     getline(cin,v1);
40     cout << t << "2 = ";
41     getline(cin,v2);
42     swapVariables(v1,v2);
43     cout << t << "1 = " << v1
44         << "\n" << t << "2 = " << v2
45         << endl;
46 }
47
48 int main()
49 {
52     demonstrateSwap<string>('s');
53 }

```

Of course, it would be possible to replace every occurrence of `T` in `demonstrateSwap<string>()` with `string`. However, this can be avoided by using an alias declaration. In line 32, `T` is declared as an alias for the type `string` with the `using` keyword. The alias declaration with `using` has been already introduced in (U. W. Eisenecker 2022). From now on, `T` is an alias for `string` in the active scope and does not need to be changed. In `main()`, the function template is called with its template argument explicitly specified as `string`, just as it was previously called for `int` and `double`.

While this allows the template instantiated for `string` to be called consistently with the other function templates, it is not a real improvement in code quality. It partially duplicates code and introduces another function into the program. Fortunately, it is possible to come up with a more elegant solution by using full specialization of templates.

Analysis of the code portions of Listing 110 relating to the swap demonstration suggests a way to refactor the code with respect to the input of values. The function template `inputValue<>()` is introduced that inputs a value in the same way as is normally done

with a text stream, where the input stops as soon as a valid input has been extracted and a space, new line or end of file character occurs. All other characters following the extracted input remain in the input stream. This also applies to the newline character with which users normally end their input. This will now be changed.

After `cin >> t;` the statement `cin.ignore(numeric_limits<streamsize>::max(), '\n');` is executed. It extracts everything contained in the input stream up to and including the newline character and discards it. The first argument of the call to the input stream member function `ignore()` specifies the maximum number of characters that can be extracted. This argument is given as `numeric_limits<streamsize>::max()`, which corresponds to the maximum length of a file. The second argument specifies the character that ends the extraction, here `'\n'`. If it is found, it is also extracted and the extraction is terminated. Both arguments specify the conditions for terminating the extraction. How many characters are extracted and discarded depends on which condition is fulfilled first. In this way, any additional input made after the expected input is discarded. If a user completes a particular input immediately by pressing the *Enter* key, no difference can be detected. However, it is no longer possible to make further subsequent entries. This general function template is completely specialized for the type `std::string`. In this case, a complete input line up to and including the newline character is extracted with `getline(cin, t);` from the input stream and stored in the string variable `t`.

Listing 115 shows the implementation of the new function template `inputValue<>()` and its specialization for `string`.

Listing 115: templates/SwapMoreSpecialized.cpp (excerpt)

```
1 // SwapMoreSpecialized.cpp by Ulrich Eisenecker, June 4, 2025
16 template <typename T>
17 T inputValue()
18 {
19     T t;
20     cin >> t;
21     cin.ignore(numeric_limits<streamsize>::max(), '\n');
22     return t;
23 }
24
25 template <>
26 string inputValue<string>()
27 {
28     string t;
29     getline(cin, t);
30     return t;
31 }
32
33 template <typename T>
34 void demonstrateSwap(char t)
35 {
36     T v1, v2;
```

```

37     cout << t << "1 = ";
38     v1 = inputValue<T>();
39     cout << t << "2 = ";
40     v2 = inputValue<T>();
41     swapVariables(v1,v2);
42     cout << t << "1 = " << v1
43         << "\n" << t << "2 = " << v2
44         << endl;
45 }
46

```

Last but not least, the implementation of `demonstrateSwap<>()` must be rewritten to use the `inputValue<>()` function template. The full specialization of `demonstrateSwap<>()` for `string` is then no longer required, because `demonstrateSwap<>()` instantiates itself with `inputValue<string>()` when instantiated with `string` as in `demonstrateSwap<string>('s')`.

The function template `inputValue<>()` and its use in `demonstrateSwap<>()` is an example of a so-called *hook* in programming. A *hook* is a point where variability occurs or can occur and where extensions can be added later. There are also other programming concepts for providing hooks, which will be introduced later.

It is important to note that it is difficult to implement `demonstrateSwap<>()` in this way if there is no previous attempt to demonstrate the swapping of two `string` variables. This is an example of the so-called *pre-planning problem*, which means that it is at least very difficult, if not impossible, to plan for unknown uses or events. Obviously, this problem occurs in software development in general, not only in programming. ***It is strongly discouraged to set a hook based only on vague assumptions and feelings. If it is not needed later, it has a negative impact on code quality because it makes code harder to understand and increases the number of entities in code as well as the code size. It is much better to refactor existing code if it turns out that a new requirement must be met that was not known before.*** Of course, refactoring is only possible if the source code is available.

Despite the intellectual elegance of the solution shown in Listing 115, it must be noted that despite removing one function template, two more function templates are added. Also, the implementation of `demonstrateSwap<>()` becomes more difficult to read because the meaning of the call to `inputValue<>()` must be understood. Considering all this, this generalized solution would not be sufficiently justified if only one case, here `std::string`, needs to be handled specifically.

If a type template parameter is used to specify a parameter in the signature of a function template, it is also possible to define an overloaded function for a specific type. This function is always automatically selected by the compiler. This applies even if a fully specialized function template is also available for this type. To call the fully specialized function template in this case, it must be called as an explicitly instantiated function template. For

- `template <class T> void someFunction(T t);` i.e. the original template,
- `template <> void someFunction<int>(int t);` as the full specialization of the original template for `int`,
- `void someFunction(int t)` as the overloaded function,
- `someFunction(5)` would result in a call to `someFunction(int)`,
- `someFunction<int>(5)` would result in a call to `someFunction<int>(int)`.

Function templates can also be overloaded, which is explained in Section *Overloading of Function Templates*.

A valid alternative to fully specializing the `demonstrateSwap<>()` function template would be to define an overloaded `demonstrateSwap()` function that internally reads, swaps, and outputs `std::string` variables. This function would be called as `demonstrateSwap('s')` in `main()`. Its call would therefore look slightly different from the previous calls to the function templates. Also, only a single overloaded `demonstrateSwap()` function can be provided, since it has no parameters involved in the overload resolution.

3.1.6. Constant Template Parameters

Functions can differ not only in terms of types. There can also be differences in non-type entities, e.g. literals. *Constant template parameters* (or *non-type template parameters*) allow dealing with them in (function) templates.

The previously presented function template `demonstrateSwap<>()` specifies a character passed by value as the only function parameter. In fact, it is also possible to turn this normal function parameter into a template, since constant template parameters are also legal template parameters. Examples of constant template parameters are integral and floating point literals. In the following, integral parameters are used to introduce constant template parameters. Listing 116 shows how to do this.

Listing 116: `templates/SwapWithIntegralTemplateParameter.cpp` (excerpt)

```
1 // SwapWithIntegralTemplateParameter.cpp by Ulrich Eisenecker, June 6, 2025
33 template <typename T, char t>
34 void demonstrateSwap()
35 {
36     T v1, v2;
37     cout << t << "1 = ";
38     v1 = inputValue<T>();
39     cout << t << "2 = ";
40     v2 = inputValue<T>();
41     swapVariables(v1, v2);
42     cout << t << "1 = " << v1
43         << "\n" << t << "2 = " << v2
44         << endl;
45 }
46
```

```

47 | int main()
48 | {
49 |     demonstrateSwap<int, 'i'>();
50 |     demonstrateSwap<double, 'd'>();
51 |     demonstrateSwap<string, 's'>();
52 | }

```

In line 33, `char t` is specified as the second template parameter of type `char`. It is separated from the first parameter `typename T` by a comma. Consequently, it is removed as an ordinary function parameter in line 34, i.e. the function parameter list of `demonstrateSwap<>()` is now empty. The following definition of the function template remains unchanged.

In `main()`, the calls to `demonstrateSwap<>()` must be changed accordingly. Now a character must be specified as the second parameter in angle brackets, while the character between the parentheses of the function call must be removed.

There is an important difference between the two versions of `demonstrateSwap<>()`. In the first version there is only one instantiation of this function template per type. In the second version, the number of instances also depends on the second parameter. This can be illustrated using the following example: `demonstrateSwap<int>('i')` and `demonstrateSwap<int>('j')` both refer to the same instantiation of `demonstrateSwap<int>()`, whereas `demonstrateSwap<int, 'i'>()` and `demonstrateSwap<int, 'j'>()` result in two different instantiations.

However, the conversion of the function argument into a constant (i.e. integral) template parameter shown here is not useful. Its only purpose is to introduce constant template parameters. A better use case for constant template parameters will be presented later.

3.1.7. Overloading of Function Templates

Just as it is possible to overload (ordinary) functions, it is also possible to *overload function templates*. This means that overloaded function templates can be defined that differ in the number of template parameters, function parameters or in the way the parameters are passed.

The program in Listing 117 swaps the pointers, more precisely the pointer variables `pX` and `pY`. The output of the values of the variables to which they point shows that the variables as such have not been swapped. In fact, the function template does what it should do, namely swap the two pointer variables, which is not explicitly demonstrated here.

Listing 117: templates/SwapPointers.cpp

```
1 // SwapPointers.cpp by Ulrich Eisenecker, May 4, 2022
2
3 #include <iostream>
4 #include <string>
5 using namespace std;
6
7 template <typename T>
8 void swapVariables(T& a,T& b)
9 {
10     T temp { a };
11     a = b;
12     b = temp;
13 }
14
15 int main()
16 {
17     auto x { 42 },
18         y { 13 };
19     auto pX { &x },
20         pY { &y };
21     swapVariables(pX,pY);
22     cout << x << '\n'<< y
23          << endl;
24 }
```

Now, one could argue that `swapVariables<>()` should actually swap the variables to which the pointers point when it is called for pointers. This can be achieved by inserting an overloaded function template, which is shown in Listing 118, under the function template `swapVariables<>()`.

Listing 118: templates/SwapOverloaded.cpp (excerpt)

```
15 template <typename T>
16 void swapVariables(T* a,T* b)
17 {
18     T temp { *a };
19     *a = *b;
20     *b = temp;
21 }
```

The call to `swapVariables(pX,pY)` then swaps the values of the variables `x` and `y`, but not the pointers `xP` and `yY`.

Furthermore, it would be possible to provide additional overloaded function templates, e.g. for performing a ring swap between three function parameters. Overloading function templates requires either a different number of template parameters, function parameters or a different way of passing function parameters. The overloading of function templates should not be confused with the specialization of function templates.

3.2. Class Templates

Just as a function template is a form for functions that can be instantiated with different template parameters, a *class template* is a form for classes that can be instantiated with different template parameters. A *ring buffer*, which is a new container type, serves as an example of a class template.

A *circular buffer*, also known as *ring buffer* (Circular Buffer 2022), is a data structure of fixed-size that is initially empty. When its capacity is reached, adding a new element overwrites the oldest element stored in the buffer. Figure 43 illustrates this behavior for a ring buffer with a capacity of 5. A new element is added in each row. The elements are named with letters, starting with A. The last element added is K. The indices are numbered from 0 to 4. After adding five elements to the ring buffer, the oldest element is overwritten with the last element added.

Figure 43: Adding the letters A to K to a ring buffer with the capacity 5

	Index 0	Index 1	Index 2	Index 3	Index 4
1. Empty buffer	-	-	-	-	-
2. Insert A	A	-	-	-	-
3. Insert B	A	B	-	-	-
4. Insert C	A	B	C	-	-
5. Insert D	A	B	C	D	-
6. Insert E	A	B	C	D	E
7. Insert F	F	B	C	D	E
8. Insert G	F	G	C	D	E
9. Insert H	F	G	H	D	E
10. Insert I	F	G	H	I	E
11. Insert J	F	G	H	I	J
12. Insert K	K	G	H	I	J

When reading from a ring buffer, always the oldest element is returned.

The crucial question now is: how can the desired behavior for putting into and getting from a ring buffer be achieved?

It is relatively simple to describe how to put an element into a ring buffer. This is done by introducing a *write index* that is initialized to 0. A new element is then put into an array at the position of the write index. The write index is then incremented by one. In this way, the capacity of the ring buffer can be reached. To prevent an overflow, a new value is assigned to the write index, which results from write index modulo capacity.

Getting an element from the ring buffer is more complicated. First, another variable is introduced that counts the number of elements. When a new element is put into the ring buffer, the number of elements is only incremented by one if it is smaller than the capacity of the buffer. It is obvious that a buffer with a capacity of 5, for example, cannot store more than 5 elements.

The next steps are a bit more difficult.

Depending on the number of elements and the actual value of the write index, the index position of the element to be extracted can be less than, equal or greater than the write index, but it must lie in the interval $[0 .. \text{capacity} - 1]$. If the read index were calculated without considering the write index, it would be equal to the capacity of the ring buffer minus the number of elements. For example, if there are 5 elements stored in a ring buffer with a capacity of 5, the read index is $5 - 5$. The number of elements must then be decreased by one, so that the next read index is $5 - 4$, and so on.

Now, the write index should be taken into account. To do this, the write index is added to the difference between the capacity and the number of elements. This can result in a value that is higher than the capacity. The result modulo capacity compensates for this. The complete formula for calculating the read index is therefore: *read index* = (*capacity* – *number of elements* + *write index*) % *capacity*. After calculating the read index, the number of elements must be decremented by one before returning the element at the position of the read index.

The RingBuffer class shown in Listing 119 fully complies with the above description. A *classic C-array* is used to store the elements, which is briefly explained here. A classic C-array is defined by specifying the type of the elements, the name and the size in square brackets. The size of a C-array should be greater than 0. The elements of an array are indexed with the index operator `[]`. The indices go from 0 to the number of elements minus one. The name of the array is also a constant pointer to the first element. Therefore, the expressions `*array_name` and `array_name[0]` both refer to the first element.

Listing 119: templates/BufferChar5.cpp (excerpt)

```
7 class RingBuffer
8 {
9     public:
10         void put(char element)
11         {
12             cout << "P" << m_writeIndex << " N" << m_noOfElements << endl;
13             if (m_noOfElements < 5)
14             {
15                 ++m_noOfElements;
16             }
17             m_elements[m_writeIndex++] = element;
18             m_writeIndex %= 5;
19         }
20         char get()
21         {
22             if (m_noOfElements == 0)
23             {
24                 throw out_of_range { "RingBuffer::get() - empty buffer" };
25             }
26             int readIndex { (5 - m_noOfElements + m_writeIndex) % 5 };
27             --m_noOfElements;
28             return m_elements[readIndex];
```

```

29     }
30 private:
31     int m_writeIndex { 0 },
32         m_noOfElements { 0 };
33     char m_elements[5];
34 };

```

The member function `Buffer::get()` first checks whether `m_noOfElements` is 0. If so, it then throws an exception of the type `std::out_of_range()` (see Section *Exceptions* for more information on exceptions).

To store elements of a different type, another ring buffer class had to be implemented. A ring buffer is therefore a good candidate to be implemented as a class template. The syntax is analogous to that of a function template. As Listing 120 shows, the template definition begins with the `template` keyword, followed by the template parameters in angle brackets. As with function templates, a type parameter is specified with either `class` or `typename`. As already explained, the use of `typename` indicates that both structured and simple types can be used for the template parameter, while `class` means that only structured types should be used.

Wherever the type `char` was used in Listing 119, it is replaced by the template parameter named `T` in Listing 120.

Listing 120: `templates/BufferTemplateT.cpp` (excerpt)

```

7  template <typename T>
8  class RingBuffer
9  {
10 public:
11     void put(T element)
12     {
13         if (m_noOfElements < 5)
14         {
15             ++m_noOfElements;
16         }
17         m_elements[m_writeIndex++] = element;
18         m_writeIndex %= 5;
19     }
20     T get()
21     {
22         if (m_noOfElements == 0)
23         {
24             throw out_of_range { "RingBuffer<>::get() - empty buffer" };
25         }
26         int readIndex { (5 - m_noOfElements + m_writeIndex) % 5 };
27         --m_noOfElements;
28         return m_elements[readIndex];
29     }
30 private:
31     int m_writeIndex { 0 },
32         m_noOfElements { 0 };

```

```
33     T m_elements[5];
34 };
```

Interestingly, template argument deduction can also be applied to a class template if its use indicates the type with which it is to be instantiated. In the following, only one way to provide the necessary information for deducing the type of the type template parameter is presented. The full guidelines for template argument deduction for class templates are quite complex and detailed, see for example (*Class Template Argument Deduction (CTAD) (since C++17)* - *Cppreference.Com* n.d.). To enable template arguments, four constructors are added to the class template `RingBuffer<>`, as shown in Listing 121.

Listing 121: templates/ClassTemplateWithTAD.cpp (excerpt)

```
7  template <typename T>
8  class RingBuffer
9  {
10     public:
11         RingBuffer()
12         {}
13         RingBuffer(T t)
14         {
15             put(t);
16         }
17         RingBuffer(T t1,T t2)
18         {
19             put(t1);
20             put(t2);
21         }
22         RingBuffer(T t1,T t2,T t3)
23         {
24             put(t1);
25             put(t2);
26             put(t3);
27         }
28         void put(T element)
29         {
30             if (m_noOfElements < 5)
31             {
32                 ++m_noOfElements;
33             }
34             m_elements[m_writeIndex++] = element;
35             m_writeIndex %= 5;
36         }
37         T get()
38         {
39             if (m_noOfElements == 0)
40             {
41                 throw out_of_range { "RingBuffer<>::get() - empty buffer" };
42             }
43             int readIndex { (5 - m_noOfElements + m_writeIndex) % 5 };
44             --m_noOfElements;
45             return m_elements[readIndex];
```

```

46     }
47 private:
48     int m_writeIndex { 0 },
49         m_noOfElements { 0 };
50     T m_elements[5];
51 };
52
53 int main()
54 {
55     RingBuffer rb1 { 'X' };
56     RingBuffer rb2 { 5.0, 1.5 };
57     RingBuffer rb3 { "Ring"s, "Buffer"s, "std::string"s };
61 }

```

The three constructors, which take one, two or three arguments of type `T`, provide the necessary hints for deducing the arguments of the template. The `RingBuffer<>` template can now be instantiated by calling one of these constructors with values of the same type, as shown in 55–57. Calling a constructor with more than one argument with parameters of different types will result in a compilation error.

The standard constructor, which takes no arguments, would be automatically generated by the compiler if no other constructor is available. Since this is not the case here, it must be specified explicitly (see Section *Constructors and List Initialization*). Otherwise, it would no longer be possible to create an empty ring buffer.

The implementation of `RingBuffer<>` can be simplified by defining a constructor that takes an *initializer list*, which is explained in Section *Initializer List*.

The next step is to turn the capacity of the class template `RingBuffer<>` into an integral template parameter, as is done in Listing 122. When the class template is instantiated, a value for the integral template parameter must now also be specified. For example, `RingBuffer<char,5>` corresponds to the `RingBuffer` class in Listing 119.

Listing 122: templates/BufferTemplateTN.cpp (excerpt)

```

7  template <typename T,int CAPACITY>
8  class RingBuffer
9  {
10 public:
11     void put(T element)
12     {
13         if (m_noOfElements < CAPACITY)
14         {
15             ++m_noOfElements;
16         }
17         m_elements[m_writeIndex++] = element;
18         m_writeIndex %= CAPACITY;
19     }
20     T get()
21     {
22         if (m_noOfElements == 0)

```

```
23     {
24         throw out_of_range { "RingBuffer<>::get() - empty buffer" };
25     }
26     int readIndex { (CAPACITY - m_noOfElements
27                     + m_writeIndex) % CAPACITY };
28     --m_noOfElements;
29     return m_elements[readIndex];
30 }
31 private:
32     int m_writeIndex { 0 },
33         m_noOfElements { 0 };
34     T m_elements[CAPACITY];
35 };
```

It is not a problem that the integral template parameter is specified as `int`. Using a negative value when instantiating the template leads to an error message from the compiler, as an array cannot have a negative size (as with *clang++*) or an attempt is made to declare an array with too many elements (as with *g++*). To avoid this, the integral template parameter `CAPACITY` could also be specified as `unsigned`.

3.2.1. More About Implicit Requirements for Template Parameters

If fundamental data types can be used as type template parameters, the template must take special care to ensure that variables of type template parameters are initialized correctly (see Sections *Fundamental Datatypes and Constructors* and *Dynamic Objects*). This is also demonstrated using the `RingBuffer<>` class template.

All operators and operations applied to the integral template parameter `CAPACITY` of `RingBuffer<>` in Listing 122 are available because `CAPACITY` is of type `int`, i.e.,

- Line 13: less-than operator
- Lines 18 and 27: modulo operator
- Line 26: subtraction operator

For its type parameter `T`, `RingBuffer<>` defines the following implicit requirements:

- Line 17: assignment operator
- Line 20 and 29: copy constructor
- Line 34: standard constructor
- destructor

Interestingly, the places where the standard constructor and destructor for `T` are called are not visible because `RingBuffer<>` has no user-defined constructor or destructor. Therefore, the compiler automatically generates the default constructor and destructor for `RingBuffer<>` as well as for `T`, unless they are user-defined.

The reason why a standard constructor for `T` is required is that `RingBuffer<>` defines the member `m_elements` as an array of `CAPACITY` elements of type `T`. When an exemplar of an instantiated `RingBuffer<>` is created, the standard constructor is called for each element of the `m_elements` array.

When `RingBuffer<>` is instantiated with `char` for `T`, there is no user-defined constructor for `char`. The automatically defined and implicitly called constructor for `char` does nothing. Therefore, each element of `m_elements` has an undefined value after the constructor has been called. This can be checked by getting `CAPACITY` elements from a `RingBuffer<>` instantiated for `char` without first putting an element into it. The result can be a constantly changing variety of different values.

As pointed out in (U. W. Eisenecker 2022) in Section 2.5.2. *Initialization of Variables*, the built-in data types of C++ behave differently when a constructor is explicitly called with an initializer list, for example `char c { };` or `char c = char();`. In this case, each variable is initialized with a value that conceptually corresponds to 0.

It should be noted that it is also possible to apply this form of initialization to `RingBuffer<>::m_elements`. By rewriting line 34 in Listing 122 as `T m_elements[CAPACITY] { };`, all array elements are initialized with a value that is conceptually zero, e.g. 0.0 if the class template is instantiated with `double` for its type template parameter `T`.

If the specified initializer list contains any values, each array element is initialized with the corresponding value, starting with the first element and the first value. If the number of initialization values is less than the size of the array, the remaining array elements are initialized with a value that is conceptually zero. If the number of initialization values exceeds the array size, the compiler reports an error.

Interestingly, there are further implicit requirements for `T` that arise from the use of an instantiated `RingBuffer<>` exemplar. The `rb1` exemplar created in the statement `RingBuffer<char,5> rb1;` can be used to create another `RingBuffer<>` exemplar with `auto rb2 { rb1 };`. In this case, the compiler automatically generates a copy constructor for `RingBuffer<char,5>`, which in turn calls the copy constructor for the type parameter `T`, which in this case is `char`. If there are `rb1` and `rb2`, one can also assign `rb2` to `rb1`, i.e. `rb1 = rb2`. Here the compiler automatically generates an assignment operator for `RingBuffer<char,T>`, which in turn calls the assignment operator for `T`, i.e. `char`. If the copy constructor or assignment operator for `T` is not accessible, the compiler issues an appropriate error message, as shown in detail in the Section *Implicit Requirements for a Template Parameter*.

3.2.2. Lazy Instantiation of Templates

The compiler instantiates only those parts of a template that are actually used in the program. This is called *lazy instantiation*. If unused parts were instantiated, this could even lead to syntactically invalid programs. In addition, no code is generated for unused parts. It is possible to force the complete instantiation of a template. This reliably detects syntactical errors caused by the use of certain template parameters. During a full template instantiation, object code is generated for the respective instantiation. Provided that the resulting object code file is available, the definitions of the template's (member) functions that were instantiated for these template parameters are not required during subsequent compilation. Only the corresponding declarations of the (member) functions need to be present. The `RingBuffer<>` class template serves as an example.

Next, a crazy extension is incorporated into the `RingBuffer<>` class template, namely a member function that multiplies each element of `m_elements` by itself, i.e. each element is squared. Obviously, this is a rather unusual function for a `RingBuffer<>`, but it is useful to demonstrate the lazy instantiation of templates. Listing 123 shows the resulting class and a `main()` function to illustrate this specific behavior.

Listing 123: templates/RingBufferSquaring.cpp (excerpt)

```
7
8 template <typename T,int CAPACITY>
9 class RingBuffer
10 {
11     public:
12         void squareElements()
13         {
14             for (auto& e : m_elements)
15             {
16                 e *= e;
17             }
18         }
19 };
20
21 int main()
22 {
23     RingBuffer<char,5> rb1;
24     rb1.squareElements();
25     RingBuffer<string,5> rb2;
26     rb2.put("How");
27     rb2.put("are");
28     rb2.put("you?");
29     cout << rb2.get() << endl;
30     cout << rb2.get() << endl;
31     cout << rb2.get() << endl;
32     rb2.squareElements(); // This line does not compile!
33 }
```

`RingBuffer<>::squareElements()` does not care whether an element is valid or not, each element is squared. As line 49 shows, it is possible to call this member function for an exemplar of `RingBuffer<char,5>`, since `char` is an integral type and so is suitable for combined assignment and multiplication by itself. If line 56 is converted to a comment, the program is compiled and executed without error. However, if line 56 is uncommented, the compiler issues an error message stating that there is no overloaded operator `*=` for the specified data type, here `std::string`.

This shows the behavior of the compiler with respect to templates: only the parts of a (class) template that are actually used in a program are instantiated. All other parts are not instantiated. This has two important consequences that are desirable for C++. First, it prevents *code bloat*, since the compiler does not generate code for the parts of a template that are not needed. Second, ***lazy instantiation makes it possible to instantiate a template with types that would otherwise result in syntactically invalid programs.***

Sometimes a programmer is not aware of the consequences of lazy instantiation. This can lead to insufficient testing. Therefore, it is possible to force full instantiation of a template. This is done by explicitly instantiating the template with all parameters that do not have default values. Here is an example of forcing a full instantiation of a `RingBuffer<>` for type `char` and a capacity of 5:

```
template class RingBuffer<char,5>;
```

It must be placed somewhere after the definition of `RingBuffer<>` and before the definition of `main()`. If a full instantiation has been forced by explicitly instantiating a template, all associated code will be generated without creating an exemplar. In this way, it is also possible to have the compiler generate object code for template instantiations without having to additionally provide the complete definition of the corresponding template for these types.

3.2.3. Template Default Arguments

It is possible to specify default arguments for template parameters. Default arguments must be specified from right to left. It is not allowed to have a parameter without a default argument before a parameter with a default argument. When a template is instantiated, parameters can be omitted from right to left. For each omitted parameter, its default argument is used. Listing 124 shows the `RingBuffer<>` class template from Listing 122 which has been augmented with template default arguments for its two parameters.

Listing 124: templates/BufferDefaultArguments.cpp (excerpt)

```
6 |  
7 | template <typename T = char,int CAPACITY = 5>  
8 | class RingBuffer  
9 | {  
35 | };  
36 |  
37 | int main()  
38 | {  
39 |     RingBuffer<char,5> rb1;  
40 |     RingBuffer<char> rb2;  
41 |     RingBuffer<> rb3;  
42 |     RingBuffer rb4;  
43 | }
```

Lines 39 – 42 show four declarations of `RingBuffer<>` exemplars, which are all of the same type. In line 40 the value for the integral template parameter is omitted so that the default argument 5 is used here. In line 41, both the type for the type template parameter and the value for the integral template parameter are omitted, so that the default arguments `char` and 5 are used. In line 42, it looks like an ordinary class is used to declare the variable `rb4`, however the class template `RingBuffer<>` is instantiated. A modern C++ compiler first attempts to apply class argument deduction. This fails because no constructor arguments are provided. Therefore, the default values for the template parameters are used.

As mentioned above, the template declaration `template <typename T = char, int CAPACITY> /*...*/` is not legal, since it contains a template parameter without a default argument on the right side of a template parameter that has a default argument.

This also applies to the instantiation of a template. With the correct declaration in line 7 in Listing 124, attempting to instantiate the class template using only a constant for `CAPACITY`—but omitting the type for the template parameter `T` on the left-hand side—would result in a compilation error.

Function templates can also have standard arguments, which are defined using exactly the same syntax as for class templates. Therefore, no code example is given here.

3.2.4. Specialization of Class Templates

Like function templates, class templates can also be fully specialized. Additionally, it is possible to partially specialize class templates. This supports structurally different template instantiations for different template parameters.

Listing 125 shows a partial specialization of `RingBuffer<>` for the type template parameter `T` as `bool`. The integral template parameter continues to vary. Therefore, every instantiation of `RingBuffer<>` with `bool` for `T` will use the `RingBuffer<>` class template specialized to `bool`.

Listing 125: templates/BoolBuffer.cpp (excerpt)

```

6
7 template <typename T,int CAPACITY>
8 class RingBuffer
9 {
35 };
36
37 template <int CAPACITY>
38 class RingBuffer<bool,CAPACITY>
39 {
40     public:
41         void put(bool element)
42         {
43             m_elements <=<= 1;
44             if (element)
45             {
46                 m_elements |= 1;
47             }
48             if (m_readIndex < CAPACITY)
49             {
50                 ++m_readIndex;
51             }
52         }
53         bool get()
54         {
55             if (m_readIndex == 0)
56             {
57                 throw out_of_range { "RingBuffer<bool>::get() - empty buffer" };
58             }
59             return (m_elements & (1 << --m_readIndex));
60         }
61     private:
62         static_assert(CAPACITY <= sizeof(unsigned long long) * 8);
63         unsigned int m_readIndex { 0 };
64         unsigned long long m_elements { 0 };
65 };
66
67 int main()
68 {
69     RingBuffer<bool,6> bb;
70     bb.put(true);

```

```
71 |     bb.put(false);  
84 | }
```

The specialization of `RingBuffer<>` for `bool` is implemented completely differently than `RingBuffer<>`.

First, the data member `m_elements` of type `unsigned long long` stores each `bool` value as a bit. Now each `bool` value is no longer represented as an independent value. Instead, each individual bit of an `unsigned long long` value represents a `bool` value. This limits the number of bits that can be stored to the size of the datatype `unsigned long long` times 8. In line 62, `static_assert` checks at compile time that this limit is not exceeded.

Second, the class template maintains a read index instead of a write index. This is because `RingBuffer<>::put()` basically shifts the bits of `m_elements` one position to the left. When a bit is shifted out to the left, an empty bit is shifted in from the right. Bit operators have already been explained in (U. W. Eisenecker 2022) in Section 3.1.1.5. *Arithmetic Operators*. Then it is checked whether the `bool` parameter is `true`. If this is the case, the first bit is changed by a bitwise OR with a corresponding bit mask. Finally, `m_readIndex` is incremented by 1 as long as its less than `CAPACITY`.

The method `RingBuffer<>::get()` uses a bit mask and bitwise AND to determine the bit at position `m_readIndex`. The pre-decrement operator is used to adjust `m_readIndex` accordingly.

This `RingBuffer<>` specialization for `bool` is not optimized for speed or memory usage. It is primarily presented as a relatively plausible use case to demonstrate the specialization of a class template.

Similarly, `RingBuffer<>` could be fully specialized with `bool` as the type template parameter and 8 as the integral template parameter. This full specialization could use a data member of type `char` to store exactly 8 bits representing `bool` values. Since it would closely resemble the partial specialization of `RingBuffer<>` for `bool`, it is not shown here.

3.2.5. Member Templates

Both an ordinary class and a class template can also define *member function templates* and static *member function templates*. In addition, a class and a class template can also define *member class templates*. The commonality of these cases is the *member template*.

Member templates are defined with exactly the same syntax as the class and function templates presented above. The access specifiers `public`, `protected` and `private` apply in the same way as for other class members with exactly the same implications for visibility and access. Member templates are used throughout the C++ standard library.

In the following, member function templates are used to implement the serialization and deserialization of objects in binary format in a type-safe manner.

In the Section *Implementation of Object Persistence With reinterpret_cast<>()*, objects are serialized and deserialized using a textual format. A textual format has the advantage that it is usually platform-independent and human-readable. Its disadvantage is that the serialized data requires more memory than necessary, and that loss of information can occur, e.g. with floating point numbers. A binary format stores the data exactly as they are represented in RAM. It therefore requires the same amount of memory on a mass storage device as in RAM, but at the expense of platform independence.

Streams, whether file streams or string streams, offer the member functions `write()` and `read()` for the unformatted, i.e. binary, output and input of data.

The member function `std::ostream::write()` returns the receiver object as a reference. Its first parameter is of type `const char*`. It points to the beginning of the memory location from which characters are to be written to the stream. The second parameter is of type `std::streamsize` and specifies how many characters are to be written to the stream from the address passed as the first parameter. The variable `d` declared and initialized in the statement `double d {1.23};` is therefore written to the output stream `o` by executing

```
o.write(reinterpret_cast<const char*>(&d), sizeof(d));
```

The `reinterpret_cast<>` is mandatory because `&d` is a pointer to `double`, and thus cannot be passed as `const char*`. In addition, the correct size of the variable must be determined by applying the `sizeof` operator to it. Of course, `sizeof()` could also be applied to `double`. But if the type of `d` were changed, the operand of `sizeof()` would have to be changed accordingly. It is therefore more robust to specify `d` twice.

The `std::istream::read()` member function returns the receiver object as a reference. Its first parameter is of type `char*`. It points to the address in RAM to which the characters read from the stream are to be written. The second parameter is of type `std::streamsize` and specifies the number of characters to be extracted from the stream and written to the address specified by the first parameter. The `double` variable `d` is read from the input stream `i` by executing

```
i.read(reinterpret_cast<char*>(&d), sizeof(d));
```

As the memory pointed to by `&d` is to be changed here, it must be cast to a `char*`. In addition, `sizeof(d)` calculates the correct number of characters to be read from `i` and written to the memory starting at `&d`.

When serializing and deserializing data in a binary format, these repeated type casts and calculations of the correct number of bytes can be tedious. Listing 126 introduces the `Writer` and `Reader` classes. Each of them defines a member function template that eliminates these repetitive and error-prone tasks.

Listing 126: templates/GenericSerialization.cpp

```
1 // GenericSerialization.cpp by Ulrich Eisenecker, April 12, 2024
2
3 #include <iostream>
4 #include <fstream>
5 using namespace std;
6
7 class Writer
8 {
9     public:
10         Writer(std::ostream& o):m_o { o }
11         {}
12         // Here is the member template.
13         template <typename T>
14         Writer& operator<<(const T& o)
15         {
16             m_o.write(reinterpret_cast<const char*>(&o),sizeof(o));
17             return *this;
18         }
19
20     private:
21         std::ostream& m_o;
22 };
23
24 class Reader
25 {
26     public:
27         Reader(std::istream& i):m_i { i }
28         {}
29         // Here is the member template.
30         template <typename T>
31         Reader& operator>>(T& o)
32         {
33             m_i.read(reinterpret_cast<char*>(&o),sizeof(o));
34             return *this;
35         }
36     private:
37         std::istream& m_i;
38 };
39
40 template <typename T>
41 void checkingConsistency(T& a,T& b)
42 {
43     cout << a << '\t' << b << '\t' << (a == b) << endl;
44 }
45
46 int main()
47 {
48     // Binary serialization
49     bool b1 { false };
50     char c1 { 'X' };
51     unsigned int u1 { 42 };
```

```

52     long double ld1 { 221 / 7 };
53     {
54         ofstream o { "binary.data" };
55         Writer w { o };
56         // In the next line, the member template is
57         // instantiated for different types.
58         w << ld1 << ui1 << c1 << b1;
59     }
60
61     // Binary deserialization
62     bool b2;
63     char c2;
64     unsigned int ui2;
65     long double ld2;
66     {
67         ifstream i { "binary.data" };
68         Reader r { i };
69         // In the next line, the member template is
70         // instantiated for different types.
71         r >> ld2 >> ui2 >> c2 >> b2;
72     }
73
74     // Checking consistency
75     checkingConsistency(b1,b2);
76     checkingConsistency(c1,c2);
77     checkingConsistency(ui1,ui2);
78     checkingConsistency(ld1,ld2);
79 }

```

The program includes the `<fstream>` header file for the file stream types `std::ofstream` and `std::ifstream`. The only data member of the `Writer` class is a reference to `std::ostream` with the name `m_o`. `std::ostream` is defined in the header file `<ostream>` and is also included in the header file `<iostream>`. `m_o` must be initialized in the constructor of `Writer` with a subtype of `std::ostream`. Valid subtypes are, for example, `std::ofstream` and `std::ostringstream`. By passing the desired stream in the constructor, the flexibility created by the inclusion polymorphism is retained, but at the cost of a slightly more complicated object creation.

The stream insertion operator `<<` is defined as a template member function. It specifies the type parameter `T`, which is used to pass the single parameter `o` as a reference to `const`. It returns a reference to `Writer`. Its implementation first serializes the parameter `o` by executing

```
m_o.write(reinterpret_cast<const char*>(&o), sizeof(o));,
```

then it returns itself by `return *this;`. The stream reference returned by calling `m_o.write()` is ignored. The reference should not be returned as the result of a call to the `<<` operator, as its type and `Writer` are different types. Returning the `Writer` reference makes it possible to concatenate several calls of the `<<` operator.

As the `main()` function shows, variables of different types can now be uniformly sent to a `Writer` exemplar using the `<<` operator (line 58). The programmer now has a type-safe way for serializing objects, as the necessary type cast is hidden in the definition of the member function template, which also ensures that the correct number of characters is written to the output stream.

The `Reader` class is implemented accordingly. Its data member `m_i` is a reference to any `std::istream` object that must be specified in the constructor to initialize `m_i`. The `istream` type is defined in the `<istream>` header, which is also included in `<iostream>`. The stream extraction operator is defined as a member function template with type parameter `T`. The only parameter `o` is passed as a reference of type `T`, as the memory to which it points is to be changed. In its definition, `&o` is cast to a `char*`, which is passed as the first parameter to `m_i.read()`. The second parameter passed is `sizeof(o)` to deserialize the correct number of characters. Again, `*this` is returned as the result of the call to the stream extraction operator defined in the `Reader` class. In `main()`, the stream extraction operator defined for `Reader` is used to extract variables of different types uniformly from the `r` exemplar (line 71).

The function `checkingConsistency()` simply takes two parameters `a` and `b`, both of the type of the template parameter `T`. It sends each parameter to the standard output and also outputs whether both parameters have the same value.

In `main()`, several variables of different types are first declared and initialized with certain values. Then a nested block is started. Here, an output file stream exemplar named `o` is declared. The *C-string* `"binary.data"` is specified as an argument for the constructor. Therefore, a new file named *binary.data* is created in the directory in which the compiled program is executed. The `Writer` exemplar `w` is then created and initialized with `o`. In the next line, the previously declared variables are sent to `w`. At the end of the nested block, `w` is automatically deleted. When the `std::ofstream` exemplar `o` is automatically deleted, the buffer content of `o` is flushed first and then the file is closed. This means that the *binary.data* file can then be found in the active directory. Stream objects are automatically closed at the end of the block in which they are defined, unless they are explicitly closed beforehand by calling the member function `close()`.

Next, the deserialization in `main()` is demonstrated. For this purpose, new variables are declared without initializing them. Then a nested block begins. First, the `ifstream` exemplar `i` is declared. Since the *C-string* `"binary.data"` is passed as a constructor argument, a file with this name is opened in the active directory. As this file was created immediately beforehand in `main()` (and hopefully not deleted in the meantime), it is assumed that it exists and can be successfully opened for input. Therefore, no error check is performed. Next, the `Reader` exemplar `r` is declared and initialized with `i`. Then, the previously uninitialized variables are deserialized. Care must be taken to ensure that they are extracted in the same order of types in which they were previously serialized. As no type check takes place during deserialization, an error here would put the program in an erroneous state. At the end of the nested block, `r` is automatically destroyed

and `i` is closed when its destructor is executed. As the file on the hard disk is not explicitly deleted here, it is still present in the active directory even after the program has been executed.

The last part of `main()` instantiates the `checkingConsistency()` function template for each pair of variables to check whether each variable has been deserialized with exactly the same values with which its counterpart was previously serialized.

Of course, it can happen that a type cannot be serialized correctly, for example `std::string`. The reason why serialization and deserialization fails with `std::string` is that a string variable does not necessarily occupy a contiguous area of memory. The characters of a string are usually stored on the heap when their number exceeds a certain limit. In this case, the value of an internal pointer is serialized. Apart from the fact, that the characters contained in the string are not serialized, the deserialization results in an invalid pointer stored in the string variable used for the deserialization. In the best case scenario, this leads to a runtime crash of the program.

Fortunately, this problem can be easily solved. Since the operators for stream insertion and extraction are defined in `Writer` and `Reader` as member function templates, it is possible to provide specializations for them *after* the class definition. Listing 127 shows the differences with respect to Listing 126.

Listing 127: templates/StringSerialization.cpp (excerpt)

```

1 // StringSerialization.cpp by Ulrich Eisenecker, June 25, 2025
2
5 #include <string>
38
39 template <>
40 Writer& Writer::operator<< <std::string>(const std::string& s)
41 {
42     *this << s.length();
43     m_o.write(s.c_str(), s.length());
44     return *this;
45 }
46
47 template <>
48 Reader& Reader::operator>> <std::string>(std::string& s)
49 {
50     decltype(s.length()) l;
51     *this >> l;
52     char * cs { new char[l] };
53     m_i.read(cs, l);
54     s.assign(cs, l);
55     delete[] cs;
56     return *this;
57 }
58
59 int main()
60 {
61     string s1 { " Serializing and deserializing std::string." };

```

```
62     s1.at(0) = '\0';
63     cout << "s1 = " << s1 << endl
64         << "s1.length() = " << s1.length() << endl;
65     {
66         ofstream o { "string.data" };
67         Writer w { o };
68         w << s1;
69     }
70
71     string s2;
72     {
73         ifstream i { "string.data" };
74         Reader r { i };
75         r >> s2;
76     }
77     cout << "s2 = " << s2 << endl
78         << "s2.length() = " << s2.length() << endl;
79 }
```

Of course, the `<string>` header file must be included so that `std::string` is available. The definitions of the `Writer` and `Reader` classes remain unchanged. New are the full specializations of the member function templates `Writer::operator<<()` and `Reader::operator>>()`.

The full specialization of `Writer::operator<<()` for `std::string` is relatively simple. First, the member function template defined in `Writer` is automatically instantiated and called in order to serialize the length of the string `s` passed as reference to `const`. Then `m_o` is used directly to serialize the C-string, which can be requested by the member function `string::c_str()`. The result is a C-string, i.e. `const char*`. Then, `s.length()` characters are serialized to `m_o`. Normally a C-string is terminated with a character with value 0, i.e. `'\0'`. This does not apply to C++-strings. As these are so-called *counted strings*, they can contain ASCII zeroes at any position and do not have to be zero-terminated. For this reason, the `string::length()` function is used to determine the correct number of characters in the string, and not the `std::strlen()` function (which is defined in the `<cstring>` header), which counts the characters until the first occurrence of `'\0'` but does not include it. Upon completion of the function, `*this` must be returned so that it is possible to concatenate calls to the stream insertion operator.

The full specialization of `Reader::operator>>()` for `std::string` is more complex. First, `decltype` is used to determine the type of the result of the call to `s.length()` in order to declare the variable `l`. As a completely equivalent alternative, the member type `string::value_type` could have been used for this purpose. Next, the member function template of the stream extraction operator defined in `Reader` is instantiated and called in order to deserialize the length of the string into the variable `l`. Then the pointer variable `cs` is declared and initialized with a pointer that points to `l` characters, which are dynamically allocated by calling the `new[]` operator. Now the input stream reference `m_i` is used to deserialize `l` characters into the character array pointed to by `cs`. The call to `m_i.read()` does not care if the deserialized character values contain ASCII zeroes, and

it is not required that the sequence of character values is terminated with `'\0'`. Then an overload of the `string::assign()` member function is used to copy 1 characters values pointed to by `cs` into the string variable `s`. Finally, the dynamic memory pointed to by `cs` is released by calling the `delete[]` operator, and `*this` is returned to enable chained calls to the stream extraction operator.

In the `main()` function, the string variable `s1` is declared and initialized with a meaningful character string. The next statement replaces the space at index position 0 in `s1` with `'\0'`. As `'\0'` is a non-printable character, it is not displayed in the output generated in the next line. However, it is still counted by the `string::length()` function, as the output statement shows.

In the following block, an output file stream is created which causes a file named *string.data* to appear in the active directory after the program has been executed. Next, a `Writer` exemplar with the name `w` is declared and initialized with `o`. `s1` is sent to `w`. At the end of the block, the `Writer` exemplar is automatically deleted and the file stream `o` is automatically flushed and closed by its destructor.

The string variable `s2` is declared before the next block, but without explicit initialization. In the following block, the input file stream exemplar `i` is created and opened. If the file with the name *string.data* has not been deleted from the mass storage in the meantime, this should be successful. No check is therefore carried out. A `Reader` exemplar is then created and initialized with `i`, and `s2` is extracted from `r`. At the end of the block, the `Reader` exemplar is automatically deleted and the output stream is automatically closed by its destructor.

Before the program ends, it outputs the content of the newly deserialized string `s2` and its length as well. Both values should correspond exactly to the preceding output of `s1` and its length.

After running the program, the file with the name *string.data* still exists in the current directory. There is a good chance that its contents can be displayed, e.g. by entering *cat string.data* in the terminal window. This is because it contains many printable characters, even though it is a binary file. Of course, the size information cannot be displayed correctly.

It must be emphasized that the serialization and deserialization of pointers as demonstrated in Section *Implementation of Object Persistence With reinterpret_cast<>()* using inclusion polymorphism would require additional code: an object identity manager would be required and it would be necessary to overload the member function templates for serialization and deserialization in the class definitions of `Writer` and `Reader` for pointers. In addition, the objects containing pointers would require a member function corresponding to `Persistable::updateAssociations()` (Listing 63) and `Person::updateAssociations()` (Listing 65). All this could be implemented using generic polymorphism, i.e. with templates only.

Member class templates are defined in the same way as member classes and member function templates. The same applies to class templates that contain templates for member functions or member classes. None of this is demonstrated here, as this would require relatively complex application examples.

3.2.6. Template Template Parameters

Constant template parameters allow variability of constant members of templates and type template parameters allow variability of types of template members. In fact, there is a third need for variability in templates, namely templates. This need is met by *template template parameters*. They are presented by means of an abstract code example.

As a first example, a class template `Pick<>` is to be designed and implemented. Its constructor accepts two arguments and picks one of them to store its value. The type of the constructor arguments is represented as a type template parameter `T`. The criterion for selecting one of the arguments is passed as another template parameter. It cannot be passed as a type template parameter, because it depends on the type template parameter `T`. That means if `T` is `int`, the smaller of the two integer values must be selected so that the less-than operator returns the smaller of the two values. If `T` is `std::string`, the less-than operator returns the string that comes first in ascending order of the ASCII values of its characters. Another criterion could be to return the greater of two values. Therefore, it is useful to represent a criterion as a class template with a type template parameter that represents the type of the two values to be compared. Consequently, this class template for the criterion must be passed as a parameter to the `Pick<>` class template. A template template parameter is required for this purpose. Listing 128 shows the corresponding class templates and gives examples of their use.

Listing 128: templates/TemplateTemplateParameter.cpp

```
1 // TemplateTemplateParameter.cpp by Ulrich Eisenecker, July 13, 2022
2
3 #include <iostream>
4 #include <string>
5 using namespace std;
6
7 template <typename T,template <typename U> class Criterion>
8 class Pick
9 {
10     public:
11         Pick(const T& t1,const T& t2):m_t { Criterion<T>::select(t1,t2) }
12         {}
13         T& value()
14         {
15             return m_t;
16         }
17     private:
18         T m_t;
19 };
20
```

```

21 template <class T>
22 class Larger
23 {
24     public:
25         static const T& select(const T& t1,const T& t2)
26         {
27             if (t1 > t2)
28             {
29                 return t1;
30             }
31             return t2;
32         }
33 };
34
35 template <class T>
36 class Smaller
37 {
38     public:
39         static const T& select(const T& t1,const T& t2)
40         {
41             if (t1 < t2)
42             {
43                 return t1;
44             }
45             return t2;
46         }
47 };
48
49 int main()
50 {
51     Pick<int,Larger> p1 { 1, 2 };
52     Pick<double,Smaller> p2 { 1.5, 1.4 };
53     Pick<string,Larger> p3 { "Hi", "there" };
54     cout << p1.value() << endl;
55     cout << p2.value() << endl;
56     cout << p3.value() << endl;
57 }

```

Each of the class templates `Larger<>` and `Smaller<>` defines a static member function `select()` that returns either the smaller or the larger of its two arguments. Here, the corresponding operators perform these comparisons. Basically, any calculation that involves both values can be performed here.

The class template `Pick<>` specifies `T` as a type template parameter and `Criterion<>` as a template template parameter. The corresponding syntax for the latter is very similar to that for defining a class template. The `template` keyword indicates, that a template template parameter is specified. Then a pair of angle brackets encloses the parameters of the template to be defined, here `<typename U>`. Actually, one could omit the name of the template parameter here, because the compiler only needs to know the type of the parameter—in this case, `<typename>`, i.e., a type template parameter. This is followed by the name of the template template parameter, in this case `Criterion`.

The member initializer list of the `Pick<>` constructor instantiates the `Criterion<>` template with the type parameter `T` and then calls its static member function `select()` with the parameters `t1` and `t2` to initialize the data member `m_t`.

In the `main()` function, `Pick<>` is instantiated with different types and class templates to demonstrate the effects of passing different criteria for selecting a value. The exemplars `p1`, `p2`, and `p3` select the one value that matches the corresponding criterion and store it during their creation.

An equivalent but simpler solution is possible using member function templates. Instead of the template-template parameter, `Pick` uses a second type template parameter with a corresponding member function template. The code that uses it in `main()` remains unchanged. The complete alternative solution is available only in the accompanying material under the name *templates/PickWithMemberTemplate.cpp*.

A valuable use case for template template parameters is presented in (pfalcon 2013), which uses template template parameters to implement a stream insertion operator for *STL* container templates as a template function with a template template parameter.

Other uses of template template parameters are template meta-functions, which correspond to higher order functions in functional programming, which is not covered here.

3.2.7. Typename and Template in Templates

Syntax checking of templates is limited by the way they are compiled. Special rules have been established to improve the diagnosis of the correct use of parts of templates. Therefore, member types and member templates of template parameters must be explicitly specified with `typename` or `template` in many cases. An abstract example is used for illustration.

Unlike other code, which is compiled exactly once, templates are compiled twice. The first time a template is compiled is when it is defined. Here the compiler checks if the syntax is correct (as far as possible). The second time a template is compiled is when it is instantiated. Here the compiler checks the syntax again, but in addition it also checks whether the current template parameters meet all implicit requirements for the given use of the template—templates are instantiated lazily—and object code is also generated.

Therefore, the template definition must be available to the compiler at both times. It is not possible to split the declaration and definition of a template into a header file and an implementation file, whereby the latter can be compiled separately. During the first compilation of a template, only syntactic checks are performed, but no object code is generated. In the second compilation step, object code is generated for the instantiated template. If this second compilation step does not take place, no object code is generated for this template instantiation, which causes the linker to complain about missing definitions. Therefore, all definitions for a template are usually implemented inline, i.e. directly in the header file or in a file that is included by the header file. All this is ex-

plained in detail with an example in Section *Structuring Template Files*. There it is also shown how to separate the declaration and the definition of a template in a way that roughly corresponds to the division into a header and an implementation file.

The compiler has a limited ability to perform syntactic checks during the initial compilation step. Of course, it is desirable to strive for optimal diagnostics of possible errors. For this reason, C++ language developers have decided to enforce disambiguation when accessing members of type template parameters whenever necessary. Two rules of thumb are generally useful:

- Firstly, whenever a member type of a type template parameter is used, it must be specified with the `typename` keyword.
- Secondly, when a member template of a type template parameter is used, it must be specified with the `template` keyword.

Consequently, the compiler checks whether these members are used correctly. Listing 129 illustrates both cases and their combination.

Listing 129: templates/TypenameTemplate.cpp

```

1 // TypenameTemplate.cpp by Ulrich Eisenecker, July 13, 2022
2
3 #include <iostream>
4 using namespace std;
5
6 template <class T>
7 class ClassTemplate
8 {
9     public:
10         ClassTemplate()
11         {
12             cout << T::A << endl;
13             typename T::B someValue { 99 };
14             cout << someValue << endl;
15             typename T:: template C<1> someClass;
16             cout << T::template C<0>::D << endl;
17             typename T:: template C<-1>::E anotherValue { 22.0 / 7.0 };
18             cout << anotherValue << endl;
19         }
20 };
21
22 class SomeClass
23 {
24     public:
25         static inline const int A { 42 };
26         using B = int;
27         template <int n>
28         class C
29         {
30             public:

```



```
31     static inline const int D { n };
32     using E = double;
33 };
34 };
35
36 class AnotherClass
37 {
38     public:
39     static inline bool A { false };
40     using B = unsigned long long;
41     template <int m>
42     class C
43     {
44     public:
45         static inline double D { m / 3.14 };
46         using E = float;
47     };
48 };
49
50 int main()
51 {
52     ClassTemplate<SomeClass> ct1;
53     ClassTemplate<AnotherClass> ct2;
54 }
```

The `ClassTemplate<>` template takes the type template parameter `T`. One should be aware that `ClassTemplate<>` is implemented without having a specific class in mind for the type template parameter `T`!

In its constructor in line 12, `T::A` is sent to the standard output. Since no further information is attached to `T::A`, the compiler assumes that `A` is a data member of `T`. When `ClassTemplate<>` is instantiated with `SomeClass` in line 52, this is an—obviously—correct assumption with appropriate usage. In line 13 the variable `someValue` is declared, which is of type `T::B`. `T::B` must be prefixed with `typename` to make it clear that `B` is a member type of the type template parameter `T`. In line 15, the member template `C<>` of `T` is instantiated with `1`. Therefore, `C<1>` must be prefixed with the `template` keyword to indicate that `C<>` must actually be a template. Otherwise, `C<1>` would be parsed as a comparison between a data member of `T` called `C` and an integer literal, immediately followed by another relational operator and an unknown identifier, causing the compiler to issue an error. Since the result of instantiating a class template is a type, this must be specified with the `typename` keyword before `T` at the beginning of line 15. Line 16 instantiates the member template `C` and outputs its data member `D`. Since `D` is neither a type nor a template, it is simply used. To indicate that the member template `C<>`—a member of `T`—is to be instantiated, `C` must be preceded by the `template` keyword. Line 17 illustrates the same constellation as in line 15, so it will not be explained further.

The `SomeClass` class defines several kinds of members, namely a data member, a type member and a template member. They are all intentionally named with single capital letters to give no hint as to what exactly they are. The `C<>` template member itself also defines a data member and a type member.

In order to clarify in which respect the classes are used as actual types for the instantiation of `ClassTemplate<>`, the class `AnotherClass` is also defined. Only the name of a class member and its usage is relevant here. The similarities between `SomeClass` and `AnotherClass` are highlighted. These similarities are important because both classes are valid type template parameters for the `ClassTemplate<>` template. To verify that `SomeClass` and `AnotherClass` are valid types for instantiating `ClassTemplate<>`, the full instantiations of `ClassTemplate<>` with both classes are forced immediately before the `main()` function.

In summary, the use of a member of a type template parameter within another template without further specification means that it is used as an ordinary element of $\mathbb{T}$, i.e. as data or function. To indicate that a member is to be treated as a type member; typename should be used, and to indicate the use of a template member; template is required.

The rules for looking up dependent names in template definitions are extensive and complex, see for example (*Dependent Names - Cppreference.Com* n.d.). In some cases template and typename can be omitted, since the compiler can infer the specific usages from the context. Not all compilers are up to date in this respect. It is therefore recommended to use typename and template for corresponding specifications in templates. This does not lead to errors during compilation and can also improve the readability of the code.

3.2.8. Structuring Template Files

All template definitions must be available in a compilation unit. It is not possible to keep a template declaration and its definition in separate compilation units, as the definition must be available to the compiler when the template is instantiated. This results in special requirements for the structuring of template source code files. In this section, the various resulting structuring options are presented. In addition, a workaround is presented that allows template definitions to be removed if the required template instantiations are made in advance.

Listing 130 shows the header file `all.hpp`, which contains the function template `swapVariables<>()` (from Listing 110) and the class template `RingBuffer<>` (from Listing 122). Here the declarations are definitions at the same time.

Listing 130: templates/all.hpp

```
1 // all.hpp by Ulrich Eisenecker, March 2, 2025
2
3 /* This header file contains a function template and
4    a class template.
5    The function template and the member functions of the
6    of the class template are declared only and defined
7    separately after their declaration.
8 */
9
10 #ifndef ALL_HPP
11 #define ALL_HPP
12
13 template <typename T>
14 void swapVariables(T& a,T& b)
15 {
16     T temp { a };
17     a = b;
18     b = temp;
19 }
20
21 template <typename T,int CAPACITY>
22 class RingBuffer
23 {
24     public:
25     void put(T element)
26     {
27         if (m_noOfElements < CAPACITY)
28         {
29             ++m_noOfElements;
30         }
31         m_elements[m_writeIndex++] = element;
32         m_writeIndex %= CAPACITY;
33     }
34     T get()
35     {
36         // The function must not be called if m_noOfElements == 0!
37         int readIndex { (CAPACITY - m_noOfElements
38                         + m_writeIndex) % CAPACITY };
39         --m_noOfElements;
40         return m_elements[readIndex];
41     }
42     private:
43     int m_writeIndex { 0 },
44         m_noOfElements { 0 };
45     T m_elements[CAPACITY];
46 };
47
48 #endif // ALL_HPP
```

The program *UseInstantiations.cpp* (Listing 135) can be used to test this header file. In this case it is necessary to replace line 3 with `#include "all.hpp"`. In addition, lines 23 and 25 can be uncommented.

The next step is to split declarations and definitions, but keep them all in the same header file named *separated.hpp* (Listing 131). Here, the function template is divided into a declaration and a definition, the latter of which appears later in the file. The class template is a valid definition, but this definition only contains the declarations of the corresponding member functions. Their separate definitions also appear later in the file.

Listing 131: templates/separated.hpp

```

1 // separated.hpp by Ulrich Eisenecker, March 2, 2025
2
3 /* This header files contains a function template and
4    a class template.
5    The function template as well as the member functions
6    of the class template are declared only and defined
7    separately after their declaration.
8 */
9
10 #ifndef SEPARATED_HPP
11 #define SEPARATED_HPP
12
13 // Declaration only
14
15 template <typename T>
16 void swapVariables(T& a,T& b);
17
18 // Definition of class template only
19 // without definitions of member functions
20
21 template <typename T,int CAPACITY>
22 class RingBuffer
23 {
24     public:
25         void put(T element);
26         T get();
27     private:
28         int m_writeIndex { 0 },
29             m_noOfElements { 0 };
30         T m_elements[CAPACITY];
31 };
32
33 // Definition
34
35 template <typename T>
36 void swapVariables(T& a,T& b)
37 {
38     T temp { a };
39     a = b;
40     b = temp;
41 }

```

```
42 |
43 | // Definitions of member functions of class template
44 |
45 | template <typename T,int CAPACITY>
46 | void RingBuffer<T,CAPACITY>::put(T element)
47 | {
48 |     if (m_noOfElements < CAPACITY)
49 |     {
50 |         ++m_noOfElements;
51 |     }
52 |     m_elements[m_writeIndex++] = element;
53 |     m_writeIndex %= CAPACITY;
54 | }
55 |
56 | template <typename T,int CAPACITY>
57 | T RingBuffer<T,CAPACITY>::get()
58 | {
59 |     // The function must not be called if m_noOfElements == 0!
60 |     int readIndex { (CAPACITY - m_noOfElements
61 |                     + m_writeIndex) % CAPACITY };
62 |     --m_noOfElements;
63 |     return m_elements[readIndex];
64 | }
65 |
66 | #endif // SEPARATED_HPP
```

The syntax for defining the function template is straightforward. It simply looks like a declaration, which is also a definition.

This is different for the definitions for the member functions of the class templates. Here, the template-specific part must be included in the definition, and the class name and the template parameters must be placed in angle brackets before the name of the member function. Although the syntax looks complicated, it is consistent.

Again, the program *UseInstantiations.cpp* (Listing 135) can be used for testing. Now line 3 must be replaced by `#include "separated.hpp"` and lines 23 and 25 can be uncommented.

Now the definitions can be moved to a separate file. It is called *definitions.t*, where the *.t* extension stands for *template definitions* (Listing 132). Unlike a *.cpp* file (also called an *implementation file*), the *.t* file cannot be compiled separately.

Listing 132: templates/definitions.t

```

1 // definitions.t by Ulrich Eisenecker, March 2, 2025
2
3 /* This header file is included from header.cpp.
4    It contains all definitions of the function template
5    and the member functions of the class template.
6 */
7
8 template <typename T>
9 void swapVariables(T& a,T& b)
10 {
11     T temp { a };
12     a = b;
13     b = temp;
14 }
15
16 template <typename T,int CAPACITY>
17 void RingBuffer<T,CAPACITY>::put(T element)
18 {
19     if (m_noOfElements < CAPACITY)
20     {
21         ++m_noOfElements;
22     }
23     m_elements[m_writeIndex++] = element;
24     m_writeIndex %= CAPACITY;
25 }
26
27 template <typename T,int CAPACITY>
28 T RingBuffer<T,CAPACITY>::get()
29 {
30     // The function must not be called if m_noOfElements == 0!
31     int readIndex { (CAPACITY - m_noOfElements
32                     + m_writeIndex) % CAPACITY };
33     --m_noOfElements;
34     return m_elements[readIndex];
35 }

```

In the new header file, which is now called *header.hpp*, these definitions are deleted (Listing 133).

Listing 133: templates/header.hpp

```
1 // header.hpp by Ulrich Eisenecker, March 2, 2025
2
3 /* This header file contains a function template and
4    a class template.
5    The function template and the member functions of the
6    of the class template are declared.
7    Their definitions are contained in definitions.t
8    which is included at the end of the file.
9    */
10
11 #ifndef HEADER_HPP
12 #define HEADER_HPP
13
14 template <typename T>
15 void swapVariables(T& a,T& b);
16
17 template <typename T,int CAPACITY>
18 class RingBuffer
19 {
20     public:
21         void put(T element);
22         T get();
23     private:
24         int m_writeIndex { 0 },
25             m_noOfElements { 0 };
26         T m_elements[CAPACITY];
27 };
28
29 #if __has_include("definitions.t")
30 #include "definitions.t"
31 #endif // __has_include
32
33 #endif // HEADER_HPP
```

It only includes the declaration of the function template and the definition of the class template. This header file largely corresponds to an ordinary header file with one exception: at its end, the preprocessor directive `#if __has_include` checks whether the file *definitions.t* exists in the same directory. If it is present, it is included. Then the `#if __has_include` directive is terminated with `#endif`. As a result, the definitions file is only included if it exists. Below is a demonstration of how this feature can be exploited.

Again, the inclusion of *header.hpp* can be tested with the *UseInstantiations.cpp* program (Listing 135), which must be modified accordingly.

The Section *Lazy Instantiation of Templates* explains how to force the full instantiation of a template for specific parameters. This is used below to illustrate the possibility of creating an object file with specific template instantiations that allows the use of a header file that does not include the *.t* file. As a prerequisite, the *instantiations.cpp* file is created (Listing 134).

Listing 134: templates/instantiations.cpp

```

1 // instantiations.cpp by Ulrich Eisenecker, March 2, 2025
2
3 #include "header.hpp"
4
5 template void swapVariables(int&,int&);
6
7 template class RingBuffer<bool,3>;

```

This file includes *header.hpp* and then forces the instantiation of the function template *swapVariables<>* for *int* as *T* and the class template *RingBuffer<>* for *bool* as *T* and 3 as *CAPACITY*. After this file has only been compiled but not linked, an object file is created. For example, *g++ -std=c++23 -c instantiations.cpp* leads to the creation of *instantiations.o*. This object file contains the binary code generated for the forced instantiations of the templates *swapVariables<>()* and *RingBuffer<>*. This object file can later be linked to another file that contains a *main()* function, for example *UseInstantiations.cpp* (Listing 135).

Listing 135: templates/UseInstantiations.cpp

```

1 // UseInstantiations.cpp by Ulrich Eisenecker, March 2, 2025
2
3 #include "header.hpp"
4
5 #include <iostream>
6 using namespace std;
7
8 int main()
9 {
10     int a { 1 }, b { 2 };
11     swapVariables(a,b);
12     cout << ((a == 2) && (b == 1 )) << endl;
13
14     RingBuffer<bool,3> c;
15     c.put(true);
16     c.put(false);
17     c.put(true);
18     cout << (c.get() == true) << endl;
19     cout << (c.get() == false) << endl;
20     cout << (c.get() == true) << endl;
21
22     double d { 2.3 }, e { 4.5 };
23     // swapVariables(d,e);

```



```
24 |  
25 | // RingBuffer<bool,2> f;  
26 | }
```

This file instantiates `swapVariables<>()` for `int` and `RingBuffer<>` for `bool` and `3`. Due to the previous step, the binary code for these instantiations is available in *instantiations.o*. Therefore, it is possible to either move *definitions.t* to another directory or temporarily rename it. Then *UseInstantiations.cpp* can be compiled and linked with *instantiations.o*, e.g. `g++ std=c++23 UseInstantiations.cpp instantiations.o`. The missing *.t* file is not a problem because of the use of `#if __has_include`. After successful compilation, the resulting file can be executed as *./a.out*. After uncommenting lines 23 and 25, *UseInstantiations.cpp* cannot be successfully compiled, or more precisely linked, because the binary code for `swapVariables<>()` instantiated for `double` and `RingBuffer<>` instantiated for `bool` and `2` is missing.

This illustrates the use of templates and the creation of an object file that contains the required instantiations of these templates. Now the header file without the definitions of the (member) functions and the object file can be passed on to a third party without disclosing the definitions of the (member) functions.

3.2.9. Name Look-up in Templates

If a name is used in a non-template part of a program, its meaning or its definition must be available to the compiler according to the given usage. Therefore, when looking up a name, the program compiled up to that point is taken into account. In the case of templates, the compiler does not consider template parameters when performing *name look-up*. If it is to do this, it must be forced explicitly. This is demonstrated with an abstract code example.

The term *name look-up* refers to the contexts and the rules that apply when the compiler looks for the definition of a name. For example, within a most nested block, the compiler first looks in the innermost block for a definition of this name. If it does not find the name there, it continues the search in the surrounding block and so on.

When looking up a name in a template, special care must be taken if the name depends on a template parameter. This situation is illustrated in Listing 136. It makes use of public inheritance. This means that a new class is not completely redefined, but refers to a base class to which it adds members. Through public inheritance, an exemplar of the derived class can be used wherever an exemplar of the base class is expected, either as a reference or as a pointer. For detailed details on inheritance, see Section *Inheritance*.

Listing 136: templates/LookUp.cpp

```

1 // LookUp.cpp by Ulrich Eisenecker, July 13, 2022
2
3 #include <iostream>
4 #include <string>
5 using namespace std;
6
7 string message()
8 {
9     return "Uh oh!"s;
10 }
11
12 class Surprise
13 {
14     public:
15         string message()
16         {
17             return "Ah!"s;
18         }
19 };
20
21 template <class T>
22 class SomeTemplate: public T
23 {
24     public:
25         SomeTemplate()
26         {
27             cout << message() << endl;
28             cout << T::message() << endl;
29             cout << (*this).message() << endl;
30         }
31     /*
32     string message()
33     {
34         return "Oh!"s;
35     }
36     */
37 };
38
39 int main()
40 {
41     SomeTemplate<Surprise> sts;
42 }

```

A particularly noticeable feature is that the `SomeTemplate<>` class inherits from its type template parameter `T`. Without going into further details, it is only worth mentioning that this is a commonly used technique in template programming. As a result, every instantiation of `SomeTemplate<>` is a subclass of `T`. Here, the candidate subclass is the class `Surprise`. Its only member is the `message()` function, which returns a `std::string`. It is worth noting that there is a global function called `message()`, which also returns a

`std::string`. In addition, `SomeTemplate<>` also defines a member function `message()`. This member function redefines `T::message()`, which must be available for the template parameter `T` because it is explicitly called in line 28.

In the constructor of `SomeTemplate<>` in line 27, the function `message()` is called. One could naively assume that the compiler calls `T::message()` here. However, this is not the case, because the compiler assumes by default that anything that is not *explicitly* associated with a template parameter is *not* associated with a template parameter. That is, the compiler does not include the template parameter in the name search. Therefore, the compiler calls the global function `message()`. Fortunately, if this were not present, the compiler would issue an error. To change this, the name must be made dependent on the type parameter. This can be achieved in two ways.

The first option is to force the compiler to look up the name directly in the template parameter `T` by writing `T::message()`. In this case, the compiler only searches for `message()` in `T`.

The second option is to use `this` to refer to the currently active exemplar of the instantiated template. In line 29, `(*this).message()` is used to accomplish this. As explained in (U. W. Eisenecker 2022) in Section 4.3.3 *Class Design Based on Reference Semantics*, `this` refers to the object that is currently in use. To avoid the problem that is caused by the dot operator having a higher precedence than the dereference operator, parentheses are placed around `(*this)`. So, the asterisk before `this` dereferences the `this` pointer. Then the member function `SomeTemplate<>::message()` is called for this object. Thus, `(*this).message()` or alternatively `this->message()` tells the compiler to look for `message()` in the instantiated template, i.e. it can be in `T`, in `SomeTemplate<>` or in both.

In the latter case, `SomeTemplate<T>` may *hide* or *override* the name defined in `T`. *Hiding* means that the name defined in `SomeTemplate<>` shadows the member of the same name in `T`, *overriding* means that `SomeTemplate<T>` provides a new implementation for a virtual member function defined in `T` with the same signature. Both cases are compiled silently.

If lines 32 – 35 are commented out, the compiler calls the global `message()` function in line 27. If these lines are not commented out, the compiler no longer considers the global `message()` function because a member function of this name is defined in `SomeTemplate<>`. Then it calls `SomeTemplate<>::message()` in line 29, because `SomeTemplate<>` redefines this member function. Therefore, `Surprise::message()` is no longer called.

3.2.10. Template Alias

In (U. W. Eisenecker 2022), Section 5.5.8 *Rule* the `using` declaration was introduced to define an alias for a type. The `using` keyword also allows the definition of an alias for a class template. It must specify exactly the same number and kind of template parameters as the original template. It can then be used in place of the original template without restriction.

Listing 137 is based on Listing 122 and introduces the template alias `CircularBuffer<>` for `RingBuffer<>`. It should be noted that the parameter names can vary, but not their kind and number. In `main()`, `CircularBuffer<>` and `RingBuffer<>` are instantiated with exactly the same parameters. As the following `static_assert` shows, the instantiated templates are of the same type. For more information, see (*Type Alias, Alias Template (since C++11)* - *Cppreference.Com* n.d.).

Listing 137: templates/AliasTemplate.cpp (excerpt)

```

1 // AliasTemplate.cpp by Ulrich Eisenecker, March 4, 2025
2
3 #include <iostream>
4 #include <type_traits>
5 using namespace std;
6
7 template <typename T,int CAPACITY>
8 class RingBuffer
9 {
32 };
33
34 template <typename TYPE,int SIZE>
35 using CircularBuffer = RingBuffer<TYPE,SIZE>;
36
37 int main()
38 {
39     using CB = CircularBuffer<char,5>;
40     using RB = RingBuffer<char,5>;
41     static_assert(is_same<CB,RB>::value,"That should not happen!");
42 }
```

3.3. Variadic Templates

Both C++ and its predecessor C offer so-called *variadic macros* and *variadic functions*. They are called *variadic* because they accept a different number of parameters. However, this flexibility comes at the expense of type safety. Variadic macros and functions are almost obsolete. Therefore, they are not presented further here. *Variadic templates* were introduced later in C++. They can be instantiated with a varying number of template parameters, but have the advantage of type safety. This is achieved by specifying

a so-called *template parameter pack*, or shorter, *parameter pack* or simply *pack*, as the template parameter. The parameter pack can be defined for non-type template parameters, type template parameters or template template parameters.

The following code examples focus exclusively on the syntax and demonstrate only a small but well thought-out selection of uses of variadic templates. Only function templates and non-type and type-parameter packs are used.

The code examples are all contained in a single source file. Table 14 shows how they are organized. The first column distinguishes between non-type (here integral) and type parameters. The second column distinguishes between two approaches to processing, *recursion* or *fold expressions*. For better understanding, there are two versions of each example, a *quiet* version, which only performs the desired calculation, and a *verbose* version, which provides additional output. The quiet versions are contained in the quiet namespace, the verbose versions in the verbose namespace. Therefore, the corresponding quiet functions are called before or in the `main()` function after using `namespace quiet;` and the verbose functions are called after using `namespace verbose;`. The last column specifies the function name that is explicitly qualified by the namespace.

Table 14: Variadic function templates

Template parameter	Processing	Version	name
integral	recursion	quiet	<code>quiet::sum1<>()</code>
integral	recursion	verbose	<code>verbose::sum1<>()</code>
integral	fold expression	quiet	<code>quiet::sum2<>()</code>
integral	fold expression	verbose	<code>verbose::sum2<>()</code>
type	recursion	quiet	<code>quiet::sum3<>()</code>
type	recursion	verbose	<code>verbose::sum3<>()</code>
type	fold expression	quiet	<code>quiet::sum4<>()</code>
type	fold expression	verbose	<code>verbose::sum4<>()</code>

Listing 138 shows the definition of the `quiet::sum1<>()` function template.

Listing 138: templates/quiet::sum1<>() (VariadicTemplates.cpp)

```

10 |     template <int n = 0,int...pack> // no whitespace around ...
11 |     auto sum1()
12 |     {
13 |         if constexpr (n == 0)
14 |         {
15 |             return 0;
16 |         }
17 |         else
18 |         {
19 |             return n + sum1<pack ...>();
20 |         }
21 |     }

```

The function template has two integral template parameters. The first, `n`, is an integral template parameter, as introduced in Listing 116. A detail that will become important later is that a default value is specified for it, here `0`. The second, named `pack`, is specified as the `pack` parameter by placing an *ellipsis* to the left of the template parameter. The *ellipsis* is represented by three points, namely `...`. The space around the *ellipsis* is optional. The name of the `pack` can be chosen arbitrarily, as long as it is a valid C++ identifier. A popular name is `rest`.

It should be noted that the use of `0` as a special value for terminating the instantiation of the recursive function template is problematic. If the `pack` contains a value equal to `0`, the calculation will be terminated prematurely, e.g. the evaluation of `quiet::sum1<0,100>()` results in `0`.

Specifying the return type of the function in line 11 as `auto` is a new feature that is independent of templates. It simplifies the function specification, especially when returning complicated types. If the return type of a function is specified as `auto`, the compiler checks whether all return statements of a function return the same type. If so, this type is used as the return type, otherwise an error is reported. It is not allowed to return values of different types.

In line 13, `if constexpr` is used to check if `n` equals `0`. This is done at compile time. If the condition evaluates to `true`, the `if` branch is included in the instantiation of the function template with the statement `return 0;` and the `else` branch is suppressed. The function template is instantiated with the `if` branch in the following cases:

- It is instantiated without specifying any parameter, e.g. `sum1<>()`.
- The parameter `pack` contains no or no further parameters.

If the condition of `if constexpr` evaluates to `false`, the `if` branch is suppressed and the `else` branch `return n + sum1<pack ...>();` is included in the instantiation of the function template. This statement uses recursion to calculate the sum. To do this, the

result of calling the instantiation of the function template for `sum1<>()` with `pack ...` is added to `n`. If the ellipsis is to the right of the pack, this means that the pack should be unpacked. Since `sum1<>()` is defined with the integral template parameter `n` and a pack, the template parameter `n` takes the first parameter of the pack and the rest is passed as a pack. This splits the (former) pack into the single parameter `n` and the rest of the (former) pack as a (new) pack. In this way, each successive call reduces the pack by one parameter until it has no more parameters and `sum1<>()` is called with the default value for `n`, namely 0. However, this case is checked by `if constexpr`, and the simple return of 0 prevents any further instantiation of the `sum1<>()` function template. It is therefore a recursive implementation of the calculation of the sum, which happens entirely at compile time.

For this implementation of `sum1<>()`, it is crucial that there are two template parameters, one normal and one pack parameter. For this reason, each recursive instantiation uses one parameter from `pack` as the first parameter and passes the rest as the template parameter `pack`. Actually, the pack is not really a second template parameter, but it bundles 0, 1 or more parameters.

Figure 44 symbolically illustrates the template instantiations for calling `quiet::sum1<3,5,1>()`.

Figure 44: Template instantiations for `quiet::sum1<3,5,1>()`

```
auto sum1<3,5,1>()
    return 3 + sum1<5,1>();
    return 3 + 5 + sum<1>();
    return 3 + 5 + 1 + sum<>();
    return 3 + 5 + 1 + 0;
    → return 9;
```

C++ Insights (Fertig n.d.) offers another, more code-centric view of the compiler's work. Listing 139 shows the program inserted to the left pane of the *C++ Insights* website. Care should be taken that C++ 20 is selected as the standard in the top toolbar. After pressing the *Run* button, *C++ Insights* shows the resulting program with instantiated templates in the right pane (Listing 140).

Listing 139: templates/InputCppInsights.cpp

```

1  #include <iostream>
2  #include <string>
3
4  using namespace std;
5
6  template<int n = 0,int...pack> // no whitespace around ...
7  auto sum1()
8  {
9      if constexpr (n == 0)
10     {
11         return 0;
12     }
13     else
14     {
15         return n + sum1<pack ...>();
16     }
17 }
18
19 int main()
20 {
21     cout << "sum1<>():\n"
22           << sum1<>() << endl;
23     cout << "sum1<3,5,1>():\n"
24           << sum1<3,5,1>() << endl;
25     cout << "=====" << endl;
26 }

```

Listing 140: templates/OutputCppInsights.cpp

```

1  #include <iostream>
2  #include <string>
3
4  using namespace std;
5
6  template<int n = 0, int ...pack>
7  auto sum1()
8  {
9      if constexpr(n == 0) {
10         return 0;
11     } else /* constexpr */ {
12         return operator+(n, sum1<pack... >());
13     }
14 }
15
16
17 #ifdef INSIGHTS_USE_TEMPLATE
18 template<>
19 int sum1<0>()
20 {
21     if constexpr(true) {

```



```
22     return 0;
23 } else /* constexpr */ {
24 }
25
26 }
27 #endif
28
29
30 #ifdef INSIGHTS_USE_TEMPLATE
31 template<>
32 int sum1<3, 5, 1>()
33 {
34     if constexpr(false) {
35     } else /* constexpr */ {
36         return 3 + sum1<5, 1>();
37     }
38 }
39 #endif
40
41
42
43 #ifdef INSIGHTS_USE_TEMPLATE
44 template<>
45 int sum1<5, 1>()
46 {
47     if constexpr(false) {
48     } else /* constexpr */ {
49         return 5 + sum1<1>();
50     }
51 }
52 #endif
53
54
55
56 #ifdef INSIGHTS_USE_TEMPLATE
57 template<>
58 int sum1<1>()
59 {
60     if constexpr(false) {
61     } else /* constexpr */ {
62         return 1 + sum1();
63     }
64 }
65 #endif
66
67
68
69 int main()
70 {
71     std::operator<<(std::cout,
72     "sum1<>():\n").operator<<(sum1()).operator<<(std::endl);
73     std::operator<<(std::cout, "sum1<3,5,1>():\n").operator<<(sum1<3, 5,
74     1>()).operator<<(std::endl);
75     std::operator<<(std::cout, "====").operator<<(std::endl);
```

```

74 |   return 0;
75 | }

```

The function template `verbose::sum1<>()` (Listing 141) produces runtime output that illustrates the template instantiations.

Listing 141: `templates/verbose::sum1<>()` (VariadicTemplates.cpp)

```

26 |   template <int n = 0,int... pack> // whitespace after ...
27 |   auto sum1()
28 |   {
29 |       cout << "sum1<" << n << ">()" << endl
30 |           << "no of parameters: " << sizeof...(pack) << endl;
31 |       if constexpr (n == 0)
32 |       {
33 |           return 0;
34 |       }
35 |       else
36 |       {
37 |           return n + sum1<pack ...>();
38 |       }
39 |   }

```

The only difference to Listing 138 is that lines 29 and 30 have been added to the beginning of the function template. Since these lines do not depend on the `if constexpr` condition, they are included in each instantiation of the template. In line 29, the name of the function and the value for which is instantiated are sent to `cout`. To emphasize that this is an integral template parameter, it is included in angle brackets in the output. In line 30, the number of parameters contained in the template parameter pack is sent to `cout`. It is calculated by applying the operator `sizeof...` to its argument `pack`, which must be enclosed in parentheses. The operators `sizeof...` and `sizeof` should not to be confused, as only the first can be applied to a template parameter pack.

Listings 138 and 141 use recursion, which was introduced as the first mechanism for processing a template parameter pack. An alternative for processing a template parameter pack are so-called *fold expressions*. They are borrowed from the functional programming paradigm and enable very compact and concise solutions. In C++, a fold expression expands a parameter pack by applying a binary operator. C++ fold expressions are explained in detail in (*Fold Expression (since C++17) - Cppreference.Com* n.d.). Table 15 summarizes the four kinds of fold expressions.

Enclosing a fold expression in parentheses is mandatory. The $\otimes$ symbol is a placeholder for a binary operator. All 32 applicable binary operators are listed in (*Fold Expression (since C++17) - Cppreference.Com* n.d.). In a binary fold, both operators $\otimes$ must be equal.

The init expression *I* must not contain an unexpanded parameter pack or operators with a lower *precedence* (synonym for *priority*) than a cast expression. The pack expression *P* must contain an unexpanded parameter pack and, like the init expression,

Table 15: Fold expressions

Name	Syntax	Expands to
Unary right fold	$(P \otimes \dots)$	$(P_1 \otimes (\dots \otimes (P_{N-1} \otimes P_N)))$
Unary left fold	$(\dots \otimes P)$	$((((P_1 \otimes P_2) \otimes \dots) \otimes P_N)$
Binary right fold	$(P \otimes \dots \otimes I)$	$(P_1 \otimes (\dots \otimes (P_{N-1} \otimes (P_N \otimes I))))$
Binary left fold	$(I \otimes \dots \otimes P)$	$(((((I \otimes P_1) \otimes P_2) \otimes \dots) \otimes P_N)$

must not contain an operator with a lower precedence than a cast expression. This restriction is quite subtle. Greatly simplified, it means that parentheses must be used to resolve ambiguous expressions. For a more detailed explanation, see (Forsum 2016), (Bolas 2016), and (Sergey 2016). For information about operators, see (*C++ Named Requirements: RangeAdaptorObject (since C++20)* - *Cppreference.Com* n.d.). The subscript for P refers to the position of a parameter in the pack.

Obviously, a left fold expression is *left-associative* and a right fold expression is *right-associative*. *Left-associative* means that a complex expression is evaluated from left to right, e.g. as in the mathematical difference $5 - 3 - 1$, which results in 1. *Right-associative* means that a complex expression is evaluated from right to left. The assignment operator in C++ is right-associative, e.g. $a = b = 2$; first assigns 2 to b and then assigns the value of b —now 2—to a .

Listing 142 shows the implementation of `quiet::sum2<>()` using a binary left fold expression.

Listing 142: `templates/quiet::sum2<>()` (`VariadicTemplates.cpp`)

```

44 |     template <int ...pack> // whitespace before ...
45 |     auto sum2()
46 |     {
47 |         return (0 + ... + pack);
48 |     }

```

The fold expression is expanded *in place*, i.e. the compiler does not instantiate the function template for each individual integral constant, but creates an instantiation for each specific function call. Listing 143 shows the output created by *C++ Insights* for the function calls `sum2<>()` and `sum2<3,5,1>()`.

The code fragment shown in Listing 144 produces an output at runtime that illustrates the effect of the fold expression.

```

53 template <typename T,typename S>
54 auto repeated(const T& n,S remaining) // helper function
55 {
56     cout << n;
57     if (remaining > 0)
58     {
59         return ", "s;
60     }
61     else
62     {
63         return ""s;
64     }
65 }
66
67 template <int ... pack> // whitespace around ...
68 auto sum2()
69 {
70     auto remaining { sizeof...(pack) };
71     cout << "sum2<";
72     ((cout << '*' ) << ... << repeated(pack,--remaining))
73         << ">()" << endl
74         << "no of parameters: " << sizeof...(pack) << endl;
75     return (0 + ... + pack);
76 }

```

The variable `remaining` is initialized with the actual number of elements in `pack` by applying the special `sizeof...` operator to the template parameter `pack`, which evaluates the number of elements in the package at compile time. Later, `remaining` is used to produce more detailed output related to the instantiation and execution of the template.

First, the expression

```
((cout << '*' ) << ... << repeated(pack,--remaining))
```

in line 72 will be analyzed with respect to Table 15. As required, there is a pair of parentheses around the fold expression. It contains two subexpressions, one at the beginning and one at the end. It is therefore a binary fold. The operator used in the fold expression is `<<`, which is left-associative, so it must be a binary left fold. Therefore, `(cout << '*')` is the init expression *I* and `repeated<>(pack,--remaining)` is the pack expression *P*. The init expression `cout << '*'` must be enclosed in additional parentheses, otherwise the binary fold would be syntactically incorrect because it contains the operator `<<` three times. Consequently, the fold expression becomes

```
(((((cout << '*' ) << repeated(pack,--remaining))  
    << repeated(pack,--remaining)) << ...)  
    << repeated(pack,--remaining)).
```

The output of the asterisk should show the evaluation of the init expression *I*.

Second, the call to `(cout << '*')` and the auxiliary function template `repeated<>()` are examined in more detail. They are necessary to produce the desired output for the init expression *I* and for the pack expression *P* respectively. Both are not elegant and are not intended to be a blueprint for practice. They are merely simple means to achieve the desired output.

The expression `(cout << '*')` corresponds to the init expression *I* and is therefore called exactly once. It outputs an asterisk to `cout` to indicate that it has been evaluated. Its evaluation results in `cout`. Further output can thus be sent to `cout` with the stream insertion operator `<<`.

The function template `repeated<>()` defines two template parameters, namely *T* and *S*. The function parameter *n* is passed as a reference to `const` of type *T*. It is the number that is sent to `cout`. The value parameter `remaining` is of type *S* and represents the number of parameters in the pack. Passing it by value of type *S* eliminates the need to find out the exact return type of the `sizeof...` operator (which is actually `std::size_t`). If `remaining` is greater than 0, the function returns a `std::string` containing a comma, otherwise an empty string. It must be emphasized that the compiler creates a new instantiation of `repeated<>()` for each different type *T* (*S* will always be the same). Since *n* is passed as a reference to `const`, a normal `if` statement is used, which is evaluated at runtime. It should be mentioned that `repeated<>()` does not take the whole pack, even if it is specified as the first actual parameter. Only the first element of `pack` is taken as a reference to `const` of type template parameter *T*. The function template `verbose::sum4<>()`

is implemented in a similar way. It should be noted that a small function that is only used once can be defined as a lambda expression instead. Lambdas are briefly mentioned in Section *Lambda Expressions as Callback Functions* and described in detail in Section *Lambda Expression*.

In line 70, the local variable `remaining` is initialized with the current number of parameters in the pack. Line 72 contains the crucial change. Here, the fold expression `(0 + ... + pack)` is converted to

```
((cout << '*' ) << ... << repeated(pack,--remaining)).
```

As a side effect, an asterisk is printed exactly once after the left angle bracket. Next, every single integer constant is sent to `cout` followed by a comma, unless it is the last parameter in the pack. This is accomplished by passing `--remaining` to `repeated<>()`. The pre-decrement operator decrements the value of `remaining` *before* it is passed. Therefore, it will be 0 if the pack contains the last parameter. In this case, an empty `std::string` exemplar is returned.

So much for using a pack of constant template parameters, or—in this case—more precisely, integral template parameters. Now a pack of type parameters is used. Listing 145 shows the function template `quiet::sum3<>()`, which specifies a type template parameter named `T` with `double` as the default type and a pack named `Pack`. Again, recursion is used to implement the function template. Therefore, the function template accepts two parameters, `n` of type `T`, for which a default value is specified with `= T { }` and `pack`, which is of type `Pack` and is passed as a pack of references to `const`.

Listing 145: `templates/quiet::sum3<>()` (`VariadicTemplates.cpp`)

```
81     template <typename T = double,typename... Pack>
82     auto sum3(const T& n = T { },const Pack& ... pack)
83     {
84         if constexpr (sizeof...(pack) == 0)
85         {
86             return n;
87         }
88         else
89         {
90             return n + sum3(pack...);
91         }
92     }
```

In some publications, an additional overloaded function without parameters is provided to stop the recursive instantiation of the function template, e.g. (Breymann 2020). This becomes redundant if `T { }` is given as the default value for `n`. As already mentioned, this results in a new exemplar of type `T` being created, which is initialized with a value that is conceptually equal to 0. The same can be achieved by calling `T()` instead of `T { }`. Therefore, the last parameter in the package is passed as `T`. The interesting point is what happens if the `quiet::sum3<>()` function template is called without parameters?

As written before, the deduction of template arguments requires that the type template parameter is used to specify at least one function parameter! This problem is solved in two steps. First, if no function parameter is explicitly specified, the default value, here `T { }`, is used. Since the type `double` is specified as the default for `T`, this type is available for creating and passing the default value.

The rest of the implementation is very similar to the previous function templates. Therefore, no further explanation is given here.

The counterpart to `quiet::sum3<>()`, which produces runtime output (`verbose::sum3<>()`), can be seen in Listing 145.

Listing 146: `templates/verbose::sum3<>()` (`VariadicTemplates.cpp`)

```
97 |     template <bool once = true, typename T = double, typename... Pack>
98 |     auto sum3(T n = T { }, const Pack& ... pack)
99 |     {
100 |         if constexpr (once)
101 |         {
102 |             cout << "sum3(";
103 |         }
104 |         cout << n;
105 |         if constexpr (sizeof...(pack) == 0)
106 |         {
107 |             cout << ')' << endl;
108 |             return n;
109 |         }
110 |         else
111 |         {
112 |             cout << ',';
113 |             return n + sum3<false>(pack...);
114 |         }
115 |     }
```

To output each element of the pack at runtime, an additional constant template parameter is introduced: `once` is specified as type `bool` with `true` as default value. Lines 100 – 103 contain an `if` statement that is evaluated at compile time, as indicated by `constexpr` before the condition. If `once` is `true`, the function name and a left parenthesis are sent to `cout`. In the `main()` function, `sum3<>()` is called without its constant template parameter. Hence, this is the only instantiation of `sum3<>()` with `true` as the value for `once`. In line 104, `n` is sent to `cout`. If `sum3<>()` is instantiated with an empty parameter list, this results in the output of the default value specified for `T`, which is conceptually 0. Line 105 checks at compile time whether the pack is empty. If so, the closing parenthesis is sent to `cout` and `n` is returned. Otherwise, the `else` branch outputs a comma to `cout` and returns `n + sum3<false>(pack...)`. It is essential to explicitly specify `false` as constant template parameter here, because this will suppress the repeated output of `"sum3("` to `cout`.

In `quiet::sum4<>()`, the processing of the pack of type parameters with a fold expression is remarkably concise, as Listing 147 shows.

Listing 147: `templates/quiet::sum4<>()` (`VariadicTemplates.cpp`)

```

120 |     template <typename... Pack>
121 |     auto sum4(const Pack& ... pack)
122 |     {
123 |         return (0.0 + ... + pack);
124 |     }

```

In the binary left fold in line 123, the init expression *I* here is simply the double literal 0.0 and the pack expression *P* consists only of the *pack* pack. If `quiet::sum4<>()` is instantiated for a type other than `double`, the initialization value 0.0 is converted—through automatic conversion or type promotion—to a value corresponding to 0 for the type being used. Otherwise, an error is reported. All in all, this is a flawless binary left fold that needs no further explanation.

The verbose `verbose::sum4<>()` function template (Listing 148) adds a runtime output. Comparing Listings 148 and 144, it becomes clear that the changes are minimal. Since the function template expands a pack of types and not integrals, the angle brackets are omitted, and `sum2` is replaced by `sum4` in the output string. The `repeated<>()` function template is reused from the implementation of `sum2<>()`. There it is placed in the verbose namespace. Since this namespace is reopened for the definition of `verbose::sum4<>()`, it is automatically available and does not need to be imported explicitly.

Listing 148: `templates/verbose::sum4<>()` (`VariadicTemplates.cpp`)

```

129 |     template <typename... Pack>
130 |     auto sum4(const Pack& ... pack)
131 |     {
132 |         auto remaining { sizeof...(pack) };
133 |         cout << "sum4(";
134 |         ((cout << '*' ) << ... << repeated(pack,--remaining))
135 |          << ')' << endl;
136 |         return (0.0 + ... + pack);
137 |     }

```

Finally, the effects of the four kinds of fold expressions (see Table 15) are explained using an example program. For this purpose, function templates are defined that use fold expressions for subtraction. They are included in Listing 149 and have self-documenting names, such as `subtractionUnaryRightFold<>()`.

Any function template that uses a unary fold must be called with at least one parameter. For example, calling `subtractionUnaryRightFold<>()` will result in a compilation error because the parameter list is empty and no init expression is used in the fold. Calling

`subtractionUnaryRightFold(10,5,2)` returns 7 as the result. This means that the fold expression is unfolded as $(10 - (5 - 2))$. The call to `subtractionUnaryLeftFold(10,5,2)` returns 3 as the result, since the fold expression is unfolded as $((10 - 5) - 2)$.

Listing 149: templates/Folds.cpp

```
1 // Folds.cpp by Ulrich Eisenecker, June 29, 2023
2
3 #include <iostream>
4 using namespace std;
5
6 template <typename ... Pack>
7 auto subtractionUnaryRightFold(const Pack& ... pack)
8 {
9     return (pack - ...);
10 }
11
12 template <typename ... Pack>
13 auto subtractionUnaryLeftFold(const Pack& ... pack)
14 {
15     return (... - pack);
16 }
17
18 template <typename ... Pack>
19 auto subtractionBinaryRightFold(const Pack& ... pack)
20 {
21     return (pack - ... - 0);
22 }
23
24 template <typename ... Pack>
25 auto subtractionBinaryLeftFold(const Pack& ... pack)
26 {
27     return (0 - ... - pack);
28 }
29
30 int main()
31 {
32     // cout << subtractionUnaryRightFold() << endl; // error
33     cout << subtractionUnaryRightFold(10,5,2) << endl; // gives 7
34     cout << subtractionUnaryLeftFold(10,5,2) << endl; // gives 3
35     cout << subtractionBinaryRightFold() << endl; // gives 0
36     cout << subtractionBinaryRightFold(10,5,2) << endl; // gives 7
37     cout << subtractionBinaryLeftFold() << endl; // gives 0
38     cout << subtractionBinaryLeftFold(10,5,2) << endl; // gives -17
39 }
```

The corresponding function templates with binary fold expressions are `subtractionBinaryRightFold<>()` and `subtractionBinaryLeftFold<>()`. Both can be called with an empty parameter list since an init expression is provided. When `subtractionBinaryRightFold(10,5,2)` is called, the fold expression unfolds to $(10 - (5 - (2 - 0)))$, which gives 7 as the result. When `subtractionBinaryLeftFold(10,5,2)` is called, the fold expression unfolds to $((0 - 10) - 5) - 2$, which evaluates to -17.

In the C++ standard library, variadic templates are used multiple times to provide powerful abstractions, e.g. `std::format<>()`, which is part of the C++20 formatting library (*Formatting Library (since C++20) - Cppreference.Com* n.d.) and (*Overview - Fmt 8.1.1 Documentation* n.d.), or `std::tie<>()`, which is part of the `<tuple>` header file and creates a tuple of references to simplify otherwise complex assignments (*std::tie - Cppreference.Com* n.d.).

(Breymann 2020) and (Gregoire 2020) both give brief introductions to variadic templates. An early introduction to this topic is (Gregor and Järvi 2007), where one of the authors is the inventor of variadic templates. More comprehensive information can be found in (Vandevoorde, N. M. Josuttis, and Gregor 2018).

3.4. Concepts

Concepts are a novel way of specifying the requirements that a template parameter must meet. Concepts allow to write short yet safe generic code. The swapping of variables is used to illustrate this.

In Section *Implicit Requirements for a Template Parameter*, it was explained that templates define implicit requirements for their template parameters. Errors in template instantiation can result in remarkably long and cryptic error messages, which can vary from compiler to compiler. Therefore, `static_assert` was presented as a means to output well-defined error messages by the compiler.

The need to correctly formulate and verify the requirements for template parameter and to provide more precise error diagnostics has encouraged the development of *concepts*. It took a long time before concepts were finally adopted in C++ 20.

Listing 150 demonstrates the use of a `requires` clause in a template specification to explicitly describe the traits that the type template parameter `T` must satisfy. Instead of `is_trait<T>::value`, the shorter form `is_trait_v<T>` is used in the following examples. Both the individual requirement and a bundle of requirements are referred to as *constraints* in this context.

Listing 150: templates/SwapWithRequires.cpp (excerpt)

```

8 | template <typename T>
9 | void swapVariables(T& a, T& b) requires
10 |     is_destructible_v<T> &&
11 |     is_copy_constructible_v<T> &&
12 |     is_assignable_v<T&, T>
13 | {
14 |     T temp { a };
15 |     a = b;
16 |     b = temp;
17 | }
```

The `requires` keyword immediately follows the name of the template and contains a logical expression that specifies the constraints that the parameter, here `T`, must satisfy. Two aspects must be emphasized: Firstly, the `requires` clause is evaluated at compile time. It must therefore not contain any runtime component. Secondly, constraints can be formulated not only for type template parameters, but also for all other kinds of template parameters. In this example, every assertion formulated with `static_assert` in Listing 112 becomes part of the `requires` clause and is combined with the logical AND.

If a template parameter does not satisfy a condition of the `requires` clause, the compiler issues a corresponding concise error message. The author does not know whether these error messages are in standardized form or whether they can vary depending on the compiler. Therefore, the use of `static_assert` can still be appropriate if the error messages are to be specified by the developer.

Depending on the implementation of the meta programming library, `std::is_copy_constructible<>::value` also checks for the presence of the destructor, see e.g. (`std::is_copy_constructible`, `std::is_trivially_copy_constructible`, `std::is_nothrow_copy_constructible` - *Cppreference.Com* n.d.). Therefore, `std::is_destructible<>::value` is checked first in the `requires` clause. This guarantees that a deleted destructor is reported as such. If `std::is_copy_constructible<>::value` were checked first, it would also be evaluated as true for a class with a deleted destructor.

Assuming that the same bundle of constraints is used in the specification of another template, this would have to be done redundantly with this form of syntax. For this reason, it is possible to formulate a `requires` clause as a *concept* and give it its own name. Listing 151 demonstrates how to do this.

Listing 151: templates/SwapWithConceptAndRequires.cpp (excerpt)

```
8 | template <typename T>
9 | concept CanBeSwapped =
10 |     is_destructible_v<T> &&
11 |     is_copy_constructible_v<T> &&
12 |     is_assignable_v<T&,T>;
13 |
14 | template <typename T>
15 | void swapVariables(T& a,T& b) requires CanBeSwapped<T>
16 | {
17 |     T temp { a };
18 |     a = b;
19 |     b = temp;
20 | }
```

A concept is defined in a similar way to a template. Therefore, its definition begins with the specification of the parameters of the template, here in line 8. In line 9, the concept `CanBeSwapped` is defined using the keyword `concept`. The equal sign `=` is followed by the expression with the and-combined constraints. The definition of a concept must be terminated with a semicolon.

Now the template definition can refer to the concept `CanBeSwapped<T>` in its `requires` clause (line 15). In this way, several different template definitions can use the same concept.

Once a concept has been defined, an even shorter syntax is possible, which is shown in Listing 152.

Listing 152: `templates/SwapWithConcept.cpp` (excerpt)

```

8  template <typename T>
9  concept CanBeSwapped =
10     is_destructible_v<T> &&
11     is_copy_constructible_v<T> &&
12     is_assignable_v<T&,T>;
13
14  template <CanBeSwapped T>
15  void swapVariables(T& a,T& b)
16  {
17     T temp { a };
18     a = b;
19     b = temp;
20  }
```

Interestingly, the developers of the C++ language came up with an even tighter syntax that makes the definition of the function template look completely different, as shown in Listing 153.

Listing 153: `templates/SwapNewSyntax.cpp` (excerpt)

```

8  template <typename T>
9  concept CanBeSwapped =
10     is_destructible_v<T> &&
11     is_copy_constructible_v<T> &&
12     is_assignable_v<T&,T>;
13
14  void swapVariables(CanBeSwapped auto& a,CanBeSwapped auto& b)
15  {
16     CanBeSwapped auto temp { a };
17     a = b;
18     b = temp;
19  }
```

Here, the concept name is used in combination with the `auto` keyword to specify the type of the function parameters `a` and `b` and the local variable `temp`. For `a` and `b` template argument deduction is applied, for `temp` type deduction. In this way, the explicit declaration of a function template with the `template` keyword becomes superfluous.

Unfortunately, `swapVariables<>()` can be instantiated incorrectly, e.g. `double a { 1.0 };int b { 2 }; swapVariables(a,b);`, which swaps variables of two different types. This is possible because the `auto` specification allows each parameter to be instanti-

ated with a different type. As Listing 154 shows, this bug can be fixed. An additional requires clause is added to the function template, which uses the `decltype` operator and the `std::is_same_v<>` template variable to check whether `a` and `b` are declared with the same type.

Listing 154: templates/SwapImproved.cpp

```
1 // SwapImproved.cpp by Ulrich Eisenecker, June 29, 2023
2 // Warning: Program does not compile!
3
4 #include <iostream>
5 #include <type_traits>
6 using namespace std;
7
8 template <typename T>
9 concept CanBeSwapped =
10     is_destructible_v<T> &&
11     is_copy_constructible_v<T> &&
12     is_assignable_v<T&,T>;
13
14 void swapVariables(CanBeSwapped auto& a, CanBeSwapped auto& b)
15     requires is_same_v<decltype(a),decltype(b)>
16 {
17     CanBeSwapped auto temp { a };
18     a = b;
19     b = temp;
20 }
21
22 int main()
23 {
24     double a { 99 };
25     swapVariables(a,a); // legal
26     int b { 42 };
27     swapVariables(a,b); // illegal
28 }
```

Now the compiler reports an error when instantiating `swapVariables<>()` with arguments of different types.

If a class is defined that does not satisfy any of the requirements included in the `CanBeSwapped` concept, only the first checked requirement is reported as an error. When the program shown in Listing 155 is compiled, the compiler only reports the missing destructor of the `AllRequirementsMissing` class as an error.

Listing 155: templates/SwapIncompleteDiagnostics.cpp (excerpt)

```

1 // SwapIncompleteDiagnostics.cpp by Ulrich Eisenecker, June 29, 2023
2 // Warning: Program does not compile!
22 class AllRequirementsMissing
23 {
24     public:
25         AllRequirementsMissing() = default;
26         AllRequirementsMissing(const AllRequirementsMissing&) = delete;
27         AllRequirementsMissing& operator=(const AllRequirementsMissing&) = delete;
28         ~AllRequirementsMissing() = delete;
29 };
30
31 int main()
32 {
33     AllRequirementsMissing * p1 { new AllRequirementsMissing },
34                             * p2 { new AllRequirementsMissing };
35     swapVariables(*p1,*p2);
36 }

```

As stated in (*Constraints and Concepts (since C++20)* - *Cppreference.Com* n.d.), the compiler evaluates a conjunction of constraints from left to right and applies Boolean short-circuit evaluation. Therefore, the evaluation is terminated as soon as the first requirement is evaluated as false. By the way, it is also possible to combine constraints in a disjunction. This disjunction is evaluated left-associatively as well, and Boolean short-circuit evaluation is also applied.

At least for *g++*, the diagnostics can be improved by reformulating the *CanBeSwapped* concept using a sequence of requirements instead of a conjunction, see e.g. (*Requires Expression (since C++20)* - *Cppreference.Com* n.d.). Listing 156 shows the correspondingly modified *CanBeSwapped* concept. When compiling the program *SwapCompleteDiagnostics.cpp*, the compiler now correctly reports all requirements that are not met by the *AllRequirementsMissing* class.

Listing 156: templates/SwapCompleteDiagnostics.cpp (excerpt)

```

8 template <typename T>
9 concept CanBeSwapped = requires
10 {
11     requires is_destructible_v<T>;
12     requires is_copy_constructible_v<T>;
13     requires is_assignable_v<T&,T>;
14 };

```

clang++ (version 22.1.7) stops compilation as soon as it encounters the first unmet requirement. It is not clear whether this behavior is compiler-specific for *g++* or *clang++* or how *clang++* can be forced to report all requirements that are not satisfied.

As mentioned earlier, the use of concepts is not restricted to type template parameters. Listing 145 introduces the `quiet::sum4()` function template, which computes the sum of an arbitrary number of arguments using a template parameter pack and a fold expression. However, it is possible to instantiate this function template for any type, even if it is not an arithmetic type. Listing 157 shows how to overcome this shortcoming.

Listing 157: `templates/SumWithConcept.cpp` (excerpt)

```
9 | template <typename T>
10 | concept arithmetic =
11 |     integral<T> ||
12 |     floating_point<T>;
20 |
21 | auto sum(const arithmetic auto& ... pack)
22 | {
23 |     return (0.0 + ... + pack);
24 | }
```

The C++ standard defines a number of useful concepts in the header file `<concepts>`. Since there is no concept `arithmetic`, so the two concepts `integral<T>` and `floating_point<T>` have been combined by a logical OR. In this way, `sum<>()` can be instantiated for any mixture of integral or double types, but not for other types.

Interestingly, the `<type_traits>` header defines the `is_arithmetic<>` template meta function. Using this trait, the concept definition in Listing 157 can be rewritten as shown in Listing 158.

Listing 158: `SumWithConcept.cpp`—modified (excerpt)

```
1 | template <typename T>
2 | concept arithmetic =
3 |     is_arithmetic_v<T>;
```

Both possibilities are completely equivalent. They have been included to show how new concepts can be defined using type traits or on the basis of already defined concepts. There is another way to describe requirements, namely through exemplary programmatic uses, which is not shown here.

It should be emphasized that the above text only gives a brief overview of concepts. Further reading can be found, for example, in (Dos Reis and Stroustrup 2006), (Sutton and Stroustrup 2011), (ACCU 2015) and (ACCU 2016).

3.5. Final Advice for Implementing Templates

Implementing templates can be a difficult task. Whenever a task is difficult, a proven strategy is to break down the task and find a solution for the simplest subtask first, expanding it in subsequent iterations until the entire task is completed.

This also applies to templates. If there is more than one template parameter or template-template parameters or parameter packs occur, it is recommended to implement a concrete function or class without template parameters in the first step. Next, code for unit tests is added. Apart from trivial cases, it is advisable to use the classic syntax for specifying a template parameter. Using `auto` in combination with a concept is usually not the best start, as the search for a suitable concept can be time-consuming. Once a concept has been found, it should be checked thoroughly to ensure that it really fulfills the intended requirements completely. The correct implementation of a new concept is even more time-consuming. Therefore, the use of `auto` and concepts should be avoided unless it is a very common and well-known case.

In the next step, the existing simple solution is templated with regard to only one template parameter. After instantiation, the same unit tests can be performed as before. This is a good opportunity to add further unit tests in terms of the variability that the newly introduced template parameter allows.

This is repeated until all the originally desired template parameters have been successfully implemented and tested.

A pre-final step may be to determine reasonable defaults for template parameters and provide them. This step may require a revision of the order in which template parameters are specified and corresponding changes in the unit tests.

As a final measure, an attempt can be made to simplify the interface of how template parameters are specified and how they need to be instantiated. This applies in particular to the use of `auto` in combination with a concept. Of course, it should not be forgotten to test these changes as well.

This section concludes the rough overview of the generic programming paradigm as realized in the C++ 20 standard. Much was left out, for example *template variables* were not mentioned at all. Some more details on generic programming are covered in the rest of the book as needed. Nowadays, generic programming is becoming more and more important, especially in conjunction with the functional programming paradigm, as it allows to write type-safe and very compact programs. In this way, the object-oriented paradigm (see Section *The Object-Oriented Programming Paradigm*) is losing its dominance, but not its importance for programming, especially in combination with the other programming paradigms.

4. Containers, Iterators and Algorithms

Many problems require programs that deal with a different number of elements of the same type, e.g.

- strings ((U. W. Eisenecker 2022), Section 3.3.3.1 *Return Type void*),
- variables or rules in an inference engine for expert systems ((U. W. Eisenecker 2022), sections 2.3.1 *Variables*, 5.5.5 *Variables Class* and 5.5.8 *Rule Class*),
- memory addresses of serialized or deserialized objects (sections *Static Members* and *Implementation of Object Persistence With reinterpret_cast<>()*) or
- child widgets (Section *Base Classes of MyWindow*).

Generally speaking, a *container* manages a variable number of elements of the same type. In this respect, a container complements a *structure* that combines related elements of possibly different types. A structure can be mapped to programming concepts such as an abstract data type or a class, which can be seen as the basis for programming paradigms such as structured programming and object-oriented programming.

In the last decade of the past millennium, the *Standard Template Library*, or *STL* for short, became part of the libraries that were later adopted as the C++ 98 standard. The *STL* was revolutionary in that it introduced *containers*, *iterators* and *algorithms*. Iterators effectively decouple algorithms from containers by providing a unifying mechanism for accessing their elements. This allows almost any algorithm to be applied to almost any container.

After the *STL* was incorporated into the C++ standard library, its parts were further developed along with the rest of the C++ standard library. Today, it is difficult to clearly distinguish the *STL* from the rest of the C++ standard library. The use of containers and algorithms has been simplified by concepts, ranges and views introduced with the C++20 standard.

The containers and algorithms of the C++ standard library follow generic polymorphism, i.e. the generic programming paradigm. They are not a fundamental programming paradigm like structured programming or object-oriented programming. Rather, they can be considered a sub-paradigm or embedded paradigm.

After a brief overview of the classic C-array for managing multiple elements of the same type, a few selected containers and algorithms are introduced. Where available, features from C++20 or newer versions, such as concepts, ranges and views, are used to simplify the use of containers and algorithms. The chapter ends with a case study on the use of containers and algorithms for statistical analysis.

A didactical note must be made. In the early days of the *STL*, C++ did not offer lambda expressions, let alone generic lambda expressions. Therefore, everything that can be done with the *STL* can be done without lambdas. Given the current state of programming, it would have been very unnatural not to use lambdas here. As a result, (generic) lambdas occur frequently. This is somewhat unfortunate, as lambdas belong to functional programming, where they are explained in detail (see Section *Lambda Expression*).

When used to simplify the code examples in this chapter and give them a modern touch, they are briefly introduced in relation to the requirements that are essential for the respective example. For didactic reasons, forward references should actually be avoided. Ultimately, however, priority was given to writing code in a modern style. Another option would have been to swap the order of this chapter with that on functional programming. However, there were several reasons against this, which will not be discussed further here.

4.1. The Classic C-Array

This section introduces the classic C-array and various ways of accessing its elements using a fictional example.

Listing 159 shows a program that declares the integer C-array values, which can be imagined as a single draw of size 10 of an urn with elements 0 to 9 without repetition, taking into account the order of the drawn elements (see Section *Urn With Order And Without Repetitions*). It illustrates some properties of C-arrays without going into too much detail.

Listing 159: cia/ClassicArray.cpp

```

1 // ClassicArray.cpp by Ulrich Eisenecker, May 16, 2025
2
3 #include <iostream>
4 #include <iterator> // Because of begin() and end().
5
6 using namespace std;
7
8 const unsigned int maxSize { 10 };
9
10 int main()
11 {
12     int values[maxSize] { 9, 0, 7, 2, 8, 1, 6, 3, 4, 5 };
13     cout << "Memory used by values: "
14           << sizeof(values)
15           << endl;
16     // Output elements
17     {
18         cout << "Elements:";
19         for (unsigned int i { }; i < maxSize; ++i)
20         {
21             cout << ' ' << values[i];
22         }
23         cout << endl;
24     }
25     // Reversing order of elements
26     {
27         cout << "Reversing order of elements ..." << endl;
28         auto start { 0u };
29         auto stop { maxSize / 2 };
30         while (start < stop)
31         {
32             auto temp { *(values + start) };
33             *(values + start) = *(values + maxSize - start - 1);
34             *(values + maxSize - start - 1) = temp;
35             ++start;
36         }
37     }
38     // Output elements
39     {
40         cout << "Elements:";
41         auto iter { begin(values) };
42         while (iter != end(values))
43         {
44             cout << ' ' << *iter++;
45         }
46         cout << endl;
47     }
48 }

```

In the first line of the `main()` function, `values` is defined as an array of integer elements by placing `int` before the `values` identifier. The important part of the syntax here is the pair of square brackets, `[]`, which indicates the declaration of an array. Inside the brackets, the constant `maxSize` of type `unsigned int` instructs the compiler to allocate contiguous memory for 10 `int` elements. To the right, a list in curly brackets specifies ten integer literals that are used to initialize the elements of the `values` array. The presence of this initializer list makes the use of `maxSize` within the square brackets unnecessary, as the compiler can also derive the size of the array from the number of elements in the initializer list. If the initializer list contains fewer than `maxSize` elements, the missing elements are initialized to zero. If the initializer list contains more elements than specified by `maxSize`, the compiler reports an error.

Starting with the first element at position zero, each element is initialized with the corresponding integer literal from the initialization list. This means that each array element is a copy of the corresponding integer literal. Once the elements of the array are fully initialized, it is irrelevant what happens to the values that were used for initialization. The array has its own elements. If a different value is assigned to a specific element of the array, the element changes but still belongs to the container. Therefore, it can be said that *the array owns its elements*. This important property applies to all container templates: ***in general, a container owns its elements***.

The next statement (lines 13 – 15) outputs the memory occupied by `values`. This is done by applying the `sizeof` operator to `values`. Assuming that `int` has a size of 4 bytes on a given platform, 40 is displayed in the terminal window. This is only possible because `values` is declared within this scope. Only within the declaring scope does the compiler know that `values` consists of ten elements—as specified by `maxSize`.

Although `values` is not explicitly declared as `const`, it is an implicit constant pointer to the first element of the array. As the pointer is constant, it must not be changed, e.g. by incrementing, decrementing or assigning. Otherwise, the reference to the first element of the array would be lost. However, after creating a copy of `values`, e.g. `auto valuesCopy { values };`, the compiler no longer has the information that `valuesCopy` points to the first element of an array with ten elements. Therefore, `sizeof(valuesCopy)` would give the size of a pointer, e.g. 8 on a computer with 64-bit architecture.

For this reason, it is always advisable to store the size of a particular array in addition, as is done here with the global constant `maxSize` (line 8). If `values` were passed to a function as a value or as a reference to `const`, its implementation could use `maxSize` to determine the number of elements to which the passed pointer validly points. If `values` were allocated on the heap rather than statically on the stack, this would not change the fact that the number of elements of the dynamically allocated array cannot be recovered from the pointer to its first element. Unless specified as a global constant that applies to all arrays, the number of elements in an array must be specified as an additional parameter when a pointer to its first element is passed to a function.

If a different number of elements were required, `maxSize` would have to be adjusted accordingly—at least in the case of a statically declared array.

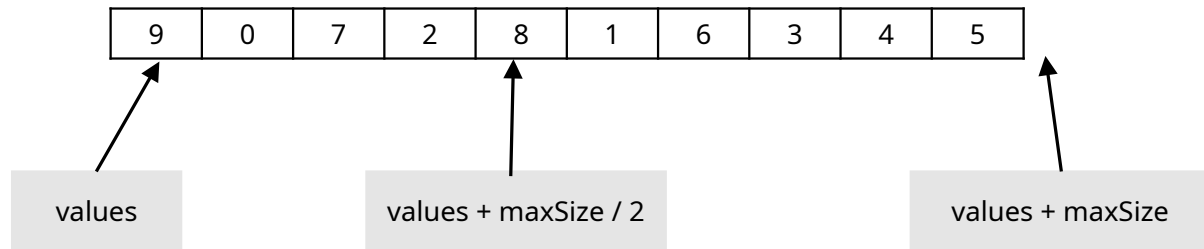
Further details about C-arrays could be mentioned, such as their alignment in the program memory, the actual size of an array when its elements occupy less than the word width of the processor or less than one byte, and so on. However, this will not be covered here. The focus of the following explanations is on accessing the elements of values and implementing simple algorithms.

The following block (lines 16 – 24) sequentially outputs the elements of values one after the other, which can be considered the first algorithm. It uses a classic for loop ((U. W. Eisenecker 2022), Section 3.1.1.1 *bool Type*) to access each element with the index operator, `[]`. In the loop header, the index variable `i` is initialized to zero, since a C-array always starts with index 0. As long as `i < maxSize` is true, the loop statement is executed. Each element at position `i` is sent to `cout`. Finally, the third part of the loop header causes `i` to be incremented by 1. This shows that the `maxSize` elements of values are located at positions 0 to `maxSize - 1`. Since the position of the elements is not determined by the array itself, the elements are said to be *externally ordered*.

Accessing the elements of the array values with the index operator is easy as long as the valid range of index positions from 0 to `maxSize - 1` is maintained. This is important because it is not checked whether a particular index position is valid. An important detail is that the elements can be accessed in any order. This form of access using the index operator is referred to—perhaps somewhat misleadingly—as *random access*.

In the next block (lines 25 – 37), another algorithm is implemented that reverses the order of the elements of values. For this purpose, so-called *pointer arithmetic* is used. This is only for illustrative purposes, as the code would be easier to read if the index operator were used instead (which also uses pointer-arithmetic, only less explicitly). The variable `start` is the loop variable, as its value is changed when the following while loop is executed. It is initialized to 0. The variable `stop` is initialized to `maxSize / 2`. It is only used in the condition of the while loop and prevents the otherwise necessary repeated evaluation of `maxSize / 2`. The while loop is executed as long as `start` is less than `stop`. This prevents the middle element from being swapped if the number of elements in values is odd. Figure 45 shows the beginning (values), the middle (values + `maxSize / 2`) and the position directly after the end (values + `maxSize`) of values.

In the body of the while loop, the auxiliary variable `temp` is initialized with the expression `*(values + start)`. This is where the pointer arithmetic comes into play for the first time. As already mentioned, the name of the array values is a constant pointer to the first element of the array. When the integer variable `start` is added to values, the compiler generates code that results in a new pointer that is advanced by the position of `start` (initially 0) relative to the index of values (which is always 0).

Figure 45: Selected positions in the C-array values

It is important to understand that `start` has no meaning as a memory address or byte offset, but rather acts as an increment of an element index that automatically takes into account the size of a single element.

Since the result of the pointer arithmetic is a pointer, this pointer must be dereferenced to obtain the element it points to. For this reason, the dereference operator, `*`, is placed before `(values + start)`. The pointer arithmetic expression must be placed in parentheses to enforce the desired order of evaluation. The initialization expression could be simplified to `auto temp { values[start] };`, which would be very useful for a better understanding of the program, but would be pointless for illustrating pointer arithmetic.

In the next statement, pointer arithmetic is used twice to swap the elements at positions `values[start]` and `values[maxSize - start - 1]` (the pointer arithmetic expressions have already been rewritten using the index operator). Please note that `values` has two meanings here: first, the name of the array, and second, the name of the pointer that points to the first element of the array. The expression `(values + start)` returns a pointer that points to the first element after the position `values`, and `(values + maxSize - start - 1)` initially points to the last element of `values`. As `start` is then incremented by 1, the resulting position is decremented by 1 in each subsequent iteration of the while loop.

In the first assignment of the swap, the pointer to the left operand runs sequentially from the first element of `values` upwards to the element at the position `maxSize / 2`, while the pointer to the right operand runs downwards from the last element of `values`. These two forms of sequential access are only due to the chosen algorithm, since the pointer type in this case would also allow random access to the elements.

The last block (lines 38 – 47) outputs the elements of `values` again, this time using iterators to access the elements. After including the header file `<iterator>` (or another of the container-related header files), the function templates `begin<>()` and `end<>()` are available. The auto variable `iter` is initialized with `begin(values)`, which results in an iterator pointing to the first element of `values`. In this case, this iterator is a pointer to `int` pointing to the same element as `values`. Since it is a non-constant copy, it can be modified, which happens later.

The condition of the while loop checks whether `iter` differs from `end(values)`. The evaluation of `end(values)` results in an iterator that points exactly to the position after the last element of `values` (`values + maxSize` in Figure 45). Since this element does not exist, it is not permitted to dereference this iterator. It is only used to represent the end of the array. Like `iter`, the iterator resulting from the call to `end(values)` is a pointer to `int`. Since the value of `end(values)` never changes, it would make sense for performance reasons to remove this function call from the loop by initializing a variable, e.g. named `stop`, with it and then using `stop` in the loop condition. The only statement in the body of the while loop is `cout << ' ' << *iter++;`. First, a space is sent to the standard output stream. Then the iterator or pointer `iter` is dereferenced by applying the dereference operator, `*`, and the resulting value is also sent to standard output. Finally, pointer arithmetic is applied by calling the post-increment operator on `iter`, which advances the pointer to the next position.

This algorithm iterates through the elements of the `values` array in strictly sequential order. Therefore, the only operation that is applied to the pointer or iterator `iter` is the post-increment operator.

Both ways of outputting values are common programming practice in *C* (not counting the use of `begin<>()` and `end<>()`). The second form shows that pointers and iterators can be used in a relatively uniform way. It is the standard way to access elements of containers. When an algorithm is implemented using iterators to access container elements, it can be applied to all container templates that provide the required iterator types, including the operations defined for them.

A *C*-array is a well-known and frequently used means of managing a series of elements of the same type. Nevertheless, it is a relatively crude concept that does not exhibit the desired characteristics of a module, such as a clear separation between interface and implementation and the hiding of information. Furthermore, there are problems for which an array is not an optimal solution, for example, when the number of elements can vary at runtime. A metaphorical comparison would be to equate the *C*-array with a traditional cast in *C*: it can do almost anything, but it is not immediately apparent how a particular use is intended. For this reason, classic *C*-arrays are not used in this book. Instead, the more powerful concepts of containers and algorithms of the *C++* standard library, including ranges and views, are explored as they combine modular design with generic polymorphism. However, it should be noted that a *C*-array may be the most efficient container for certain problems.

4.2. Migrating from Array to Vector<>

The next step is to replace the C-array from Listing 159 by a `std::vector<>` from the C++ standard library. Listing 160 shows the program with the necessary adaptations.

Listing 160: cia/VectorA.cpp

```
1 // VectorA.cpp by Ulrich Eisenecker, July 28, 2025
2
3 #include <iostream>
4 #include <vector>
5
6 using namespace std;
7
8 int main()
9 {
10     vector values { 9, 0, 7, 2, 8, 1, 6, 3, 4, 5 };
11     cout << "Memory used by values: "
12         << sizeof(values)
13         << endl;
14     // Output elements
15     {
16         cout << "Elements:";
17         for (decltype(values)::size_type i { }; i < values.size(); ++i)
18         {
19             cout << ' ' << values[i];
20         }
21         cout << endl;
22     }
23     // Reversing the order of elements
24     {
25         cout << "Reversing order of elements ..." << endl;
26         decltype(values)::size_type start { 0 };
27         auto stop { values.size() / 2 };
28         while (start < stop)
29         {
30             auto temp { *(values.begin() + start) };
31             *(values.begin() + start) = *(values.end() - start - 1);
32             *(values.end() - start - 1) = temp;
33             ++start;
34         }
35     }
36     // Output elements
37     {
38         cout << "Elements:";
39         auto iter { begin(values) };
40         while (iter != end(values))
41         {
42             cout << ' ' << *iter++;
43         }
44         cout << endl;
45     }
46 }
```

To use `std::vector<>`, the header file `<vector>` must be included. This header file also provides `begin<>()` and `end<>()`, so `<iterator>` does not need to be included. Since a `vector<>` dynamically adjusts its size, it is not necessary to define a `maxSize` constant. It should be emphasized that `vector<>` does not correspond to or implements the mathematical concept of a *vector*. It merely refers to an abstraction that is more flexible than an array because it can adjust its size at runtime.

In `main()`, template argument deduction for class templates (see Section *Class Templates*) is used to declare a `vector<>` that is automatically instantiated for `int` and initialized with the list containing the elements of the fictitious draw from an urn.

The statement after the declaration of `values` outputs the size of `values`. This is very likely to differ from the size of the `C`-array. The reason for this is that a `vector<>` stores its elements differently than a `C`-array, e.g. by using memory space on the heap. The `sizeof` operator does not take into account the memory space dynamically allocated by a `vector<>`. Furthermore, a `vector<>` can contain additional private data members. Therefore, applying the `sizeof` operator to a container is more or less pointless, as it does not indicate how much memory the container and its elements actually occupy.

A significant change has been made in the block that outputs the elements of `values`. The `for` loop uses a complex expression for declaring the loop variable `i`, namely `decltype(values)::size_type`. The reason for this is, that each container manages a type member for representing sizes as a public member type alias called `size_type`. Since no type alias was defined for the declaration of `values`, the `decltype` operator is applied to the variable `values` to determine its type. The scope operator is then used to access `vector<>::size_type`.

It would have been possible to declare `i` as type `int` or `unsigned int` or another suitable type. However, this might require conversions to the type expected by `values` when `i` is used to access an element. Since the original type of a member type can vary, it is advisable to use a member type of a container whenever it is available. Normally, `vector<>::size_type` is `std::size_t`, but this is not guaranteed.

The condition of the `for` loop is `i < values.size()`, since the member function `vector<>::size()` returns the number of elements currently stored in the vector. Instead of calling the member function, `size(values)` could also be used. ***In modern C++, the use of the free function templates (if available) is preferred over the use of the corresponding member functions*** for several reasons that are not explained in detail here. However, the following programs use the free function templates are used—if available—, since not all algorithms have yet been adopted by the ranges library.

The loop body remains unchanged. This is possible because `vector<>` also provides the index operator, `[]`, as a member function. As with the `C`-array, the use of the index operator is not checked. If checked access is required, the member function `vector<>::at()` should be used. This throws an exception if called with an invalid index.

The next block reverses the order of the vector elements.

The variable `start` is declared and initialized with 0. To avoid type mismatch, the type of values is first determined using the `decltype` operator, as before. The scope operator is then applied to access the member type `size_type`. In this way, the type of `start` is exactly the integer type to be used as the index. The variable `stop` is declared and initialized in the same way in the block that reverses the order of the elements in Listing 159. The following algorithm is implemented as the reversal of the order in Listing 159, with the change that `values.begin()` is used instead of `values` and `values.end()` instead of `values + maxSize`. The underlying mechanism is the same pointer arithmetics that applies to iterators.

The last block, which outputs the elements of values again, is an exact copy of the corresponding code in Listing 159. Both times, the function templates `begin<>()` and `end<>()` are used instead of the corresponding member functions.

Overall, the program `cia/VectorA.cpp` replaces the C-array values in `cia/ClassicArray.cpp` with a `std::vector<>` of the same name, and all uses of values as pointer to the first element with `values.begin()` (or `begin(values)`) and all uses of `values + maxSize` with `values.end()` (or `end(values)`). This demonstrates how a program that uses a classic C-array can be transformed into a program that uses `vector<>`.

4.3. Use of Algorithms From the C++ Standard Library

Next, the above algorithm implementations are gradually replaced by algorithms from the C++ standard library. Listing 161 replaces the user-defined swap of two vector elements with `std::swap<>()` and the for loop that outputs the vector elements with `std::for_each<>()`.

Listing 161: `std::swap<>()` and `std::for_each<>()` in `cia/VectorB.cpp` (excerpt)

```
1 // VectorB.cpp by Ulrich Eisenecker, July 28, 2025
2
3 #include <algorithm> // Because of swap<>() and for_each<>().
4 #include <iostream>
5 #include <vector>
6
7 using namespace std;
8
9 int main()
10 {
11     vector values { 9, 0, 7, 2, 8, 1, 6, 3, 4, 5 };
12     cout << "Memory used by values: "
13         << sizeof(values)
14         << endl;
15     // Output elements
16     {
17         cout << "Elements:";
18         for (auto element : values)
19             {
```

```

20         cout << ' ' << element;
21     }
22     cout << endl;
23 }
24 // Reversing the order of elements
25 {
26     cout << "Reversing order of elements ..." << endl;
27     auto start { size(values) };
28     auto stop { start / 2 };
29     while (start > stop)
30     {
31         swap(*(end(values) - start),*(begin(values) + start - 1));
32         --start;
33     }
34 }
35 // Output elements
36 {
37     cout << "Elements:";
38     for_each(begin(values),end(values),[](auto element)
39             { cout << ' ' << element; });
40     cout << endl;
41 }
42 }

```

The use of `swap<>()` explicitly expresses what the corresponding code does, but only represents a minor simplification. The use of `for_each<>()` replaces a while loop. This programming style is referred to as *abstraction of control structures* and gives the program a *declarative* character. In this respect, it is an interesting improvement.

However, there are also disadvantages: The need to explicitly specify `begin(values)` and `end(values)` seems unnecessary, and last but not least, a lambda expression, more precisely a generic lambda, is used to specify the action to be applied to each element.

Lambdas are briefly mentioned in the Section *Lambda Expressions as Callback Functions* and explained in detail in the Section *Lambda Expression*. The empty pair of square brackets at the beginning indicates that no variables from the calling environment are made available to the lambda. The use of an `auto` parameter indicates that this is a generic lambda. The parameter enclosed in parentheses, named `element`, is passed by value and is sent to `cout` in the lambda. Since no return value is specified, the lambda implicitly returns `void`. Even if the elements of `values` were of a different type, this code block would not need to be changed as long as a stream insertion operator is available for the type of the elements.

Compared to a range-based for loop, the implementation with `for_each<>()` can be considered inferior, since the range-based for loop is also declarative and avoids the use of `begin(values)`, `end(values)` and the generic lambda.

The program `cia/VectorC.cpp` (which is only included in the accompanying material) replaces the entire block for reversing the order of the elements of `values` with the statement `reverse(begin(values),end(values));`. `std::reverse<>()` is a function tem-

plate that is also defined in the `<algorithm>` header. It requires the availability of a bidirectional iterator that can iterate through the elements of a container in both forward and backward directions. Applying `std::reverse<>()` to a container changes the state of the container. This is a so-called *mutating algorithm*.

After calling a mutating algorithm, all iterators previously obtained for the corresponding container, become invalid. If necessary, they must be obtained again. Using invalid iterators is an error that can be difficult to detect and locate.

If modifying the container is not desired and the only goal is to output the elements once in reverse order, an alternative would be to use backward iterators instead of forward iterators when calling `for_each<>()`, i.e.

```
for_each(rbegin(values), rend(values),
[](auto element){ cout << ' ' << element; }));
```

4.4. Further Simplification by Using Ranges

The program presented in Listing 160 and the `cia/VectorB.cpp` program are somewhat inconvenient, because an algorithm always requires the specification of an iterator pair, namely a begin and an end iterator or a start and a stop iterator. Specifying them is essential if the algorithm is only to be applied to only part of the container, but redundant and even error-prone if all elements of a container are to be processed. The second line of Table 16 shows how `begin()` and `end()` refer to all elements of a container, while iterators with the hypothetical names `start` and `stop` (line 3) can refer to a contiguous range of container elements. What is missing is a concept to simply referring to all elements of a container (line 2).

Table 16: `begin()/end()` and `start/stop` iterators

<code>begin()</code>										<code>end()</code>
9	0	7	2	8	1	6	3	4	5	
		<code>start (= begin() + 2)</code>					<code>stop (= begin() + 7)</code>			

The ranges library extends and generalizes the algorithm and iterator libraries. A first and simple application is the use of `reverse<>()` and `for_each<>()`, which are provided by the ranges library. Instead of a pair of iterators, they accept a container (in fact, they accept a range, but this will not be discussed further here). The ranges-specific algorithms are contained in the namespace `std::ranges`. To make it clear when a ranges-algorithm is being used, it is prefixed with either `std::ranges::` or `ranges::` if the namespace `std` has been imported previously. This is necessary anyway if header files with both versions of the algorithms are included, because the compiler selects the algorithm that accepts a pair of iterators and tries to convert the provided container to an iterator, complaining that too few arguments were provided.

The program in Listing 162 demonstrates the use of ranges-algorithms.

Listing 162: `std::ranges::reverse<>()` and `std::ranges::for_each<>()` in `cia/VectorD.cpp` (excerpt)

```

1 // VectorD.cpp by Ulrich Eisenecker, July 28, 2025
2
9 int main()
10 {
24     // Reversing the order of elements
25     ranges::reverse(values);
26     // Output elements
27     {
28         cout << "Elements:";
29         ranges::for_each(values, [](auto element)
30                             { cout << ' ' << element; });
31         cout << endl;
32     }
33 }
```

Comparing this program with the original versions in Listings 159 and 160, it becomes obvious that much less code is required and that the resulting code is easier to read and modify, which is also due to its declarative nature.

One way to reverse the order without changing the container is to use a view, as shown in Listing 163.

Listing 163: `std::ranges::reverse_view<>` and `std::ranges::reverse` in `cia/VectorE.cpp` (excerpt)

```

24 // Output elements in reversed order
25 {
26     // Remember lambda in a variable.
27     auto lambda { [](auto element)
28                     { cout << ' ' << element; }
29                 };
30     // 1st version
31     cout << "Elements:";
32     ranges::for_each(ranges::reverse_view(values), lambda);
33     cout << endl;
34     // 2nd version
35     cout << "Elements:";
36     ranges::for_each(values | ranges::views::reverse, lambda);
37     cout << endl;
38 }
```

While a range is a concept that defines specific requirements for iterations, i.e. it provides abstractions for iterations, a view is an *adapter for a range* that can apply a function to the elements of the underlying range or container without changing them. ***Unlike a container, a view does not own the elements.*** Views are also defined in the

`<ranges>` header in the `std::ranges` or in the `std::ranges::views` namespace. The alias `std::views` is also available for the latter (see (*Ranges Library (since C++20)* - *Cppreference.Com* n.d.)).

The first version (line 32) creates a temporary `reverse_view` exemplar that is initialized with `values`. To emphasize this, the expression `ranges::reverse_view(values)` could also have been written as `ranges::reverse_view { values }`, but this is not common. The temporary `reverse_view` exemplar adapts `values`, but does not own the elements of `values`. They belong exclusively to `values`.

The second version (line 36) uses a range adapter object that accepts a range and returns a view (see (*C++ Named Requirements: PODType (Deprecated in C++20)* - *Cppreference.Com* n.d.); (*Containers Library* - *Cppreference.Com* n.d.)). The bitwise OR operator, `|`, has been overloaded to accept a container, a range or view as the left operand and a view as the right operand (roughly speaking). The effect is as if the second operand were called as a function with the first operand as an argument. Some readers may notice the similarity to the pipe operator, `|`, in a terminal window, which is intentional. This makes it easy to compose complex processing pipelines, which will be explained in more detail later. Composing function calls with piping is also an important technique in functional programming (see Section *Lambda Expression*). line

A nice detail is the declaration of the `lambda` variable (lines 27ff). Since the same lambda must be passed as the second argument for both calls to `ranges::for_each<>()`, it is previously assigned to the `lambda` variable. This avoids redundant specification of the lambda.

4.5. Containers

The C++ containers library is part of the standard library. It offers a wealth of generic containers for many purposes. These can be divided into three main categories: *sequence containers*, *associative containers*, and *unordered associative containers*. There are other containers that do not fit exactly into these categories.

4.5.1. Sequence Containers

`std::vector<>` is a so-called *sequence container*. Its elements can be accessed sequentially, e.g. from the first to the last. Further examples of sequence containers are `std::array<>`, `std::deque<>`, `std::list<>` and `std::forward_list<>` (for a complete list see (*Containers Library* - *Cppreference.Com* n.d.)). All sequence containers provide sequential access to their elements. As they do not sort their elements according to an internal criterion, they are referred to as *externally ordered*. The program in Listing 164 illustrates their use.

Listing 164: cia/Containers.cpp (excerpt)

```

1 // Containers.cpp by Ulrich Eisenecker, July 28, 2025
2
3 #include <algorithm>
4 #include <array>
5 #include <deque>
6 #include <forward_list>
7 #include <iostream>
8 #include <list>
9 #include <ranges> // Because of concept ranges::range.
10 #include <string>
11 #include <vector>
12
13 using namespace std;
14
15 void myMain(ranges::bidirectional_range auto& values)
16 {
17     cout << "Memory used by values: "
18           << sizeof(values)
19           << endl;
20     // Output elements
21     cout << "Elements:";
22     for (auto element : values)
23     {
24         cout << ' ' << element;
25     }
26     cout << endl;
27     // Reversing the order of elements
28     ranges::reverse(values);
29     // Output elements
30     cout << "Elements:";
31     ranges::for_each(values, [](auto element)
32                       { cout << ' ' << element; });
33     cout << endl;
34 }
35
36 int main()
37 {
38     array cArray { 9, 0, 7, 2, 8, 1, 6, 3, 4, 5 };
39     myMain(cArray);
40     vector stdVector { 9, 0, 7, 2, 8, 1, 6, 3, 4, 5 };
41     myMain(stdVector);
42     deque stdDeque { "9"s, "0"s, "7"s, "2"s, "8"s,
43                     "1"s, "6"s, "3"s, "4"s, "5"s };
44     myMain(stdDeque);
45     list stdList { 9, 0, 7, 2, 8, 1, 6, 3, 4, 5 };
46     myMain(stdList);
47
48     forward_list stdFwdList { 9, 0, 7, 2, 8, 1, 6, 3, 4, 5 };
49     // myMain(stdFwdList); // Does not compile
50
51     string stdString { "9072816345"s };

```



```
52 |     myMain(stdString);  
53 | }
```

It defines the function template `myMain<>()`, which replaces the `main()` function of the previous programs. It accepts a parameter whose type must conform to the `std::ranges::bidirectional_range` concept, i.e. the member functions `begin()` and `end()` must be available, the iterators they return must be input iterators, and they can iterate through the elements in forward or backward direction. This is important because `reverse<>()` requires both iteration direction. To use `std::ranges::bidirectional_range`, the `<ranges>` header must be included.

First, `myMain<>()` outputs the memory space occupied by values on the function call stack. Second, a range-based for loop sends all elements of values to standard output. Third, `std::ranges::reverse<>()` is used to reverse the order of the elements in values. Fourth, `std::ranges::for_each<>()` is used to output the elements to `cout`.

Both output options are presented so that one can compare them and decide which one one would prefer to use.

The `main()` function declares exemplars of different container templates:

- `std::array<>` is a wrapper for a classic C-array. It has a fixed size that cannot be changed after it has been declared. Since its elements are managed internally in a C-array, they are located in a single contiguous memory area. `array<>` does not support the inserting or deleting an element in the sense that other elements are automatically moved toward the end or the beginning of the array. Apart from that, `std::array<>` provides the member functions and types that a container of the C++ standard library must define. Its use was demonstrated in Listing 160.
- `std::vector<>` is closest to a classic C-array, but it is a container whose size can be adjusted during runtime. Elements can be inserted at the end in constant time by calling `vector<>::push_back()`, as long as this does not change the size. Inserting or deleting elements at other positions requires moving other elements or even enlarging the `vector<>`, which happens automatically. Inserting or deleting an element invalidates all previously acquired iterators. This also applies when an element is appended to the end of a `vector<>`, as this may require resizing the `vector<>`. Apart from that, all elements of a `vector<>` are allocated contiguously in memory. When a `vector<>` is created, a certain number of elements are allocated, which is referred to as its *capacity*. Therefore, adding some elements does not necessarily result in an immediate change in size. If the size of a `vector<>` is changed, a few more elements than necessary are usually allocated, so that the capacity is increased by more than just one element. The container `vector<>` has already been used frequently, so no separate program is needed to demonstrate its use.
- `std::deque<>` is a double-ended queue. Elements can be inserted quickly (and in constant time) at either end, and iterators pointing to another element of a `deque<>` exemplar are never invalidated. The elements of a `deque<>` can be allocated in non-

contiguous memory locations. Like `vector<>`, it provides an iterator with random access. Its overall performance is slightly worse than that of `vector<>`, but when inserting elements at the beginning of the container is required, it performs significantly better than `vector<>`.

- `std::list<>` is normally implemented as a doubly linked list. Like a `deque<>`, it allows elements to be inserted and deleted at both ends in constant time. Inserting an element does not invalidate previously acquired iterators. When an element is deleted, only the iterator for the deleted element becomes invalid; all other iterators remain valid. A `list<>` provides forward and backward iterators. Unlike a `deque<>`, it does not support random access, i.e. it has no index operator and no `at()` member function. The elements of a `list<>` can be stored in completely different memory locations. Normally, the performance of `list<>` is worse than that of `vector<>` or `deque<>`, but if elements need to be frequently inserted into or removed from the middle of a container, the performance of `list<>` is better.
- `std::forward_list<>` is a unidirectional linked list that only provides a forward iterator. If a backward iterator is not required, a `forward_list<>` is a better choice than `list<>`, as it requires less memory. Commenting out the line `myMain(std::forward_list);` results in a compilation error. The reason for this is that `myMain<>()` cannot be instantiated for a `std::forward_list<>` because it does not provide a backward iterator as required by the `std::ranges::range` concept, which the compiler would output as an error message. If `myMain<>()` were defined as an ordinary generic function without a concept or requirements, it would also fail to compile, but the error message would most likely be much more complex. As with a `list<>`, the elements of a `forward_list<>` can be located in completely different memory locations. As for `deque<>` and `list<>`, there is also no programming example for `forward_list<>`, as these containers are only relevant in special cases and their use does not differ significantly from that of a `vector<>`.
- Although `std::string` is not related to the earlier *STL* containers, it also meets the requirements of the `std::ranges::range::bidirectional_range` concept. Therefore, it is possible to instantiate and call `myMain<>()` with `std::string` as an argument. The container `std::basic_string<>` (and its instantiation for the type `char`, `std::string`) was not part of the *STL*. Over time, however, `basic_string<>` was modified to meet the requirements of an *STL* container.

Some variables are instantiated for types other than `int`, namely `std::deque` with *C++-strings* and `std::string` with a single *C++-string*, i.e. the individual characters of the string are treated as elements. This is done to illustrate the possibility of instantiating different container types for different element types and to show that `myMain<>()` can be instantiated for the resulting exemplars.

The function template `myMain<>()` demonstrates the expressiveness of concepts. The `std::ranges::bidirectional_range` ensures that all containers whose type meets the requirements of this concept can be used. This makes `myMain<>()` a very powerful

generic function. If the template of the container and the type of its elements had to be specified separately, the container would have to be passed as a template parameter. This would lead to a complex and cumbersome definition.

4.5.2. Associative Containers

Associative containers store elements that link keys to values. Since they are sorted internally by their keys, the associated values can be searched quickly. There are currently four associative containers:

- `std::set<>` stores unique elements, i.e. an element can occur at most once. In this respect, `std::set<>` resembles a mathematical set. Unlike a mathematical set, it is sorted internally, which requires a comparison function for its elements. The elements of a set are also referred to as *self-associative*, i.e. they associate themselves keys and values. Operations familiar from mathematical sets, such as intersection, union and difference are available in the algorithms library. They can also be applied to any other container that is internally sorted or has been previously sorted externally by calling `sort<>()` with a suitable comparison function.
- `std::map<>` associates unique keys with values. The elements are sorted in ascending order according to the criterion specified by a comparison function. While keys are generally unique, the associated values can occur multiple times. Therefore, a `map<>` realizes a unidirectional mapping.
- `std::multiset<>` stores elements that can occur multiple times.
- `std::multimap<>` is similar to `std::map<>`. It can link multiple identical keys with multiple identical or different values.

Since `std::map<>` has already been introduced in Listings 28 and 63, the following program examples only deal with `std::set<>`, `std::multiset<>`, and `std::multimap<>`. They also introduce various algorithms, ranges and views.

The programs are implemented as unit tests with *Catch2* v3. Normally, types and functions from a proven standard library would not be unit tested. However, this helps to assess whether their use and the expected results are correctly understood. In addition, some functions and details are added that really should be tested.

4.5.2.1. `std::set<>`

Listing 165 introduces the generic function `to_string<>()` as a helper function that gives a representation of the elements of a range as `std::string`. Then, some basic behaviors of `set<>` and algorithms for sets are checked, which can basically also be applied to other containers with sorted elements.

Listing 165: cia/Set.cpp

```

1 // Set.cpp by Ulrich Eisenecker, August 28, 2025
2 // Compiles with clang version 20.1.5 .
3 // Does not compile with GCC 15.1.0 .
4
5 #include <catch2/catch_test_macros.hpp>
6 #include <algorithm> // Because of set operations.
7 #include <iterator> // Because of begin<>(), end<>() and inserter<>().
8 #include <numeric> // Because of accumulate<>().
9 #include <ranges> // Because of set operations in the ranges library
10 // and forward_range concept.
11 #include <set> // Because of set<>.
12 #include <string> // Because of string.
13
14 // The following function template creates a string representation
15 // for all elements of a set.
16 std::string to_string(const std::ranges::forward_range auto& c)
17 {
18     // accumulate<>() is not yet available in the ranges library.
19     return std::accumulate(begin(c), end(c), std::string { });
20 }
21
22 TEST_CASE("Set")
23 {
24     using namespace std;
25
26     set set1 { "two"s, "one"s, "zero"s, "four"s },
27             set2 { "four"s, "six"s, "eight"s, "one"s, "nine"s, "six"s }; // "six" occurs
28                                                                    // twice!
29
30     set<string> result { }; // Empty set of strings.
31     SECTION("size")
32     {
33         REQUIRE(size(set1) == 4);
34         REQUIRE(size(set2) == 5); // Not 6 elements, as "six" occurs twice above.
35     }
36     SECTION("Internally sorted")
37     {
38         REQUIRE(to_string(set1) == "fouronetwozero"s);
39         REQUIRE(to_string(set2) == "eightfournineonesix"s);
40         // Or with the same effect
41         REQUIRE(ranges::is_sorted(set1));
42         REQUIRE(ranges::is_sorted(set2));
43     }
44     SECTION("Inserting, erasing & containing")
45     {
46         REQUIRE(set1.contains("five"s) == false);
47         set1.insert("five"s);
48         REQUIRE(size(set1) == 5);
49         REQUIRE(set1.contains("five"s) == true);
50         set1.erase("five"s);
51         REQUIRE(set1.size() == 4);
52         REQUIRE(set1.contains("five"s) == false);
53     }
54 }

```

```
52 | }
53 | SECTION("Union")
54 | {
55 |     // An inserter is required to insert elements into the
56 |     // result set. The container and an iterator (either begin()
57 |     // or end()) must be passed to the inserter as arguments.
58 |     ranges::set_union(set1, set2,
59 |                       inserter(result, begin(result)));
60 |     REQUIRE(to_string(result) == "eightfournineonesixtwozero"s);
61 | }
62 | SECTION("Intersection")
63 | {
64 |     ranges::set_intersection(set1, set2,
65 |                              inserter(result, begin(result)));
66 |     REQUIRE(to_string(result) == "fourone"s);
67 | }
68 | SECTION("Difference")
69 | {
70 |     ranges::set_difference(set1, set2,
71 |                            inserter(result, begin(result)));
72 |     REQUIRE(to_string(result) == "twozero"s);
73 | }
74 | SECTION("Symmetric difference")
75 | {
76 |     ranges::set_symmetric_difference(set1, set2,
77 |                                       inserter(result, begin(result)));
78 |     REQUIRE(to_string(result) == "eightninesixtwozero"s);
79 | }
80 | }
```

The purpose of `to_string<>()` is to give a stringified representation of all the elements in a forward range. All the elements themselves need to be strings, which are concatenated from the first to the last element to make one big string. This fits perfectly with the variables `set1`, `set2`, and `result`, whose elements are of type `std::string`. In this way, the expected result of calling `to_string<>()` for a set can be compared with the actual result of a set-specific function, as is done in many sections.

The implementation of `to_string<>()` returns the result of calling `std::accumulate<>()`, an algorithm function template contained in the `<numeric>` header file. Unfortunately, the version that accepts a range is not yet available. For this reason, the first two parameters must be a begin and end iterator, here `begin(c)` and `end(c)`. The third parameter is an initialization value, here `std::string { }`, which results in an empty string as the initial value. The fourth parameter is normally a pointer to a function that accepts the accumulated value as the first argument and a value to be added to the accumulated value as the second argument, and returns the result. If none is provided, the specified default generic function pointer is used, which adds the first and second arguments by applying the plus operator. Since the `+` operator is overloaded for `std::string`, it meets the requirements needed here. Therefore, this argument is not specified, as the default generic function pointer is sufficient.

For readers interested in what a suitable generic lambda expression might look like, the following line shows one possible definition:

```
[](auto result, auto element) { return result + element; }
```

Since the pair of square brackets is empty, no variables from the call context are available in the implementation of the lambda. In the pair of parentheses two auto parameters are specified. `result` is the intermediate result (either the initial value or the result of the previous call to the lambda), `element` is the actual element for which the lambda is invoked. The only statement enclosed in curly brackets returns the concatenation of `result` and `element` using the `+` operator. Since no return type is explicitly specified, the compiler automatically deduces it from the returned value.

`std::accumulate<>()` performs a left-fold operation as presented for a template parameter pack in the Section *Variadic Templates*. Beginning with C++23, the `std::ranges::left_fold<>()` algorithm is included in the `<algorithm>` header file. Listing 166 shows the implementation of `to_string<>()` with `std::ranges::left_fold<>()`. For compiling it, the option `-std=c++23` is required for *clang++ 20.1.5* and *g++ 14.2.0_1*. Unlike `std::accumulate<>()`, `std::ranges::left_fold<>()` does not specify a generic function pointer as the default for the operation that is called with the intermediate value and the element. Therefore, a corresponding lambda must be provided. This is the lambda presented earlier.

Listing 166: `to_string<>()` with `std::ranges::fold_left` (excerpt from `cia/FoldLeft.cpp`)

```

10 | std::string to_string(const std::ranges::forward_range auto& c)
11 | {
12 |     return std::ranges::fold_left(c, std::string { },
13 |                                 [] (auto result, auto element)
14 |                                 { return result + element; });
15 | }
```

All identifiers from the C++ standard library are fully qualified. This allows `to_string<>()` to be placed later in a separate header file that can be included by other programs without them having to implicitly import all identifiers from the `std` namespace. This follows the advice given in (U. W. Eisenecker 2022), Section *Header and Implementation Files*.

One could argue that `to_string<>()` has limited applicability, because it cannot be instantiated for containers whose elements are not of type `std::string`. One possibility would be to call `std::to_string()` with `element` as an argument, i.e. rewriting the return statement as

```
return result + std::to_string(element);
```

This works for all element types except `std::string`. If the type of the elements is `std::string`, instantiation of the function template fails. The reason for this is that there is no overload of `std::to_string()` that accepts a `std::string` exemplar as an argument. This becomes obvious when trying to compile the statement

```
cout << std::to_string("Not possible."s) << endl;
```

which is included as a comment in the program *cia/FoldLeft.cpp*).

There is a solution to this problem that does not require any modification of `to_string<>()`. First, a generic lambda is defined that accepts a parameter of any type and returns the result of calling `std::to_string()` for that parameter, i.e.

```
auto lambda { [](auto e){ return std::to_string(e); } };
```

This generic lambda expression can be applied to any value that is not a `std::string` and for which an overload of `to_string()` is available.

Second, the non-string elements of a container are converted into a range-adapter, also known as a *view*, with a transformation function that creates a string representation for a container element:

```
vector vi { 1,3,8 };  
cout << to_string(vi | std::views::transform(lambda)) << endl;.
```

The operator `|` is overloaded so that it accepts a range as the left operand (here `vi`) and a range-adapter as its right operand. The range-adapter is created by calling the function template `std::views::transform<>()`, passing `lambda` as an argument.

There is one important detail. The transformation with a view does not change the elements of the underlying range. Instead, the transformation function of the view, here `lambda`, is only applied to an element when it is called, i.e. *it is evaluated lazily* and returns a new element that contains the transformed value. This element can also have a different type than the corresponding element of the underlying range. One possible disadvantage is, that the transformation must be repeated when the transformed values are accessed again.

All the above code is contained in the program *cia/FoldLeft.cpp*, which can be found in the accompanying material.

There is also the algorithm `std::transform<>()`, which requires the specification of start and end iterators for the elements to be transformed, as well as an iterator for the container in which the transformed elements are to be stored. If the source and target containers are identical, i.e., it is an in-place transformation, the type of the elements must not change. If the transformation function produces a result that differs from the element type, a destination container with the corresponding element type must be provided.

In the test case `Set`, all identifiers from the namespace `std` are imported. This is adequate because the code of the unit tests is not intended to become part of a library.

The sets `set1` and `set2` are initialized with a list of string literals. Therefore, they are automatically instantiated for `std::string` as the element type. The use of the suffix `s` is essential. Without this suffix, the sets would be instantiated for ordinary C-strings and sorted in ascending order with respect to their pointer values. Since `result` is initialized with an empty list, the type of its elements cannot be inferred and it must be explicitly instantiated with the desired element type, i.e., `set<string> result;`.

The section named `size` checks whether the number of elements is correct. Since `set1` is initialized with four unique elements, its size is 4. Please note the use of the free function template `size<>()` to determine the number of elements instead of `set1.size()`, which follows the above recommendation to prefer free functions (or function templates) over member functions for containers. The size of `set2` is actually 5, not 6, as the element "six"s occurs twice in the initialization list. Since `set2` is a normal set, it cannot contain duplicate elements.

The next section checks whether `set1` and `set2` are sorted internally in ascending order. A naive attempt would be to obtain a string representation of each by calling the `to_string<>()` function defined above and comparing the actual result with the expected result. However, there is an easier method, which is to call the `ranges::is_sorted<>()` function template for each set. The result must, of course, be `true`.

The following section demonstrates how to check for the presence of an element using the member function `contains()`. Initially the result of calling `set1.contains("five"s)` is `false`. However, after inserting "five"s with the call `set1.insert("five"s)`, the check for the presence of "five"s returns `true`. The element "five"s is then removed by calling `set1.erase("five"s)`. As a consequence, calling `set1.contains("five"s)` subsequently returns `false`. In addition, the size of `set1` is checked after inserting and removing an element.

In the section `Union`, `set1` and `set2` are merged by calling `std::ranges::set_union<>()`. This operation corresponds to the mathematical operator *union* for sets. In C++, it can be applied to a variety of different container types. The most important prerequisite is that the elements of both containers must be sorted in ascending order, which can also be achieved by a preceding call to `std::sort<>()`. Please note that the expected result contains only one occurrence of "four"s and only one occurrence of "one"s. This is correct, as the resulting set must not contain duplicate elements.

Naively, one might assume that calling `ranges::set_union<>()` returns a new container containing all elements that occur in the sets passed as arguments. Although this would fit well a functional programming style, the usage is different. Instead, an insert iterator must be specified as the third argument. This is created by calling the convenience function template `std::inserter<>()`, which is defined in the header file `<iterator>`.

It must be called with a container and an iterator associated with that container that specifies the position where the resulting elements are to be inserted, and it returns the corresponding insert iterator.

The programmer is responsible for passing an iterator position as the second argument that is valid for the container passed as the first argument. They are also responsible for specifying a valid iterator position. For example, passing `begin(result)` would be invalid if the container were a `vector<>`, since a `vector<>` only allows elements to be inserted at its end; therefore, only `end(result)` is a valid choice in this case. The section `Union` uses the `result` that was defined at the beginning of the test case and initialized to the empty set. Since it is empty, after calling `ranges::set_union<>()`, it contains only the distinct elements of `set1` and `set2`. Since it is a set, it is equally valid to specify `begin(result)` or `end(result)` as the second parameter for the call to `inserter<>()`, since new elements can be inserted on either side of a set; they are sorted anyway when inserted.

It is also possible to insert new elements somewhere in the middle if a container does support this, e.g. a `list<>`. This can be demonstrated easily by inserting the code snippet shown in Listing 167 before the end of the `Union` section (including `<list>` at the beginning of `cia/Set.cpp` is required):

Listing 167: The insertion in the middle of a `std::list<>`

```
1 | list brackets { "{ "s," }"s };
2 | ranges::set_union(set1,set2,
3 |                   inserter(brackets,++begin(brackets)));
4 | REQUIRE(to_string(brackets) == "{ eightfournineonesixtwozero }"s);
```

As can be seen, the elements of the resulting set union are inserted between the first and the last element of the `brackets` list. There are two important details. First, the list named `brackets` is initialized with a pair of curly braces as the first and last element, second, the iterator is incremented before it is passed as the second parameter to `inserter<>()`, i.e. `++begin(brackets)`. This algorithm is commutative with respect to `set1` and `set2`, i.e. it is not relevant which set is passed as the first argument and which as the second argument.

The `Intersection` section demonstrates the use of `std::ranges::set_intersection<>()`. It inserts those elements into `result` that occur in both `set1` and `set2`, which corresponds to the mathematical set intersection. The structure of this operation is similar to set union. Therefore no further explanations are given here. The set intersection is also commutative.

The `std::ranges::set_difference<>()` algorithm is presented in the Difference section. The resulting set only contains the elements that occur exclusively in `set1`. Elements that occur in both sets are excluded. Although this algorithm is structurally equivalent to the preceding one, it is not commutative, i.e. changing `set1` and `set2` when calling the algorithm can lead to different results.

The Symmetric difference section shows the use and effect of the `std::ranges::set_symmetric_difference<>()` algorithm. The result only contains the elements that occur in either `set1` or `set2`. Elements that occur in both sets are excluded. This algorithm is also structurally equivalent to the two aforementioned algorithms. The algorithm is commutative.

4.5.2.2. `std::multiset<>`

There are two changes in the *cia/MultiSet.cpp* program with respect to *cia/Set.cpp*. The first difference is that *cia/MultiSet.cpp* includes the header file *to_string.hpp*, which contains the definition of the function template `to_string<>()` for containers in the namespace *cia*. The second difference is that `multiset<>` is used throughout instead of `set<>`. Unlike a `set<>`, a `multiset<>` can contain multiple occurrences of an element. Therefore, all set algorithms take into account the cardinality of an element. Apart from these two differences, the programs are structurally equivalent. Therefore, the files *cia/to_string.hpp* and *cia/MultiSet.cpp* are only included in the accompanying material and are not reproduced here, and no further explanation is given.

The `set<>` and `multiset<>` containers can be closely linked to their mathematical counterparts. However, it can sometimes be advantageous not to be too constrained by this link. A `set<>` and, in particular, a `multiset<>` can also be regarded as containers that are immediately reordered when a new element is added. Reading data from the keyboard or from a file can be time-consuming. If the exact order of the read data does not need to be preserved and sorting is required later anyway, one option is to fill a `multiset<>` with the data, as shown in Listing 168.

The corresponding algorithm is called *insertion sort*. It is not included in the C++ standard library. As Listing 168 shows, this is not a disadvantage, since `multiset<>::insert()` performs this equivalently—in the scenario of individual input of each element.

Listing 168: cia/InsertionSort.cpp

```
1 // InsertionSort.cpp by Ulrich Eisenecker, July 92, 2025
2
3 #include <iostream>
4 #include <ranges> // Because of views::transform<>().
5 #include <set>
6
7 #include "to_string.hpp" // Because of cia::to_string<>().
8
9 using namespace std;
10
11 using ElementType = double;
12 using ContainerType = multiset<ElementType>;
13
14 ContainerType readData()
15 {
16     ContainerType data;
17     ElementType value;
18     while (cin)
19     {
20         cin >> value;
21         if (cin)
22         {
23             data.insert(value); // The insertion sort takes place here.
24         }
25     }
26     return data;
27 }
28
29 int main()
30 {
31     cout << "Please enter floating point values.\n"
32           << "When finished, enter EOF "
33           << "(Ctrl-D on macOS and Linux, Ctrl-Z on MS Windows)."
34           << endl;
35     auto data { readData() };
36     size_t counter { };
37     cout << "You entered " << size(data) << " values.\n"
38           << "Following, they are printed in ascending order.\n"
39           << cia::to_string(data | views::transform([&counter](auto e)
40                                           { return to_string(++counter) +
41                                             ": " + to_string(e) + '\n'; }))
41           << endl;
42 }
43 }
```

At the beginning, `ElementType` is defined as alias for `double`. This provides a customization point to switch to `float` or `long double` if desired. For simplicity, `ContainerType` is defined as alias for `set<ElementType>`. The `readData()` function returns an exemplar of `ContainerType` as a new value. In its implementation, it defines a local variable named `data` of type `ContainerType` and a local variable named `value` of type `ElementType`. Next,

a while loop starts which runs as long as `cin` can be converted to the Boolean value `true`. In the loop, the variable `value` is extracted from `cin`. If `cin` is still good after the extraction, i.e. it evaluates to `true`, `value` is inserted into `data`. This is exactly where insertion sorting takes place. As soon as the end of file is reached for `cin` or an error has occurred, the while loop is terminated and `data` is returned by `value`.

At the beginning of `main()`, some instructions for data input are sent to `cout`. Next, the auto variable `data` is initialized with the result of the call to `readData()`. Then, a local variable of type `size_t` named `counter` is defined and zero-initialized. Next, a brief information about the following output and the result of calling `cia::to_string<>()` are sent to `cout`.

The expression passed as an argument to `cia::to_string<>()` is similar to the one used in the *`cia/FoldLeft.cpp`* program. The operator takes the container `data` as its left operand and a range adapter as the right operand. The range adapter is created by calling `views::transform<>()`. Here, a lambda is passed as an argument.

It shows a new syntax detail. For the first time, the capture clause, the part in the square brackets, is not empty. Here, the variable `counter` from the enclosing context is made available as a reference by specifying `[&counter]`. The leading ampersand, `&`, means that `counter` is accessible as a reference in the lambda expression, i.e. it can be modified by the lambda, and this modification also affects the `main()` function that defines `counter`. A single container element is passed as an auto parameter named `e`. The implementation of the lambda assembles a `std::string` consisting of `counter` incremented by 1, converted into a string, the C-string `": "` (which is automatically converted to a `std::string`), `e` converted to a C++-string, and finally the newline character. The resulting string is returned as the value.

4.5.2.3. `std::multimap<>`

The program in Listing 169 presents the function `allSubstrings()` which generates all substrings of a given source string, including the source string itself. The result can be considered a kind of a power set of substrings plus the position where a substring starts in the source string. The result returned by this function is used to demonstrate some properties of `multimap<>`, the use of views, and in particular the `multimap<>::merge()` member function.

Listing 169: cia/MultiMap.cpp

```
1 // MultiMap.cpp by Ulrich Eisenecker, Agust 27, 2025
2 // Compiles with clang version 20.1.5 .
3 // Does not compile with GCC 15.1.0 .
4
5 #include <catch2/catch_test_macros.hpp>
6 #include <algorithm> // Because of is_sorted<>() abd equal<>().
7 #include <iostream>
8 #include <iterator> // Because of begin<>(), end<>() and inserter<>().
9 #include <map> // Because of multimap<>.
10 #include <ranges> // Because of set operations in the ranges library,
11 // forward_range concept and is_sorted<>().
12 #include <string> // Because of string.
13 //
14 #include "to_string.hpp" // Because of cia::to_string<>().
15
16 auto allSubstrings(std::string s)
17 {
18     using StringType = decltype(s);
19     using SizeType = StringType::size_type;
20     std::multimap<StringType, SizeType> result;
21     for (SizeType i { }; i < size(s); ++i)
22     {
23         for (SizeType l { 1 }; l <= size(s) - i; ++l)
24         {
25             result.insert({ s.substr(i,l), i });
26         }
27     }
28     return result;
29 }
30
31 TEST_CASE("MultiMap")
32 {
33     using namespace std;
34     using namespace cia;
35
36     auto abacab { allSubstrings("ABACAB"s) };
37     auto aba { allSubstrings("ABA"s) };
38     auto abc { allSubstrings("abc"s) };
39
40     SECTION("to_string, transform, keys & values")
41     {
42         auto pairs { abc | views::transform([](const auto& p)
43             { return " "s + p.first +
44               "->" + to_string(p.second);
45             })
46         };
47
48         REQUIRE(to_string(pairs) == " a->0 ab->0 abc->0 b->1 bc->1 c->2"s);
49         REQUIRE(to_string(abc | views::keys) == "aababcbbcc"s);
50         auto values { abc | views::values
51             | views::transform([](const auto& v)
```

```

52         { return std::to_string(v);}
53     }
54 };
55     REQUIRE(to_string(values) == "000112"s);
56 }
57 SECTION("size")
58 {
59     REQUIRE(size(abacab) == 21);
60     REQUIRE(size(aba) == 6);
61 }
62 SECTION("Internally sorted")
63 {
64     REQUIRE(ranges::is_sorted(abacab));
65     REQUIRE(ranges::is_sorted(aba));
66 }
67 SECTION("Merge, keys & equal")
68 {
69     auto abacabCopy { abacab },
70         abaCopy { aba };
71     abacab.merge(aba);
72     REQUIRE(size(abacab) == size(abacabCopy) + size(abaCopy));
73     REQUIRE(size(aba) == 0);
74     abaCopy.merge(abacabCopy);
75     REQUIRE(size(abacabCopy) == 0);
76     auto v1 { views::keys(abacab) };
77     auto v2 { views::keys(abaCopy) };
78     REQUIRE(ranges::equal(v1,v2));
79 }
80 }

```

Before discussing the implementation of function, the task it solves will be presented in more detail.

The string *ABACAB* has 21 substrings including the string itself, namely

A AB ABA ABAC ABACA ABACAB B BA BAC BACA BACAB A AC ACA ACAB C CA CAB A AB B.

Sorting all these substrings results in

A A A AB AB ABA ABAC ABACA ABACAB AC ACA ACAB B B BA BAC BACA BACAB C CA CAB.

The sorted list shows that some substrings occur more than once. If it is desired to reconstruct the source string from all these substrings, the simplest option would of course be to search for the longest string. In some cases, however, it is desirable to know at which position in the source string a particular substring starts. The length of the substring does not need to be recorded separately, as a substring has its own length. With this information, a source string can be fully related to its substrings, as shown in Table 17. String segmentation and alignment are important operations in various domains, such as DNA sequencing.

Table 17: Aligning substrings from ABACAB

A	B	A	C	A	B
A					
A	B				
A	B	A			
A	B	A	C		
A	B	A	C	A	
A	B	A	C	A	B
	B				
	B	A			
	B	A	C		
	B	A	C	A	
	B	A	C	A	B
		A			
		A	C		
		A	A	A	
		A	C	A	B
			C		
			C	A	
			C	A	B
				A	
				A	B
					B

To record this information, an instantiation of `std::multimap<>` is used, which maps a key of type `std::string` to the starting position of the string in the source string as value of type `std::string::size_type`. Such a multimap is returned by the function `allSubstrings()` whose return type is specified as `auto`.

Initially, `allSubstrings()` defines `StringType` as an alias for `decltype(s)`. This is done to avoid redundantly specifying `std::string` twice. Furthermore, this makes the function implementation resilient to a possible future change in the parameter type of the function argument. Next, `SizeType` is defined as an alias for `StringType::size_type`. Then, an exemplar of `std::multimap<StringType,SizeType>` is created with the name `result`.

Two nested for loops generate all possible substrings, including the source string `s` itself. The outer loop with the variable `i` runs from 0 to the size of `s` minus 1, the inner loop with the variable `l` runs from 1 up to and including the size of `s` minus `i`. The statement in the inner loop calls the member function `string::substr()` to extract a substring starting at position `i` of length `l` from `s`. The resulting substring and `i` are inserted into `result`. At the end, the function returns `result`.

In the `MultiMap` test case, three `auto` variables called `abacab`, `aba`, and `abc` are initialized by calling `allSubstrings()` with `"ABACAB"s`, `"ABA"s` and `"abc"s` as source strings.

The first section tests `cia::to_string<>()`, `transform<>()` and the elements of a `multimap<>`, which are represented as `std::pair<key,value>`. First, the `auto` variable `pairs` is initialized by combining `abc` and a view adapter with the overloaded bitwise OR operator, `|`. The view is created by calling `transform<>()`, passing a lambda as a parameter. The capture clause, `[]`, of the lambda is empty, its argument list specifies a reference to `const` named `p`, which is declared as `auto`, making it a generic lambda. Its return statement constructs a `std::string` from a string containing a single space, `p.first`, a C-string containing an arrow, and the result of calling `to_std::string(p.second)`. Since `p` is a pair, it consists of the data members `first` of type `std::string` and `second` of type `string::size_type`. Since `p.first` is a string, it can be used directly in a string concatenation expression, but `p.second`, being an integer, must be converted to a string.

The first `REQUIRE` macro checks whether `pairs` matches the expected result string, which is created manually. The next `REQUIRE` macro has an interesting expression. First, a view adapter is applied to `abc`, which extracts the keys. `views::keys` takes a range of tuple-like values and extracts their first element. A `pair<>` is also a tuple-like value, so its first element (the member `first`), which acts as the key, is extracted. Tuples will be explained later. Then `cia::to_string<>()` is called with the result as an argument. This should match the string specified as the right operand of the comparison operator containing all substrings that act as keys. The `auto` variable `values` is initialized with an expression that starts with `abc` (referred to as *arg* below), and then applies `views::values`, a pre-defined view from the C++ standard library, (referred to as *f₁*() below). This extracts

the values associated with the keys, to which a transform view is finally applied that converts each element to a string (referred to as *f₂*() below). This can be thought of as applying the functions *f₁*() and *f₂*() to the argument *arg* as follows:

f₂(*f₁*(*arg*)).

The REQUIRE macro in the next line calls `cia::to_string<>()` for values and compares it with the expected string, which contains only the values, i.e. the second member of a `pair<>`.

The third section demonstrates the member function `multimap<>::merge()` and the `ranges::equal()` function template. Initially, two copies with the names of `abacab` and `aba` are created. Then, `abacab` is merged with `aba` by calling `abacab.merge(aba)`. This moves all values from `aba` to `abacab`. After that, `abacab` has the same number of elements as `abacab` and `aba` had before calling `multimap<>::merge()`.

Therefore, the REQUIRE macro checks whether the size of the modified `abacab` is equal to the sum of `size()` called for `abacab` and `aba`. The following REQUIRE macro checks whether `aba` is now empty. Next, `abaCopy` is merged with `abacabCopy`. Although one might assume that the merge operation is commutative, this is not the case. The reason for this is that all substrings of `aba` must be contained in `abacab`, but can start at different positions. Therefore, the values resulting from the two possible merges may be different. This difference can be mitigated by examining the keys alone. For this purpose, the auto variables `v1` and `v2` are initialized with the range adapter that extracts the keys from `abacab` and `abaCopy`. This time, a different syntax is used that makes the call to `views::keys()` look like a function call. The syntax with the overloaded bitwise OR operator can be used equivalently.

4.5.2.4. Another Look at `std::multiset<>`

`std::set<>` and `std::multiset<>` are often thought of as their mathematical counterparts, which store individual elements that are either unique or can occur multiple times. In addition, `set<>` and `multiset<>` can also be considered as sorted containers that can store not only single elements, but also pairs or tuples of heterogeneous types. This makes them similar to `map<>` and `multimap<>`.

The program in Listing 170 shows a set that is instantiated with a pair of strings as the element type.

Listing 170: cia/SetOfPairs.cpp

```

1 // SetOfPairs.cpp by Ulrich Eisenecker, June 5, 2025
2
3 #include <iostream>
4 #include <set>
5 #include <string>
6 #include <utility>
7 #include <ranges>
8
9 #include "to_string.hpp"
10
11 using namespace std;
12
13 int main()
14 {
15     set<pair<string,string>> s { { "X"s,"W"s }, { "X"s,"B"s },
16                                { "A"s,"Z"s }, { "A"s,"C"s },
17                                { "A"s,"C"s }, { "X"s,"B"s } };
18     cout << cia::to_string(s | views::transform([](const auto& e)
19                                                { return e.first + e.second + '\n'; }))
20          << endl;
21 }

```

As the output illustrates, it is sorted first by `e.first` and then by `e.second`, i.e. *AC* comes before *AZ*, *AZ* before *XB* and *XB* before *XW*. It is also possible to instantiate a `multiset<>` instead. Then, *AC* and *XB* occur twice.

A tuple is a generalization of a pair. It comprises any number of possibly different types. For using tuples, the header file `<tuple>` must be included. It provides many helper templates for working with tuples, among them the `std::get<>()` function template for accessing a specific element of a tuple. The index of a tuple element must be in the range `0 .. tuple_size_v<> - 1`. If `i` refers to the index constant and `t` to the tuple, `std::get<i>(t)` returns the element at the `i`th position of the tuple `t`.

The program in Listing 171 demonstrates the use of a tuple as a type for the elements of a set.

Listing 171: cia/SetOfTuples.cpp

```

1 // SetOfTuples.cpp by Ulrich Eisenecker, June 5, 2025
2
3 #include <iostream>
4 #include <set>
5 #include <string>
6 #include <utility>
7 #include <ranges>
8 #include <tuple>
9
10 #include "to_string.hpp"

```

```
11
12 using namespace std;
13
14 int main()
15 {
16     set<tuple<char,unsigned,string>>
17     s { { 'u',2,"X"s }, { 'u',0,"Y"s }, { 'u',1,"Z"s },
18         { 'u',2,"W"s }, { 'A',42,"nn"s }, { 'A',42,"n"s },
19         { 'A',42,"nn"s }, { 'u',0,"Y"s }, { 'u',2,"Z"s } };
20     cout << cia::to_string(s | views::transform([](const auto& e)
21                                             { return get<0>(e) +
22                                                         to_string(get<1>(e)) +
23                                                         get<2>(e) + '\n';
24                                             })))
25         << endl;
26     cout << "The tuple size is "
27         << tuple_size_v<decltype(s)::value_type>
28         << '.'
29         << endl;
30 }
```

Executing the compiled program generates an output that proves that the elements of the set are sorted by the first value of the tuple, then by the second element and finally by the third element. No element occurs twice. At the end of the program, the `decltype` operator is used to determine the type of `s` which is more convenient than repeating the complete instantiation of its declaration. From the result type, its member type `value_type` is accessed, which in turn is used to instantiate the `tuple_size_v<>` template variable which gives the size of the tuple.

It is also possible to replace the `std::set<>` template with `std::multiset<>`. In this case, all duplicated elements are retained.

Since a tuple is a generalization of a pair, it is permissible to treat a pair as a tuple with two elements.

All this allows for more creative uses of sets and multisets than just with their mathematical counterparts. There is also an overlap in functionality with `std::map<>` and `std::multimap<>`. The program *cia/MapOfPairs.cpp* (which is only included in the accompanying material) is based on the program in Listing 170. Instead of a `set<>`, it instantiates a `map<>` to create a map called `s`, i.e. `map<string,string> s`. Since a `map<>` requires two type template parameters, instantiation with a `pair<>` as a single parameter is not possible. After compiling and executing the program, there are only two outputs, namely *AZ* and *XW*, which differs from the output of the program in Listing 170. This is because `map<>` only considers the first part of the element type for which it is instantiated, here `pair<>::first`. As *AZ* and *XW* are inserted before *AC* and *XB*, the latter are considered to be duplicates and are therefore not inserted.

Replacing `std::map<>` with `std::multimap<>` in the program *cia/MapOfPairs.cpp* does not lead to the behavior of the *cia/SetOfPairs.cpp* program, as the output now contains two duplicates of *AC* and *XB*, since each key/value pair is now inserted. However, it is possible to overload the function template `std::less<>` from the `<functional>` header file, which is internally used by `set<>` to check which of two elements is the smaller one. This is demonstrated in Listing 172.

Listing 172: Specialization of `std::less<>` for `std::pair<>` (excerpt from *cia/SetOfPairsAdapted.cpp*)

```

1 // SetOfPairsAdapted.cpp by Ulrich Eisenecker, June 6, 2025
2
3 #include <functional> // Because of less<>
4
11
12 using namespace std;
13
14 template <typename T1,typename T2>
15 struct less<pair<T1,T2>>
16 {
17     bool operator()(const pair<T1,T2>& lhs,
18                     const pair<T1,T2>& rhs) const
19     {
20         return lhs.first < rhs.first;
21     }
22 };
23

```

The only changes compared to Listing 170 are the inclusion of the header file `<functional>` and the partial specialization of the `less<>` class template for `pair<>`. This template specifies two type template parameters. As a consequence, the template definition also matches pairs that are instantiated with different types for their first and second data members. The only requirement of the template is that the less-than comparison operator, `<`, is defined for the type of `pair<>::first`. The `main()` function is not changed. This customization is very interesting in that the only intervention is that part of the code is compiled before the code that is to be adapted. One could argue that the hook for this adaptation is also present in the parametrization and implementation of `std::set<>` which is correct.

If `std::set<>` is replaced by `std::multiset<>` in this program, its output is the same as that of the program *cia/MapOfPairs.cpp* after `std::map<>` has been replaced by `std::multimap<>`.

The last endeavor of this section is the adaptation of `std::set<>`, which is instantiated for `std::tuple<>` as element type to only consider the first component of the tuple when sorting and inserting. This is demonstrated in Listing 173.

Listing 173: Specialization of `std::less<>` for a parameter pack (excerpt from `cia/SetOfTuplesAdapted.cpp`)

```
1 // SetOfTuplesAdapted.cpp by Ulrich Eisenecker, June 6, 2025
2 #include <tuple>
3
4 #include "to_string.hpp"
5
6 using namespace std;
7
8 template <typename ... TYPES>
9 struct less<std::tuple<TYPES ...>>
10 {
11     bool operator()(const tuple<TYPES ...>& lhs,
12                     const tuple<TYPES ...>& rhs) const
13     {
14         return get<0>(lhs) < get<0>(rhs);
15     }
16 };
17
18
```

Here too, the only changes are the inclusion of the `<tuple>` header file and the specialization of the class template `less<>` for `std::tuple<>`. The latter is, to put it mildly, a syntactic challenge. As there can be any number of types, a parameter pack called `TYPES` is specified as a template parameter. The `std::less<>` template is then specialized for each tuple with `less<std::tuple<TYPES ...>>`. The parameters passed to the function call operator are specified accordingly as `tuple<TYPES ...>`. The implementation of the function call operator in turn uses the helper function template `std::get<>()` to compare only in relation to the first component of the pair at index position 0. Of course, it is possible to replace `set<>` by `multiset<>` in `main()` to insert all specified elements, but only sorted with respect to the first element.

If the `std::less<>` class template is specialized as shown above, it applies to all instantiations of `std::pairs<>` or `std::tuple<>` respectively. As this may not be desirable, a narrower partial specialization or a full specialization for a specific type of the corresponding class template should be considered.

The preceding code examples and the related discussion may seem purely academic. This is true insofar as there is no need to use `std::set<>` or `std::multiset<>` as a replacement for `std::map<>` or `std::multimap<>`. If the requirements clearly indicate the use of `std::map<>` or `std::multimap<>`, then they should definitively be used, as this is more intentional and avoids unnecessary complexity and tweaks. In practice, however, there are cases where the requirements do not point so clearly to the use of a specific container template. In these cases, it is highly advisable to think about possible details of use, e.g. how much data should be stored in the container or how often the data is iterated when the container is filled, or is it really essential that the data is always sorted?

And in case of doubt, performance measurements should be carried out on the target platform with representative data, realistic data volumes and usage scenarios with different container templates in order to select the one that best meets the requirements.

4.5.3. Unordered Containers

The third category comprises the unordered containers

- `std::unordered_set<>`,
- `std::unordered_map<>`,
- `std::unordered_multiset<>` and
- `std::unordered_multimap<>`.

Their most prominent feature is that their elements are not sorted, as the name of the category suggests. In contrast to the corresponding containers from the previous category, they use hash data structures to store their elements. Since no sorting is performed when inserting a new element into a container, repeated iterations over all elements of a container can yield a performance advantage. The disadvantage is that a function must be available that calculates a hash value for a specific element. It can happen that identical hash values are calculated for different elements. In this case, the elements that have the same hash value are stored linearly, which increases the search and access time. Before selecting a specific container template, it is strongly recommended to perform benchmarks with representative data in terms of data type and data size.

Iteration over the elements of an unordered container takes place in an undefined order, as the program in Listing 174 shows.

Listing 174: cia/UnorderedSet.cpp

```

1 // UnorderedSet.cpp by Ulrich Eisenecker, June 7, 2025
2
3 #include <iostream>
4 #include <unordered_set>
5 #include <string>
6 #include <ranges>
7
8 #include "to_string.hpp"
9
10 using namespace std;
11
12 int main()
13 {
14     unordered_set s { 99, -42, 42, 16, 0, 20, -1, 99, 42 };
15     cout << cia::to_string(s | views::transform([](const auto& e)
16                                     { return to_string(e) + ' '; }))
17          << endl;
18 }
```

When the compiled program is executed, all elements of `s` are output in an unspecified order, which may change for different platforms. The output does not contain any duplicate elements, as the elements of a mathematical set should be unique but not ordered. Therefore, the container template `std::unordered_set<>` corresponds more to the mathematical set than `std::set<>`.

The other unordered container templates are not shown in detail, as they largely correspond to their ordered counterparts—only without sorting.

4.5.4. More Container Types

There are generic containers that do not fit into the categories described above, e.g. *container adapters*:

- `std::stack<>` offers the functionality of a stack, i.e. a *last-in, first-out (LIFO)* for short) data structure.
- `std::queue<>` offers the functionality of a queue, i.e. a *first-in, first-out (FIFO)* for short) data structure.
- `std::priority_queue<>` gives the element with the highest priority in constant time. The elements are ordered according to a priority that can be explicitly defined by a comparison function.

A container adapter does not implement its own data structure for storing its elements. Instead, it uses another generic container for this purpose.

The C++ standard library also offers `std::vector<bool>`, a specialization of `std::vector<>` specifically designed to store `bool` values in a compressed format that requires only a single bit for a `bool` value. The size of a `vector<bool>` exemplar can vary during runtime (see (*Containers Library - Cppreference.Com* n.d.); (`std::vector<bool>` - *Cppreference.Com* n.d.)) for details).

Another sequence container is `std::bitset<>`. It also stores bits in compressed form, but its size must be specified when the template is instantiated, i.e. it cannot change its size during runtime (see (`std::bitset` - *Cppreference.Com* n.d.) for more information).

Other parts of the C++ standard library also define containers. For example, `std::basic_string<>` is as well a full-fledged sequence container. Within certain restrictions, even streams can be treated as containers.

For a more comprehensive list of categories and containers, see (*Containers Library - Cppreference.Com* n.d.).

Some of the containers mentioned above are presented in more detail below.

4.5.4.1. `std::stack<>`

In the following, `std::stack<>` is introduced as a representative of the category of generic container adaptors using a mathematical game. The other two examples of this category, `std::queue<>` and `std::priority_queue<>`, offer a different functionality. As they are technically very similar to `std::stack<>`, no additional examples of their use are presented.

The container adapter `std::stack<>` offers the functionality of a stack, i.e. a LIFO data structure (see above). It accepts two template parameters, namely the type of the elements `T` and the type of another container instantiated for `T`, which is used internally to store the elements. If the second template parameter is not specified, `std::deque<T>` is used by default.

Interestingly, both parameters are type template parameters. This means that only an instantiated container template can be used as the second parameter. It is the programmer's responsibility to ensure that the type `T` specified as the first template parameter and the type `T` used to instantiate the container template as the second parameter are identical; otherwise, the program is ill-formed. When the *STL* was included in the C++ standard, there was no widely used compiler that implemented template template parameters that would have allowed a container class template to be passed to `std::stack<>` as the second template parameter. In order to keep the interface for instantiating `std::stack<>` stable, this early decision has not been revised to this day.

Towers of Hanoi (*Tower of Hanoi* 2025) serves to illustrate their use. The game has three pegs, here called left, middle and right. There are a number of different sized disks on the left peg and the aim is to move all the disks to the right peg. One rule must be observed: It is only allowed to place a smaller disk on top of a larger disk. There are several algorithms to solve this task. The recursive algorithm is very compact and easy to understand. It is demonstrated in Listing 175.

Listing 175: `cia/HanoiSimple.cpp`

```

1 // HanoiSimple.cpp by Ulrich Eisenecker, July 11, 2025
2
3 #include <iostream>
4
5 using namespace std;
6
7 using NaturalNumber = unsigned short int;
8
9 void move(NaturalNumber disks, char left, char middle, char right)
10 {
11     if (disks > 0)
12     {
13         move(disks - 1, left, right, middle);
14         cout << "Moving top disk from " << left << " to " << right << "." << endl;
15         move(disks - 1, middle, left, right);
16     }

```



```
17 }
18
19 int main()
20 {
21     NaturalNumber disks;
22     cout << "Number of disks: ";
23     cin >> disks;
24     move(disks, 'l', 'm', 'r');
25 }
```

The type alias `NaturalNumber` serves as a single customization point to change the integer type for the number of disks. It must be emphasized that numbers above 20 can lead to long execution times, as the minimum number of moves to solve the game is $2^n - 1$, where n is the number of disks. The `move()` function has four parameters, namely the number disks to be moved and character variables representing the different pegs. To enable the recursion, the first parameter must be passed by value. There are no restrictions regarding the passing of parameters for the remaining parameters. When `move()` calls itself recursively, which happens twice in its implementation, it changes the order of the parameters representing the pegs. Since the sole purpose of the `move()` function is to inform the user of the minimal number of moves to solve the game, these parameters are required to correctly identify the pegs to be used in a move.

When imagining a graphical representation of the game, it is obvious that a peg can be considered as a stack. Indeed, it is relatively straightforward to replace the parameters two through four of `move()` by stacks of natural numbers representing the different sizes of the disks. This is illustrated in Listing 176.

Listing 176: `cia/HanoiStack.cpp`

```
1 // HanoiStack.cpp by Ulrich Eisenecker, July 11, 2025
2
3 #include <iostream>
4 #include <stack>
5
6 using namespace std;
7
8 using NaturalNumber = unsigned short int;
9 using Peg = stack<NaturalNumber>;
10
11 Peg init(NaturalNumber numberOfValues)
12 {
13     Peg result;
14     while (numberOfValues)
15     {
16         result.push(numberOfValues--);
17     }
18     return result;
19 }
20
21 void move(NaturalNumber disks, Peg& left, Peg& middle, Peg& right)
```

```

22 {
23     if (disks > 0)
24     {
25         move(disks - 1, left, right, middle);
26         cout << "Moving disk " << left.top();
27         if (right.size())
28         {
29             cout << " on top of disk "
30                 << right.top()
31                 << '.' << endl;
32         }
33         else
34         {
35             cout << " on empty peg." << endl;
36         }
37         right.push(left.top());
38         left.pop();
39         move(disks - 1, middle, left, right);
40     }
41 }
42
43 int main()
44 {
45     NaturalNumber disks;
46     cout << "Number of disks: ";
47     cin >> disks;
48     Peg left { init(disks) },
49         middle,
50         right;
51     move(disks, left, middle, right);
52     cout << "Right peg (in reverse order):";
53     while (right.size())
54     {
55         cout << ' ' << right.top();
56         right.pop();
57     }
58     cout << endl;
59 }

```

This program defines Peg as a type alias for `stack<NaturalNumber>`. The `move()` parameters `left`, `middle` and `right` are passed as references of type Peg and the function `main()` defines its local variables with the same names, also of the type Peg.

Moving disks on real stacks is a bit laborious. First, a function `init()` is defined which returns a peg with as many disks as the user has entered and which are stacked in the correct order. The return value is used as an argument for copy constructing `left` in `main()`. Second, there is no name associated with a peg that could be used to identify the source peg and the target peg in `move()`. Therefore, the number of the peg to be moved plus the information whether it is placed on an empty peg, or otherwise the number of the peg on which it is placed, is output. Since this is too little feedback, the destination

peg right is output to cout at the end of main(). The values of the stack are output in reverse order, as there is no way to apply a range-based for loop to a stack, since stack<> does not define iterators for begin and end.

The essence of using stacks in move() can be found in lines 37f. The member function push() places a new element at the top of the stack, the member function top() returns the top element of the stack as the new value, and the member function pop() removes the top element of the stack. The member functions top() and pop() should only be called for stacks that are not empty. Otherwise an exception is thrown. In the while loop at the end of main(), stack<>::top() and stack<>::pop() are called again to output the elements of the stack in reverse order.

A closer look at move() reveals an annoying detail. It is the need to pass the number of plates as the first parameter. Passing the three pegs left, middle and right actually gives the move() function all the information it needs. Currently, any natural number other than the number of elements of left could be passed as a value, which would result in the function executing incorrectly. Listing 177 shows how this design deficit can be mitigated.

Listing 177: cia/HanoiSmarter.cpp (excerpt)

```
21 void move(Peg& left,Peg& middle,Peg& right)
22 {
23     auto impl { [](NaturalNumber count,Peg& l,Peg& m,Peg& r,auto&& impl) -> void
24                 {
25                     if (count > 0)
26                     {
27                         impl(count - 1,l,r,m,impl);
28                         cout << "Moving disk " << l.top();
29                         if (r.size())
30                         {
31                             cout << " on top of disk "
32                                 << r.top()
33                                 << '.' << endl;
34                         }
35                         else
36                         {
37                             cout << " on empty peg." << endl;
38                         }
39                         r.push(l.top());
40                         l.pop();
41                         impl(count - 1,m,l,r,impl);
42                     }
43                 }
44     };
45
46     impl(left.size(),left,middle,right,impl);
47 }
48
49 int main()
50 {
```

```

57 |     move(left,middle,right);
65 | }

```

The relevant detail for a hypothetical client programmer when using `move()` is the call with only three parameters, namely the pegs `left`, `middle` and `right`, as shown in `main()`.

The `move()` function has been completely revised. As it is not possible to define functions within functions in C++, a lambda expression is defined as a workaround. A pointer to this lambda is stored in the auto variable `impl` in the first statement. It accepts five parameters. The first is `count`, which is passed as a value and specifies the number of disks to be moved. This is followed by three parameters named `l`, `m` and `r`, which are references of type `Peg`. The fifth parameter, `impl`, is specified as `auto` and is passed as an rvalue reference. The use of `auto` turns the lambda function into a generic lambda function. Its great advantage is that it makes the explicit specification of a pointer to this lambda superfluous. Its specification as an rvalue reference allows the passing of any object, i.e. also a temporary and a constant object.

The underlying mechanism is the so-called *reference collapsing*, whose rules apply in templates and typedefs or using declarations that introduce type aliases (see (*Reference Declaration - Cppreference.Com* n.d.) for more information; the topic is also related to *perfect forwarding* and move semantics; more information on this can be found in (N. Josuttis 2020)). This parameter allows the lambda to call itself recursively, as is the case twice, e.g. by calling `impl(count - 1,m,l,r,impl)`. The reason for this is that `impl` stores a pointer to this lambda. It can therefore be regarded as the name of the lambda, which is only available in `move()`, but not yet in the lambda itself. The explicit passing of `impl` as a parameter makes it available in the implementation of the lambda and thus allows recursive calls to itself. This recursive call of the lambda assumes that its complete declaration is available. As the return type of a lambda cannot be specified at the beginning of the declaration, it must be specified at the end with the syntax `->`. Here `-> void` means that the lambda returns nothing. In this way, the declaration of the lambda is complete before it calls itself in its implementation.

A reader who is already familiar with lambda expressions in C++ might object that explicitly passing the pegs as parameters is superfluous, as they could be made available to the lambda by specifying the capture clause as `[&]`, for example. Although this is correct, it would lead to a mess, as the order of the parameters is changed when the lambda is called recursively. This could not be taken into account when using the peg parameters of `move()` directly.

It should be emphasized that many features of lambdas are used in this example without having been properly introduced beforehand. This is unfortunate insofar as lambdas are introduced in detail in the Section *Lambda Expression*. Everything that is done

with lambdas could also be done with with functions and function pointers. However, a modern programmer would definitely not do this, as lambdas are more concise and appropriate in this case.

Despite this small increase in desirable security for client programs using `move()`, the solution presented is of considerable size and still does not provide a meaningful representation of how disks are moved via pegs. This should be changed in the next step. Listing 178 shows a program that generates a visual representation of each move using character graphics.

Listing 178: `cia/HanoiVisualization.cpp` (excerpt)

```
1 // HanoiVisualization.cpp by Ulrich Eisenecker, July 12, 2025
2
3 #include <algorithm> // Because of std::max<>().
4 #include <iostream>
5 #include <stack>
6 #include <string>
7
22
23 void display(Peg a,Peg b,Peg c,NaturalNumber d = 0)
24 {
25     auto prepareLine { [](size_t disks,size_t maxHeight,Peg& p)
26     {
27         string s { };
28         if (p.size() < maxHeight)
29         {
30             if (maxHeight > 1)
31             {
32                 s += string(disks * 2 + 1, ' ');
33             }
34             else
35             {
36                 s += string(disks * 2, '_');
37                 s += ' ';
38             }
39         }
40         else
41         {
42             s += string((disks - p.top()), ' ');
43             s += string(p.top() * 2, 'X');
44             s += string((disks - p.top()) + 1, ' ');
45             p.pop();
46         }
47         return s;
48     } };
49
50     auto disks { a.size() + b.size() + c.size() };
51     auto maxHeight { max({ a.size(),b.size(),c.size() }) };
52     while (maxHeight)
53     {
54         cout << prepareLine(disks,maxHeight,a)
55              << prepareLine(disks,maxHeight,b)
56              << prepareLine(disks,maxHeight,c)
```

```

57         << endl;
58         --maxHeight;
59     }
60     if (d)
61     {
62         cout << "\nMoving disk " << d << ':' << std::endl;
63     }
64 }
65
66 void move(Peg& left,Peg& middle,Peg& right)
67 {
68     auto impl { [&](NaturalNumber count,Peg& l,Peg& m,Peg& r,auto&& impl) -> void
69     {
70         if (count > 0)
71         {
72             impl(count - 1,l,r,m,impl);
73             display(left,middle,right,l.top());
74             r.push(l.top());
75             l.pop();
76             impl(count - 1,m,l,r,impl);
77         }
78     }
79 };
80
81     impl(left.size(),left,middle,right,impl);
82 }
83
84 int main()
85 {
93     display(left,middle,right);
94 }

```

The program introduces the `display()` function, which creates a character-based visual representation of the pegs and the disks stacked on them. It is called twice, the first time in the lambda defined in `move()`, where it replaces all code that outputs sparse information about progress, and the second time at the end of `main()`, where it is used to display the final state of all pegs. It is important to note that the position of the call to `display()` in the lambda within `move()` is actually before the execution of the move. The state before the move is therefore displayed. The following explains how to create the impression that the pegs display the state after the move has been executed.

The `display()` function takes four parameters. The first three, `a`, `b` and `c`, are of type `Peg` and are passed as a value. This is important because they are changed during the execution of `display()` and changes to the original pegs are not desired. The different names were chosen to emphasize a context change when using them. However, it could be discussed whether the names `left`, `middle` and `right` would be more appropriate in terms of consistency. The last parameter named `d` is of type `NaturalNumber` and is specified with the default value 0. As can be seen at the end of `display()`, this parameter

is used to output information about which disk will be moved (next). If the value is 0, no output is made. This is relevant when `display()` is called at the end of the `main()` function to display the final state of the pegs.

The following explanations first skip the `lambda`, which is referred to in the `prepareLine` variable.

After defining and initializing the variable `prepareLine`, the auto variable `disks` is initialized with the total number of disks by adding the sizes of the pegs `a`, `b` and `c`. The maximum height of the pegs is then calculated using the function template `std::max<>()`, which is defined in the header file `<algorithm>`.

The lines representing the pegs with their disks must be created from top to bottom. Fortunately, this corresponds to the LIFO order of the pegs.

Now a while loop is executed, which runs as long as `maxHeight` is greater than 0. In its loop body, each character string resulting from the calls of `prepareLine()` with `a`, `b` and `c` is sent to `cout`, a line break is inserted by the output of `endl`. Then `maxHeight` is decremented so that the while loop is finally terminated.

The `lambda` to which `prepareLine` refers will now be explained in detail. It may be helpful to look at the output of the program for three disks (Figure 46).

It is evident that the number of lines used to display the pegs corresponds to the maximum number of disks on a peg. The smallest disk, size 1, is represented by two `X` characters. This allows for the vertical alignment of odd and even sized disks when using a fixed-width font, as is usually the case in terminal windows. In addition, there is exactly one extra space to the right of each peg to visually separate it from its neighbor. If a peg is empty, a number of underscores is printed that corresponds to twice the maximum size of a disk. This makes it easy to recognize an empty peg.

Admittedly, this is a primitive graphical representation, but its chronological order provides a basic insight into the moves. One could argue that a program with a graphical user interface that only visualizes the current move would be less valuable, as the history of the moves would be missing. Adding the history of the moves to a corresponding GUI program would involve considerable effort.

The `lambda` referred to by the variable `prepareLine` accepts three parameters: the number of disks, the maximum height of one of the three pegs, and a `Peg` named `p`. Passing `p` by reference is essential because it needs to be changed within the `lambda` function during repeated calls. To prevent unwanted changes to the original pegs, they are passed by value to `display()`.

The `lambda` defines the `std::string` variable `s`. First, it checks whether `p.size()` is less than `maxHeight`. If this is the case, another `if` statement is executed. This checks whether `maxHeight` is greater than 1. If this is the case, a constructor of `std::string` is called, which takes a number for the number of repetitions and a character to be repeated to create the new string. The number of repetitions is calculated as the number

Figure 46: Output of cia/HanoiVisualization.cpp for 3 disks

```

Number of disks: 3
  XX
XXXX
XXXXXX  _____

Moving disk 1:
  XXXX
XXXXXX  _____  XX

Moving disk 2:
XXXXXX  XXXX  XX

Moving disk 1:
      XX
XXXXXX  XXXX  _____

Moving disk 3:
      XX
_____  XXXX  XXXXXX

Moving disk 1:
  XX  XXXX  XXXXXX

Moving disk 2:
      XXXX
  XX  _____  XXXXXX

Moving disk 1:
      XX
      XXXX
_____  _____  XXXXXX

```


of disks (corresponding to the size of the largest disk) times 2, since each unit of disk size comprises two characters with a fixed width. The value 1 is added to separate two pegs. The character to be repeated is a space. The resulting temporary string is appended to `s`. If this is not the case, a new string is created that repeats the underscore number of disks times 2. Here, too, the resulting string is appended to `s`. A space is then appended to `s`.

If the condition of the first if statement evaluates as false, a temporary string consisting of `disks` minus `p.top()` repetitions of the space character is created and appended to `s`. Next, a temporary string representing the disk is created and appended to `s`. It consists of `p.top()` times 2 repetitions of the character `X`. Then another temporary string with spaces is created—now `disks` minus `p.top()` plus 1 (for the space as a separator)—and appended to `s`. Finally, the top element of `p` is removed. The last statement returns `s` as the result.

A very important change to the lambda defined within `move()` must be mentioned. In order to generate a correct visualization, the `&` in the capture clause of the lambda specifies that all local parameters available up to this point in `move()` are made directly accessible as references in the lambda. This allows the lambda to call `display()` with these parameters, i.e., `display(left,middle,right,l.top());`. This is necessary because `l`, `m`, and `r` are swapped when the lambda referenced by `impl` calls itself recursively. In the first recursive call, `right` and `middle` are swapped, and in the second recursive call, `middle` and `left` are swapped.

Some critical considerations should be made regarding the use of the container adapters `std::stack<>`, `std::queue<>`, and `std::priority_queue<>`. They offer exactly the functionality they promise, but nothing more. For example, the elements of a `stack<>` can only be accessed via the member functions `top()` and `pop()`. If, for whatever reason, it is desirable to iterate through the elements of a container adapter, this is not possible. Interestingly, other generic containers also offer functions that correspond to those of the container adapters. The following member functions are equivalent:

- `vector<>::push_back()` and `stack<>::push()`,
- `vector<>::back()` and `stack<>::top()`,
- `vector<>::pop_back()` and `stack<>::pop()`,
- `deque<>::push_back()` and `queue<>::push()`,
- `deque<>::front()` and `queue<>::front()` and
- `deque<>::pop_front()` and `queue<>::pop()`.

To mimic the functionality of a `std::priority_queue<>`, a `std::vector<>` or a `std::deque<>` can be sorted according to the desired criterion before accessing the first element.

For this reason, the choice of `stack<>`, `queue<>`, or `priority_queue<>` as a data structure should be carefully considered, especially if the functions of `vector<>` or `deque<>` are also required in a program. If choosing `vector<>` or `deque<>` helps to avoid writing code to adapt, for example, a `stack<>` to a `vector<>` and also prevents the inclusion of an additional header file, e.g., for `stack<>`, then not using `stack<>` may be a valid decision.

Since C++23, there are also adapted versions for sets and maps whose elements are stored in adjacent memory spaces, enabling faster sequential access by minimizing cache trashing. These are not explained further here.

4.5.4.2. Views for Containers

There are also *views* that can adapt containers. Since C++20 `std::span<>` can represent a contiguous sequence of elements in a container. For example, if a container has 10 elements, a span can be used to create a view that includes the five elements from index positions 2 to 6.

The `std::mdspan<>` view introduced in C++23 is very interesting. C and C++ only allow one-dimensional data structures. The usual workaround, e.g., for creating matrix-like data structures, is to nest data structures, e.g., a `vector<>` instantiated with `vector<>` as elements. Furthermore, it is impossible to access such a nested element with an index operator with more than one comma-separated index values, e.g., `[j,k]`. All this changes with `mdspan<>`. It allows the creation of any multidimensional view for a data structure and provides index operators that can handle the required number of index values.

This is illustrated using an abstract example in Listing 179.

Listing 179: cia/MultiDimensions.cpp (excerpt)

```

1 // MultiDimensions.cpp by Ulrich Eisenecker, July 14, 2025
2
3 #include <iostream>
4 #include <mdspan>
5 #include <numeric> // Because of iota<>().
6 #include <vector>
7
8 using namespace std;
9
10 int main()
11 {
12     cout << "1..24 as one dimension with 24 elements\n";
13     vector<unsigned int> oneDimensional(24);
14     iota(begin(oneDimensional), end(oneDimensional), 1);
15     for (size_t i { 0 }; i < size(oneDimensional); ++i)
16     {
17         cout << " i" << i << ": " << oneDimensional[i];
18     }
19
20     cout << "\n\n1..24 as two dimensions with 4 rows and 6 columns\n";
21     auto twoDimensional { mdspan(oneDimensional.data(), 4, 6) };

```

```
22     for (size_t i { 0 }; i < size(oneDimensional) / 6; ++i)
23     {
24         for (size_t j { 0 }; j < size(oneDimensional) / 4; ++j)
25         {
26             cout << " i" << i << "|j" << j << ": "
27                 << twoDimensional[i,j];
28         }
29         cout << endl;
30     }
31
32     cout << "\n\n1..24 as three dimensions with width = 2, "
33          << "height = 3 and depth = 4\n";
34     auto threeDimensional { mdspan(oneDimensional.data(),2,3,4) };
35     for (size_t i { 0 }; i < size(oneDimensional) / 3 / 4; ++i)
36     {
37         for (size_t j { 0 }; j < size(oneDimensional) / 2 / 4; ++j)
38         {
39             for (size_t k { 0 }; k < size(oneDimensional) / 2 / 3; ++k)
40             {
41                 cout << " i" << i << "|j" << j << "|k" << k << ": "
42                     << threeDimensional[i,j,k];
43             }
44             cout << endl;
45         }
46         cout << endl;
47     }
48     cout << endl;
49 }
```

First, a vector for unsigned `int` is instantiated and created with 24 elements. Then, the algorithm `std::iota<>()` is called to assign the elements in ascending order with the values 1 to 24. Next, the 24 elements of `oneDimensional` are sent to the standard output.

Then, a view called `twoDimensional` is created, which accesses the elements of the underlying vector `oneDimensional` as if it were a matrix with 4 rows and 6 columns. When accessing the elements for output, the index operator is used with two indices. It should be noted that no range check is applied. If range checking is desired, the `at()` member function should be used instead.

In the last third of the `main()` function, a three-dimensional view is created for `oneDimensional`. Now the index operator is called with three index values.

In this example program literal values are used to specify the cardinality of the dimensions of a view. Instead, dynamically calculated values also can be used, but this is not shown here.

The program should be compiled with the `std=c++23` option. Currently, it can only be compiled with `clang++ 20.1.8`; `g++ 15.1.0` reports an error due to the missing header file `<mdspan>`.

4.5.5. Closing Remarks on Containers, Iterators and Algorithms

To facilitate generic programming, the containers in the C++ standard library share some common features. Most of them define the following member types, with a few exceptions indicated in parentheses:

- `value_type`: The type of elements stored in the container.
- `reference`: A reference of the type of the elements, i.e. `value_type&`.
- `const_reference`: A reference to `const` of the type of the elements, i.e. `const value_type&`.
- `iterator`: The type of an iterator that can be used to read or modify an element (not available for container adaptors).
- `const_iterator`: The type of an iterator that can be used to read an element but not to modify it (not available for container adaptors).
- `difference_type`: A signed integer type representing the distance between iterators (not available for container adaptors).
- `size_type`: An unsigned integer type representing the number of elements of a container.

To access a member type, the instantiated container type must be followed by the scope operator, `::`, and the member type name, e.g. `std::vector<int>::value_type` (which should evaluate to `int`). To access a member type of a container variable, the type of the container variable must be first determined by applying the `decltype` operator, e.g. `vector<char> vc; decltype(vc)::value_type` (which should evaluate to `char`).

The generic containers—not the adaptors—provide the following member functions:

- Standard constructor, copy constructor, destructor and assignment operator.
- `empty()`: Returns `true` if the container has no elements, `false` otherwise.
- `size()`: Gives the number of elements in the container.
- `max_size()`: Returns the maximum number of elements that the container can hold.
- `swap()`: Exchanges the contents of the container with another container of the same type.
- `clear()`: Removes all elements of the container; afterwards, `empty()` returns `true`.
- `begin()`, `end()`/`cbegin()`, `cend()`: Returns (read-write/read) iterators for the beginning and end (not available for container adaptors).
- `rbegin()`, `rend()`/`crbegin()`, `crend()`: returns (read-write/read) reverse iterators (not available for `std::forward_list<>` and container adaptors).
- Comparison operators, namely `==`, `!=`, `<`, `<=`, `>` as well as `>=` before C++20, and `==` as well as `<=` since C++20 (the remaining comparison operators are automatically generated by the compiler if necessary when the latter two are present); they compare two containers of the same type lexicographically.

There are also free function templates that can be instantiated and called for generic containers. They all accept a container variable as a parameter, which is denoted by `c` in the following list:

- `std::begin(c)`, `std::end(c)`: Return the result of calling `c.begin()` and `c.end()` respectively.
- `std::cbegin(c)`, `std::cend(c)`: Return the result of calling `c.cbegin()` and `c.cend()`.
- `std::rbegin(c)`, `std::rend(c)` `std::crbegin(c)`, `std::crend(c)`: Call the corresponding member function of `c`, which returns a reverse iterator.
- `std::empty(c)`: Returns `true` if `c` is empty, otherwise `false`.
- `std::size(c)`: Returns the size of a container as an unsigned integer value.
- `std::ssize(c)`: Returns the size of a container as a signed integer value.

Interestingly, there does not seem to be a free function template for removing all elements from a container (`clear<>()`) or determining the maximum number of elements in a container (`max_size<>()`).

If available, a free function template calls the corresponding member function of a container—provided that it is defined by the container. Otherwise, the function template can execute other code to accomplish the task. Accordingly, calling the free function template always invokes the optimal version of an algorithm—be it the general or the specific version of a container. The use of a free function template also offers a customization point for library developers (and sometimes for library users). In addition, it supports a more functional programming style. While it was formerly recommended to call the container-specific member function of an algorithm to achieve optimal performance, this is no longer the case. Today, the free function template typically retrieves a specialized member function and calls it if available. Cases where this is not yet implemented will become increasingly rare over time. For this reason, it is generally recommended to use free function templates.

The difference between mutating and non-mutating algorithms could also be explained in more detail. This is important insofar because using of a mutating algorithm invalidates the previously acquired iterators for a container (see Section *Use of Algorithms From the C++ Standard Library*). However, this problem is mitigated when using the ranges versions of algorithms, as these automatically use up-to-date iterators. A similar argument applies to the use of views, since the positions and numbers of elements are used instead of iterators. Since the use of views and ranges reduces the need to deal directly with iterators in many usage scenarios, this aspect of algorithms and iterators is not covered in detail.

Further information is not presented here. Since the C++ standard library contains many generic containers and algorithms, it does not make sense to explain each one in detail. In practice, it is strongly recommended to carefully consider what requirements a con-

tainer or algorithm must meet for a specific purpose, and then search for a suitable versions and learn about possible candidates in order to reliably assess, which one is most suitable and how it is used.

4.6. Descriptive Statistics—A Case Study

4.6.1. (Pseudo-)Random Numbers

This section briefly introduces pseudo-random numbers and generators for their creation in C++. Later, pseudo-random number sequences will be generated for benchmarking purposes.

It is not really possible to judge whether a single number is a random number or not. This can only be determined retrospectively for a (larger) sequence of random numbers. If the next number in a sequence of numbers cannot be predicted any better than by pure chance, there is a high probability that it is a *random number*. Generating random numbers is laborious, e.g., by rolling dice for a long time, or requires complex equipment, e.g., by measuring the decay of radioactive substances. Without special hardware, a classic computer cannot generate true random numbers (for more information, see (*Random Number Generation 2025*) for more information).

This makes it necessary to use *pseudo-random numbers* as a substitute. There are several mathematical methods for generating sequences of numbers that appear more or less random. One such method is called a *pseudo-random number generator* or, in the context of pseudo-random numbers, simply an *engine*. The C++ standard library offers a selection of such engines (see (*Pseudo-Random Number Generation - Cppreference.Com* n.d.)).

Listing 180 shows the use of `mt19937`. This engine uses the *Mersenne Twister* algorithm. It is popular in many programming languages and associated libraries and generates pseudo-random numbers of good quality (see (*Random Number Generation 2025*)), but it is not suitable for cryptographic purposes.

Listing 180: `cia/PRN_Generator`

```

1 // PRN_Generator.cpp by Ulrich Eisenecker, May 2, 2025
2
3 #include <iostream>
4 #include <random>
5
6 using namespace std;
7
8 int main()
9 {
10     cout << "Mersenne Twister Engine 32 as pseudo random number generator" <<
11         endl;
12     mt19937 generator { };
13     for (int i { }; i < 5; ++i)

```

```
13 | {  
14 |     cout << generator() << endl;  
15 | }  
16 | }
```

The `main()` function declares the variable `generator` of type `mt19937`. The empty initializer list could have been omitted. It serves as a hint that default values are used as arguments for the constructor call. This will be changed in the following. The `for` loop calls the function call operator five times for `generator`. Each call generates a new pseudo random number as an unsigned integer of 32 bit size and sends it to `cout`.

When a pseudo-random number generator is instantiated, e.g. `mt19937` (defined in the `<random>` header file), it always generates the same sequence of pseudo-random numbers. Depending on the specific engine, the generated numbers appear more or less random, and more or less numbers can be generated before repetitions occur.

It is desirable to always generate the same sequence of pseudo-random numbers if the calculations dependent on them must be reproducible. Otherwise, a generator can be initialized with a so-called *seed value*, which itself can be generated as a (pseudo-)random number by a so-called *random number generator*. The C++ standard library defines such a random generator called `random_device` in the header file `<random>` (see (*std::random_device* - *Cppreference.Com* n.d.)). Whether this device generates true random numbers using a hardware device or pseudo-random numbers depends on the implementation. Even if it generates pseudo-random numbers, these are usually based on a specific state, such as the current time in milliseconds, so that each run of the generator is highly likely to produce a different sequence of pseudo-random numbers.

Listing 181 shows the use of `random_device` and outputs some of its properties.

Listing 181: `cia/RandomDevice.cpp`

```
1 | // RandomDevice.cpp by Ulrich Eisenecker, May 2, 2025  
2 |  
3 | #include <iostream>  
4 | #include <random>  
5 |  
6 | using namespace std;  
7 |  
8 | int main()  
9 | {  
10 |     cout << "Directly using a (pseudo) random number device." << endl;  
11 |     random_device rnd_device;  
12 |     cout << "Entropy = \t" << rnd_device.entropy()  
13 |         << "\nMinval = \t" << rnd_device.min()  
14 |         << "\nMaxval = \t" << rnd_device.max()  
15 |         << endl;  
16 |     for (int i { }; i < 5; ++i)  
17 |     {  
18 |         cout << rnd_device() << endl;
```

```

19 |     }
20 | }

```

It declares a `random_device` named `rnd_device` by calling its standard constructor, as indicated by the missing initializer list. Next, three member functions are called for `rnd_device` and their results are sent to `cout`. The first function returns an estimate of entropy (*entropy* is a measure of disorder or randomness in a system.). No detailed information is provided for this member function in (*std::random_device* - *Cppreference.Com* n.d.). The estimated entropy is related to the number of truly random numbers that can be obtained within a certain period of time.

If too many numbers are generated within a certain period of time, they become less random, i.e., the entropy pool is exhausted because the operating system does not have enough opportunities to collect entropy (or randomness) from various sources such as mouse events, keyboard events, network traffic, etc. For this reason, only a single value from a random device is usually requested for the initialization of a pseudo-random number generator. After that, a large number of pseudo-random numbers can be generated quickly. The member function `min()` returns the smallest random number that can be generated, and `max()` returns the largest random number that can be generated.

Listing 182 shows the use of a seed value generated by a `random_device` exemplar to initialize an `mt19937` generator. When the generator is instantiated, a temporary exemplar of `random_device` is created using list initialization, and the function call operator is called for the resulting temporary random device. When the resulting program is run several times, it should generate different sequences of pseudo-random numbers.

Listing 182: cia/SeedValue.cpp

```

1 // SeedValue.cpp by Ulrich Eisenecker, May 2, 2025
2
3 #include <iostream>
4 #include <random>
5
6 using namespace std;
7
8 int main()
9 {
10     cout << "Pseudo random number generator with seed value" << endl;
11     mt19937 generator { random_device { }() }; // Creating temporary random
12     for (int i { }; i < 5; ++i)                // device and obtaining a value
13     {                                           // as a seed value.
14         cout << generator() << endl;
15     }
16 }

```


A random device and a pseudo-random number generator generate (integer) numbers that fall within a specific range, which is ultimately limited by the range of the random number type. These numbers are usually distributed uniformly within their range. In many use cases, specific distributions are required, such as the *normal distribution* (also known as the *Gaussian distribution* or *bell curve*).

The C++ standard library offers several templates that generate specific distributions (see (C++ *Named Requirements: RandomNumberDistribution (since C++11)* - *Cppreference.Com* n.d.)), e.g. `normal_distribution`. A distribution exemplar can be created with parameters that characterize the specific distribution. If constructor parameters can be omitted, suitable default values are chosen. The function call operator, `()`, of a distribution exemplar requires a generator whose type at least meets the requirements of a *UniformRandomBitGenerator* (a so-called *named concept* in the C++ standard, which means that it is not implemented as such) and returns a single conforming value.

Listing 183 illustrates the use of a distribution, in this case `normal_distribution`. This time, `mt19937_64` is used as the type of the generator exemplar. Its function call operator returns an unsigned integer with a size of 64 bits. Next, `normal_distribution` is used to declare `bellCurve`. Since the constructor parameters are omitted, as shown by the empty initializer list, 0.0 is chosen as the default value for the mean value and 1.0 as the default value for the standard deviation. When the function call operator is called in the for loop, `generator` is provided as an argument. Each time the compiled program is executed, the same sequence of pseudo-random numbers is generated because the generator is not initialized with a seed value when it is created.

Listing 183: `cia/Distribution.cpp`

```
1 // Distribution.cpp by Ulrich Eisenecker, May 4, 2025
2
3 #include <iostream>
4 #include <random>
5
6 using namespace std;
7
8 int main()
9 {
10     cout << "Using normal_distribution without seed value." << endl;
11     mt19937_64 generator { };
12     normal_distribution bellCurve { };
13     for (int i { }; i < 5; ++i)
14     {
15         cout << bellCurve(generator) << endl;
16     }
17 }
```

The last code example in this section (Listing 184) shows how a sequence of pseudo-random numbers is generated for a bell curve that differs each time the compiled program is executed.

Listing 184: cia/GermanFemales.cpp

```

1 // GermanFemales.cpp by Ulrich Eisenecker, May 4, 2025
2
3 #include <iostream>
4 #include <random>
5
6 using namespace std;
7
8 int main()
9 {
10     cout << "Distribution of german females, taken from\n"
11           << "https://en.wikipedia.org/wiki/Average_human_height_by_country."
12           << endl;
13     mt19937 generator { random_device {}() };
14     normal_distribution bellCurve { 165.0, 6.7 };
15     for (int i { }; i < 5; ++i)
16     {
17         cout << bellCurve(generator) << endl;
18     }
19 }

```

A normal distribution for the height of German women in cm is generated based on a sample from 2014–2017 with a mean of 165 cm and a standard deviation of 6.7 cm (according to (*Average Human Height by Country* 2025)). As in Listing 182, the `mt19937` generator is seeded by calling the `()` operator of a temporary `random_device` exemplar. In the `for` loop, the `()` operator is called for the `bellCurve` distribution, passing `generator` as an argument. Repeated execution of the compiled program is highly likely to generate different sequences of height values.

When the program is executed, the five height values are output with an accuracy of hundredths of a millimeter, i.e. with three decimal places. This is neither realistic nor useful, as a person's height fluctuates slightly over the course of 24 hours. For example, a person is usually slightly taller immediately after getting up in the morning than in the evening after standing for a long time. Furthermore, a measurement of such accuracy suggests a degree of precision that is not possible when measuring human height. For this reason, the height measurements are truncated to full centimeters. Since they are not rounded commercially, their mean value will be slightly below 165 cm.

The above explanations should be sufficient to encourage self-study of (pseudo-)random number generators and possible distributions.

4.6.2. Descriptive Statistics as an Application Example

In the following, benchmarking will be used out to compare the efficiency of different container templates and algorithms for calculating basic statistical parameters, which requires own implementations of statistical calculations. Therefore, none of the existing statistical libraries for C++ is used.

In addition to unit tests, *Catch2* also offers simple but sufficient benchmarking options. Thus, no separate implementation of benchmarking is required.

The most well known statistical parameters are:

- *Mean*: The arithmetic mean of the values in a data set (*location parameter*).
- *Standard deviation*: Parameter that provides information about the dispersion of the data; in a normal distribution, the range between mean minus and plus one standard deviation contains roughly 68% of the data (*dispersion parameter*); only the uncorrected standard deviation is used below.
- *Median*: The value in the middle of a data set, i.e. there are 50% of the data is to the left of it and 50% to the right of it (*location parameter*).
- *Mode(s)*: The value(s) in a data set that occur most frequently (*location parameter(s)*); there may be several values with the same highest frequency.
- *Minimum*: The lowest value in a data set.
- *Maximum*: The highest value in a data set.

Readers who are unfamiliar with descriptive statistics or who are interested in learning more about this field are referred to the numerous resources available on the Internet or in textbooks. It should be emphasized that it does not make sense to develop new programs or libraries for statistical analysis. Instead, one of the existing tools should be used, e.g., *R*.

Table 18 contains the parameter names and, where applicable, the mathematical formulas used to calculate them. It also includes *variance*, which was not mentioned above because its definition simplifies the calculation of standard deviation. The calculation of the mode or modes and the minimum and maximum is not included because it is more difficult to specify these mathematically than to implement a function that determines them.

Next, a program will be presented in small chunks that generates pseudo-random data for testing, determines modes, calculates and returns all of the above statistical parameters, and outputs the data and statistical parameters. All functions are implemented generically. For the generation of random data, this means that the numerical type of the values to be generated, the type of the container containing the elements, a distribution, and a random number generator can be freely selected.

Table 18: Parameter names and formulas of frequency statistics

Name	Smy-bol	Formula
Size of data set	n	
Data ele-ment	x	
Fre-quency	f	$f_1 + \dots f_k = n$
Mean value	x	$\bar{x} = \frac{\sum_{i=1}^n x_i}{n} \equiv \frac{\sum_{i=1}^n x_i \cdot f_i}{n}$
Vari-ance	s^2	$s^2 = \frac{\sum_{i=1}^n (x_i - \bar{x})^2}{n} \equiv \frac{\sum_{i=1}^n x_i^2}{n} - \bar{x}^2$
Stan-dard devia-tion	s	$s = \sqrt{s^2}$
Me-dian	M	$M = \begin{cases} x_{m+1} & \text{for odd } n = 2m + 1 \\ \frac{x_m + x_{m+1}}{2} & \text{for even } n = 2m \end{cases}, \text{ provided all } x_i \text{ are sorted}$

To determine the modes, a range of individual elements sorted in ascending order is required. It should be possible to calculate the statistical parameters for any range of numerical values, whereby a specific accuracy should be used for floating point types for the parameters mean, standard deviation, and median. If this accuracy is not explicitly specified, double should be used as the default.

The data should be output in CSV format with a maximum of 8 values per line, with the values separated by tab characters. The statistical parameters should be output individually per line, first the name of the parameter, followed by a tab character and then the value. If there is more than one mode value, the values should be output separated by tab characters in a single line.

Overall, the statistical parameters should also be output in CSV format. This allows them to be easily imported into spreadsheet programs or converted into tables in word processing programs. Finally, there should be a `main()` function that provides an example of the height of German women in cm, in which all of the aforementioned functions are called once (see Section *(Pseudo-)Random Numbers*).

Listing 185 shows the function template that generates the random data.

Listing 185: `generateRandomData<>()` (excerpt from `cia/DescriptiveStatistics.cpp`)

```
15 | template <typename ELEMENTTYPE = double,  
16 |         template <class T, class Allocator = std::allocator<T>>  
17 |         class CONTAINER = std::vector,  
18 |         class DISTRIBUTION,  
19 |         class GENERATOR  
20 |         >  
21 | [[nodiscard]] auto generateRandomData(DISTRIBUTION&& distribution,  
22 |                                     GENERATOR&& generator,  
23 |                                     std::size_t n)  
24 | /* Passing distribution and generator as rvalue references allows  
25 |  * to use variable and temporaries, but not const objects.  
26 |  */  
27 | {  
28 |     CONTAINER<ELEMENTTYPE> result(n);  
29 |     std::ranges::generate(result, [&] () { return distribution(generator); });  
30 |     return result;  
31 | }
```

The template defines four template parameters. `ELEMENTTYPE` is the type of the individual values to be generated. This allows specifying whether the type of an individual value should be a small unsigned integer type or a floating point type with maximum precision. The double type is set as the default for `ELEMENTTYPE`. Restrictions that check whether valid numeric types are provided by the client programmer have been omitted.

The second parameter, `CONTAINER`, is a template template parameter. The client programmer should specify a container type that supports sequential access to individual elements in both forward and backward directions. The container template `std::vector` is specified as the default for `CONTAINER`. Again, no corresponding constraints have been added.

Next, the distribution type and generator type for pseudo-random numbers are specified as type template parameters, again without specifying any constraints. It is up to the reader to specify appropriate constraints.

Only three parameters are passed in the function header, namely a distribution exemplar, a generator exemplar, and the number of data items to be generated, here `n`. `ELEMENTTYPE` and `CONTAINER` are not passed as function parameters. If they are not explicitly specified in the instantiation of the function template, the provided default values `double` and `std::vector` are automatically chosen. Both parameters are passed as rvalue references, which triggers reference collapsing. Consequently, distribution and generator exemplars can be passed by reference. It is also permissible to pass temporary exemplars, which are created ad hoc when the function is called. The use of `const` exemplars is not possible.

The implementation is relatively simple. The `CONTAINER` template is instantiated with `ELEMENTTYPE` to declare a local variable called `result`. No initializer list is used for initialization, but rather a classic constructor call. The parameter `n`, the third parameter specified for `generateRandomData<>()`, is passed to it. As a consequence, a container with `n` pre-allocated elements is created. For small `n`, the effect may be negligible. However, for large `n`, this can result in significant time savings, since generic containers such as `std::vector<>()` in particular do not have to repeatedly reallocate memory and copy elements. All of this is done only once. It should be emphasized that only suitable container templates should be used for instantiating the function template. Invalid generic containers include `std::multiset<>` or `std::map<>`. Using an invalid container can lead to cryptic or long error messages, as no constraints are specified for the template parameters.

Next, the function template `generate<>()` from the ranges library is called. The first parameter is `result` and the second parameter is a lambda expression. This lambda is applied to each element of `result`. Here, the capture clause contains an ampersand, `[&]`, which makes all local variables of `generateRandomData<>()` that have been declared so far available for reading and writing within the lambda. Therefore, distribution and generator are accessible within the lambda. Its return value is implicitly determined by its return statement, which returns the call to the function operator of distribution with generator as an argument. The `generate<>()` function template assigns the resulting value internally to each individual element of `result`.

The last statement returns `result`. Of course, the return type of `generateRandomData<>()` could have been specified explicitly as `CONTAINER<ELEMENTTYPE>`. Specifying it with the keyword `auto` is simpler and would automatically adjust if, for whatever reason, the type of the return value were changed within the function.

The function template `modes<>()` was extracted from `calculateParameters<>()` and implemented as a separate function (Listing 185). This reduces the number of lines of code in `calculateParameters<>()` and improves the comprehensibility of the implementation. The only redundant part of the code in `mode<>()` is the initialization of the auto variable `n` by calling `size(data)`, since a variable with the same name and meaning already exists in the calling function template `calculateParameters<>()`, but is not passed as a parameter to `modes<>()`. Please note that the auto parameter `data` is passed as an rvalue reference, which allows temporary variables and variables to be passed as actual parameters.

Listing 186: `modes<>()` (excerpt from `cia/DescriptiveStatistics.cpp`)

```
33 | auto modes(std::ranges::forward_range auto&& data)
34 | /* This function assumes that the elements of
35 |  * data are sorted in ascending order. This is
36 |  * not checked by the function itself!
37 |  */
38 | {
39 |     auto n { size(data) };
40 |     auto recentValue { data[0] };
41 |     std::size_t frequency { },
42 |         modeFrequency { };
43 |     std::set<decltype(recentValue)> modeValues;
44 |     for (std::size_t counter { }; auto& v : data)
45 |     {
46 |         ++counter;
47 |         if (v == recentValue)
48 |         {
49 |             ++frequency;
50 |             if (frequency > modeFrequency)
51 |             {
52 |                 modeFrequency = frequency;
53 |                 modeValues.clear();
54 |                 modeValues.insert(recentValue);
55 |             }
56 |             else if (frequency == modeFrequency)
57 |             {
58 |                 if (modeValues.contains(recentValue) == false)
59 |                 {
60 |                     modeValues.insert(recentValue);
61 |                 }
62 |             }
63 |         }
64 |         else if (counter < n)
65 |         {
```

```

66         recentValue = v;
67         frequency = 1;
68     }
69 }
70 return std::make_pair(modeFrequency, modeValues);
71 }

```

The auto variable `recentValue` is initialized with `data[0]`. Since no range check is performed when accessing a container element with the index operator, this code may or may not throw an exception. Therefore, this function should not be called for empty containers, which is not checked in its implementation. It should be noted that the auto declaration of the variable `recentValue` removes the specification of `data[0]` as a reference. Since a range does not own the elements, these are usually references or references to `const`.

Next, the variables `frequency` and `modeFrequency` are zero-initialized. In addition, the empty set `modeValues` is declared. To obtain the type for which `std::set<>` is to be instantiated, the `decltype` operator is applied to `recentValue`.

If the `decltype` operator were applied directly to `data[0]`, the result would be a reference to the element type. However, this would not be suitable for instantiating `std::set<>`. Alternatively, the template metafunction `std::remove_reference<>::type` or the template variable `std::remove_reference_t<>` or `std::remove_cvref<>::type` or `std::remove_cvref_t<>` (all contained in the header file `<type_traits>`) could be applied to the result of the `decltype` operator evaluation. However, this was not done for reasons of clarity.

Now, a range-based for loop is started, which manages an additional variable called `counter`. The loop variable `v` is an auto reference and iterates through all elements of `data`. At the beginning of each iteration, `counter` is incremented by 1. In each iteration, a check is made to see if `v` is equal to `recentValue`. If this is the case, the variable `frequency` is incremented by 1. It should not be confusing that `recentValue` is initialized with the first element of `data` and the comparison initially compares a copy of the first element with the first element. Since `frequency` is initially 0, `frequency` is correctly set to 1 after this comparison.

The following nested if statement checks whether `frequency` is greater than `modeFrequency`. If this is true, `modeFrequency` is assigned the value of `frequency`, `modeValues` is cleared, and `recentValue` is inserted as the new and only element in `modeValues`.

Why is that? The variable `modeFrequency` contains the highest frequency of a value found so far. There may be more than one value with the same `modeFrequency`. However, if the value `recentValue` occurs one more time, this frequency must become the new `modeFrequency`. Since it is now the highest `modeFrequency`, all values stored in `modeValues` have a lower frequency than `recentValue`. Therefore, `modeValues` is deleted and the value `recentValue` is inserted as the only mode value.

A second possibility is checked in the else branch, which tests whether frequency and modeFrequency are equal. If this is the case, a check is made to see whether modeValues does *not* contain recentValue. If this is the case, recentValue is inserted into modeValues.

Now the focus is on the else branch of the top-level if statement. Here, the program checks whether counter is less than n. If so, recentValue is assigned the value v and frequency is assigned the value 1. Then the next loop iteration begins.

Finally, a pair is created by calling `std::make_pair<>()` with modeFrequencies and modeValues as arguments, which is returned as the result of the function. This function result can ideally be captured by *structured binding*, e.g., `auto [modeFrequency, modeValues] = modes(someData);`. Subsequently, modeFrequency indicates the frequency of the mode values, and `size(modeValues)` indicates how many modes there are. Each element of modeValues corresponds to a value from someData that is a mode.

The function template `modes<>()` illustrates that a function can, in a sense, return more than a single value as its result. Of course, `std::make_pair<>()` combines the two results into a single value. However, with structured binding, it is possible to capture the two values in different variables with meaningful names in an almost intuitive way. Alternatively, a new data type could have been defined with one data element representing the mode frequency and another representing the mode values. However, this would involve extra effort and introduce another entity into the program that would need to be managed. In the following, the idea of returning a pair is extended to returning a tuple.

The function template `calculateParameters<>()` calculates—as its name suggests—all of the above mentioned parameters of descriptive statistics (Listing 187).

Listing 187: `calculateParameters<>()` (excerpt from `cia/DescriptiveStatistics.cpp`)

```

73 template <typename PRECISION = double>
74 [[nodiscard]] auto calculateParameters(std::ranges::forward_range auto&& data)
75 {
76     // IMPORTANT: accumulate<>() is not yet available in ranges!
77     auto n { size(data) };
78     PRECISION sum { std::accumulate(data.begin(), data.end(), PRECISION { 0.0 }) };
79     PRECISION mean { sum / n };
80     PRECISION stdDev
81     { std::sqrt(std::accumulate(data.begin(), data.end(), PRECISION { 0.0 },
82     [=](auto sum, auto val)
83     { return sum + (val - mean) * (val - mean); } ) / n)
84     };
85     std::ranges::sort(data);
86     auto median { n % 2 ? data[n / 2] :
87     (data[n / 2 - 1] + data[n / 2]) /
88     PRECISION { 2.0 } };
89     auto [modeFrequency, modeValues] = modes(data);
90     using namespace std::string_literals;
91     return std::make_tuple(std::make_pair("n"s, n),

```

```

92         std::make_pair("mean value"s,mean),
93         std::make_pair("standard deviation"s,stdDev),
94         std::make_pair("median"s,median),
95         std::make_pair("mode frequency"s,modeFrequency),
96         std::make_pair("mode value(s)"s,modeValues),
97         std::make_pair("minimal value"s,*(data.begin())),
98         std::make_pair("maximal value"s,*(data.rbegin())));
99     }

```

The first template parameter, `PRECISION`, is explicitly specified with `double` as the default type. The type used for `PRECISION` should be a floating-point type that is used by the function to perform floating-point operations. Note that despite specifying `float`, the operations are performed with double precision. As a result, each floating-point operation first converts a `float` to a `double`, while the final result of the operation—when assigned to a `float` variable—is converted back from `double` to `float` (see (U. W. Eisenecker 2022), Section 3.1.1.8 *Floating Point Types*). Please note that no constraints are specified for `PRECISION`. Due to the presence of a default type, `calculateParameters<>()` can be called like a normal function. If a different type is to be used, it must be explicitly specified when instantiating the template, e.g., `calculateParameters<long double>(...)`.

Since the `auto` parameter is passed as an rvalue reference, it is possible to use a variable or a temporary value as the actual parameter.

Since it would be pointless to call the function without using its `auto` result, the attribute `[[nodiscard]]` is prefixed to it.

In the following, the parameters of descriptive statistics are calculated in exactly the same order as they are listed in Table 18. The exception is variance, which is calculated as part of the expression that calculates the standard deviation. Therefore, it only exists temporarily and is not included in the returned result.

Since the number of elements contained in `data` is needed repeatedly, it is stored in the `auto` variable `n` in the first statement. The next statement calculates the sum of all values contained in `data`. The generic function `std::accumulate<>()` is used for this purpose. It performs a left-fold operation, which sums the elements in the range specified by `begin(data)` and `end(data)` with the start value `PRECISION { 0.0 }`.

Unfortunately, `std::ranges::accumulate<>()` is not (yet) available in the C++ standard library. If this were the case, it would simplify the code a little. As an alternative, `std::ranges::fold_left<>()` can be used as of C++23, but the summation of the elements is not a default, i.e. a lambda expression must always be passed as the third parameter. These details are of some historical interest, as `std::accumulate<>()` was inspired by functional programming when it became part of the *STL*. By specifying the range from `end(container)` to `begin(container)`, `std::accumulate<>()` becomes a right fold. When calling an algorithm of the ranges library, exactly one range is passed as the only parameter. So if `std::ranges::accumulate<>()` is to be used as a right fold, the passed range must first be reversed. It would therefore perhaps be more intu-

itive and simpler to offer only `std::ranges::fold_left<>()` and `std::ranges::fold_right<>()` instead of combining both purposes in `std::accumulate<>()`. Programming language design is not an easy profession and it will be interesting to see whether `std::ranges::accumulate<>()` will ultimately make it into the C++ standard library and what semantics it will then have.

The following statement calculates the mean value according to the formula given in Table 18.

Next, the standard deviation is calculated according to the first formula for the variance and then the square root is taken from this. The `accumulate<>()` algorithm is also used here. The difference in the calculation of the sum lies in the explicit definition of a lambda expression as the fourth parameter. Its capture clause only specifies an equals sign, `[=]`, which means that each local variable of the outer function is only made available for reading purposes. This is necessary because the mean value is to be subtracted from each data value. Therefore, the mean value must be accessible in the lambda function. The statement of the lambda body returns `sum` plus the product of `(val - mean)` with itself. This somewhat complicated expression is necessary because there is no operator or function in C++ that calculates the square. The result of the call to `accumulate<>()` is first divided by `n` before the `sqrt()` function is applied to it.

The following operations assume that the data values are sorted in ascending order. This is done by calling `std::ranges::sort(data)`.

The median is now calculated exactly as described in the corresponding formula in Table 18.

The auto variables `modeFrequency` and `modeValues` are then initialized with the result of the call to `modes(data)` via a structured binding. As it is explained in detail how `modes()` is implemented, it is not discussed further here.

Next, the namespace `std::string_literals` is imported. This is necessary to define string literals. Actually, the import of namespaces in header files should be strictly avoided (well, *DescriptiveStatistics.cpp* is not a header file, but most of its functions are later moved to a header file), but here it does no harm, as the effect of the import ends at the closing curly bracket of the function definition.

What follows is a complex return statement. A tuple is created with elements that represent both names and values of statistical parameters. Each element of a tuple is an instantiation of `std::pair<>`. The first component of each pair is always a string literal with the name of the corresponding statistical parameter. The second component is the value of the statistical parameter itself. With one exception, this is always a scalar, i.e. a single value. The exception is the mode values. Since there can be more than one mode, in principle a set is returned that contains all mode values. The advantage of using a set is that the mode values are also sorted in ascending order when iterating over the set elements. The last two statistical parameters became available by sorting the data. The first element of data contains the minimum value. The last element of data is the max-

imum value. To access this, `end()` cannot be used, as this results in an iterator exactly right from the last element. Decrementing the result of `end()` can also be problematic and is not elegant. Therefore, `rbegin()` is used. It is an abbreviation for *reverse begin* and returns an iterator that points exactly to the last element (if it exists).

The motivation for returning pairs as elements of a tuple is that the code should be resistant to changes in the order in which the pairs are specified. When a parameter is output, the first component is output first, followed by the second component. In this way, a change in the order of the components of the tuple is visible in the output, but each output is still correctly labeled.

The purpose of `outputData<>()` is to output the values of its data parameter in a CSV format (Listing 188). It returns nothing and takes an ostream reference called `os` as the first parameter. The second parameter is the auto parameter `data`, which is passed as an rvalue reference.

Listing 188: `outputData<>()` (excerpt from `cia/DescriptiveStatistics.cpp`)

```

101 void outputData(std::ostream& os, std::ranges::forward_range auto&& data)
102 {
103     constexpr std::size_t valuesPerLine { 8 };
104     std::size_t n { 0 };
105     for (auto value : data)
106     {
107         os << value;
108         if ( ++n < valuesPerLine)
109         {
110             os << '\t';
111         }
112         else
113         {
114             os << std::endl;
115             n = 0;
116         }
117     }
118     os << std::endl;
119 }
```

Its implementation is straightforward. The variable `valuesPerLine` is declared as `constexpr` so that it can also be evaluated at compile time—a feature that is not used in the rest of the function. The use of 8 as a value allows the output of eight values in a row, which can be assumed to be 80 characters wide. This allows 9 characters per value plus a separator, e.g. a space. The variable `n` is then initialized to 0 and serves as a counter.

A range-based for loop then iterates through all elements of `data`. In the loop body, the loop variable `value` is output first. After incrementing `n` by 1, a check is made to see whether `n` is less than `valuesPerLine`. If this is the case, a tab character is inserted in `os`, otherwise a line break is inserted in `os`, the output buffer is emptied and `n` is set to

0. When the range-based for loop is completed, a `std::endl` is sent to `os` again. This can create a superfluous empty line in a few cases, but in most cases a data line with less than eight elements is terminated. Since `n` is still available, its current value could be checked to decide whether sending `std::endl` to `os` is redundant or not. This should serve as an exercise for the reader.

Listing 189 shows the function template `outputParameters<>()` and two auxiliary function templates preceding it.

Listing 189: `outputParameters<>()` (excerpt from `cia/DescriptiveStatistics.cpp`)

```
121 | template <class T1,class T2>
122 | std::ostream& output(std::ostream& os,const std::pair<T1,T2>& p)
123 | {
124 |     os << p.first << '\t' << p.second;
125 |     return os;
126 | };
127 |
128 | template <class T1,class T2>
129 | std::ostream& output(std::ostream& os,const std::pair<T1,std::set<T2>>& p)
130 | {
131 |     os << p.first;
132 |     for (auto e : p.second)
133 |     {
134 |         os << '\t' << e;
135 |     }
136 |     return os;
137 | };
138 |
139 | template <typename ...Pair>
140 | void outputParameters(std::ostream& os,const std::tuple<Pair...>& params)
141 | {
142 |     std::apply([&os](const Pair& ...param)
143 |               { ((output(os,param) << std::endl), ...); },params);
144 | }
145 |
```

First, `outputParameters<>()` is explained. Its parameters are an `ostream` reference called `os` and a tuple that is instantiated for a pack called `Pair`. This tuple exemplar is passed as a reference to `const` named `params`. Please note that the pack template parameter is explicitly specified with the name `Pair` before the function declaration. The name `Pair` was chosen because the components of the tuple in this case are actually `std::pair<>` instantiations.

In a single statement, the function template instantiates `std::apply<>()` and calls it. Its purpose is similar to that of `std::for_each<>()` or `std::ranges::for_each<>()`. While the latter applies a function to the specified range, the former applies a function to each component of a tuple. Just like `std::tuple<>`, `std::apply<>()` is defined in the `<tuple>` header file. The function template `std::apply<>()` takes two parameters. The first is a lambda expression that is applied to the first component of the tuple pack, here called

param, the second is the rest of the pack, here called params. The capture clause of the lambda specifies os as a reference. This is necessary because the lambda performs the actual output. The only parameter of the lambda is param. It is a pack of type Pair which is passed as a reference to const.

It is important not to confuse params, which is a parameter passed to outputParameters<>(), and param, which is passed as a parameter to the lambda. The parameter params of outputParameters<>() represents a complete tuple, while param of the lambda represents the currently processed pack. Initially, both parameters are the same, but each time the lambda is called, the first pack component of param is removed.

The body of the lambda is a binary left fold expression. It uses the comma operator, which first evaluates and discards the argument on the left-hand side. Then the argument on the right-hand side is evaluated, which is returned as the result of applying the comma operator. The comma operator can be used in a chain that is evaluated left-associated, but this is not relevant here.

To the left of the comma operator is an expression in parentheses that is evaluated and calls the helper function template output<>() to output the current component referred to by param to os. Since output<>() returns os as the result, std::endl is inserted next to os. This ends the expression to the left of the comma operator. To the right of this is the ellipsis, which evaluates to the rest of the pack without the parameter that was sent to output immediately before. The enclosing pair of parentheses is always required for a folding expression. The semicolon terminates the only statement of the lambda whose body is terminated by the closing curly bracket.

Now there is a comma again. However, this is not part of the lambda expression. It separates the second argument of std::apply<>(), namely the tuple named params.

There is one more thing that needs to be explained: the output<>() helper function templates.

The problem is that the second component of the various pairs is not always a scalar, but can also be a std::set<>. Therefore, both situations must be considered appropriately. The first output<>() function template can be considered as a general case. It specifies two template type parameters. They are used to instantiate the std::pair<> template, which specifies the second parameter of output<>(). No further constraints are specified. So values of any type can be used to instantiate the std::pair<> parameter, but compilation will fail if the stream insertion operators for these types are not available.

The second overload of output<>() also uses type template parameters, but when specifying the parameter std::pair<>, the type of the second component is specified as std::set<T2>. This forces the type of the second component to be an instantiation of std::set<>. As can be seen, the implementation uses a range-based for loop to output all available elements of the set to os.

When the `apply<>()` function template is instantiated, each instantiation selects the appropriate `output<>()` function template, depending on the type of the second pair component. Although this works, it is not elegant. Having the two overloads of `output<>()` in the same namespace can be irritating, as they should not be used by a client programmer. A possible solution could be to define them in a separate namespace. Another aspect is that the second overloading is only valid for pairs where the second component is of an instantiated set type. Perhaps a concept could be found or defined that would allow other suitable container templates to be used. A developer of the `serios` library had to deal with these problems (and a few more).

Admittedly, this function template and its helper function templates are very complex despite their small number of lines of code. Designing, implementing and testing such functions can take hours, even for experienced programmers.

There is one more function that deserves an explanation, namely `main()`. It is reproduced in Listing 190 including a preceding line.

Listing 190: `main()` (excerpt from `cia/DescriptiveStatistics.cpp`)

```
146 | using namespace std;
147 |
148 | int main()
149 | {
150 |     auto femaleHeights
151 |         { generateRandomData<int,vector>(normal_distribution { 165.0,6.7 },
152 |                                           mt19937_64 { },10)
153 |     };
154 |     cout << "Heights of females in cm:" << endl;
155 |     outputData(cout,femaleHeights);
156 |     outputParameters(cout,calculateParameters(femaleHeights));
157 |     cout << endl;
158 | }
```

Before `main()` is started, all identifiers are imported from the namespace `std`. In `main()`, the `auto` variable `femaleHeights` is declared and initialized with the result of the call to `generateRandomData<>()`. Here, `int` is specified as the type of the values and `vector` as the container template. Since the first two parameters, namely the distribution and the pseudo-random number generator, are passed as rvalue references, temporary values can be created on the spot. They use the values as specified in the Section *(Pseudo-)Random Numbers*. The next statement outputs a header line before calling `outputData<>()` with `cout` and `femaleHeights` as parameters. Then `outputParameters<>()` is called with `cout` as the first parameter and the result of the call to `calculateParameters(femaleHeights)` as the second parameter. The last statement sends `endl` to `cout` again.

For safety reasons, the output of the *DescriptiveStatistics.cpp* program should be compared with the corresponding calculations of another program, e.g. a spreadsheet program. Figure 47 shows a screenshot of a table created with *Numbers*. *LibreOffice Calc* or *MS Excel* could also have been used as equivalent alternatives.

Figure 47: Spreadsheet replicating calculations of *DescriptiveStatistics.cpp*

Data	160	170	166	165	164	163	150	166	168	153
n	10									
mean value	162,5									
standard deviation	6,103									
median	164,5									
mode value	166									
minimum value	150									
maximum value	170									

As can be seen, all statistical parameters highlighted in green are correct. This is sufficient as a first, very soft test of correctness.

Two details need to be mentioned, one of which is very important. Since there are different formulas for calculating the standard deviation in spreadsheet programs, care should be taken to choose the formula that calculates the standard deviation for the population. This means dividing the variance by n . Otherwise, the variance is divided by $(n - 1)$, which is called the *population variance* and serves as an estimate of the (unknown) population variance. The second point is that the spreadsheet programs can only deal with one modal value. Fortunately, there is only one modal value in the small random data set.

All the necessary prerequisites and knowledge are now in place to delve into the details of benchmarking.

4.6.3. Benchmarking

In programming, the term *benchmarking* refers to the assessment of the performance of hardware, software, or a combination of both. In software, a function, a data structure, a specific part of a program, or an entire program can be subjected to a benchmark. Possible purposes include comparing implementation alternatives or determining estimates for a specific implementation under different circumstances. Planning valid and representative benchmarks is a challenging task. First, the purpose of the benchmark must be clearly defined. Then, the execution of the benchmark must be carefully planned. To distinguish benchmarking parts of a program rather than the entire program, this form of benchmarking is referred to as *micro-benchmarking*.

A frequently used related term is *profiling*. Viewed from a distance, benchmarking and profiling are almost identical. Upon closer inspection, however, profiling focuses on identifying the parts of a program that are called most frequently and the parts where a program spends most of its time. ***Therefore, profiling focuses more on creating a valid basis for targeted optimizations, while benchmarking provides more general information about the consequences of using a particular part of the software under different circumstances.***

In the following, the templates presented in `cia/DescriptiveStatistics.cpp` and a few others serve as a playground for benchmarks. For this purpose, they are moved to a header file called `cia/statistics.hpp`.

Catch2 v3 also offers micro-benchmarking functions that have been adopted from the open source framework *Nonius*. This is useful because it means that no other tools need to be installed or understood apart from *Catch2* v3.

However, the documentation currently available for *Catch2* v3 is rudimentary, to say the least. Figuring out how to do something with *Catch2* v3 or how to interpret the results it produces can be quite time-consuming and is not necessarily correct. This must be stated clearly.

4.6.4. Simple Benchmarking with Catch2

When using *Check2* v3, a benchmark is part of a test case. Listing 191 shows the test case named `RandomData` and a benchmark named `100`.

Listing 191: cia/BM_RandomData100.cpp

```

1 // BM_RandomData100.cpp by Ulrich Eisenecker, July 21, 2025
2
3 #include "statistics.hpp"
4
5 #include <catch2/catch_test_macros.hpp>
6 #include <catch2/benchmark/catch_benchmark.hpp>
7
8 using namespace std;
9 using namespace statistics;
10
11 TEST_CASE("RandomData")
12 {
13     BENCHMARK("100")
14     {
15         return generateRandomData<int,vector>(normal_distribution { 165.0,6.7 },
16                                             mt19937_64 { },10);
17     };
18 }

```

The header file `statistics.hpp` is only included in the accompanying material (in the *sources/cia* directory). Since it contains additional parts and some adaptations that slightly improve the code quality, it is recommended to take a closer look at it.

Every *Catch2* v3 application must include at least `<catch2/catch_test_macros.hpp>` as a basic header file. For the implementation of benchmarks, `<catch2/benchmark/catch_benchmark.hpp>` must also be included. As already mentioned, an enclosing test case must be defined. This can contain any number of benchmarks. Here, the benchmark with the name `100` is the only one. When implementing a benchmark, two details must be taken into account. First, when calling a function, its result should be returned. This prevents the compiler from potentially optimizing away the function call. This optimization would, of course, lead to maximum performance. Second, the closing bracket of the benchmark must be terminated with a semicolon, as the corresponding macro is extended to a lambda.

When compiling a *Catch2* v3 application, the linker must be instructed to link the object code of the *Catch2* library and that of the *Catch2* `main()` function. This is done by specifying the options `-lCatch2Main -lCatch2`. This must be done exactly in this order and after the filename of the source file. For example, the command `g++ -std=c++23 BM_RandomData100.cpp -lCatch2Main -lCatch2` can be used to compile the program shown in Listing 191. This naturally requires that *Catch2* v3 has been installed correctly and that the path to its libraries has been added to the library search path.

Compiling (without specifying an optimization level) and running the program on a *MacBook Max Pro M3* with 128 GiB RAM produces the output shown in Figure 48.

Figure 48: Output of executing `cia/BM_RandomData100.cpp`

```
Randomness seeded to: 3257435833

~~~~~

a.out is a Catch2 v3.8.1 host application.
Run with -? for options

-----
RandomData
-----
BM_RandomData100.cpp:11
.....

benchmark name                samples      iterations    est run time
                                mean          low mean      high mean
                                std dev      low std dev   high std dev
-----
100                            100          8             1.6512 ms
                                2.05645 us   2.04858 us    2.08061 us
                                64.1829 ns   25.0389 ns    142.147 ns

=====
test cases: 1 | 1 passed
assertions: - none -
```

To understand the output, further information about how a benchmark is executed is required.

Since the computers on which a benchmark is executed can vary greatly in terms of their resources and performance, the first step is to examine the actual environment, in particular the clock frequency resolution and the cost of accessing it.

But there is also the size of the RAM and, in particular, the size and speed of the caches. For this reason, in a second step, the *user code*, i.e., the code within the benchmark, is executed several times to obtain an estimate of the required number of runs in each sample. A *run* is a single execution of the user code, a *sample* comprises all runs of the same user code. A welcome side effect is the warming up of the cache, i.e., code and data for the actual measurement are loaded into the cache.

The third step is to perform the number of runs previously estimated for each sample. A sample may in fact contain several runs, especially if the clock resolution is too low to measure a single run accurately.

An important consequence of this is that ***benchmarks must be repeatable***. Otherwise, their results are invalid. *Repeatable* means that the execution of a benchmark must not change the state of the program in which the benchmarks are executed.

Back to the output in Figure 48. The legend explaining the benchmark result begins with the benchmark name. This name appears four lines below and is *100*. The two data items to the right of the benchmark name are *samples* and *iterations*. The number of samples is given as 100, and the number of iterations as 8. Unfortunately, the available documentation is not very precise regarding the meaning of these terms. For the time being, this can be interpreted to mean that *each sample is executed as many times as the number of iterations indicates*. It is possible that the authoritative source of information has yet to be found or will be available in the future.

The estimated *run time* is reported as 1.6512 ms. Obviously, *ms* stands for *milliseconds*. A millisecond is one thousandth of a second, or 1,000 microseconds. Other units of time listed below are *us* and *ns*. The unit *us* can be assumed to be *microseconds*. A microsecond is one millionth of a second or 1,000 nanoseconds. The unit *ns* should stand for *nanoseconds*. A nanosecond is one billionth of a second.

This helps to relate the specified numbers to each other. The multiplication expression $2.05645 \text{ us} \times 100 \times 8$ results in 1651.6 us , which is quite close to the reported estimated run time of 1.6512 ms.

Below the *mean*, the *std dev* (short for *standard deviation*) is given, which is 64.1829 ns, corresponding to 0.0641829 us. This results in a small interval around the mean value, in which approximately 68% of all values can be found.

If the above assumption that a sample, here 100 runs, is executed as often as *iterations* specifies (here 8) is correct, this results in 8 values for the mean and standard deviation. Therefore, it is possible to specify the lowest and highest values for both statistical

parameters, which correspond to the remaining numbers. As can be seen, the mean is relatively stable, but there is some fluctuation in the standard deviation, which is given in *ns*. This may be due to tasks being executed in parallel.

However, the number of primary interest is the one that represents the mean value. As long as the standard deviation is not too large, the mean value can be considered a reliable statistical parameter.

But what does this benchmark tell us?

Not much, actually. It only becomes interesting when comparing the result described above with the results of the same benchmark achieved on other computers or for different sizes of data to be generated, different container templates, or different element types.

Next, it is analyzed whether it makes a difference whether 1 times 100,000 random values or 10 times 10,000 random values or 100 times 1,000 random values or 1,000 times 100 random values are generated. As already mentioned, a test case can contain any number of benchmarks. But writing four very similar benchmarks would be tedious. There is a better way, which will be explained in the next section.

4.6.5. Catch2 Generators

Catch2 v3 provides *generators* that manage lists of values (and more) and generate test cases or benchmarks, each of which is parameterized with a value from the list. Listing 192 shows how to create and use a generator.

Listing 192: cia/BM_RandomData1_1000.cpp

```
1 // BM_RandomData1_1000.cpp by Ulrich Eisenecker, July 21, 2025
2
3 #include "statistics.hpp"
4 #include <string>
5
6 #include <catch2/catch_test_macros.hpp>
7 #include <catch2/benchmark/catch_benchmark.hpp>
8 #include <catch2/generators/catch_generators.hpp>
9
10 using namespace std;
11 using namespace statistics;
12
13 TEST_CASE("RandomData")
14 {
15     auto iterations { GENERATE(1,10,100,1000) };
16     string name { to_string(iterations) + 'x' +
17                 to_string(100000 / iterations)
18                 };
19     BENCHMARK(name.c_str())
20     {
21         for (size_t i { 0 }; i < iterations; ++i)
```

```

22     {
23         auto result { generateRandomData<int,vector>
24                       (normal_distribution { 165.0,6.7 },
25                        mt19937_64 { },100000 / iterations)
26                       };
27     }
28 };
29 }

```

The header file `<catch2/generators/catch_generators.hpp>` must be included so that all functions related to generators are available in the program. In the test case, the auto variable `iterations` is declared, which is initialized with a call to the `GENERATE` macro, specifying the four parameters 1, 10, 100, and 1,000. Next, a string named `name` is declared and initialized with an expression that identifies the upcoming benchmark. On the first run, `iterations` has the value 1. This is converted to a string, to which the character `x` is appended, representing the multiplication operator, and finally the result of dividing 100,000 by `iterations`, converted to a string, is appended.

The following benchmark is created with `name.c_str()` as name. This is important because passing a string results in a compilation error, as the compiler complains about a missing constructor.

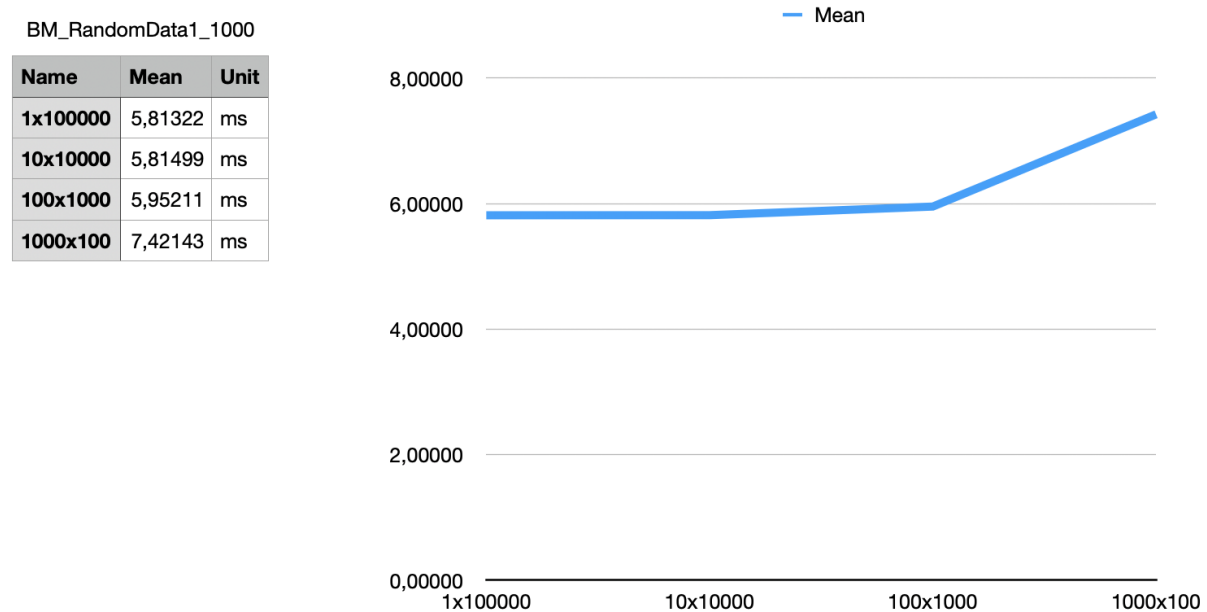
The benchmark user code now contains a for loop with the loop variable `i`, which is executed as long as `i` is less than `iterations`. It calls `generateRandomData<>()`, passing the expression `100,000 / iterations` as the third parameter, and initializes the auto variable `result` with the returned tuple exemplar. Hopefully, this should also be sufficient to prevent the call to `generateRandomData<>()` from being optimized away.

Compared to the benchmark in Listing 191, the current benchmark causes a certain amount of additional overhead. First of all, there is the loop. The more often it is executed, the more often its condition must be evaluated and the loop variable incremented. Although this is not directly visible, the variable `result` is not only created in the loop body, but also destroyed at the end of the loop body as often as the loop is executed. It can be assumed that this causes overhead that is proportional to the number of loop executions. However, depending on the selected optimization level, a loop variable can be cached directly in a processor register, which makes its processing extremely fast.

The program now takes significantly longer to run. Figure 49 shows the average values for the benchmarks on a *MacBook Max Pro* M3 with 128 GiB RAM.

The mean values of the first three benchmarks are relatively similar. The last one shows a considerable increase. This may be due to the possible reasons mentioned above, i.e., checking the loop condition and increasing the loop variables, as well as deleting the result variables.

With such a table, it is easy to create a chart with just a few mouse clicks. Since it can be tedious to manually extract the mean values from a long list of benchmark results, it is advisable to use a tool for this purpose. One such tool is described in (JohT 2022).

Figure 49: Mean values of benchmarks from BM_RandomData1_1000.cpp

Since *Catch2* v3 can generate various output formats such as *XML* and *JSON*, *Vega-Lite*, a textual domain-specific language for creating graphics, can be used to visualize the benchmark results. However, since this would require the installation of several additional software components and learning how to use them, it was ultimately decided to do this manually, as it only needs to be done a few times.

It is strongly recommended to pay attention to the units of the reported values. If the units differ from each other, this must be adjusted manually. Otherwise, a very false impression is created, as there is always a factor of 1,000 between milliseconds and seconds, microseconds and milliseconds, and nanoseconds and milliseconds. This should not be ignored.

4.6.6. Benchmarking of Sequence Containers

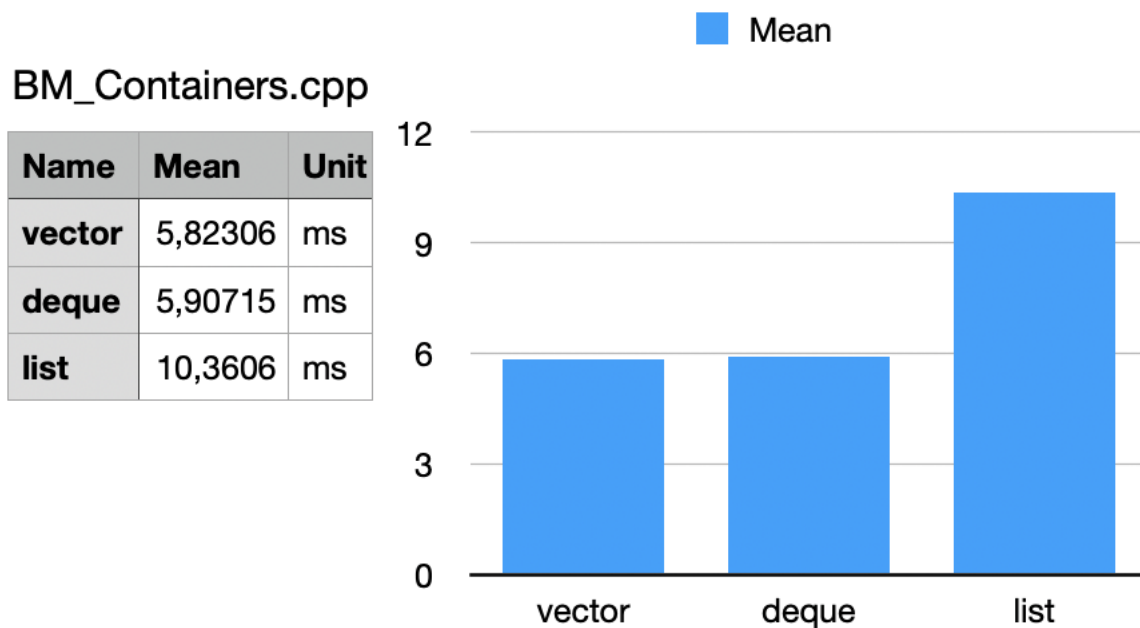
With `int` as the element type and `vector<>` as the generic container, the function templates in *cia/statistics.hpp* appear to be relatively insensitive to the amount of data being processed. The following section examines the impact of choosing a specific sequential container on performance.

It does not seem possible to use the `GENERATE` macro from *Catch2* v3 to automatically create tests for different types or templates. For this reason, the benchmarks for `vector<>`, `deque<>`, and `list<>` were created manually by *copy, paste and adapt*. This is not a prob-

lem, as the code for a benchmark is simple and short. Since the implementation of *BM_Containers.cpp* is very similar to that of *BM_RandomData100.cpp*, it is not reproduced here. It can be found in the *sources/cia* directory of the accompanying material.

The number of pseudo-random numbers in each benchmark is 100,000, and the element type is always specified as `int`. Figure 50 shows the mean values of the benchmarks and a diagram that was also created on a *MacBook Pro Max M3* with 128 GiB RAM. It should be noted that the chart uses vertical bars rather than connected lines. This is appropriate because `vector<>`, `deque<>`, and `list<>` cannot be interpolated.

Figure 50: Mean values of benchmarks from *BM_Containers.cpp*



As can be seen, `vector<>` and `deque<>` perform almost identically. The use of `list<>` results in a significant increase in execution time.

It should be noted that despite this finding, it is not appropriate to simply generalize these results to data sets of different sizes. To be sure, each combination of container and size must be subjected to a benchmark test. Nor should it be assumed that these results will be identical on different computers—not in absolute terms, of course, but in terms of the proportions of the data. Here, too, if reliable findings are desired, the benchmark tests must be performed on the target computer.

4.6.7. Comprehensive Benchmarking

All data generated by `generateRandomData<>()` so far is stored in sequence containers. One could also consider storing the data in a map. The value would then be the key (note the choice of words) and the frequency would be the value, e.g., `somePair.first` would be a generated pseudo-random number and `somePair.second` would be the frequency of the pseudo-random number. As Listing 193 shows, this supports a compact implementation for storing a new value.

Listing 193: `generateRndDtaAsMap<>()` (excerpt from `cia/extended_statistics.hpp`)

```
13     template <typename T = double,  
14               class DISTRIBUTION,  
15               class GENERATOR  
16             >  
17     [[nodiscard]] auto generateRndDtaAsMap(DISTRIBUTION&& distribution,  
18                                           GENERATOR&& generator,  
19                                           std::size_t n)  
20     {  
21         std::map<T, std::size_t> result;  
22         while (n)  
23         {  
24             result[distribution(generator)] += 1;  
25             --n;  
26         };  
27         return result;  
28     }
```

Since the container type is defined as `std::map<>`, it is not specified as a template parameter. The type template parameter for the values is now more conventionally referred to as `T`. The template parameters `DISTRIBUTION` and `GENERATOR` have the same names and meanings as in Listing 185.

The function has a different name than its counterpart `generateRandomData<>()` in Listing 185 because it cannot be overloaded for `std::map<>`. For purely practical reasons, its return type is specified as `auto`.

The variable `result` is of type `std::map<T, std::size_t>`. If `T` is not explicitly specified when instantiating the function template, `double` is used as the default value. While the element type is `T`, the frequency type is `std::size_t`, the largest unsigned integer type. Individual values will probably never reach the upper limit of this type. When implementing a library for statistics, this should also be a parameterizable type.

There is another weakness in the design: the function template does not specify any constraints for its parameters. Therefore, choosing non-arithmetic types for `T` can lead to cryptic error messages whose cause is difficult to find.

This is followed by a while loop that is executed as long as `n` is greater than 0. In the loop body, the distribution function is called with the random number generator as an argument, and its return value is used as the index for the index operator of `result`. This index differs from the usual indexes in that it can be of any type. It serves only as a key. Calling the index operator creates a `std::pair<>` with the result of calling `distribution(generator)` as the first component if the key resulting from the call to `distribution(generator)` does not exist. *Since a pair can only be created completely, a default constructor that performs zero-initialization must be available for the type of the second component.* Now it is guaranteed that a pair with this value as the key exists. The combined plus assignment adds 1 to the associated value. If the pair with the key has been newly created, the second component is then 1, otherwise it is the previous value of the second component plus 1.

However, this is not the only action performed when creating a pair with a new key. Since a map is always ordered, the resulting pair is inserted into and sorted by the internal data structure managed by the map. This makes each insertion a relatively expensive operation. Choosing an unordered map is not a valid option. Although it avoids the cost of sorting during insertion, in order to calculate the median, the pairs managed by the unordered map must ultimately be sorted by their key components. However, this is definitely not possible because an unordered map cannot be sorted. It accesses its elements via hash values, and these hash values are not valid keys for sorting. Using a `std::flat_map<>` (since C++23) is also not an option, as the pairs it manages must either be sorted after insertion or accessing them can become extremely costly, as this would require a linear search.

The final statement in the loop body decrements the variable `n`. Afterwards, `result` is returned.

The function template `modes<>()` is shown in Listing 194. It basically uses the same algorithm to determine the mode values. In fact, this is simplified because the frequencies of a value are already available in the second component of a key-value pair. As a reminder, a single value serves as the key here, and the value associated with the key is the frequency of the value. Therefore, a range-based for loop iterates through all pairs in a map and deletes and updates or updates only the `modeValues` accordingly. The frequency of the mode is stored in the variable `modeFrequency`.

Just like with `generateRndDtaAsMap<>()`, there is no template parameter for specifying the generic container to be used, as it is hard-wired as `std::map<>`.

Listing 194: `modes<>()` (excerpt from `cia/extended_statistics.hpp`)

```
30 | template <typename T = double>
31 | auto modes(std::map<T,std::size_t>& data)
32 | {
33 |     std::size_t frequency { },
34 |         modeFrequency { };
35 |     std::set<T> modeValues;
36 |     for (auto& p : data)
37 |     {
38 |         if (p.second > modeFrequency)
39 |         {
40 |             modeFrequency = p.second;
41 |             if (!modeValues.contains(p.first))
42 |             {
43 |                 modeValues.clear();
44 |                 modeValues.insert(p.first);
45 |             }
46 |         }
47 |         else if (p.second == modeFrequency)
48 |         {
49 |             modeValues.insert(p.first);
50 |         }
51 |     }
52 |     return std::make_pair(modeFrequency,modeValues);
53 | }
```

Calculating the median of a map is a daunting task. For this reason, it has been moved to its own function template (Listing 195). Please note that in addition to data, the number of all values is also passed as a second parameter named `n`. Otherwise, it would first be necessary to iterate over all pairs in order to sum their second components, i.e., the frequencies.

Listing 195: `median<>()` (excerpt from `cia/extended_statistics.hpp`)

```
55 | template <typename PRECISION,typename T = double>
56 | PRECISION median(std::map<T,std::size_t>& data,std::size_t n)
57 | {
58 |     std::pair<T,std::size_t> belowHalf { },
59 |         aboveOrEqualHalf { };
60 |     auto nHalf { n / PRECISION { 2.0 } };
61 |     for (auto& p : data)
62 |     {
63 |         if (belowHalf.second + p.second <= nHalf)
64 |         {
65 |             belowHalf.first = p.first;
66 |             belowHalf.second += p.second;
67 |         };
68 |         aboveOrEqualHalf.first = p.first;
69 |         aboveOrEqualHalf.second += p.second;
70 |         if (aboveOrEqualHalf.second > nHalf)
```

```

71         {
72             break; // Exit range-based for loop.
73         }
74     }
75     PRECISION median { };
76     if (nHalf + 1 == aboveOrEqualHalf.second)
77     {
78         median = (belowHalf.first + aboveOrEqualHalf.first) / PRECISION { 2.0 };
79     }
80     else
81     {
82         median = aboveOrEqualHalf.first;
83     }
84     return median;
85 }

```

It is perhaps noteworthy that the variables `belowHalf` and `aboveOrEqualHalf` are not declared as auto variables and initialized with `*begin(data)`, for example. This would lead to compilation errors if `belowHalf.first` or `aboveOrEqualHalf.first` were assigned a new value. Compared to `vector<>`, this is counterintuitive, since in this case it is always possible to assign a new value to an element. However, a `std::vector<>` is an unordered sequence and thus differs significantly from a `std::map<>`, which is an ordered sequence. Assigning a new value to the first component of its elements would change the key. Now, one could argue that this should create a new key. But what happens to the old key and the value associated with it? Thinking about this thoroughly, one encounters a series of semantic puzzles. To avoid this, *the first component of the pairs that serve as elements of a map is always const*. For this reason, `belowHalf` and `aboveOrEqualHalf` are declared as variables of type `std::pair<T, std::size_t>`. This makes it possible to assign a new value to the first component of a pair, which is essential for the functioning of the following algorithm.

Once the two pairs have been identified with their values and corresponding frequencies below half of the values and equal to or above half of the values, a distinction must be made between two cases, namely the first case in which `n` is even and the second case in which `n` is odd – according to the median formula in Table 18.

While it is relatively easy to determine the median of a series of numbers, this becomes difficult with a map. It is hard to describe the algorithm for determining the median value of a map in words. A useful alternative to improve understanding may be to prepare some examples of data with odd and even `n` on a sheet of paper (or in a spreadsheet) and perform all steps manually, recording the intermediate results separately and explicitly. A written medium such as this book is not suitable for applying this method.

The function template `calculateParameters<>()` (Listing 196) underwent significant changes compared to its counterpart in *statistics.hpp*.

Listing 196: calculateParameters<>() (excerpt from cia/extended_statistics.hpp)

```

87 template <typename PRECISION = double,typename T = double>
88 [[nodiscard]] auto calculateParameters(std::map<T,std::size_t>& data)
89 {
90     std::size_t n { };
91     PRECISION sum { },
92         sumOfSquares { };
93     std::ranges::for_each(data,[&n,&sum,&sumOfSquares](auto p)
94     {
95         n += p.second;
96         sum += p.first * p.second;
97         sumOfSquares += p.first * p.first * p.second;
98     });
99     PRECISION mean { PRECISION { sum } / n };
100    PRECISION stdDev { std::sqrt(sumOfSquares / static_cast<PRECISION>(n) -
101        mean * mean) };
102    auto medianValue { median<PRECISION>(data,n) };
103    auto [modeFrequency,modeValues] = modes(data);
104    using namespace std::string_literals;
105    return std::make_tuple(std::make_pair("n"s,n),
106        std::make_pair("mean value"s,mean),
107        std::make_pair("standard deviation"s,stdDev),
108        std::make_pair("median"s,medianValue),
109        std::make_pair("mode frequency"s,modeFrequency),
110        std::make_pair("mode value(s)"s,modeValues),
111        std::make_pair("minimal
112        value"s,data.begin()->first),
113        std::make_pair("maximal
114        value"s,data.rbegin()->first));
115 }

```

Its first parameter is specified as `std::map<>`, so the function template overloads the corresponding function template from *statistics.hpp*.

The first half of the implementation is new. Of course, the number of elements can be queried by calling `size()`, either as a free function template or as a member function. However, the returned value is not valid for statistical purposes, as it does not provide any information about the total frequency of the values. For this reason, `n` must first be calculated. Since the map contains pairs with value and frequency as components, the second variant of the formulas listed in Table 18 must be applied. The basic operations for their application are the sum and the sum of squares.

To calculate `n`, `sum`, and `sumOfSquares`, the `for_each<>()` algorithm from the ranges library is used. It takes `data` as its first parameter and a lambda as its second parameter. The capture clause of the lambda makes `n`, `sum`, and `sumOfSquares` available as references that can be accessed within the lambda. The only parameter of the lambda is `p`, which is specified as `auto`. The lambda increases `n` by the frequency of the current `p`,

then updates `sum` by adding the product of the value and its frequency, and finally updates `sumOfSquares` by adding the squared value times its frequency. Alternatively, all of this could have been done in a range-based for loop.

Next, the mean value is calculated, followed by the standard deviation, by applying the second variant for calculating the variance in Table 18 and taking the square root of the result.

Since the median is now determined by the function template `median<>()`, the corresponding value is called `medianValue` to avoid conflicts. The variables `modeFrequency` and `modeValues` are initialized using structural binding with the result of the call to `mode<>()`. The rest of the implementation is almost identical. Only `medianValue` is used instead of `median`.

Value-frequency pairs must be output differently. Therefore, the function template `outputData<>()` is also overloaded for `std::map<>`. It outputs each pair on a new line, with the value and frequency separated by a tab character.

Inspired by the implementation of `calculateParameters<>()` for `std::map<>`, an alternative implementation for sequence containers is provided, called `calcParamsFaster<>()` (Listing 197). It must be renamed because there is no easy way to overload `calculateParameters<>()` accordingly.

Listing 197: `calcParamsFaster<>()` (excerpt from `cia/extended_statistics.hpp`)

```

125     template <typename PRECISION = double>
126     [[nodiscard]] auto calcParamsFaster(std::ranges::forward_range auto&& data)
127     {
128         auto n { size(data) };
129         PRECISION sum { },
130             sumOfSquares { };
131         std::ranges::for_each(data, [&sum, &sumOfSquares](auto v)
132             {
133                 sum += v;
134                 sumOfSquares += v * v;
135             });
136         PRECISION mean { sum / n };
137         PRECISION stdDev { std::sqrt(sumOfSquares / n - mean * mean) };
138         std::ranges::sort(data);
139         auto median { n % 2 ? data[n / 2] :
140             (data[n / 2 - 1] + data[n / 2]) /
141             PRECISION { 2.0 } };
142         auto [modeFrequency, modeValues] = modes(data);
143         using namespace std::string_literals;
144         return std::make_tuple(std::make_pair("n"s, n),
145             std::make_pair("mean value"s, mean),
146             std::make_pair("standard deviation"s, stdDev),
147             std::make_pair("median"s, median),
148             std::make_pair("mode frequency"s, modeFrequency),
149             std::make_pair("mode value(s)"s, modeValues),
150             std::make_pair("minimal value"s, *(data.begin())));

```

```
151 |         std::make_pair("maximal value"s,*(data.rbegin())));  
152 |     }  
153 | }
```

Since `data` now reports the number of values correctly, `n` does not need to be calculated separately. Therefore, the lambda passed to `for_each<>()` calculates only the sum and the sum of squares in a single pass. Based on `n`, `sum`, and `sumOfSquares`, the mean and standard deviation can be calculated.

To determine the median and mode, the data elements must be sorted. Once this has been done, median and `modeFrequency` and `modeValues` can be calculated in the usual way. The rest of the implementation remains unchanged.

A final remark on the design of `"extended_statistics.hpp"` is useful. One option would have been to create a copy of `"statistics.hpp"` and add all the content that is now contained in `"extended_statistics.hpp"`. However, this was not desirable. Therefore, `"extended_statistics.hpp"` includes `"statistics.hpp"` and reopens the `statistics` namespace. This way, a program can simply include `"extended_statistics.hpp"` instead of `"statistics.hpp"` and use all the add-ons. At a later stage, after all function templates have been improved with constraint specifications and exceptions have been added, after thorough testing and documentation, and after a certain degree of stability has been achieved, the functions then available could be included in a new header file and thus transition into a library.

This section focuses on a comprehensive evaluation of the various functions for calculating parameters for different containers. The efficiency with which pseudo-random numbers are generated is not of interest, as they are not the subject of the statistics. Nevertheless, pseudo-random numbers must be generated, as they are stored in the containers of interest. Fortunately, *Catch2* v3 provides a macro for advanced benchmarks that solves this problem: the `BENCHMARK_ADVANCED` macro.

The resulting benchmarking program is relatively compact and is shown in Listing 198.

Listing 198: `cia/BM_Comprehensive.cpp`

```
1 | // BM_Comprehensive.cpp by Ulrich Eisenecker, July 24, 2025  
2 |  
3 | #include <deque>  
4 | #include <map>  
5 | #include <vector>  
6 |  
7 | #include <catch2/catch_test_macros.hpp>  
8 | #include <catch2/benchmark/catch_benchmark.hpp>  
9 |  
10 | #include "extended_statistics.hpp"  
11 |  
12 | using namespace std;
```

```

13 using namespace statistics;
14
15 TEST_CASE("Comprehensive")
16 {
17     BENCHMARK_ADVANCED("vector")(Catch::Benchmark::Chronometer meter)
18     {
19         auto sample
20             { generateRandomData<int,vector>(normal_distribution { 165.0,6.7 },
21                                             mt19937_64 { },100000)
22         };
23         meter.measure([&sample]() { return calculateParameters(sample); });
24     };
25     BENCHMARK_ADVANCED("deque")(Catch::Benchmark::Chronometer meter)
26     {
27         auto sample
28             { generateRandomData<int,deque>(normal_distribution { 165.0,6.7 },
29                                             mt19937_64 { },100000)
30         };
31         meter.measure([&sample]() { return calculateParameters(sample); });
32     };
33     BENCHMARK_ADVANCED("map")(Catch::Benchmark::Chronometer meter)
34     {
35         auto sample
36             { generateRndDtaAsMap<int>(normal_distribution { 165.0,6.7 },
37                                     mt19937_64 { },100000)
38         };
39         meter.measure([&sample]() { return calculateParameters(sample); });
40     };
41     BENCHMARK_ADVANCED("faster")(Catch::Benchmark::Chronometer meter)
42     {
43         auto sample
44             { generateRandomData<int,vector>(normal_distribution { 165.0,6.7 },
45                                             mt19937_64 { },100000)
46         };
47         meter.measure([&sample]() { return calcParamsFaster(sample); });
48     };
49 }

```

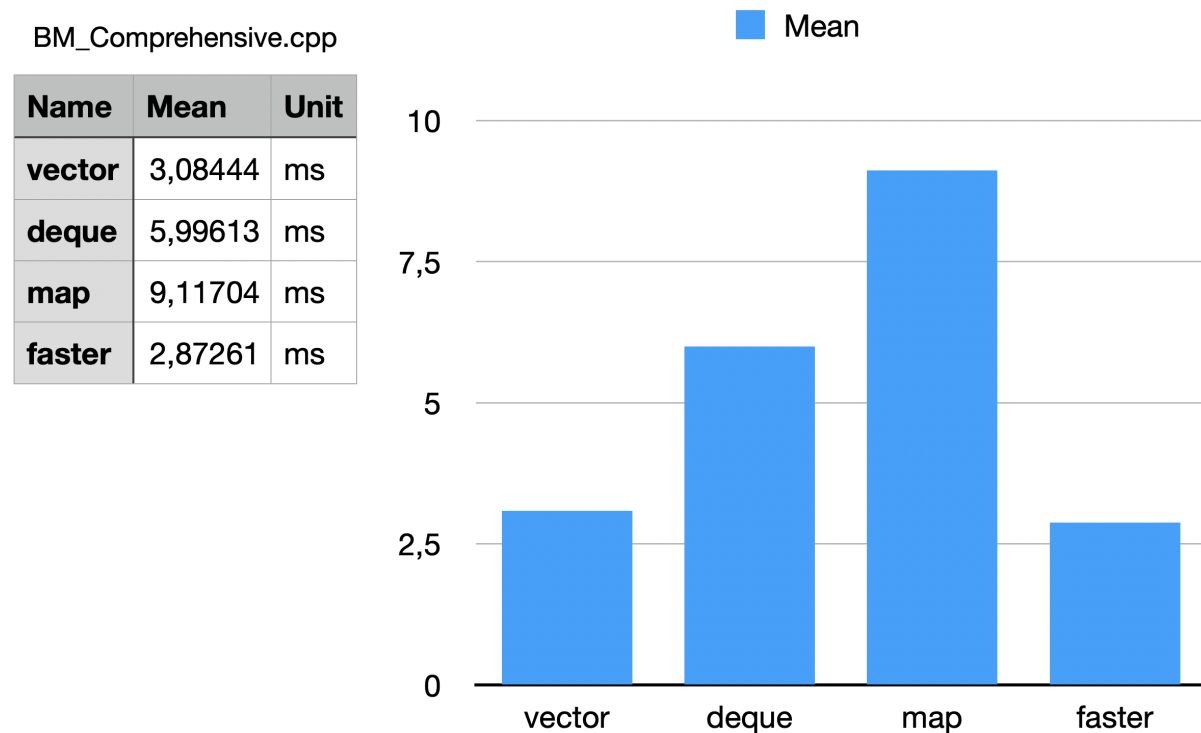
The `BENCHMARK_ADVANCED` macro takes the name of the benchmark as a parameter. This is followed by an expression in parentheses that specifies a variable of type `Catch::Benchmark::Chronometer`. Although it is tempting to speculate about what is going on under the hood, this desire is futile. It is better to stick to the (sparse) documentation that specifies what to do here.

Next, the `auto` variable `sample` is declared and initialized with the result of calling `generateRandomData<>()`. Then `meter.measure()` is called with a lambda that specifies `sample` as a reference in its capture clause and returns the result of the call to `calculateParameters(sample)`. This is necessary to prevent the function call from being optimized away by the compiler.

The following extended benchmarks all follow this pattern with minor adjustments where necessary, e.g., by calling `generateRndDtaAsMap<>()` or `calcParamsFaster<>()`. Therefore, they are not discussed in detail here.

Executing *BM_Comprehensive.cpp* on a *MacBook Pro Max* M3 with 128 GiB RAM (compiled with *clang++* without specifying optimization levels) yields the results shown in Figure 51.

Figure 51: Mean values of benchmarks from *BM_Comprehensive.cpp*



In terms of generating pseudo-random data, `vector<>` and `deque<>` were almost neck and neck, as shown by the benchmark results in Figure 50. When calculating statistical parameters, `vector<>` is significantly more powerful than `deque<>`, while `map<>` takes about three times as long as `vector<>` to perform the same task. Interestingly, however, calling `calcParamsFaster<>()` for a stack yields a small performance gain. If performance is critical in practice, this would be an indication to investigate further performance improvements.

The following two sections deal with the specifics of unit testing the results of statistical parameter calculations and a significant weakness in using a `map` to store values for statistical calculations.

4.6.8. Unit Testing for Calculating Statistical Parameters

The focus of the previous sections was on benchmarking function templates related to statistics. However, unit tests of these function templates are also highly desirable. It is relatively easy to generate a few data sets of different sizes, insert them into a spreadsheet, and apply the statistical functions to obtain these parameters. The results of one's own functions can then be visually compared with those of the spreadsheet. However, this is not true unit testing, as it is not automated. Nevertheless, it can be a good starting point for creating an initial version of a function template for calculating statistical parameters.

When testing new generic containers and variations of function templates for calculating statistical parameters, this first function template or these first function templates can serve as a reference.

For example, one can compare the calculation results of a `vector<>`, a `map<>`, or using the *faster* implementation (see Table 19). Their visual comparison demonstrates the consistency of the calculations. Assuming that the first result is correct, one can conclude that the other results are also correct.

Table 19: Output of different implementations of calculating statistical parameters

vector		map		vector/faster	
n	100000	n	100000	n	100000
mean value	164.514	mean value	164.514	mean value	164.514
standard deviation	6.7191	standard deviation	6.7191	standard deviation	6.7191
median	165	median	165	median	165
mode frequency	5929	mode frequency	5929	mode frequency	5929
mode value(s)	165	mode value(s)	165	mode value(s)	165
minimal value	134	minimal value	134	minimal value	134
maximal value	193	maximal value	193	maximal value	193

Well, that's another visual comparison that should be automated. And through automation, all possible pairs of results—somewhat paranoid and possibly a waste of time—could be tested for equality, as shown in Listing 199.

Listing 199: Parts of test case Calculations (excerpt from cia/UnitTesting.cpp)

```
16 TEST_CASE("Calculations")
17 {
18     auto dv { generateRandomData<int,vector>
19               (normal_distribution { 165.0,6.7 },
20                mt19937_64 { },100000)
21     };
22     auto rv { calculateParameters(dv) };
23
24     auto dd { generateRandomData<int,deque>
25               (normal_distribution { 165.0,6.7 },
26                mt19937_64 { },100000)
27     };
28     auto rd { calculateParameters(dd) };
29
30     auto md { generateRndDtaAsMap<int>
31               (normal_distribution { 165.0,6.7 },
32                mt19937_64 { },100000)
33     };
34     auto rm { calculateParameters(md) };
35
36     auto fd { generateRandomData<int,vector>
37               (normal_distribution { 165.0,6.7 },
38                mt19937_64 { },100000)
39     };
40     auto rf { calcParamsFaster(fd) };
52     REQUIRE(rv == rd);
53     //REQUIRE(rv == rm); // Will probably fail.
54     //REQUIRE(rv == rf); // Will probably fail.
55     //REQUIRE(rd == rm); // Will probably fail.
56     //REQUIRE(rd == rf); // Will probably fail.
57     REQUIRE(rm == rf);
78 }
```

All REQUIRE statements with a trailing comment indicating that they may fail are likely to fail. The preceding code, which outputs all parameters of each calculation, is not shown. Part of this output is included in Table 19.

How can that be?

In (U. W. Eisenecker 2022), *Section 3.1.1.8 Floating Point Types* it is demonstrated that calculations with floating point numbers are fundamentally imprecise and that important mathematical properties that apply to real numbers in mathematics, such as associativity and commutativity, no longer apply. A glance at the various implementations immediately reveals a differences. The first version of `calculateParams<>()` uses the first variant of the formula to calculate the variance and standard deviation, while the other version uses the second variant with the sum of the values and the sum of the squares. This difference is sufficient to produce differences in the resulting values that go beyond the digits displayed when sending the numbers to a text stream.

Of course, one could use one of the matchers of *Catch2* v3 as described in (U. W. Eisenacker 2022), *Section 4.6.7 Using Catch2 in a ModernWay*. However, it would be tedious to do this repeatedly for three floating-point values (mean, standard deviation, median) and again for each combination of result pairs. It is more convenient to create the text output and compare them as shown in Listing 200. This part of the "Calculations" test case follows the part shown in Listing 199.

Listing 200: Part of test case Calculations (excerpt from cia/UnitTesting.cpp)

```

16 TEST_CASE("Calculations")
17 {
59     ostringstream os;
60     outputParameters(os,rv);
61     auto rvs { os.str() };
62     os.str("");
63     outputParameters(os,rd);
64     auto rds { os.str() };
65     os.str("");
66     outputParameters(os,rm);
67     auto rms { os.str() };
68     os.str("");
69     outputParameters(os,rf);
70     auto rfs { os.str() };
71
72     REQUIRE(rvs == rds);
73     REQUIRE(rvs == rms);
74     REQUIRE(rvs == rfs);
75     REQUIRE(rds == rms);
76     REQUIRE(rds == rfs);
77     REQUIRE(rms == rfs);
78 }

```

To prepare the `REQUIRES` assertions, each result is output to the string stream named `os`. A string is then requested from `os` and used to copy and initialize a corresponding string. After repeated execution, the `REQUIRE` assertions check whether the combinations of the result strings are identical. This time, they do not fail.

Of course, all previous `REQUIRE` statements that inevitably fail must be converted from comments to compilable statements in order to achieve this point.

4.6.9. A Systematic Measurement Error and Maps

An interesting question is whether the actual values of the parameters change when different data sizes are selected. Compiling and executing the program shown in Listing 201 and redirecting its output to a CSV file yields the results shown in Table 20.

Listing 201: cia/WrongMean.cpp

```
1 // WrongMean.cpp by Ulrich Eisenecker, July 24, 2025
2
3 #include <iostream>
4 #include <tuple>
5 #include <vector>
6
7 #include "extended_statistics.hpp"
8
9 using namespace std;
10 using namespace statistics;
11
12 int main()
13 {
14     cout << "Size";
15     for (size_t i { 10 }; i <= 100000; i *= 10)
16     {
17         cout << '\t' << i;
18     }
19     cout << endl;
20     cout << "Mean";
21     for (size_t i { 10 }; i <= 100000; i *= 10)
22     {
23         cout << '\t' << get<1>(calcParamsFaster(
24                                 generateRandomData<int,vector>
25                                 (normal_distribution { 165.0,6.7 },
26                                 mt19937_64 { },100000)))>.second;
27     }
28     cout << endl;
29 }
```

The program outputs the mean values resulting from the evaluation of five data sets of different sample sizes. It is relatively simple, with the exception of the statement in the second loop body. Here, the function template `get<>()` is used to access the mean value of the tuple returned by `calcParamsFaster<>()`. Since this is the second element of the tuple, the literal constant 1 is specified, as the elements of a tuple are numbered starting with 0. The only parameter that this overload of `get<>()` accepts is the tuple. This is returned by calling the function to calculate the statistical parameters, which in turn is called with the result of the call to generate random data.

Please note that the last parameter of this function is now the loop variable `i`. Since the result of calling `get<>()` is a pair, its second component, which contains the value of the mean, must be accessed explicitly with `.second`. Although these four lines may seem a little intimidating, hopefully their explanation was relatively easy to understand.

Table 20: Mean values of data sets of different size

Size	10	100	1000	10000	100000
Mean	164.514	164.514	164.514	164.514	164.514

Obviously, the mean values are identical with regard to the three decimal places displayed. Of course, they may differ in the subsequent places after the decimal point, which is likely to be the case. However, one observation may be surprising if one does not take into account the note from the Section *(Pseudo-)Random Numbers*: they are all below the specified mean value of 165.0!

This is because the generated values are of type `int`, which simply truncates the decimal places of the pseudo-random numbers, which are of type `double` by default. Instead, it may be noticeable that the mean values are not closer to 164.5. However, the reason for the deviation of the mean values from the specified mean value is a systematic measurement error. In reality, every measurement is flawed. However, it is assumed that the errors deviate randomly from the *true* value. This is not the case here, as most of the measurements are systematically too small.

Implementing commercial rounding directly in `generateRandomData<>()` does not make sense, as it would then always be applied. One option would be to specify `int` as the type for the results of generating pseudo-random numbers and distribution numbers, which could be sufficient if they apply internal rounding.

Currently, the problem is solved by selecting `double` as the type of values returned by `generateRandomData<>()`, although this is not realistic, as no height measurement with such accuracy would be reliable in practice.

This is done in the *CorrectMean.cpp* program, which is not reproduced here but is included in the accompanying material (folder *sources/cia*). This program now reports a mean value of 165.014 for all data sets of different sizes—which again shows this suspicious increase of 0.014.

Next, a map is used to store these double values (Listing 202).

Listing 202: *cia/MapWithDoubles.cpp*

```

1 // MapWithDoubles.cpp by Ulrich Eisenecker, July 24, 2025
2
3 #include <iostream>
4 #include <map>
5
6 #include "extended_statistics.hpp"
7
8 using namespace std;
9 using namespace statistics;
10
```

```
11 | int main()
12 | {
13 |     auto data { generateRndDtaAsMap<double>
14 |         (normal_distribution { 165.0, 6.7 }, mt19937_64 { }, 100000) };
15 |     auto results { calculateParameters(data) };
16 |     outputParameters(cout, results);
17 |     cout << endl;
18 | }
```

Compiling and executing this program generates an enormous amount of output on the author's platform. In fact, the program reported 100,000 mode values, each with a frequency of 1. Due to the limited accuracy of the text output, some values appear identical but are not. Depending on the platform selected, it is possible that genuine duplicate values may be generated if the data set is large enough. However, the probability of this seems to be low.

Why is this important?

This is very important when using a map. Using a map makes sense if there is a sufficient number of values with frequencies greater than 1. If all values have a frequency of 1, the map actually becomes a sequence container with the special property of being very inefficient. If, on the other hand, there is a lot of data with a frequency greater than 1, which may even be distributed across the values, this would be a strong indication that the data is not distributed according to a bell curve, which is a prerequisite for a valid interpretation of the mean and standard deviation.

All of this clearly argues against using an associative container such as a map to store values for statistical calculations. Since a map is always sorted, inserting, retrieving, and updating the value associated with a key are complex operations.

In general, careful consideration must be given to which container and algorithm to select for specific data. Sometimes the type of data is relevant, sometimes the size of the data. In case of doubt, benchmarks should be performed with different containers and algorithms for representative data on the target platform. Last but not least, it should be emphasized that pseudo-random numbers and even random numbers are not always representative. Therefore, it often makes sense to collect sufficiently large amounts of real data.

This case study has once again demonstrated how closely intertwined the problem space and solution space are, and how carefully the problem space must be analyzed and the solution space designed in order to create a useful and valid mapping from the problem space to the solution space.

4.7. The Final Draw

The chapter began with a program example that uses a hypothetical draw from an urn without repetitions, where the order is important, i.e., so-called *permutations* (Section *The Classic C-Array*). The C++ standard library provides the function templates `std::next_permutation<>()` and `std::prev_permutation<>()` (both defined in the header file `<algorithm>`), which do exactly what their names suggest: If possible, they change the order of the unique elements in a container to their next or previous permutation in lexicographical order. If this is not possible, they move the elements to either the first or last permutation.

Listing 203 illustrates how the fictitious draw 9, 0, 7, 2, 8, 1, 6, 3, 4, 5 can be permuted until the final permutation has been generated. Since all remaining permutations are sent to `cout` as a string, the program generates extensive output. It also demonstrates the equivalence of an `urn::Urn0` exemplar (see Sections *Urn With Order And Without Repetitions* and *Implementation of the Urn0 Class*), for which all possible draws are generated by applying `std::next_permutation<>()` to a `std::vector` containing the elements 0, 1, 2.

Listing 203: cia/FinalDraw.cpp

```

1 // FinalDraw.cpp by Ulrich Eisenecker, August 8, 2025
2
3 #include <algorithm> // Because of next_permutation<>();
4 #include <array>
5 #include <iostream>
6 #include <iterator> // Because of begin() and end().
7 #include <ranges>   // Because of views::transform<>().
8 #include <string>
9 #include <vector>
10
11 #include "to_string.hpp"
12 #include "urn.hpp"
13
14 using namespace std;
15 using namespace urn;
16
17 int main()
18 {
19     array<int,10> values { 9, 0, 7, 2, 8, 1, 6, 3, 4, 5 };
20     do
21     {
22         cout << cia::to_string(values | std::views::transform([](auto e)
23                                     { return std::to_string(e); })) << endl;
24     } while (next_permutation(begin(values),end(values)));
25
26     Urn0 urn1 { 3,3 };
27     vector urn2 { 0, 1, 2 };
28     do
29     {
30         string draw1 { },

```



```
31         draw2 { };
32     for (uint i { }; i < urn1.k(); ++i)
33     {
34         draw1 += to_string(urn1.at(i));
35         draw2 += to_string(urn2.at(i));
36     }
37     if (draw1 != draw2)
38     {
39         cerr << "Error comparing urn results!" << endl;
40         return 1;
41     }
42     next_permutation(begin(urn2), end(urn2));
43 } while (urn1.next());
44 cout << "The urns give identical results." << endl;
45 }
```

A classic C array is not a valid range because the end iterator is undefined when passed to `cia::to_string<>()`. For this reason, a `std::array<>()` is used as a generic container to store the fictitious draw. Next, a do loop is started that runs as long as `next_permutation(begin(urn2), end(urn2))` returns true. As can be seen, a start and end iterator must be passed as arguments to `next_permutation<>()`, as this algorithm is not yet available in the ranges library. The loop body uses the function template `cia::to_string<>()` to create a string representation of the values. Since the elements of values are not of type `std::string`, a view is applied that converts each element to a string by calling `std::to_string()` on it. This technique has already been used several times, so no further details are provided.

The program includes `"../objects/urn.hpp"`. The specified path corresponds to the directory structure in the accompanying material. The first two dots switch to the parent directory (which is *sources*), from where the program switches to the *objects* directory. From there, the header file *urn.hpp* is included.

Next, two urns are declared. The first, `urn1`, is of type `urn::Urn0`. It specifies 3 as the number of elements, which are automatically numbered from 0 to 2, and 3 as the size of the draw, i.e., the maximum number of balls that can be drawn. The second urn, named `urn2`, initializes a vector with the elements 0, 1, and 2. In the following do loop, two string variables are declared. In the nested for loop, they are filled with string representations of the elements in `urn1` and `urn2`. Since `urn::Urn0` does not implement an interface corresponding to *STL* containers, a more classic way is used to access the elements of `urn1` and `urn2`.

If the strings `draw1` and `draw2` were different, a corresponding error message would be sent to `cerr` and the function `main()` would be terminated with 1 as the result. Return values from `main()` that differ from 0 indicate an incorrect termination of the program.

The last statement in the body of the do loop is the call to `next_permutation<>()` for `urn2`, which advances the state of `urn2` to the next permutation. The condition of the do loop evaluates `urn.next()`. As long as this call evaluates to true, the do loop continues.

If everything goes well, the program will display a corresponding success message.

As you can see, the *C++* standard library offers a wealth of code that can be used or reused, even when it comes to specific problems such as permutations. It is strongly recommended to always first search the *C++* standard library thoroughly for code that contributes to solving a specific problem, and then search other publicly available libraries. This initially incurs high costs. However, over time, human capital builds up, meaning that one remembers potentially suitable code snippets from previous searches. It always pays off when existing, frequently used, and well-tested code can be (re)used. This is fundamentally more sustainable than developing one's own or proprietary solutions that have to be tested, documented, and maintained.

5. The Functional Programming Paradigm

The functional programming paradigm has its roots in mathematics, more specifically in lambda calculus (*Lambda Calculus* 2026), which was developed by Alonzo Church (*Alonzo Church* 2026), Alan Turing's doctoral advisor at Princeton University. One of its features is to avoid state as much as possible, whether in the entire program or in parts of it. ***Functional programming in its purest form knows neither classical variables (which is a consequence of avoiding state) nor control structures such as branching constructs and loops, which are replaced by pattern matching and recursion.*** Another feature is its a-temporality. This means that expressions are not evaluated in a specific order. Instead, an expression is evaluated when its value is needed, and not before. This enables extensive optimizations by the compiler. Originating in academia, the characteristics of functional programming found their way into modern programming languages. Since functional programming makes it possible to write shorter programs that are easier to test and extend, the functional approach is becoming increasingly attractive in programming practice.

Interestingly, templates are a Turing-complete sublanguage of C++, which was discovered shortly after their introduction into the language. This sparked great interest in the functional programming community and led to many concepts of functional programming being adopted by C++ and its standard library.

This chapter does not cover every aspect of functional programming in detail, nor is it an introduction to template metaprogramming in C++. As a curiosity, it starts with a brief explanation of template metaprogramming, making use of the many functions that support it in the C++ standard library.

5.1. Template Metaprogramming

Template metaprogramming (TMP for short), also known as *static metaprogramming*, was discovered by Erwin Unruh. At a meeting of the C++ Standard Committee, he distributed a program with templates that caused the compiler to output the first prime numbers as error messages. Interested readers are referred to (Czarnecki and U. Eisenecker 2000), Section *Static Metaprogramming in C++*, which contains additional information about the history of template metaprogramming in C++, including the famous listing for outputting prime numbers. It also offers a comprehensive introduction to the state of the art in this field in the late 1990s.

A lot has changed since then. Template metaprogramming has become widely used, especially in the creation of the C++ standard library. It also generated much interest in the functional programming community, which has since been involved in the further development of the C++ language. Listing 204 shows a program that calculates factorial numbers during compilation and outputs them at runtime.

Listing 204: functional/FactorialTMP.cpp (excerpt)

```
1 // FactorialTMP.cpp by Ulrich Eisenecker, August 21, 2025
2
3 #include <iostream>
4
5 template <unsigned long long int n>
6 struct factorial
7 {
8     static constexpr auto value { n * factorial<n - 1>::value };
9 };
10
11 template <>
12 struct factorial<0ull>
13 {
14     static constexpr auto value { 1ull };
15 };
16
17 template <unsigned long long int n>
18 constexpr auto factorial_v { factorial<n>::value };
19
20
21 using namespace std;
22
23 int main()
24 {
25     cout << factorial_v<5> << endl;
26     cout << endl;
27     output<10> { };
28 }
```

First, the class template `factorial<>` is defined as a `struct` with `n` as the only integral, i.e. constant parameter. Defining it as a `struct` is convenient because all its members are public. If it were defined as a `class`, the members that need to be accessed from outside would have to be declared `public`, which would actually be all members of the structure.

Although `factorial<>` is a class template, it is referred to as a *template metafunction* (TMF for short). It is not used to create exemplars, but is only executed at compile time.

Its sole member is the static variable `value`, which is declared as `constexpr`. As a result, it is fully evaluated during compilation time. It has a recursive definition, as its value is calculated by multiplying `n` by the instantiation of `factorial<>` for `n - 1` and accessing the member `value` of this nested instantiation. This is the classic definition of a recursive function.

To terminate the recursive instantiation of `factorial<>`, a full specialization for `n = 0` is required. As can be seen, `factorial<0ull>` also defines a static constant named `value`, which is initialized to `1ull`. Before instantiating a template, the compiler first checks whether a suitable template instantiation is already available that can be used instead. In this way, recursion is effectively terminated by this full specialization of `factorial<>`.

The mathematical definition of the factorial function encompasses two cases, namely $n! = n \times (n - 1)$ and $0! = 1$. Both are mapped to the TMF `factorial<>` and its full specialization.

The process of template instantiation and lookup can be considered as sort of *pattern matching*. There is no direct equivalent for ordinary functions. It is not possible to overload an ordinary function for a specific value. Therefore, a branching statement has to be used.

Browsing through the C++ standard library, particularly the header file `<type_traits>`, it becomes clear that a TMF that calculates a value stores it in a *static* data member called `value`, as shown in the example above. When it calculates a type, it stores it in a static member type called `type`.

For convenience, additional template variables are defined. Those ending in `_v` represent TMFs that calculate a value, while those ending in `_t` yield a type. Template variables have not been introduced in the Chapter *The Generic Programming Paradigm*, but it is sufficient to explain them briefly here. As can be seen, a template variable is defined using a syntax that is very similar to that of a class template. Since `factorial_v<>` is initialized with the call to `factorial<>`, `auto` is used to deduce its type. To force initialization at compile time, it is declared as `constexpr`. From now on, `factorial<>` can be used like any other TMF in the C++ standard library that also calculates a value.

The `main()` function contains the statement

```
std::cout << factorial_v<5> << std::endl;
```

Since the call to `factorial_v<5>` is evaluated at compile time, the generated object code contains only the result, namely the unsigned integer 120. This can be verified by disassembling the resulting object file, but this is not shown here.

As this program demonstrates, a mathematical function definition can be mapped directly to the implementation of a corresponding TMF. Instead of a branching construct, pattern matching of the various parts of a function definition is used. In addition, the loop construct from procedural programming is replaced by recursion, in this case by the recursive instantiation of the TMF `factorial<>`. Of course, this TMF does not contain any state. Although the various instantiations of `factorial<>` all have a member named `value`, these members are all unique because each belongs to a different instantiation of `factorial<>`. Therefore, the member `value` is not a classic variable. It can be thought of as a name that has a value connected to it. A TMF cannot accept input from a human operator when it is executed during compilation. If output is required, runtime programming concepts must be used for this purpose.

There is another interesting aspect: once the factorial of 10 has been calculated, there is no need to calculate any further factorials of numbers between 1 and 10, as the compiler simply looks them up in its internal tables. This is known as *memoization*, which is also a desirable feature of functional programming.

TMP in C++ fully meets the strict requirements of functional programming: TMFs are guaranteed to be stateless, referential transparency (this term will be explained shortly) applies to both individual TMFs and complete template metaprograms, and there are no traditional variables or control structures. It is in the nature of template metaprograms that they are executed exclusively at compile time, and there is no way to accept user input during compile time, although there are limited possibilities for generating output during compile time.

In C++, TMP is well integrated into the further implementation of functional programming. It ensures type safety and offers several opportunities for optimization by the compiler. However, this book does not provide any further details on TMP. Interested readers can take a look at the program *FactorialTMP.cpp*, which is located in the *sources/functional* directory. It contains additional TMFs for printing sequences of factorial numbers, which are commented in detail.

5.2. Basic Principles of Functional Programming

Some principles of functional programming have already been briefly discussed in (U. W. Eisenecker 2022) and in some previous sections of this book. While TMP corresponds to pure functional programming—which was purely coincidental—, C++ and its predecessor *C* were definitely not designed as functional programming languages. However, even *C* has some features that can be used to emulate principles of functional programming. This set of language features has been extended in C++ in such a way that, in particular, the C++ standard library offers a wealth of functionality that, when used,

conveys a functional programming perspective, even though it may be implemented using other means. For simplicity, the outlined principles are summarized below, along with possible procedural implementations designed to align them stylistically with functional programming.

In strictly functional programming, there is no state. For this reason, a function call can neither modify state (since there is none) nor have a stateful implementation (since there is no state). In C++, this restriction is relaxed in that while a function may not modify external state, it may have a stateful implementation, e.g., to simplify implementation or for performance reasons (see Section *Functions With Memory*). In this book, functions of this type are referred to as *pure functions*, although they do not fully conform to the functional programming paradigm.

There are functions that can return different results even when called with the same parameters, such as a parameterless function that returns pseudo-random numbers. Other examples that inevitably must alter state include functions for reading and writing files. This also applies to functional programming, which sometimes offers solutions that are difficult to understand, using highly complex terminology that merely obscures the fact that the principle of purity cannot apply everywhere. In the following, functions in this category will be referred to as *impure functions*—just as is the case in some functional programming languages. In functional programming, impure functions should only be used when there is no alternative to their use. The state changes they cause should be limited to the necessary minimum.

In relation to purity and the canonical form stands the concept of *referential transparency*. It states that an expression can be replaced by the result of its evaluation without changing the program. This also applies to the subexpressions of a complex expression. Referential transparency is a prerequisite for the automatic rewriting of a program by the compiler, e.g. to perform specific optimizations.

To avoid states, variables should only be initialized or assigned once. In C++, it is common practice to initialize a variable when it is defined. Ideally, the value used for initialization should not be changed afterwards—at least from a functional programming perspective. If this cannot be avoided for legitimate reasons, the variable should be zero-initialized when it is defined and later assigned a value only once.

Following this principle results in a *value-based design*. Consequently, user-defined data types should not have member functions that explicitly change state or support implicit state changes. In functional programming, ADTs or classes usually only have constructor functions, getter functions and functions that return new values, but not setter functions. This fits well to mathematical entities as they are regularly viewed as being immutable. It can become difficult if an object has a fixed identity, but a rich and variable lifecycle, e.g. a representation of a complex machine or a living organism.

Because an object should not be modified after its creation or—on an exceptional basis—after its first assignment, it can become difficult to test for the equalities of two objects. For example, the rational numbers $3/4$ and $6/8$ are equal. To figure this out, both objects have to be brought first into a unique representation with respect to their internal values. This unique representation is called the *canonical form* of an object. Usually, the canonical form of an object (or of an expression) cannot be further reduced.

As functions are primary building blocks, it is desirable to keep their number low with respect to comprehensibility, redundancy, and maintainability. While overloading functions is one possibility to create a family of semantically related functions—at the expense of rising the number of functions—sometimes the carefully crafted use of default values for function parameters can achieve the same result with only one function. Another possibility to extend the generality of a function are function parameters that refer to functions. Such a function is called a *higher-order function* (see Section *Higher Order Functions*). One more possibility is to provide function templates that can be instantiated with type-parameters, non-type template parameters, and even template-template parameters. And last but not least, a class can also serve as a blue-print for instantiating different exemplars that can be used like functions. Such objects are called *function objects*.

Sometimes, functions cannot be called with specific argument values for which they would give undefined results, terminate or throw an exception. The best-known example for this is the division by zero. From a mathematical point of view, a function that is not defined for the entire range of its argument types, is a *partial function* (see Section *Partial Function*). A further possibility to mitigate this problem is to extend the return type of a partial function such that it also gives a defined result for invalid argument values (see Section *std::optional<>*).

In functional programming the term *partial function* has one more meaning, which refers to a way of creating a new, more specialized function by fixing one or more arguments of the original function (see Section *Partial Function Application*). The resulting function is called with fewer arguments. Ultimately, this allows to create functions that take exactly one argument which can also be achieved by a related technique (see Section *Currying*).

The following sections present functions in their traditional form in *C* and in their more modern form in *C++* as well as function objects and anonymous functions which are specific to *C++* from a functional programming perspective.

5.3. The Classic C Function and its Modern Form

The classic C function looks like Listing 205. It specifies the return type of the result and how it is returned. Here, a new value of type `unsigned long long int` is returned, followed by the function name, in this case `factorial1`, and the parameters enclosed in parentheses, in this case `n` of type `unsigned long long int`, which is passed by value.

Listing 205: Classic C function (excerpt from `functional/Factorial.cpp`)

```

5 | unsigned long long int factorial1(unsigned long long int n)
6 | {
7 |     if (n > 0)
8 |     {
9 |         return n * factorial1(n - 1);
10 |    }
11 |    return 1;
12 | }
```

If these three parts (return type, function name and parameter list) were terminated by a semicolon, they would form a *function declaration*. This is not the case here, as the function body enclosed in curly brackets follows. Together, these four parts form a *function definition*. If it is not preceded by a corresponding declaration, it is a declaration that is also a definition.

The function body begins with an if statement. Its condition checks whether `n` is greater than 0. If this is true, it returns the result of the recursive call to the `factorial1()` function, which corresponds to the general case of the mathematical definition. The last statement of the function returns 1, which is not guarded by an if statement because it cannot be called if the condition of the preceding if statement was evaluated as true. This shows how the pattern matching of functional programming in Listing 204 is replaced by an if statement in its runtime equivalent.

5.3.1. New Syntax for Specifying the Return Type

There is an alternative syntax for specifying functions, inspired by functional programming. It is shown in Listing 206.

Listing 206: Trailing return type specification (excerpt from functional/Factorial.cpp)

```
14 | auto factorial2(unsigned long long int n) -> unsigned long long int
15 | {
16 |     if (n == 0)
17 |     {
18 |         return 1;
19 |     }
20 |     return n * factorial2(n - 1);
21 | }
```

Here, `auto` is used as a placeholder for the return type of the function, which means that *the return type of the function is not yet known at this point*. The actual return type is specified as the last part of the function header after `->`, namely `unsigned long long int`.

Apart from the fact that this syntax may be more familiar to functional programmers, it is not really relevant for ordinary functions. However, the situation is different in the context of function templates whose return type depends on the argument types specified as template parameters. With this knowledge in mind, the alternative syntax will be used below where necessary.

Please note that the condition of the `if` statement is different here, as it checks whether `n` is equal to 0. In principle, it can be assumed that it makes no difference which case comes first and testing both variants always gives the same result. However, covering the simplest case first can be one of the prerequisites for compiler optimization. This will be examined in the next section.

5.3.2. Tail Recursion

As already mentioned in (U. W. Eisenecker 2022), Section 3.3.7 *Recursion*, recursion in C++ is considered relatively expensive, since each recursive call of a function creates a new entry on the function call stack, which costs time and memory and can even lead to an overflow of the function call stack. However, if a specific pattern called *tail recursion* is followed when implementing a recursive function, a compiler can automatically convert, i.e. *rewrite* the recursive implementation into an iterative implementation that is more efficient and requires less memory. Tail recursion has two requirements:

- The recursive call must appear as the last statement in the function implementation.
- The final statement must only call the recursive function.

Looking at the `factorial2()` function in Listing 206, the first requirement is obviously met. The second requirement is not fulfilled, since the recursive call to `factorial2()` is the second operand in a multiplication with `n` as the first operand.

Therefore, the `factorial2()` function must be manually rewritten. As shown in Listing 207, it is split into two functions, namely the `factorial2()` function, which is to be called by the client, and the `factorial_impl()` function which should exclusively be called by `factorial2()`.

Listing 207: Factorial function for tail recursion (excerpt from `functional/BM_TailRecursion.cpp`)

```

15 unsigned long long int factorial_impl(unsigned long long int n,
16                                     unsigned long long int& product)
17 {
18     if (n == 0)
19     {
20         return product;
21     }
22     /* [[clang::musttail]] */ return factorial_impl(n - 1, product *= n);
23 }
24
25 unsigned long long int factorial2(unsigned long long int n)
26 {
27     unsigned long long int product { 1 };
28     return factorial_impl(n, product);
29 }
```

The comment before the second return statement of `factorial_impl()` is an attribute specific to *clang++*. If it is uncommented, it forces tail recursion optimization. Please note that the presence of this compiler-specific attribute makes the program non-portable.

To verify whether a C++ compiler actually performs the tail recursion optimization, a corresponding source program must be compiled with various optimization options and then the generated object or assembler code must be examined. Since this can be quite challenging, it is easier to use benchmarking for this purpose.

The benchmarks compare the two recursive function implementations. In addition, another benchmark is performed for the `factorial3()` function, which is implemented iteratively. Listing 208 shows only the newly introduced `factorial3()` function and the benchmarks. The complete program is included in the accompanying material (*function/BM_TailRecursion.cpp*).

Listing 208: functional/BM_TailRecursion.cpp (excerpt)

```
1 // BM_TailRecursion.cpp by Ulrich Eisenecker, September 8, 2025
2
3 #include <catch2/catch_test_macros.hpp>
4 #include <catch2/benchmark/catch_benchmark.hpp>
51 unsigned long long int factorial3(unsigned long long int n)
52 {
53     auto product { 1 };
54     while (n > 0)
55     {
56         product *= n--;
57     }
58     return product;
59 }
60
61 TEST_CASE("Factorial")
62 {
63     BENCHMARK_ADVANCED("Recursion")(Catch::Benchmark::Chronometer meter)
64     {
65         meter.measure([]() { return factorial1(20); });
66     };
67
68     BENCHMARK_ADVANCED("Tail Recursion")(Catch::Benchmark::Chronometer meter)
69     {
70         meter.measure([]() { return factorial2(20); });
71     };
72
73     BENCHMARK_ADVANCED("Iterative")(Catch::Benchmark::Chronometer meter)
74     {
75         meter.measure([]() { return factorial3(20); });
76     };
77 }
```

Since the use of *Catch2* v3 for benchmarking has been described in detail in the Section *Benchmarking* and the following sections, it is not discussed again here. However, it should be mentioned that the `BENCHMARK_ADVANCED` macro must be used, since calls to the various factorial functions for the `BENCHMARK` macro are optimized away even when the most moderate optimization level is selected.

In the following, *BM_TailRecursion.cpp* is compiled and executed eight times. For the creation and execution of the first four binary files, the attribute `[[clang::musttail]]` remains as a comment; for the second half of the binary files, it is uncommented and thus influences the compilation. The next dimension covers the degree of optimization. If the optimization flag is not present, no optimization is applied. The `-O1` option enables moderate optimizations, the `-O2` option increases the number of optimizations performed (debugging information is no longer generated), and the `-O3` option enables

the most aggressive optimizations, which can increase compilation time and also enlarge the size of the generated binary file. Figure 52 shows the average values of the results using a *MacBook Pro Max* M3 with 128 GiB RAM.

The first row of the table at the top of Figure 52 shows that the recursive implementation of `factorial1()` is slower than the iterative implementation of `factorial3()`. At a first glance, it may seem surprising that `factorial2()`, which is optimized for tail recursion, performs the worst. However, no optimization is applied because this can hinder debugging. This should not be confused with generating debugging information using the `-g` option for compilation. Rather, the structure of the program is preserved when no optimization takes place. Therefore, it is fundamentally possible to step through the function calls to `factorial2()` and `factorial_impl()` without debugging symbols being available. Preserving this possibility is the cause of the poor performance due to additional function calls.

As soon as an optimization flag is specified, the execution time drops to about a quarter of a nanosecond, with the exception of `-O1` and tail recursion. Thus, the tail-recursive implementation of `factorial2()` is obviously still treated differently in this case.

After removing the comment for the `[[clang::musttail]]` attribute, the tail-recursive version suddenly performs better than the other implementations, even without specifying an optimization flag. Obviously, specifying `[[clang::musttail]]` makes a difference. However, it should be noted that the execution times are still very long.

When specifying `-O1`, the tail-recursive implementation has the longest execution time, although it is slightly faster than without optimization. However, the other two versions have already dropped to around a quarter of a nanosecond.

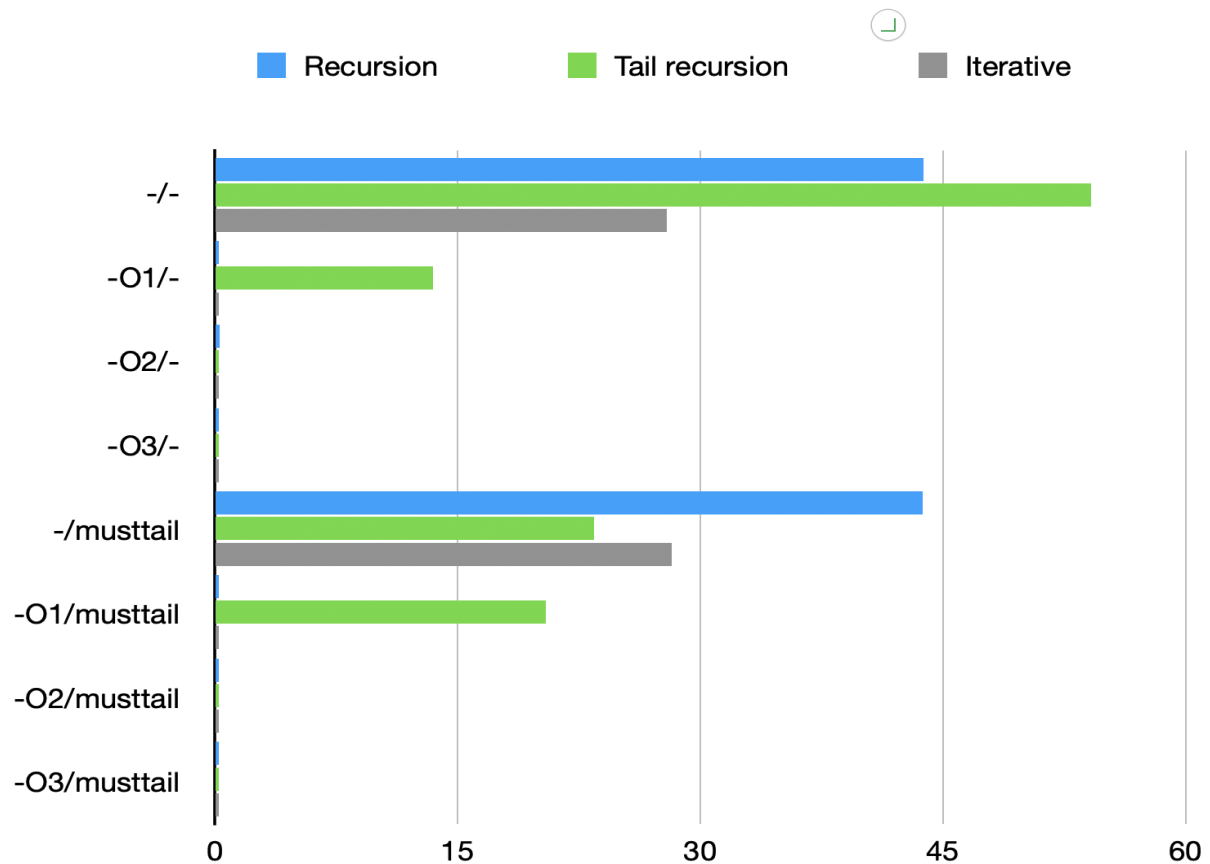
When specifying `-O2` or `-O3`, the execution times of all versions become very fast and very similar.

What conclusions can be drawn from these measurements?

- There is only one case in which the non-portable compiler attribute `[[clang::musttail]]` has a significant effect, namely when no optimization flag is specified.
- The attribute `[[clang::musttail]]` apparently prevents some optimizations when `-O1` is specified, as performance is worse than without optimization.
- Manually providing a tail-recursive implementation either negatively impacts performance (if no optimization or `-O1` is specified) or has no effect at all (in all other cases).
- The recursive and iterative implementations perform equally well for each optimization level. Therefore, manually converting a recursive implementation to an iterative implementation is not worthwhile—at least in this case, since the compiler can apparently do this automatically during optimization.

Figure 52: Benchmarking results for various optimization levels**BM_TailRecursion.cpp**

Optimization	Recursion	Tail recursion	Iterative	Unit
-/-	43,7817	54,1491	27,9238	ns
-O1/-	0,246689	13,4684	0,246517	ns
-O2/-	0,272332	0,246673	0,247861	ns
-O3/-	0,246546	0,246546	0,251511	ns
-/musttail	43,7464	23,4132	28,243	ns
-O1/musttail	0,246794	20,4645	0,251261	ns
-O2/musttail	0,246767	0,246704	0,246694	ns
-O3/musttail	0,246594	0,246707	0,246736	ns



- It can be assumed that the compiler performs additional optimizations beyond tail recursion.

5.3.3. Further Optimizations

It is indeed possible to check which optimizations *clang++* applies. By specifying the option *-foptimization-record-file=optimizations.txt*, the compiler records the optimizations applied in the file *optimizations.txt* (a different name for the file can also be used). Interested readers can check this using the files *FR.cpp* and *FI.cpp*, both of which are included in the *sources/functional* directory of the accompanying material. The *g++* compiler also allows the specification of optimization flags, but this is not discussed here. Only a few hints will be given as to which additional optimizations could be applied when calculating the factorial.

One of these is known as *loop unrolling*. If the compiler can assume that the factorial is only being calculated for values greater than or equal to five, it can initialize the local variable `product` of `factorial3()` with 120—the value resulting from unrolling the first 5 iterations of the loop to $1 \times 2 \times 3 \times 4 \times 5$ —and then execute the remaining loop iterations starting with this value. In addition, it can even provide a version with a partially unrolled loop and a version with an unrolled loop or with some results stored in an internal table.

Another option is *parallelization*. When calculating the factorial of 10, it can be divided into the calculation of the product of 0 to 5 and 6 to 10. Both operations can be performed in parallel on two different cores. The two resulting products are then multiplied to obtain the final result.

There are other optimizations that a compiler can apply. It is important to note that using them can change the structure of the source program. In addition, some optimizations require the prior application of another optimization. Without tail recursion (in the case of a recursive function), neither loop unrolling nor parallelization become possible.

What can be learned from this for (functional) programming in C++ (without necessarily being limited to C++)?

- When writing source code, the focus should be on the correctness and the comprehensibility of the code. ***Paying attention to optimization does not pay off.***
- Obviously, recursion—which is the de-facto standard in functional programming—can be used without (too much) concern in C++. Manually transforming recursive functions into tail-recursive functions is not necessary—at least when using an aggressively optimizing compiler.
- If a program's performance is too slow, benchmarking can help to identify the areas that are critical to performance. Only these should be considered for performance-oriented reprogramming. And—very importantly—the previous correct imple-

mentation should never be deleted. Rather, it should be kept as a secure backup. It does not make sense to apply profiling to binary code whose structure has been optimized in a way that does not preserve the structure.

5.3.4. Higher Order Functions

If a function accepts another function as a parameter or returns another function as a parameter, it is referred to as a *higher-order function*. Higher-order functions have been around since the early days of C. They are represented as function pointers and are used, for example, in the `qsort()` function of the C library. Another common use is callback functions, such as in the *FLTK* framework (see Section *Menus and Callbacks*).

For the sake of completeness, Listing 209 shows an example that accumulates the values of the calls to a function passed as a parameter for a range defined by a start value, a stop value, and an increment value.

Listing 209: functional/HOFasParameter.cpp

```
1 // HOFasParameter.cpp by Ulrich Eisenecker, August 27, 2025
2
3 #include <iostream>
4
5 auto sum(auto func, auto start, decltype(start) stop, decltype(start) increment)
6 {
7     decltype (func(start)) result { }; // Zero initialization
8     while (start < stop)
9     {
10         result += func(start);
11         start += increment;
12     }
13     return result;
14 }
15
16 long long unsigned int identical(long long unsigned int n)
17 {
18     return n;
19 };
20
21 long double reciprocal(int n)
22 {
23     return 1.0L / n;
24 }
25
26 double squared(double n)
27 {
28     return n * n;
29 }
30
31 using namespace std;
32
33 int main()
```

```

34 {
35     cout << "Sum of numbers from 0 .. 10 step 1: "
36         << sum(identical,0,10,1) << endl;
37     cout << "Sum of reciprocals from 1 .. 10 step 1: "
38         << sum(reciprocal,1,10,1) << endl;
39     cout << "Sum of squares from -3 .. 0 step 1: "
40         << sum(squared,-3,0,1) << endl;
41 }

```

Declaring a function pointer type can be cumbersome. For this reason, the first parameter of the `sum()` function is declared as `auto func`, which effectively makes this function a function template. Since no concept is used to specify restrictions on the type, `func` can be of any type. Next, the start value is passed, which is also specified as an `auto` parameter. The following two parameters, `stop` and `increment`, should be of the same type as `start`. If they were also specified as `auto` parameters, different types could be used for them. For this reason, the `decltype` operator is used to specify them as the same type as `start`.

The `result` variable is declared in the first line of the `sum()` function template. The question is what type this variable should have. Here, too, the `decltype` operator is used to determine this type. It is applied to `func(start)`, which is not evaluated in this process. Only the type that `func()` would return if it were applied to `start` is determined. This is also an initial restriction based on usage. If the function call operator is not available for `func` and if `func()` cannot be called with exactly one parameter, the compiler issues an error message.

Since `sum()` calculates the sum of repeated calls to `func()`, `result` is zero-initialized.

Next, a while loop is started whose condition is true as long as `start` is less than `stop`. This is neither elegant nor safe, as the relationships between `start`, `stop`, and `increment` are not taken into account. In principle, it may be necessary for the repeated addition of the `increment` to result in exactly the value `stop`, or for the loop to be executed at least once—even if `start` is equal to or greater than `stop`—or for the loop to run downwards rather than upwards.

In the body of the while loop, `sum` is incremented by calling `func()` with `start` as an argument. Then `start` is incremented by `increment`.

As soon as the while loop evaluates to false, the `sum()` function terminates returning `result`.

Next, some functions are defined that return the argument, calculate the reciprocal of the argument, or return the square of the argument.

In the `main()` function, the sums of different ranges for these functions are calculated and sent to `cout`.

Listing 210 shows how a function pointer is returned. It defines several functions that accept two double arguments, in this case the basic arithmetic functions `add()`, `subtract()`, `multiply()`, and `divide()`, each of which returns the corresponding double result. The function `classify()` accepts two operands and `result` as parameters and compares the values with exactly the previously specified order using one of the above functions. If a call matches, it returns a pointer to the corresponding function. The `classify()` function is not a function template, as the use of `auto` as its return type might suggest. Since all of its `if` statements are evaluated at runtime, they must all return a value of the same type. Furthermore, all functions in `classify()` are known, so the type of the return value cannot vary.

Listing 210: `functional/HOFasReturnValue.cpp`

```
1 // HOFasReturnValue.cpp by Ulrich Eisenecker, March 31, 2026
2
3 #include <iostream>
4 #include <map>
5 #include <utility> // Because of make_pair<>()
6
7 double add(double a,double b)
8 {
9     return a + b;
10 }
11
12 double subtract(double a,double b)
13 {
14     return a - b;
15 }
16
17 double multiply(double a,double b)
18 {
19     return a * b;
20 }
21
22 double divide(double a,double b)
23 {
24     return a / b;
25 }
26
27 double unknown(double a,double b)
28 {
29     return { };
30 }
31
32 auto classify(double a,double b,double result)
33 {
34     if (add(a,b) == result)
35     {
36         return add;
37     }
38     if (subtract(a,b) == result)
39     {
```

```

40     return subtract;
41 }
42 if (multiply(a,b) == result)
43 {
44     return multiply;
45 }
46 if (divide(a,b) == result)
47 {
48     return divide;
49 }
50 return unknown;
51 }
52
53 using namespace std;
54
55 int main()
56 {
57     map f2n { make_pair(add,"add"),
58              make_pair(subtract,"subtract"),
59              make_pair(multiply,"multiply"),
60              make_pair(divide,"divide"),
61              make_pair(unknown,"unknown")
62     };
63     cout << f2n[classify(17.0,4.0,21.0)] << endl;
64     cout << f2n[classify(1.1,2.2,0.0)] << endl;
65 }

```

Of course, it is possible that no call to the known functions matches the triple consisting of the two operands and the result. In this case, `classify()` returns `unknown`, a pointer to a function that always returns a double value initialized to zero. This zero value is not really of interest, but the function pointer is relevant.

The implementation of `classify()` is straightforward, so no further explanation should be necessary. However, the `main()` function has some interesting details.

First, a `std::map<>` named `f2n` is instantiated, which maps the function pointers to names. Since the required keys/values are specified when initializing `f2n`, template argument deduction can be used to deduce the types of the keys and values. Then, the result of accessing `f2n` with the result of calling `classify()` for three given parameters is sent to `cout` using the index operator. This is a declarative method of converting the function pointer returned by `classify()` to a corresponding name.

There are a few points that can be criticized. First, it is not possible to dynamically extend the functions considered by `classify()`. Currently, `classify()` has very limited applicability because the functions it knows are hard-coded into its implementation. Second, `classify()` returns the pointer to the first function that matches the result. In some cases, however, several functions may match. For example, the group of three one's, i.e. 1, 1, 1, matches both `multiply()` and `divide()`. This could also be taken into account. Fixing these shortcomings is left as an exercise for the reader.

5.3.5. Functions With Memory

In *C* and *C++*, a function can maintain an internal state that can be used to implement impure functions as well as memoization.

As explained in (U. W. Eisenecker 2022), Section 2.8.1 *Unit Tests*, a function is the smallest module: its header corresponds to the interface of a module and its body to the implementation of a module. Listing 16 in the Section *Global and Static Variables* shows that a function can even be stateful, similar to an exemplar of a class in object-oriented programming. This property is relevant, for example, when programming a function that generates pseudo-random numbers, since the result it returns can be different each time it is called, even if the same arguments are specified.

In addition, this property can be used to cache the results of previous function calls and return a result from the cache instead of recalculating it. Memoization is an important optimization technique that stores the return values of previous calls to the same function, as demonstrated in Section *Template Metaprogramming*. While being automatically available at compile time for TMP due to the peculiarities of template instantiation and the lookup of template instantiations, it is not available at runtime, i.e. during the execution of a program. With the help of functions with memory, memoization can be implemented for runtime. Listing 211 illustrates this using functions that calculate factorials.

Listing 211: functional/FactorialMemo.cpp

```
1 // FactorialMemo.cpp by Ulrich Eisenecker, October 8, 2025
2
3 #include <iostream>
4 #include <vector>
5 #include <array>
6
7 static_assert(sizeof(unsigned long long int) == 8);
8
9 unsigned long long int factorial3(unsigned long long int n)
10 {
11     static std::vector results { 1ull, 1ull, 2ull, 6ull };
12     for (auto i { size(results) }; i <= n; ++i)
13     {
14         results.push_back(results.back() * i);
15     }
16     return results[n];
17 }
18
19 constexpr auto factorial4(unsigned long long int n)
20 {
21     constexpr static std::array<unsigned long long int,21> results =
22         []() { std::array<unsigned long long int,21> f;
23             f[0] = 1;
24             for (unsigned long long int i { 1 }; i <= 20; ++i)
25             {
26                 f[i] = f[i - 1] * i;
```

```

27         }
28         return f;
29     } ();
30     return results[n];
31 }
32
33 using namespace std;
34
35 int main()
36 {
37     for (unsigned long long int i { 0 }; i <= 20; ++i)
38     {
39         cout << i << '\t' << factorial3(i) << '\t' << factorial4(i) << endl;
40     }
41     return 0;
42 }

```

The `factorial3()` function takes `n`, the number for which the factorial is to be calculated, as an argument. First, it declares `std::vector<> results` as a static variable. It could have been initialized as a completely empty vector, but here the first four factorial numbers are provided for initialization, allowing the compiler to deduce the type of the vector elements.

Then a classic for loop is executed. It specifies `i` as the loop variable and runs from `size(results)` until `i` is greater than `n`. The loop statement calculates the actual factorial number by multiplying the last factorial number (which is different for each execution of the for loop) by the current value of `i`. The result of the multiplication is appended as the last element to the end of the vector. Note that this loop is not executed if the factorial of `n` has already been calculated.

Therefore, it is generally sufficient to return the `results` element at position `n`, regardless of whether the loop was executed immediately before or not.

This function is executed at runtime and fills the cache, in this case `results`, during runtime. Calculating a factorial is a relatively inexpensive operation that only becomes expensive when it is called repeatedly for larger `n` without caching. In addition, factorials grow very quickly. If `unsigned long long int` has a size of 8 bytes, the largest factorial that can be represented correctly is `20!`. Consequently, only 21 factorials in the range from `0!` to `20!` need to be cached or stored. This can also be done in a classic C array being encapsulated in a `std::array<>` and initialized during runtime. The required storage space of 168 bytes is almost negligible.

The `factorial4()` function uses this option. It declares the static variable `results` of type `std::array<>`. Since a `std::array<>` must be instantiated for a fixed element size, the required type and non-type template parameters must be explicitly specified in angle brackets. Note that 21 elements are allocated, since `0!` is also cached.

In addition, the static variable `results` is declared as `constexpr`. This means that it must be initialized during compilation. Otherwise, it would be permissible to initialize it during runtime, precisely at the start time, before the body of the `factorial4()` function is executed for the first time.

The keyword `constexpr` also applies to the initialization of `results`, which is somewhat complicated syntactically. A convenient way to do this is to call a lambda, which is explained in detail in the Section *Lambda Expression*.

Since a lambda is a function object, an initializer list cannot be used. Instead, the `=` operator is used to force the compiler to call the lambda. Because a lambda must be called like any other function using the function call operator, `()` appears before the semicolon that ends the variable declaration. The lambda is defined between `=` and `;`. Since no parameters from the environment are made available to the lambda, the corresponding list is empty, i.e. `[]`. No arguments are passed to the lambda either. Therefore, the corresponding parameter list is also empty, i.e. `()`.

The first statement of the lambda declares a variable `f`, which is an array instantiated with `unsigned long long int` and 21 elements, corresponding to the type of the static variable `results` to be initialized. The variable name `f` was chosen to avoid confusion with `results` above. Please note that `f` is neither declared as `static` (which is not necessary) nor marked as `constexpr`, since the `constexpr` for the declaration of `results` is still valid for its initialization. The first element is manually set to 1. Next, a classic `for` loop is executed, which calculates the factorial numbers from 1 to 20 and assigns them to the corresponding element of `f`. The last statement returns `f`. Since this is the only return statement, the compiler can infer the return type of the lambda, so it does not need to be redundantly specified in its header using the `->` syntax. This also applies to the `factorial4()` function itself.

Whether the elements of `f` are moved to `results` or actually copied is not really relevant, since all this happens at compile time. The only thing that remains in the runtime code is the return of the element at position `n` from `results`.

Both functions use tables to determine a factorial number. In the first case, this table is created dynamically during runtime; in the second case, this table is created completely during compile time. In this respect, it is similar to the `factorial` TMF from Listing 204, perhaps with slightly improved readability, but with the added advantage that it can be evaluated both at compile time and at runtime.

It is up to the reader to program benchmarks to assess whether the effect of these manual optimizations justifies the effort required to create them.

5.4. C++-specific Functions

The previous sections demonstrate the enormous range of classic *C* functions and their capabilities in the context of functional programming. Their power is further extended by combining them with generic polymorphism, which is covered in the Section *Function Templates*. Next, functions specific to C++ are introduced. They are special in that they are not functions in the strictly traditional sense, but rather function-like. This is sometimes summarized under the colloquial term *duck typing*, which means that *if something looks like a duck and quacks like a duck, one can assume it is a duck*. This also applies to function objects and lambda expressions: they look (at least sometimes) like classic functions and can be used like classic functions (which also opens up new uses), so they can be considered functions.

5.4.1. Function Objects

If a class overloads the function call operator as a member function, an exemplar of this class can be called like a function. Listing 212 illustrates this.

Listing 212: functional/FunctionObject.cpp

```

1 // FunctionObject.cpp by Ulrich Eisenecker, October 16, 2025
2
3 #include <iostream>
4 #include <vector>
5
6 using ulli_t = unsigned long long int;
7
8 class Factorial
9 {
10 public:
11     Factorial():m_results { 1ull, 1ull, 2ull, 6ull }
12     {
13     }
14     [[nodiscard]] ulli_t operator()(ulli_t n) const
15     {
16         for (auto i { size(m_results) }; i <= n; ++i)
17         {
18             m_results.push_back(m_results.back() * i);
19         }
20         return m_results[n];
21     }
22 private:
23     mutable std::vector<ulli_t> m_results;
24 } const factorial5;
25
26 using namespace std;
27
28 int main()
29 {
30     for (ulli_t i { 10 }; i <= 22; ++i)

```



```
31 | {  
32 |     cout << i << '\t' << factorial5(i) << endl;  
33 | }  
34 | return 0;  
35 | }
```

First, `ulli_t` is defined as a type alias for `unsigned long long int`, because this integer type is used repeatedly in the program (and in the following programs as well).

The `Factorial` class corresponds to the `factorial3()` function in Listing 211, i.e., it uses the `vector<>` member variable `m_results` to cache previously calculated factorial numbers at runtime. Please note that this member variable is declared as `mutable`, which means that it can be changed even if the exemplar that owns it is `const`. In the default constructor of the class, `m_results` is initialized with the first four factorial numbers, just as in `factorial3()`. In `factorial3()`, a static `vector<>` variable is used for temporary storage. This is not the case here, i.e., each exemplar of `Factorial` has its own `m_results` variable.

Next, the function call operator is defined as a member function. The name of this operator is `operator()`. In addition, its arguments must be specified in the following pair of parentheses. Here, this is `n`, the number for which the factorial is to be calculated. Please note that the function call operator is also marked as `const`. This means that it does not change the state of the object—it is still a pure (member) function. Updating the cache is not considered a state change that is relevant to the caller of the function object and its use. Therefore, it is declared as `mutable`. The implementation of the function call operator is similar to the implementation of `factorial3()`, with the difference that `m_results` is updated and used instead of `results`.

After the definition of `Factorial` as before, `factorial5` is declared as a `const` exemplar. It is perfectly permissible to create exemplars immediately after the class definition. As shown, the exemplar can also be `const`, which is consistent in that calling a function should not change any state, so that calling it with the same arguments will return the same result. The fact that `factorial5` is defined in the surrounding namespace (here as a global variable) is fine, as it coexists with the `factorial3()` function, which is defined in the same namespace.

The `main()` function calls `factorial5()` in a for loop that runs from 10 to 22. The call is not different from calling `factorial4()`, even though it is an exemplar and not a function. For this reason, `factorial5` is referred to as a *function object*. Sometimes the term *functor* is used to refer to a function object. In the context of functional programming, this is not appropriate, as *functor* has a different meaning here.

5.4.1.1. Partial Function

If the unsigned long long int type has a size of 8 bytes, the last factorial that can be represented correctly is 20!. Therefore, the reported values for 21 and 22 when the program is executed are incorrect.

The factorial5() function—like the previous functions for calculating factorials—cannot calculate valid factorials for all possible arguments, but only for some of them, in this case the numbers 0 to 20. A function that cannot be applied to all possible arguments is called a *partial function* (as mentioned before, the term partial function has one more meaning which is discussed in Section *Partial Function Application*).

However, the case of factorial5() is special in that it is conceptually a partial function, but technically not, i.e., factorial5() can be called with invalid arguments, which is not recognized by the program.

What could be done to change this?

Using classic C functions, a function checkedFactorial() could be implemented that accepts the upper limit for which the factorial can be calculated correctly as an additional parameter, e.g., checkedFactorial(10,20). Here, 20 (the second parameter) would be the upper limit and 10 (the first parameter) would be the number for which the factorial is to be calculated. Constantly specifying this second parameter, which represents the upper limit, is tedious and prone to errors. One way to avoid this would be to define 20 as the default value for the limit parameter. If this value is omitted when the function is called, 20 is inserted by the compiler as the default value. This is fine as long as the size of unsigned long long int is 8 bytes. If this size changes, the default value is no longer correct and a different limit value must be specified explicitly each time.

This problem can be mitigated by a function checkedFactorial() that does not define a default value for its limit parameter, as well as by the additional definition of a function, e.g., factorial8b() with n as its only parameter, which calls checkedFactorial() with n and 20 in its implementation. If unsigned long long int only had a size of 4 bytes, another function, such as factorial4b(), would have to be defined that calls checkedFactorial() with n and 12 as the limit.

Both variants have complementary advantages and disadvantages. The positive aspect of the first variant is that it only requires one function, but the negative aspect is that it cannot be adapted as flexibly as the second variant. The negative aspect of the second variant is that it requires more than one function, but the positive aspect is that it can be adapted more flexibly.

The implementation of both variants is left as an exercise for the reader. Before looking at the sample solution in the accompanying material (*functional/CheckedFactorial.cpp*), the reader should attempt to program their own solution.

Both variants have a common disadvantage: they are based on the comprehensive solution, which always performs a check. This additional effort is also incurred when it is known with certainty that no factorials for numbers greater than 20 will be calculated (on systems with unsigned long long int with a size of 8 bytes).

5.4.1.2. `std::optional<>`

The following solution to this problem uses composition to obtain both a fast but unsafe factorial calculation and a factorial calculation as partial function that returns a special value when it is called with an invalid argument. For this purpose, function objects are used again and `std::optional<>` is introduced as a means of indicating erroneous situations (Listing 213).

Listing 213: functional/Composition.cpp (excerpt)

```
1 // Composition.cpp by Ulrich Eisenecker, October 17, 2025
2
3 #include <iostream>
4 #include <optional>
5 #include <vector>
6
26
27 class Checking
28 {
29     public:
30         Checking(ulli_t limit):m_limit { limit }
31         {}
32         [[nodiscard]] std::optional<ulli_t> operator()(ulli_t n) const
33         {
34             if (n > m_limit)
35             {
36                 return std::nullopt; // or alternatively: return { };
37             }
38             return m_function(n);
39         }
40     private:
41         Factorial m_function;
42         ulli_t m_limit;
43 } const factorial6 { 20 };
44
45 std::ostream& operator<<(std::ostream& os,const std::optional<ulli_t>& i)
46 {
47     if (i)
48     {
49         os << i.value();
50     }
51     else
52     {
53         os << "overflow";
54     }
55     return os;
56 }
```

```

57
58 using namespace std;
59
60 int main()
61 {
62     for ( ulli_t i { 10 }; i <= 22; ++i)
63     {
64         cout << i << '\t' << factorial5(i) << '\t'
65             << factorial6(i) << endl;
66     }
67     return 0;
68 }

```

To use `std::optional<>`, the header file `<optional>` must be included. The definition of the type alias `ulli_t` and the class `Factorial` are omitted because they remain unchanged from Listing 212.

Next, the `Checking` class is defined. It has two private data elements, namely `m_factorial`, which is an exemplar of `Factorial`, and `m_limit`, which stores the upper limit for which the factorial can be calculated correctly.

In the constructor the limit must be specified which is used to initialize `m_limit` subsequently. Then, the function call operator is overloaded. It does not return a result of type `ulli_t`. Instead it returns `std::optional<ulli_t>`.

A `std::optional<>` exemplar contains an optional value (for detailed information, see (*std::optional* - *Cppreference.Com* n.d.)). The type template parameter defines the type of which the value is created, if it exists. For this reason, the type template parameter cannot be something that does not result in a new object, e.g. a reference.

The class template `std::optional<>` defines several member functions, among them:

- `operator=()`, assigns content
- `operator bool()`, returns true if the object contains a value, otherwise false
- `has_value()`, equivalent to `operator bool()`
- `value()`, returns the contained value, if any, which must be checked beforehand
- `value_or()`, returns the contained value, if any, or the value passed as an argument; both values must be of the same type

The relational and comparison operators are overloaded for `std::optional<>` as free functions, and `std::make_optional<>()` creates an `optional<>` exemplar.

To create an empty `std::optional<>` exemplar, it can be initialized with the empty list, which conceptually corresponds to zero initialization. Alternatively, `std::nullopt` can be used, which is an `optional<>` exemplar without a value.

The function call operator of `Checking` takes `n` as an operand. Its implementation checks whether `n` is greater than `m_limit`. If this is the case, it returns `std::nullopt`. Otherwise, it returns the result of calling `m_function()` with `n` wrapped in `std::optional<>`.

Immediately after completing the definition of `Checking`, a `const` exemplar named `factorial6` is created with a value of 20 as its upper limit.

The important difference between the function call operators of `Factorial` and `Checking` is that the former returns a value of type `ulli_t`, while the latter returns a result of type `std::optional<ulli_t>`.

In the `for` loop in `main()`, `factorial5()` and `factorial6()` are called without considering the different result types. This is achieved by the beforehand overloading of the stream insertion operator for `const std::optional<ulli_t>&` as the type of the second argument. The implementation first checks whether a value is present by calling the `bool()` operator of `std::optional<>`. Alternatively, the member function `std::optional<>::has_value()` could have been used to check this. If the result is `true`, `i.value()` is inserted into the output stream. Otherwise, the C string "overflow" is sent to `os`.

The reason for the somewhat complicated implementation is that `Checking::operator()()` returns an optional value. It is debatable whether this is really necessary or not. One argument against it is that a return value of 0 could be used instead to indicate overflow. Unfortunately, this would mean that a `Checked` exemplar would technically no longer be a partial function, as it would accept any value as an argument and return a value of the same type even for invalid arguments. In this case, the caller of the function must be informed of the special meaning of the result 0 and take appropriate action.

On the other hand it is desirable to simply output "overflow" for invalid arguments. Since an integer and a C-string are different types, the implementation of the overloaded stream insertion operator cannot use the ternary operator for this, as it requires that the types of the value before and after the colon be identical. Incidentally, this counterargument also apply to a return value of 0. For this reason, the chosen implementation of overloading the stream insertion operator is probably the simplest.

After compiling and running the program, it can be seen that when calculating the factorials of 21 or 22, the result "overflow" is output.

To be honest, the solution to this problem described above is cumbersome. With a TMF for multiplication and another TMF for checked multiplication, the upper limits for different unsigned integer types could be calculated during compilation and stored in a compilation data structure. In addition, a function template with a template parameter of type `bool` could use `if constexpr` in combination with a default value to insert code for checking when the `bool` parameter is `true`. This would be a much simpler solution with the same functionality. Implementing this solution is left as an exercise for the reader.

On the other hand, the previous examples show only a small selection of the possibilities offered by function objects. These capabilities increase significantly when combined with generic polymorphism. Function objects and generic function objects are the technical basis for lambda expressions, known as lambdas for short. These are discussed in detail below.

5.4.2. Lambda Expression

A *lambda expression* (or *lambda* for short) can be roughly thought of as an anonymous function that is defined ad-hoc. It has been used several times before, namely as a substitute for a callback function that is invoked when a specific event occurs (Section *Lambda Expressions as Callback Functions*), or a function to be applied to the elements of a container, a range or a view (e.g. Sections *Use of Algorithms From the C++ Standard Library* and following), or to define a specific selection criterion.

Below are details on the syntax for defining a lambda, passing variables from the lexical scope to the lambda, and generic lambdas.

5.4.2.1. Lambda Syntax

The program in Listing 214 defines a simple lambda and calls it.

Listing 214: functional/BasicLambda.cpp

```

1 // BasicLambda.cpp by Ulrich Eisenecker, November 14, 2025
2
3 #include <iostream>
4 #include <string>
5
6 using namespace std;
7
8 int main()
9 {
10     cout << [](int i) { return i % 2 == 0 ? "even"s : "odd"s; } (42) << endl;
11 }
```

The first lines up to and including the beginning of the body of the `main()` function should be familiar to the reader. The statement that starts with `cout` defines and calls a lambda.

A lambda expression contains a declaration that is also a definition, which evaluates to a value. For this reason it is an expression.

The first syntactical part of the lambda expression is the pair of angular brackets, `[]`, which is called *capture clause* in this text. In (*Lambda Expressions (since C++11) - Cppreference.Com* n.d.), the part between the brackets is called *capture* or *captures*, depending on the number of the captured variables. Sometimes the brackets including their content are referred to as *captures group* and other terms occur as well. The capture clause

can specify variables from the lambda's lexical scope and how they are passed to the lambda, e.g. by value or by reference. Here, the capture clause is empty, which means that nothing from the lexical scope is passed to the lambda. More details about the capture clause follow below.

The syntax for specifying the parameter list is the same as for ordinary functions. Here, in the pair of parentheses only one parameter named `i` of type `int` is specified.

The important syntactic difference between a function and a lambda is that a function's parameter list is preceded by the function name, whereas a lambda is defined without a name. For this reason, the term *defined* is used here instead of *declared*, since a declaration introduces a name.

As explained in the Section *The Virtual Function Table*, the name of a function without the function call operator, `()`, is a function pointer which points to the corresponding function. Its type can be declared manually, which can be cumbersome (e.g. Listing 25), or it can be obtained by applying the `decltype` operator to the function name, i.e. the pointer to the function.

Because a lambda is anonymous, i.e. it has no name, the result is a type, more precisely a class type. The C++ compiler generates for a lambda expression a class type with an overloaded function call operator, `()`, plus a temporary exemplar of this class. The automatically generated class type has an internal name which is implementation defined and should not be used. The underlying anonymous class of a lambda expression is known as the *closure type* (see (Lambda Expressions (since C++11) - Cppreference.Com, n.d.)). The exemplar created for this capture type is called a *closure* (ibid.). **Therefore, the term *closure type* precisely identifies the underlying function object class, while *closure* unambiguously refers to the corresponding function object.**

In the program of Listing 214, the function call operator is immediately called for the closure with 42 as an argument (another use would be to pass it as a parameter to a function). Afterwards, the closure is automatically deleted. Not using it, would also result in its immediate deletion.

The code in curly brackets after the function parameter list is the function body. Here it contains a single statement which returns the result of evaluating a complex expression with the ternary operator, `? :`. To the left side of the question mark the remainder of dividing `i` integrally by 2 is calculated, and it is checked if the result is 0. If this is true, the second operand of the ternary operator between `?` and `:` is returned, which is the C++-string "even"s. Otherwise, the third operand is returned, which is to the right side of `:`, namely "odd"s. Of course, the same effect could be implemented with less code, which is left as an exercise to the reader.

It has to be noted, that the return type, here `std::string`, is automatically deduced by the compiler. As stated in Section *Variadic Templates*, each return statement must return the same type. Otherwise, the automatic deduction of the return type fails and a compilation error is reported.

If the compiler has not enough information to automatically deduce the return type, it must be made available before the function implementation. In this case, the return type has to be specified with the `->` syntax, e.g. `[](int i) -> string` (see Section *New Syntax for Specifying the Return Type*).

Since a lambda defines a class, i.e. the capture type, attributes and specifiers can be used. It is also possible, to turn a lambda into a template. Listing 215 shows an example for all of these possibilities.

Listing 215: functional/ComplexLambda.cpp

```

1 // ComplexLambda.cpp by Ulrich Eisenecker, November 28, 2025
2
3 #include <iostream>
4 #include <string>
5 #include <type_traits>
6
7 using namespace std;
8
9 int main()
10 {
11     auto sqr
12     {
13         [] <class T> [[nodiscard]] (T n) noexcept -> T requires is_arithmetic_v<T>
14         { return n * n; }
15     };
16     cout << sqr(2) << '\t' << sqr(2.1) << endl;
17     sqr(1); // The compiler issues a warning.
18 }
```

Because the same lambda is called repeatedly, the auto variable `sqr` is declared and initialized with the result of the lambda expression.

The capture clause is still empty. The part `<class T>` specifies a template lambda expression with the type template parameter `T`. Afterwards, the `[[nodiscard]]` attribute is specified which causes the compiler to issue a warning if the result of calling the lambda is not used. Then the parameter list specifies the parameter `n` of type-template parameter `T`. This is followed by a `noexcept` specification, i.e. the lambda promises to throw no exception. This is followed by an explicit specification of the return type, `-> T`. Finally, a `requires` clause specifies that `T` must be an arithmetic type. The implementation of the lambda is simple: it returns `n * n`, i.e. the square of `n`.

Afterwards, `sqr()` is called twice with different types of numeric literals and the results are sent to `cout`. Eventually, `sqr(1)` is called without using the result. When compiling the program, this line therefore causes a corresponding warning.

The lambda is called three times, which is quite unusual. To avoid writing the same lambda expression three times, the variable `sq` is copy-initialized with the capture.

The lambda has a simple implementation, namely to return the result of n multiplied by itself. But its overall definition is very complex and extends over two lines. Because of this, the definition of the lambda is difficult to read and understand, which is not in the spirit of a lambda. A lambda should be of local relevance, usually called only once, and have a short definition.

In the following, several simplifications are presented to realize these qualities for the lambda shown in Listing 215.

The first measure is to delete the `[[nodiscard]]` attribute. Of course, it is principally correct to warn if the result of a function call is not used if this is the only effect of the function call. But in the narrow context of a lambda expression it is obvious that ignoring the return value is non-sensical.

A second measure is to omit the exception specification. This is usually relevant for a function that is part of a library and the client programmer should be informed that this function promises to not throw any exception—especially as the implementation of the corresponding function might be deeply buried in the source code or not available at all. Specifying a (member) function template with `noexcept` is questionable per se as it is not known whether the types used for its instantiation throw exceptions or not. In the context of the lambda it should also be obvious if this lambda can throw an exception or not.

The next aspect is that the lambda is a template with a type-template parameter. Here, the classic syntax is chosen which is also the most explicit one. As an alternative, the keyword `auto` could be used to specify n . The `requires` clause can be simply omitted. The only consequence of this would be a longer and less comprehensible error message when trying to instantiate the lambda for an unsuitable type. To improve the error diagnosis a concept could be used in combination with `auto`. Furthermore, the explicit specification of the return type is redundant as the compiler can safely deduce it in this case. Listing 216 shows the resulting program.

Listing 216: functional/SimplifiedLambda.cpp

```
1 // SimplifiedLambda.cpp by Ulrich Eisenecker, November 28, 2025
2
3 #include <iostream>
4 #include <string>
5 #include <type_traits>
6
7 template <typename T>
8 concept arithmetic = std::is_arithmetic_v<T>;
9
10 using namespace std;
11
12 int main()
13 {
14     auto sqr { [] (arithmetic auto n) { return n * n; } };
15     cout << sqr(2) << '\t' << sqr(2.1) << endl;
```

```
16 |     sqr(1);  
17 |     // sqr("Ups"s); // does not compile.  
18 | }
```

Unfortunately, the arithmetic concept is not (yet) available in the C++ standard library. Therefore it is defined at the beginning as shown in Listing 157. Now, the lambda expression fits into a single line and is easier to understand. Removing the initial comment from the last statement in `main()` demonstrates the improved error diagnostics due the use of the concept. However, this comes at the price of an extra definition of the arithmetic concept and its use for declaring the `n` parameter.

Something more has to be pointed out. As lambdas are declared and used on the fly, it is difficult to test them. Unit testing a lambda would require declaring it separately and copy-initializing a variable with it which would contradict the purpose of a lambda. In this case it would be more appropriate to define a classic function object. Because of this, lambdas are usually not unit tested directly but only indirectly by thoroughly testing the function that declares and calls it. Since in most cases the lambda is not called in the declaring function but in another function called by it, white box testing should be applied to design and implement suitable test cases.

Another aspect is documentation. Because a lambda is used ad-hoc, there is no need for the usual documentation of a library function. Furthermore, it can also be counter-productive to intersperse the definition of a lambda with (documentation) comments, as its definition is then spread over several lines, making it more difficult to read and understand.

The general recommendation is to keep a lambda expression as simple as possible.

5.4.2.2. Capture Clause

Next, selected specifications that can be used in a capture clause are presented. A comprehensive overview of more possibilities is available at (*Lambda Expressions (since C++11)* - *Cppreference.Com* n.d.).

A *capture entity*, or in short *capture*, specifies a variable from the lexical scope, which is also called *environment* or *outside* in this context, that is made available in the lambda body. The lexical scope includes all variables which are accessible immediately before the declaration of the lambda expression.

Listing 217 introduces a few specifications.

Listing 217: functional/CaptureClause.cpp

```
1 // CaptureClause.cpp by Ulrich Eisenecker, November 19, 2025
2
3 #include <iomanip> // Because of boolalpha
4 #include <iostream>
5
6 using namespace std;
7
8 int main()
9 {
10     int a { 3 },
11         b { 4 };
12     cout << "a = " << a << " b = " << b
13          << boolalpha << endl;
14
15     cout << "Empty capture clause, smaller value: " <<
16          [] (int a,int b) { return a < b ? a : b; } (a,b) << endl;
17
18     cout << "Capturing all local variables by value, smaller value: " <<
19          [=] () { return a < b ? a : b; }() << endl;
20
21     cout << "Capturing a by value, larger value: " <<
22          [a] (int b) { return a > b ? a : b; }(5) << endl;
23
24     cout << "Capturing all local variables by reference, swapping a and b: ";
25     [&] () { int c { a }; a = b; b = c; }();
26     cout << a << ' ' << b << endl;
27
28     cout << "As before, but providing c initialized with a, swapping a and b: " ;
29     [&,c { a } ] () { a = b; b = c; }();
30     cout << a << ' ' << b << endl;
31 }
```

In `main()` the local variables `a` and `b` are declared and initialized. The two lines with the empty capture clause pass both variables by value as function arguments. The lambda returns the smaller value as a new value which is sent to `cout`.

In the following two lines, the capture clause contains a single `=` character. This has the effect that all variables that are accessible in `main()` at this very point are accessible as values in the lambda. Therefore, the lambda is now called with a function call operator without an argument. However, the variables `a` and `b` of `main()` are available as copies with the names `a` and `b` within the lambda. Again, the smaller value is returned as a new value and sent to `cout`. Apparently this reduces the size of the lambda expression by a few characters. This is a nice side-effect, but not the primary intention for making outer variables accessible in a lambda body. The primary reason will be explained soon.

It has to be added, that a specification with `=` as a capture creates a constant copy of the corresponding variables. For example, in `[=] () { return a < b ? a : b; }` it would not be legal to modify `a` in the body of the lambda, e.g. `a -= 1; /* ... */`. If this would

be desired, the lambda expression had to be specified with `mutable`, i.e. `[=] () mutable { a -= 1; return a < b ? a : b; }`. As shown before, `a`—and `b` as well—can now be modified in the function body. This modification affects only the local copy, but not the variable of the defining environment as can be demonstrated by sending `main()`'s `a` to `cout` afterwards.

Specifying `=` as a capture in a member function implicitly passes `this`, i.e. the pointer to the currently active object. Since C++20, accessing a member of the currently active object issues a warning by the compiler. If it is desired to additionally access members of the object—besides local member function variables—, it is advised to redundantly specify `this` after `=` in the capture clause, i.e. `[=, this]`.

Next, the capture clause specifies `a`. This makes `a` available as a copy with the same name in the lambda. For checking if `a` is larger than `b`, `b` is passed by value as an argument to the lambda expression. The rest of the definition is similar to the preceding ones.

The next two lines specify a capture clause with an ampersand. This makes all variables which are available at this point in `main()` available as references with the same name in the lambda. Making `a` and `b` of `main()` available is adequate, because the lambda swaps their contents. For this purpose, the helper variable `c` is declared in the lambda body. While having `a` and `b` available as references is intended here, other variables that are accessible in `main()` at this point also become available as references in the lambda which may be definitively undesirable. Therefore, it is preferable to explicitly specify only those variables that should become available as references in the capture clause, e.g. `[&a, &b] () // ...`. There is another possible source of subtle errors when passing references in the capture clause to a lambda: If the lambda—for whatever reasons—outlives its declaring function, it contains dangling references, a situation that should be definitively avoided. Therefore, specifying references in the capture list should be done with maximum care and well-thought.

If only `=` or `&` are specified in the capture clause, they represent the capture default. To their right hand side exceptions can be specified, e.g. `[=, &a, &b]`

means, that all variables are made accessible by value, but `a` and `b` become accessible as references. Conflicting or redundant specifications in the capture clause should be avoided. Usually, they are reported as errors by the compiler.

The last two lines illustrate the additional option of specifying a new variable in the capture clause, which in this case must always be initialized. Here, `c` is introduced and initialized with `a`. In this way, the declaration and initialization of the helper variable `c` in the lambda body becomes redundant. However, this simply moves the position of the declaration and initialization of `c` without providing any special benefit in this case.

5.4.2.3. Capturing this and *this

It is also possible to specify `this` and `*this` as capture. While `this` creates a copy of the `this` pointer pointing to the currently active object, `*this` creates a copy of the entire currently active object. In both cases, the lambda expression can access every member of the corresponding object, regardless of the visibility specified for it.

Listing 218 details both options, including the consequences of using them. First, the use case is briefly explained. Raw statistical data can contain so-called *outliers* because of various reasons, e.g. due to measurement errors or incorrect data entry. There are basically two approaches to dealing with outliers. The first is to delete the individual data rows that contain one or more outliers. The second is to trim lower outliers to a lower limit and upper outliers to an upper limit. Both approaches have their advantages and disadvantages, which will not be discussed here. The approach implemented below is to trim outliers. This measure is also known as *winsorizing*. The design chosen for this is not particularly elegant. It introduces a `Winsorizer` class that can be initialized with limits. In addition, there is a member function `limits()` for explicitly setting or resetting previously selected limits. Furthermore, there are the member functions `winsorize()` and `winsorize2()`, which combine a range of data with a lambda that performs the winsorizing and returns a corresponding object.

Listing 218: functional/CaptureThis.cpp

```
1 // CaptureThis.cpp by Ulrich Eisenecker, November 27, 2025
2
3 #include <iostream>
4 #include <limits>
5 #include <ranges>
6 #include <stdexcept>
7 #include <vector>
8
9 constexpr auto verbose { true };
10
11 class Winsorizer
12 {
13 public:
14     Winsorizer(double lowerLimit = std::numeric_limits<double>::lowest(),
15               double upperLimit = std::numeric_limits<double>::max() ):
16         m_lower { lowerLimit },
17         m_upper { upperLimit }
18     {
19         if (m_lower >= m_upper)
20         {
21             throw std::invalid_argument("Error: m_lower >= m_upper");
22         }
23         if constexpr(verbose)
24         {
25             std::cout << "Winsorizer::Winsorizer("
26                       << m_lower << ', ' << m_upper
27                       << ') ' << std::endl;
```

```

28     }
29 }
30 Winsorizer(const Winsorizer& w):m_lower(w.m_lower),
31                               m_upper(w.m_upper)
32 {
33     if constexpr(verbose)
34     {
35         std::cout << "Winsorizer::Winsorizer(const Winsorizer&)"
36                     << std::endl;
37     }
38 }
39 Winsorizer(Winsorizer&& w):m_lower(w.m_lower),
40                               m_upper(w.m_upper)
41 {
42     if constexpr(verbose)
43     {
44         std::cout << "Winsorizer::Winsorizer(Winsorizer&&)"
45                     << std::endl;
46     }
47 }
48 ~Winsorizer()
49 {
50     if constexpr(verbose)
51     {
52         std::cout << "Winsorizer::~~Winsorizer("
53                     << m_lower << ',' << m_upper
54                     << ')' << std::endl;
55     }
56 }
57 Winsorizer& limits(double l = std::numeric_limits<double>::lowest(),
58                   double u = std::numeric_limits<double>::max())
59 {
60     if (l >= u)
61     {
62         throw std::invalid_argument("Error: l >= u");
63     }
64     m_lower = l;
65     m_upper = u;
66     return *this;
67 }
68 [[nodiscard]] auto winsorize(auto& data)
69 {
70     return data | std::views::transform(
71         [this] (double v)
72         {
73             if (v < m_lower) return m_lower;
74             if (v > m_upper) return m_upper;
75             return v;
76         } );
77 }
78 [[nodiscard]] auto winsorize2(auto& data)
79 {
80     return data | std::views::transform(
81         [*this] (double v)
82         {

```

```
83         if (v < m_lower) return m_lower;
84         if (v > m_upper) return m_upper;
85         return v;
86     } );
87 }
88 private:
89     double m_upper { };
90     double m_lower { };
91 };
92
93 void output (const std::string& message,
94             const std::ranges::forward_range auto& v,
95             std::ostream& os = std::cout)
96 {
97     os << message;
98     for (auto e : v)
99     {
100         os << ' ' << e;
101     }
102     os << std::endl;
103 }
104
105 using namespace std;
106
107 int main()
108 {
109     {
110         cout << "Capturing this" << endl;
111         vector d { -2.5, 4.0, 1.1 };
112         output("original:",d);
113         Winsorizer w { -2.0, 2.0 };
114         auto wd { w.winsorize(d) };
115         output("winsorized [-2.0,2.0]:",wd);
116         w.limits();
117         output("not winsorized:",wd);
118     }
119     {
120         cout << "\nCapturing *this" << endl;
121         vector d { -2.5, 4.0, 1.1 };
122         output("original:",d);
123         Winsorizer w { -2.0, 2.0 };
124         auto wd { w.winsorize2(d) };
125         output("winsorized [-2.0,2.0]:",wd);
126         w.limits();
127         output("still winsorized:",wd);
128     }
129 }
```

First, the code that is not directly related to the functional requirements will be presented in more detail. The `constexpr bool` flag `verbose` controls the amount of additional information that the program outputs during execution. If it is `false`, there is no

additional output regarding the creation and deletion of objects; otherwise, calls to conversion constructors, copy constructors, and move constructors, as well as destructors, are reported.

The constructors and destructors of `Winsorizer` contain code that is included in the binary file by `if constexpr` when `verbose` is `true`.

The `output()` function takes a message and a range parameter and prints the range elements to `cout`, unless another output stream is explicitly specified as the third parameter.

The `main()` function contains two blocks that are almost identical. The first block illustrates the semantics of capturing `this`, the second block illustrates capturing `*this`. In both blocks, the vector `d` is defined and initialized with four values. Then `d` is passed to `output()`. Next, a `Winsorizer` exemplar named `w` is created with the limits `-2.0` and `2.0`. Now there is a difference. In the first block, the member function `winsorize()` is called for `w`, passing `d` as a parameter and copying the result of the function to the auto variable `wd`. In the second block, the function `winsorize2()` is called for `w`.

Then, the result of winsorizing the data referred to by `wd` is passed to `output()`. Next, `w.limits()` is called without specifying any parameters. This resets the previously selected limits. When `wd` is output in the first block, this change is displayed in the terminal window. Therefore, the message "not winsorized" was chosen. However, when `wd` is output in the second block, the winsorized data is still printed, i.e., the call to `w.limits()` obviously has no effect. Therefore, "still winsorized" was chosen as the message.

A closer look at the implementations of `winsorize()` and `winsorize2()` reveals the reason for this difference.

In Section `std::set<>`, a range-adapter is introduced that lazily transforms the elements of a range. This enables a syntax similar to piping in a terminal window. For this purpose, the `|` operator is overloaded so that its left operand is a range that is combined with its second operand, a range adapter. Here, this range adapter is created by calling the function `std::views::transform()`, passing a lambda expression as an argument. The capture clause of this lambda expression specifies `this`, referring to the object for which the member function `winsorize()` is currently being executed. The pointer `this` is passed by value. This makes every member of the currently active object available to the lambda body. The single parameter of the lambda expression is specified as `double v`, meaning it is passed by value. The lambda is called by `std::views::transform()`. ***In this way, the variables used in the lambda are a mixture of the defining side and the calling side.***

The only difference in the member function `winsorize2()` is that it specifies `*this` instead of `this` as the capture. As a result, a copy of the currently active object is created and made accessible to the body of the lambda expression. The lifetime of this copy ends when the lifetime of the function object underlying the lambda expression ends.

The consequence of this difference are illustrated by the two blocks of `main()`.

In the first block, the auto variable `wd` is initialized by calling `w.winsorize(d)`. Therefore, `wd` is a range adapter associated with `w`. The following statement outputs the elements of `d` that have been winsorized according to the limits with which `w` was previously initialized. However, the following statement `w.limits();` resets `w`. Therefore, printing `wd` again prints the elements with their original values without winsorizing. This is the result of specifying `this` as a capture. Therefore, `w` and the object pointed to by the captured `this` are identical.

In the second block, `wd` is initialized with the result of calling `w.winsorize2(d)`. This creates a copy of `w` that is used in the body of the lambda expression. Therefore, resetting `w` by calling `w.limits()` has no effect on the range adapter, as this only changes the original, not the copy used in the lambda. Consequently, passing `wd` to output continues to output the winsorized elements.

Specifying `this` in the capture clause allows subsequent changes to the object that is used in the lambda expression. Although this possibility seems fascinating at a first glance, it leads to state changes in the program, which is not really compatible with value-oriented design and functional programming. It also carries the risk that the lambda will outlive its defining member function and use a dangling pointer, as this copy no longer refers to a valid object.

These problems are completely avoided when `*this` is specified as a capture. This excludes possible state changes in the range adapter, and the copied object remains in existence as long as the underlying function object of the lambda expression exists.

It is instructive to compile the program with `verbose` initialized to `true` for different compilers.

When this version of the program is compiled with *g++ 15.2.0* on *macOS*, the output of the first block in `main()` remains unchanged. The reason for this is that copying the `this` pointer is not reported. The second block is different because specifying `*this` as the capture reports all corresponding constructor and destructor calls, as shown in Figure 53.

Compiling and executing the same source program with *clang++ 21.1.6* on *macOS* produces the output for the second block of `main()`, as shown in Figure 54.

Obviously, the processing of lambda expressions varies in some details between different compilers. *clang++* reports another call to a move constructor and the corresponding destructor, and creates and deletes objects in a different order. These differences in behavior should not be a cause for concern, but it is strongly recommended not to write code that depends on such specific behavior.

It is left as an exercise for the interested reader to modify the program `functional/CaptureThis.cpp` to demonstrate the effects of a lambda that outlives its defining function when passing `this` or `*this` as a capture.

Figure 53: Output of 2nd block in main() of functional/CaptureThis.cpp for g++ 15.2.0

```

Capturing *this
original: -2.5 4 1.1
Winsorizer::Winsorizer(-2,2)
Winsorizer::Winsorizer(const Winsorizer&)
Winsorizer::Winsorizer(Winsorizer&&)
Winsorizer::Winsorizer(Winsorizer&&)
Winsorizer::Winsorizer(Winsorizer&&)
Winsorizer::~~Winsorizer(-2,2)
Winsorizer::~~Winsorizer(-2,2)
Winsorizer::~~Winsorizer(-2,2)
winsorized [-2.0,2.0]: -2 2 1.1
still winsorized: -2 2 1.1
Winsorizer::~~Winsorizer(-2,2)
Winsorizer::~~Winsorizer(-1.79769e+308,1.79769e+308)

```

Figure 54: Output of 2nd block in main() of functional/CaptureThis.cpp for clang++ 21.1.6

```

Capturing *this
original: -2.5 4 1.1
Winsorizer::Winsorizer(-2,2)
Winsorizer::Winsorizer(const Winsorizer&)
Winsorizer::Winsorizer(Winsorizer&&)
Winsorizer::Winsorizer(Winsorizer&&)
Winsorizer::~~Winsorizer(-2,2)
Winsorizer::Winsorizer(Winsorizer&&)
Winsorizer::Winsorizer(Winsorizer&&)
Winsorizer::~~Winsorizer(-2,2)
Winsorizer::~~Winsorizer(-2,2)
Winsorizer::~~Winsorizer(-2,2)
winsorized [-2.0,2.0]: -2 2 1.1
still winsorized: -2 2 1.1
Winsorizer::~~Winsorizer(-2,2)
Winsorizer::~~Winsorizer(-1.79769e+308,1.79769e+308)

```

This concludes the overview of functions in C++ in traditional and modern form from the perspective of functional programming. The following sections highlight further concepts and techniques of functional programming.

5.4.3. Currying

A function can have zero, one, or more arguments. In functional programming, a function without arguments is suspicious, because it could be impure. In a few cases, functions without arguments are justified, e.g. to generate pseudo-random numbers or to intentionally always return the same value.

A (pure) function with exactly one argument is relatively simple as it is sufficient to know the types of its single argument and its result to be informed about the mapping from the argument type to the result type—optionally with argument values for which the function is undefined. Examples are a function that returns the square of a floating point number as a floating point number, and a function that returns the square root of a floating point number as a floating point number, of which the latter can not be called for negative floating point numbers.

This can become more complex with a function that has two or more arguments. An example of a function with two arguments is the calculation of the logarithm of a number with a specific base—where the arguments and the result are floating-point numbers. A question that can immediately arise concerns the order of the parameters.

This can be changed by transforming a function that is called with two or more arguments into a series of functions, each of them being called with exactly one argument. This is called *currying*, a term that has nothing to do with the famous spice mixture, but rather refers to the name of the logician *Haskell Curry*, who frequently used this concept (*Currying* 2025). Listing 219 shows how to implement currying using a lambda expression.

Listing 219: functional/Currying.cpp

```

1 // Currying.cpp by Ulrich Eisenecker, January 12, 2026
2
3 #include <cmath>
4 #include <iostream>
5
6 auto logarithm(double base, double number)
7 {
8     return std::log(number) / std::log(base);
9 }
10
11 auto logarithm(double base)
12 {
13     return [base](double number) { return std::log(number) / std::log(base); };
14 }
15
16 using namespace std;
17
18 int main()
19 {
20     cout << logarithm(2.0, 16.0) << endl;
21     cout << logarithm(2.0)(16.0) << endl;

```

```
22 | // logarithm to the base 2.0
23 | auto ld { logarithm(2.0) };
24 | cout << ld(16.0) << endl;
25 | // logarithm to the base 10
26 | const auto log10 { logarithm(10.0) };
27 | cout << log10(10000.0) << endl;
28 | }
```

The `logarithm()` function has two arguments, `base` and `number`, both of type `double`, in exactly that order. Its return type is specified as `auto`, as it is deduced from the return value. The function calls `std::log()`, which calculates the natural logarithm (its base is the *Euler* number) of its argument. According to the rules for logarithms, the logarithm of number divided by the logarithm of base gives the logarithm of number for base.

The first line of the `main()` function calls this function with the arguments `2.0` for base and `16.0` for number, and sends the result `4` to `cout`.

The following overload of the `logarithm()` function accepts only one `double` parameter, namely `base`. Its return type is specified as `auto` and is inferred to be the closure type of the lambda that is returned as a copy. The implementation returns a lambda, passing `base` as a capture and `number` as an argument. The implementation of the lambda corresponds to the first overload of `logarithm()`.

The result of calling `logarithm(double)` is a lambda that also accepts a `double` value as a parameter. This is demonstrated in the second statement of `main()`. Here, `logarithm()` is called with `2.0` as a parameter. The result is a temporary object, which is a lambda that internally binds `2.0` as the value for the base. This temporary function is immediately called with `16.0` as an argument, i.e. the number for which the logarithm is to be calculated. This executes the lambda that calculates and returns the desired logarithm. The resulting statement does not resolve the issue with the order of the parameters and can look even more complex.

This changes in the following lines of `main()`. The `auto` variable `ld` is initialized with a copy of `logarithm(2.0)`. `ld()` is now a function that calculates the *logarithmus dualis* (*ld* for short). This is exactly what is intended.

In fact, `ld` could be assigned a new value, which would contradict the spirit of functional programming. Defining `log10` as `const auto` shows how this can be prevented.

It is important to note that the C++ standard library includes the functions `log2()` and `log10()`, which calculate both the dual logarithm and the decimal logarithm.

It is also worth noting that `logarithm(double)` should not call `logarithm(double, double)` when implementing its lambda, as this would introduce a directed coupling between these two functions, which is probably undesirable. The `logarithm(double, double)` function is included here for didactic reasons only. It would be enough to just have the `logarithm (double)` function available.

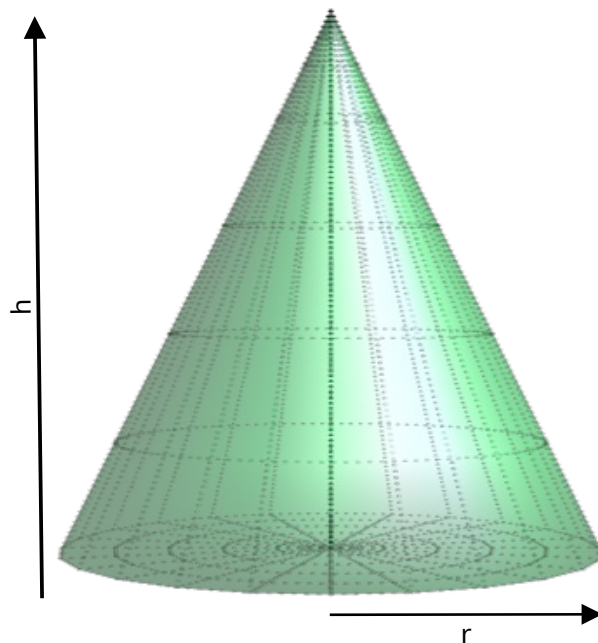
5.4.4. Partial Function Application

Currying supports the creation of families of related functions, but it is limited to binding only the first parameter of a function. It is also possible to bind arbitrary parameters of a function. This is another technique for creating a new function based on an existing one which is called *partial function application*. It can be considered more general as currying. Both techniques should not be confused.

Partial function application not only supports the creation of families of related functions based on a single general function, but can be also be useful for adapting a function for use in a framework that expects a semantically compatible function but a different interface.

The calculations of the cone volume and the conical segment volumes are used for illustration purposes. Figure 55 shows a green cone and its defining dimensions r and h .

Figure 55: Cone



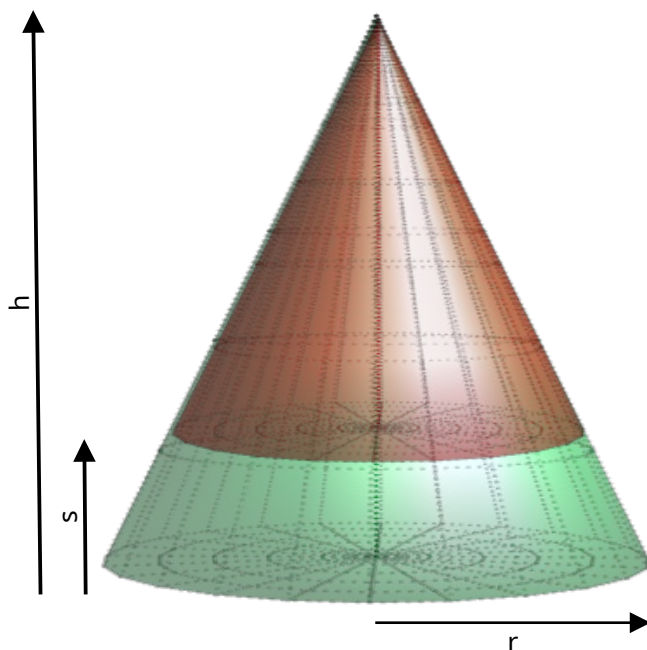
Its volume is calculated according to the formula $V_{cone} = \frac{1}{3}\pi r^2 h$ that corresponds 1:1 to its definition as a C++ function (Listing 220). It must be emphasized that `coneVolume()` is also a partial function in the first sense (see Section *Partial Function*). It should only be called for positive values of r and h , but this is not checked.

Listing 220: coneVolume() (excerpt from functional/Cone.cpp)

```
11 | [[nodiscard]] auto coneVolume(double r, double h)
12 | {
13 |     return 1.0 / 3.0 * r * r * std::numbers::pi * h;
14 | }
```

Figure 56 shows a green conical segment of height s , defined as a cone with its red upper part cut off.

Figure 56: Conical segment



To calculate its volume the volume of the red upper cone is subtracted from the volume of the complete cone that consists of the green conic segment plus the red part. The height of the red cone t is given by $h - s$ and its radius is calculated as $\frac{r \times t}{h}$. This translates to the function shown in Listing 221, which deserves some explanation.

Listing 221: conicalSegmentVolume() (excerpt from functional/Cone.cpp)

```

16 [[nodiscard]] double conicalSegmentVolume(double r,double h,double s)
17 {
18     double t { h - s };
19     return (r == 0.0 || h == 0.0) ?
20         0.0 :
21         r * r * std::numbers::pi * h / 3.0 - ((s < h) ?
22             conicalSegmentVolume(r * t / h,t,t) :
23             0.0);
24 }

```

It has one more parameter than `coneVolume()`, namely `s`. As with `r` and `h`, only positive values should be passed for `s`. In addition, `s` interacts with `h`, because `s` cannot be greater than `h`. If `s` is equal to `h`, the volume should be identical to that of a cone with the given radius and `h`.

The implementation of the function is recursive, as it calculates the volume of the entire cone (line 21) and subtracts the volume of the cone to be cut away (line 22). Since the height of the cone to be cut away is needed several times, the local variable `t` is initialized accordingly at the beginning of the function. Next, the ternary operator is used to check whether `r` or `h` is `0.0`. If so, the function returns `0.0` as the result. Otherwise, the volume of a cone with height `h` is calculated. The volume of the smaller cone must be subtracted from the resulting value, but only if `s` is less than `h`, which is again checked using the ternary operator in the last sub-expression in line 21. If the condition is true, `conicalSegmentValue()` is called recursively with the radius $\frac{r \times t}{h}$ and `t` as second and third parameters. If the condition is false, `0.0` is subtracted from the cone volume. The result of this complex expression is returned as the function result.

To improve comprehensibility, this complex expression could be broken down into several smaller smaller statements.

One small detail is worth mentioning. The return type of `coneVolume()` is specified as `auto`. This is not possible for `conicalSegmentVolume()` due to its recursive implementation. In this case the `->` syntax had to be used to specify the return type, i.e. `[[nodiscard]] auto conicalSegmentVolume(double r,double h,double s) -> double`. Specifying the return type as `double` before the function name is simpler.

Now all prerequisites for demonstrating partial function application are complete. The following code snippets are all taken from *Catch2* v3 unit tests contained in *functional/Cone.cpp*.

As explained above, `conicalSegmentVolume()` is the more general function. When its third parameter is identical to the second parameter, it corresponds to `coneVolume()`. For this purpose, a function call adapter is required that accepts two arguments and calls `conicalSegmentVolume()` with the actual second argument as expected arguments two and three. The C++ standard library provides the function template `std::bind<>()`

in the `<functional>` header file for precisely this purpose (*std::bind* - *Cppreference.Com* n.d.). The first argument of `bind<>()` is the pointer to the function to be adapted. In Listing 222 this is `conicalSegmentVolume`. So-called *placeholders* are used to specify the arguments for calling the customized function. These are also defined in the `<functional>` header file and contained in the namespace `std::placeholders` (*std::placeholders::_1*, *std::placeholders::_2*, ..., *std::placeholders::_N* - *Cppreference.Com* n.d.).

Listing 222: TEST_CASE("BIND1") (excerpt from functional/Cone.cpp)

```
47 | using namespace std::placeholders;
48 | auto coneVolume2 { std::bind(conicalSegmentVolume,_1,_2,_2) };
49 | std::cout << coneVolume2(6.0,5.0) << std::endl;
```

In Listing 222, `conicalSegmentVolume()` is adapted so that it can be used instead of `coneVolume()`. Since the partially applied function only accepts two parameters, these must be mapped to the three parameters expected by `conicalSegmentVolume()`. This is done by specifying the placeholder `_1` as the first and the placeholder `_2` as the second and third parameters. Because `std::bind<>()` has a relatively complicated return type, its value is used to initialize the auto variable `coneVolume2`. After that, `coneVolume2()` can be called just like `coneVolume()`.

The next example deals with a so-called *equilateral cone*. Its diameter, which is two times the radius, equals its height, thus its volume depends on only one parameter. When adapting `conicalSegmentVolume()`, the placeholder `_1` represents the radius of the equal cone. But as the values of the parameters two and three the radius times two is required. Listing 223 shows a possibility how to do this.

Listing 223: TEST_CASE("BIND2") (excerpt from functional/Cone.cpp)

```
58 | using namespace std::placeholders;
59 | auto times2 { [](double n) { return 2.0 * n; } };
60 | auto equilateralConeVolume { std::bind(conicalSegmentVolume,_1,
61 |                                     std::bind(times2,_1),
62 |                                     std::bind(times2,_1)) };
63 | std::cout << equilateralConeVolume(6.0) << std::endl;
```

First, a lambda with a single double argument `n` is defined. The product of `2.0` and `n` is returned as the result. Because the lambda is needed twice, the auto variable `times2` is initialized with it. Then `bind<>()` is called with `conicalSegmentVolume` as the first parameter. The first argument, the radius, is specified with the placeholder `_2`. To specify the second and third arguments, `bind<>()` is called repeatedly. It binds `times2` as a function and the placeholder `_1` as an argument. To obtain the actual values for parameters two and three of `conicalSegmentVolume()`, the lambda `times2` is therefore called with `_1` as an argument.

The auto variable `equilateralConeVolume` is initialized with the result of calling `bind<>()`. Afterwards, `equilateralConeVolume()` can be called as a function with only one parameter.

Listing 224 shows an alternative way to achieve the same result. Here, `coneVolume2()` is first created as explained in Listing 223. Then `coneVolume2()` is adapted so that it is called with the radius as the only parameter. Because the lambda that multiplies the radius by two is now only needed once, it is specified directly as the first argument for `bind<>()` in line 4. In this case, three functions are now involved: the original functions `conicalSegmentVolume()` and `coneVolume()`, as well as `equilateralConeVolume()`. While this may be intriguing, there is also a drawback: `equilateralConeVolume()` depends on `coneVolume2()`, and `coneVolume2()` depends on `conicalSegmentVolume()`. These dependencies lead to direct and indirect couplings between all these functions, thereby violating the principles of good modular design. These advantages and disadvantages must be carefully weighed against each other.

Listing 224: TEST_CASE("BIND3") (excerpt from functional/Cone.cpp)

```

70 |     using namespace std::placeholders;
71 |     auto coneVolume2 { std::bind(conicalSegmentVolume,_1,_2,_2) };
72 |     auto equilateralConeVolume { std::bind(coneVolume2,_1,
73 |                                           std::bind([](double n)
74 |                                           { return 2.0 * n; },_1)) };
75 |     std::cout << equilateralConeVolume(6.0) << std::endl;

```

A last possibility is demonstrated in Listing 225. Here, partial function application is used to obtain `equilateralConeVolume()` directly from `coneVolume()`. As the simplest case, this needs no further explanation.

Listing 225: TEST_CASE("BIND4") (excerpt from functional/Cone.cpp)

```

82 |     using namespace std::placeholders;
83 |     auto equilateralConeVolume { std::bind(coneVolume,_1,
84 |                                           std::bind([](double n)
85 |                                           { return 2.0 * n; },_1)) };
86 |     std::cout << equilateralConeVolume(6.0) << std::endl;

```

Partial function application is a powerful tool. Its use in creating families of related functions must be carefully considered, as it creates a coupling between the functions. If this is deemed acceptable, it must still be taken into account that each function call ultimately calls the most basic and potentially most complex function. In cases where performance is critical, benchmarking can help to assess whether this is an issue that needs to be addressed.

The most important use case for partial function application is providing adapters for functions so that they can be used in frameworks that expect semantically similar functions with different interfaces. Here, partial function application is a well-known and easy-to-use technology.

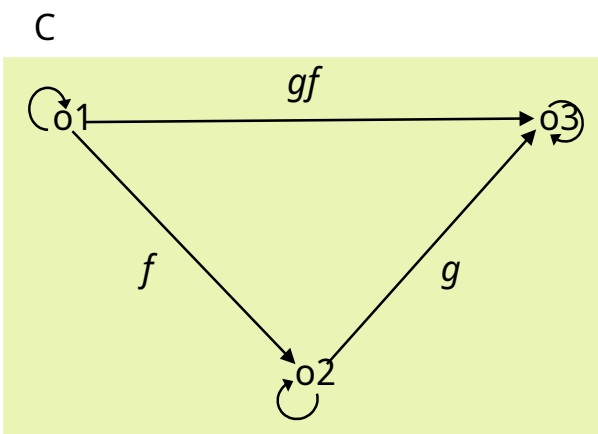
5.4.5. Functor

In C++, the term *functor* is often used to refer to objects of a class with an overloaded function call operator. To avoid confusion with functional programming, such objects should be referred to as *function objects* (Section *Function Objects*). In functional programming, the term functor originates from mathematical *category theory*. Its meaning is presented in a very abbreviated form below. Some concrete examples are given and functors in C++ are discussed.

A *category* comprises *objects* (which do not correspond to objects in object-oriented programming) and *morphisms* between two objects (*Category (Mathematics)* 2025). It is important not to confuse a *category* with a *set* and an *object* with an *element*, because from a mathematical point of view category theory replaces or supersedes set theory. A suitable example is therefore the category of *natural number objects*. A morphism can be understood as a mapping from one object to another, for example as a successor, i.e. *2 is the successor of 1*. Mathematically, a morphism f is denoted as $f: o_1 \rightarrow o_2$. There are three defining properties of a category C (here with objects o_1, o_2 and o_3), see Figure 57:

- There exists an identity mapping that maps each object onto itself (*identity*).
- Morphisms must be composable i.e., if a morphism f maps o_1 to o_2 and a morphism g maps o_2 to o_3 , then gf must map o_1 to o_3 (*composition*). Mathematically, the composition of the morphisms f and g is denoted as $f: o_1 \rightarrow o_2$ and $g: o_2 \rightarrow o_3$ or, or gf for short.
- The composition of morphisms must be associative, i.e., $h(gf) = (hg)f$.

Figure 57: Category



It must be emphasized that there are no restrictions on the type of objects in a category. The number of objects in a category should be at least 1, and there are also no restrictions on morphisms, apart from the requirement that a morphism maps an object of one category to another object of the same category. In case of the identity morphism, the two objects involved are identical.

An example of a well known mathematical category is *Set*, the category of *sets* and *set functions* (Category (Mathematics) 2025). This is noteworthy in that some theorists hold the view that the set of all sets cannot be a set, since as such it would have to contain both the empty set and the power set of itself as elements. This problem is circumvented in other theories, which exclude a set of all sets and instead assume a specific class (here referred to as the *Universe*) that contains all subsets, i.e., elements of itself, but not itself. Furthermore, there are other variants based on different assumptions and taking other restrictions into account. Since it is quite possible to feel queasy in this field of study, this topic will be left here and not revisited.

In category theory, a functor F maps a category C onto a category D such that

- each object o of C is associated to an object $F(o)$ in D
- and every morphism f that maps o_1 to o_2 in C is associated with a morphism $F(f)$ that maps $F(o_1)$ to $F(o_2)$ (remember: $F(o_1)$ and $F(o_2)$ are objects of category D); in mathematical notation: every $f: o_1 \rightarrow o_2$ is associated with $F(f): F(o_1) \rightarrow F(o_2)$,

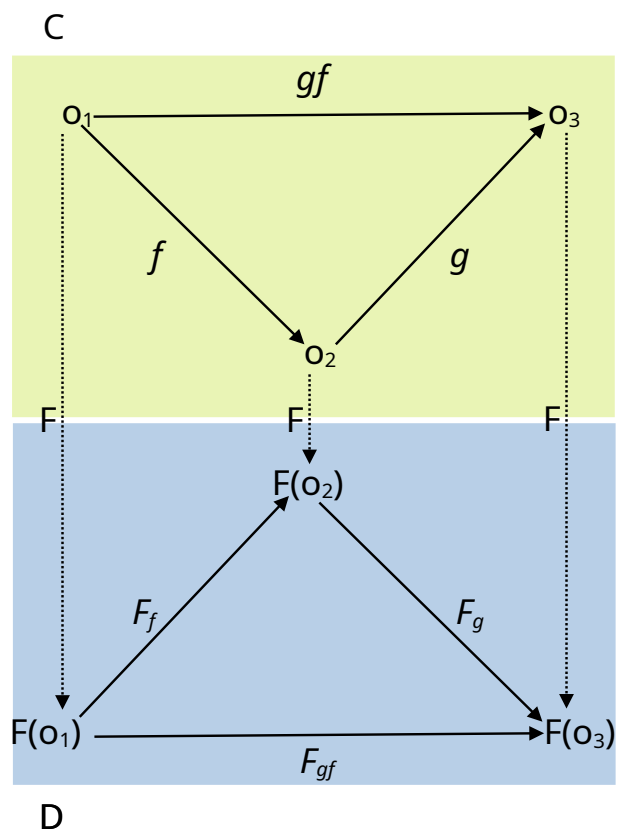
where two conditions must apply:

- F , applied to the identity morphism id of o , is equal to the identity morphism of F applied to o for all objects o in category C , i.e. $F(id_o) = id_{F(o)}$ for every object o in C .
- F , applied to the combined morphisms g and f , is equal to F , applied to g , combined with F , applied to f , for all morphisms f that map o_1 to o_2 , and g that map o_2 to o_3 , i.e., $F(gf) = F(g) F(f)$ for all morphisms $f: o_1 \rightarrow o_2$ and $g: o_2 \rightarrow o_3$ in C (Functor 2025).

In summary, a functor F preserves the composition of morphisms g and f (Figure 58).

From a purely theoretical perspective, a functor is a mapping, and in category theory, the goal is to map morphisms to another category. In the specific case where a functor maps a category onto itself, it is called the *identity functor*.

Functors can be implemented in different ways. Listing 226 shows an example that implements a functor as a class template. More precisely, it is an *identity functor*, since it maps a category onto itself, which in most cases is of only limited use.

Figure 58: Functor

Listing 226: functional/Functor.cpp

```

1 // Functor.cpp by Ulrich Eisenecker, February 18, 2026
2
3 #include <iostream>
4
5 using namespace std;
6
7 using ulli_t = unsigned long long int;
8
9 ulli_t output(ulli_t n)
10 {
11     cout << n << endl;
12     return n;
13 }
14
15 [[nodiscard]] ulli_t plus1(ulli_t n)
16 {
17     return n + 1ull;
18 }
19
20 [[nodiscard]] ulli_t factorial(ulli_t n)
21 {
22     return n == 0ull ? 1ull : n * factorial(n - 1ull);
23 }
24
25 template <typename T>
26 class Functor
27 {
28     private:
29         const T m_value;
30     public:
31         Functor(T v):m_value { v }
32         {}
33         auto map(auto f) const
34         {
35             return Functor<decltype(f(m_value))> { f(m_value) };
36         }
37 };
38
39 template <typename T>
40 auto operator|(const Functor<T>& v, auto f)
41 {
42     return v.map(f);
43 }
44
45 int main()
46 {
47     const Functor f5 { 5ull };
48     output(5ull);
49     /* --> */ f5.map(output);
50     /* --> */ f5 | output;
51     output(plus1(5ull));

```

```
52 |     /* --> */ f5.map(plus1).map(output);
53 |     /* --> */ f5 | plus1 | output;
54 |     output(factorial(plus1(5ull)));
55 |     /* --> */ f5.map(plus1).map(factorial).map(output);
56 |     /* --> */ f5 | plus1 | factorial | output;
57 | }
```

The class template `Functor<>` takes the type template parameter `T` and defines a private `const` data member `m_value` of type `T`. Its public constructor accepts an argument of type `T`, which copy-initializes `m_value`. The public member function `map()` is a higher-order function because it takes one `auto` parameter named `f` which it calls with `m_value` as an argument. The function `f()` therefore has access to `m_value` and can use it to calculate its return value. The return value is used to initialize a new `Functor<>` exemplar, which is returned as the result of calling `map()`. Together, the constructor and the member function `map()` form the essential characteristics of a functor. In functional programming, `map()` is also referred to synonymously as `fmap()` or `transform()`.

Although there is no public member function that returns `Functor<>::m_value`, the function `f()` passed to `Functor<>::map()` can access it. This makes a functor both a secure capsule for its member data and a generalized interface for applying functions to it. In the example above, these are the functions `output()`, `plus1()` and `factorial()`. Each of these functions takes an argument of type `ulli_t`, which is an alias for unsigned long long int, and also returns a result of type `ulli_t`.

The `main()` function demonstrates the use of the aforementioned functions. First, `f5` is declared as `const Functor<>` and initialized with `5ull`. The next statement directly calls `output(5ull)`. It is not possible to call `output()` with `f5` as an argument because no corresponding type conversion is available. Instead, `f5.map(output)` is called. The name of a function without the function call operator `()` and its arguments is a pointer to the function (see Section *The Virtual Function Table*). In `Functor<>::map()` the function is called with `m_value` as its argument. The desired side effect is the output of the value 5 in the terminal window.

The following line will be addressed later.

The next statement contains the nested function calls `output(plus1(5ull))`. Using `f5` and `map()`, this can be rewritten sequentially, i.e., `f5.map(plus1).map(output)`. The statement two lines further contains three nested function calls, `output(factorial(plus1(5ull)))`, which are rewritten as `f5.map(plus1).map(factorial).map(output)`. The sequence resulting from the use of `f5` and `map()` is more readable. If another function call had to be inserted somewhere, this would be easier with sequential calls than with nested function calls. From an object-oriented programming perspective, one could argue that chaining member function calls, as is done here with `map()`, is considered bad style. However, this argument

is not valid because this chaining does not depend on the internal structure of one or more objects and therefore does not result in a path expression that breaks when the internal structure of the participating objects is changed.

There is another subtle aspect. As already mentioned, functional programming is a-temporal, i.e., a function is called when its result is needed. Therefore, the compiler can freely optimize the order of function calls. The situation is different with nested or chained function calls. They force the compiler to call the functions in the specified order.

The higher-order function `map()` is a uniform interface for calling functions on a functor. This forms the basis for further extensions or syntactic sugar. Although the functor in combination with `map()` converts nested function calls into a sequence of function calls, the latter still requires some typing. Some functional programming languages use the pipe operator, `|`, to compose function calls. Typically, using the pipe operator also causes delayed evaluation. Here, however, it is only overloaded to simplify coding, not for the purpose of delayed evaluation.

In Listing 226, the `|` operator is overloaded by taking a functor `v` and a function pointer `f` as arguments. It simply calls `v.map(f)`. This allows the chained `map()` calls to be shortened while maintaining their order. In this way, `f5.map(output)` becomes `f5 | output`. The longest sequence is shortened to `f5 | plus1 | factorial | output`. In terms of software qualities such as readability and changeability, this is a significant improvement. Without `Functor<>`, this would not be possible because the operator `|` cannot be overloaded for basic data types, and it would be very difficult to develop multiple overloads of `|` for different user-defined data types that can be combined as easily as the single overload of `|` for `Functor<>`.

One could argue that the implementation of `Functor<>::map()` is more complex than necessary, as it could simply return `Functor { f(m_value) }`. However, this would prevent the functions called for `m_value` from returning a type other than that of `m_value`. Therefore, the type returned by `f()` is determined by applying the `decltype` operator to the expression `f(m_value)`. This also allows to return functors instantiated for different types, as shown in Listing 227.

Listing 227: functional/TypePlay.cpp

```

1 // TypePlay.cpp by Ulrich Eisenecker, February 19, 2026
2
3 #include <iostream>
4 #include <string>
5
6 using namespace std;
7
8 template <typename T>
9 [[nodiscard]] T output(T n);
10
11 template<>
```



```
12 [[nodiscard]] double output(double n)
13 {
14     cout << n << endl;
15     return n;
16 }
17
18 template<>
19 [[nodiscard]] string output(string n)
20 {
21     cout << n << endl;
22     return n;
23 }
24
25 [[nodiscard]] double grossTotal(double n)
26 {
27     return n * 1.19;
28 }
29
30 [[nodiscard]] string withCurrency(double n)
31 {
32     return to_string(n) + " EUR"s;
33 }
34
35 template <typename T>
36 class Functor
37 {
38     private:
39         const T m_value;
40     public:
41         Functor(T v):m_value { v }
42         {}
43         auto map(auto f) const
44         {
45             return Functor<decltype(f(m_value))> { f(m_value) };
46         }
47 };
48
49 int main()
50 {
51     const Functor amount { 11.0 };
52     amount.map(output<double>).map(grossTotal)
53         .map(withCurrency).map(output<string>);
54 }
```

Since `Functor<>::map()` accepts a function pointer as an argument, the function pointer must be precisely identified, which is not possible in the context of overload polymorphism, i.e. different overloads of the `output()` function cannot be used here. For this reason, the declaration of the `output()` function template contains special definitions that overload it for `double` and `string`. This way, the function pointers can be effectively distinguished using `output<double>` and `output<string>`.

The `main()` function first declares a `Functor<double>` named `amount`. In the following statement, its value is first output, then the gross sum is calculated, which is then converted into a string with a currency abbreviation and output again. This clearly shows that the type for which a functor is instantiated can be changed by applying `map()`, which becomes relevant in the following section.

The hard-coding of the tax rate and currency abbreviation in the functions `grossTotal()` and `withCurrency()` is not flexible. This can be easily improved using the methods already presented, namely currying and `bind()`. It is strongly recommended to implement one's own solutions before looking at the programs *functional/TypePlayCurrying.cpp* and *TypePlay/Bind.cpp* in the accompanying material.

In terms of efficiency, one could argue that it would be more advantageous to manually create specific functions that combine the functionality of `factorial()`, `plus1()`, and `output()` as needed. However, this would result in functions serving more than one purpose, which is considered poor design. A valid argument is that the number of function calls doubles for nested function calls compared to sequential function calls with functors and `map()`. Inlining and requesting aggressive optimizations during compilation can eliminate some of the resulting overhead. If there is indeed an unacceptable performance degradation, benchmarking or profiling should be used to identify the bottlenecks.

It is important to understand that only the fulfillment of the relevant requirements is decisive for a functor to become a functor. For example, `std::vector<>` and `std::for_each<>()` also form a functor because `vector<>` encloses any number of objects and `for_each<>()` provides a uniform interface for applying unary functions to these objects, even though `for_each<>()` does not support chaining function calls. The two mandatory features of a functor are

- a data structure capable of managing any number of objects—that is, in principle, from 0 to infinity (though infinity does not typically occur in practice)—and
- an interface that supports the uniform invocation of unary functions on the objects managed by the data structure. Ideally, this interface should be a function separate from the data structure and allow for chaining of calls. However, these are optional features that, while desirable, are not strictly required.

For more examples, see (Čukić 2019), *Section 10: Monads*. (Allsopp 2016) shows how to implement a much more general functor template for creating functors for *C++14*, which requires several adjustments to bring it up to the level of *C++23*. However, the primary goal is not to implement functors using standardized technical or mathematical terms. Rather, it is more relevant to correctly implement the defining requirements of a functor in appropriate terms of the problem domain.

5.4.6. Monad

In Listing 227, all functions used in compositions return a normal value, i.e., a value that is not in a functional context. Listing 228 presents a contrived example that changes this situation. Here, a function is passed to `Functor<>::map()` that itself returns a `Functor<>` exemplar, namely `check()`, which leads to a problem. `check()` takes an `ullli_t` value as an argument and returns a `Functor<ullli_t>` with the value 0 if `n` is greater than 20, and `n` otherwise.

Listing 228: functional/NestedFunctor.cpp

```
1 // NestedFunctor.cpp by Ulrich Eisenecker, February 18, 2026
2
3 #include <iostream>
4
5 using namespace std;
6
7 using ullli_t = unsigned long long int;
8
9 template <typename T>
10 class Functor
11 {
12     private:
13         const T m_value;
14     public:
15         Functor(T v):m_value { v }
16         {}
17         auto map(auto f) const
18         {
19             return Functor<decltype(f(m_value))> { f(m_value) };
20         }
21         T flatten() const
22         {
23             return m_value;
24         };
25         auto bind(auto f) const
26         {
27             return flatten().map(f);
28         }
29 };
30
31 ullli_t output(ullli_t n)
32 {
33     cout << n << endl;
34     return n;
35 }
36
37 [[nodiscard]] ullli_t plus1(ullli_t n)
38 {
39     return n + 1ull;
40 }
41
42 [[nodiscard]] auto check(ullli_t n)
```

```

43 {
44     return Functor<ulli_t> { n > 20 ? 0 : n };
45 }
46
47 int main()
48 {
49     const Functor f { 20ull };
50     // f.map(output).map(plus1).map(check).map(output);
51     //                                     ^ Error
52     f.map(output).map(plus1).map(check).flatten().map(output);
53     //                                     ^ now ok!
54     f.map(output).map(plus1).map(check).bind(output);
55     //                                     ^ also ok!
56 }

```

Calling `check()` for 21 is fine, but `map()` returns a `Functor<Functor<ulli_t>>` with a value of 0 as the result. Commenting out the second line of `main()` causes exactly that to happen at the end of the expression. Here, `map()` is called for a `Functor<Functor<ulli_t>>` exemplar and attempts to call `output()` for a `Functor<ulli_t>` exemplar. However, such an overload of `output()` does not exist, and `Functor<>` does not provide a corresponding type conversion operator. Therefore, this attempt fails and causes a compilation error.

To fix this problem, the data element `Functor<ulli_t>` must first be extracted from the `Functor<Functor<ulli_t>>` exemplar. This is done using the member function `flatten()`. It simply returns `m_value`.

Removing a level from a (nested) functor is referred to as *flattening*. Therefore, the corresponding function in functional programming is called `flatten()`. The opposite of this is converting a value into a functor, which is known as *lifting*. The term *join* is used as a synonym for *flatten*, and the term *pure* is used as a synonym for *lift*.

The second statement (line 4) of `main()` illustrates the use of `flatten()`. It is called on the result of evaluating `...map(check)`, which removes one level of `Functor<>`. As a result, `map(output)` can then be safely applied.

Since `flatten()` and `map()` are often used together, functional programming offers a special function that combines both. This function is referred to synonymously as `bind()`, `flatMap()`, or `mbind()`. In the example in Listing 228, it is called `bind()`. Its implementation is simple: it returns the result that is first achieved by calling `flatten()` and then by a chained call to `map(f)`. The last statement of `main()` illustrates its use.

That's basically all it takes to turn a functor into a monad! Roughly speaking, one could say that **a monad is a functor plus `flatten()` and `bind()`**. The class template `Functor<>` in Listing 228 could therefore actually be called `Monad<>`. Functional programming also distinguishes between other entities such as *applicative functors* and *pointed functors* and requires that the higher-order functions `map()`, `flatten()`, and

`bind()` (or whatever they may be called) are free functions. For a first introduction to functors and monads, the simplified examples used here should be sufficient for didactic purposes.

A well-known monad in C++ is `std::optional<>`, which is presented below using a practical example. Next, the monad `std::expected<>` is briefly introduced. Using a purely constructed example, the vector monad is demonstrated in combination with transformation and flattening.

5.4.6.1. The Optional Monad

In Listing 229, the unary function `check()` is called with `n` of type `ulli_t` as an argument. `ulli_t` is a type alias for unsigned long long int. On 64 bit platforms, this type usually occupies 8 byte which allows the factorial of 20ull to be represented correctly. The return value of `check()` is specified as `optional<ulli_t>` (see Section *std::optional<>*) which contains a value of type `ulli_t` if successful and is empty otherwise. In its implementation, the function checks whether `n` is greater than 20ull. If so, it returns an empty `optional<>` exemplar. Otherwise, an `optional<>` exemplar with `n` is created and returned. In this way, calling `check()` creates a *wrapped object*.

Listing 229: functional/Optional.cpp

```
1 // Optional.cpp by Ulrich Eisenecker, February 12, 2026
2
3 #include <iostream>
4 #include <optional>
5
6 using namespace std;
7
8 using ulli_t = unsigned long long int;
9
10 [[nodiscard]] optional<ulli_t> check(ulli_t n)
11 {
12     if (n > 20ull)
13     {
14         return { };
15     }
16     return n;
17 }
18
19 [[nodiscard]] ulli_t plus1(ulli_t n)
20 {
21     return n + 1ull;
22 }
23
24 ulli_t factorial(ulli_t n)
25 {
26     return n ? n * factorial(n - 1) : 1;
27 }
28
```

```

29 [[nodiscard]] ulli_t output(ulli_t n)
30 {
31     cout << n << endl;
32     return n;
33 }
34
35 template <typename T>
36 auto operator/(const optional<T>& v, auto f)
37 {
38     return v.transform(f);
39 }
40
41 template <typename T>
42 auto operator%(const optional<T>& v, auto f)
43 {
44     return v.and_then(f);
45 }
46
47 int main()
48 {
49     check(5ull).transform(plus1).and_then(check)
50         .transform(output).transform(factorial)
51         .transform(output);
52     /* --> */ (check(5ull) / plus1).and_then(check) / output
53               / factorial / output;
54     check(20ull).transform(output).transform(plus1)
55         .transform(output).and_then(check).transform(factorial)
56         .transform(output);
57     /* --> */ check(20ull) / output / plus1 / output % check / output
58               / factorial / output;
59 }

```

The remaining functions `plus1()`, `factorial()`, and `output()` all accept one argument of type `ulli_t` and return a value of type `ulli_t`. The two overloaded operators will be discussed later.

In the first line of the `main()` function, evaluating `check(5ull)` results in an `optional<>` exemplar. `std::optional<>` has a member function `transform()`, which corresponds to `Functor<>::map()`. Here, `transform()` is called for the temporary `optional<>` exemplar with `plus1` as an argument. Internally, `optional<>::transform()` calls `plus1()` with the encapsulated value, if it exists, and returns the result of the call as another `optional<>` exemplar. If the value does not exist, `transform()` does not call anything and returns an empty `optional` instance. In this respect, `optional<>` is a functor.

For the result of `check(5ull).transform(plus1)`, it should be checked again whether the contained value is less than 21. If `check` were passed as an argument to `transform()` for this purpose, `check()` would be called for the value (if present). However, the `optional<>` exemplar returned by `check()` would again be enclosed as `optional<>`, which would result in a compilation error. Therefore, another function is required that

removes one level of `optional<>` before returning the result of the call to the function passed as an argument, namely the member function `and_then()`. It corresponds to the function `bind()` and converts a functor into a monad.

For this reason, `.and_then(check)` is appended to the expression `check(5ull).transform(plus1)`. From this point on, only unary functions that return `ull_i_t` values are applied. Therefore, `transform()` is used to chain the calls to `factorial()` and `output()`.

The third statement in `main()` uses `check(20ull)` to illustrate what happens when further function calls are chained together but the `optional<>` exemplar is empty. In all these cases, the function pointer passed as an argument to `transform()` is not called, and `transform()` returns an empty `optional<>` exemplar.

In Listing 226, the operator `|` is overloaded to support piping syntax. This would also be possible for `std::optional<>`, allowing every call to `transform()` to be replaced by `|`. Unfortunately, it is not feasible to overload another operator for `optional<>::and_then()`, as there is no other operator with the same precedence as `|`, namely 13 (C++ *Named Requirements: RangeAdaptorObject (since C++20)* - *Cppreference.Com* n.d.). Choosing an operator with a different precedence would result in an undesirable order of evaluation of subexpressions. Therefore, operators must be chosen from a category that contains more than one operator with the same precedence. Furthermore, the operator category must be left-associative, as this is the desired order for evaluating the function composition expression. Here, the operators `/` and `%` are selected and overloaded correspondingly. This allows the sequenced function calls to be rewritten in a more compact form, as shown in statements two and four of `main()`. However, it is not recommended to do this in serious code. This demonstration has two goals: first, to show that this is possible and how it can be done, and second, to remind the reader of the special properties of operators and operator overloading in C++ in case such a solution is explicitly desired.

After returning an empty `optional<>` exemplar, the following calls to `transform()` silently do not call the function passed to them as an argument. No attempt is made to correct the error situation, nor is any information provided about it. The member function `std::optional<>::or_else()` can change this. Listing 230 shows the two new functions `error()` and `truncate()`. Both are nullary functions, i.e., they do not have a parameter, and both return an `optional<ull_i_t>` exemplar.

Listing 230: functional/OptionalOrElse.cpp

```
1 // OptionalOrElse.cpp by Ulrich Eisenecker, February 12, 2026
2
3 #include <iostream>
4 #include <optional>
5
6 using namespace std;
7
8 using ulli_t = unsigned long long int;
9
10 [[nodiscard]] optional<ulli_t> check(ulli_t n)
11 {
12     if (n > 20ull)
13     {
14         return { };
15     }
16     return n;
17 }
18
19 [[nodiscard]] optional<ulli_t> error()
20 {
21     cerr << "some error happened" << endl;
22     return { };
23 }
24
25 [[nodiscard]] optional<ulli_t> truncate()
26 {
27     return 20ull;
28 }
29
30 [[nodiscard]] ulli_t plus1(ulli_t n)
31 {
32     return n + 1ull;
33 }
34
35 ulli_t factorial(ulli_t n)
36 {
37     return n ? n * factorial(n - 1) : 1;
38 }
39
40 [[nodiscard]] ulli_t output(ulli_t n)
41 {
42     cout << n << endl;
43     return n;
44 }
45
46 int main()
47 {
48     check(21ull).or_else(error).transform(factorial).transform(output);
49
50     check(5ull).transform(plus1).and_then(check)
51         .or_else(error)
```



```
52         .or_else(truncate)
53         .transform(output).transform(factorial)
54         .transform(output);
55
56     check(20u11).transform(output).transform(plus1)
57         .transform(output).and_then(check).transform(factorial)
58         .transform(output).or_else(truncate)
59         .transform(factorial).transform(output);
60 }
```

The `error()` function sends an error message to `cerr`, so it is not a pure function, and then returns an empty `optional<>` exemplar afterwards. This ensures that the error situation continues.

The `truncate()` function returns an `optional<>` exemplar with the value `20u11`. This is the largest unsigned integer for which the factorial can be correctly calculated. In this case the error situation is silently corrected and thus resolved.

The three statements in `main()` illustrate the possible uses of `or_else()`.

In the first statement, `check(21u11)` creates an empty `optional<>` exemplar. Calling `or_else()` for it with `error` as an argument outputs the error message to the standard error stream, which is normally tied to the terminal window. The subsequent calls to `factorial()` and `output()` via `transform()` are ignored because the `optional<>` exemplar is always empty.

The second statement illustrates that calls to `or_else()`, whether with `error` or `truncate` as arguments, have no effect if the `optional<>` exemplar is not empty. Although this is interesting in itself, in practice, code containing such scattered questionable monadic calls should not be written.

The third statement illustrates the use of `or_else(truncate)`. After this transformation, the error is no longer present, and the transformations calculate and output the factorial for `20u11`. It must be emphasized that the measure chosen here is not a magic bullet for fixing errors in functional programming. The sole intention is to illustrate the use of `std::optional<>.or_else()`.

5.4.6.2. The Expected Monad

The optional monad is a coarse-grained tool for error handling. An empty `optional<>` exemplar indicates that something went wrong, but does not provide any additional information about what exactly went wrong. The situation is different with `std::expected<>`. This class template is instantiated with two types. The first type represents values that are expected when no error occurs. The second type represents values that are returned when an error occurs. This can be a user-defined enumeration type with enumeration constants for specific error causes, or simply `std::string`, which conveys information for users, or another relevant type.

In Listing 231 `ulli_t_or_string` is defined as a type alias for `expected<ulli_t,string>`. It is used to specify the return type of the functions `check()` and `error()`.

Listing 231: functional/Expected.cpp

```

1 // Expected.cpp by Ulrich Eisenecker, February 12, 2026
2
3 #include <expected>
4 #include <iostream>
5 #include <string>
6
7 using namespace std;
8
9 using ulli_t = unsigned long long int;
10
11 using ulli_t_or_string = expected<ulli_t,string>;
12
13 [[nodiscard]] ulli_t_or_string check(ulli_t n)
14 {
15     if (n > 20ull)
16     {
17         return unexpected { "argument "s + to_string(n) + " too large."s };
18     }
19     return n;
20 }
21
22 [[nodiscard]] ulli_t plus1(ulli_t n)
23 {
24     return n + 1ull;
25 }
26
27 ulli_t factorial(ulli_t n)
28 {
29     return n ? n * factorial(n - 1) : 1;
30 }
31
32 [[nodiscard]] ulli_t output(ulli_t n)
33 {
34     cout << n << endl;
35     return n;
36 }
37
38 [[nodiscard]] ulli_t_or_string error(string s)
39 {
40     cerr << s << endl;
41     return unexpected { s };
42 }
43
44 int main()
45 {
46     check(20ull).transform(output).transform(plus1)
47         .transform(output).and_then(check).transform(factorial)
48         .transform(output)
49         .or_else(error);

```

50 | }

The `check()` function, which takes an `ulli_t` value as an argument, creates an exemplar of `std::unexpected<>` if `n > 20`. This exemplar is instantiated for `std::string` with a detailed error message, which is implicitly used to create the `ulli_t_or_string` exemplar returned by `check()`.

The `error()` function takes a string as an argument, which is sent to `cerr`. An `unexpected<>` exemplar is then instantiated for `string` and initialized with `s`, which is then used to create the `ulli_t_or_string` exemplar returned by `error()`. This continues the error situation and ensures that a chained call to `transform()` does not execute the function passed as an argument. If another error were to occur in a subsequent function call, this cannot happen because the function is not executed. This means that the preceding error must be corrected before a successive error can occur.

The statement in `main()` illustrates the use and effect of `expected<>`. The subexpression `check(20ull)` creates an `expected<>` exemplar. The following transformations output its value, add one to `20ull`, and output this new value again. Then, `and_then(check)` is executed for the current temporary `expected<>` exemplar. Since `21ull` is too large, the resulting `expected<>` exemplar contains an `unexpected<>` value. Consequently, all subsequent transformations do not call the functions passed as arguments. Therefore, the factorial of `21ull` is not calculated and no output is produced. Finally, `or_else(error)` is called for the current temporary `expected<>` exemplar, which sends an error message to `cerr`. Since `error()` returns a new `expected<>` exemplar which is initialized with the `unexpected<string>` `s`, the error situation continues.

As shown in Listing 230, another function could be defined instead of `error()` to correct the error. Individual measures could be taken depending on the specific `unexpected` value.

5.4.6.3. The Vector Monad

In Listing 232, the initials of a person's names are selected and combined into an acronym in an overly complex manner.

Listing 232: functional/Flatten.cpp

```
1 // Flatten.cpp by Ulrich Eisenecker, February 16, 2026
2
3 #include <print>
4 #include <ranges>
5 #include <string>
6 #include <vector>
7
8 using namespace std;
9
10 int main()
```

```

11 {
12     vector names { "Turi"s,"Uria"s,"Rubio"s,"Nipka"s,"Isa"s,"Peeves"s };
13     println("names: {}",names);
14     auto initials { names | views::transform([](const auto& n)
15                                           { return n | views::take(1);
16                                           }
17                                           )
18     };
19     println("initials: {}",initials);
20     auto flattened { initials | views::join };
21     println("flattened: {}",flattened);
22     auto acronym { flattened | ranges::to<string>() };
23     println("acronym: {}",acronym);
24 }

```

The string vector `names` is initialized with six C++-strings. Together, they form a person's full name. It should be emphasized that each element of `names` of type `string` is also a kind of vector, resulting in a nested data structure.

The next line uses the `std::println()` function from the C++ standard library to output `names` in the console window. The first argument of this overload of `println()` is a so-called *format string*. Each pair of curly brackets represents an output parameter. The function can accept as many parameters as specified in the format string. In addition, it can output not only values of elementary data types, but also values of ranges and tuples, even if they are nested. After completing the output, `println()` starts a new line. If this is not desired, `print()` should be used. To use these functions, the header file `<print>` must be included. Only one pair of curly brackets appears here. Accordingly, only the vector `names` is output.

In the following statement, `names` is transformed so that the first element, a `char`, is selected and returned from each of its elements. The result is used to initialize the `auto` variable `initials`. This is a vector whose elements are also vectors with exactly one `char` as an element, which is clearly illustrated by the output of `println()` in the next line. The vector is used here as a functor. Its transformation results in a doubly enclosed functor. In the next step, one level of the enclosure must be removed, i.e., it must be flattened.

This is achieved by applying the `std::views::join()` function to `initials`. It could be called explicitly as `std::ranges::join_view()`, but here too, pipe syntax is used to apply it to `initials`. The expression is used to initialize the `auto` variable `flattened`. As the `println()` statement shows, this operation has successfully removed one level of nesting. The resulting vector has only one dimension, and its elements are characters.

This is still not exactly what is desired, namely a `std::string`. Fortunately, the ranges library contains the conversion function `std::ranges::to<>()` with many overloads. Therefore, the `auto` variable `acronym` is initialized with the expression `flattened | ranges::to<string>()`, which concatenates the individual characters into a string that is output in the last statement.

Figure 59 shows the result of compiling and executing the program.

Figure 59: Output of functional/Flatten.cpp

```
i@compi functional % g++-15 -std=c++2c -Wall -pedantic -o flatten Flatten.cpp
i@compi functional % ./flatten
names: ["Turi", "Uria", "Rubio", "Nipka", "Isa", "Peeves"]
initials: [['T'], ['U'], ['R'], ['N'], ['I'], ['P']]
flattened: ['T', 'U', 'R', 'N', 'I', 'P']
acronym: TURNIP
i@compi functional %
```

`std::views::transform<>()` or `std::ranges::transform_view<>()` and `std::views::join<>()` or `std::ranges::join_view<>()` turn vectors (and some other containers) as well as ranges and views into monads. A `bind()` function that combines `join()` and `transform()` as in other functional programming languages is not currently part of the C++ standard library. The program in Listing 232 is called *Flatten.cpp*, because removing a nesting level in functional programming is usually referred to as *flattening*. As a reminder, in a sense, *flattening* is the complementary action to *lifting*. *Lifting* means moving a value or function up one level, while *flattening* means moving it down one level.

To illustrate the transformation steps, some intermediate states were bound to variables. All of them could be omitted, resulting in a single expression that only uses pipe syntax. The method presented is absurdly complex. There are much simpler ways to find an acronym in this case. It is left to the reader as an exercise to find a simpler and shorter solution that also uses functional programming techniques.

Monads are often used in functional programming with C++. They can also occur in procedural or object-oriented programs. Other monads in the C++ standard library are `std::future<>` and `std::promise<>`, information about which can be found on the Internet.

5.4.7. Algebraic Data Types

In accordance with the mathematical origins of functional programming, data types—like functions—are viewed from a mathematical perspective, namely as *algebraic data types*.

The first category is *sum types*. A sum type comprises mutually exclusive values. Common examples are *integers* with unique values such as ..., -1, 0, 1, 2, ... and *Booleans* with the values *False* and *True*. However, the values of a sum type can also be linked to different data. For example, the type *Sex* with the values *Male* and *Female*, where *numberOfPregnancies* is a piece of data that is exclusively linked to *Female*. If two or more types are combined into a sum type, its values correspond to the sum of all the values of the combined types. A sum type can be even simpler. Previously, the *optional monad*

was introduced, which either has a value or does not, and is therefore also a sum type. A sum type does not necessarily correspond to a basic data type in C++ or a class. Sum types whose values are not associated with different data can usually be represented relatively easily in C++ using enumeration types.

A product type combines two or more types. For example, two integer types can be combined into a *RationalNumber* type. Typically, the values that can be represented by a product type correspond to the product of all values of the combined types. Interestingly, *RationalNumber* shows that this is not always true, as *RationalNumber* values with a denominator of 0 are not valid.

In addition, a type can be *recursive*. An example of this is a list that can be empty or contain any number of elements.

5.4.7.1. Sum Type

In C++, the basic type `bool` represents the truth values `false` and `true` with literals of the same name. Historically, `bool` values were treated like integer values: the integer value 0 corresponded to `false`, and every other integer value corresponded to `true`. This meant that many integer values actually represented the Boolean value `true`. This still applies today for the automatic conversion from `int` to `bool`. The Boolean enumeration type is presented below as a separate definition for representing Boolean values (Listing 233). The program *BooleanEnumUT.cpp* contains unit tests for all entities defined in the header file. It is included in the *sources/functional* directory of the accompanying material.

Listing 233: functional/booleanenum.hpp

```

1 // booleanenum.hpp by Ulrich Eisenecker, February 25, 2026
2
3 #ifndef BOOLEANENUM_HPP
4 #define BOOLEANENUM_HPP
5
6 #include <iostream>
7
8 namespace booleanenum
9 {
10     enum class Boolean
11     {
12         False,
13         True
14     };
15
16     std::ostream& operator<<(std::ostream& os, const Boolean& b)
17     {
18         os << (b == Boolean::False ? "False" : "True");
19         return os;
20     }
21
22     constexpr Boolean operator!(const Boolean& b)
23     {

```

```
24     if (b == Boolean::False)
25     {
26         return Boolean::True;
27     }
28     return Boolean::False;
29 }
30
31 constexpr Boolean operator&&(const Boolean& p1,const Boolean& p2)
32 {
33     return (p1 == Boolean::True) && (p2 == Boolean::True) ?
34         Boolean::True : Boolean::False;
35 }
36
37 constexpr Boolean operator||(const Boolean& p1,const Boolean& p2)
38 {
39     return (p1 == Boolean::True) || (p2 == Boolean::True) ?
40         Boolean::True : Boolean::False;
41 }
42
43 } // namespace booleanenum
44
45 #endif // BOOLEANENUM_HPP
```

The enumeration is defined in a modern way using the additional keyword `class`. Therefore, the enumeration constants `False` and `True` must be prefixed with `Boolean::` when used. Since all enumeration types are represented internally as integers, relational operators such as `==`, `!=`, `<`, etc. are automatically available. This does not apply to the stream insertion operator or logical operators. Despite the use of the keyword `class`, an enumeration type is not a class. For this reason, all required operators must be defined as free functions, which is done for `<<`, `!`, `&&`, and `||`. Since all operators are declared with `constexpr`, they can be evaluated at compile time. All implementations of the operators use the operator `==`, which, as already mentioned, is automatically available.

Additional logical operators can be implemented in addition to the available operators. A stream extraction operator could also be implemented.

This self-defined `Boolean` type can be used almost interchangeably with `bool`—in some cases with little effort, which also involves some strange uses. For example, the following expressions are all valid: `true > false`, `Boolean::false < Boolean::true`. It seems reasonable to ask why `false` and `true` should be relationally comparable beyond equality or inequality. For the enumeration type, this is a consequence of the underlying integer representation, and for `bool` and the enumeration types, the argument of preserving backward compatibility of the code may apply. The use of arithmetic operators is not valid for enumeration types, i.e., `Boolean::True - Boolean::True` results in a compilation error. However, `true - true` is valid, either because of the automatic conversion from `bool` to `int`, i.e., coercion polymorphism, or because `bool` is an integer type and therefore supports some arithmetic operators (since C++17, `false++` and `true--` are no

longer supported). However, given the availability of the basic type `bool`, implementing the enumeration type `Boolean` is not worthwhile. It is only done here for didactic reasons.

Next, two attempts to eliminate these undesirable strange uses are presented. Both convert the values *false* and *true* into classes and make them full-fledged elements of the type system. This measure is called *reification*. It is to be distinguished from lifting, which means bringing something into a different context or wrapping it.

Listing 234 shows the implementation of the types `False` and `True` as subtypes of `Boolean` and additionally offers the advantages of inclusion polymorphism. Comprehensive unit tests are included in *BooleanClassUT.cpp*, which is available in the *sources/functional* directory of the corresponding directory.

Listing 234: functional/booleanclass.hpp

```

1 // booleanclass.hpp by Ulrich Eisenecker, March 4, 2026
2
3 #ifndef BOOLEANCLASS_HPP
4 #define BOOLEANCLASS_HPP
5
6 #include <concepts> // Because of std::is_derived_from<>
7 #include <iostream>
8 #include <typeinfo> // Because of typeid()
9 #include <type_traits> // Because of std::is_same_v<>
10
11 namespace booleanclass
12 {
13     class Boolean
14     {
15     public:
16         virtual ~Boolean() = default;
17         virtual void output(std::ostream& os) const = 0;
18     };
19
20     class False final : public Boolean
21     {
22     public:
23         void output(std::ostream& os) const override
24         {
25             os << "False";
26         }
27     };
28
29     class True final : public Boolean
30     {
31     public:
32         void output(std::ostream& os) const override
33         {
34             os << "True";
35         }
36     };

```



```
37     std::ostream& operator<<(std::ostream& os,const False& f)
38     {
39         f.output(os);
40         return os;
41     }
42
43
44     std::ostream& operator<<(std::ostream& os,const True& t)
45     {
46         t.output(os);
47         return os;
48     }
49
50     std::ostream& operator<<(std::ostream& os,const Boolean& b)
51     {
52         b.output(os);
53         return os;
54     }
55
56     constexpr const True operator!(const False&)
57     {
58         return True { };
59     }
60
61     constexpr const False operator!(const True&)
62     {
63         return False { };
64     }
65
66     const Boolean& operator!(const Boolean& b)
67     {
68         static const True t { };
69         static const False f { };
70         if (typeid(b) == typeid(True))
71         {
72             return f;
73         }
74         return t;
75     }
76
77     template <class T1,class T2>
78     constexpr const False operator==(const T1&,const T2&)
79         requires std::derived_from<T1,Boolean> &&
80                 std::derived_from<T2,Boolean> &&
81                 (!std::is_same_v<T1,Boolean>) &&
82                 (!std::is_same_v<T2,Boolean>)
83     {
84         return False { };
85     }
86
87     template <class T>
88     constexpr const True operator==(const T&,const T&)
89         requires std::derived_from<T,Boolean>
90     {
91         return True { };
```

```

92     }
93
94     const Boolean& operator==(const Boolean& p1,const Boolean& p2)
95     {
96         static const True t { };
97         static const False f { };
98         if (typeid(p1) == typeid(p2))
99             {
100                 return t;
101             }
102         return f;
103     }
104
105     constexpr const False operator&&(const False&,const Boolean&)
106     {
107         return False { };
108     }
109
110     constexpr const False operator&&(const True&,const False&)
111     {
112         return False { };
113     }
114
115     constexpr const True operator&&(const True&,const True&)
116     {
117         return True { };
118     }
119
120     const Boolean& operator&&(const Boolean& p1,const Boolean& p2)
121     {
122         static const True t { };
123         static const False f { };
124         if (typeid(p1) == typeid(True) && typeid(p2) == typeid(True))
125             {
126                 return t;
127             }
128         return f;
129     }
130
131     constexpr const False operator||(const False&,const False&)
132     {
133         return False { };
134     }
135
136     constexpr const True operator||(const False&,const True&)
137     {
138         return True { };
139     }
140
141     constexpr const True operator||(const True&,const False&)
142     {
143         return True { };
144     }
145
146     constexpr const True operator||(const True&,const True&)

```

```
147     {
148         return True { };
149     }
150
151     const Boolean& operator||(const Boolean& p1,const Boolean& p2)
152     {
153         static const True t { };
154         static const False f { };
155         if (typeid(p1) == typeid(True) || typeid(p2) == typeid(True))
156         {
157             return t;
158         }
159         return f;
160     }
161 } // namespace booleanclass
162
163 #endif // BOOLEANCLASS_HPP
```

Boolean is an abstract class because the member function `output()` is declared as a pure virtual function. Even if this member function did not exist, it would still be a polymorphic (base) class because it has a defaulted virtual destructor.

False is a final subclass of Boolean, which means that it cannot serve as a base class itself. It overrides the member function `output()` and thus implements it specifically for this class. True is structurally identical to False and therefore requires no further explanation.

None of these classes defines the operators `<<`, `!`, `==`, `&&`, or `||` as member functions. All of these operators are overloaded multiple times as free functions, some of them even as function templates. The reason for this is that these operators are needed both in static variants and in variants that correspond to inclusion polymorphism. In addition, all operators that can be evaluated at compile time are declared with `constexpr`.

The stream insertion operator is overloaded three times, namely for False as the second argument, for True as the second argument, and for Boolean as the second argument. In all cases, the second argument is passed as a reference to `const`. Thanks to the availability of overloads for False and True, the specific version of stream insertion is called when the static type of the second argument is known to the compiler. Even the call to the virtual function `output()` can be optimized away. If only the base type Boolean is known, the function `output()` is called via late binding, which is acceptable for a polymorphic type.

The operator `!` is also overloaded for the types False and True. In the case of False, negation returns a True exemplar, while negation of True returns a False exemplar. Please note that both results are returned as `const` values to ensure consistency with the following operator overloads, which return a Boolean value or a subclass thereof. Overloading the `!` operator for Boolean values is tricky. If `b` is a True exemplar, a False exemplar must be returned as the Boolean value, and vice versa. However, the compiler

prohibits returning values of different types. Furthermore, it is not possible to return a more specific type than the one specified as the return type. In fact, specialization of the return type for references or pointers is possible when overriding member functions, which is called *covariant specialization*. However, this possibility will not be explored further here. Instead, this overload of the `!` operator defines a static `const True` exemplar named `t` and a static `const False` exemplar named `f`. Depending on the result of comparing the application of the `typeid` operator to `b` and `True`, either `f` or `t` is returned. The static variables `t` and `f` are initialized once when the program starts and cannot be changed later because they are `const`. Therefore, they are consistent with the immutability of values in functional programming.

The static version of the comparison operator `==` is overloaded in the form of two templates. The first overload of `==` takes two operands, which can be of different types. However, it is required that both types inherit from `Boolean`. Furthermore, neither of these types may be `Boolean` itself. It always returns `False`. If this were the only overload, it would return `false` if both operands were of the same type. For this case, the overload of the `==` operator is relevant for only one template type parameter. It also takes two operands, but both must be of the same type. Furthermore, `T` must be a subtype of `Boolean`. This overload always returns `True`. It must be emphasized that neither of these two overloads specifies names for the operands and therefore cannot access them. This is also not necessary, as the compiler compares the types with the two different function template overloads and selects the more specific one. This largely corresponds to pattern matching in functional programming. The overload of the `==` operator as a free function that takes two `Boolean` arguments as references to `const` is called when the static type of the operands is not available to the compiler. In this case, the `std::type_info` objects of operands `a` and `b` are compared. If they are identical, `t` is returned, otherwise `f`. Here, too, two static `const` objects of type `True` and `False` are declared and used in exactly the same way as in the overload of the `!` operator for `Boolean` values.

The operator `&&` is overloaded four times. The first three overloads cover the cases where the result of evaluating the operator depends entirely on the available static types. The fourth overload covers the polymorphic case.

The first overload specifies references to `const` of the types `False` and `Boolean`. Since the left operand is `False`, the operator must evaluate to the result `False`. Therefore, `Boolean` is used as the right operand, which includes the two subtypes, `False` and `True`. The second overload takes `True` as the left operand and `False` as the right operand, which also evaluates to `False`. An attempt to define this second overload for `Boolean` and `False`—the inverse of the first overload, so to speak—would fail, as it would lead to an ambiguity with the first overload. The third overload takes `True` as both the left and right operands; this is the only case that returns `True`. The fourth overload of the operator `&&` covers the entirely polymorphic case when the true type is not available to the compiler. The structure corresponds to the overloads of the operators `!` and `==` for `Boolean`. Only if both `std::type_info` objects of `p1` and `p2` are equal to that of `True`, `t` is returned, otherwise `f`.

There are five overloads for the `||` operator. The first four overloads cover the static cases of each combination of `False` and `True`. Although each implementation is very simple, each combination is explicitly represented. It is left to the reader as an exercise to find an alternative solution that requires fewer overloads. The last overload of the `||` operator deals with the polymorphic case and is very similar to the corresponding implementation of the `&&` operator for `Boolean`. The only difference is the use of the `||` operator (the built-in logical OR operator) to combine the results of comparing the `std::type_info` objects involved.

The difference between static evaluation of operators and their dynamic evaluation is obvious. But that's not all! The operators `==`, `&&`, and `||` take two operands. Cases with a mixture of known and unknown true types should also be considered. To check these, *BooleanClassUT.cpp* contains a third test case called "MIXED". To be honest, it should be mentioned that after running this test case, an error was discovered regarding the `==` operator, which was fixed by adding the restrictions that neither the type of the first parameter (`T1`) nor the type of the second parameter (`T2`) may be identical to `Boolean`. The concept `std::derived_from<>` evaluates to `true` not only for a class and its base class, but also for a class and itself. A concept such as `is_true_subtype<>` is currently not available in the concepts library and could be defined by the user. It is left to the reader as an exercise to verify that all possible combinations of static and dynamic operands are covered in the test cases.

As shown, it is possible to implement a sum type using the object-oriented means of inheritance and late binding for both static and dynamic types. If at least one dynamic type is involved, the result is always a reference to `const` of type `Boolean`, i.e., the base type, since only `Boolean` can be either `False` or `True`. If only static types are involved, the return type is always either `False` or `True`. If one of the types `True` or `False` had a more specialized interface, i.e., additional member functions and possibly internal (`const`) data, that interface would be preserved in this case. In the case of returning a reference to `const` of type `Boolean`, the true type would have to be restored by applying a dynamic cast in order to access the more specialized interface. It should be noted that the trick used in the previous example always returns the same `True` or `False` exemplar that was created internally in the corresponding operators during initialization time. In summary, this option is feasible, but requires considerable effort and may still have some subtle shortcomings.

Next, `False` and `True` are defined as completely independent types. As a result, inclusion polymorphism is excluded. Listing 235 shows only the first few lines of the corresponding header file, as the definitions are quite simple, but also boring.

Listing 235: functional/booleanvariant.hpp (excerpt)

```

1 // booleanvariant.hpp by Ulrich Eisenecker, February 25, 2026
2
3 #ifndef BOOLEANVARIANT_HPP
4 #define BOOLEANVARIANT_HPP
5
6 #include <iostream>
7 #include <variant>
8
9 namespace booleanvariant
10 {
11     class False
12     {};
13
14     class True
15     {};
16
17     using Boolean = std::variant<False,True>;
18
19     std::ostream& operator<<(std::ostream& os,False)
20     {
21         os << "False";
22         return os;
23     }
24

```

False and True do not define any explicit member functions. Constructors, destructors, and possibly copy constructors are automatically generated as needed. The sum type is defined as `std::variant<False,True>` with `Boolean` as the type alias.

The `variant<>` class template can be instantiated with any number of types, of which an exemplar of an instantiated `variant<>` can contain exactly one exemplar. There are several (member) functions for accessing the currently contained object of a variant, including `visit()`, which accepts a (generic) function that is then applied to the contained object. This makes `std::variant<>` a functor.

The file *functional/BooleanVariantUT.cpp* contains comprehensive unit tests for False, True, and Boolean. Listing 236 shows an excerpt from the test case "BOOLEAN", which illustrates the use of some selected (member) functions of `std::variant<>`. A comprehensive overview can be found at (*std::variant* - *Cppreference.Com* n.d.).

Listing 236: functional/BooleanVariantUT.cpp (excerpt)

```
125 Boolean isOdd(unsigned long long int i)
126 {
127     if (i % 2)
128     {
129         return True { };
130     }
131     return False { };
132 }
133
134 TEST_CASE("BOOLEAN")
135 {
136     ostringstream os;
137     auto visitor { [&os](auto v) { os << v; } };
138
139     SECTION("FALSE")
140     {
141         auto result { isOdd(2) };
142         REQUIRE(get_if<False>(&result));
143         REQUIRE(is_same_v<False,variant_alternative_t<0,decltype(result)>>);
144         REQUIRE(result.index() == 0);
145         result.visit(visitor);
146         REQUIRE(os.str() == "False");
147     }
```

The return type of the `isOdd()` function is specified as `Boolean`. Returning `True { }` for odd values results in a new `Boolean` exemplar being created and initialized with a value of type `True`. Otherwise, the `Boolean` exemplar is initialized with a value of type `False`.

Moving on to the "BOOLEAN" test case. After declaring the `ostringstream` exemplar `os`, the `auto` variable `visitor` is initialized with a generic lambda expression that specifies `os` as a reference in its capture clause. Since *Catch2* v3 resets `os` each time before executing a new section, `os` contains only the expected content in each section. Since `v` is passed as an `auto` variable, it can be called with any concrete type. In the body of the lambda, `v` is sent to `os`, which requires that the stream insertion operator for the type `os` is defined with `v` as the second argument.

In the "FALSE" section, the value returned by calling `isOdd(2)` is used to initialize the `auto` variable `result`. The following `REQUIRE` assertion macro instantiates and calls the free function template `std::get_if<>()`. A type is specified in the angle brackets, and a pointer to a `variant<>` exemplar is passed as a function argument in the parentheses. If the object currently held by the `variant<>` exemplar is of the specified type, a pointer to that exemplar is returned. Otherwise, `nullptr` is returned. The reason why a pointer to a `variant<>` exemplar must be passed as a function argument is either mysterious or subtle, or both. Since `False` is specified as the type of the `Boolean` variant, the returned pointer is not null and points to the corresponding `False` exemplar.

There is also the function template `std::get<>()`, which is not shown here. It, too, takes the inferred type as an argument in angle brackets, but accepts a reference to a `variant<>` exemplar and returns a reference to the object managed by the `variant<>` exemplar. If it does not exist, `std::get<>()` throws a `std::bad_variant_access` exception.

In the next `REQUIRE` statement, the template variable `std::variant_alternative_t<>` is used to retrieve the type specified as the second parameter in the Boolean variant. The index must be specified as the first template parameter and starts at 0 for the first type with which `variant<>` is instantiated. Internally, the template variable is initialized with the result of the corresponding template metafunction `std::variant_alternative_t<>::type`. For simplicity, Boolean could have been specified directly as the second template parameter. Instead, the `decltype` operator is applied to `result` to determine its type. This is done to remind one of this possibility. Next, it must be checked whether the resulting type is `False`. This is done by applying the template variable `is_same_v<>`, which is true if both types specified as template arguments are identical, and false otherwise.

The next `REQUIRE` statement calls the member function `index()`. It checks the type of the object currently held by the `variant<>` exemplar and returns its index in the order of the types specified when instantiating `variant<>`. The index values start at 0. Since the Boolean exemplar contains a `False` object, the value returned by calling `result.index()` should be 0.

Next, the member function template `visit<>()` is called with `visitor` as an argument. `visitor` refers to the lambda template and is automatically instantiated internally for the type of the object currently held by `variant<>`. The specified implementation of the lambda simply sends the object to `os`, which is captured as a reference, using the stream insertion operator. The last `REQUIRE` assertion macro checks whether the content of `os` is equal to `"False"`.

Although the interface of `std::variant<>`, i.e., its member functions and associated free functions, including function templates, may seem complex at first glance, it is the standard method for representing sum types in functional programming with C++. There are other (member) functions that are not presented here, including the ability to assign a new value to a `variant<>` exemplar, though this should not be necessary in functional programming. Interested readers are again referred to (*std::variant - Cppreference.Com* n.d.) for more comprehensive and detailed information.

As a final example, the program *objects/Butterfly.cpp* (Listing 78) is converted into a functional variant—at least almost—which is shown in Listing 237.

Listing 237: functional/Butterfly.cpp

```
1 // Butterfly.cpp by Ulrich Eisenecker, February 26, 2026
2
3 #include <iostream>
4 #include <string>
5 #include <variant>
6
7 using namespace std;
8
9 using ulli_t = unsigned long long int;
10
11 class Id
12 {
13     public:
14         Id():m_id { ++s_id }
15         { };
16         string id() const
17         {
18             return to_string(m_id);
19         }
20     protected:
21         ulli_t m_id;
22         inline static ulli_t s_id { 0 };
23 };
24
25 class Egg;
26 class Caterpillar;
27 class Chrysalis;
28 class Imago;
29
30 using Butterfly = variant<Egg,Caterpillar,Chrysalis,Imago>;
31
32 class Egg
33 {
34     public:
35         Caterpillar hatch() const;
36         string id() const
37         {
38             return m_id.id() + " Egg ";
39         }
40     private:
41         Id m_id;
42 };
43
44 class Caterpillar
45 {
46     public:
47         Caterpillar(Id id):m_id { id }
48         { };
49         Chrysalis pupate() const;
50         string id() const
51         {
```

```
52         return m_id.id() + " Caterpillar ";
53     }
54     private:
55         Id m_id;
56 };
57
58 class Chrysalis
59 {
60     public:
61         Chrysalis(Id id):m_id { id }
62         { };
63         Imago emerge() const;
64         string id() const
65         {
66             return m_id.id() + " Chrysalis ";
67         }
68     private:
69         Id m_id;
70 };
71
72 class Imago
73 {
74     public:
75         Imago(Id id):m_id { id }
76         { };
77         Egg layEgg() const;
78         string id() const
79         {
80             return m_id.id() + " Imago ";
81         }
82     private:
83         Id m_id;
84 };
85
86 Caterpillar Egg::hatch() const
87 {
88     return m_id;
89 }
90
91 Chrysalis Caterpillar::pupate() const
92 {
93     return m_id;
94 }
95
96 Imago Chrysalis::emerge() const
97 {
98     return m_id;
99 }
100
101 Egg Imago::layEgg() const
102 {
103     return Egg { };
104 }
105
106 Butterfly nextStage(Butterfly b)
```

```
107 {
108     switch (b.index())
109     {
110         case 0: return get<0>(b).hatch();
111         case 1: return get<1>(b).pupate();
112         case 2: return get<2>(b).emerge();
113         case 3: return get<3>(b).layEgg();
114     }
115     return b;
116 }
117
118 int main()
119 {
120     Butterfly b { Egg { } };
121     ulli_t n { 10 };
122     while (n--)
123     {
124         b.visit([](auto v) { cout << v.id() << endl; });
125         b = nextStage(b);
126     }
127 }
```

Id is a helper class that generates and manages unique IDs. A unique ID is required to identify an object that may exist as an egg, caterpillar, chrysalis, or imago, as there is no enclosing object whose address in memory serves as a substitute for a unique ID. Whenever a new exemplar of Id is created with its default constructor, the static variable `s_id` is incremented and used to initialize the data member `m_id`. When an Id exemplar is copied, the `m_id` member is also copied. Therefore, such a copy should not be considered a unique exemplar, but merely a copy. The element function `id()` returns the value of `m_id` converted to a C++ string.

Next, there are pending declarations for all stages a butterfly can go through, i.e. Egg, Caterpillar, Chrysalis and Imago. Then the type alias `Butterfly` is defined for `variant<Egg, Caterpillar, Chrysalis, Imago>`.

The class `Egg` has a member function `hatch()`, which returns the next stage of development of a butterfly, namely a `Caterpillar`. Since `Caterpillar` is currently an incomplete type, the member function is implemented after the `Caterpillar` class. There is also a member function `id()`. It returns `m_id.id()` with the suffix " Egg ". The class has only one data member, namely `m_id` of type `Id`. It should be noted that `Egg` does not define a standard constructor. Therefore, an exemplar of `Egg` is created by calling the automatically generated standard constructor, which calls the standard constructor of all data members of the class. Since the standard constructor of `Id` requests a new ID, an `Egg` exemplar is always created with a new ID.

The constructor of the `Caterpillar` class requires an `Id` argument named `id`. In its initializer list, `m_id` is initialized with `id`. This way, the unique ID of the `Egg` exemplar is preserved. The `pupate()` member function returns a new `Chrysalis` exemplar and is defined after the `Chrysalis` class. The rest of `Caterpillar` is identical to that of `Egg` and not further explained.

The `Chrysalis` class is structurally identical to `Caterpillar`. Its member function `emerge()` returns a new `Imago` exemplar.

The same applies to the `Imago` class. Its member function `layEgg()` differs slightly from `Egg::hatch()`, `Caterpillar::pupate()` and `Chrysalis::emerge()`, whose implementations return `m_id`, thereby initializing the returned exemplars of `Caterpillar`, `Chrysalis` and `Imago`. This preserves the identity of the returned exemplars. However, `Imago::layEgg()` returns a newly created `Egg` exemplar. This results in a butterfly stage with a new identity.

The `nextStage()` function takes a `Butterfly` exemplar as an argument and returns a new `Butterfly` exemplar containing the next stage of a butterfly. In its implementation, it executes a `switch` statement for the result of `b.index()`. The member function `variant::index()` returns an integer that identifies the currently managed exemplar, starting with 0 for the first specified type. Now, `get<>()` can be used to safely access the corresponding exemplar. Therefore, no `std::bad_variant_access` exception should be thrown. In each branch, the returned object is returned despite its different type in order to initialize the `Butterfly` exemplar returned by the function. The `nextStage()` function always returns a `Butterfly` exemplar, but the `Butterfly` exemplar can contain objects of different types. In this way, a form of polymorphism is realized. Unlike inclusion polymorphism, the types managed by `std::variant<>` do not need to have a common base type, and no late binding and overriding is required. Although the two programs are not completely identical in terms of functionality, the implementation of *functional/Butterfly.cpp* appears to be less complex than that of *objects/Butterfly.cpp*.

5.4.7.2. Product Type

A type defines the possible values that an expression, such as a variable, can take. Mathematically, a type corresponds to a *set*, and the number of its possible values is the *cardinality* of that set. For example, the *Boolean* type has two elements, namely *false* and *true*, and the *UnsignedInteger8Bit* type has 256 values, namely the numbers 0 to 255.

The combination of two (or more) types results in a product type with a cardinality that is usually the product of the cardinalities of the individual types. However, this does not generally apply, as some combinations may be invalid, for example when two integers are combined to represent the type *RationalNumber*, where the denominator must not be 0. In this example, the problem could be solved by defining the type `Natural` with a lower limit of 1. The product type consisting of *Integer* and *Natural* cannot represent invalid rational numbers.

The usual method of combining types in procedural or object-oriented programming languages is to use structures and classes, but without inheritance. Functional programming supplements these with *pairs*, *triples*, and *tuples*. The C++ standard library provides the struct template `std::pair<>`, which can be instantiated for two (even different) types, and its generalization, the class template `std::tuple<>`, which can be instantiated for (almost) any number of types. Therefore, it can also serve as a pair, triple, or whatever. Both templates have already been used several times in this book. For this reason, Listing 238 only presents selected functions and member functions that use `std::tuple<>`.

Listing 238: functional/Tuple.cpp

```
1 // Tuple.cpp by Ulrich Eisenecker, February 27, 2026
2
3 #include <print>
4 #include <string>
5 #include <sstream>
6 #include <tuple>
7 #include <vector>
8
9 using namespace std;
10
11 struct Person
12 {
13     string name;
14     unsigned short int age;
15 };
16
17 bool operator<(const Person& p1,const Person& p2)
18 {
19     return tie(p1.name,p1.age) < tie(p2.name,p2.age);
20 }
21
22 struct ExtendedPerson
23 {
24     string name;
25     unsigned short int age;
26     double weight;
27     vector<string> hobbies;
28 };
29
30 int main()
31 {
32     Person p1 { "Pascale"s,90 };
33     // println("{} ",p); // not possible
34     auto t1 { make_tuple(p1.name,p1.age) };
35     println("{} ",t1);
36
37     Person p2 { "Torre"s,14 };
38     println("{}\n", {}) < ("{}\n", {}) -> {},
39         p1.name,p1.age,p2.name,p2.age,p1 < p2);
40     using PersonT = decltype(t1);
```

```

41     PersonT t2 { p2.name,p2.age };
42     println("{} > {} -> {}",t1,t2,t1 > t2);
43
44     ExtendedPerson p3 { "Moonie"s,20,65.5,{ "Surfing"s,"Reading"s,"Math"s } };
45
46     PersonT tPart { "Joao"s,24 };
47     auto attributes { make_tuple(64.25,vector { "Tennis"s,"Magic"s }) };
48     auto t3 { tuple_cat(tPart,attributes) };
49     println("{} ",t3);
50
51     auto p4 { make_from_tuple<ExtendedPerson>(t3) };
52     println("{} {} {} {} ",p4.name,p4.age,p4.weight,p4.hobbies);
53
54     apply([](auto e1:auto e2)
55           { println("{} {} ",e1,e2); },t2);
56     apply([](auto e1:auto e2:auto e3:auto e4)
57           { println("{} {} {} {} ",e1,e2,e3,e4); },t3);
58     apply([]<typename ... E>(const E& ... e)
59           { ((println("{} ",e)), ...); },t3);
60     ostream os;
61     apply([&os]<typename ... E>(const E& ... e)
62           { ((print(os,"{} ",e)), ...); },t3);
63     println("{} ",os.str());
64 }

```

The struct `Person` is what used to be called a *plain old datatype* (C++ *Named Requirements: RandomNumberDistribution (since C++11)* - *Cppreference.Com* n.d.), because it has no custom constructor, no custom destructor, does not use inheritance, and has no virtual functions. As such, it is compatible with types in the C programming language and can be exchanged with C libraries in its binary form. Another consequence of this is that an exemplar of `Person` can be list initialized, as demonstrated in the first line of `main()`. Even an exemplar of `ExtendedPerson`, which contains a data member of type `vector<string>` can be initialized in this way, as shown a few statements later with `p3`.

Although `Person` is a struct with only public data elements, it cannot be printed using `std::println<>()` because no information about its structure is available. Removing the comment from the second statement in `main()` results in a corresponding compilation error. The data elements of `Person` can only be output piece by piece using `println()`—as shown a few statements later for `p1` and `p2`—or using the stream insertion operator `<<`.

It is also not possible to compare two `Person` exemplars lexicographically, as the necessary relational operators cannot be generated automatically. The overloading of the `<` operator for two `Person` operands shows a potentially surprising implementation. It uses the `std::tie<>()` function template to convert both `Person` exemplars into `tuple<>` exemplars, which are then compared for their smallness. This possibility was already briefly introduced in the Section *Another Look at std::multiset<>*.

This is different for tuples. The auto variable `t1` in `main()` is initialized with a call to `std::make_tuple<>()`, which creates a `tuple<>` exemplar with `p1.name` and `p1.age` as elements. Since the structure of a `tuple<>` type is always the same, the template function `println<>()` can output any `tuple<>` exemplar as long as its elements meet the requirements for formatted printing, which are not covered here. The tuple exemplar `t2` is defined as `PersonT`, an alias for `decltype(t1)`, to illustrate this possibility. `t2` is initialized with the data elements of `p2`. Then, a comparison is made to see if `t1` is less than `t2`, and `t1`, `t2`, and the result of the comparison are output with `println<>()`. The relational operators for tuples of the same type are available for free!

`ExtendedPerson` is defined completely independently, without inheriting from `Person`. This is inconvenient in that the data elements `name` and `age` are specified redundantly. On the positive side, however, there is absolutely no coupling between the two structures.

Tuples offer an interesting alternative to inheritance, namely *tuple concatenation*. `tPart` is an exemplar of `PersonT`, which corresponds to the `Person` struct. The auto variable `attributes` is then initialized with the result of calling `std::make_tuple<>()` with a double and a `vector<string>` as arguments. The auto variable `t3` is initialized with `tuple_cat(tPart, attributes)`. `std::tuple_cat<>()` combines the tuples specified as arguments into a new tuple, which it returns as the result. Therefore, `t3` corresponds to an exemplar of `ExtendedPerson`.

A tuple cannot be created automatically from a struct or class. However, there is support for creating a struct or class exemplar from a `tuple<>` using `std::make_from_tuple<>()`. This is demonstrated in `main()` by creating an `ExtendedPerson` exemplar from the `t3` tuple. The function template initializes the data elements of the struct or class if no user-defined constructor is available, or otherwise calls the constructor whose arguments match the elements of the tuple.

The last part in the `main()` function demonstrates various uses of the `std::apply<>()` function template. The arguments of `apply<>()` are a function pointer and a `tuple<>` exemplar. The first statement with `apply<>()` defines a lambda template that accepts two auto parameters. Alternatively, the arguments could be defined using the element types of `PersonT`, namely `std::string` and `int`. Since exactly two parameters are specified, this lambda can be only called for tuples with two elements, e.g. `t2`. The next statement with `apply<>()` specifies a lambda template with four parameters. Consequently, the second parameter must be a tuple with four elements. This use of `std::apply<>()` is relevant if each of the elements passed as arguments is to be treated differently.

The last two uses of `std::apply<>()` show how to pass tuples of arbitrary size and process their elements uniformly. This is achieved by using a parameter pack and a fold expression, as already shown in Section *Descriptive Statistics as an Application Example* (Listing 189). In both lambdas, `E` is defined as the name of the pack, and `e` is passed as an argument to the lambda. In the first case, each `e` is output separately on a new line using `println<>()`. As can be seen in the terminal window, the `vector<string>` data

member is correctly treated as a single element. The second use case shows how the printed representations of each element can be combined into a single line. To achieve this, a reference to the `ostream` exemplar `os` is passed as a capture. In the lambda, `os` is specified as the first argument in the call to `print<>()` followed by the format string and `e`. In both cases, a left fold expression is specified. After completing the second call to `std::apply<>()` with a generic lambda that takes a parameter pack, the contents of `os` are retrieved by the member function `str()` and output via `println<>()`. Of course, this is not functional programming and was shown out of curiosity.

The element-wise access to tuple elements demonstrated above using `std::apply<>()` can be replaced by using only core language features starting with C++26 (see (C++ Templates 2025)). The corresponding code, which uses structured bindings with parameter packs, can be compiled with `clang++ 22.1.0` and is available via the reference provided before.

In summary, tuples are a powerful means of representing product types and are not limited to fundamental types, since the type of a tuple element can also be a class or function type. Compared to abstract data types, there is a clear disadvantage in that there is no name to identify a tuple element. The only way to access an element of a tuple is to use the free function template `std::get<>()`, which accepts an integer index as a template parameter and a tuple as an argument. Encapsulation and information hiding, which are core features of abstract data types, can be replaced—at least partially—by functions and techniques such as currying.

There are non-standard solutions for named types that allow access to a tuple element with a string literal, e.g. (Jansen 2025). It can be assumed that with the growing interest in functional programming with C++, named types will become part of the C++ standard library.

5.4.7.3. Recursive Type

The definition of a recursive type includes at least two constructors. The first constructor creates an empty exemplar. It usually has no arguments, for example `List() → emptyList`. The second constructor accepts an exemplar of the type plus a second argument and creates a non-empty exemplar that contains the second argument, for example `List(list, element) → listWithElement`.

Viewed abstractly, an *emptyList* can be considered a product type of *List* and *Nothing*, and *listWithElement* can be considered a product type of nested lists. Consequently, a recursive type can be considered a sum type of product types. Therefore, **an algebraic data type is a potentially recursive sum type of product types** (Algebraic Data Type 2026).

The classic example of a recursive type in functional programming is the *list*. An exemplar of a recursive type is immutable, like any other exemplar in functional programming. Consequently, appending an element to a list or combining two lists always results

in a new list. This differs from the generic containers in the C++ standard library, whose exemplars are stateful, as inserting or removing an element changes them. Since the C++ standard library does not currently contain immutable recursive types, no code examples for their use are presented, as this would require the additional definition of such immutable recursive types. There are non-standard libraries that provide these, e.g., (*Contents - Immer 0.0.0 Documentation* n.d.), which interested readers can study.

If a recursive data type is required in a functional program, it is advisable to select a suitable generic container from the C++ standard library and to deliberately use functions that violate immutability.

5.4.8. Lazy Evaluation

As shown in Listing 239, initialization statements in C++ are evaluated eagerly in a strict order.

Listing 239: functional/StrictOrder.cpp

```
1 // StrictOrder.cpp by Ulrich Eisenecker, March 11, 2026
2
3 #include <cmath>
4 #include <iostream>
5
6 using namespace std;
7
8 int main()
9 {
10     auto a { (cout << "a", sqrt(4.0)) };
11     auto b { (cout << "b", 2 / 4.0) };
12     auto c { (cout << "c", 0) };
13     cout << (cout << "d", a + b) * c << endl;
14 }
```

To illustrate the initialization of a variable, a trick is used. The expressions in the initializer lists are enclosed in parentheses and separated by the *sequence operator*, `,`. The sequence operator is left-associative, and the result of its evaluation corresponds to the result of the sub-expression on the far right. Here, the operand before the comma is an expression that inserts a character into the standard output stream. The stream exemplar is then returned but discarded, since only the second operand to the right of the comma is evaluated, returned, and used to initialize the corresponding variable.

When the compiled program is run, `abcd0` is displayed in the terminal window. This reflects the fact that each statement in the sequence (one of the mandatory control structures for Turing completeness) is executed exactly in the order in which it appears in the sequence.

For the sake of completeness, it should be noted that there are a few cases in which an expression is not evaluated at all, for example, when it is used as an operand of the `decltype` or `sizeof` operator. The statement `auto bytes { sizeof(cout << 5 + 3 << e_`

`endl) };` produces no output in the terminal window. The only effect is that `bytes` is initialized to the number of bytes occupied by an exemplar of an output text stream on the function call stack.

In contrast, functional programming ideally follows the principle of *lazy evaluation*. ***Lazy evaluation means that an expression is evaluated only when its value is needed.*** Some functional programming languages apply lazy evaluation by default, others offer it as an option, and a few do not support it at all. The program in Listing 239 illustrates some of the consequences of this.

The variables `a`, `b`, and `c` are not used before the last statement in `main()`. Therefore, their creation and initialization could be deferred until that point. This would also apply to the expressions used for initialization. Furthermore, the order in which `a`, `b`, and `c` are created and initialized is irrelevant. It could be any permutation, e.g., `a`, `c`, `b`, without changing the semantics of the programs. This essentially allows for the parallel execution of these declaration statements.

Another point to note is that—due to referential transparency—all occurrences of `a`, `b`, and `c` in the fourth statement of `main()` can be replaced by the expressions used to initialize them (Listing 240).

Listing 240: functional/ReferentialTransparency.cpp

```

1 // ReferentialTransparency.cpp by Ulrich Eisenecker, March 12, 2026
2
3 #include <cmath>
4 #include <iostream>
5
6 using namespace std;
7
8 int main()
9 {
10     cout << (cout << "d",
11              (cout << "a", sqrt(4.0)) +
12              (cout << "b", 2 / 4.0)) *
13             (cout << "c", 0) << endl;
14 }
```

The parentheses containing expressions of the sequence operator have been retained, so the output is still *bacd0*.

An interesting detail emerges after almost all sub-expressions have been evaluated and the output expressions have been removed, namely `cout << (2.0 + 0.5) * 0 << endl;`.

According to the rules of arithmetic, the product of any number and zero must be 0. Therefore, the entire arithmetic expression can be replaced with 0, which can already be done at compile time here, since the fact that one of the operands in the multiplication is zero is already known at compile time.

In eager evaluation, all calculations are performed even though they are redundant. Lazy evaluation allows these to be skipped entirely if the compiler detects that they are redundant. This results in a massive performance gain even compared to a parallelized execution, since performing unnecessary calculations also constitutes a waste of resources.

To be honest, it should be noted that once the sub-expressions that output characters to the console window are omitted, a C++ compiler can generally recognize that the relevant calculations are unnecessary and can therefore omit them—at least when an aggressive optimization level is used.

The following sections describe approaches for implementing lazy evaluation in C++ using the C++ standard library.

5.4.8.1. From Containers to Ranges to Views

Containers and ranges are explained in detail in the Chapter *Containers, Iterators and Algorithms*. Roughly speaking, ***a (generic) container internally manages elements in a specific structure that occupies memory***. There are member functions and free functions for accessing, adding, removing, and modifying the elements. Most functions that operate on the elements of a container require the use of a pair of iterators. Special iterators are the begin iterator, which points to the first element, and the end iterator, which points to the position of the element immediately to the right of the last element. Both are of the same type.

A range is an abstraction of a container that provides a uniform interface for using different containers. An iterator pair is no longer required to access elements. Furthermore, the end of a range can also be marked by a so-called *sentinel*. The resulting flexibility is fully utilized in the Section *Descriptive Statistics—A Case Study* and its subsections. Algorithms for containers and ranges are used extensively in this context.

The situation is different with views, which are also known as range adapters: they are *lazy*. ***This means that algorithms are applied to views on a delayed basis, and it even means that views are, in a sense, created on a delayed basis, as explained below.***

Views are a relatively new addition to the C++ standard library. Their underlying concepts are closely related to ranges and containers, which has led to intense discussions among language developers and ongoing changes during the C++ standardization process. For this reason, the description above is highly simplified and kept rather general. Interested readers are encouraged to search the Internet for current and authoritative sources that deal with ranges and views in depth.

Some of the relationships between containers, ranges, and areas are shown in Listing 241.

Listing 241: functional/CRV.cpp

```

1 // CRV.cpp by Ulrich Eisenecker, March 16, 2026
2
3 #include <algorithm>
4 #include <ranges>
5 #include <print>
6 #include <vector>
7
8 using namespace std;
9
10 int main()
11 {
12     vector c { 1,2,3 };
13     println("container: {}",c);
14
15     transform(begin(c),end(c),begin(c),[](auto e) { return e * e; });
16     println("container + transform: {}",c);
17
18     ranges::range auto& r { c };
19     ranges::transform(r,begin(r),[](auto e) { return e + 1; });
20     println("range + transform: {}",r);
21
22     auto v1 { views::transform(c,[](auto e) { return -e; }) };
23     println("view(c) + transform: {}",v1);
24
25     auto v2 { views::transform(r,[](auto e) { return e / 2; }) };
26     println("view(r) + transform: {}",v2);
27
28     auto v3 { views::transform(r,[](auto e)
29                                     { print("{} ",e);return 10 * e; }) };
30     println("before outputting v3");
31     println("views are lazy: {}",v3);
32
33     println("container: {}",c);
34 }

```

First, a container—in this case, a `std::vector<>` named `c`—is created and initialized. The subsequent output shows that it contains exactly the desired elements, namely 1, 2, 3.

Next, the `std::transform<>()` algorithm is applied to `c`. To do this, a pair of iterators must first be specified—in this case, `begin(v)` and `end(v)`. Then, an iterator is specified to indicate where the transformed values should be written—in this case, `begin(v)`. This means that the elements of `v` are overwritten starting with the first element, which is safe only in non-parallel execution. Next, a lambda is passed that takes an element and returns its squared value. The following output statement prints all elements of `v`, namely 1, 4, 9.

Next, a reference to a range named `r` is declared. To do this, the `ranges::range` concept is used in combination with `auto` and `&`. The reference `r` is initialized with `v`. This is an important detail, as otherwise a new range exemplar with its own elements would be created. Next, the `ranges::transform<>()` algorithm is applied. Now, instead of a pair of iterators, the range reference `r` is passed. However, the second argument must specify the location where the transformed elements are to be stored, in this case `begin(r)`. If `r` were a range exemplar rather than a reference, this would result in only `r` being modified, not `v`. Here, `begin(r)` is identical to `begin(v)`. The third argument is a lambda that returns the passed argument plus 1. The next statement outputs the elements of `r`, namely 2, 5, 10. `c` could also be used instead of `r`. This group of statements underscores that ranges are abstractions of containers that serve as interfaces for their use.

Next, `views::transform<>()` is called with `c` and a lambda expression as arguments. The lambda expression returns `-e`. No other parameters are required. The call to `views::transform<>()` returns a view exemplar, which is used to initialize the variable `v1`. The resulting view exemplar `v1` records the operation to be performed on its first argument, but does not execute it! This explains why no destination needs to be specified where the transformed elements are to be stored. The evaluation—that is, the execution of the operation—takes place when an element is accessed. Here, this happens indirectly when `println<>()` is called. This outputs the values -2, -5, -10. Note that these are new values, since the values in the container `r` are not modified. This is neither a disadvantage nor a problem, as it aligns with the principle of functional programming that functions return new values and do not alter the state of variables.

The next group of statements is very similar. Instead of `c`, it passes `r` as the first argument to `views::transform<>()`. The implementation of the lambda expression is also different. It returns the argument `e` integrally divided by 2. The result of the call to `views::transform<>()` is used to initialize the auto variable `v2`. The next statement prints 1, 2, 5, which is clear evidence that the elements of the original container `c` were transformed, not the elements generated by `v1`.

Next, the auto variable `v3` is initialized by a call to `views::transform<>()`. Here, the lambda itself generates output. It first prints its argument `e` before returning `e` multiplied by 10. However, the execution of this statement does not produce any output, which is a clear indication of the delayed evaluation of views. The following statement generates its output, and only the execution of `println<>()` with `v3` as an argument outputs the elements 2 5 10 (commas are intentionally omitted) before the transformed elements are printed.

Finally, `c` is printed again, yielding the values 2, 5, and 10, demonstrating that the original container `c` was not modified by the view transformations.

Views are an ideal tool for functional programming in C++, as they enable lazy evaluation and immutability (by design) in a standardized way.

5.4.8.2. A View as Functor and Monad

The Section *The Vector Monad* explains that a vector—like some other containers—is both a functor and a monad. Since ranges are abstract interfaces for containers and views are adapters for containers, ranges, and views, they too are functors and monads. The following briefly highlights the functions required to become a functor and a monad. The function template `std::views::transform<>()` converts a view into a functor. The function template `std::ranges::for_each<>()` allows access to every single element of a view, and `std::ranges::join_view<>()` and `std::views::join` handle the flattening. These are the functions required for monads. `std::ranges::for_each<>()` is demonstrated in Listing 242.

Listing 242: functional/ViewMonad.cpp

```

1 // ViewMonad.cpp by Ulrich Eisenecker, March 17, 2026
2
3 #include <algorithm>
4 #include <ranges>
5 #include <print>
6 #include <string>
7 #include <vector>
8
9 std::string asString(const std::ranges::range auto& v)
10 {
11     std::string s { };
12     std::ranges::for_each(std::views::transform(v, [](auto e)
13                                     { return std::to_string(e); })),
14                           [&s](auto e) { s += e; } );
15     return s;
16 }
17
18 using namespace std;
19
20 int main()
21 {
22     vector many { 2,3,5 };
23     println("many {}", asString(many));
24
25     auto one { views::single(22.0 / 7) };
26     println("one {}", asString(one));
27
28     auto none { views::empty<bool> };
29     println("none {}", asString(none));
30 }
```

The `asString<>()` function template takes a reference to `const` that meets the requirements of the `std::views::view` concept. It is referred to as `v`. The local `std::string` variable `s` is zero-initialized. Next, the `std::ranges::for_each<>()` function template is automatically instantiated and called. Its first parameter is the result of calling `std::views::transform<>()` with `v` and a lambda that returns a string representation of

each element of `v`. The second parameter is also a lambda that captures `s` by reference, i.e. it can modify `s`. Its implementation appends each element to the `std::string s`. The last statement of `asString<>()` returns `s`. The ongoing modification of the local string variable `s` by the lambda passed as second parameter to `std::ranges::for_each<>()` does not comply with the immutability principle. But since it is well hidden and encapsulated in `asString<>()` it can be tolerated.

In the `main()` function, the vector `many` is created and initialized with the values 2, 3, 5. Then `println<>()` is called, passing the result of the `asString(many)` call as the second parameter. As previously shown, a container or a range can be adapted using a view, since a view is essentially a range adaptor.

The second group of statements is interesting. It initializes the auto variable `one` with the result of calling `views::single(22.0 / 7)`. This returns a view whose sole element is the value resulting from the evaluation of `22.0 / 7`. `one` is passed as the second parameter to `println<>()`, which prints exactly one value. `one` is a view that demonstrates that a view can also be adapted using another view.

The last group of statements shows that views can also be empty. `views::empty<bool>` creates a view for boolean values that contains no value. However, it is permissible to pass the resulting view to `asString<>()`, which consequently returns an empty string. This clearly demonstrates that the concept of a view is orthogonal to containers, single values, and emptiness.

Listing 243 illustrates the function templates `std::ranges::join_view<>()` and `std::views::join_with<>()`.

Listing 243: functional/Join.cpp

```
1 // Join.cpp by Ulrich Eisenecker, March 17, 2026
2
3 #include <algorithm>
4 #include <ranges>
5 #include <print>
6 #include <string>
7 #include <vector>
8
9 std::string asString(const std::ranges::range auto& v)
10 {
11     auto strings { std::views::transform(v, [](auto e)
12                                     { return std::to_string(e); }) };
13     auto joined { std::views::join_with(strings, ' ') };
14     std::string s { };
15     std::ranges::for_each(joined, [&s](auto e) { s += e; });
16     return s;
17 }
18
19 using namespace std;
20
21 int main()
```

```

22 | {
23 |     vector numbers { 1,2,3,4,5 };
24 |     println("numbers: {}\\nnumbers as string: \\\"{}\\\"", numbers, asString(numbers));
25 |
26 |     vector<vector<int>> nested { { 1,2 }, { 3,4 }, { 5,6 } };
27 |     println("nested vector: {}\\nflattened string: \\\"{}\\\"",
28 |           nested,
29 |           asString(ranges::join_view(nested)));
30 | }

```

The function template `asString<>()` replaces `asString<>()` in Listing 242. When using the previous function template `asString<>()`, the string elements are simply concatenated. The new function template `asString<>()` inserts a space between two adjacent string elements. This is achieved by using `std::views::join_with()`. Its first parameter is a range, and its second parameter is a separator, which can be a single element or a view.

The first statement in `asString<>()` converts the elements of `v` into strings and initializes the local variable `strings` with the result. The following statement calls `std::views::join_with<>()` with `strings` and a space. The return value is used to initialize the local variable `joined`. Next, the string variable `s` is zero initialized. The next statement calls `std::ranges::for_each<>()` with `joined` as the first argument and a lambda expression as the second argument. The lambda captures `s` by reference and successively appends `e` to `s`. Finally, `asString<>()` returns `s` as the result. Many local variables were introduced here to clearly separate the various steps. By applying referential transparency, all of these variables could be eliminated. Readability could be improved by consistently using the piping syntax. Both remain as exercises for the reader.

The first group of statements in the `main()` function demonstrates the use of `asString<>()` with a one dimensional vector.

The second group of statements first creates and initializes a two-dimensional vector named `nested`. To demonstrate its two-dimensional nature it is output with `println<>()`. The third argument of `println<>()` is obtained by flattening `nested` via a call to `ranges::join_view<>()`, which is then passed as an argument to `asString<>()`. The output clearly shows that this results in a single string whose elements are separated by spaces.

The code and its explanations overlap in part with the Section *The Vector Monad*. However, they demonstrate how well containers, ranges and views are coordinated. Next, lazy evaluation is applied to the calculation of the prime factors of a natural number.

5.4.8.3. Prime Factorization

Every natural number can be expressed as the product of its prime factors; for example, 20 is equal to $2 \times 2 \times 5$. This also applies to a prime number, which is simply the product of 1 and itself; for example, 23 is equal to 23, with the factor 1 omitted.

Below, a simple and inefficient algorithm is implemented functionally. Due to its inefficient implementation, its execution is relatively costly. For this reason, views are used to enable lazy evaluation, which becomes relevant when the prime factorization is performed only for a few natural numbers or for a single one.

In addition, Listing 244 presents a selection of new options for creating and processing views.

Listing 244: functional/PrimeFactors.cpp

```
1 // PrimeFactors.cpp by Ulrich Eisenecker, March 18, 2026
2
3 #include <iostream>
4 #include <functional> // Because of std::multiplies<>()
5 #include <list>       // Because of list<>
6 #include <numeric>    // Because of std::accumulate<>()
7 #include <print>      // Because of println<>()
8 #include <ranges>
9 #include <tuple>      // Because of get<>()
10 #include <utility>    // Because of pair<>
11
12 using namespace std;
13
14 using ulli_t = unsigned long long int;
15
16 auto twoFactors(ulli_t n)
17 {
18     auto factor { n / 2ull };
19     while(factor && n % factor)
20     {
21         --factor;
22     }
23     return pair { factor, factor ? n / factor : 1 };
24 }
25
26 auto primeFactors(ulli_t n) -> list<ulli_t>
27 {
28     list<ulli_t> primes { };
29     auto factors { twoFactors(n) };
30     if (factors.second > 1)
31     {
32         primes.prepend_range(primeFactors(factors.first));
33         primes.push_front(factors.second);
34     }
35     return primes;
36 }
37
```

```

38 int main()
39 {
40     auto f1 { views::zip(views::iota(0),
41                        views::iota(0) | views::transform(primeFactors))
42                | views::drop(2)
43                | views::take(20) };
44     println("Numbers and prime factors:\n{}\n",f1);
45
46     auto f2 { f1 | views::transform([](auto p) { return get<1>(p); }) };
47     println("Only prime factors:\n{}\n",f2);
48
49     auto f3 { f2 | views::transform([](auto f)
50                                   { return accumulate(begin(f),end(f),1,
51                                                         multiplies<ulli_t>());
52                                   }) };
53     println("Only products:\n{}\n",f3);
54
55     cout << "Number to calculate prime factors for: ";
56     ulli_t n;
57     cin >> n;
58     println("Prime factors of {}: {}\n",n,
59            views::iota(0) | views::transform(primeFactors)
60                | views::drop(n)
61                | views::take(1)
62                | views::join);
63
64 }

```

The function `twoFactors()` attempts to factor its argument `n` of type `ulli_t` (a type alias for unsigned long long int) into a product of two factors. This is done in a straightforward manner. It initializes the local `auto` variable `factor` with `n` integrally divided by 2. Then it starts a while loop that runs as long as `factor` is greater than 0 and `n` modulo `factor` is not equal to 0. As long as this compound condition is true, `factor` is decremented by 1. So when `factor` becomes 0, the loop is no longer executed. If `factor` is greater than zero and `n` modulo `factor` is equal to 0, this means that a valid factor has been found. Therefore, the while loop also terminates. The function returns a pair whose first element is `factor` and whose second element is the result of `n` divided by `factor`, provided that `factor` is greater than 0; otherwise, it returns 1. Dividing `n` by 0 is not a good idea, and returning 1 as the second element is safe, since every natural number can be divided by 1 and 1 is the identity element for multiplication.

This function represents the bottleneck of the algorithm, as it checks all numbers starting from $n / 2$ and working downward to find the largest number that is a valid divisor. It would be more efficient to check only prime numbers as divisors.

The `primeFactors()` function takes `n` of type `ulli_t` as an argument and returns a `std::list<ulli_t>` containing all prime factors in ascending order.

First, the local variable `primes` of type `list<ulli_t>` is created and zero-initialized. Next, the auto variable `factors` is created and initialized with the result of calling `twoFactors(n)`. The condition of the if statement checks whether `factors.second` is equal to 1. If this is the case, `primes.prepend_range<>()` is called, which inserts the elements of the range passed as an argument before `begin()`. Therefore, the resulting list contains the prime factors in ascending order. To calculate the remaining prime factors, `primeFactors()` is called recursively here with `factors.first` as the argument. The following statement inserts `factors.second` as the first element of the list.

A list was chosen over a vector here because it allows new elements to be inserted at either end in constant time. It is not necessary to reorganize a list's internal memory when additional elements need to be stored.

The last statement returns `primes` as the result of calling `primeFactors()`.

The first block of statements in `main()` shows how to calculate the prime factors for the numbers 2 through 21 and print each number along with its prime factors.

Associating the elements of two or more ranges is the purpose of `std::views::zip<>()`. Its operation can be thought of zipping two stripes (or more—this is, when the metaphor is no longer natural). Zipping stops, as soon as the elements of one range are exhausted. The result of a single zip action is a tuple with as many elements as there are ranges.

Only two ranges are involved here. The first range is actually a range generator that produces a sequence of elements by incrementing a starting value. An ending value can be specified as the second argument. In this case, it is a bounded range factory. If the second parameter is omitted, as is the case here, it is an unbounded range factory. For this reason, `views::iota(0)` can, in a sense, be regarded as a potentially very large object that is created incrementally as needed, which can also be viewed as a form of laziness.

The range used as the second argument for `views::zip<>()` also starts with `views::iota(0)`. Now, the pipe syntax is used to apply functions sequentially to the preceding expression. First, the sequence potentially generated by `views::iota()` is transformed by calling `primeFactors()`. `views::drop()` removes the specified number of elements from the beginning of the range. Here, the first two elements are removed, since 0 and 1 have no prime factors. Next, 20 elements are taken with `views::take(20)`, which causes the next 20 elements, starting from 2 inclusive, to be generated by the range factory. The result of this expression is used to initialize the auto variable `f1`. All of this happens in constant time, since computations are only specified but not evaluated!

The evaluation takes place when the following output statement is executed. It prints the numbers from 2 through 21, inclusive, along with their corresponding prime factors, on a single line (Figure 60).

Figure 60: Numbers and prime factors

```
[(2, [2]), (3, [3]), (4, [2, 2]), (5, [5]), (6, [2, 3]), (7, [7]), (8,
[2, 2, 2]), (9, [3, 3]), (10, [2, 5]), (11, [11]), (12, [2, 2, 3]),
(13, [13]), (14, [2, 7]), (15, [3, 5]), (16, [2, 2, 2, 2]), (17,
[17]), (18, [2, 3, 3]), (19, [19]), (20, [2, 2, 5]), (21, [3, 7])]
```

In the next group of statements, the auto variable `f2` is initialized with the result obtained by manually extracting only the prime factors from each element of `f1`. In fact, this is an excellent example of how worthwhile it is to read the documentation, since `std::views::elements<>()` exists, allowing the code to be rewritten equivalently as follows: `println("Only prime factors:\n{}\n", views::elements<1>(f1));`. The resulting output is shown in Figure 61.

Figure 61: Only prime factors

```
[2], [3], [2, 2], [5], [2, 3], [7], [2, 2, 2], [3, 3], [2, 5], [11],
[2, 2, 3], [13], [2, 7], [3, 5], [2, 2, 2, 2], [17], [2, 3, 3], [19],
[2, 2, 5], [3, 7]]
```

The third group of statements performs a simple test. It converts each element of `f2` into the product of its prime factors. This is done by applying `std::accumulate<>()` to an element containing the prime factors. Since this is still an algorithm from the old STL, both the start and end iterators must be specified. The third argument is 1, which is required as the initial value for the accumulation. The auto variable `f3` is initialized with the result of evaluating this expression. It should be emphasized once again that all of this takes place in constant time, since no products have been calculated yet. This happens when `f3` is printed. The resulting products correspond exactly to the numbers 2 through 21, which means that the previously calculated prime factors were correct (Figure 62).

Figure 62: Only products

```
[2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20,
21]
```

The last group of statements shows how to calculate the prime factors of any number. First, the user is prompted to enter a value for `n`. This is the number that will then be factored into its prime factors.

Currently, there is no subtype of `std::ranges::view_interface` that provides its own index operator (`std::ranges::view_interface<D>::operator[]` - *Cppreference.Com* n.d.), which means that no view currently defines the `[]` operator. A specific individual element can be identified by removing the preceding elements and then selecting only one element.

First, an unbound range factory is created that starts at 0. This is transformed by applying `primeFactors()`. Since the starting value of the range factory is 0, exactly `n` elements must be removed to access the `n`th element; then, one element is extracted.

Since `views::iota(0)` is a range and each of its elements is also a range—namely, the prime factors—`println<>()` would enclose the resulting prime factors in an additional pair of square brackets. To remove these, the resulting single element is flattened using `views::join`.

It should be noted that the prime factors of a single number can be calculated and printed in a much simpler way, namely: `println("Prime factors of : ", n, primeFactors(n));`. The method shown using views is intended merely to demonstrate that this general case also encompasses the specific case of prime factorization and printing for a single number.

This example illustrates the laziness of views, the use of pipe syntax for function composition, and various useful functions for ranges and views. Although performance is low, it is acceptable when the prime factorization is performed for only a few numbers or a single number. Interested readers might first consider possible ways to speed up the calculation of prime factors for individual numbers, e.g., by prioritizing the extraction of larger factors. The resulting implementations should be carefully benchmarked in comparison to the algorithm presented above.

Another idea is to factorize larger ranges of natural numbers, starting with 2, into prime factors. In this case, all previous prime factors can be retained and reused, which could potentially lead to a significant increase in speed. Of course, this comes with high memory requirements, illustrating the common trade-off between time and memory in algorithms and programming.

A basic approach would be to implement prime factorization algorithms from a textbook and systematically compare them using benchmarks.

In addition, the output format could be adapted to meet different requirements. Here, only the standard output generated by `println<>()` was used. Other formats could also be generated using, for example, `std::views::join_with<>`.

5.4.9. Game of Life

In the 1970s, computer magazines frequently featured articles on the *Game of Life*, which was developed by the British mathematician John Horton Conway (*Conway's Game of Life* 2025). It generated considerable interest and became the subject of intensive research on cellular automata. In the 1980s, several implementations of the *Game of Life* were available for personal computers.

The game is relatively simple. The game board consists of a grid of square cells that are either *alive* or *dead*. The alive or dead status can be represented by a color, a shape, or both. At the start, all cells are dead, and the player (this is a single-player game) creates some alive cells. The following rules then apply:

1. Any living cell that has fewer than two other living cells as immediate neighbors dies.
2. Any living cell that has four or more living cells as immediate neighbors dies.
3. Any living cell that has two or three living immediate neighbors remains unchanged.
4. Any dead cell that has exactly three living neighbors becomes alive.

From this point on, the game continues on its own without any further intervention. There are several possible outcomes: either all the cells die off and the game ends, or patterns of living cells keep reforming, or a static pattern emerges that persists indefinitely.

Many programmers, especially those familiar with the object-oriented programming paradigm, begin by designing a two-dimensional grid consisting of cell elements that are either dead or alive. A next design step might involve enabling a cell—whether dead or alive—to count its neighboring cells. Then the question must be clarified as to whether it is the cell’s responsibility or the game board’s responsibility to let a cell die or bring it to life. Another object-oriented idea would be to develop a game coordinator object that takes on this responsibility. Somewhere in the midst of all this design work, it becomes clear that one more problem needs to be solved: The dimensions of the grid are limited. What should happen when a boundary is crossed? Can the playing field be re-entered from the opposite side? Or should cells that leave the playing field always die?

The following text presents a functional program inspired by (*Conway’s Game of Life: A Functional Approach* 2024).

A fundamental question is: *Should a dead cell actually be represented in a program?* If we rephrase *dead cell* as *non-existent cell*, this question can be more easily answered with a clear *no*. Another relevant follow-up question is: *Should a grid be explicitly displayed?* If there are no dead cells, most of the grid’s nodes will likely contain nothing. But what is a node that contains nothing? Obviously, it is not a node. Viewed this way, the need to display anything is limited to live cells.

This raises the question: *What does a living cell know?* In fact, it must know its coordinates—in the case of a Cartesian world, the x- and y-coordinates. And if one initially considers this world to be infinite, one need not even think only of positive coordinates or a necessary limit on the coordinates. The basic data type `int` can serve as a starting point. If necessary, other integer types with a larger range of values can be selected.

Depending on a cell's coordinates and the geometry of the world, the coordinates of neighboring cells must be calculated. This can be done using a function called `neighbors()`, which takes a cell as an argument. Its return value is a set of (neighboring) cells.

To apply the rules of the *Game of Life*, one only needs to know a cell's neighbors. A function called `next()` takes a container containing the current cells (the *old generation*) as an argument and calculates the next generation of cells (the *new generation*), which it returns as the result.

These are the two essential functions for the *Game of Life*—nothing more.

The `main()` function manages two variables for the old and new cell generations—yes, this is a stateful part of the program—and generates a rudimentary text output. Another stateful variable is used to count the rounds.

After reading these alternative ideas, an object-oriented programmer might be inspired to develop a similar design. ***However, this thought experiment demonstrates the influence a programming paradigm can have. This is a characteristic of paradigms in most fields, as they provide concepts and the relationships between them to perceive, describe, understand, and design new artifacts in a domain of interest.***

The resulting program is shown in Listing 245.

Listing 245: functional/LifeSimple.cpp

```
1 // LifeSimple.cpp by Ulrich Eisenecker, March 19, 2026
2 // Inspired by
3 // https://strangeleaflet.com/conways-game-of-life-a-functional-approach/
4
5 #include <iostream>
6 #include <map>
7 #include <print>
8 #include <set>
9 #include <utility>
10
11 using namespace std;
12
13 using Position = int;
14 using Frequency = int;
15 using Cell = pair<Position,Position>;
16 using Cells = set<Cell>;
17 using Frequencies = map<Cell,Frequency>;
18
19 auto neighbors(Cell cell)
20 {
21     Cells result;
22     for (Position x : { -1,0,1 })
23         for (Position y : { -1,0,1 })
24             if (!(x == 0 && y == 0))
25                 result.insert(make_pair(cell.first + x,cell.second + y));
```

```

26     return result;
27 }
28
29 auto next(const Cells& cells)
30 {
31     Frequencies f;
32     for (auto cell : cells)
33     {
34         auto n { neighbors(cell) };
35         for (auto e : n)
36             f[e] += 1;
37     }
38     Cells result;
39     for (auto e : f)
40         if (cells.contains(e.first) && (e.second == 2 || e.second == 3))
41             result.insert(e.first);
42     else
43         if (e.second == 3)
44             result.insert(e.first);
45     return result;
46 }
47
48 int main()
49 {
50     Cells ng { { 1,0 }, { 1,1 }, { 1,2 } },
51              og;
52     unsigned long long int round { };
53     do
54     {
55         println("Round {}: {}", round++, ng);
56         og = ng;
57         ng = next(og);
58     } while (ng != og);
59 }

```

At the beginning of the program, several type aliases are declared. `Position` is the integer type used to represent coordinates. If the range of coordinate values needs to be changed, this is the right place to do so. `Frequency` is the integer type used to represent the number of live neighbors of a cell. Since a cell in a Cartesian world can have a maximum of eight neighbors, a smaller integer type could be chosen, but this would not result in any performance gain on most computers. It would be possible to save a few bytes. In a Cartesian world, a cell consists of a pair of coordinates. Therefore, `Cell` is an alias for `std::pair<Position, Position>`. Since two cells can never occupy the same position (perhaps a variation of the *Pauli exclusion principle*?), a set of cells is best represented as `std::set<>`. Therefore, `Cells` is a type alias for `std::set<Cell>`. The rules of the *Game of Life* depend on the number of adjacent cells to a given cell. To represent this information, `Frequencies` is defined as an alias for `std::map<Cell, Frequency>`. Now aliases for all necessary types have been declared and can be adjusted as needed.

The `neighbors()` function takes a cell—that is, a pair of `x/y` coordinates—as an argument. Using two nested for loops, it calculates the `x/y` coordinates of the neighboring cells and stores the result, which is a set of cells. For clarity, the statements of the two for loops are written as individual statements. The inner if statement prevents the cell itself from becoming a neighbor of itself. The two for loops are range-based for loops whose ranges are defined as literals.

The `next()` function takes a set of cells as an argument. It consists of two parts. In the first part, the variable `f` of type `Frequencies` is declared. It then iterates through all the cells passed as an argument and determines the neighbors of each cell. Next, it iterates through the set of neighbors, using each neighbor as a key in the map `f` and incrementing the corresponding value. Upon completion of this part, `f` contains all neighboring cells of all cells, along with information on how often they appear as neighbors.

In the second part, the variable `result` of type `Cells` is declared. Now, a range-based for loop iterates through the map `f`. Here, too, the loop statement is coded as a single statement rather than a compound statement for the sake of brevity, as is the case throughout the rest of the book. The if-else statements precisely encode the rules of the *Game of Life*. The first if condition checks whether cell `e` is alive and has two or three neighbors. If this is the case, it is added to `result`. This corresponds to the third rule mentioned above. The entire condition is subject to a Boolean short-circuit evaluation. If it evaluates to false, the else branch is followed. There, it is checked whether the number of neighbors is exactly 3. If this is the case, cell `e` is added to `result`, which corresponds to rule 4. With regard to rules 1 and 2, no action is required, as they signify death, which is implemented by not adding cell `e` to `result`. Finally, `result` is returned.

The `main()` function declares the variables `ng` and `og` of type `Cells`. `ng` is initialized with a seed pattern, which is a so-called *blinker* that reproduces itself endlessly after two rounds. If a different seed pattern is desired, this is where adjustments can be made. Note that `ng` is initialized with the starting pattern. The reason for this is that `ng` is later assigned to `og` first in the do-while loop.

The variable `round` of type `unsigned long long int` counts the rounds.

Next, a do-while loop begins. It prints `round` and `ng`. Then `ng` is assigned to `og`, and the result of `next(og)` is assigned to `ng`. The loop continues as long as `ng` and `og` are different. Consequently, the blinker program will never stop. To terminate it, it must be interrupted by pressing *Ctrl-C*.

Admittedly, this version of the *Game of Life* isn't particularly entertaining. Staring at boring rows of coordinates offers no entertainment whatsoever. The argument that the user interface aligns with the parsimony of functional programming isn't helpful. As a final step, a slightly improved text-based user interface is implemented using *Ncurses*, which was introduced in the Section An Example of a Text-based User Interface and the following sections.

Listing 246 shows only an excerpt from the program. The type aliases and the functions `neighbors()` and `next()` are identical to those in Listing 245. The complete program is available in the accompanying material.

Listing 246: functional/LifeTUI.cpp (excerpt)

```

1 // LifeTUI.cpp by Ulrich Eisenecker, March 19, 2026
7 #include <ncurses.h>
50 Cells seed()
51 {
52     vector<pair<string,Cells>>
53     seeds { { "block"s,{ {2,2},{2,3},{3,2},{3,3} } },
54             { "beehive"s,{ {1,2},{2,1},{2,3},{3,1},{3,3},{4,2} } },
55             { "blinker"s,{ {1,0},{1,1},{1,2} } },
56             { "pentadecathlon"s,{ {3,5},{3,10},
57                                   {4,3},{4,4},{4,6},
58                                   {4,7},{4,8},{4,9},{4,11},{4,12},
59                                   {5,5},{5,10} } },
60             { "acorn"s,{ {5,6},{6,8},{8,5},{7,6},{7,9},{7,10},{7,11} } }
61     };
62     while (true)
63     {
64         for (int i { 0 };auto s : seeds)
65             cout << ++i << " - " << s.first << endl;
66         cout << "\nYour choice: ";
67         uintmax_t answer;
68         cin >> answer;
69         if (answer >= 1 && answer <= size(seeds))
70             return seeds[answer - 1].second;
71         else
72             cout << "Wrong selection, please try again.\n" << endl;
73     }
74     return { };
75 }
76
77 bool quit()
78 {
79     return getch() == 27;
80 }
81
82 void display(const Cells& cells)
83 {
84     static unsigned long long int round { 0 };
85     for (auto cell : cells)
86         mvaddch(cell.first,cell.second,'+');
87     mvprintw(getmaxy(stdscr) - 1,1,
88              ("Generation "s + to_string(round++)).c_str());
89 }
90
91 int main()
92 {
93     Cells og,
94     ng { seed() };
95

```

```
96 |  initscr();    // Initialize Ncurses.
97 |  noecho();    // Disable echoing of an entered character.
98 |  curs_set(0); // Make the cursor transparent.
99 |  timeout(200); // Non-blocking input for 200 ms
100 |  do
101 |  {
102 |      erase(); // Clear entire screen
103 |      display(ng);
104 |      og = ng;
105 |      ng = next(og);
106 |  } while (ng != og && !quit());
107 |
108 |  timeout(-1); // Restore blocking input
109 |  getch();    // Wait for key being pressed
110 |
111 |  curs_set(1); // Restore the normal appearance of the cursor.
112 |  endwin();    // Stop using Ncurses; restore the normal
113 |  }           // behavior of the terminal window.
```

The `seed()` function provides a degree of variability. It defines a small selection of seed patterns from which the user can choose. The seed patterns can be easily expanded as needed. There are numerous online resources that provide additional seed patterns and related information. The function does not fully adhere to the principles of purely functional programming, as there may be repeated assignments to the variable `answer`. It is left as an exercise for the reader to eliminate these assignments. There is one more interesting detail. As soon as the user has selected a starting pattern, the function returns it. To avoid compiler warnings that not all possible execution paths are returned, a zero-initialized `Cells` exemplar is returned at the end.

The `quit()` function simply checks whether a key with *ASCII* code 27 has been pressed, i.e., whether the *Esc* key has been pressed. If so, it returns `true`; otherwise, it returns `false`.

The `display()` function is specific to *Ncurses*. However, it also manages the static variable `round`, which counts the rounds. The range-based `for` loop places a plus sign at every position in the active terminal window where a cell is located. The information about the current round is then displayed on the bottom line.

In the `main()` function, `og` and `ng` are declared as in the previous program. The only difference is that `ng` is now initialized with the result of the `seed()` call. The usual steps for initializing *Ncurses* are then performed. An interesting detail is the call to `timeout(200)`. This function sets a time limit of one-fifth of a second during which non-blocking input is allowed. Normally, a terminal application waits until users have finished entering their input. Now, a user can enter input, and if they do not, the program continues executing.

In the `do-while` loop, the screen is first cleared, then `ng` is displayed. Next, `ng` is assigned to the variable `og`, and the result of calling `next()` is assigned to `ng`. The loop continues as long as `ng` and `og` are different and the user has not pressed the *Esc* key.

When the loop ends, the usual behavior—blocking input—is restored, and the program waits for the next keystroke. This is important because it allows the user to continue seeing the result of the last round. *Ncurses* is then exited, and normal terminal mode is restored.

When compiling the program, it is important to remember to include the *-l* option along with the *ncurses* argument, i.e., *-lncurses*. Otherwise, the link will fail due to a missing library.

When running the program, the screen may go blank. In this case, it is recommended to wait a while. Usually, a pattern will reappear in the visible area of the screen. Interestingly, *Ncurses* appears to manage a virtual screen that also includes negative coordinates. Therefore, there seems to be no need for clipping.

It is fascinating to see how well the procedural and functional parts of the program work together. Creating a text-based user interface with *Ncurses* required very little effort. It is left as an exercise for the reader to develop a GUI version of the program, e.g., using *FLTK*. There are further possibilities for extension or modification. For example, it would be interesting to provide a kind of automatic pattern-tracking function. This could be a minimal rectangle that contains all cells and is always displayed centered. Another feature could involve detecting cycles longer than 1. In this case, the program could output information about the cycle size and ask the user whether it should be terminated. A further playful extension would be the design and implementation of an editor for initial patterns that generates a format which could be interactively loaded into the functional version of the *Game of Life*. Last but not least, one could consider populating 3D or higher-dimensional worlds with cells. This increases the number of a cell's direct neighbors and certainly requires modified or additional rules for cell death or generation. It is recommended to think through any problems that arise and possible solutions oneself before searching the internet for fully developed programs.

5.4.10. Discussion

Despite existing similarities, the functional programming paradigm gives a different feeling than the programming paradigms discussed so far, namely the procedural paradigm, abstract data types, generics, and the object-oriented paradigm. This impression is often reinforced when a programmer has initially been familiarized only with the so-called imperative paradigms—namely procedural and object-oriented programming, as well as abstract data types as an intermediate step—and only comes into contact with functional programming at a later stage.

Some of the quality goals of functional programming, such as terseness, seem to conflict with more traditional software qualities like readability. However, terseness results in significantly fewer lines of code, which is certainly a desirable code quality. Functional programming also strives for highly abstracted code that is more widely applicable, which likewise reduces code size and benefits potential code reuse. The near-complete elimination of state—whether variables in the traditional sense or program

state in general—also eliminates a common source of errors and thus contributes to overall software quality. Admittedly, it can feel easier to think in terms of objects, which usually give the impression of being somehow tangible, than in terms of abstract functions and concepts such as functors and monads. However, it is questionable whether this is a necessity—that is, an innate characteristic of some, many, or all people—or whether it is a consequence of not having been taught functional programming from the start.

Undoubtedly, functional programming has its roots in mathematics, while object-oriented programming originated in simulation, with *Simula* considered the first object-oriented programming language and used in physical modeling (*Object-Oriented Programming* 2026). Mathematical entities are not mutable. Even complex entities such as matrices are created and initialized but not modified. It is even conceivable to relax strict immutability to allow for monotonic growth, i.e. accumulation of state, of objects, still forbidding the removal of state information. But this can be cumbersome when modeling physical objects such as humans and their basic physical properties. Of course, instead of age the date of birth can become an attribute, so the age of a human is a result of calculating the span between the date of birth and e.g. the present date. But what about weight and hair color? Well, all these information could be recorded with an internal container attribute of a human associating some time and the attribute of interest, e.g. *March 20, 2026* and *80.6 kg* as a single element. This can be difficult questions and even more difficult solutions, that should definitively be within the bounds of the given software requirements.

The programming examples presented in this chapter have a strong mathematical focus. Since most texts on functional programming rely on mathematical examples, this appears to be a proven approach, backed by many years of practical experience, for introducing functional programming. The content of this chapter will hopefully serve as a starting point for interested readers to further explore functional programming and apply it in practical programming. The implementation of the *Game of Life* with a text-based user interface provides a first impression of how the functionally programmed parts of a program can be combined with parts implemented in other paradigms. However, one should resist the temptation to develop a kind of pidgin language with an idiosyncratic mix of functional programming and other paradigms.

Bibliography

- ACCU (Oct. 2015). *Introducing Concepts*. Overload, 23(129):4-8. URL: https://accu.org/journals/overload/23/129/sutton_2157/ (visited on 05/27/2022).
- (Feb. 2016). *Defining Concepts*. Overload, 24(131):4-8. URL: https://accu.org/journals/overload/24/131/sutton_2198/ (visited on 05/27/2022).
- Algebraic Data Type* (Feb. 17, 2026). In: *Wikipedia*. URL: https://en.wikipedia.org/w/index.php?title=Algebraic_data_type&oldid=1338804607 (visited on 03/03/2026).
- Allsopp, Toby (Oct. 12, 2016). *Free Monads in C++*. The Things That Toby Typed. URL: <https://toby-allsopp.github.io/2016/10/12/free-monads-in-cpp.html> (visited on 02/10/2026).
- Alonzo Church* (Mar. 29, 2026). In: *Wikipedia*. URL: https://en.wikipedia.org/w/index.php?title=Alonzo_Church&oldid=1346015065 (visited on 05/03/2026).
- Average Human Height by Country* (Apr. 1, 2025). In: *Wikipedia*. URL: https://en.wikipedia.org/w/index.php?title=Average_human_height_by_country&oldid=1283384842 (visited on 05/04/2025).
- Benyon, David (2017). *Designing User Experience*. Fourth edition. Harlow, United Kingdom: Pearson Education Limited. ISBN: 978-1-292-15551-7.
- Biggerstaff, T.J. (1994). “The Library Scaling Problem and the Limits of Concrete Component Reuse”. In: *Proceedings of 1994 3rd International Conference on Software Reuse*, pp. 102–109. DOI: 10.1109/ICSR.1994.365806.
- Bolas, Nicol (July 11, 2016). *Answer to “C++17: What Does “Operator with Precedence below Cast” Mean in “Fold” Concept?”*. Stack Overflow. URL: <https://stackoverflow.com/a/38299340> (visited on 05/24/2022).
- Booch, Grady and Michael Vilot (Oct. 1990). “The Design of the C++ Booch Components”. In: *ACM SIGPLAN Notices* 25.10, pp. 1–11. ISSN: 0362-1340, 1558-1160. DOI: 10.1145/97946.97947. URL: <https://dl.acm.org/doi/10.1145/97946.97947> (visited on 01/15/2024).
- Breymann, Ulrich (2020). *C++ programmieren: C++ lernen - professionell anwenden - Lösungen nutzen*. ISBN: 978-3-446-46386-8.
- C++ Named Requirements: PODType (Deprecated in C++20) - Cppreference.Com* (n.d.). URL: https://en.cppreference.com/w/cpp/named_req/PODType (visited on 02/27/2026).

- C++ Named Requirements: RandomNumberDistribution (since C++11)* - *Cppreference.Com* (n.d.). URL: https://en.cppreference.com/w/cpp/named_req/RandomNumberDistribution (visited on 05/02/2025).
- C++ Named Requirements: RangeAdaptorObject (since C++20)* - *Cppreference.Com* (n.d.). URL: https://en.cppreference.com/w/cpp/named_req/RangeAdaptorObject (visited on 05/24/2025).
- C++ Templates* (Oct. 12, 2025). *C++ Templates: How to Iterate through Std::Tuple: C++26 Packs and Expansion Statements*. C++ Stories. URL: <https://www.cppstories.com/2025/tuple-iteration-cpp26/> (visited on 03/04/2026).
- Cardelli, Luca and Peter Wegner (Dec. 1985). "On Understanding Types, Data Abstraction, and Polymorphism". In: *ACM Comput. Surv.* 17.4, pp. 471–523. ISSN: 0360-0300. DOI: 10.1145/6041.6042. URL: <http://doi.acm.org/10.1145/6041.6042>.
- Category (Mathematics)* (Nov. 6, 2025). In: *Wikipedia*. URL: [https://en.wikipedia.org/w/index.php?title=Category_\(mathematics\)&oldid=1320721384](https://en.wikipedia.org/w/index.php?title=Category_(mathematics)&oldid=1320721384) (visited on 01/27/2026).
- Circular Buffer* (Apr. 26, 2022). In: *Wikipedia*. URL: https://en.wikipedia.org/w/index.php?title=Circular_buffer&oldid=1084776648 (visited on 05/09/2022).
- Class Template Argument Deduction (CTAD) (since C++17)* - *Cppreference.Com* (n.d.). URL: https://en.cppreference.com/w/cpp/language/class_template_argument_deduction (visited on 07/11/2022).
- Combination* (Mar. 7, 2023). In: *Wikipedia*. URL: <https://en.wikipedia.org/w/index.php?title=Combination&oldid=1143342260> (visited on 03/08/2023).
- Combinatorics* (Mar. 2, 2023). In: *Wikipedia*. URL: <https://en.wikipedia.org/w/index.php?title=Combinatorics&oldid=1142506872> (visited on 03/08/2023).
- Configuration Management Plan - Fast Light Toolkit (FLTK)* (n.d.). URL: <https://www.fltk.org/cmp.php> (visited on 02/16/2024).
- Constraints and Concepts (since C++20)* - *Cppreference.Com* (n.d.). URL: <https://en.cppreference.com/w/cpp/language/constraints> (visited on 06/29/2023).
- Containers Library* - *Cppreference.Com* (n.d.). URL: https://en.cppreference.com/w/cpp/container#Sequence_containers (visited on 04/25/2025).
- Contents - Immer 0.0.0 Documentation* (n.d.). URL: <https://sinusoid.es/immer/index.html> (visited on 03/03/2026).
- Conway's Game of Life* (Aug. 22, 2025). In: *Simple English Wikipedia, the Free Encyclopedia*. URL: https://simple.wikipedia.org/w/index.php?title=Conway%27s_Game_of_Life&oldid=10471656 (visited on 03/19/2026).
- Conway's Game of Life: A Functional Approach* (Aug. 23, 2024). *Conway's Game of Life: A Functional Approach*. Strange Leaflet. URL: <https://strangeleaflet.com/conways-game-of-life-a-functional-approach/> (visited on 03/19/2026).
- Costantini, F., D. Gibson, et al. (n.d.). *FLTK 1.3.9: FLTK Programming Manual*. URL: <https://www.fltk.org/doc-1.3/index.html> (visited on 02/06/2024).
- Costantini, F., M. Melcher, et al. (Apr. 29, 2025). *FLTK 1.4.3: FLTK Programming Manual*. *FLTK Programming Manual*. URL: <https://www.fltk.org/doc-1.4/> (visited on 07/08/2025).

- Čukić, Ivan (2019). *Functional Programming in C++*. Shelter Island, New York: Manning Publications Company. 296 pp. ISBN: 978-1-61729-381-8.
- Currying (Dec. 24, 2025). In: *Wikipedia*. URL: <https://en.wikipedia.org/w/index.php?title=Currying&oldid=1329137515#Definition> (visited on 01/12/2026).
- Czarnecki, Krzysztof and Ulrich Eisenecker (2000). *Generative Programming: Methods, Tools, and Applications*. Boston: Addison Wesley. 832 pp. ISBN: 978-0-201-30977-5.
- Dependent Names - *Cppreference.Com* (n.d.). URL: https://en.cppreference.com/w/cpp/language/dependent_name (visited on 06/26/2023).
- Diaper, D. and Neville A. Stanton, eds. (Jan. 9, 2019). *The Handbook of Task Analysis for Human-Computer Interaction*. Mahwah, NJ: Lawrence Erlbaum. 650 pp. ISBN: 978-0-8058-4432-0.
- Dos Reis, Gabriel and Bjarne Stroustrup (Jan. 12, 2006). "Specifying C++ Concepts". In: *ACM SIGPLAN Notices* 41.1, pp. 295–308. ISSN: 0362-1340, 1558-1160. DOI: 10.1145/1111320.1111064. URL: <https://dl.acm.org/doi/10.1145/1111320.1111064> (visited on 05/27/2022).
- Eisenecker, Ulrich W. (Aug. 30, 2022). *Basic Programming Paradigms*. 1.3. Leipzig: Leipzig University. 339 pp. URL: <https://www.wifa.uni-leipzig.de/institut-fuer-wirtschaftsinformatik/professuren/professur-insbesondere-softwareentwicklung/studium/lehrveranstaltungen/bpp>.
- Fertig, Andreas (n.d.). *C++ Insights*. URL: <https://cppinsights.io/> (visited on 05/24/2022).
- Fleischer, Willi (Dec. 1, 2021). *Installation CodeBlocks with FLTK on Windows - Google Drive*. URL: https://drive.google.com/file/d/1RMSzBlc4e_tjOKBE07nmZiY8epULXQmx/view?pli=1 (visited on 02/02/2024).
- Fltk/Fltk (Jan. 23, 2024). The FLTK Team. URL: <https://github.com/fltk/fltk> (visited on 01/23/2024).
- Fold Expression (since C++17) - *Cppreference.Com* (n.d.). URL: <https://en.cppreference.com/w/cpp/language/fold> (visited on 05/24/2022).
- Formatting Library (since C++20) - *Cppreference.Com* (n.d.). URL: <https://en.cppreference.com/w/cpp/utility/format> (visited on 03/04/2024).
- Forsum, Hind (July 11, 2016). C++17: What Does "Operator with Precedence below Cast" Mean in "Fold" Concept? Stack Overflow. URL: <https://stackoverflow.com/q/38298565> (visited on 05/24/2022).
- Functor (Aug. 17, 2025). In: *Wikipedia*. URL: <https://en.wikipedia.org/w/index.php?title=Functor&oldid=1306406256> (visited on 01/27/2026).
- Gamma, Erich et al. (1995). *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley Professional Computing Series. Reading, Mass: Addison-Wesley. 395 pp. ISBN: 978-0-201-63361-0.
- GNOME Human Interface Guidelines (n.d.). URL: <https://developer.gnome.org/hig/> (visited on 03/25/2024).
- Gregoire, Marc (2020). *Professional C++*. 5th ed. Indianapolis: John Wiley and Sons. ISBN: 978-1-119-69540-0.

- Gregor, Douglas and Jaakko Järvi (2007). “Variadic Templates for C++”. In: *Proceedings of the 2007 ACM Symposium on Applied Computing - SAC '07*. The 2007 ACM Symposium. Seoul, Korea: ACM Press, p. 1101. ISBN: 978-1-59593-480-2. DOI: 10.1145/1244002.1244243. URL: http://www.jot.fm/issues/issue_2008_02/article2/ (visited on 05/25/2022).
- Homebrew The Missing Package Manager for macOS (or Linux)* (n.d.). URL: <https://brew.sh/> (visited on 02/02/2024).
- Human Interface Guidelines* (n.d.). Apple Developer Documentation. URL: <https://developer.apple.com/design/human-interface-guidelines> (visited on 03/25/2024).
- Human Interface Guidelines - Fast Light Toolkit (FLTK)* (n.d.). URL: <https://www.fltk.org/hig.php> (visited on 03/04/2024).
- Hunt, John (2003). “Inheritance Considered Harmful!” In: *Guide to the Unified Process featuring UML, Java and Design Patterns*, pp. 363–381.
- Jansen, Jean-Bernard (Oct. 17, 2025). *Duckie/Named_types*. URL: https://github.com/duckie/named_types (visited on 03/04/2026).
- JohT (Sept. 5, 2022). *Visualize Catch2 Benchmarks with Vega-Lite*. Johtizen. URL: <https://joht.github.io/johtizen/data/2022/09/05/visualize-catch2-benchmarks-with-vega-lite.html> (visited on 07/22/2025).
- Josuttis, Nicolai (Aug. 30, 2020). *C++ Move Semantics - The Complete Guide*. First Edition. Leanpub. 262 pp. ISBN: 978-3-96730-900-3.
- juwmsft (Apr. 27, 2021a). *Get Started Developing UI for Windows Apps - Win32 Apps*. URL: <https://learn.microsoft.com/en-us/windows/win32/appuistart/getting-started-developing-user-interfaces-portal> (visited on 03/25/2024).
- (Feb. 9, 2021b). *Guidelines - Win32 Apps*. URL: <https://learn.microsoft.com/en-us/windows/win32/uxguide/guidelines> (visited on 03/25/2024).
- Lambda Calculus* (Mar. 26, 2026). In: *Wikipedia*. URL: https://en.wikipedia.org/w/index.php?title=Lambda_calculus&oldid=1345568754 (visited on 05/03/2026).
- Lambda Expressions (since C++11) - Cppreference.Com* (n.d.). URL: <https://en.cppreference.com/w/cpp/language/lambda.html> (visited on 11/14/2025).
- Lazar, Jonathan (2017). *Research Methods in Human Computer Interaction*. 2nd edition. Cambridge, MA: Elsevier. ISBN: 978-0-12-805390-4.
- Liskov, Barbara H. and Jeannette M. Wing (Nov. 1994). “A Behavioral Notion of Subtyping”. In: *ACM Transactions on Programming Languages and Systems* 16.6, pp. 1811–1841. ISSN: 0164-0925, 1558-4593. DOI: 10.1145/197320.197383. URL: <https://dl.acm.org/doi/10.1145/197320.197383> (visited on 01/07/2023).
- Liskov Substitution Principle* (Dec. 5, 2022). In: *Wikipedia*. URL: https://en.wikipedia.org/w/index.php?title=Liskov_substitution_principle&oldid=1125662440 (visited on 01/07/2023).
- Mask (Computing)* (Jan. 28, 2024). In: *Wikipedia*. URL: [https://en.wikipedia.org/w/index.php?title=Mask_\(computing\)&oldid=1200079124](https://en.wikipedia.org/w/index.php?title=Mask_(computing)&oldid=1200079124) (visited on 02/23/2024).

- Mikhajlov, Leonid and Emil Sekerinski (1998). "The Fragile Base Class Problem and Its Impact on Component Systems". In: *Object-Oriented Technologys*. Ed. by Jan Bosch and Stuart Mitchell. Vol. 1357. Berlin, Heidelberg: Springer Berlin Heidelberg, pp. 353–358. ISBN: 978-3-540-64039-4. DOI: 10.1007/3-540-69687-3_72. URL: http://link.springer.com/10.1007/3-540-69687-3_72 (visited on 01/16/2024).
- Model-View-Controller* (Mar. 16, 2024). In: *Wikipedia*. URL: <https://en.wikipedia.org/w/index.php?title=Model%E2%80%93view%E2%80%93controller&oldid=1213978727> (visited on 03/25/2024).
- Ncurses* (Dec. 8, 2023). In: *Wikipedia*. URL: <https://en.wikipedia.org/w/index.php?title=Ncurses&oldid=1188864922> (visited on 01/18/2024).
- NCURSES - New Curses* (n.d.). URL: <https://invisible-island.net/ncurses/> (visited on 01/18/2024).
- Norman, Donald A. (2013). *The Design of Everyday Things*. Revised and expanded edition. New York, New York: Basic Books. 347 pp. ISBN: 978-0-465-05065-9.
- Object-Oriented Programming* (Mar. 10, 2026). In: *Wikipedia*. URL: https://en.wikipedia.org/w/index.php?title=Object-oriented_programming&oldid=1342659733 (visited on 03/20/2026).
- Observer Pattern* (Jan. 25, 2024). In: *Wikipedia*. URL: https://en.wikipedia.org/w/index.php?title=Observer_pattern&oldid=1198846919#cite_note-6 (visited on 03/19/2024).
- Overview - Fmt 8.1.1 Documentation* (n.d.). URL: <https://fmt.dev/latest/index.html> (visited on 07/05/2022).
- Pereyra, Irene (2023). *Universal Principles of UX: 100 Timeless Strategies to Create Positive Interactions between People and Technology*. Rockport Universal vol 4. Beverly, MA, USA: Rockport Publishers, an imprint of The Quarto Group. 223 pp. ISBN: 978-0-7603-7804-5.
- Permutation* (Feb. 11, 2023). In: *Wikipedia*. URL: <https://en.wikipedia.org/w/index.php?title=Permutation&oldid=1138689249> (visited on 03/08/2023).
- pfalcon (Jan. 14, 2013). *Answer to "What Are Some Uses of Template Template Parameters?"*. Stack Overflow. URL: <https://stackoverflow.com/a/14311714> (visited on 07/10/2022).
- PImpl - Cppreference.Com* (n.d.). URL: <https://en.cppreference.com/w/cpp/language/pimpl> (visited on 08/30/2023).
- Pseudo-Random Number Generation - Cppreference.Com* (n.d.). URL: <https://en.cppreference.com/w/cpp/numeric/random> (visited on 05/02/2025).
- RAII - Cppreference.Com* (n.d.). URL: <https://en.cppreference.com/w/cpp/language/raii> (visited on 08/23/2023).
- Random Number Generation* (Mar. 29, 2025). In: *Wikipedia*. URL: https://en.wikipedia.org/w/index.php?title=Random_number_generation&oldid=1282983147 (visited on 05/02/2025).
- Ranges Library (since C++20) - Cppreference.Com* (n.d.). URL: <https://en.cppreference.com/w/cpp/ranges> (visited on 05/24/2025).

- Reference Declaration - Cppreference.Com* (n.d.). URL: <https://en.cppreference.com/w/cpp/language/reference> (visited on 03/16/2021).
- Requires Expression (since C++20) - Cppreference.Com* (n.d.). URL: <https://en.cppreference.com/w/cpp/language/requires> (visited on 06/29/2023).
- Rogers, Yvonne, Helen Sharp, and Jenny Preece (2023). *Interaction Design: Beyond Human-Computer Interaction*. Sixth edition. Hoboken, New Jersey: Wiley. 689 pp. ISBN: 978-1-119-90109-9.
- Sergey (July 11, 2016). *Answer to "C++17: What Does "Operator with Precedence below Cast" Mean in "Fold" Concept?"*. Stack Overflow. URL: <https://stackoverflow.com/a/38299456> (visited on 05/24/2022).
- Shneiderman, Ben et al. (2017). *Designing the User Interface: Strategies for Effective Human-Computer Interaction*. Sixth Edition. Boston: Pearson. 580 pp. ISBN: 978-0-13-438038-4.
- Shotts, William E. (2012). *The Linux Command Line: A Complete Introduction*. San Francisco: No Starch Press. 446 pp. ISBN: 978-1-59327-389-7. URL: <http://linuxcommand.org/tlcl.php>.
- State Pattern* (Apr. 26, 2023). In: *Wikipedia*. URL: https://en.wikipedia.org/w/index.php?title=State_pattern&oldid=1151901651 (visited on 08/28/2023).
- Static_cast Conversion - Cppreference.Com* (n.d.). URL: https://en.cppreference.com/w/cpp/language/static_cast (visited on 03/22/2023).
- std::bind - Cppreference.Com* (n.d.). URL: <https://en.cppreference.com/w/cpp/utility/functional/bind.html> (visited on 01/16/2026).
- std::bitset - Cppreference.Com* (n.d.). URL: <https://en.cppreference.com/w/cpp/utility/bitset> (visited on 05/18/2025).
- std::exception - Cppreference.Com* (n.d.). URL: <https://en.cppreference.com/w/cpp/error/exception> (visited on 08/21/2023).
- std::is_copy_constructible, std::is_trivially_copy_constructible, std::is_nothrow_copy_constructible - Cppreference.Com* (n.d.). URL: https://en.cppreference.com/w/cpp/types/is_copy_constructible (visited on 06/27/2023).
- std::move - Cppreference.Com* (n.d.). URL: <https://en.cppreference.com/w/cpp/utility/move> (visited on 08/30/2023).
- std::optional - Cppreference.Com* (n.d.). URL: <https://en.cppreference.com/w/cpp/utility/optional.html> (visited on 10/17/2025).
- std::placeholders::_1, std::placeholders::_2, ..., std::placeholders::_N - Cppreference.Com* (n.d.). URL: <https://en.cppreference.com/w/cpp/utility/functional/placeholders.html> (visited on 01/16/2026).
- std::printf, std::fprintf, std::sprintf, std::snprintf - Cppreference.Com* (n.d.). URL: <https://en.cppreference.com/w/cpp/io/c/fprintf> (visited on 03/04/2024).
- std::random_device - Cppreference.Com* (n.d.). URL: https://en.cppreference.com/w/cpp/numeric/random/random_device (visited on 05/02/2025).
- std::ranges::view_interface<D>::operator[] - Cppreference.Com* (n.d.). URL: https://en.cppreference.com/w/cpp/ranges/view_interface/operator_at.html (visited on 03/18/2026).

- `std::tie` - *Cppreference.Com* (n.d.). URL: <https://en.cppreference.com/w/cpp/utility/tuple/tie> (visited on 07/05/2022).
- `std::unique_ptr` - *Cppreference.Com* (n.d.). URL: https://en.cppreference.com/w/cpp/memory/unique_ptr (visited on 08/30/2023).
- `std::unique_ptr<T,Deleter>::operator=` - *Cppreference.Com* (n.d.). URL: https://en.cppreference.com/w/cpp/memory/unique_ptr/operator%3D (visited on 08/30/2023).
- `std::unique_ptr<T,Deleter>::unique_ptr` - *Cppreference.Com* (n.d.). URL: https://en.cppreference.com/w/cpp/memory/unique_ptr/unique_ptr (visited on 08/30/2023).
- `std::variant` - *Cppreference.Com* (n.d.). URL: <https://en.cppreference.com/w/cpp/utility/variant.html> (visited on 02/25/2026).
- `std::vector<bool>` - *Cppreference.Com* (n.d.). URL: https://en.cppreference.com/w/cpp/container/vector_bool (visited on 05/18/2025).
- `std::weak_ptr` - *Cppreference.Com* (n.d.). URL: https://en.cppreference.com/w/cpp/memory/weak_ptr (visited on 09/05/2023).
- Sutton, Andrew and Bjarne Stroustrup (July 3, 2011). *Design of Concept Libraries for C++*, p. 118. 97 pp. ISBN: 978-3-642-28829-6. DOI: 10.1007/978-3-642-28830-2_6.
- Tower of Hanoi* (July 11, 2025). In: *Wikipedia*. URL: https://en.wikipedia.org/w/index.php?title=Tower_of_Hanoi&oldid=1299921763 (visited on 07/11/2025).
- Type Alias, Alias Template (since C++11)* - *Cppreference.Com* (n.d.). URL: https://en.cppreference.com/w/cpp/language/type_alias (visited on 05/27/2022).
- UTF-8* (Feb. 19, 2024). In: *Wikipedia*. URL: <https://en.wikipedia.org/w/index.php?title=UTF-8&oldid=1208945767> (visited on 02/20/2024).
- Vandevoorde, David, Nicolai M. Josuttis, and Douglas Gregor (2018). *C++ Templates: The Complete Guide*. Second edition. Boston: Addison-Wesley. 788 pp. ISBN: 978-0-321-71412-1.